

JAVA | JVM | CONCURRENCY | SYSTEM DESIGN

JAVA ENGINEERING INTERVIEW FIELD GUIDE

Java Foundations to Advanced Engineering

A Complete SDE-2 Interview Preparation Guide

BY

Vinay Reddy Kalluri

A first-principles guide to Java semantics, JVM internals, collections, concurrency, performance, algorithms, and production engineering.

LANGUAGE | JVM | COLLECTIONS | CONCURRENCY | DSA | BACKEND

58 CHAPTERS | 7 APPENDICES | JAVA 25 BASELINE | 8-WEEK STUDY PLAN

Java 21 baseline - Java 17 and Java 21 features - 2026 edition

Open educational printable interview study edition

Copyright and Disclaimer

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and published editions are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders. Company names are used only to describe common interview markets.

The Java platform, JVM implementations, tools, flags, support policies, and licensing terms evolve. Examples target Java 21 unless a section states otherwise. Verify release-specific and vendor-specific behavior against the primary sources in Appendix G before production use.

The examples are provided without warranty. Review correctness, security, performance, operational, and legal requirements before adapting any example to a production system.

Build provenance: generated from the ordered Markdown chapter sources in this project. The PDF and DOCX editions use the same content inventory.

Contents

Parts, chapters, front matter, and appendices. PDF bookmarks also expose section-level navigation.

CONTENTS NAVIGATION

Start Here - Choose Your Route	7
Preface	8
About the Author	11
How to Use This Book	12
Study Roadmap	16
Part I - Java and the Computing Model	19
Chapter 1 - Why Java Exists	20
Chapter 2 - JDK, JRE, JVM, Editions, and Distributions	27
Chapter 3 - Compilation, Bytecode, and the Execution Pipeline	34
Part II - JVM Architecture and Memory	42
Chapter 4 - JVM Architecture	43
Chapter 5 - Class Loading, Linking, and Initialization	50
Chapter 6 - Runtime Data Areas	58
Chapter 7 - Object Creation and Memory Layout	66
Chapter 8 - Java Stacks, Method Calls, and Recursion	74
Chapter 9 - Garbage Collection	81
Chapter 10 - Execution Engine, JIT Compilation, and Safepoints	89
Chapter 11 - The Java Memory Model from First Principles	97
Part III - Java Language Engineering	107
Chapter 12 - Variables, Primitive Types, Literals, and Numeric Semantics	108
Chapter 13 - Operators, Expressions, and Control Flow	116
Chapter 14 - Methods, Overloading, Varargs, and Pass-by-Value	124
Chapter 15 - Arrays, Strings, Text Blocks, and Unicode	132

CONTENTS NAVIGATION

Chapter 16 - Classes, Objects, Access Control, and Packages	140
Chapter 17 - Inheritance, Polymorphism, and Composition	148
Chapter 18 - Interfaces, Abstract Classes, Sealed Types, and Pattern Matching	157
Chapter 19 - Equality, Hashing, Immutability, and Records	166
Chapter 20 - Exceptions and Resource Management	175
Chapter 21 - Nested Types, Enums, Annotations, and Reflection	183
Chapter 22 - Generics, Variance, Type Erasure, and Heap Pollution	192
Chapter 23 - Lambdas, Method References, and Functional Interfaces	201
Chapter 24 - Java 17 and Java 21 Language and Platform Features	209

Part IV - Collections, Streams, and I/O 218

Chapter 25 - Collections Framework Architecture	219
Chapter 26 - ArrayList, LinkedList, and List Trade-offs	229
Chapter 27 - HashMap, HashSet, and Hashing Internals	238
Chapter 28 - TreeMap, TreeSet, Ordering, and Navigable Collections	249
Chapter 29 - Queues, Deques, PriorityQueue, and Heaps	258
Chapter 30 - Comparable, Comparator, Sorting, and Selection	267
Chapter 31 - Streams, Collectors, Optional, and Spliterators	277
Chapter 32 - Java I/O, NIO.2, Files, Buffers, and Serialization Boundaries	288

Part V - Concurrency and Multithreading 299

Chapter 33 - Threads, Lifecycle, Interruption, and Cancellation	300
Chapter 34 - Synchronization, Intrinsic Locks, and Explicit Locks	309
Chapter 35 - Volatile, Atomics, CAS, and Happens-Before in Practice	317
Chapter 36 - Executors, Futures, CompletableFuture, and Work Scheduling	325
Chapter 37 - Concurrent Collections and Virtual Threads	335
Chapter 38 - Concurrency Failure Modes, Testing, and Design Patterns	344

Part VI - Performance, Diagnostics, and Reliability 353

Chapter 39 - Performance Methodology and JMH Benchmarking	354
---	-----

CONTENTS NAVIGATION

Chapter 40 - JVM Diagnostics with jcmd, jstack, jmap, JFR, and Mission Control	363
Chapter 41 - Memory Leaks, GC Incidents, and Tuning Playbooks	372

Part VII - Data Structures and Algorithms in Java 382

Chapter 42 - Complexity and the SDE-2 Problem-Solving Method	383
Chapter 43 - Arrays, Strings, Hashing, Two Pointers, Sliding Windows, and Prefix Sums	393
Chapter 44 - Linked Lists, Stacks, Queues, and Monotonic Structures	404
Chapter 45 - Trees, BSTs, Heaps, and Tries	416
Chapter 46 - Graphs, Topological Sort, Shortest Paths, and Union-Find	427
Chapter 47 - Recursion, Backtracking, Greedy Reasoning, and Dynamic Programming	439
Chapter 48 - The Java Coding Interview Playbook	450

Part VIII - Engineering Practice and Interview Readiness 462

Chapter 49 - Clean Java APIs, SOLID Design, and Low-Level Design Patterns	463
Chapter 50 - Backend Java Boundaries: JDBC, Transactions, Serialization, and Services	473
Chapter 51 - Testing, Build Tools, Static Analysis, and Dependency Management	483
Chapter 52 - Secure and Reliable Java	493
Chapter 53 - SDE-2 Java Interview Question Bank	504
Chapter 54 - Eight-Week Study Plan and Mock Interview Loops	519

Part IX - Core Library Depth and the Modern Java Baseline 532

Chapter 55 - Dates, Times, Zones, and the java.time API	533
Chapter 56 - Regular Expressions and Text Processing	543
Chapter 57 - The Module System, JPMS, and Strong Encapsulation	554
Chapter 58 - Java 22 to 25: The Modern LTS Baseline	565

Appendices 577

Appendix A - Java Syntax and Language Quick Reference	578
Appendix B - Collection Complexity and Selection Matrix	589
Appendix C - JVM Tools, Flags, and Incident Commands	595
Appendix D - Java 17 and Java 21 Feature Matrix	605

CONTENTS NAVIGATION

Appendix E - Exercise Hints and Selected Solutions613

Appendix F - Glossary631

Appendix G - Primary References and Further Reading648

Start Here - Choose Your Route

This book is deliberately broad. Start where your explanation, implementation, or production judgment first becomes unreliable.

One-minute placement rule: choose the earliest route below that describes a real gap. Complete its entry check, follow the named path, and move forward only when you can demonstrate the exit evidence without notes.

Pick one primary route

Your situation	Start here	Exit evidence
New to Java syntax	JAVA-01, then DSA-01 through DSA-03	Write a method, loop, array, validation, and tests
Rebuilding foundations	JAVA-01, then DSA-01 through DSA-17	Solve representative problems with invariants and complexity
Experienced; eight weeks	Diagnostic baseline, then Chapter 54	Pass coding, Java-depth, debugging, design, and behavioral mocks
Interview in two to three weeks	Chapters 24, 27, 31, 33-38, and 42-53; timed mocks select gaps	Implement under time and defend production trade-offs
Java backend specialist	JAVA-01 through JAVA-08, then the relevant FW and SD books	Defend a Spring service, persistence strategy, and distributed workflow

Open the [focused PDF Series Index](#) for individual modules, clickable navigation, and entry/exit gates.

Pair the schedule with the learning loop

Schedule decision	Repeat for every topic
12-week foundation: Study Roadmap; rebuild from basics	1. Recognize: name the contract, signal, or pattern
Eight-week accelerated: Chapter 54; skip only through diagnostics	2. Understand: restate the mechanism, invariant, or guarantee
Two-to-three-week sprint: revision driven by timed mocks	3. Implement: type the Java and test its boundaries
	4. Explain: give complexity, trade-offs, and failure modes aloud
	5. Prove readiness: produce working code or a defensible design without notes

Preface

What this book is for

Java interviews at the SDE-2 level are rarely tests of syntax alone. A strong candidate can move between several layers of explanation without losing accuracy. They can explain what a line of Java means, what bytecode and runtime machinery make it work, what performance costs are plausible, what the API contract actually guarantees, and how the same choice behaves inside a production service.

This book rebuilds that stack of understanding from the bottom up. It starts with the portability problem Java set out to solve, follows source code through compilation and execution, opens the JVM memory model, develops the language and libraries systematically, and then applies those ideas to collections, concurrency, performance diagnosis, coding interviews, and backend design.

The target reader already writes Java and may use Spring Boot, databases, queues, caches, and cloud services every day. Practical familiarity is valuable, but it can hide gaps. Frameworks make productive defaults easy; interviews deliberately remove those defaults and ask why a mechanism is correct. The aim here is to turn practical experience into an explicit engineering model.

The standard of explanation

An SDE-2 answer should be correct before it is impressive. This book therefore separates three kinds of claims:

- **Language and platform guarantees** come from the Java Language Specification, Java Virtual Machine Specification, and Java SE API contracts.
- **Typical implementation behavior** describes OpenJDK HotSpot when that detail helps build intuition.
- **Engineering guidance** is a contextual recommendation, not a universal law.

That distinction matters. For example, Java guarantees pass-by-value semantics; it does not guarantee that every object is physically allocated in a particular heap location. The JVM specification defines runtime data areas; a production HotSpot process also uses native memory structures that are outside the abstract machine. `HashMap` promises functional behavior and broad performance expectations; its precise table layout is an implementation detail that can evolve.

When an interviewer asks a broad question, begin with the guaranteed model, then add implementation detail only after labeling it. This habit prevents a common senior-level failure: giving a technically sophisticated answer that is confidently over-specific.

Version scope

Java 21 is the main executable baseline because it combines a mature LTS platform with records, sealed types, pattern matching, sequenced collections, and production virtual threads. Java 17 features and migration concerns are covered directly because Java 17 remains common in backend fleets. Earlier Java 8 and Java 11 idioms appear where they still shape interviews or deployed systems.

As of July 2026, Java 25 is also an LTS release. This book mentions that ecosystem fact but does not silently rewrite Java 21 behavior using later APIs. Whenever a feature was preview, incubating, or finalized in a particular release, the chapter says so. Always check the release notes and support policy for the exact distribution used by your employer.

How the chapters are built

Most chapters follow the same learning loop:

- 1 Build an intuitive model.
- 2 Define the formal vocabulary.
- 3 inspect the Java API or language mechanism.
- 4 Trace a concrete example.
- 5 Map the example to memory, bytecode, or execution behavior where useful.
- 6 Analyze complexity and failure modes.
- 7 Connect the mechanism to production backend work.
- 8 Practice explaining it under interview constraints.

The repeated structure is deliberate. Recognition is not recall, and recall is not judgment. Reading a definition creates recognition. Dry-running code develops recall. Comparing trade-offs and diagnosing failures develops judgment.

A note about code and measurements

Small examples are designed for Java 21 unless the surrounding text states otherwise. Some snippets isolate one idea and omit production concerns such as dependency injection, logging, retries, or observability. The separate [examples/java](#) project contains compilable examples used for verification.

Performance numbers are intentionally rare. JVM optimization, hardware, operating system, heap size, warm-up, and workload shape can reverse a microbenchmark result. Treat asymptotic complexity as a model, measurement as evidence, and benchmark design as part of the claim. Chapter 39 develops that discipline with JMH.

Disclaimer

This book is an independent educational resource. Java and related marks belong to their respective owners. Company names are used only to describe common interview markets and do not imply endorsement. Runtime flags, support schedules, licensing terms, and preview features change; consult the primary vendor documentation before making production or licensing decisions.

The examples are provided without warranty. Review security, correctness, operational, and legal requirements before adapting them to production systems.

Acknowledgments and source discipline

The conceptual authorities for this book are the Java Language Specification, the Java Virtual Machine Specification, Java SE API documentation, OpenJDK JEPs, and official JDK troubleshooting and tuning guides. Appendix G provides direct links. The explanations, examples, interview rubrics, and exercises are original instructional material built around those primary sources.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of *Java Foundations to Advanced Engineering* and the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through [AUTHORS.md](#) and the public Git history.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this book exists

Vinay created this guide to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The book reflects the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

[LinkedIn](#) | [GitHub](#)

The open-source edition and contributor registry are available in the [SDE2 Interview Handbook repository](#).

How to Use This Book

Choose a path, not a page count

The complete book is intentionally larger than a last-week revision guide. Do not read it linearly under every deadline. Use one of these paths.

The foundation rebuild

Use this path if everyday framework work has made core Java feel implicit:

- 1 Chapters 1-11: execution model, JVM, memory, GC, and the memory model.
- 2 Chapters 12-24: language semantics and modern Java.
- 3 Chapters 25-38: collections, streams, I/O, and concurrency.
- 4 Chapters 39-41: measurement and diagnosis.
- 5 Chapters 42-48: DSA practice in Java.
- 6 Chapters 49-54: design, production boundaries, and interview loops.

Plan eight to twelve weeks and write code while reading. The eight-week schedule in Chapter 54 is an accelerated version.

The interview sprint

Use this path when an interview is two to three weeks away:

- Read the summaries and revision checklists for Chapters 3-11.
- Study Chapters 19, 22, 25-31, and 33-38 in full.
- Complete the coding templates in Chapters 42-48 without copying.
- Answer Chapter 53 aloud under a two-minute limit.
- Run at least two mock loops from Chapter 54.

Return to detailed mechanics when an answer exposes a gap. Do not spend the sprint memorizing isolated JVM flags or collection trivia.

The production incident path

For performance, memory, or thread incidents, start with the symptom:

- High allocation or pauses: Chapters 6, 7, 9, 39, and 41.
- CPU saturation or latency after startup: Chapters 10, 39, and 40.
- Deadlock, thread starvation, or stuck shutdown: Chapters 33-38 and 40.
- Data race or visibility anomaly: Chapters 11, 34, and 35.
- Collection hot spot: Chapters 25-31 and 39.

The book is educational, not a replacement for an incident runbook. Preserve evidence before changing flags or restarting a process.

Use active recall

At the end of a section, close the book and explain the idea from memory. A useful explanation has four parts:

- 1 **Definition:** What is it?
- 2 **Mechanism:** How does it work?
- 3 **Trade-off:** What does it optimize and what does it cost?
- 4 **Application:** When would you use, avoid, or diagnose it?

For `volatile`, for example, a definition-only answer is weak. A complete answer states that volatile reads and writes participate in synchronization order, explains the happens-before edge, says that a compound read-modify-write is not made atomic, and gives a correct publication or cancellation example.

Keep an evidence notebook

Create four columns for every difficult concept:

Concept	Guarantee	Typical implementation	Experiment
Object allocation	The language exposes object creation and reference semantics.	HotSpot commonly allocates in a thread-local allocation buffer and may eliminate an allocation.	Compare allocation profiles with JFR; inspect generated code only after warm-up.
HashMap iteration	No general sorted-order contract.	Current implementations walk table structure and bins.	Insert controlled keys; never convert observed order into a contract.
<code>final</code> field publication	Proper construction gives special visibility guarantees.	Barriers and compiler constraints implement those semantics.	Write a safe-publication test; understand why a failing test is not a proof of correctness.

This format prevents accidental promotion of observation into specification.

Practice code in three passes

For each coding pattern:

- 1 **Correctness pass:** State inputs, outputs, invariants, and edge cases. Write the simplest correct solution.
- 2 **Complexity pass:** Identify the operations that dominate time and auxiliary space. Improve only with a clear reason.
- 3 **Communication pass:** Re-solve while narrating decisions, testing examples, and naming trade-offs.

Use the Java standard library confidently, but know the cost and semantics of what you call. `ArrayDeque` is normally a better stack than legacy `Stack`; a `PriorityQueue` does not provide sorted iteration; `List.subList` is a backed view; `Collectors.toMap` needs a merge policy when duplicate output keys are possible.

Turn questions into answer structures

Different interview questions need different shapes:

- **Compare A and B:** shared purpose, semantic difference, complexity/operational difference, choice criteria.
- **How does X work?:** contract, data/control flow, important state, exceptional path, implementation caveat.
- **Why did this fail?:** evidence, candidate mechanisms, discriminating tests, safest mitigation, prevention.
- **Design a component:** requirements, invariants, API, data structures, concurrency model, failure handling, tests.

Model answers in this book are reference structures, not scripts. Interviewers will probe any phrase that sounds memorized.

Verify with tools

The command line makes abstract explanations concrete:

BASH

```
javac --release 21 Example.java
javap -c -v Example
java -Xlog:gc* Example
jcmd <pid> VM.version
jcmd <pid> VM.flags
jcmd <pid> GC.heap_info
jcmd <pid> Thread.print
```

Tool output is implementation- and version-dependent. Use it to test a hypothesis, not to redefine the platform contract.

Definition of interview readiness

You are ready on a topic when you can:

- explain it accurately at three depths: 30 seconds, two minutes, and ten minutes;
- write or repair a representative example without documentation;
- identify at least two edge cases or failure modes;
- distinguish the specification from the implementation;
- connect it to one production decision;
- answer a follow-up that changes an assumption.

Checkmarks in revision lists are promises to yourself, not completion badges. Re-test them after several days.

Study Roadmap

Focused 40-book path

The complete master book remains the umbrella reference. For a finishable basics-to-SDE-2 route, use the **Java SDE-2 Interview Preparation Series** in [dist/](#). Its 40 focused books are grouped into four clearly ordered shelves: Java (9), DSA (17), frameworks and data systems (12), and system design (2). Each PDF is independently printable and includes previous/next navigation within its shelf.

Start with the sibling [series index](#). A reader rebuilding from basics should study Java Foundations first, followed by Time and Space Complexity, Number Systems, and the remaining DSA sequence. Continue with the rest of the Java shelf, then choose the frameworks and system-design books required by the target role. Git/GitHub and Maven/Gradle can be studied before the first portfolio project or before entering the framework shelf.

Use the displayed shelf code and sequence number as the source of truth: **JAVA-01** through **JAVA-09**, **DSA-01** through **DSA-17**, **FW-01** through **FW-12**, and **SD-01** through **SD-02**. When revising, begin at the first completion check that exposes a gap instead of rereading material you can already explain and implement.

Diagnostic baseline

Before choosing a schedule, answer these prompts aloud and write one code sample for each weak area:

- 1 Trace `javac` output from a class file through loading, verification, interpretation, and JIT compilation.
- 2 Distinguish heap, Java stack, metaspace, code cache, and native process memory.
- 3 Explain `equals` and `hashCode` as a joint contract, then predict a broken-key failure in `HashMap`.
- 4 Explain happens-before and show why `volatile int count++` is not an atomic counter.
- 5 Compare `ArrayList`, `LinkedList`, `ArrayDeque`, `HashMap`, `TreeMap`, and `ConcurrentHashMap` for a concrete workload.
- 6 Design cancellation for a task submitted to an executor.
- 7 Diagnose a service with rising old-generation occupancy and normal request volume.
- 8 Solve a medium array or graph problem while stating an invariant before coding.

Score each answer from zero to three:

- 0: cannot form a model;
- 1: recognizes terms but cannot explain mechanism;
- 2: gives a correct working explanation with minor gaps;
- 3: handles follow-ups, trade-offs, and production implications.

Prioritize zeros and ones. A balanced candidate is usually stronger than a candidate with one spectacular specialty and silent foundations.

Twelve-week foundation plan

Week	Core reading	Coding and evidence	Interview output
1	Chapters 1-3	Compile, disassemble, and package a small program	Explain source-to-CPU execution in five minutes
2	Chapters 4-8	Draw JVM memory and call-stack diagrams	Answer stack, heap, class-loader, and pass-by-value probes
3	Chapters 9-11	Capture GC logs; write safe and unsafe publication examples	Explain reachability, collectors, JIT, and happens-before
4	Chapters 12-18	Implement language edge-case exercises	Predict code output and defend specification claims
5	Chapters 19-24	Build immutable values, generic APIs, and pattern switches	Compare records, classes, sealed types, and generics
6	Chapters 25-30	Reimplement a small map and heap; benchmark only after reasoning	Select collections for five production workloads
7	Chapters 31-32	Build stream and NIO pipelines with explicit resource handling	Explain laziness, collectors, back pressure boundaries, and I/O
8	Chapters 33-35	Write cancellation, visibility, lock, and atomic examples	Solve race-condition and synchronization follow-ups
9	Chapters 36-38	Build bounded executor and virtual-thread experiments	Design a concurrent component and failure strategy
10	Chapters 39-41	Run JMH and JFR; analyze a synthetic memory leak	Present an evidence-first performance diagnosis
11	Chapters 42-48	Two timed problems per day, rotating patterns	Complete two coding mocks
12	Chapters 49-54	One low-level design and one backend scenario per day	Complete full mock loops and close weak topics

Daily practice block

A sustainable two-hour block is more valuable than an occasional eight-hour binge:

- 1 25 minutes: recall yesterday's concepts without notes.
- 2 35 minutes: read one focused section and annotate guarantees versus implementation.
- 3 45 minutes: write, run, and test code.
- 4 15 minutes: answer two questions aloud and record unclear phrases.

On coding-heavy days, use 20 minutes for problem framing, 35 minutes for implementation, 15 minutes for tests and complexity, and 10 minutes for a clean verbal recap.

Spaced review cadence

Review a concept after one day, three days, seven days, and fourteen days. Each review should retrieve, not reread:

- draw the mechanism;
- state the contract;
- write the smallest example;
- answer one adversarial follow-up;
- update the evidence notebook.

If retrieval fails, shorten the next interval. If it succeeds cleanly twice, move the topic to weekly rotation.

Mock loop design

A realistic SDE-2 loop should sample different evidence:

- 45 minutes: coding and complexity;
- 45 minutes: Java/JVM depth;
- 45 minutes: low-level design and concurrency;
- 45 minutes: backend design and operational reasoning;
- 30 to 45 minutes: behavioral examples with ownership and trade-offs.

After each mock, repair the first point where the answer lost precision before collecting more questions.

Readiness gates

Do not use confidence as the only signal. Use observable gates:

- At least 80 percent of revision checklist items can be explained without notes.
- Medium coding problems are normally correct within 35 minutes, including tests and complexity.
- Collection and concurrency choices are justified by workload and correctness requirements.
- JVM answers label implementation details.
- Performance answers begin with measurement and preserve evidence.
- Design answers state invariants, failure modes, and observability before exotic optimizations.
- Behavioral stories show decisions, measurable impact, and learning; every answer should make reasoning inspectable and trustworthy.

JAVA FOUNDATIONS TO ADVANCED ENGINEERING

Part I - Java and the Computing Model

A focused part of the complete SDE-2 preparation guide

SDE-2 CORE CHAPTER

Chapter 1 - Why Java Exists

1.1 Learning objectives

By the end of this chapter, you should be able to:

- Explain the portability, safety, and productivity problems Java was designed to address.
- Describe the relationship among source code, bytecode, a JVM implementation, and an operating system.
- Separate the Java language promise from marketing shorthand such as "write once, run anywhere."
- Compare Java with C++, C#, Python, Go, Kotlin, and Rust without reducing the choice to speed alone.
- Defend or reject Java for a backend workload using engineering constraints.

WHY IT WORKS | 1.2 Why this matters at SDE-2

An SDE-2 is expected to choose technology rather than merely use it. "Java is portable" is an incomplete answer. A strong engineer can explain what is portable, which compatibility assumptions are required, and what is still platform dependent. This model also supports later reasoning about deployment artifacts, native libraries, container images, startup time, garbage collection, and operational diagnostics.

Interviewers often use Java's history as a route into trade-offs. They may ask why bytecode exists, why Java uses managed memory, or why a JVM can outperform simplistic expectations about a virtual machine. The useful answer connects design choices: an intermediate instruction set creates a stable distribution format; verification and runtime services increase safety; adaptive compilation recovers much of the performance cost; and a mature library ecosystem reduces the total cost of backend development.

WHY IT WORKS | 1.3 First-principles model

A processor executes instructions for one instruction set, such as x86-64 or AArch64. An operating system also defines executable formats, system-call interfaces, file conventions, and dynamic linking rules. If a compiler turns a C or C++ program directly into one machine's executable, that binary normally cannot simply move to another machine and operating system.

Java inserts a specified virtual machine between program and platform:

TEXT

```
Java source
  |
  | compiler checks language rules
  v
class files containing JVM bytecode
  |
  | a JVM implementation for this host
  v
host machine code, OS services, and hardware
```

The portable artifact is ordinarily the class file, not a running process and not every interaction with the environment. A Linux x86-64 JVM and a macOS AArch64 JVM can implement the same class-file and runtime contracts while using different native code, garbage collectors, and operating-system facilities.

Specification boundary: The Java Language Specification (JLS) defines the language, while the Java Virtual Machine Specification (JVMS) defines the class-file format and abstract JVM behavior. Neither specification promises that all programs produce identical environmental results on every host. File systems, default character sets, native methods, timing, resource limits, and optional providers can differ.

1.4 Core terminology

- **Source language:** The syntax and semantics programmers write, defined for Java by the JLS.
- **Bytecode:** Instructions in a class file for the abstract JVM instruction set.
- **JVM:** An implementation of the JVMS that loads and executes class files and provides runtime services.
- **Portability:** The ability to move an artifact across conforming environments with limited or no changes.
- **Managed runtime:** A runtime that participates in memory management, type safety, execution, and diagnostics.
- **Ahead-of-time compilation:** Translation before the program runs, commonly to a platform-specific form.
- **Just-in-time compilation:** Translation during execution, often guided by observed behavior.
- **Foreign function interface:** A boundary for calling non-Java code; Java's historical interface is JNI.

1.5 Detailed mechanics

Before Java, portable source code was common, but portable binaries were harder. C standardized much of a language, yet recompilation, conditional compilation, compiler differences, data-model differences, and platform libraries remained practical concerns. C++ added expressive object-oriented and generic facilities but also more undefined behavior, manual lifetime complexity, ABI differences, and build-system complexity. These languages remain excellent choices; the point is that their usual delivery model did not supply Java's uniform managed runtime.

Java began in Sun Microsystems' Green Project in the early 1990s. The language was first called Oak and targeted networked consumer devices. That market did not become its defining success, but the design suited the emerging web: compact downloadable code, a platform-neutral instruction form, runtime checks, automatic storage management, and a substantial standard library. The name changed to Java, and the browser-applet era made the platform visible. Applets later disappeared for security and deployment reasons, while server-side Java grew.

"Write once, run anywhere" captures the intended distribution model. It does not eliminate dependencies. A class compiled for a newer class-file version will not run on an older JVM. Code can depend on OS paths, native libraries, locale, time zone, fonts, cryptographic providers, or unspecified iteration order. A more accurate engineering statement is: compile to a specified portable representation, then run on a compatible JVM and required environment.

Enterprise adoption came from more than portability. Java supplied garbage collection, exceptions, reflection, dynamic loading, threading primitives, networking, database APIs, and strong tooling under a relatively stable compatibility culture. Organizations could run long-lived services, inspect their state, upgrade hardware, and hire from a large talent pool. Frameworks later standardized web, dependency injection, transactions, messaging, and observability patterns.

Java remains relevant because the ecosystem compounds. The platform has high-quality collectors, profile-guided JIT compilers, profilers, flight recording, debuggers, build tools, libraries, and production knowledge. Java 17 and 21 also modernized the language and runtime with records, sealed classes, pattern matching, and virtual threads. Backward compatibility is treated seriously, although it is never absolute.

The strengths have costs. A managed runtime adds startup and memory overhead. Garbage collection changes latency behavior. Type erasure constrains some generic operations. Checked exceptions and verbosity can be contentious. A large dependency ecosystem can become a supply-chain and complexity burden. Peak control over layout and deterministic resource use is generally lower than in C++ or Rust.

Language comparisons should be workload-specific:

Alternative	Typical advantage relative to Java	Typical Java advantage
C++	Explicit layout, native integration, deterministic destruction	Memory safety, uniform runtime, deployment and diagnostics
C#	Tight .NET integration and polished language evolution	Broad JVM/server ecosystem and cross-vendor runtime choices
Python	Concise code and data/ML ecosystem	Static checking, throughput, parallel execution, large-service tooling
Go	Fast builds, simple deployment, lightweight concurrency	Richer type/library ecosystem, adaptive optimization, mature JVM tooling
Kotlin	More concise null-aware JVM language	Simpler toolchain baseline and maximum Java-source familiarity
Rust	Ownership-based memory safety without GC, layout control	Lower learning cost, dynamic runtime capabilities, mature enterprise stack

These are tendencies, not universal rankings. C# is cross-platform, Go has excellent services tooling, and Rust can build highly reliable servers. Kotlin interoperates closely with Java but introduces its own compiler and language semantics.

TRACE IT | 1.6 Worked Java example

This program deliberately avoids host-specific assumptions:

JAVA

```
public final class PortabilityDemo {
    private PortabilityDemo() {}

    static long checksum(byte[] input) {
        long result = 0;
        for (byte value : input) {
            result = (result * 31 + (value & 0xff)) & 0xffff_ffffL;
        }
        return result;
    }

    public static void main(String[] args) {
        byte[] message = {74, 97, 118, 97};
        System.out.println(checksum(message));
    }
}
```

Java fixes the widths and two's-complement behavior of integral primitive types. The `& 0xff` expression interprets each signed `byte` as an unsigned value from 0 to 255, and `& 0xffff_ffffL` keeps the low 32 bits. A conforming implementation must preserve these language semantics.

TRACE IT | 1.7 Execution or memory walkthrough

- 1 `javac` parses and type-checks the source and emits `PortabilityDemo.class`.
- 2 The class file records bytecode, symbolic references, method descriptors, and constants.
- 3 A host-specific Java launcher creates a JVM process and loads required platform classes plus `PortabilityDemo`.
- 4 The JVM verifies constraints before executing the methods.
- 5 `main` allocates an array object and calls `checksum`.
- 6 Bytecode operations perform specified integer arithmetic. The JVM may interpret them or compile them to native instructions.
- 7 `System.out` ultimately crosses into host facilities to write output. That environmental boundary is less portable than the arithmetic.

The source and class file can remain unchanged across hosts. The launcher executable, JVM internals, and final machine instructions differ.

1.8 Complexity and performance

`checksum` visits `n` bytes, so it uses $O(n)$ time and $O(1)$ auxiliary space. That algorithmic statement survives every compliant JVM. Exact time does not: compilation tier, CPU, bounds-check elimination, warm-up, and surrounding load matter.

Bytecode is not inherently "slow." It is an input to an execution strategy. A JVM can observe frequent call paths and concrete receiver types, inline across method boundaries, remove redundant checks, and compile optimized native code. Conversely, a short-lived command may finish before adaptive optimization pays back its compilation cost.

HotSpot note: HotSpot commonly begins with interpreted or lower-tier execution, collects profiles, and compiles hot code through tiered compilation. These thresholds and compiler strategies are implementation details, not Java language guarantees.

WATCH OUT | 1.9 Edge cases and common mistakes

- Saying Java is platform independent without naming the compatible JVM and environment.
- Confusing source portability with binary portability. Java emphasizes portable class files, although source compatibility also matters.
- Claiming Java has no pointers. Java has references; it does not expose general pointer arithmetic in ordinary source code.
- Claiming garbage collection prevents leaks. Reachable but useless objects still consume memory.
- Treating Java as only interpreted or only compiled.
- Assuming a deterministic result implies deterministic timing.
- Depending on default locale, character encoding, file separator, or time zone and then calling the program portable.

1.10 Production engineering notes

Java is a strong default for long-lived APIs, transaction systems, stream processors, data platforms, and services that benefit from throughput, observability, library depth, and maintainability. It is less compelling for tiny utilities where startup and distribution size dominate, hard real-time control loops, kernel code, extremely constrained devices, or components requiring exact object layout and direct hardware access.

Containerization does not remove JVM portability concerns. Pin the JDK distribution and version, record JVM flags, set locale and time zone explicitly where results matter, test native dependencies for the target architecture, and size container memory with heap plus non-heap usage in mind. Treat the class file as one layer in a full deployment contract.

TRY IT | 1.11 Interview questions and model answers

Why did Java use bytecode instead of directly compiling only to native code?

Bytecode is a stable, platform-neutral distribution format. A host-specific JVM maps it to local execution while providing verification, managed memory, dynamic loading, and profiling. Native compilation is still possible, but it trades some adaptive behavior and portability for startup or footprint benefits.

What does write once, run anywhere really mean?

It means a compatible class-file artifact can run on conforming JVMs without recompilation, provided its library and environmental assumptions are met. It does not make native libraries, paths, encodings, timing, or resource limits identical.

Why is Java popular for enterprise backends?

The important combination is stable compatibility, static typing, automatic memory management, concurrency support, a large library/framework ecosystem, strong diagnostics, and high steady-state performance. No one feature explains adoption.

When would you choose Rust or Go instead?

I would consider Rust when layout control, predictable resource ownership, native integration, or no-GC latency is central. I would consider Go when simple deployment, quick builds, and a smaller language/runtime model dominate. I would choose based on the system's constraints and team capability, not a generic language ranking.

TRY IT | 1.12 Exercises

- 1 List five assumptions that can make an otherwise portable Java service host dependent.
- 2 Explain the boundary among JLS semantics, JVMS bytecode, and JVM implementation choices.
- 3 Modify `PortabilityDemo` to read text. Specify a charset explicitly and explain why.
- 4 Compare Java and one alternative for a low-latency trading gateway, a CRUD service, and a command-line utility.
- 5 Write a two-minute answer to "Why Java?" that includes two strengths and two trade-offs.

RECAP | 1.13 Chapter summary

Java's central design is an explicit portability and runtime boundary. The compiler produces class files for an abstract machine, and a host JVM supplies execution, managed memory, safety checks, and services. This model helped Java become a durable server platform, but it does not erase operating-system dependencies or runtime costs. Technology selection must consider startup, throughput, latency, memory, ecosystem, control, and team factors together.

TRY IT | 1.14 Revision checklist

- ☐ I can explain why portable source and portable class files are different.
- ☐ I can state what the JLS, JVMS, and a JVM implementation each control.
- ☐ I can give a precise interpretation of write once, run anywhere.
- ☐ I can explain why managed execution can still achieve high throughput.
- ☐ I can compare Java with alternatives using workload constraints.
- ☐ I can identify cases where Java is a poor fit.

SDE-2 CORE CHAPTER

Chapter 2 - JDK, JRE, JVM, Editions, and Distributions

2.1 Learning objectives

- Distinguish the JVM, runtime image, JRE concept, and JDK.
- Explain Java SE, Jakarta EE, OpenJDK, Oracle JDK, and vendor distributions.
- Reason about source, binary, class-file, runtime, and behavioral compatibility.
- Summarize the practical significance of Java 8, 11, 17, and 21.
- Select and use core JDK command-line tools during development and incidents.

WHY IT WORKS | 2.2 Why this matters at SDE-2

Production failures often hide behind imprecise statements such as "the Java version is the same." The build may use JDK 21 while emitting Java 17 class files; the container may use a different vendor build; an agent may depend on internal APIs; or a library may be binary compatible but behaviorally different. An SDE-2 should capture the complete runtime contract and use built-in tools before guessing.

The JDK/JRE/JVM question is common in interviews because it tests whether the candidate sees a layered platform. The senior answer also covers modern modular runtime images, class-file versions, LTS policy, distribution support, and diagnostic tooling.

WHY IT WORKS | 2.3 First-principles model

The JVM is the abstract execution target and its concrete implementation. A runtime needs that JVM plus core libraries and supporting files. A development kit adds compilers, packagers, documentation generators, debuggers, and diagnostics.

TEXT

```
JDK
+-- Java compiler and development/diagnostic tools
+-- Java runtime image
    +-- JVM implementation
    +-- standard modules and native support
```

Historically, vendors shipped a separately installable JRE. Since Java 9 introduced the module system and custom runtime images, production deployments frequently use a full JDK or an application-specific image produced by [jlink](#). "JRE" remains a useful conceptual term, but one should not assume a separate official JRE download exists for every modern distribution.

2.4 Core terminology

- **JVM:** Loads and executes class files according to the JVMs.
- **JRE:** The runtime concept: JVM plus libraries and resources needed to run applications.
- **JDK:** A development and diagnostic distribution containing a runtime and tools.
- **Java SE:** The standard language and platform APIs forming the base Java platform.
- **Jakarta EE:** Enterprise specifications layered on Java SE and implemented by compatible servers/frameworks; it is not another JVM.
- **OpenJDK:** The open-source reference implementation project and code base used by many builds.
- **Distribution:** A tested vendor build of OpenJDK, potentially with different packaging, support, patches, and optional components.
- **LTS:** A release for which a vendor offers a longer support window; the duration and terms are vendor policy.
- **Class-file version:** A version marker constraining which JVM releases can load a class.

2.5 Detailed mechanics

Java SE includes the language, class-file/JVM contracts, and APIs such as collections, I/O, concurrency, networking, JDBC, and cryptography. Jakarta EE defines additional APIs for enterprise concerns such as persistence, transactions, dependency injection, REST, and messaging. A Jakarta EE specification requires an implementation. A Spring Boot service may use selected Jakarta APIs without running a full Jakarta EE application server.

OpenJDK is not synonymous with one downloadable binary. Oracle, Eclipse Temurin, Amazon Corretto, Azul Zulu, Microsoft Build of OpenJDK, Red Hat builds, BellSoft Liberica, and others distribute JDKs. GraalVM distributions add technologies such as a high-performance compiler and Native Image. IBM Semeru can use OpenJ9 in some editions rather than HotSpot. Selection factors include supported platforms, update cadence, cryptographic requirements, container images, support contracts, and operational validation.

Oracle JDK and OpenJDK builds share substantial code and conformance goals, but license and support terms must be evaluated for the chosen version and use. Do not infer licensing from the words "Java" or "OpenJDK" alone.

LTS is not a language property. Java 8, 11, 17, and 21 have been treated as LTS releases by major vendors, but update availability differs. Feature releases occur on the Java release cadence, and teams can choose to upgrade every feature release or remain on supported LTS lines.

Version compatibility has several dimensions:

- **Source compatibility:** Whether old source still compiles. New keywords, removed APIs, and stricter checks can matter.
- **Binary compatibility:** Whether previously compiled clients can link against a changed library. Removing a method or changing a superclass can break it even when source could be adjusted.
- **Class-file compatibility:** Whether the runtime supports the emitted class-file major version.

- **Runtime compatibility:** Whether required APIs, modules, native libraries, flags, and environment exist.
- **Behavioral compatibility:** Whether results and operational behavior remain acceptable despite implementation or library changes.

Specification boundary: A JVM must reject an unsupported class-file version. The Java platform defines extensive compatibility rules, but it does not promise that every internal API, removed module, command-line flag, timing characteristic, or vendor extension remains available.

A compact release map:

Release	Interview and production significance
Java 8	Lambdas, streams, default interface methods, <code>java.time</code> ; still a major historical baseline
Java 11	Standard HTTP client, single-file source launch, post-Java-8 module-era baseline
Java 17	Records, sealed classes, pattern matching for <code>instanceof</code> ; common modern LTS baseline
Java 21	Virtual threads, record patterns, pattern matching for <code>switch</code> , sequenced collections; modern LTS baseline

Features arrived across intermediate releases. For example, records became final before Java 17; a release table is a migration summary, not a claim that every listed feature originated in that exact LTS release.

The essential tools form a pipeline:

- `javac`: compile source to class files; `--release 17` targets the documented Java 17 API and class-file level.
- `java`: launch a class, JAR, module, or source file in supported modes.
- `jar`: create, inspect, and update JAR archives.
- `javap`: inspect class signatures and bytecode; it is not a Java decompiler that reconstructs exact source.
- `javadoc`: generate API documentation from source comments and declarations.
- `jcmd`: send supported diagnostic commands to a running JVM.
- `jstack`: print Java thread stacks; `jcmd <pid> Thread.print` is often preferred in modern operations.
- `jmap`: inspect heap configuration or request heap information/dumps; collection impact must be considered.
- `jfr`: manage and inspect Java Flight Recorder files, depending on JDK version and command usage.

TRACE IT | 2.6 Worked Java example

JAVA

```
public final class RuntimeIdentity {
    public static void main(String[] args) {
        System.out.println("runtime=" + Runtime.version());
        System.out.println("vm=" + System.getProperty("java.vm.name"));
        System.out.println("vendor=" + System.getProperty("java.vendor"));
        System.out.println("home=" + System.getProperty("java.home"));
    }
}
```

Compile for a Java 17 deployment while using a newer JDK:

BASH

```
javac --release 17 RuntimeIdentity.java
javap -verbose RuntimeIdentity.class
java RuntimeIdentity
```

`--release 17` is stronger than merely using `-source 17 -target 17`: it also constrains compilation against the documented Java 17 platform API signature set. It cannot validate every third-party library or environmental dependency.

TRACE IT | 2.7 Execution or memory walkthrough

Suppose CI runs JDK 21 with `--release 17` and production runs a Java 17 runtime.

- 1 The compiler accepts Java 17 language constructs and documented Java 17 APIs.
- 2 It emits the class-file version associated with Java 17.
- 3 `javap -verbose` exposes that version and constant-pool metadata.
- 4 A Java 17 JVM can load the class if dependencies are compatible.
- 5 At launch, the host JDK supplies its own JVM implementation, core modules, native libraries, default properties, and security providers.
- 6 `Runtime.version()` reports the executing runtime, not the compiler that produced the class.

The JVM process has heap and non-heap state independent of the tool binaries on disk. Running `jcmd` attaches or communicates with that process through implementation-supported mechanisms; it is not executing inside application source semantics.

2.8 Complexity and performance

Choosing a distribution should rarely depend on a single microbenchmark. Compare representative startup, steady-state throughput, tail latency, memory, collector behavior, and diagnostics under the exact release and flags. Vendor patches can change performance without changing language behavior.

A custom `jlink` image can reduce installed modules and distribution size, which may improve image transfer and attack surface. It also creates ownership: the exact module graph and update process become part of deployment. A full JDK consumes more disk space but keeps diagnostic tools nearby. Some organizations use a slim runtime image and an approved sidecar or matching diagnostic image.

WATCH OUT | 2.9 Edge cases and common mistakes

- Calling Jakarta EE a JDK or JVM edition. It is a set of specifications layered on Java SE.
- Assuming all OpenJDK distributions have identical support, packaging, certificates, fonts, or native integrations.
- Believing LTS means free updates forever. Support is vendor-specific.
- Compiling on JDK 21 with `-target 17` while accidentally referencing Java 21 APIs.
- Assuming a lower class-file target makes every dependency compatible.
- Parsing `java -version` manually when deployment metadata could record an exact immutable image digest.
- Shipping a JRE-only image and discovering incident tools are unavailable.
- Running heap-dump commands without checking disk space, data sensitivity, and pause impact.

2.10 Production engineering notes

Record at least distribution, full version/build, architecture, base image digest, enabled modules, JVM flags, and collector for each service release. Reproduce incidents with the same build where possible. Patch releases contain security and correctness fixes, so "LTS" should support an update policy, not justify freezing forever.

Useful first-response commands include:

BASH

```
java -version
jcmd -l
jcmd 12345 VM.version
jcmd 12345 VM.flags
jcmd 12345 VM.system_properties
jcmd 12345 Thread.print
jcmd 12345 GC.heap_info
jfr print recording.jfr
```

Commands vary by release and JVM implementation. Confirm with `jcmd <pid> help`, test operational cost, protect diagnostic outputs, and avoid making write-heavy dumps during an already capacity-constrained incident unless justified.

HotSpot note: Tools such as `jcmd`, HotSpot-specific commands, many `-XX` flags, and details of JFR integration are implementation/version specific. Their presence is not guaranteed by the JLS or JVMs.

TRY IT | 2.11 Interview questions and model answers

What is the difference among JDK, JRE, and JVM?

The JVM executes class files. A runtime combines a JVM with platform libraries and support files. A JDK contains a runtime plus development and diagnostic tools. In modern modular Java, a runtime may be a custom image rather than a separately branded JRE installation.

Is OpenJDK different from Oracle JDK?

OpenJDK is the open-source project/code base; Oracle JDK is a distribution. They share substantial implementation, but packaging, licensing, support, update schedules, and optional components must be assessed for the exact releases.

Why can an `UnsupportedClassVersionError` occur?

The class file was produced for a newer JVM class-file level than the runtime supports. The durable fix is to run a compatible newer runtime or compile the application and all dependencies for the intended older release.

What does binary compatibility mean?

It asks whether already compiled client code can link and run against a new library version without recompilation. It differs from source compatibility and does not guarantee identical behavior.

TRY IT | 2.12 Exercises

- 1 Install or inspect two JDK distributions and compare `java -version` output and available modules.
- 2 Compile a class with `--release 17`, inspect it with `javap -verbose`, and identify its major version.
- 3 Produce a JAR with a main class and run it using `java -jar`.
- 4 Design a runtime-version inventory record for a fleet of services.
- 5 Explain when a slim `jlink` image is worth the operational trade-off.
- 6 On a safe local process, use `jcmd` to inspect flags and thread stacks.

RECAP | 2.13 Chapter summary

The JVM is the execution layer, a runtime combines it with required libraries, and the JDK adds engineering tools. Java SE defines the base platform; Jakarta EE adds enterprise specifications. OpenJDK underlies many distributions whose packaging and support still differ. Compatibility must be discussed across source, binary, class-file, runtime, and behavior dimensions. Exact version and distribution metadata are production requirements, not trivia.

TRY IT | 2.14 Revision checklist

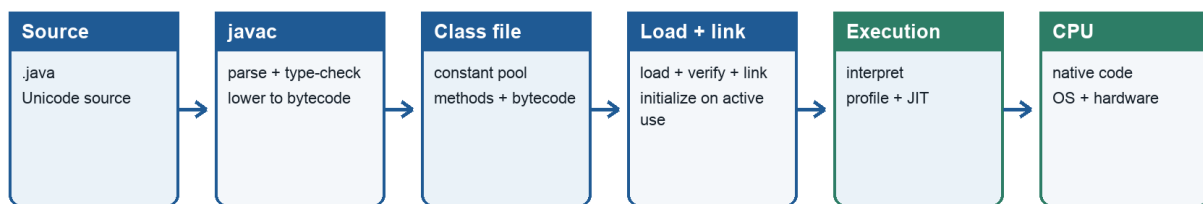
- ☐ I can explain JDK, runtime/JRE, and JVM without relying on old packaging assumptions.
- ☐ I can distinguish Java SE, Jakarta EE, OpenJDK, and a distribution.
- ☐ I know why LTS is a vendor support concept.
- ☐ I can explain source, binary, class-file, runtime, and behavioral compatibility.
- ☐ I can summarize Java 8, 11, 17, and 21 accurately.
- ☐ I can select the right basic JDK diagnostic tool and state its risk.

SDE-2 CORE CHAPTER

Chapter 3 - Compilation, Bytecode, and the Execution Pipeline

Java compilation and execution

From source text to optimized native instructions



Key interview boundary

The JVM specification defines an abstract machine and class-file format.
Interpretation, tiered compilation, profiling, and exact machine-code generation are implementation strategies.

Figure 3.1 - Compilation and execution from source to native instructions

3.1 Learning objectives

- Trace a Java program from source characters to CPU instructions.
- Explain lexical analysis, parsing, type checking, class-file generation, loading, linking, and initialization.
- Read representative output from `javap -c`.
- Distinguish symbolic bytecode from resolved runtime structures and native code.
- Explain why an IDE Run action is orchestration rather than a separate Java execution model.

WHY IT WORKS | 3.2 Why this matters at SDE-2

This pipeline locates failures. A syntax error, linkage error, initialization failure, verifier error, JIT deoptimization, and native crash occur in different layers and require different evidence. SDE-2 interviews also expect more than "Java compiles to bytecode": candidates should describe when types are checked, how symbolic references are resolved, why warm-up exists, and how a source line may map to multiple execution forms.

WHY IT WORKS | 3.3 First-principles model

A compiler converts one representation into another while preserving specified meaning. `javac` is primarily an ahead-of-time source-to-class-file compiler. The JVM then performs dynamic work that cannot or need not be finalized at source compilation time.

TEXT

```
.java text
-> tokens
-> syntax tree
-> attributed/type-checked program
-> class-file structures and bytecode
-> loaded Class objects and metadata
-> verified, prepared, optionally resolved classes
-> initialized class
-> interpreted/compiled execution
-> native instructions on a CPU
```

There can be other paths: annotation processors can generate source; alternative compilers can emit valid class files; JVM languages can target bytecode; and ahead-of-time tools can build native executables. None changes the conceptual contract between valid class files and a conforming JVM.

3.4 Core terminology

- **Lexing:** Grouping characters into tokens such as identifiers, literals, and operators.
- **Parsing:** Checking grammatical structure and constructing a syntax tree.
- **Attribution/type checking:** Resolving names and validating types, overloads, access, and language rules.
- **Class file:** A binary structure containing a version, constant pool, fields, methods, attributes, and bytecode.
- **Constant pool:** A per-class table of literals and symbolic references used by class-file structures.
- **Descriptor:** A compact JVM encoding of field or method types, such as `(II)I`.
- **Verification:** Checking structural and type-safety constraints before unsafe bytecode can execute.
- **Resolution:** Converting a symbolic reference into a concrete runtime entity.
- **Interpreter:** Executes bytecode according to its meaning without first producing a fully optimized method body.
- **JIT compiler:** Compiles selected runtime code to native instructions.
- **Deoptimization:** Transfers execution from optimized code when an assumption is invalidated.

3.5 Detailed mechanics

The compiler first recognizes Unicode input according to Java's lexical rules. It forms tokens, parses declarations and expressions, resolves imported and qualified names, infers applicable generic types, chooses overloads, performs definite-assignment checks, and reports errors. Some constructs are translated into lower-level class-file patterns. For example, an enhanced `for` loop becomes index-based or iterator-based logic, and string concatenation can use an invokedynamic-based recipe in modern compilers.

Compilation does not load every application class in the same sense as the runtime class-loader subsystem. It reads declarations needed to check code. Successful compilation also does not prove the runtime classpath contains compatible definitions.

A class file contains methods expressed in an operand-stack instruction set. Many Java source variables map to local-variable slots; expressions push and consume operand-stack values. Object member references are symbolic constant-pool entries until runtime resolution associates them with actual definitions.

Loading finds bytes and creates the runtime representation. Linking includes verification, preparation, and resolution. Preparation allocates static storage and gives fields their default values; initialization later executes class initialization code. Resolution may be eager or lazy within JVM constraints.

Specification boundary: The JVM defines class-file validity, instruction behavior, loading/linking/initialization constraints, and observable error conditions. It does not require an interpreter, a particular JIT compiler, a hotness threshold, or one-to-one mapping from bytecode to machine instructions.

At invocation, a JVM may interpret a method, compile it, invoke a precompiled form, or combine strategies. Adaptive JVMs gather profiles such as branch frequencies and receiver types. Optimized native code can inline methods and eliminate allocations based on assumptions. If a newly loaded subclass or uncommon branch invalidates an assumption, the runtime can deoptimize to a less specialized form while preserving Java semantics.

An IntelliJ IDEA Run action ordinarily performs orchestration:

- 1 Save or use the in-memory project state according to settings.
- 2 Invoke a compiler or incremental build.
- 3 Construct an output directory/module path/classpath.
- 4 Build a command containing a selected JDK launcher, JVM options, main class, and arguments.
- 5 Start an operating-system process and connect console/debug channels.
- 6 The Java launcher creates the JVM and locates `main`.

The IDE does not bypass class loading or cause the CPU to execute Java source directly.

TRACE IT | 3.6 Worked Java example

JAVA

```
public final class BytecodeTour {
    static int max(int left, int right) {
        return left >= right ? left : right;
    }

    public static void main(String[] args) {
        int answer = max(4, 9);
        System.out.println(answer);
    }
}
```

Run:

BASH

```
javac -g BytecodeTour.java
javap -c -p BytecodeTour
```

Representative output for `max` is conceptually:

TEXT

```
0: iload_0
1: iload_1
2: if_icmplt 9
5: iload_0
6: goto 10
9: iload_1
10: ireturn
```

`iload_0` pushes the integer in local slot 0. `if_icmplt` pops two integers and branches if the first is less than the second. One branch pushes `left`, the other pushes `right`, and `ireturn` returns the top integer. Actual offsets or branch shape may differ by compiler release while behavior remains the same.

Representative `main` instructions include `iconst_4` or a constant push, `bipush 9`, `invokestatic` for `max`, `istore_1`, `getstatic` for `System.out`, `iload_1`, and `invokevirtual` for `println`.

TRACE IT | 3.7 Execution or memory walkthrough

For `main`:

- 1 The JVM creates a frame whose local slots include `args` and later `answer`.
- 2 Constants 4 and 9 are pushed on `main`'s operand stack.
- 3 `invokestatic` invokes `max`; a new frame receives the arguments in local slots 0 and 1.
- 4 `max` compares them and returns 9. Its frame is removed.
- 5 The value 9 appears as the invocation result on `main`'s operand stack, then is stored in `answer`'s slot.
- 6 `getstatic` obtains the `PrintStream` reference represented by `System.out`.
- 7 The receiver and integer argument are placed on the operand stack and `println(int)` is invoked.
- 8 `return` completes `main`; non-daemon thread state then determines process lifetime.

Resolution may occur when an instruction first uses `System.out`, `println`, or `max`, or earlier. Initialization of `BytecodeTour` must occur before its active use, though it has no explicit static initializer.

TEXT		
Source local <code>answer</code>	JVM frame representation local variable slot	Possible native form register, stack location, or optimized away

Debug metadata can preserve source names and line mappings, but the runtime does not need a source-level local variable object.

3.8 Complexity and performance

`max` is $O(1)$ time and $O(1)$ auxiliary space. Bytecode instruction count is not a stable performance measure. One instruction can have complex effects; JIT compilation may inline the entire method; and machine code depends on profiles and CPU.

Compilation time, startup time, warm-up time, and steady-state time are distinct. Incremental IDE builds reduce compilation work. Class-data sharing, lazy class loading, JIT activity, and library initialization affect startup. Hot methods can become faster after profiling, but compilation consumes CPU and code-cache space. Use a harness such as JMH for microbenchmarks rather than timing one invocation around `System.nanoTime()`.

HotSpot note: HotSpot's tiered compilation commonly uses an interpreter and multiple compiled tiers, with C1 and C2 compilers in mainstream builds. Profiling counters, inlining policies, on-stack replacement, and deoptimization are HotSpot mechanisms and can change by version.

WATCH OUT | 3.9 Edge cases and common mistakes

- Treating `javac` as the only valid Java compiler or assuming bytecode uniquely identifies source.
- Assuming all type errors are caught at compile time. Separate compilation, raw types, reflection, malformed bytecode, and linkage can move failures later.
- Confusing the class-file constant pool with the runtime string pool.
- Saying resolution always happens completely at startup.
- Reading `javap` output as the final native instructions.
- Assuming local slot numbers equal physical CPU registers.
- Forgetting static initialization can throw `ExceptionInInitializerError`, followed by later `NoClassDefFoundError` for the erroneous class.
- Benchmarking IDE debug mode and attributing all overhead to Java.

3.10 Production engineering notes

Preserve build provenance: compiler JDK, `--release`, dependency lock state, generated sources, and artifact digest. At runtime, capture the actual command, classpath/module path, flags, and environment. A failure that appears after a library upgrade may be a binary linkage issue even though the service compiled in another module.

Use `javap -c -p -s` to validate overload descriptors, bridge methods, or generated bytecode. Use `-verbose` for constant-pool and class-file metadata. For JIT questions, prefer JFR and supported diagnostics over conclusions drawn from source alone. Optimized code can omit allocations and frames that source structure suggests.

TRY IT | 3.11 Interview questions and model answers

Describe Java execution from source to CPU.

`javac` lexes, parses, resolves names, checks types, and emits class files. At runtime, class loaders find definitions; the JVM verifies, prepares, resolves, and initializes them under specified rules. The execution engine interprets or compiles bytecode. Native instructions then execute on the CPU, with runtime services handling calls, allocation, GC, synchronization, and deoptimization.

Why does Java need bytecode verification if `javac` checked the source?

The JVM may receive class files from any compiler or transformer, not necessarily trusted `javac`. Verification protects runtime invariants such as operand types, control-flow consistency, access, and valid initialization before unsafe operations execute.

What is in the constant pool?

It is a class-file table containing constants and symbolic references used by the class. Runtime structures derive from it during loading and resolution. It is not merely a pool of Java `String` objects.

Can the JIT change program behavior?

It may change implementation and timing but must preserve allowed Java behavior. For data-racy programs, the JVM may permit multiple outcomes, so optimization can expose an already legal outcome rather than violate semantics.

TRY IT | 3.12 Exercises

- 1 Compile the example and annotate every `main` instruction from `javap -c`.
- 2 Add an instance method and compare `invokevirtual` with `invokestatic`.
- 3 Add a static initializer that prints a message; predict and observe when it runs.
- 4 Compile with and without `-g`, then compare `javap -l -v` output.
- 5 Break runtime binary compatibility by compiling against a method and removing it from the runtime class; identify the error phase.
- 6 Draw the complete timeline for your IDE's Run command, including OS process creation.

RECAP | 3.13 Chapter summary

Java execution is a sequence of representations and validation boundaries. Source compilation produces specified class-file structures and stack-oriented bytecode. Runtime loading, linking, initialization, and execution connect symbolic definitions to live classes and host instructions. Interpretation and JIT compilation are implementation strategies; optimized native state can differ radically from source structure while preserving specified behavior.

TRY IT | 3.14 Revision checklist

- ☐ I can name the major compiler front-end phases.
- ☐ I can explain class files, descriptors, and the constant pool.
- ☐ I can read loads, stores, branches, field access, calls, and returns in `javap -c`.
- ☐ I can distinguish loading, linking, initialization, interpretation, and JIT compilation.
- ☐ I can explain an IDE Run action through to CPU execution.
- ☐ I do not confuse bytecode with native code or JVM implementation policy with the JVM.

JAVA FOUNDATIONS TO ADVANCED ENGINEERING

Part II - JVM Architecture and Memory

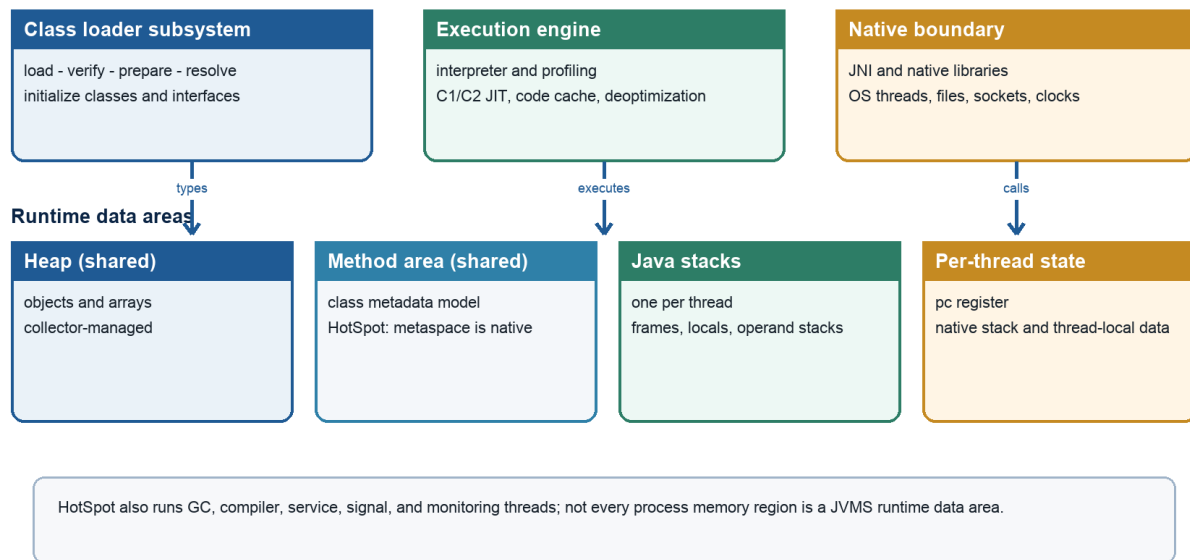
A focused part of the complete SDE-2 preparation guide

SDE-2 CORE CHAPTER

Chapter 4 - JVM Architecture

JVM architecture

Abstract runtime areas plus a typical HotSpot execution environment



Java Foundations to Advanced Engineering

Figure 4.1 - JVM architecture and representative HotSpot runtime components

4.1 Learning objectives

- Describe the major JVM subsystems and how they cooperate.
- Distinguish specified JVM runtime concepts from one implementation's process layout.
- Explain the roles of class loading, runtime data areas, execution, garbage collection, JNI, and native libraries.
- Place application, compiler, GC, service, and operating-system threads in one process model.
- Explain safepoints and stop-the-world operations without claiming that every pause is a GC pause.

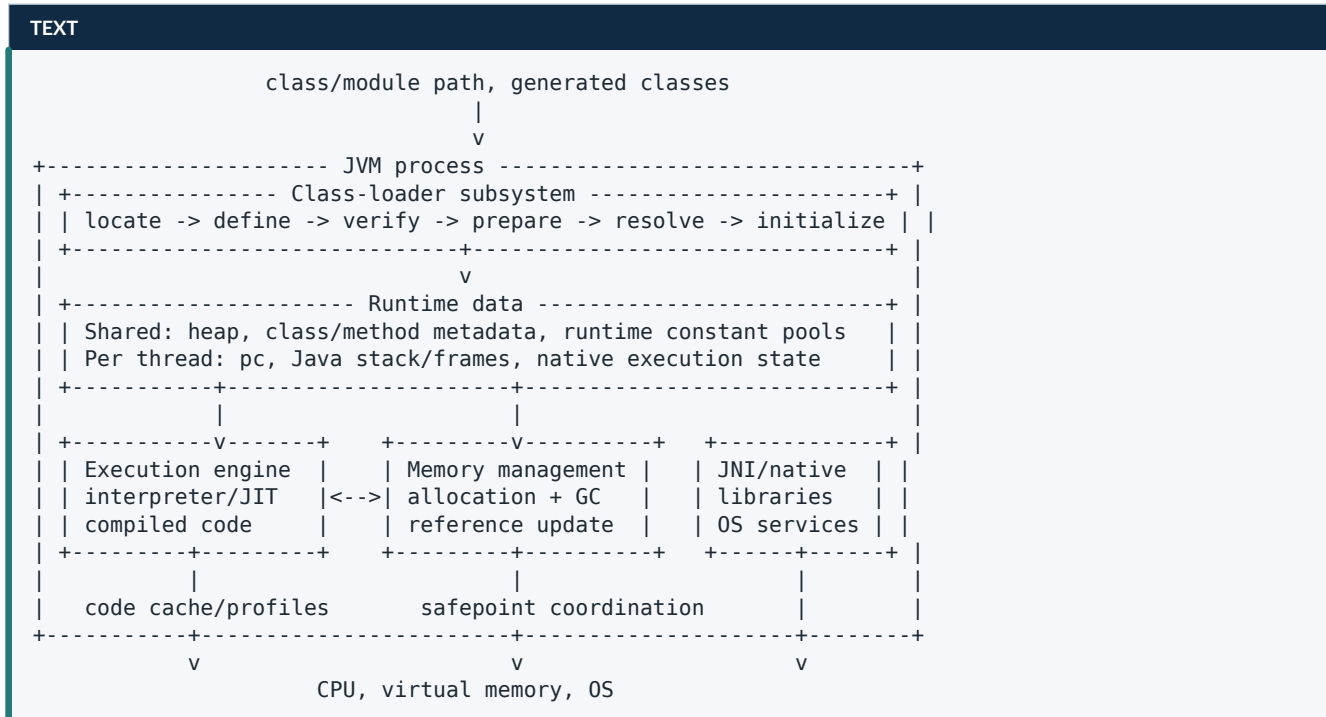
WHY IT WORKS | 4.2 Why this matters at SDE-2

A production Java process is not simply "the heap plus application threads." Startup failures, allocation stalls, code-cache exhaustion, native memory growth, class-loader leaks, JNI bugs, and safepoint pauses live in different subsystems. A useful incident hypothesis begins by locating evidence in the correct region.

Architecture is also a common interview gateway. The interviewer is testing whether you can connect components rather than recite a box diagram: a class loader supplies verified code and metadata; stack frames execute bytecodes that reference heap objects; the JIT places machine code in a code cache; the collector coordinates with all threads to discover and update references; JNI crosses into native code with additional safety obligations.

WHY IT WORKS | 4.3 First-principles model

The JVM describes an abstract machine. A concrete JVM is a native process that realizes that model while integrating with a host OS and CPU. Its high-level data and control flow is:



The arrows matter. Execution triggers class loading and allocation. GC needs precise knowledge of references in frames and compiled code. Class unloading depends on reachability of defining loaders. Native calls can retain references and block coordination. No subsystem is operationally isolated.

4.4 Core terminology

- **Class-loader subsystem:** Finds class-file bytes and creates runtime class definitions under loader-specific namespaces.
- **Runtime data areas:** Memory regions described by the JVM, some shared and some per thread.
- **Execution engine:** The implementation that performs method semantics, possibly via interpretation and compilation.
- **Code cache:** Implementation memory holding generated native code and related data.
- **Garbage collector:** Reclaims storage for objects that can no longer affect execution.
- **JNI:** Java Native Interface, a standard native interoperability boundary.
- **Safepoint:** An implementation state where selected runtime invariants allow a global VM operation to inspect or modify managed state.
- **Stop the world (STW):** A period in which relevant Java application threads are paused for a VM operation.
- **VM thread:** An implementation thread coordinating certain JVM-wide operations.

4.5 Detailed mechanics

The class-loader subsystem receives a binary name request, delegates or searches according to loader policy, obtains bytes, and asks the JVM to define a class. The JVM protects type safety through class-file validation and linking. Runtime class identity includes both binary name and defining loader. Initialization executes static initialization logic before specified active uses.

The runtime data areas provide execution state. Each Java thread has a program counter and JVM stack with a frame per active method. Frames hold local-variable slots, an operand stack, and linkage/return information. Objects and arrays usually inhabit a shared heap. Per-class runtime constant pools and method/class structures represent loaded definitions. Native method execution may require a native stack or equivalent host mechanism.

The execution engine obeys bytecode semantics. An implementation can interpret, compile, or mix both. Compiled code still cooperates with the runtime: it contains metadata identifying reference locations, points where a thread can stop, exception mappings, and paths back to less optimized execution.

Memory management handles allocation and reclamation. Allocation often has a fast path, but a failed fast path can request a new allocation region or trigger collection. The collector identifies roots in threads, class metadata, JNI handles, and runtime structures, then traces references. Depending on algorithm, it marks, copies, compacts, or reclaims regions and updates references.

JNI permits native functions to receive Java values and controlled object handles. Native code can call Java methods and use platform libraries. It must respect JNI reference lifetimes and cannot treat managed object addresses as permanently stable. Critical native sections and long native calls can interfere with collection or safepoint coordination depending on implementation.

Threads in a JVM process can include:

- application platform threads mapped to OS threads;
- carrier threads used to run virtual threads;
- GC workers and concurrent marking/relocation workers;
- JIT compiler threads;
- signal, attach, service, timer, and reference-processing threads;
- VM coordination threads;
- threads created by native libraries.

Specification boundary: The JVM specification defines abstract runtime data areas and execution behavior. It does not require HotSpot's Metaspace, named compiler tiers, TLABs, a particular collector, one OS thread per platform thread, or a particular internal thread inventory.

Safepoints support operations that require a stable view, such as some GC phases, deoptimization, class redefinition, biased-lock cleanup in historical releases, or certain diagnostic operations. Threads do not necessarily stop at the exact instant a request is made. They reach a safe state through polling or transitions, and the delay until all required threads arrive is distinct from the duration of the operation itself.

Stop-the-world is a coordination property, not a synonym for full GC. A young collection may pause the world; a mostly concurrent collector still has short pause phases; deoptimization or a diagnostic command may also stop threads. Conversely, much GC work can run concurrently with application threads.

TRACE IT | 4.6 Worked Java example

JAVA

```
import java.util.ArrayList;
import java.util.List;

public final class ArchitectureTour {
    private static final List<byte[]> RETAINED = new ArrayList<>();

    static int compute(int value) {
        return value * 31 + 7;
    }

    public static void main(String[] args) throws Exception {
        for (int i = 0; i < 10_000; i++) {
            compute(i);
        }
        RETAINED.add(new byte[1024 * 1024]);
        Thread worker = new Thread(() -> System.out.println(compute(5)), "worker");
        worker.start();
        worker.join();
    }
}
```

This small program touches every main box: classes are loaded, calls create frames, an array is allocated, a static collection retains it, repeated execution can create JIT profiles, a new application thread appears, and `join` coordinates completion.

TRACE IT | 4.7 Execution or memory walkthrough

- 1 The launcher creates a JVM process and establishes the initial thread and runtime services.
- 2 `ArchitectureTour` and referenced platform classes are loaded and linked. Static initialization creates `RETAINED`.
- 3 Each `compute` invocation has method state. The optimizing runtime may later inline the calculation into the loop, eliminating a physical call frame in compiled execution.
- 4 `new byte[...]` requests heap storage. The local reference is passed to `ArrayList.add` and stored in the shared list's backing array.
- 5 Because `RETAINED` is reachable through a static field of a live class, the byte array remains reachable after `main`'s local expression finishes.
- 6 `worker.start()` asks the runtime and OS scheduling layer to arrange concurrent execution. Its lambda invokes `compute` and prints.
- 7 `join()` blocks the main thread until the worker terminates and also establishes a Java Memory Model ordering relationship.
- 8 At a collection pause, reference maps for frames and compiled code allow the collector to find live references. The exact internal algorithm depends on the chosen collector.

4.8 Complexity and performance

The loop is $O(n)$ time and $O(1)$ Java-level auxiliary space. The retained allocation is $O(m)$ space for an m -byte array. Architecture adds non-algorithmic costs: loading and verification, compilation CPU, metadata, thread stacks, heap reservation/commit, and GC work.

JVM process memory is broader than maximum heap:

TEXT

```
resident/process memory
= committed heap portions
+ class metadata
+ code cache
+ thread stacks
+ direct/native buffers
+ GC/compiler/runtime native structures
+ loaded native libraries and allocator overhead
```

This is a conceptual accounting identity, not a promise that OS tools categorize every byte cleanly. Thread count can dominate native stack reservation, and class generation can dominate metadata even when the heap appears stable.

WATCH OUT | 4.9 Edge cases and common mistakes

- Drawing Metaspace inside the heap and calling that a JVM requirement.
- Claiming every method call creates a visible frame after JIT inlining.
- Assuming an empty Java heap explains low process memory.
- Calling every application pause a GC pause.
- Assuming a concurrent collector never stops the world.
- Treating JNI as ordinary safe Java; native code can corrupt memory or crash the process.
- Forgetting static fields store references, not the referenced objects "inside the class."
- Assuming all JVM threads correspond to business request threads.

4.10 Production engineering notes

Measure the whole process. Heap metrics, native memory tracking where available, thread counts, class counts, code-cache state, direct-buffer usage, GC telemetry, and OS RSS answer different questions. A container limit must cover non-heap memory and transient native needs, not just `-Xmx`.

For long pauses, separate time-to-safepoint from VM-operation time and inspect CPU starvation, native transitions, and thread states. For high CPU, distinguish application, compiler, GC, and native threads using profiles. For class growth, group by class loader. For native crashes, preserve `hs_err`-style error logs and native library versions.

HotSpot note: HotSpot exposes architecture details through JFR, unified logging, `j cmd`, serviceability agents, Native Memory Tracking, and many `-XX` options. Names and availability change by release; they must be validated against the deployed build.

TRY IT | 4.11 Interview questions and model answers

What are the main JVM components?

I group them as class loading/linking, runtime data areas, execution, memory management, and native integration. Then I explain their interactions: execution uses frames and heap objects; the JIT emits code plus GC metadata; the collector coordinates with threads; loaders define type identity; JNI crosses into host libraries.

What is a safepoint?

It is an implementation-defined safe execution state used for global VM operations that need consistent managed state. A safepoint request may pause relevant Java threads, and reaching the safepoint can itself take time. The JLS/JVMS do not mandate HotSpot's safepoint mechanism.

Does stop-the-world mean full GC?

No. STW describes thread suspension. Young GC and short phases of concurrent collectors can be STW, and non-GC VM operations can also require it. A full GC describes collection scope or a collector-specific event, not every pause.

Why can process memory exceed `-Xmx`?

`-Xmx` limits the Java heap, not class metadata, code cache, thread stacks, native buffers, JVM structures, libraries, or allocator overhead.

TRY IT | 4.12 Exercises

- 1 Redraw the architecture diagram and add an arrow for every allocation and native call.
- 2 Run the example, capture a thread dump, and classify visible threads as application or runtime support.
- 3 Explain why the retained byte array survives collection.
- 4 Build a memory budget for a 1 GiB container with a 600 MiB heap.
- 5 Give three STW operations, at least one not caused by GC.

RECAP | 4.13 Chapter summary

A JVM is an interacting runtime, not a single bytecode loop. Class loading establishes definitions and identity; runtime data areas hold shared and per-thread state; the execution engine interprets or compiles; memory management allocates and reclaims; JNI connects native facilities. Safepoints coordinate operations requiring stable managed state. The abstract specification and a concrete HotSpot process must be discussed at different levels.

TRY IT | 4.14 Revision checklist

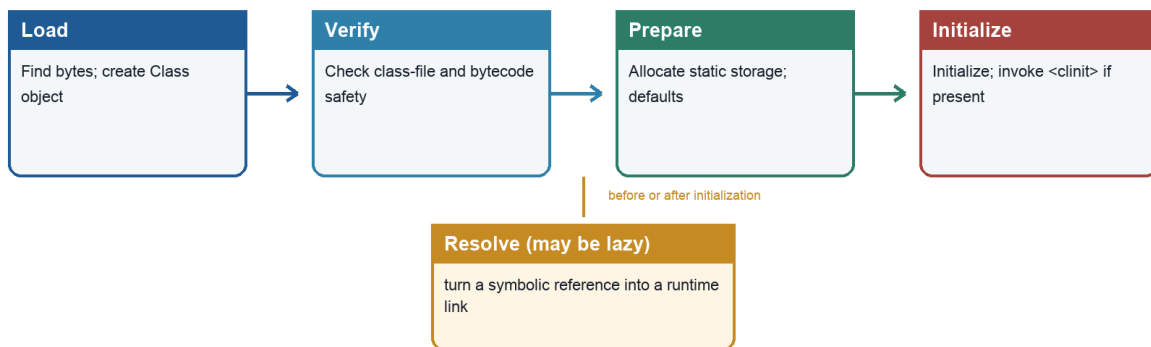
- ☐ I can explain the full JVM architecture as a connected system.
- ☐ I can separate shared runtime state from per-thread state.
- ☐ I can identify non-heap contributors to process memory.
- ☐ I can distinguish a safepoint, an STW pause, and a full GC.
- ☐ I understand why JIT code and GC require shared metadata.
- ☐ I label HotSpot-specific structures as implementation details.

SDE-2 CORE CHAPTER

Chapter 5 - Class Loading, Linking, and Initialization

Class lifecycle

Loading, linking, and initialization are distinct events



Parent delegation controls lookup; class identity is (binary name, defining loader).

Initialization uses a unique per-class initialization lock and follows specified dependency rules.

`NoClassDefFoundError` can report a missing definition or a class whose earlier initialization failed.

Java Foundations to Advanced Engineering

Figure 5.1 - Loading, linking, and initialization lifecycle

5.1 Learning objectives

- Explain loading, verification, preparation, resolution, and initialization.
- Describe bootstrap, platform, and application class loaders in modern Java.
- Reason about parent delegation, custom loaders, namespaces, and class identity.
- Predict static initialization order, including failure and circularity cases.
- Diagnose `ClassNotFoundException`, `NoClassDefFoundError`, and common linkage errors.

WHY IT WORKS | 5.2 Why this matters at SDE-2

Class loading enables plugin systems, application servers, agents, hot deployment, JDBC drivers, and framework discovery. It also creates difficult failures: a class can exist in a JAR but be invisible to a particular loader; two classes with the same name can be different types; an initialization failure can poison later use; and retained class loaders can leak entire application generations.

At SDE-2, "check the classpath" is only the start. You should identify the initiating and defining loader, distinguish lookup failure from definition/linkage failure, and explain why changing delegation order affects both isolation and security.

WHY IT WORKS | 5.3 First-principles model

A class name is not sufficient to create a runtime type. The JVM needs class-file bytes and a defining class loader. A useful pipeline is:

TEXT

```
binary-name request
  |
  v
loading: find bytes and create Class representation
  |
  v
linking:
  verification -> preparation -> resolution
  |
  v
initialization: execute class initialization method once
  |
  v
active runtime use
```

These phases have ordering constraints but can be lazy. A class may be loaded without being initialized. Resolution of some symbolic references can be delayed until first use.

Specification boundary: The JLS defines when class or interface initialization is required and its synchronization/failure behavior. The JVMs defines loading, linking, class-file verification, and runtime constraints. The names and exact search behavior of built-in loaders are platform API/implementation matters rather than language semantics.

5.4 Core terminology

- **Binary name:** Runtime name such as `com.example.OrderService`.
- **Initiating loader:** A loader through which loading of a class was initiated.
- **Defining loader:** The loader that calls into the JVM to define that class.
- **Loader namespace:** The set of type definitions visible under a loader's delegation relationships.
- **Parent delegation:** Asking a parent loader before attempting a local definition.
- **Preparation:** Allocating static field storage and assigning default values.
- **Initialization:** Executing static field initializers and static blocks in textual order through the generated class initialization method.
- **Active use:** An operation that triggers initialization, such as certain static field access, static method invocation, construction, or reflective requests.
- **Linkage error:** An `Error` indicating incompatible or unsatisfied class relationships at runtime.

5.5 Detailed mechanics

Loading begins from a request for a binary name. A loader may return a class it already loaded, delegate, read a class from a JAR/network/generated source, transform bytes, then define it. The JVM associates the definition with that loader. Arrays are created specially by the JVM, with their component type influencing loader identity.

Verification rejects malformed or type-unsafe class files. Checks include structural validity, valid instruction operands and control flow, legal stack states, access constraints, and correct object initialization patterns. Verification protects the JVM even when bytes came from a compiler other than `javac` or were transformed by an agent.

Preparation creates static field storage with default values. It is important not to confuse this with executing Java initializers. Given `static int size = 42`, preparation establishes `size` as 0; initialization later assigns 42. Compile-time constant variables can be represented through a `ConstantValue` attribute and have special initialization/use behavior.

Resolution turns symbolic references from runtime constant pools into concrete classes, fields, methods, or interfaces, checking access and compatibility. Implementations may resolve lazily. Therefore an incompatible method can remain unnoticed until a path first executes its invocation.

Initialization executes a class's generated `<clinit>` logic, if present. Before a class initializes, its superclass initializes. Static field initializers and static blocks execute in source order. Initialization is synchronized per class so only one thread performs it while other threads wait, subject to specified recursive handling.

Merely referring to `SomeType.class` does not necessarily initialize `SomeType`. Reading a compile-time constant through a class name can inline the value into the client and may not initialize the declaring class. Creating an instance, invoking a static method, or reading a non-constant static field triggers initialization.

Modern built-in loaders are commonly described as:

- **Bootstrap loader:** Loads foundational runtime classes, represented as `null` by some reflection APIs.

- **Platform class loader:** Loads selected platform modules/classes not loaded by bootstrap.
- **Application (system) class loader:** Loads application classes from the configured class path/module path.

The application loader is not guaranteed to be a `URLClassLoader` in modern Java. Code that casts it based on Java 8 behavior is fragile.

Parent-first delegation prevents an application from casually substituting foundational platform definitions and promotes a single shared definition. Some containers use carefully designed child-first or selective policies for application isolation. A custom loader should normally call `loadClass` through the established locking/delegation protocol and implement `findClass` for local lookup.

Class identity is approximately `(binary name, defining loader)`. If loader A and loader B independently define bytes named `plugin.Message`, instances are not assignment compatible, even if byte-for-byte identical. A shared parent-loaded interface can bridge plugin implementations.

TRACE IT | 5.6 Worked Java example

JAVA

```
public final class InitializationOrder {
    static int first = trace("first", 10);
    static int second = first + trace("second", 20);

    static {
        System.out.println("block: first=" + first + ", second=" + second);
    }

    static int trace(String label, int value) {
        System.out.println(label + " -> " + value);
        return value;
    }

    public static void main(String[] args) {
        System.out.println("main: " + second);
    }
}
```

Expected output:

TEXT

```
first -> 10
second -> 20
block: first=10, second=30
main: 30
```

Preparation first gives `first` and `second` the value 0. Initialization then follows textual order: assign 10, evaluate `10 + 20`, run the block, and finally invoke `main`.

TRACE IT | 5.7 Execution or memory walkthrough

Consider two threads concurrently invoking `InitializationOrder.main` through reflection:

- 1 The class can already be loaded, verified, and prepared, with static fields at defaults.
- 2 Thread T1 requests active use and obtains the class initialization lock.
- 3 T2 requests active use and waits.
- 4 T1 executes the initializers in order. References needed during execution live in its frames; static values belong to runtime class state.
- 5 T1 completes normally. The class is marked initialized.
- 6 T2 resumes and observes the completed initialization effects, then proceeds without rerunning `<clinit>`.

If T1's initializer throws an unchecked exception, first active use commonly observes `ExceptionInInitializerError` (unless the throwable already has special `Error` treatment). The class is marked erroneous. A later use can fail with `NoClassDefFoundError: Could not initialize class ...`.

Circular initialization is subtler than a dead loop. If class `A` initializes and reads `B.value`, `B` may initialize and read a not-yet-assigned `A.value`, observing its prepared default. Recursive initialization by the same thread is allowed to proceed rather than reentering the initializer. Cross-thread cycles can deadlock when two initialization locks and mutually dependent initializers interact.

5.8 Complexity and performance

Loading cost scales with class-file input, verification, metadata construction, I/O, and dependency resolution. Initialization cost is arbitrary Java code and can include network calls, locks, or large allocations, although such behavior is usually a design smell. Many small generated classes increase metadata and startup work. Class lookup caches make repeated successful loads cheap, but custom loaders can defeat sharing.

Unloading is normally collective: a class can become unloadable only when its defining loader is unreachable and its classes/instances are no longer strongly reachable under implementation criteria. One retained loader can therefore retain class metadata, static fields, generated proxies, and related graphs.

WATCH OUT | 5.9 Edge cases and common mistakes

- Calling loading and initialization the same operation.
- Saying static fields receive explicit initializer values during preparation.
- Using `Class.forName(name)` without realizing common overloads initialize the class; `ClassLoader.loadClass` normally does not initialize it.
- Assuming same class name means same type across loaders.
- Casting the system loader to `URLClassLoader` on modern JDKs.
- Catching only `ClassNotFoundException` when failures include `NoClassDefFoundError`, `UnsupportedClassVersionError`, `VerifyError`, `NoSuchMethodError`, or `IncompatibleClassChangeError`.
- Performing slow external I/O in static initialization.
- Using a child-first loader without protecting platform/shared API packages.

`ClassNotFoundException` is checked and typically reported by an explicit lookup API that cannot find a requested definition. `NoClassDefFoundError` is unchecked and means the JVM or runtime tried to use a definition that cannot be provided, including cases where it existed during compilation or its initialization previously failed. The message and causal chain are essential evidence.

5.10 Production engineering notes

For a class-loading incident, capture the exact runtime artifact set, module/class path, class name, exception chain, initiating context, and defining loader. Unified class-load logging can reveal sources and loaders. Dependency-tree output can expose version conflicts, but the effective packaged artifact is authoritative.

Plugin boundaries should place shared API types in a parent loader and implementations in isolated child loaders. On unload, close loaders/resources and remove listeners, thread locals, executor threads, JDBC drivers, logging contexts, and caches that point back to the plugin loader. Threads inherit context class loaders, which are a frequent retention path.

HotSpot note: HotSpot stores class metadata in native Metaspace and can log class loading with unified logging such as class-related tags. Exact flags, log formats, and unloading policy depend on version and collector.

TRY IT | 5.11 Interview questions and model answers

What are the class life-cycle phases?

Loading finds bytes and creates a definition. Linking verifies it, prepares static state, and resolves symbolic references, potentially lazily. Initialization executes static initialization when required by active use. Loading does not imply initialization.

How do `ClassNotFoundException` and `NoClassDefFoundError` differ?

`ClassNotFoundException` is a checked lookup failure from APIs such as class-loader or reflection operations. `NoClassDefFoundError` is a linkage-time/runtime inability to provide a class definition required by executing code, and it can also follow failed class initialization. I would inspect the cause and loader context rather than classify only by name.

Why can identical classes fail a cast?

Runtime identity includes the defining loader. Two loaders that each define `com.example.Message` create distinct types. The solution is usually a shared parent-loaded contract, not a forced cast.

What is parent delegation for?

It promotes consistent shared definitions and protects foundational classes from accidental replacement. Isolation systems may alter the policy selectively, but then must manage type boundaries and security carefully.

TRY IT | 5.12 Exercises

- 1 Predict the output of the worked example, then inspect `<clinit>` using `javap -c`.
- 2 Add a `static final int CONSTANT = 7` and a side-effecting initializer. Test which references trigger initialization.
- 3 Create two `URLClassLoader` instances with no shared application parent for a sample class and demonstrate failed assignment compatibility.
- 4 Construct a safe example of failed static initialization and compare first and second errors.
- 5 Draw a class-loader leak path involving a thread context class loader.

RECAP | 5.13 Chapter summary

Class loading turns named byte streams into loader-scoped runtime types. Linking verifies safety, prepares default static state, and resolves symbolic relationships. Initialization later runs Java code under a synchronized, once-only protocol. Loader identity enables isolation but makes namespaces and leaks important. Precise phase and loader reasoning turns vague classpath failures into diagnosable engineering problems.

TRY IT | 5.14 Revision checklist

- ☐ I can distinguish all loading, linking, and initialization phases.
- ☐ I know preparation assigns defaults before Java initializers run.
- ☐ I can explain built-in loaders without outdated inheritance assumptions.
- ☐ I can state class identity as name plus defining loader.
- ☐ I can explain normal, failed, and circular initialization.
- ☐ I can diagnose the major class-not-found and linkage error categories.

SDE-2 CORE CHAPTER

Chapter 6 - Runtime Data Areas

Runtime data areas

Shared state, per-thread state, and native implementation memory

Shared by JVM threads

Heap

objects, arrays, GC regions
logical generations depend on collector

Method area model

runtime constant pools and type data
HotSpot stores class metadata in native metaspace

Specified per-thread areas and optional implementation state

Java stack

frame per active invocation
locals + operand stack + frame data

pc register

current JVM instruction location
undefined during a native method

Native method stack

per thread if the JVM provides one
native calling state is
implementation-specific

Optional HotSpot state

TLAB may be enabled or absent
other thread-local runtime structures

Java Foundations to Advanced Engineering

Figure 6.1 - Shared and per-thread runtime data areas

6.1 Learning objectives

- Identify JVM runtime data areas and whether they are shared or per thread.
- Describe frames, local-variable arrays, operand stacks, program counters, heaps, runtime constant pools, and native method stacks.
- Place HotSpot Metaspace, code cache, and TLABs outside the strict JVMS abstraction.
- Trace references across stack frames, heap objects, and class metadata.
- Diagnose common exhaustion errors by region rather than treating all memory as heap.

WHY IT WORKS | 6.2 Why this matters at SDE-2

Memory incidents require a map. A heap dump cannot explain every native leak; increasing `-Xmx` does not fix stack overflow; lowering thread count can recover native memory without changing live heap; generated classes can exhaust metadata; and JIT compilation can fail when a code cache fills. The map also grounds pass-by-value, recursion, allocation, garbage collection, and concurrency.

Interview answers often misuse phrases such as "the object is on the stack" or "static variables live in Metaspace." An SDE-2 should distinguish an object, a reference value, specified logical storage, and one runtime's physical representation.

WHY IT WORKS | 6.3 First-principles model

Execution needs three broad kinds of state:

- 1 Shared program and object state available to multiple threads.
- 2 Per-thread control state describing active calls and next execution points.
- 3 Runtime implementation state supporting compilation, collection, and native integration.

TEXT

```
JVM process
|
+-- shared logical state
|   +-- heap: objects and arrays
|   +-- per-class structures and runtime constant pools
|
+-- thread T1
|   +-- pc
|   +-- JVM stack: frame -> frame -> frame
|   +-- native method execution support
|
+-- thread T2
|   +-- pc
|   +-- JVM stack: frame -> frame
|   +-- native method execution support
|
+-- implementation memory
    +-- generated code, compiler/GC data, native allocations
```

Specification boundary: The JVM specification defines logical runtime data areas and permitted failures. It deliberately allows implementation freedom over contiguous memory, exact location, allocation strategy, and whether some areas are fixed or dynamically expanded.

6.4 Core terminology

- **Heap:** Shared runtime area from which storage for all class instances and arrays is allocated in the JVM model.
- **JVM stack:** Per-thread area containing frames for Java method invocations.
- **Frame:** Per-invocation state with local variables, operand stack, and dynamic linking/return support.
- **Local-variable array:** Indexed frame slots holding parameters and locals in the abstract execution model.
- **Operand stack:** LIFO workspace used by JVM instructions.
- **PC register:** Per-thread current instruction address/position for non-native execution in the abstract model.
- **Method area:** Shared logical area for per-class structures, method data/code, and runtime constant pools.
- **Runtime constant pool:** Per-class runtime representation derived from the class-file constant pool.
- **Native method stack:** Implementation support for methods outside JVM bytecode.
- **Metaspace:** HotSpot native-memory implementation for much class metadata.
- **TLAB:** HotSpot thread-local allocation buffer carved from shared heap space.

6.5 Detailed mechanics

The heap stores arrays and class instances. A reference value in a local slot or field can designate a heap object or be `null`. The specification does not expose a raw address, and a collector can relocate an object while preserving reference semantics. Heap storage is shared, but reachability and synchronization determine whether threads can safely use an object.

Each started thread has a JVM stack. Invoking a Java method creates a new frame for that invocation; normal return or abrupt exception unwinding destroys it. A frame's size can be derived from class-file method metadata plus implementation needs. Excessive call depth can produce `StackOverflowError`. Failure to create or expand a stack can lead to `OutOfMemoryError` in permitted circumstances.

The local-variable array uses slots. Instance method slot 0 initially holds `this`; subsequent slots hold parameters. `long` and `double` have historical two-slot treatment in class-file frame representation, although implementations can optimize physical storage. A source local can share a slot with a later non-overlapping local. Names exist only if optional debug metadata preserves them.

The operand stack evaluates expressions and passes invocation arguments in bytecode. Instructions push constants or loaded locals, consume operands, and push results. Its maximum depth is stored in method class-file metadata. It is not the same thing as the entire JVM stack.

The PC register lets each independently executing thread track its current JVM instruction. For a native method its value is undefined by the abstract specification. A context switch saves enough host/runtime state to resume execution; the JLS does not prescribe OS scheduler mechanics.

The method area is a logical shared area holding per-class structures such as field/method information, method code, and runtime constant pools. The name does not imply a specific physical region. The runtime constant

pool includes numeric/string constants and symbolic field/method/type references derived from the class file, with resolution state maintained at runtime.

Native methods use host ABI and implementation machinery, often involving native stacks. A deep or faulty native call can exhaust or corrupt native state independently of Java heap correctness.

HotSpot note: Modern HotSpot stores substantial class metadata in Metaspace, backed by native memory. Generated machine code is placed in segmented code heaps commonly called the code cache. These do not redefine the JVM's method area; they are implementation structures satisfying and extending the abstract model.

TLABs improve allocation scalability. A thread receives a region within the heap and can often allocate by advancing a pointer without contending on a global allocator. The buffer is thread local as an allocation mechanism, but objects allocated there are ordinary heap objects and can immediately escape to other threads. Large allocations or exhausted TLABs take other paths.

TRACE IT | 6.6 Worked Java example

JAVA

```
public final class AreaWalkthrough {
    private static int created;

    private final int id;

    AreaWalkthrough(int id) {
        this.id = id;
        created++;
    }

    static int score(AreaWalkthrough item, int bonus) {
        int base = item.id * 10;
        return base + bonus;
    }

    public static void main(String[] args) {
        AreaWalkthrough item = new AreaWalkthrough(7);
        int result = score(item, 3);
        System.out.println(result + ":" + created);
    }
}
```

The likely output is `73:1`. The important lesson is where values conceptually participate, not a claim that a particular JIT leaves each value in the named region.

TRACE IT | 6.7 Execution or memory walkthrough

Before construction, initialization establishes class state including static `created`. In `main`:

TEXT	
Shared heap/class state	main frame
+-----+	+-----+
object #A	local: item reference
id = 7	local: result = 73
+-----+	operand stack ...
class state: created = 1	+-----+
+-----+	

Step by step:

- 1 `new` obtains object storage, generally from the heap model, and default-initializes `id` to 0.
- 2 The constructor frame receives the new reference as `this` and integer 7 as a parameter.
- 3 `putfield`-style behavior assigns 7 to the object's `id`; static state `created` is read, incremented, and written.
- 4 The constructor frame returns. The reference remains in `main`'s local state.
- 5 Calling `score` creates a frame. Its locals include the copied reference value and copied integer 3.
- 6 Bytecodes load the reference, read `id`, multiply, store or retain `base`, add `bonus`, and return 73.
- 7 The `score` frame disappears; the integer result is stored in `main`'s frame.
- 8 After `main` returns, `item` no longer exists as a local root. The object becomes collectible unless another reference was stored elsewhere.

Under optimization, `score` can be inlined, `base` can remain in a register, and the object could potentially be scalar-replaced if escape analysis proves it unnecessary. The source-level semantics remain the same.

6.8 Complexity and performance

`score` is $O(1)$ time and space. Each non-inlined invocation consumes frame state proportional to method metadata and implementation needs. Recursion consumes $O(\text{depth})$ logical call state unless optimized in ways Java does not guarantee.

Allocation through a TLAB can be extremely cheap, often resembling pointer bump plus initialization. Cost is deferred rather than absent: the collector later processes live/dead objects, and TLAB refills and large objects require slower paths. Code-cache pressure can reduce compilation effectiveness; metadata pressure can trigger class unloading attempts; too many threads can consume large native stack reservations.

WATCH OUT | 6.9 Edge cases and common mistakes

- Saying local variables always physically live on a native stack. Optimized values can live in registers or disappear.
- Saying all objects always physically remain in heap memory. The JVM heap is the semantic allocation model, but scalar replacement may eliminate an allocation.
- Calling Metaspace the heap or saying static object values are stored "in Metaspace." Static state contains values/references associated with class runtime data; referenced objects remain ordinary objects.
- Confusing the operand stack with the per-thread JVM stack.
- Assuming thread-local allocation means thread-confined objects.
- Confusing the runtime constant pool with string interning.
- Tuning only `-Xmx` when native stacks or metadata cause process exhaustion.
- Expecting a heap dump to contain direct native memory or all JVM structures.

6.10 Production engineering notes

Match symptom to evidence:

Symptom	First regions/evidence to inspect
Java heap space	live-set growth, allocation rate, heap dump, GC behavior
Metaspace	loaded class counts, class loaders, generated classes
StackOverflowError	recursion/call cycle and per-thread stack sizing
unable to create native thread	thread count, stack size, process/container limits, native memory
code cache warnings	compilation logs/JFR and code-cache status
container OOM kill without Java OOME	total RSS, native buffers, stacks, JVM/native structures

Do not react to reserved address space as if it were committed resident memory. OS virtual size, committed JVM memory, and RSS are different. For native growth in HotSpot, Native Memory Tracking can help if enabled with an acceptable overhead and the relevant allocations are tracked.

TRY IT | 6.11 Interview questions and model answers

Which JVM areas are shared and which are per thread?

The heap and method-area/runtime class structures are shared. Each thread has a PC, JVM stack with frames, and native execution support. Implementation structures such as a shared code cache and per-thread TLABs must be labeled separately.

What is in a stack frame?

The abstract frame contains local-variable slots, an operand stack, and information supporting dynamic linking, return, and exceptions. A JIT may inline or optimize physical frames, so a source invocation is not always a materialized native frame.

Is a TLAB outside the heap?

No. In HotSpot it is a thread-owned allocation slice of the heap. Ownership reduces allocation contention; it does not imply that allocated objects are forever thread local.

What is the relationship between method area and Metaspace?

The method area is a JVM specification concept for per-class structures. Metaspace is HotSpot's native-memory implementation for much class metadata. They should not be treated as specification synonyms.

TRY IT | 6.12 Exercises

- 1 Draw frames and heap state for the worked example at each method call.
- 2 Use `javap -c -v` to find `max_stack` and `max_locals` for `score`.
- 3 Write bounded recursion and observe how failure changes with stack-size settings in a safe local process.
- 4 Create many dynamic proxy classes with isolated loaders and predict the affected region.
- 5 Build a complete memory budget including heap, stacks, metadata, code cache, direct buffers, and margin.

RECAP | 6.13 Chapter summary

The runtime data-area model separates shared objects and class state from per-thread execution state. Frames use local slots and operand stacks; the PC tracks instruction progress; native calls require host support. HotSpot adds concrete structures such as Metaspace, code cache, and TLABs. Correct reasoning follows references and execution state while avoiding unsupported claims about exact physical placement.

TRY IT | 6.14 Revision checklist

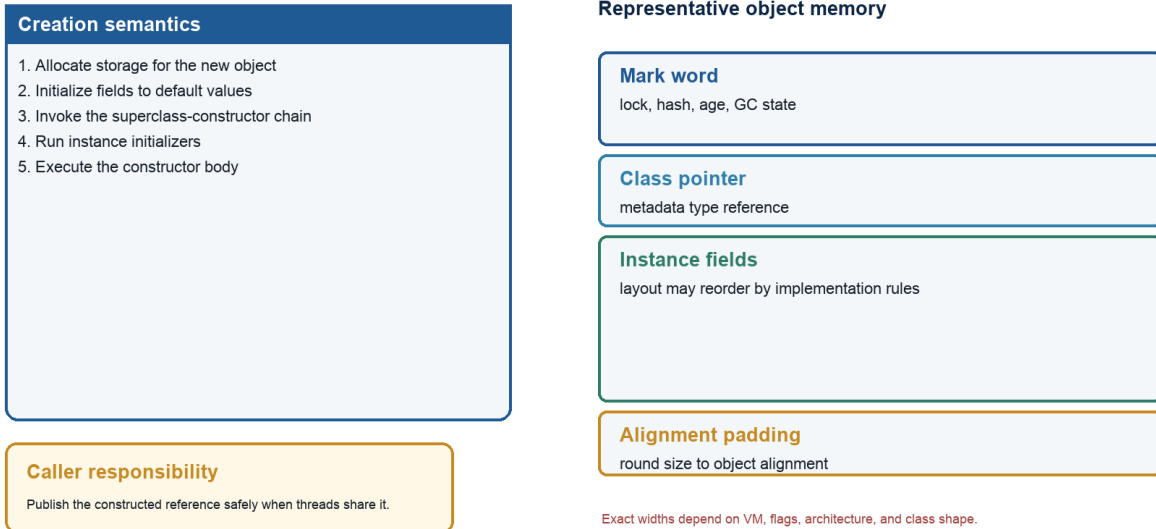
- ☐ I can classify each runtime data area as shared or per thread.
- ☐ I can distinguish a JVM stack, frame, local array, and operand stack.
- ☐ I can trace a reference from a local slot to a heap object.
- ☐ I can explain method area versus Metaspace and heap versus TLAB.
- ☐ I know why process memory can exceed heap memory.
- ☐ I can map common exhaustion symptoms to the right evidence.

SDE-2 CORE CHAPTER

Chapter 7 - Object Creation and Memory Layout

Object creation and layout

Language semantics on the left; representative 64-bit HotSpot layout on the right



Java Foundations to Advanced Engineering

Figure 7.1 - Object creation semantics and representative HotSpot layout

7.1 Learning objectives

- Trace a **new** expression through class readiness, allocation, zeroing, and constructor execution.
- Distinguish object semantics from HotSpot headers, alignment, TLABs, and compressed references.
- Explain default values, reference assignment, and constructor failure.
- Reason about escape analysis, allocation elimination, and the "stack allocation" misconception.
- Distinguish shallow size, deep size, and retained size.

WHY IT WORKS | 7.2 Why this matters at SDE-2

Allocation rate, not just live heap, drives garbage-collection work. Object layout affects cache locality and capacity planning. Unsafe publication can expose constructor-related correctness bugs. Heap analysis depends on understanding that an object's retained size is graph-dependent, not simply its field total.

Interviewers also use object creation to see whether a candidate can separate layers. Java guarantees default initialization and constructor semantics. It does not guarantee a mark word, an 8-byte alignment boundary, compressed ordinary object pointers, or that every syntactic allocation becomes a heap allocation in optimized machine code.

WHY IT WORKS | 7.3 First-principles model

An object combines identity, class membership, and instance state. Evaluating a class instance creation expression conceptually:

- 1 Ensures the target class is loaded, linked, and initialized as required.
- 2 Obtains storage for a new instance.
- 3 Assigns default values to every instance field.
- 4 Evaluates constructor arguments left to right.
- 5 Invokes the selected constructor, beginning with a superclass-constructor chain.
- 6 Executes instance field initializers and initializer blocks in specified order for each class level.
- 7 Produces a reference to the initialized object if construction completes normally.

TEXT

```
new Account("A-1", 100)
|
+-- allocate identity and default state
|    id=null, balance=0
+-- Object constructor
+-- Account field initializers / initializer blocks
+-- Account constructor body
v
reference value to constructed Account
```

If construction throws, the expression produces no normal result. The allocated object can still escape if constructor code published `this` before failure, which is one reason early escape is dangerous.

7.4 Core terminology

- **Allocation:** Obtaining storage and runtime identity for an object or array.
- **Default initialization:** Setting reference fields to `null`, numeric fields to zero, `boolean` to `false`, and `char` to zero before Java initializer code.
- **Object header:** Implementation metadata preceding or accompanying instance fields.
- **Mark word:** HotSpot header word commonly encoding identity hash, lock/age/GC state in a layout that varies by mode and release.
- **Klass pointer:** HotSpot metadata reference identifying the runtime class.
- **Alignment:** Placement at address multiples chosen by the VM, potentially requiring padding.
- **Compressed oops:** HotSpot encoding that represents many object references in fewer bits under supported heap/addressing conditions.
- **Escape analysis:** Compiler analysis of whether a reference escapes a method, thread, or analysis scope.
- **Scalar replacement:** Replacing an aggregate allocation with individual scalar values when identity/escape constraints allow.
- **Shallow size:** Storage directly occupied by one object, excluding referenced objects.
- **Retained size:** Storage that becomes unreachable if a particular object is removed from the reachability graph.

7.5 Detailed mechanics

The bytecode instruction `new` names a class through the runtime constant pool and creates an uninitialized reference used under verifier-enforced rules. `dup` commonly preserves a reference while constructor arguments are prepared, and `invokespecial <init>` invokes the constructor. A constructor is not an ordinary dynamically dispatched method and has no return type, not even `void`.

Before constructor code observes fields, allocation supplies their default values. Source field initializers are compiled into constructors after the superclass constructor invocation and before the rest of that constructor's body. If one constructor delegates to another using `this(...)`, the delegated constructor performs initialization once; Java requires a `this(...)` or `super(...)` invocation to be the constructor's first statement under the applicable language rules.

A JVM must preserve object semantics, but it need not use one allocation algorithm. A general allocator could find a free block. A compacted region supports fast pointer-bump allocation: return the current top and advance it by aligned object size. Concurrent threads require coordination.

Specification boundary: The JLS defines evaluation, initialization, constructor, field, identity, and reference semantics; the JVMS defines object/array allocation instructions and verification constraints. Neither specification fixes an object header, byte offset, field order, alignment, TLAB policy, or compressed-reference encoding.

HotSpot note: HotSpot usually gives allocating threads TLABs in the shared heap. A fast-path allocation can advance a thread-local pointer, reducing contention. TLAB refill, large-object handling, slow-path allocation, and collector-specific regions vary by collector and release.

Physical object layout is implementation-specific. A common HotSpot mental model is:

TEXT

```
+-----+
| mark word          | implementation metadata
+-----+
| class/Klass metadata ref | possibly compressed
+-----+
| instance fields     | superclass fields included
+-----+
| internal/tail padding | alignment-dependent
+-----+
```

Field declaration order is not a portable layout contract. A JVM may arrange fields to reduce padding while honoring semantic constraints. Object size depends on VM bitness, compressed-reference modes, field mix, header scheme, and alignment. Arrays add length metadata and repeated element storage. A `boolean` field has language values `true` and `false`; its exact physical bit/byte size inside an object is not specified.

Compressed ordinary object pointers encode references using a base and scale or related addressing mode, allowing a 32-bit encoded value to address a larger aligned heap. Compressed class pointers can separately reduce the metadata reference. Whether these modes activate and their address range are HotSpot/version/configuration details.

Assignment copies a reference value; it does not copy the object:

JAVA

```
Account second = first;
```

Afterward, both variables can designate the same object. Reassigning `second` changes only that variable. Mutating through either reference changes shared object state.

Escape analysis allows optimization beyond the source model. If an object never escapes an analyzed compilation scope and identity is not observably required, a JIT may replace its fields with scalar values, eliminate synchronization, or eliminate the allocation. This is often loosely described as "putting the object on the stack," but scalar replacement can mean no object exists at all in generated code. Java does not guarantee stack allocation, and an allocation can be materialized during deoptimization.

Size terminology needs graph precision. Deep size recursively adds a chosen object's reachable graph but must define how shared objects are counted. Retained size uses dominators: if every path from a GC root to object B passes through A, A dominates B, so B contributes to A's retained set. Removing one ordinary reference to A does not necessarily collect A if another root path exists.

TRACE IT | 7.6 Worked Java example

JAVA

```
public final class Account {
    private static int nextSequence;

    private final int sequence = ++nextSequence;
    private final String id;
    private long cents;
    private boolean active;

    Account(String id, long openingCents) {
        if (id == null || openingCents < 0) {
            throw new IllegalArgumentException();
        }
        this.id = id;
        this.cents = openingCents;
        this.active = true;
    }

    long balance() {
        return cents;
    }

    public static void main(String[] args) {
        Account first = new Account("A-1", 10_000);
        Account alias = first;
        alias.cents += 500;
        alias = new Account("A-2", 2_000);
        System.out.println(first.balance() + ":" + alias.sequence);
    }
}
```

The output is **10500:2** in this single-threaded program. The first assignment aliases; the second **new** plus reassignment does not change what **first** references.

TRACE IT | 7.7 Execution or memory walkthrough

For the first construction:

- 1 `Account` initialization establishes `nextSequence` at 0.
- 2 Storage is obtained. Fields initially contain `sequence=0`, `id=null`, `cents=0`, and `active=false`.
- 3 `Object` construction completes.
- 4 The instance initializer expression `++nextSequence` writes 1 to class state and 1 to `sequence`.
- 5 Constructor validation executes. `id`, `cents`, and `active` receive their explicit values.
- 6 The resulting reference is stored in `first`.
- 7 `alias = first` copies the same reference. Mutation through `alias` changes the one object to 10,500 cents.
- 8 The second construction increments class state to 2 and produces another object. Reassigning `alias` changes the local reference graph only.

TEXT

```
main locals          heap graph
first -----> Account #1
                  sequence=1
                  id -----> String "A-1"
                  cents=10500
alias -----> Account #2
                  sequence=2
                  id -----> String "A-2"
                  cents=2000
```

The shallow size of `Account #1` includes its header, field encodings, and padding, not the `String` object's storage. Its retained size may include the `String` only if no other root path retains that string and `Account #1` dominates it.

7.8 Complexity and performance

Fast-path allocation is often amortized $O(1)$, while initialization time is at least proportional to storage that must be zeroed or otherwise made semantically zero. Collector work accounts for allocation rate and survival. Constructor body complexity is arbitrary.

Reducing object count can improve locality and lower GC metadata/tracing work, but manual pooling is often harmful. Pools retain objects, add synchronization and reset complexity, and defeat generational collection's strength with short-lived objects. Pool scarce external resources such as connections, not ordinary DTOs, unless measurement proves a need.

Field layout can create padding and cache effects, but changing domain design solely to save a few bytes is premature until a heap histogram shows multiplicity makes it material. Measure the deployed JVM configuration with a trustworthy layout tool; never hard-code an assumed shallow size into correctness logic.

WATCH OUT | 7.9 Edge cases and common mistakes

- Publishing `this` from a constructor via listeners, static collections, callbacks, or starting a thread.
- Calling an overridable method from a constructor; the subclass override can observe uninitialized subclass fields.
- Believing a `final` reference makes the referenced object immutable.
- Believing reference assignment clones an object.
- Treating header size or field order as a Java guarantee.
- Equating escape analysis with guaranteed stack allocation.
- Using shallow size to estimate the memory released by clearing a cache.
- Forgetting arrays are objects with header/length/alignment overhead.
- Assuming a failed constructor means the object could never have escaped.

7.10 Production engineering notes

Use allocation profiling to find hot allocation sites and heap histograms/dominator analysis to find retention. These are different questions. A high allocation rate with stable live set can cause GC CPU but not a leak. A slowly growing retained graph can leak with a modest allocation rate.

Prefer immutable objects with constructor validation and no `this` escape. Final-field visibility has useful JMM guarantees only when construction and publication rules are respected. For memory estimates, include backing arrays, object graphs, duplicate strings, alignment, and load factors in collections.

HotSpot note: Header formats have evolved, including locking changes and optional compact-object-header work in some releases/configurations. Always label diagrams with exact JDK, VM, architecture, alignment, and compression settings when numbers matter.

TRY IT | 7.11 Interview questions and model answers

What happens when Java executes `new`?

The runtime ensures the class is ready, obtains and default-initializes object storage, then constructor invocation runs the superclass chain, instance initializers, and constructor body. A conforming JVM preserves those effects, while TLAB allocation and header layout are implementation choices.

Are Java objects allocated on the stack or heap?

Semantically, object and array storage comes from the JVM heap. An optimizing JIT may prove an allocation non-escaping and scalar-replace it, so the physical object may not exist. Java provides no general stack-allocation guarantee.

What are mark word and Klass pointer?

They are HotSpot object-header concepts. The mark word can encode GC, locking, age, and hash-related state; the class metadata reference identifies the runtime type. Their exact format is not defined by Java specifications.

Shallow size versus retained size?

Shallow size is direct object storage. Retained size is the size of objects that would become unreachable if this object were removed, determined by root paths and dominance. It is a graph property, not a sum of field types.

TRY IT | 7.12 Exercises

- 1 Draw default and post-constructor state for `Account`.
- 2 Inspect constructor bytecode and identify `new`, `dup`, and `invokespecial` at a call site.
- 3 Create a constructor that leaks `this`; explain possible observations without relying on a reproducible failure.
- 4 Estimate the shallow size of a sample object under stated assumptions, then list why the result is non-portable.
- 5 Given a heap graph with shared children, compute shallow, deep-under-a-stated-rule, and retained sizes.

RECAP | 7.13 Chapter summary

Object creation combines allocation, mandatory default state, a superclass/initializer/constructor sequence, and reference production. Java specifies semantic effects but delegates headers, field placement, alignment, TLABs, and compressed references to implementations. Escape analysis may eliminate allocations without creating a portable stack-allocation feature. Memory analysis must distinguish allocation rate, shallow storage, and graph-based retention.

TRY IT | 7.14 Revision checklist

- ☐ I can trace `new` through default initialization and constructors.
- ☐ I can separate Java guarantees from HotSpot layout details.
- ☐ I can explain TLABs without calling their objects thread local.
- ☐ I understand reference aliasing and reassignment.
- ☐ I can explain escape analysis and scalar replacement precisely.
- ☐ I can distinguish shallow, deep, and retained size.

SDE-2 CORE CHAPTER

Chapter 8 - Java Stacks, Method Calls, and Recursion

8.1 Learning objectives

- Trace method invocation through argument evaluation, frame state, dispatch, return, and exceptions.
- Explain parameters and local variables as values, including copied reference values.
- Distinguish mutation through a reference from reassignment of a parameter.
- Model recursive calls and diagnose stack overflow.
- Explain why Java does not guarantee tail-call optimization.

WHY IT WORKS | 8.2 Why this matters at SDE-2

Call mechanics answer practical questions about aliasing, side effects, recursion depth, stack traces, synchronization boundaries, and debugging optimized code. Pass-by-reference folklore causes API bugs and poor interview answers. Recursive algorithms that look elegant can fail at production input depths because Java does not promise constant-space tail calls.

An SDE-2 also needs a layered view: the bytecode frame model is specified; the source language defines evaluation and pass-by-value; an optimizing runtime may inline calls, keep values in registers, or reconstruct frames for debugging. All three descriptions can be true at once.

WHY IT WORKS | 8.3 First-principles model

A method call transfers control while giving the callee new parameter variables initialized from argument values. Each active invocation has logically independent local state.

TEXT

```
caller expression
  1. evaluate target, if any
  2. evaluate arguments left to right
  3. select/invoke method according to call kind
      |
      v
callee invocation state
  parameters = copies of argument values
  locals + operand stack + return/linkage state
      |
      v
normal return value OR abrupt exception propagation
```

Java has only pass-by-value. For an object expression, the value is a reference. Copying that reference creates another route to the same object; it does not make the parameter an alias for the caller's variable itself.

8.4 Core terminology

- **Argument:** Expression/value supplied at a call site.
- **Parameter:** Callee variable initialized with the argument value.
- **Frame:** Per-invocation storage and control state in the JVM model.
- **Dynamic dispatch:** Runtime selection of an overriding instance method based on receiver class.
- **Static binding:** Selection not based on receiver override, as with static methods and constructors.
- **Recursion:** A method invocation chain in which a method eventually invokes itself.
- **Base case:** Condition that terminates recursive expansion.
- **Stack overflow:** Exhaustion caused by excessive frame depth or related stack limits.
- **Tail call:** A call whose result is immediately returned with no remaining caller computation.
- **Inlining:** Compiler replacement of a call with callee operations; unrelated to source-level recursion guarantees.

8.5 Detailed mechanics

Java evaluates the target expression and argument expressions left to right. Each evaluation can have side effects or throw before invocation begins. Overload selection occurs at compile time based on declared types and applicable conversions. Overriding dispatch occurs at runtime for eligible instance methods based on the actual receiver type.

Representative JVM invocation instructions are:

- `invokestatic` for static methods;
- `invokespecial` for constructors, private methods in relevant class-file forms, and special superclass/interface calls;
- `invokevirtual` for ordinary class instance dispatch;
- `invokeinterface` for interface method dispatch;
- `invokedynamic` for dynamically linked call sites such as common lambda and modern string-concatenation translations.

These are bytecode categories, not a performance ranking. A JIT can inline interface calls when profiles show a stable receiver and can deoptimize if assumptions change.

A JVM frame contains local-variable slots and an operand stack. Parameters occupy initial local slots. For an instance method, local slot 0 initially contains `this`. Bytecodes load operands, perform calculations, and store results. Invocation consumes receiver/argument operands and establishes callee state; return places a result in the caller's computation state. Exceptions skip ordinary return and search handlers while unwinding frames.

Specification boundary: Java specifies argument evaluation order, pass-by-value, method selection/dispatch, and exception behavior. The JVM specification specifies abstract frames and invocation instructions. Neither guarantees that optimized native execution materializes one physical frame per source call.

For primitives, the copied parameter contains the primitive value. Reassigning it cannot change the caller variable. For references, the copied parameter and caller variable can initially designate the same object. Mutating that object is visible through other aliases. Reassigning the parameter makes only the parameter hold another reference.

Arrays follow the same rule. A method can mutate elements through its copied array reference; it cannot replace the caller's variable by assigning its parameter to a new array. Wrapper types are objects but immutable, so apparent mutation such as `value++` unboxes and reboxes, then reassigns only the local parameter.

Recursion creates a new logical invocation for each level. Each has independent parameters and locals, while references may point to shared heap objects. A base case stops growth; returns then unwind in reverse order. Direct recursion calls the same method; mutual recursion cycles through methods.

Tail recursion does not change the language rule. The JVM specifications do not require elimination of tail frames, and mainstream Java code must budget $O(\text{depth})$ stack for recursive calls. A JIT may inline a bounded recursion prefix or optimize details, but an algorithm cannot rely on tail-call optimization for correctness.

TRACE IT | 8.6 Worked Java example

JAVA

```
public final class CallSemantics {
    static final class Box {
        int value;
        Box(int value) { this.value = value; }
    }

    static void change(int number, Box box) {
        number = 99;
        box.value = 20;
        box = new Box(30);
        box.value++;
    }

    static long factorial(int n) {
        if (n < 0) throw new IllegalArgumentException("negative");
        return n <= 1 ? 1 : n * factorial(n - 1);
    }

    public static void main(String[] args) {
        int number = 5;
        Box box = new Box(10);
        change(number, box);
        System.out.println(number + ":" + box.value);
        System.out.println(factorial(4));
    }
}
```

The output is:

TEXT

```
5:20
24
```

`number` remains 5 because the callee reassigns its copy. The original `Box` becomes 20 through shared-object mutation. Reassigning callee `box` to a second object does not redirect caller `box`.

TRACE IT | 8.7 Execution or memory walkthrough

At entry to `change`:

TEXT

main frame	change frame	heap
number = 5	number = 5	Box #1 {value=10}
box ----- reference R1 ----->	box = copy of R1 ----->	^

- 1 `number = 99` changes only the `change` frame slot.
- 2 `box.value = 20` follows R1 and mutates Box #1.
- 3 `new Box(30)` creates Box #2; assigning its reference changes only `change.box`.
- 4 `box.value++` changes Box #2 to 31.
- 5 On return, the `change` frame disappears. If no other reference escaped, Box #2 is collectible. `main.box` still points to Box #1 with value 20.

For `factorial(4)`, logical frames expand:

TEXT

```
factorial(4): waiting for 4 * result
factorial(3): waiting for 3 * result
factorial(2): waiting for 2 * result
factorial(1): returns 1
```

Unwinding computes 2, then 6, then 24. At depth `d`, there are $O(d)$ active invocations. `long` overflow occurs after relatively small `n`; correct recursion structure does not guarantee numerically correct unbounded factorials.

When an exception is thrown for negative input, the runtime searches the current frame's exception table. If no compatible handler covers the instruction, the frame is removed and search continues in its caller. `finally` behavior compiled into control flow must execute according to language rules.

8.8 Complexity and performance

`change` is $O(1)$ time and allocates one additional $O(1)$ -sized object. `factorial(n)` is $O(n)$ time and $O(n)$ call depth. An iterative factorial is $O(n)$ time and $O(1)$ auxiliary call-stack space.

Method calls are not automatically expensive. JIT inlining can eliminate dispatch and expose further optimizations. Excessive large methods can inhibit inlining, while tiny abstraction methods often disappear in hot compiled code. Measure rather than manually flattening well-designed APIs.

Recursion is appropriate when problem depth is naturally bounded, such as a balanced tree with controlled height. For adversarial linked lists, arbitrary graph DFS, parsers, or user-supplied nesting, an explicit heap-backed stack gives controllable capacity and avoids process-threatening `StackOverflowError`.

HotSpot note: HotSpot stack size, inlining thresholds, frame layout, guard pages, and stack-bang mechanisms are implementation and platform dependent. The `-Xss` option is common but exact supported syntax and minimums vary.

WATCH OUT | 8.9 Edge cases and common mistakes

- Saying Java passes objects by reference. It passes a reference value by value.
- Confusing overload selection with override dispatch.
- Assuming a `final` parameter creates caller immutability; it only prevents parameter reassignment in that method.
- Forgetting argument expressions run left to right and can fail before the method begins.
- Catching `StackOverflowError` and attempting normal recovery on the same deeply recursive design.
- Assuming tail recursion is constant-space Java.
- Increasing stack size without estimating total memory across thousands of threads.
- Reading an optimized stack trace/profile as a perfect history of physical frames.
- Using recursion on a cyclic graph without a visited set.

8.10 Production engineering notes

API documentation should state mutation behavior. Prefer returning a value or an immutable result over hidden mutation unless mutation is intentional and clear. Defensive copies protect boundaries when callers must not retain mutable aliases.

Diagnose stack overflow from the repeated frame pattern. A repeating short cycle indicates mutual recursion; a single repeated frame suggests missing/late base case or unexpectedly deep input. Capture the triggering input shape. Do not blindly increase `-Xss`: larger per-platform-thread stacks reduce the number of threads supportable under a memory limit.

Virtual threads still preserve Java call semantics, but their stack implementation and scheduling differ from traditional one-OS-thread-per-platform-thread expectations. Never use thread count alone as a substitute for call-depth control.

TRY IT | 8.11 Interview questions and model answers

Is Java pass-by-reference?

No. Every parameter receives a copied value. For objects, that value is a reference, so caller and callee can reach the same object and observe mutation. Reassigning the callee parameter cannot reassign the caller variable.

What is stored in a method frame?

In the JVM model, local-variable slots, an operand stack, and control/linkage state for return and exceptions. Optimized execution can inline calls or keep values elsewhere, so physical state need not mirror source frames.

Why can recursion overflow when the algorithm has a base case?

The base case may be reached only after more active calls than the thread stack supports. Input depth, frame size, and stack configuration matter. A base case ensures logical termination, not sufficient memory.

Does Java optimize tail recursion?

Java and the JVMs do not guarantee general tail-call elimination. Production Java must assume tail recursion consumes $O(\text{depth})$ stack and convert unbounded cases to iteration or an explicit stack.

TRY IT | 8.12 Exercises

- 1 Modify `change` to return the new `Box`; show how the caller can deliberately replace its reference.
- 2 Draw frames for `factorial(5)` during expansion and unwinding.
- 3 Convert factorial to an iterative version and compare overflow behavior.
- 4 Implement depth-first traversal both recursively and with `ArrayDeque`.
- 5 Create overload and override examples and identify compile-time versus runtime decisions.
- 6 Explain how an exception propagates through three calls with one matching handler.

RECAP | 8.13 Chapter summary

Method calls copy argument values into independent invocation state. A copied reference can reach a shared object, so mutation propagates through aliases while parameter reassignment does not. Frames model locals, operand computation, return, and exceptions. Recursion creates $O(\text{depth})$ active state, and Java offers no portable tail-call elimination. Optimized physical execution may inline or reconstruct frames without changing these semantics.

TRY IT | 8.14 Revision checklist

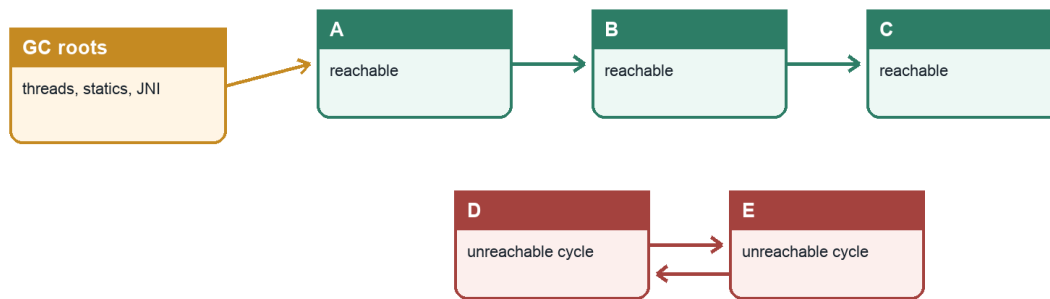
- ☐ I can explain Java pass-by-value with primitives and references.
- ☐ I can separate mutation from parameter reassignment.
- ☐ I know the main JVM invocation instruction categories.
- ☐ I can draw frame expansion and unwinding for recursion.
- ☐ I can choose recursion versus an explicit stack based on depth risk.
- ☐ I do not rely on tail-call optimization or physical frame layout.

SDE-2 CORE CHAPTER

Chapter 9 - Garbage Collection

Reachability and generational collection

Collectors reclaim unreachable objects; algorithms and heap organization vary



Generational heuristic (not universal)

Most objects die young, so many collectors separate young and old work; collector/version behavior differs.

G1, ZGC, Shenandoah, Parallel, and Serial differ in generations, barriers, concurrency, compaction, pauses, and throughput.

The illustrated roots are representative rather than an exhaustive implementation list.

Java Foundations to Advanced Engineering

Figure 9.1 - Reachability and the generational collection heuristic

9.1 Learning objectives

- Explain reachability, GC roots, tracing, reclamation, and relocation.
- Use strong, soft, weak, and phantom references with accurate expectations.
- Compare marking, sweeping, compaction, copying, and generational collection.
- Describe Serial, Parallel, G1, ZGC, and Shenandoah at an engineering level.
- Read basic GC evidence, explain leaks in managed languages, and tune from goals and measurements.

WHY IT WORKS | 9.2 Why this matters at SDE-2

Garbage collection removes manual deallocation but not capacity engineering. An SDE-2 must distinguish allocation pressure from retention, throughput from tail latency, a young pause from a full-heap event, and Java heap exhaustion from native or metadata exhaustion. The same knowledge prevents harmful fixes such as forcing `System.gc()`, caching with soft references, or increasing heap until container OOM kills replace useful Java errors.

GC is an interview favorite because it connects graphs, runtime architecture, performance, references, and operations. Strong answers describe goals and trade-offs without presenting collector folklore as specification.

WHY IT WORKS | 9.3 First-principles model

Memory can be reclaimed when no future legal execution can observe an object. Tracing collectors approximate this condition through reachability from roots known to the runtime.

TEXT

```
GC roots
+-- active thread frames -----> Request -> byte[]
+-- static/class state -----> Cache -> Entry
+-- JNI/runtime handles -----> NativePeer

unreachable cycle:
  Node A -> Node B -> Node A      no path from any root, collectible
```

Reference counting alone cannot collect the unreachable cycle. A tracing collector starts at roots, follows references, marks or copies what is reachable, and treats the rest as reclaimable, subject to special reference processing and finalization-related rules.

Specification boundary: Java specifies reachability categories, reference APIs, finalization semantics where retained for compatibility, and observable requirements such as not reclaiming strongly reachable objects. It does not require generations, Eden, survivor spaces, specific pause names, or a particular collector algorithm.

9.4 Core terminology

- **GC root:** Runtime starting point for reachability analysis, such as selected thread/frame, class, JNI, and VM references.
- **Live set:** Objects considered live under the collector/reference-processing rules at a point in time.
- **Strong reference:** Ordinary reference that keeps an object strongly reachable.
- **Soft reference:** Reference intended for memory-sensitive caches, cleared at GC's discretion under specified policy constraints.
- **Weak reference:** Does not keep a weakly reachable object alive; commonly used with a [ReferenceQueue](#).
- **Phantom reference:** Post-mortem notification/cleanup coordination mechanism, always observed through `get()` as `null`.
- **Mark-sweep:** Mark reachable objects, then reclaim unmarked storage.
- **Compaction:** Move live objects to reduce fragmentation and update references.
- **Copying:** Move live objects from one region to another, reclaiming the source wholesale.
- **Generational hypothesis:** Most newly allocated objects die young, while survivors are more likely to remain.
- **Pause:** Interval when selected application threads cannot execute.

9.5 Detailed mechanics

Root discovery uses runtime knowledge of where references reside. In compiled code and frames, precise maps can identify reference locations. Static state associated with live classes, active thread locals, synchronization/runtime structures, and JNI handles can participate. A local that appears in source may cease to be a root after its last use under allowed optimization.

Strong references are the default. Weak references support canonical maps and metadata associations when combined with careful cleanup. [WeakHashMap](#) weakens keys, not values; if a value strongly references its key, the entry can remain reachable through the map value path. Soft references are not deterministic cache eviction and can cause unstable latency. Bounded caches with explicit policy are usually better. Phantom references plus a [ReferenceQueue](#) and [Cleaner](#)-style mechanisms can coordinate cleanup of non-memory resources, but explicit [AutoCloseable](#) ownership is primary.

Mark-sweep avoids moving live objects but can fragment free memory. Mark-compact pays relocation cost to create contiguous free space. Copying cost is proportional mainly to survivors and gives cheap bulk reclamation, which suits young regions with high mortality. Concurrent collectors perform substantial tracing or relocation while application threads run, using barriers and additional metadata to preserve correctness.

Generational collectors classify storage by object age or regions serving young/old roles. A common young layout has Eden for new allocation and survivor spaces for copied survivors. Objects surviving enough collections, or meeting other conditions, move to an old region. References from old to young require remembered sets/card tables so a young collection need not scan the entire old population.

"Minor," "major," and "full" GC are widely used but not rigorously portable terms. Minor commonly means young-only. Major sometimes means old-generation work, but log/tool usage varies. Full commonly means a broad stop-the-world collection of much or all heap, often with compaction, but exact causes and algorithms are collector-specific. Use actual event names and scope from the deployed JVM.

Collector overview for modern HotSpot deployments:

Collector	Primary orientation	Important trade-off
Serial GC	Simple single-worker collection, small heaps	Longer pauses as heap/work grows
Parallel GC	Throughput using parallel stop-the-world workers	Pause-time predictability is secondary
G1 GC	Region-based generational collector with pause targets	Remembered-set/concurrent-cycle overhead; targets are goals
ZGC	Very low pauses through concurrent work and barriers	Additional concurrent CPU/metadata; release capabilities vary
Shenandoah	Low pauses through concurrent evacuation/compaction	Availability and generational mode depend on distribution/release

G1 divides the heap into regions rather than fixed contiguous young/old blocks. Young collections evacuate live objects; concurrent marking identifies old live data; mixed collections can include selected old regions. Humongous objects receive special region handling. A pause target guides heuristics but is not a deadline guarantee.

ZGC uses colored/metadata-bearing reference techniques, load barriers, and concurrent phases to keep pauses largely independent of heap size. Modern releases have evolved, including generational capabilities in relevant versions. Shenandoah also performs concurrent evacuation with barriers. Do not transfer flags or mental models between releases without checking documentation and logs.

TRACE IT | 9.6 Worked Java example

JAVA

```
import java.lang.ref.ReferenceQueue;
import java.lang.ref.WeakReference;

public final class ReferenceDemo {
    static final class TrackedReference extends WeakReference<Object> {
        final String label;

        TrackedReference(Object value, ReferenceQueue<Object> queue, String label) {
            super(value, queue);
            this.label = label;
        }
    }

    public static void main(String[] args) throws Exception {
        ReferenceQueue<Object> queue = new ReferenceQueue<>();
        Object strong = new byte[1_000_000];
        TrackedReference weak = new TrackedReference(strong, queue, "payload");

        System.out.println(weak.get() != null); // true here
        strong = null;

        for (int i = 0; i < 100 && queue.poll() == null; i++) {
            System.gc();
            Thread.sleep(10);
        }
        System.out.println(weak.get());
    }
}
```

The first line is predictably `true` because `strong` still holds the object. The final line is not a portable timing test: `System.gc()` is only a request, collection need not happen immediately, and weak-reference clearing/enqueueing timing is nondeterministic.

TRACE IT | 9.7 Execution or memory walkthrough

Initially:

TEXT

```
main frame --strong-----> byte[]
           --weak--> TrackedReference -weak-> byte[]
           --queue--> ReferenceQueue
```

After `strong = null`, the byte array has only the weak edge. At an appropriate GC cycle:

- 1 Roots are scanned and the `TrackedReference` plus queue are strongly reachable.
- 2 The byte array is not strongly or softly reachable through the shown graph.
- 3 Weak-reference processing clears the referent according to the reference rules.
- 4 The reference is registered/enqueued for consumer observation, with timing depending on processing.
- 5 The byte-array storage can eventually be reclaimed or its region reused.

The `TrackedReference` itself is not automatically removed; the program must drain the queue and clean associated metadata. A reference object and its referent are distinct objects.

For ordinary young allocation, a common HotSpot generational dry run is:

TEXT

```
allocate in Eden -> Eden fills -> STW young collection
  dead objects: not copied
  survivors: copied to survivor/old regions, age updated
  references: adjusted
resume application
```

This is a collector model, not a universal Java execution requirement.

9.8 Complexity and performance

Tracing cost relates to roots, live graph, remembered-set work, and changed-reference tracking, not simply maximum heap. Copying cost is driven by survivors. Sweep cost can involve address-space/region metadata. Concurrent work consumes CPU and memory bandwidth while the application runs.

Key workload measurements are allocation rate, promotion rate, live-set size after a sufficiently complete collection, pause distribution, concurrent-cycle duration, GC CPU, and headroom. A larger heap can reduce collection frequency but increase footprint and sometimes cycle/pause work. A smaller heap may improve locality but collect too often or fail under bursts.

Latency and throughput are different goals. Parallel GC can maximize application throughput with larger pauses. Low-pause collectors shift work concurrent with the application and use barriers, often buying tail-latency control with CPU and complexity. Collector choice must follow an SLO and measured workload.

WATCH OUT | 9.9 Edge cases and common mistakes

- Saying reference cycles leak automatically. Tracing collectors collect unreachable cycles.
- Treating any high allocation rate as a leak.
- Assuming GC means resources such as sockets are closed promptly.
- Using finalization for correctness; finalization is deprecated for removal and nondeterministic.
- Assuming `System.gc()` forces an immediate full collection.
- Using soft references as the only cache bound.
- Forgetting thread locals, listeners, static maps, queues, and class loaders are common retention paths.
- Calling every old-generation event a full GC.
- Choosing a collector from generic benchmark claims without application SLO evidence.
- Ignoring non-heap OOME causes and container-level kills.

Memory leaks exist whenever reachable objects are no longer useful. Examples include an unbounded cache, completed requests retained in a queue, forgotten listener registrations, stale thread-local values on pool threads, and class loaders retained after redeployment.

`OutOfMemoryError` is a family of failures. Messages can indicate Java heap, GC overhead, array-size limits, Metaspace, direct buffer memory, or native thread creation. The JVM may be severely resource constrained; error-handling code requiring more allocation may fail too.

9.10 Production engineering notes

Start with service goals and evidence, not flags:

- 1 Confirm the exact JDK, collector, heap/container sizing, and recent changes.
- 2 Capture GC logs/JFR and OS/container memory/CPU data.
- 3 Determine whether the problem is allocation rate, retained live set, fragmentation, insufficient headroom, concurrent-cycle lateness, or non-heap memory.
- 4 Fix retention or allocation behavior when code is the cause.
- 5 Adjust heap/collector/limited options one variable at a time under representative load.
- 6 Validate throughput, p95/p99/p999 latency, CPU, memory, and failure recovery.

Unified GC logging in HotSpot can expose event type, phases, causes, region occupancy, and pause time. Use a tested rotation/retention plan because logs are incident evidence. A heap dump may contain secrets and can require substantial disk and pause/CPU; secure it and rehearse acquisition.

HotSpot note: Collector defaults, supported collectors, flags, log tags, reference-processing policy, and `System.gc()` handling vary by JDK distribution and release. Confirm the running process with supported commands instead of trusting launch documentation alone.

TRY IT | 9.11 Interview questions and model answers

How does GC decide an object is garbage?

A tracing collector starts from JVM-known roots and follows references under Java's reachability rules. Objects with no applicable root path are candidates for reclamation, with special processing for soft, weak, phantom, and legacy finalization states.

Can Java have memory leaks?

Yes. GC removes unreachable objects, not reachable objects the application no longer needs. Unbounded caches, thread locals, listeners, queues, and class loaders can retain useless graphs.

Compare G1 and ZGC.

G1 is a region-based generational collector balancing throughput and pause goals with evacuation and concurrent marking. ZGC moves much more work, including relocation, concurrently to target very low pauses

using barriers and metadata techniques. Exact generational capabilities and trade-offs depend on JDK version; I would choose using SLOs and load tests.

What is stop-the-world?

It means relevant application threads are paused for a VM operation. Young collections and short phases of concurrent collectors can be STW. It does not necessarily mean full GC, and not every STW operation is GC.

How would you investigate frequent GC?

First classify events and inspect allocation rate, live set, promotion, pause and concurrent-cycle timing, heap occupancy, and CPU. Then determine whether traffic, object churn, retention, heap sizing, or collector inability to keep up is causal before tuning.

TRY IT | 9.12 Exercises

- 1 Draw root paths for a static cache, a thread-local leak, and an unreachable cycle.
- 2 Implement weak-key metadata cleanup using `ReferenceQueue`; explain why polling is required.
- 3 Given GC-log excerpts, classify young pauses, concurrent phases, and broad full-heap events using their actual labels.
- 4 Design a load test comparing throughput and p99 latency under two collectors.
- 5 Explain why a 2 GiB heap in a 2 GiB container is unsafe.
- 6 Create a bounded local heap experiment, capture a histogram, and distinguish allocation from retention.

RECAP | 9.13 Chapter summary

Garbage collection traces from roots and reclaims objects that can no longer participate under Java's reference rules. Marking, sweeping, copying, compaction, generations, regions, and concurrency are implementation strategies with different CPU, memory, throughput, and pause trade-offs. Managed memory still leaks when useless objects remain reachable. Successful operations begin with workload goals, GC evidence, whole-process memory, and code-level causes before flags.

TRY IT | 9.14 Revision checklist

- ☐ I can explain tracing reachability and why cycles can be collected.
- ☐ I know strong, soft, weak, and phantom reference roles and limitations.
- ☐ I can compare sweep, compact, copy, and concurrent work.
- ☐ I can explain generations, remembered sets, and promotion.
- ☐ I can compare Serial, Parallel, G1, ZGC, and Shenandoah without overclaiming.
- ☐ I can distinguish allocation pressure, retention leaks, and non-heap failures.
- ☐ I can propose an evidence-driven GC investigation and tuning loop.

SDE-2 CORE CHAPTER

Chapter 10 - Execution Engine, JIT Compilation, and Safepoints

10.1 Learning objectives

- Explain how bytecode execution can combine interpretation and multiple compilation tiers.
- Describe profiling, inlining, speculative optimization, on-stack replacement, and deoptimization.
- Distinguish the code cache and compilation work from Java heap behavior.
- Explain safepoints, safepoint polling, time-to-safepoint, and stop-the-world coordination.
- Investigate warm-up, compilation, and safepoint issues using evidence rather than folklore.

WHY IT WORKS | 10.2 Why this matters at SDE-2

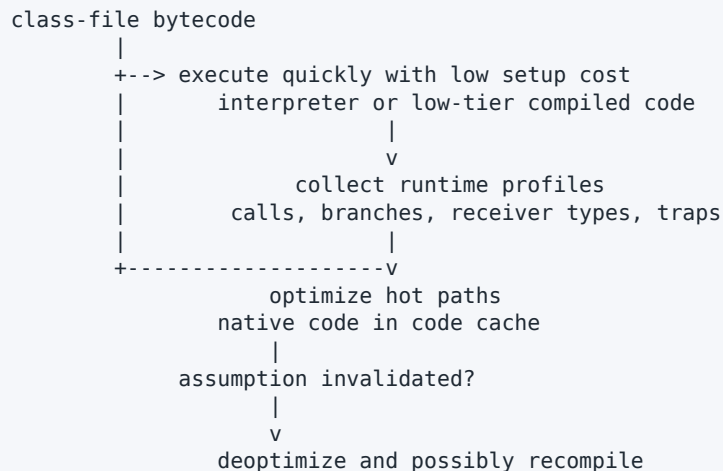
Java service performance changes over time. Cold startup can be dominated by loading and interpretation; a warm service can run highly optimized native code; a traffic-shape change can invalidate profiles; compiler CPU can compete with requests; and a global VM operation can pause threads even when GC work is small. These effects matter for autoscaling, canary comparison, benchmark design, and tail-latency incidents.

The SDE-2 standard is not memorizing C1 and C2. It is explaining why adaptive optimization exists, which assumptions it can make, how correctness survives invalidation, and how safepoint coordination interacts with all execution forms.

WHY IT WORKS | 10.3 First-principles model

Bytecode is a portable semantic program, not the only runtime representation. An execution engine may trade immediate startup for later optimization:

TEXT



The JVM can make an optimization that is valid under a guarded assumption, retain metadata to recover a legal execution state, and abandon the optimized form if the assumption becomes false. This feedback loop is the foundation of adaptive JIT compilation.

10.4 Core terminology

- **Interpreter:** Executes bytecode with relatively low compilation delay and rich profiling opportunities.
- **Hot method/loop:** Code selected for compilation based on implementation counters and policy.
- **Tiered compilation:** Movement through execution/compilation levels balancing profiling speed and optimization quality.
- **Inlining:** Substitution of callee operations at a call site, often enabling further optimization.
- **Profile-guided optimization:** Use of observed receiver types, branches, and call frequencies.
- **On-stack replacement (OSR):** Replacement of an executing loop/method state with compiled code before invocation returns.
- **Speculative optimization:** Optimization based on a guarded, currently valid runtime assumption.
- **Uncommon trap:** Transfer out of optimized code when a rare path or failed assumption occurs.
- **Deoptimization:** Reconstruction of less optimized/interpreted state from compiled execution metadata.
- **Code cache:** Native memory used for generated machine code and associated runtime structures.
- **Safepoint poll:** A check through which a thread cooperates with a safepoint request.
- **Time to safepoint (TTSP):** Delay between requesting a global safe state and all relevant threads reaching it.

10.5 Detailed mechanics

Interpretation gives fast availability: the runtime can start executing a method without spending time on aggressive optimization. It also observes behavior. A JIT compiler spends CPU and memory to make repeated execution cheaper. Tiering attempts to optimize total time by using quick compilation for moderately hot code and expensive optimization for code likely to repay it.

Inlining is the gateway optimization. Once a callee is merged into a caller, the compiler can propagate constants, eliminate dead branches, remove redundant null/bounds checks, specialize virtual calls, combine arithmetic, and analyze allocations across the old call boundary. Inlining decisions consider hotness, call-site profile, method size, recursion depth, and compilation budget.

Dynamic dispatch does not prevent optimization. If a call site has observed one receiver class, compiled code can guard that class and call or inline the known target. Class-hierarchy analysis can prove that only one target currently exists. A later class load can invalidate such a dependency. The runtime makes affected code non-entrant, transfers active execution safely when needed, and can recompile with a broader dispatch path.

Escape analysis can prove that a newly created object is confined to a compilation scope. Scalar replacement then represents its fields as independent values and removes allocation. Lock elimination can remove synchronization on a proven non-escaping object. These transformations require inlining and analysis visibility; small source changes can alter the outcome.

Loops deserve special treatment because a single method invocation can run for a long time. Waiting for it to return before using compiled code loses the benefit. OSR compiles and enters a loop at a supported bytecode position using the live state of the current invocation.

Optimized code must support exceptions, garbage collection, stack walking, debugging, and deoptimization. It therefore includes metadata mapping machine positions and registers to Java-level values, reference locations, and possible reconstructed frames. An inlined method can appear as a logical frame even though no independent physical call occurred.

The code cache is separate from the Java heap. Generated code has a life cycle: compilation installs it, dependencies can invalidate it, sweeper/runtime policy can reclaim dead forms, and finite capacity can fill. If effective compilation is disabled or constrained by code-cache pressure, throughput may degrade without a heap OOME.

Specification boundary: The JLS and JVMs require correct language/JVM behavior. They do not require an interpreter, JIT, tiered compilation, C1/C2, OSR, a code cache, escape analysis, or deoptimization. A conforming implementation could use ahead-of-time compilation or another execution strategy.

Safepoints solve a coordination problem. Some runtime operations need every relevant thread at a state where object references and execution can be described reliably. Compilers place polling opportunities at selected transitions, returns, or loop paths, and runtime stubs/transitions cooperate. Once a safepoint is requested, threads eventually observe it and enter a safe state; the VM operation executes; threads resume.

TTSP and safepoint operation time are separate:

TEXT

```
request issued ----- all threads stopped ----- operation ends
      time to safepoint                safepoint operation time
```

A long TTSP can arise from CPU starvation, a thread in a difficult native/critical transition, or generated code with delayed polling under a particular implementation defect or path. A short TTSP followed by a long pause points to the operation itself. Some JVM operations can instead use thread-local handshakes, coordinating only selected threads, but this is implementation evolution rather than a Java guarantee.

HotSpot note: Mainstream HotSpot uses tiered compilation with an interpreter, C1, and C2 in common configurations. It uses invocation/back-edge counters, OSR, speculative dependencies, uncommon traps, and safepoint polling. Exact levels, thresholds, compiler availability, and polling mechanisms vary by build and release.

TRACE IT | 10.6 Worked Java example

JAVA

```
public final class AdaptiveExecution {
    interface PriceRule {
        long apply(long cents);
    }

    static final class Discount implements PriceRule {
        public long apply(long cents) {
            return cents - cents / 10;
        }
    }

    static long total(PriceRule rule, long[] prices) {
        long sum = 0;
        for (long price : prices) {
            sum += rule.apply(price);
        }
        return sum;
    }

    public static void main(String[] args) {
        long[] prices = {100, 200, 300, 400};
        PriceRule rule = new Discount();
        long checksum = 0;
        for (int i = 0; i < 1_000_000; i++) {
            checksum ^= total(rule, prices);
        }
        System.out.println(checksum);
    }
}
```

The repeated call site has one receiver type. An adaptive compiler may inline `Discount.apply` through the interface call, inline `total`, and optimize loop mechanics. "May" is essential: compilation policy, command-line modes, profiling, and program context determine the result.

TRACE IT | 10.7 Execution or memory walkthrough

A plausible HotSpot timeline is:

- 1 Classes load and methods begin in interpreted or early-tier execution.
- 2 Invocation and loop back-edge counters grow. Profiles record that `rule` is always `Discount` at the call site.
- 3 `total` or `main` is queued for compilation on compiler threads. Application execution continues using an available form.
- 4 Compiled code is installed in the code cache. A future invocation enters it, or OSR transfers a currently executing hot loop.
- 5 The interface call is guarded/specialized. After inlining, `cents / 10` and summation appear in a larger optimization graph.
- 6 A safepoint request can stop the application at a poll. Metadata tells GC where `prices`, `rule`, and any other live references reside, even if not in source-like frame slots.
- 7 If another implementation of `PriceRule` later appears at that site or a hierarchy dependency changes, a guard can choose a fallback or an uncommon trap can deoptimize.
- 8 Deoptimization reconstructs logical Java frames and values, materializing objects if scalar replacement had removed them, then continues in a valid less-specialized form.

The JIT cannot delete the entire computation if the printed value depends on it, though algebraic simplification may still be possible. A benchmark whose result is unused is more vulnerable to dead-code elimination.

10.8 Complexity and performance

For `p` prices and `n` repetitions, source algorithmic time is $O(np)$, with $O(1)$ auxiliary algorithmic space beyond the input. JIT compilation does not change Big-O here, but it can drastically change constants. Inlining can reduce dispatch overhead and expose vectorization or range-check elimination.

Optimization has a budget:

- interpretation/low tier reduces startup delay but executes slower;
- profiling consumes counters and some runtime work;
- compiler threads consume CPU and native memory;
- optimized code consumes code cache;
- speculation can pay off, but repeated deoptimization/recompilation wastes work;
- aggressive inlining increases code size and instruction-cache pressure.

Warm-up is workload-specific. A fixed number of iterations is not proof of steady state. Observe compilation and profile stability, use forked processes, consume results, and rely on JMH for microbenchmarks. For services, load testing should include startup, ramp, representative polymorphism, and traffic shifts.

WATCH OUT | 10.9 Edge cases and common mistakes

- Calling JIT compilation a Java language guarantee.
- Assuming hotness means elapsed time only; policy often uses invocation and loop counters plus tier state.
- Believing interface calls cannot inline.
- Equating source calls with physical native frames after inlining.
- Treating deoptimization as an error; it is a normal correctness mechanism.
- Benchmarking a constant or unused result that the compiler removes.
- Mixing cold and warm samples without labeling them.
- Assuming the code cache is part of heap or fixed by `-Xmx`.
- Calling every safepoint pause GC or every long pause TTSP.
- Disabling compilation or changing thresholds as a first response without evidence.

10.10 Production engineering notes

Java Flight Recorder is a strong first choice for low-overhead visibility into compilation, code cache, execution sampling, allocation, locks, and safepoint/GC activity. Pair it with unified logging for targeted compiler/safepoint questions and OS CPU scheduling evidence. Diagnostic flag names and overhead must be tested on the deployed version.

For startup-sensitive services, consider class loading, framework initialization, profile maturity, and readiness policy together. Declaring readiness before representative code is warm can shift warm-up latency to users. Artificial warm-up can help only if it exercises realistic paths without unsafe external side effects.

For latency spikes, build a timeline:

- 1 Did application threads run, block, or stop?
- 2 Was a safepoint requested, and what operation ran?
- 3 How much was TTSP versus operation duration?
- 4 Was CPU saturated by GC, compilers, application, or neighbors?
- 5 Did deoptimization or a compilation storm coincide with a traffic/code-shape change?

Code-cache exhaustion, excessive dynamic class generation, megamorphic call sites, and unstable speculation are advanced hypotheses, not default explanations. Validate them with events and profiles.

TRY IT | 10.11 Interview questions and model answers

Why use both an interpreter and JIT?

Interpretation or low-tier execution starts quickly and gathers profiles. JIT compilation spends more work on hot paths, using runtime information unavailable to a static compiler. Tiering balances startup, profiling quality, compilation cost, and steady-state speed.

What is speculative optimization?

The runtime optimizes for an observed condition, such as one receiver type, while guarding the assumption. If it fails, execution takes a generic path or deoptimizes to reconstructed legal state. Correctness does not depend on the speculation remaining true.

What is OSR?

On-stack replacement transfers a currently running invocation, commonly at a hot loop back edge, into compiled code. It avoids waiting for a long-running method to return before benefiting from compilation.

What is a safepoint?

It is an implementation safe state used for VM operations needing coordinated managed state. Threads reach it through polls or transitions. Time to reach the safepoint and time spent performing the operation must be measured separately.

Can JIT optimizations violate Java semantics?

No. They can exploit all outcomes the JLS/JMM permits, including surprising outcomes in data-racy code, but must preserve specified behavior. Guards, metadata, and deoptimization maintain correctness when assumptions change.

TRY IT | 10.12 Exercises

- 1 Identify potential inlining and constant/range optimizations in the worked example.
- 2 Add a second `PriceRule` selected unpredictably and reason about monomorphic versus polymorphic profiles.
- 3 Explain how OSR helps one very long loop invocation.
- 4 Design a JMH benchmark that consumes results and separates warm-up from measurement.
- 5 Draw a pause timeline that distinguishes TTSP, VM operation, and application recovery.
- 6 Explain how an inlined, scalar-replaced object could reappear during deoptimization.

RECAP | 10.13 Chapter summary

Adaptive execution moves code among low-setup and optimized forms using runtime profiles. Inlining unlocks specialization, escape analysis, and check elimination; speculation remains safe through guards and deoptimization. OSR accelerates long-running loops, while the code cache stores generated native code outside the heap. Safepoints coordinate runtime operations and must be analyzed as time-to-safepoint plus operation time. All of these are implementation strategies under Java's semantic contract.

TRY IT | 10.14 Revision checklist

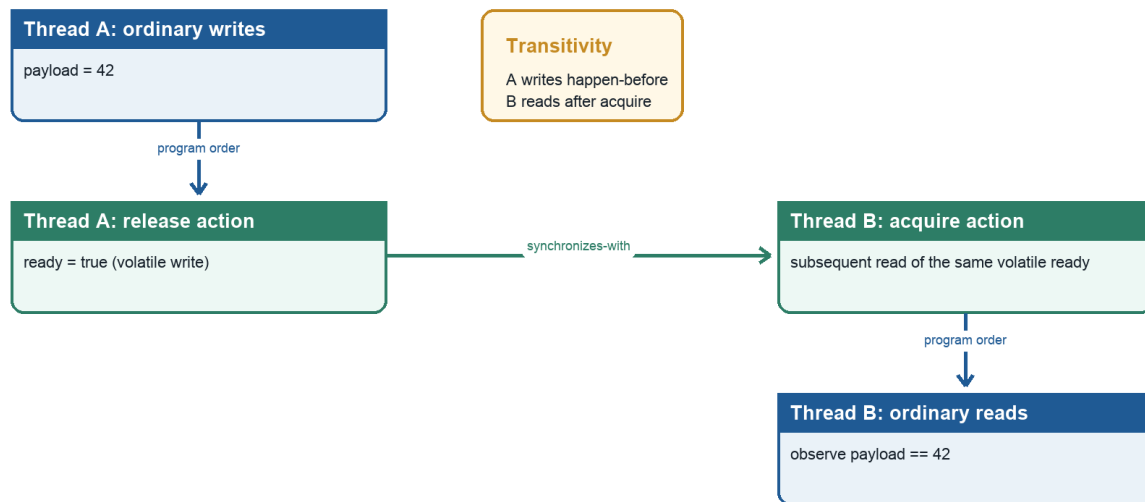
- ☐ I can explain interpreter/JIT tiering as a cost trade-off.
- ☐ I can describe inlining, profiles, speculation, OSR, and deoptimization.
- ☐ I know why optimized frames and objects may be reconstructed.
- ☐ I can distinguish Java heap from code cache.
- ☐ I can explain safepoint polling and TTSP.
- ☐ I can propose evidence for warm-up, compilation, and pause investigations.

SDE-2 CORE CHAPTER

Chapter 11 - The Java Memory Model from First Principles

Happens-before reasoning

Visibility follows synchronization edges, not elapsed wall-clock time



Java Foundations to Advanced Engineering

Figure 11.1 - A happens-before chain through volatile synchronization

11.1 Learning objectives

- Separate atomicity, visibility, and ordering in concurrent Java.
- Use happens-before to prove which writes a read is guaranteed to observe.
- Explain monitor, volatile, thread start/join, task synchronization, default initialization, and final-field rules.
- Understand data races, safe publication, immutable objects, and broken double-checked locking.
- Avoid reasoning only from source interleavings or a particular CPU cache story.

WHY IT WORKS | 11.2 Why this matters at SDE-2

Concurrency bugs often pass tests and fail under production load, another architecture, or a new JIT. Locks are not merely mutual exclusion. They, volatile operations, and lifecycle APIs establish ordering that makes writes visible. Without such relationships, "thread A ran first" is not a proof.

An SDE-2 should be able to review a publication pattern, explain why `volatile` does not make `count++` atomic, select a synchronization mechanism, and state the proof in happens-before terms. This skill underlies concurrent collections, executors, atomics, virtual threads, and reliable service shutdown.

WHY IT WORKS | 11.3 First-principles model

Each thread executes actions governed by its program order, but compilers and processors may transform implementation order when a single thread cannot tell the difference. Multiple threads can observe effects only within constraints imposed by the Java Memory Model (JMM).

The JMM is not a literal diagram of each core copying values to "main memory." It is a formal contract defining legal observations. The central proof relation is happens-before (HB): if action A happens-before action B, then A's effects are ordered before B, and conflicting memory observations must respect that order.

TEXT

```

writer thread                reader thread
data = 42;                   if (ready) {
ready = true; // volatile W ----HB----> volatile R of true
                                   use(data); // guaranteed to see 42
                                   }

```

The volatile write of `ready` synchronizes-with a subsequent volatile read that observes the relevant write in the volatile order. Program-order edges place `data=42` before that write and the `data` read after the volatile read. HB transitivity orders the data write before the data read.

11.4 Core terminology

- **Memory action:** Read/write of a variable, volatile action, lock/unlock, thread lifecycle action, and other JMM-relevant operation.
- **Program order:** Per-thread order consistent with intra-thread semantics.
- **Synchronization action:** Operation participating in the global synchronization order, such as volatile access or monitor lock/unlock.
- **Synchronizes-with:** Specified cross-thread edge created by matching synchronization actions.
- **Happens-before:** Transitive ordering built from program order and synchronizes-with plus other specified rules.
- **Data race:** Two conflicting accesses to the same variable, at least one a write, not ordered by happens-before.
- **Sequential consistency (SC):** Behavior explainable by one total interleaving preserving each thread's program order.
- **Safe publication:** Making an object reachable by other threads through an ordering mechanism that exposes required initialized state.
- **Visibility:** Guarantee about which writes a read can observe.
- **Atomicity:** Indivisibility of an operation relative to other threads.
- **Ordering:** Constraints preventing observable reordering across actions.

11.5 Detailed mechanics

Three properties must be separated:

- **Atomicity:** A read or write may be indivisible, yet a read-modify-write expression is multiple actions. `count++` reads, adds, and writes; two threads can lose an update.
- **Visibility:** One thread may not be guaranteed to observe another thread's ordinary write without HB, even if wall-clock time passed.
- **Ordering:** Operations can be implemented in a different order when allowed, and another thread with a data race can observe results not predicted by a naive source interleaving.

Reads and writes of references and most primitive values are atomic under JMM rules. The specification still permits a non-volatile `long` or `double` access to be implemented as two 32-bit accesses, so a racing read could theoretically tear; volatile `long` and `double` accesses are guaranteed atomic. Mainstream 64-bit HotSpot platforms commonly perform aligned 64-bit accesses atomically, but correctness must not rely on that implementation fact. Atomic individual access still does not make compound invariants atomic. Array elements and fields are distinct variables, and implementations must prevent a write to one from tearing into an adjacent variable.

Happens-before includes these crucial rules:

- 1 Every action in a thread happens-before each later action in that thread's program order.

- 2 An unlock of a monitor happens-before every subsequent lock of that same monitor.
- 3 A write to a volatile variable happens-before every subsequent read of that variable in the synchronization order.
- 4 A call to `Thread.start()` happens-before actions in the started thread.
- 5 All actions in a thread happen-before another thread successfully returns from `join()` on it.
- 6 Default initialization of an object happens-before any other actions, providing zero/default state safety.
- 7 HB is transitive.

Library specifications add edges. For example, executor submission, `Future.get`, concurrent collections, latches, and queues state memory-consistency effects. Use those documented contracts; do not assume that "it uses threads" provides publication.

Monitors provide both mutual exclusion and ordering. Exiting a `synchronized(lock)` block unlocks; later entering a block on the same monitor locks. Synchronizing on different objects creates no matching edge. Reentrancy lets a thread reacquire a monitor it owns but does not weaken cross-thread rules.

Volatile provides ordering/visibility for a variable without making arbitrary multi-step invariants mutually exclusive. A volatile read sees some write consistent with volatile synchronization order; if it sees a publication flag's write, earlier writer effects are visible after that read. A volatile write has release-like effects and a volatile read acquire-like effects as an intuition, while the formal Java rule remains synchronizes-with/HB.

Volatile is suitable for independent state such as a shutdown flag or immutable snapshot reference. It is insufficient for `if (stock > 0) stock--`, `counter++`, or updates across multiple fields that must be observed consistently. Use a lock, an atomic read-modify-write primitive, or an immutable state transition through CAS.

Data-race-free programs whose synchronization is correctly used receive a powerful guarantee often summarized as DRF-SC: executions appear sequentially consistent. This is why synchronization should define the program, not merely add barriers after a bug appears. A data race does not mean "the latest value eventually appears"; allowed observations can be unintuitive while still constrained by causality and safety rules.

Final fields have special initialization safety. If an object's constructor completes and the reference does not escape during construction, another thread that obtains the reference, even through some racy forms, receives specified guarantees for correctly constructed final fields and reachable state covered by the final-field semantics. This is not a license for racy publication: non-final fields, later mutations, compound object graphs, and the reference itself still demand robust publication. Use final fields plus safe publication.

Safe publication idioms include:

- constructing before storing into a `volatile` reference;
- storing while holding a monitor that readers also acquire;
- static initialization/class initialization;
- placing into a concurrent collection or synchronization-aware queue;
- passing state before `Thread.start()`;

- completing a [Future](#), promise-like API, or latch under its documented memory effects.

Thread confinement avoids sharing entirely. Immutability reduces proof surface because state cannot change after construction. Neither helps if mutable components leak or an object's constructor publishes [this](#) prematurely.

Double-checked locking is correct only with a volatile publication field in its standard modern form:

JAVA

```
private static volatile Service instance;

static Service instance() {
    Service local = instance;
    if (local == null) {
        synchronized (Service.class) {
            local = instance;
            if (local == null) {
                local = new Service();
                instance = local;
            }
        }
    }
    return local;
}
```

Without volatile, publication can race with construction effects, and the unsynchronized read has no required HB connection. Simpler static-holder initialization is often preferable.

Specification boundary: The JMM is part of Java language semantics. It defines allowed observations through actions and ordering relations, not cache-flush instructions, CPU fences, or HotSpot compiler phases. A correct proof should remain valid across processors and conforming JVMs.

TRACE IT | 11.6 Worked Java example

JAVA

```

public final class Publication {
    private int payload;
    private volatile boolean ready;

    void publish(int value) {
        payload = value;
        ready = true;
    }

    int await() {
        while (!ready) {
            Thread.onSpinWait();
        }
        return payload;
    }

    public static void main(String[] args) throws InterruptedException {
        Publication publication = new Publication();
        Thread reader = new Thread(() ->
            System.out.println(publication.await()));
        reader.start();
        publication.publish(42);
        reader.join();
    }
}

```

If `await` terminates after reading `ready == true`, it must return 42. The volatile edge publishes the earlier ordinary write. `Thread.onSpinWait()` is only a hint and supplies no visibility guarantee; `volatile` supplies the ordering.

This is an educational pattern, not a recommended blocking design. Busy waiting consumes a carrier/CPU resource. A latch, queue, future, or higher-level synchronizer usually expresses waiting more efficiently.

TRACE IT | 11.7 Execution or memory walkthrough

Name the actions:

TEXT

Main/writer	Reader
S: reader.start()	
P: payload = 42	
W: volatile ready = true	
	R: volatile read ready == true
	Q: read payload
	T: reader terminates
J: reader.join() returns	

Proof:

- 1 S happens-before actions in the started reader, though this alone does not publish the later write P because P occurs after start.

- 2 Writer program order gives $P \text{ HB } W$.
- 3 The volatile rule gives $W \text{ HB } R$ for the subsequent read of the published true write.
- 4 Reader program order gives $R \text{ HB } Q$.
- 5 Transitivity gives $P \text{ HB } Q$, so Q must observe `payload=42` or a later HB-consistent write, and none exists here.
- 6 All reader actions happen-before J , so after `join`, the main thread can safely observe reader effects as defined.

If `ready` is ordinary, P and W are not connected to reader R/Q by synchronization. Source order and physical elapsed time are insufficient. The loop could fail to terminate, or the reader could observe an allowed stale/independently ordered state.

If two writers call `publish` concurrently, volatile does not make the pair (`payload`, `ready`) an atomic transaction. The design assumes a single publication event. For repeated messages, use a queue, lock, sequence protocol, or immutable snapshot held in one volatile reference.

11.8 Complexity and performance

Each volatile read/write is $O(1)$ algorithmically, but its constant cost includes compiler and hardware ordering constraints and cache-coherence traffic. An uncontended monitor can also be fast, while contention introduces queueing, park/unpark, scheduler delay, and convoy effects. Big-O alone is not useful for synchronization choices.

The spin loop performs $O(k)$ reads until publication and consumes CPU while waiting. With no publication, time is unbounded. Blocking synchronization trades wake-up latency for released CPU. Hybrid spin-then-park strategies belong in well-tested concurrency libraries, not ad hoc business code.

False sharing is an implementation/hardware performance issue where independent frequently written variables share cache-coherence granularity. It does not change JMM correctness. Padding or special annotations are JVM-specific optimization techniques requiring measurement.

HotSpot note: HotSpot maps JMM requirements to compiler barriers, machine instructions, cache-coherence protocols, monitor implementations, and runtime stubs appropriate to the target CPU. x86-64 and AArch64 can require different instruction sequences. Those mappings are not portable source-level explanations.

WATCH OUT | 11.9 Edge cases and common mistakes

- Saying `volatile` makes a variable "thread-safe" without defining the invariant.
- Using `volatile int count` and expecting `count++` to be atomic.
- Synchronizing writers but allowing readers to access the same fields without the same lock or another HB edge.
- Locking on different objects, mutable lock references, boxed values, or publicly accessible strings.
- Assuming `sleep`, logging, a debugger, or elapsed time flushes memory.
- Treating `ConcurrentHashMap` safety as automatically making mutable values safely updated under arbitrary operations.
- Starting a thread from a constructor and leaking partially constructed `this`.
- Assuming final reference means deeply immutable graph.
- Using double-checked locking without `volatile`.
- Believing data races merely return "old or new" values in every compound scenario.
- Reasoning from one observed run as proof.

Publication and mutation are separate. Putting a mutable `ArrayList` into a concurrent map can safely publish the list reference according to the map contract, but unsynchronized mutations of that list can still race. Similarly, `Collections.synchronizedList` requires its documented locking discipline during compound iteration.

Deadlock freedom and memory visibility are separate properties. A correct HB proof can still deadlock; a lock-free algorithm can still be logically wrong. Progress properties such as obstruction freedom, lock freedom, wait freedom, fairness, and starvation need separate analysis.

11.10 Production engineering notes

Prefer ownership and higher-level primitives:

- immutable request/config snapshots published atomically;
- executor task boundaries and futures;
- blocking queues for producer-consumer transfer;
- concurrent maps with atomic methods such as `compute` for key-scoped transitions;
- atomics for small independent state machines;
- locks for multi-field invariants;
- structured lifecycle with interruption and join/close semantics.

Document invariants next to guarded fields, for example "guarded by `lock`" or "immutable snapshot published through volatile `current`." Code review should identify every conflicting access and the HB path ordering it, not only verify that some `synchronized` appears.

Race testing is probabilistic. Stress tools, randomized scheduling, high iteration counts, and multiple architectures can expose bugs but cannot prove absence. Static analyzers and disciplined design help. Thread dumps reveal blocking/deadlock states but not all memory-order races. JFR and profiles can reveal lock contention; they do not replace a correctness proof.

When optimizing synchronization, first preserve a written invariant and establish a benchmark representing contention. Replacing a lock with multiple atomics can weaken snapshot consistency. A single volatile immutable state object is often both clearer and faster than coordinating several volatile fields.

TRY IT | 11.11 Interview questions and model answers

What does happens-before mean?

It is the JMM ordering relation used to guarantee visibility and constrain observations. It is built from per-thread program order, synchronizes-with edges such as monitor unlock-to-lock and volatile write-to-read, thread lifecycle rules, and transitivity. It is not simply wall-clock ordering.

What does volatile guarantee?

Individual volatile access is atomic and participates in a total synchronization order. A volatile write happens-before subsequent reads of that variable, publishing earlier writer actions to code after the read. Volatile does not make compound operations or multi-field invariants atomic.

How does `synchronized` affect memory?

It provides mutual exclusion for the same monitor. Unlocking that monitor happens-before a subsequent lock, so writes before unlock are visible to actions after the matching lock. Different monitors do not create that edge.

Why is `count++` unsafe even if reads/writes of `int` are atomic?

Increment is read, compute, write. Two threads can read the same old value and both write the same incremented value. Use `AtomicInteger.incrementAndGet`, a lock, or an appropriate aggregate concurrency design.

How do you safely publish an object?

Construct it without leaking `this`, preferably make state final/immutable, then publish through a specified edge: volatile write/read, matching monitor unlock/lock, class initialization, thread start, concurrent collection, queue, future, or another synchronizer's documented memory effect.

Does `final` make an object thread-safe?

No. Final prevents reassignment of that field after construction and supplies special initialization safety for correctly constructed objects. The referenced object may still be mutable, and later mutations require synchronization.

Why can a racy loop fail to see a flag update?

Without synchronization, there is no HB edge requiring the read to observe the other thread's write. Compiler optimizations and hardware execution may reuse or reorder values within JMM permissions. Volatile or a synchronizer establishes the required relation.

TRY IT | 11.12 Exercises

- 1 Draw the HB graph for the worked example and remove each edge in turn.
- 2 Implement a lost-update counter using plain `int`, then correct it with a lock and an atomic.
- 3 Review a mutable object placed in `ConcurrentHashMap`; identify which operations remain unsafe.
- 4 Implement lazy initialization using the static-holder idiom and compare its proof with volatile double-checked locking.
- 5 Design an immutable configuration snapshot with one volatile reference.
- 6 Explain the memory effects of submitting a task and obtaining its `Future` result from library documentation.
- 7 Find a `this`-escape pattern in a constructor and redesign it with a factory.
- 8 State correctness, liveness, and performance properties separately for a proposed concurrent queue use.

RECAP | 11.13 Chapter summary

The JMM defines legal cross-thread observations through actions and ordering relations, not a literal cache architecture. Atomicity, visibility, and ordering are distinct. Happens-before proofs connect program order with volatile, monitors, thread lifecycle, class initialization, and synchronization-library contracts. Data-race-free designs gain sequentially consistent reasoning. Safe construction, safe publication, immutability, ownership, and high-level synchronizers are the practical path to reliable concurrency.

TRY IT | 11.14 Revision checklist

- ☐ I can distinguish atomicity, visibility, and ordering.
- ☐ I can define data race, synchronizes-with, and happens-before.
- ☐ I can list monitor, volatile, start, join, and initialization edges.
- ☐ I can prove the volatile publication example by transitivity.
- ☐ I know why `volatile count++` is unsafe.
- ☐ I can safely publish immutable and mutable state.
- ☐ I understand final-field guarantees and `this` escape limitations.
- ☐ I can explain double-checked locking and prefer simpler alternatives.
- ☐ I do not substitute CPU-cache folklore or tests for a JMM proof.

JAVA FOUNDATIONS TO ADVANCED ENGINEERING

Part III - Java Language Engineering

A focused part of the complete SDE-2 preparation guide

SDE-2 CORE CHAPTER

Chapter 12 - Variables, Primitive Types, Literals, and Numeric Semantics

12.1 Learning objectives

By the end of this chapter, you should be able to:

- distinguish a variable, a value, a type, and an object reference;
- choose an appropriate primitive type and write unambiguous literals;
- predict integer promotion, floating-point behavior, narrowing, and overflow;
- explain local-variable initialization, `final`, `var`, boxing, and unboxing; and
- design numeric code whose domain assumptions are explicit and testable.

WHY IT WORKS | 12.2 Why this matters at SDE-2

Many production failures are not algorithm failures. They are representation failures: an order total overflows an `int`, a nullable `Integer` is unboxed, a monetary comparison uses `double`, or a shift silently masks its distance. An SDE-2 engineer must see these risks during review, not after an incident.

Interviewers also use short numeric expressions to test whether a candidate reasons from Java rules instead of intuition borrowed from another language. The goal is not memorizing puzzles. It is building a reliable model for conversions and evaluation.

Learning-path note: Build the Java language model in this chapter first. After Java Foundations (JAVA-01) and Time and Space Complexity (DSA-01), continue to Number Systems (DSA-02) for digit algorithms, base conversion, overflow-safe arithmetic, large-number strings, GCD/LCM, modular arithmetic, powers, roots, and number-oriented practice.

WHY IT WORKS | 12.3 First-principles model

A Java variable is a named storage location with a compile-time type. A primitive variable contains a primitive value. A reference variable contains either `null` or a reference to an object or array; it does not contain the object itself.

Java has eight primitive types: `boolean`, `byte`, `short`, `char`, `int`, `long`, `float`, and `double`. Except for `boolean`, these participate in numeric operations. Integral types use fixed-width two's-complement representations in modern Java's specified numeric model. Floating-point types follow IEEE 754 binary floating-point semantics.

A literal is source syntax for a value. Its spelling affects its type: `1` is an `int`, `1L` is a `long`, `1.0` is a `double`, and `1.0F` is a `float`. A conversion may preserve the mathematical value, approximate it, wrap it, or be rejected.

Specification boundary: Java specifies primitive ranges, conversions, and expression results. It does not expose a source-level memory address for a reference or guarantee that a local primitive occupies a particular number of physical bytes in a stack frame.

12.4 Core terminology

- **Declaration:** introduces a variable and its type, such as `long count;`.
- **Initialization:** supplies its first value.
- **Assignment conversion:** conversion allowed when storing an expression into a variable.
- **Widening conversion:** moves to a type with a broader representable domain, although `long` to `float` can lose precision.
- **Narrowing conversion:** explicitly converts to a smaller or different domain and may discard information.
- **Numeric promotion:** converts operands before unary or binary arithmetic.
- **Overflow:** a result outside an integral type's range; ordinary Java integer arithmetic wraps modulo the type's width.
- **Boxing/unboxing:** conversion between a primitive and its wrapper, such as `int` and `Integer`.
- **Compile-time constant:** a restricted expression of primitive or `String` type whose value the compiler can determine.

12.5 Detailed mechanics

Types, ranges, and defaults

`byte` is signed 8-bit, `short` signed 16-bit, `int` signed 32-bit, and `long` signed 64-bit. `char` is an unsigned 16-bit UTF-16 code unit, not necessarily a complete Unicode character. `float` is 32-bit binary floating point and `double` is 64-bit.

Fields and array elements receive default values: numeric zero, `false`, or `null`. Local variables do not. Java's definite-assignment analysis rejects a path that might read an uninitialized local.

JAVA

```
public class InitializationDemo {
    private int field;           // default 0

    public int choose(boolean useField) {
        int local;
        if (useField) {
            local = field;
        } else {
            local = 10;
        }
        return local;           // assigned on every reachable path
    }
}
```

`final` means a variable can be assigned only once. For a reference, it freezes the reference, not the referenced object's state. A blank `final` field must be assigned by every constructor.

`var`, available for local variables since Java 10, asks the compiler to infer a static type from the initializer. It is not dynamic typing, cannot be used for fields or method parameters, and requires an initializer. Prefer it when the inferred type remains obvious.

Literals

Integral literals can be decimal, hexadecimal (`0xFF`), binary (`0b1111`), or octal (`017`). Underscores may group digits but cannot occur at the literal's beginning, end, or next to a radix prefix or decimal point. An integer literal without `L` has type `int` and must satisfy the `int` literal range rules; use uppercase `L` for `long` because lowercase `l` resembles `1`.

JAVA

```
long nanos = 1_000_000_000L;
int permissions = 0b110_100_100;
int mask = 0xFF;
double ratio = 6.25e-2;           // 0.0625
float sample = 3.5F;
char newline = '\n';
char omegaCodeUnit = '\u03A9';
```

A special rule lets a constant `int` expression be assigned to `byte`, `short`, or `char` if the value fits. The same value held in a non-constant variable requires a cast.

JAVA

```
byte a = 100;           // constant fits
final int fixed = 100;
byte b = fixed;         // constant variable fits
int runtime = 100;
// byte c = runtime;    // compile-time error
byte c = (byte) runtime;
```

Promotion and evaluation

Unary numeric promotion changes `byte`, `short`, and `char` to `int`. Binary numeric promotion normally chooses `double`, then `float`, then `long`, otherwise `int`. Thus two `byte` operands add as `int`.

JAVA

```
byte x = 10;
byte y = 20;
int sum = x + y;
// byte bad = x + y;
x += y;           // compound assignment includes an implicit cast
```

Compound assignment `E1 op= E2` behaves roughly like `E1 = (T)(E1 op E2)`, while evaluating the left side once. That implicit narrowing can hide wraparound.

Integer division truncates toward zero. The remainder has the dividend's sign. Division by integral zero throws `ArithmeticException`, but floating-point division by zero produces infinity or NaN under IEEE 754 rules.

JAVA

```
System.out.println(-7 / 3);    // -2
System.out.println(-7 % 3);    // -1
System.out.println(1.0 / 0.0); // Infinity
System.out.println(0.0 / 0.0); // NaN
```

Overflow and exact arithmetic

Ordinary integer overflow does not throw. `Integer.MAX_VALUE + 1` becomes `Integer.MIN_VALUE`. Use `Math.addExact`, `Math.subtractExact`, `Math.multiplyExact`, `Math.incrementExact`, or conversion helpers such as `Math.toIntExact` when overflow is an invalid domain event.

Floating-point values model a finite subset of binary fractions. Decimal `0.1` is not exactly representable, so repeated addition can accumulate visible error. `BigDecimal` is appropriate for decimal business arithmetic, but construction matters: prefer a string or `BigDecimal.valueOf(double)` over `new BigDecimal(0.1)`.

NaN is unordered: every relational comparison with NaN is false, and `NaN == NaN` is false. Positive and negative zero compare equal with `==`, although some library operations distinguish their bit patterns.

Boxing and unboxing

Wrapper objects enable primitives in generic APIs. Boxing may allocate, but the language does not promise object identity. `Integer` commonly caches a small range, so `==` can appear to work and then fail. Compare wrapper values with `equals` or `Objects.equals`.

Unboxing `null` throws `NullPointerException` at the conversion site. A conditional expression or overloaded call can trigger unboxing less visibly than a direct assignment.

TRACE IT | 12.6 Worked Java example

This invoice example treats cents as exact integral units and rejects overflow.

JAVA

```
import java.math.BigDecimal;

public class InvoiceMath {
    static long lineTotalCents(long unitCents, int quantity) {
        if (unitCents < 0 || quantity < 0) {
            throw new IllegalArgumentException("negative price or quantity");
        }
        return Math.multiplyExact(unitCents, (long) quantity);
    }

    static BigDecimal dollars(long cents) {
        return BigDecimal.valueOf(cents, 2);
    }

    public static void main(String[] args) {
        long first = lineTotalCents(1_299L, 3);
        long second = lineTotalCents(250L, 4);
        long total = Math.addExact(first, second);
        System.out.println(total);           // 4897
        System.out.println(dollars(total));  // 48.97
    }
}
```

The public unit is explicit in the names. Multiplication promotes quantity to `long` before evaluation, and exact helpers turn an impossible invoice into an exception rather than corrupted data.

TRACE IT | 12.7 Execution or memory walkthrough

For `lineTotalCents(1_299L, 3)`, the literal `1_299L` is already `long`; `3` is `int`. The method receives copies of both primitive values. The range checks evaluate without conversion surprises. Casting `quantity` to `long` ensures 64-bit multiplication, producing `3_897L`.

The second line produces `1_000L`. `Math.addExact` returns `4_897L`. `BigDecimal.valueOf(4897, 2)` represents an unscaled integer of 4897 with scale 2, so its decimal value is exactly 48.97.

If multiplication exceeded `long`, `multiplyExact` would throw before a wrapped value reached persistence. No wrapper is needed on this hot path. The values may be held in registers or stack-frame slots, but that placement is a JVM implementation decision.

12.8 Complexity and performance

Primitive arithmetic is constant time for fixed-width types. Exact arithmetic helpers are also $O(1)$, with a small overflow check. `BigInteger` and `BigDecimal` costs grow with precision; their operations allocate immutable result objects and are unsuitable as automatic replacements for every primitive.

Boxing can create objects and increase garbage-collection pressure. Specialized primitives, primitive streams, or domain objects backed by `long` often help in high-volume paths. Measure before optimizing, because escape analysis and JIT compilation can remove some allocations.

HotSpot note: HotSpot may keep values in registers, scalar-replace non-escaping wrappers, and use cached wrapper instances. None of these optimizations changes Java's observable value semantics or makes wrapper identity safe to depend on.

WATCH OUT | 12.9 Edge cases and common mistakes

- `long result = intA * intB;` performs `int` multiplication before widening. Cast an operand first.
- `Math.abs(Integer.MIN_VALUE)` is still negative because its positive magnitude does not fit. `Math.absExact` throws instead.
- A shift distance is masked: for `int`, only the low five bits are used; `x << 32` is effectively `x << 0`.
- `>>` sign-extends; `>>>` shifts zero bits into the high end.
- Narrowing a floating-point value truncates toward zero, clamps out-of-range finite values to the target endpoint, and converts NaN to zero.
- `0.1 + 0.2 == 0.3` is false in binary floating point.
- Autounboxing a nullable wrapper can fail during comparison, arithmetic, or overload selection.
- `char` arithmetic promotes to `int`; a surrogate pair requires two `char` values.
- Never use wrapper `==` for value comparison.

12.10 Production engineering notes

Put units in names and APIs: `timeoutMillis`, `amountCents`, and `bytesPerSecond`. Validate at boundaries, use exact helpers where overflow violates the domain, and test minimum, maximum, zero, and just-outside-range cases.

For money, decide whether fixed minor units or `BigDecimal` best fits currency and rounding requirements. Specify scale and `RoundingMode`; do not let defaults leak into accounting behavior. For scientific measurements, `double` is often correct, but define tolerances and NaN handling.

Be cautious when database, JSON, and Java numeric domains differ. A JavaScript client cannot exactly represent every `long` as a number. Database `DECIMAL` scale may reject or round a `BigDecimal`. Treat conversion as part of the contract.

TRY IT | 12.11 Interview questions and model answers

Why does `long n = 1_000_000 * 1_000_000`; not produce one trillion?

Both operands are `int`, so multiplication occurs in 32 bits and overflows before assignment widens the wrapped result. Make either operand `long`, for example `1_000_000L * 1_000_000`.

Is widening always lossless?

No. Widening integral conversion preserves values except conversions to floating point can lose precision. A sufficiently large `long` cannot be represented exactly by `float` or even by `double`.

What does `final List<String> names` guarantee?

The variable cannot be reassigned after initialization. It does not make the list immutable; callers can still mutate the referenced list unless the object itself prevents mutation.

Why can `Integer a = 100; Integer b = 100; a == b` be true?

Boxing implementations reuse some wrapper instances, and the language requires caching for certain constant values. `==` asks about reference identity, not numeric equality, so code must not rely on that result outside the specified caching case. Use `a.equals(b)` or `Objects.equals`.

When would you choose `BigDecimal` over scaled `long`?

Choose `BigDecimal` when variable decimal scale, large magnitudes, or explicit decimal rounding are central. Scaled `long` is compact and fast when the unit and range are fixed. In either design, encode scale and rounding in the domain API.

TRY IT | 12.12 Exercises

- 1 Predict the type and value of `byte b = 127; b += 1;`. Rewrite it so overflow throws.
- 2 Implement `average(long a, long b)` without overflowing their sum. Consider negative values.
- 3 Explain every conversion in `double d = 3 + 4L / 2.0F`.
- 4 Write tests for converting an incoming decimal price into cents with a required scale of two.
- 5 Find and fix nullable unboxing in `boolean active = map.get(id);`.
- 6 Compare `new BigDecimal("0.10")` and `new BigDecimal("0.1")` using both `equals` and `compareTo`; explain the different contracts.

RECAP | 12.13 Chapter summary

Java's numeric rules are deterministic: literals have types, operands are promoted before evaluation, narrowing may discard information, and ordinary integral overflow wraps. Primitive variables hold values while reference variables hold references. `final` restricts assignment, `var` preserves static typing, and wrappers introduce identity and nullability concerns. Robust systems make units, range, precision, scale, and overflow policy visible in their APIs.

TRY IT | 12.14 Revision checklist

- ☐ I can state all eight primitive types and their roles.
- ☐ I can distinguish default field values from definite assignment of locals.
- ☐ I can trace unary and binary numeric promotion.
- ☐ I cast before arithmetic when the operation needs a wider type.
- ☐ I know how integer division, remainder, shifts, and overflow behave.
- ☐ I can explain NaN, infinity, signed zero, and decimal approximation.
- ☐ I use exact arithmetic or `BigDecimal` when the domain requires it.
- ☐ I avoid wrapper identity comparisons and defend against null unboxing.

SDE-2 CORE CHAPTER

Chapter 13 - Operators, Expressions, and Control Flow

13.1 Learning objectives

By the end of this chapter, you should be able to:

- evaluate Java expressions using precedence, associativity, promotion, and short-circuit rules;
- distinguish statement forms from value-producing expressions;
- design readable branching and iteration with explicit invariants;
- use modern `switch` expressions safely; and
- identify control-flow bugs caused by side effects, fall-through, labels, and abrupt completion.

WHY IT WORKS | 13.2 Why this matters at SDE-2

Control flow is where requirements become executable decisions. At SDE-2 level, correctness includes more than reaching the happy-path result. You must reason about whether a side effect runs, whether a loop terminates, whether cleanup executes, and whether all domain states are represented.

Interview code often compresses these questions into a loop or conditional. Production code spreads them across authorization, retries, state transitions, and pricing rules. A precise expression model makes both settings easier.

Focused series path: Finish Java Foundations (JAVA-01) first, then study Time and Space Complexity (DSA-01). Continue to Number Systems (DSA-02) for modulo and overflow-safe numeric patterns, Bit Manipulation (DSA-04), and Loop Mastery (DSA-05). The stable filenames and recommended order are listed in [Java-SDE2-Interview-Preparation-Series-Index.pdf](#).

WHY IT WORKS | 13.3 First-principles model

An expression is evaluated to produce a value, a variable, or a side effect. A statement controls execution: declaration, expression statement, block, branch, loop, transfer, or exception. Operators define how operand values are combined, but evaluation order determines when operands and side effects occur.

Java evaluates operand expressions from left to right. Precedence determines grouping, not temporal reordering. Parentheses should communicate intent even when precedence already makes code legal.

Control normally completes a statement and continues to the next one. `break`, `continue`, `return`, and `throw` complete abruptly and transfer control. Loops are best understood through an invariant: a property true before and after each iteration, plus a progress measure that proves termination.

Specification boundary: Java specifies left-to-right operand evaluation and short-circuit behavior. An optimizing JVM may rearrange machine instructions only if the observable Java behavior, including exceptions and synchronization semantics, is preserved.

13.4 Core terminology

- **Operand:** input expression to an operator.
- **Precedence:** which operator binds more tightly in source parsing.
- **Associativity:** how equal-precedence operators group.
- **Short-circuiting:** skipping the right operand when the left determines a boolean result.
- **Side effect:** observable state change, I/O, synchronization action, or exception.
- **Statement expression:** an expression allowed as a statement, such as assignment or method invocation.
- **Loop invariant:** claim maintained by every completed iteration.
- **Abrupt completion:** transfer caused by `break`, `continue`, `return`, or `throw`.
- **Exhaustiveness:** every possible selector value is handled by a `switch` expression.

13.5 Detailed mechanics

Operator families

Arithmetic operators are `+`, `-`, `*`, `/`, and `%`. Unary `+` and `-` apply numeric promotion. Prefix and postfix `++` and `--` both mutate a variable; postfix yields the old value and prefix yields the new value. Avoid embedding them in larger expressions.

Relational operators (`<`, `<=`, `>`, `>=`) compare numeric values. Equality (`==`, `!=`) compares primitive values or reference identity after permitted conversions. It does not invoke `equals`. Boolean logical operators are `!`, `&&`, `||`, `&`, `|`, and `^`. With booleans, `&` and `|` evaluate both sides; `&&` and `||` short-circuit.

Bitwise `&`, `|`, `^`, and `~` operate on promoted integral values. Shifts are `<<`, `>>`, and `>>>`. Assignment operators include simple `=` and compound forms such as `+=` and `&=`.

The conditional operator `condition ? whenTrue : whenFalse` evaluates exactly one branch. Its result type follows detailed rules involving numeric constants, boxing, reference least-upper-bound analysis, and `null`; do not assume it is simply the type of one textual branch.

Evaluation and short-circuiting

JAVA

```
static boolean valid(String value) {  
    return value != null && !value.isBlank();  
}
```

If `value` is `null`, `value != null` is false and `isBlank` is never called. Replacing `&&` with `&` would evaluate it and throw. Short-circuit operators also encode ordering, so avoid hiding expensive or state-changing operations in their right operands.

Java evaluates the target and arguments of a method invocation from left to right before calling the method. In `array[index()] += delta()`, the array reference and index are evaluated once, then the current element and right operand are combined.

Branches

An `if` condition must be `boolean` or `Boolean` subject to unboxing; Java does not treat integers or arbitrary references as truthy. Braces prevent misleading indentation and reduce maintenance risk.

Classic `switch` statements use colon labels and can fall through. Arrow labels do not fall through. A `switch` expression produces a value and must be exhaustive.

JAVA

```
enum Tier { FREE, STANDARD, PREMIUM }  
  
static int retentionDays(Tier tier) {  
    return switch (tier) {  
        case FREE -> 7;  
        case STANDARD -> 30;  
        case PREMIUM -> 365;  
    };  
}
```

For an enum selector, covering every declared constant makes this source exhaustive. A compiler-generated defensive path can still throw if separately compiled code supplies a newer enum constant at runtime. `switch` on traditional supported reference selectors throws `NullPointerException` for `null` unless modern pattern-switch labels explicitly handle `null`.

A block arm in a switch expression uses `yield` to provide its value:

JAVA

```
int cost = switch (tier) {  
    case FREE -> 0;  
    case STANDARD -> {  
        audit("standard pricing");  
        yield 20;  
    }  
    case PREMIUM -> 50;  
};
```

Loops and transfer

while checks before an iteration. **do-while** runs the body once before checking. A basic **for** has initialization, condition, and update components. Enhanced **for** delegates traversal to an iterator for **Iterable** objects or uses array indexing semantics for arrays.

JAVA

```
static int firstAtLeast(int[] values, int threshold) {
    for (int i = 0; i < values.length; i++) {
        if (values[i] >= threshold) {
            return i;
        }
    }
    return -1;
}
```

break exits the nearest loop or switch statement. **continue** moves to the next loop iteration; in a basic **for**, the update expression still runs. A labeled **break** or **continue** can target an enclosing statement or loop, but extraction into a method is often clearer.

return exits a method, and **throw** propagates an exception. A **finally** block runs during most abrupt transfers, but a **return** or **throw** from **finally** can replace the original outcome and should be avoided.

TRACE IT | 13.6 Worked Java example

The following retry policy combines an exhaustive switch expression with a loop whose termination is explicit.

JAVA (continued 1/2)

```
import java.time.Duration;

public class RetryPolicy {
    enum Result { SUCCESS, RETRYABLE, PERMANENT_FAILURE }

    interface Operation {
        Result run(int attempt);
    }

    static boolean execute(int maxAttempts, Operation operation) {
        if (maxAttempts < 1) {
            throw new IllegalArgumentException("maxAttempts must be positive");
        }

        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            Result result = operation.run(attempt);
            boolean stop = switch (result) {
                case SUCCESS -> true;
                case PERMANENT_FAILURE -> true;
            };
        }
    }
}
```

JAVA (continued 2/2)

```

        case RETRYABLE -> attempt == maxAttempts;
    };

    if (stop) {
        return result == Result.SUCCESS;
    }
    Duration delay = Duration.ofMillis(100L * attempt);
    System.out.println("retry after " + delay);
}
throw new AssertionError("loop is exhaustive");
}

public static void main(String[] args) {
    boolean ok = execute(3, attempt ->
        attempt < 3 ? Result.RETRYABLE : Result.SUCCESS);
    System.out.println(ok);
}
}

```

The last throw documents an unreachable condition based on the validated positive limit and loop exits. In a real service, sleeping should respect interruption and policy jitter; the example focuses on control flow.

TRACE IT | 13.7 Execution or memory walkthrough

`maxAttempts` is validated once. Iteration one invokes the operation with `1`, which returns `RETRYABLE`. The switch selects its third arm; `attempt == maxAttempts` is false, so `stop` is false. A 100 millisecond duration is printed.

Iteration two follows the same path and prints 200 milliseconds. The for-update increments `attempt` after each normally completed body. Iteration three returns `SUCCESS`; the first switch arm yields true. The `if` executes and returns `true`, so neither the update nor another condition check runs.

At every loop entry, the invariant is `1 <= attempt <= maxAttempts`. Progress is the increment of `attempt`, and the upper bound proves termination. Only one switch arm executes on each iteration.

13.8 Complexity and performance

Expression operators on fixed-width primitives are $O(1)$. A branch is $O(1)$, while loop cost is the iteration count multiplied by body cost. The example makes at most `maxAttempts` operation calls, so it is $O(n)$ time and $O(1)$ extra space, excluding work performed by the operation.

Branch shape can influence CPU prediction, and switch compilation may use tables, lookups, or comparison chains. Such choices are implementation details and rarely justify distorting domain code. First minimize unnecessary work and choose an appropriate algorithm; then profile representative loads.

HotSpot note: HotSpot may inline branches, eliminate unreachable paths, unroll loops, or compile a switch to different machine structures based on profiling. Source semantics and exception behavior remain the contract.

WATCH OUT | 13.9 Edge cases and common mistakes

- `a < b < c` is illegal; the first comparison is boolean. Write `a < b && b < c`.
- `if (flag = true)` assigns and then tests. Prefer `if (flag)` and enable static analysis.
- `&` and `|` do not short-circuit boolean operands.
- Post-increment returns the previous value: `a[i++] = i` is legal but hard to review.
- A missing `break` in a colon-style switch falls through, sometimes intentionally and often accidentally.
- A semicolon after `if` or `while` creates an empty controlled statement.
- Mutating a collection structurally during enhanced-for traversal usually triggers fail-fast behavior; use the iterator's removal operation or another strategy.
- Comparing references with `==` does not compare object content.
- Floating-point NaN makes all relational comparisons false, including both `x < y` and `x >= y`.
- A `continue` can skip state updates placed in the loop body and create nontermination.
- Deep labels often signal that the body should become a named method.

13.10 Production engineering notes

Keep decision tables centralized. An exhaustive enum switch can make a missing business state a compiler concern, but account for enum evolution across independently deployed components. For workflows with many transitions, model legal state transitions explicitly rather than building a maze of conditionals.

Separate predicates from side effects. A name such as `isEligible` should not write to a database. This makes short-circuiting safe to understand and lets tests cover decision logic deterministically.

Loops over external work need budgets: maximum attempts, deadlines, pagination termination, cancellation, and metrics. Check thread interruption when doing blocking or long-running work. In retry code, distinguish an attempt count from a retry count and test off-by-one boundaries.

TRY IT | 13.11 Interview questions and model answers

What is the difference between precedence and evaluation order?

Precedence decides how tokens group, as multiplication grouping before addition. Java then evaluates operand expressions left to right. Parentheses can change grouping but do not authorize arbitrary operand reordering.

When should `&` be used with booleans?

Only when both boolean operands intentionally must be evaluated, which is uncommon in application predicates. `&&` is usually safer because it encodes conditional evaluation. `&` remains useful as a bitwise operator for integral masks.

How is a switch expression different from a classic switch statement?

A switch expression produces a value, must be exhaustive, and commonly uses non-fall-through arrow arms. A classic colon-style statement controls execution and may fall through. A block expression arm supplies its result with `yield`.

How do you prove a loop terminates?

State an invariant and a bounded progress measure. For an index loop, show that the index begins in range, advances on every continuing path, and is bounded by a finite length. External loops also need a deadline or other finite budget.

Does `finally` always execute?

It executes for normal and most abrupt completion of its `try`, but not if the JVM process terminates or cannot continue, such as `System.exit`, a crash, or forced kill. It is also dangerous for `finally` to return because that replaces an earlier return or exception.

TRY IT | 13.12 Exercises

- 1 Trace `int x = 1; int y = x++ + ++x;` using left-to-right evaluation. Then rewrite it for clarity.
- 2 Convert a fall-through day-category switch into an exhaustive switch expression.
- 3 State an invariant and progress measure for binary search.
- 4 Implement pagination that stops on an empty page, a repeated cursor, a maximum page count, or interruption.
- 5 Explain why `user != null & user.active()` is unsafe and whether changing to `&&` fully solves the design problem.
- 6 Write a truth table for `&&`, `||`, `^`, and equality on booleans, noting which operands execute.

RECAP | 13.13 Chapter summary

Java expressions follow fixed grouping, conversion, left-to-right evaluation, and short-circuit rules. Statements organize those expressions into branches, loops, and abrupt transfers. Modern switch expressions make value mapping and exhaustiveness explicit. Reliable iteration requires an invariant, a progress measure, and a finite budget. Readable control flow minimizes hidden side effects and makes every exit path intentional.

TRY IT | 13.14 Revision checklist

- ☐ I distinguish precedence, associativity, and evaluation order.
- ☐ I know which boolean operators short-circuit.
- ☐ I avoid mutation hidden inside complex expressions.
- ☐ I can use switch expressions, arrow arms, and `yield`.
- ☐ I understand fall-through and null selector behavior.
- ☐ I can state a loop invariant and prove termination.
- ☐ I understand `break`, `continue`, `return`, and `throw` as abrupt completion.
- ☐ I place explicit budgets around retries and external iteration.

SDE-2 CORE CHAPTER

Chapter 14 - Methods, Overloading, Varargs, and Pass-by-Value

14.1 Learning objectives

By the end of this chapter, you should be able to:

- explain method declarations, signatures, return contracts, and invocation;
- predict compile-time overload selection and runtime overriding separately;
- use varargs without ambiguity, mutation leaks, or heap-pollution surprises;
- demonstrate Java's pass-by-value rule for primitives and references; and
- design methods with cohesive responsibilities and stable API boundaries.

WHY IT WORKS | 14.2 Why this matters at SDE-2

Methods are Java's primary unit of behavior and API design. Subtle errors arise when overloaded APIs accept `null`, generic and varargs methods interact, or a developer expects a callee to replace the caller's reference. These issues appear in framework APIs, backward-compatible library evolution, and interviews about dispatch.

At SDE-2 level, a correct method is not enough. Its contract must communicate nullability, mutation, failure, ownership, cost, and concurrency expectations. Overload sets must remain predictable as the codebase evolves.

WHY IT WORKS | 14.3 First-principles model

A method declaration gives code a name, parameter list, return type, modifiers, optional type parameters, and declared checked exceptions. On invocation, Java evaluates the receiver and arguments, creates a new invocation context, and initializes each parameter with a copy of its corresponding argument value.

For a primitive argument, the copied value is the primitive. For a reference argument, the copied value is the reference. Caller and callee can therefore refer to the same mutable object, but assigning the callee's parameter cannot replace the caller's variable. Java has no pass-by-reference parameter mode.

Invocation has two major decisions. The compiler selects an applicable declaration using the compile-time types and overload rules. If the chosen method is an overridable instance method, runtime dynamic dispatch selects the most specific override for the receiver's actual class.

Specification boundary: Java is always pass-by-value. References are values that can be copied. The JVM's physical calling convention may use registers, stack locations, or optimized inlining, but these do not create source-language pass-by-reference semantics.

14.4 Core terminology

- **Formal parameter:** variable declared by a method.
- **Argument:** expression supplied at a call site.
- **Signature:** method name plus type parameters after adaptation and formal parameter types; return type is not part of overload identity.
- **Overloading:** same method name with different parameter lists, resolved at compile time.
- **Overriding:** subclass instance method supplies behavior for an inherited method, dispatched at runtime.
- **Arity:** number of arguments or parameters.
- **Varargs:** variable-arity final parameter declared with `T...` and represented as an array.
- **Applicable method:** overload that can accept the arguments in a particular invocation phase.
- **Covariant return:** override narrows a reference return type.
- **Bridge method:** compiler-generated adapter that preserves polymorphism after type erasure.

14.5 Detailed mechanics

Declaration and contracts

JAVA

```
public static <T> T require(T value, String message) {  
    if (value == null) {  
        throw new IllegalArgumentException(message);  
    }  
    return value;  
}
```

`public static` are modifiers, `<T>` declares a method type parameter, the next `T` is the return type, and two formal parameters follow. A non-`void` method must not complete normally without returning a compatible value. Returning a reference does not copy the object.

Parameter names are generally not part of binary method identity. Overloading cannot differ only by return type, parameter names, `throws` clauses, or generic type arguments erased to the same signature.

Overload resolution phases

Conceptually, Java searches in phases to preserve compatibility:

- 1 fixed-arity methods applicable without boxing or unboxing;
- 2 fixed-arity methods allowing boxing and unboxing; then
- 3 variable-arity methods.

Within a phase, it chooses a most-specific method. Widening a primitive is preferred over boxing because it succeeds in an earlier phase.

JAVA

```
static String choose(long value) { return "long"; }
static String choose(Integer value) { return "Integer"; }
static String choose(int... values) { return "varargs"; }

System.out.println(choose(5)); // long
```

The `int` literal widens to `long` during strict fixed-arity invocation. Boxing and varargs are considered only if no earlier-phase candidate works.

`null` can make an overload ambiguous when unrelated reference parameter types are equally specific:

JAVA

```
static void send(String value) {}
static void send(StringBuilder value) {}
// send(null); // ambiguous
```

Casting documents the intended overload, but a public API that repeatedly needs such casts probably has a poor overload set.

Static selection and dynamic dispatch

JAVA

```
class Parent {
    String label(Object value) { return "Parent:Object"; }
}
class Child extends Parent {
    @Override String label(Object value) { return "Child:Object"; }
    String label(String value) { return "Child:String"; }
}

Parent p = new Child();
System.out.println(p.label("x")); // Child:Object
```

The compile-time type of `p` exposes only `Parent.label(Object)`, so overload selection chooses that signature. Runtime dispatch then finds `Child`'s override of `label(Object)`. It does not restart overload selection to discover `label(String)`.

Static methods are hidden, not overridden. Private methods are not inherited as overridable members. Constructors are not methods and are not inherited. `final` instance methods cannot be overridden.

Varargs

`void log(String... messages)` is compiled using a `String[]` parameter. A call such as `log("a", "b")` causes the call site to create an array, while `log(existingArray)` can pass the existing array directly. This creates an ownership risk if the callee mutates it.

Only the final formal parameter may be varargs. A method is treated as fixed arity first, which is why passing an existing array does not require another wrapper array.

Generic varargs can create non-reifiable array types. The compiler warns because the runtime array cannot fully check its element type. `@SafeVarargs` is a promise by the author that an eligible method does not perform unsafe operations on or expose its varargs array. It suppresses warnings; it does not make unsafe code safe.

JAVA

```
@SafeVarargs
static <T> java.util.List<T> immutableList(T... values) {
    return java.util.List.copyOf(java.util.Arrays.asList(values));
}
```

This method reads the array and returns an independent unmodifiable list. It does not store incompatible values or expose the array.

Pass-by-value and mutation

JAVA

```
static void change(StringBuilder builder, int number) {
    builder.append("!");
    builder = new StringBuilder("replacement");
    number = 99;
}

StringBuilder text = new StringBuilder("hello");
int count = 1;
change(text, count);
System.out.println(text); // hello!
System.out.println(count); // 1
```

The copied reference lets `change` mutate the original builder. Reassigning the parameter changes only the callee's local copy. The copied integer is similarly independent.

TRACE IT | 14.6 Worked Java example

This API avoids an ambiguous overload set and defensively handles its variable-arity input.

JAVA

```
import java.time.Instant;
import java.util.List;
import java.util.Objects;

public final class AuditEvent {
    private final String action;
    private final Instant occurredAt;
    private final List<String> tags;

    private AuditEvent(String action, Instant occurredAt, List<String> tags) {
        this.action = Objects.requireNonNull(action);
        this.occurredAt = Objects.requireNonNull(occurredAt);
        this.tags = List.copyOf(tags);
    }

    public static AuditEvent now(String action, String... tags) {
        Objects.requireNonNull(tags, "tags");
        return at(action, Instant.now(), List.of(tags.clone()));
    }

    public static AuditEvent at(
        String action, Instant occurredAt, List<String> tags) {
        return new AuditEvent(action, occurredAt, tags);
    }

    public List<String> tags() {
        return tags;
    }
}
```

Named factories distinguish convenience construction from deterministic construction. The full method takes a collection rather than adding many same-name overloads. `List.copyOf` validates elements and creates an unmodifiable snapshot when necessary.

TRACE IT | 14.7 Execution or memory walkthrough

For `AuditEvent.now("login", "security", "user")`, the call site constructs a `String[]` containing two references. `now` receives a copy of the array reference. It checks that reference, clones the array, and creates a list view over the clone. The clone prevents a caller-supplied array from being affected by internal processing.

`at` receives copies of three references. The constructor receives another set of copied references.

`List.copyOf` rejects null elements and returns an unmodifiable list whose contents cannot be changed through the original list. The final fields receive references once; immutability also depends on the referenced values. `String` and `Instant` are immutable, and the tag list is unmodifiable with immutable string elements.

The JVM may inline these calls and eliminate some intermediate allocations, but semantically every parameter is initialized from a copied value.

14.8 Complexity and performance

A direct method invocation is $O(1)$ before considering method work. Virtual dispatch is also constant time conceptually. Varargs array creation is $O(n)$ time and space in the argument count; cloning adds another $O(n)$. `List.copyOf` may copy in $O(n)$, depending on its input.

Tiny methods can be inlined by a JIT, so architectural clarity usually matters more than avoiding calls. Varargs on a high-frequency logging or serialization path can allocate even when the callee does little. Offer collection, builder, or fixed-arity forms only when measurement and API clarity support them.

HotSpot note: HotSpot uses profiling and class-hierarchy knowledge to inline or devirtualize calls and may scalar-replace short-lived varargs arrays. These are optimizations, not guarantees that allocation has no cost.

WATCH OUT | 14.9 Edge cases and common mistakes

- Return type alone cannot distinguish overloads.
- `null` can be ambiguous or select a surprising most-specific reference overload.
- Widening, boxing, and varargs do not have equal priority.
- Autounboxing during invocation can throw `NullPointerException`.
- Overloading a method in a subclass does not override a different parameter signature.
- A static call is selected using compile-time type, even when invoked through an instance expression.
- Passing an existing array to a varargs method may let the callee mutate caller-owned data.
- Calling a varargs method with plain `null` can mean a null array rather than one null element and may produce a warning. Cast explicitly if unavoidable.
- Generic varargs plus array writes can produce heap pollution and delayed `ClassCastException`.
- A method that returns an internal mutable collection leaks representation.
- Recursive methods need both a base case and progress toward it; large depth can exhaust the thread stack.

14.10 Production engineering notes

Prefer cohesive methods named for business intent. Keep validation near boundaries and make ownership explicit: snapshot, view, mutable result, or caller-owned input. Public overloads should share one canonical implementation so fixes do not diverge.

Evolving an API requires source, binary, and behavioral compatibility analysis. Adding an overload can make existing source calls ambiguous or change which declaration recompilation selects. Changing a return type is not overloading; covariant changes are limited to overriding. Framework reflection may also observe bridge, synthetic, or parameter metadata differently.

Avoid boolean parameter clusters such as `send(data, true, false)`. An options object, enum, or separate named operation communicates more. Document nullability and checked failures; do not make callers reverse-engineer them from implementation.

TRY IT | 14.11 Interview questions and model answers

Is Java pass-by-reference for objects?

No. The argument value is a reference, and Java copies that value into the parameter. Callee and caller can use their copies to reach the same object, so mutation is visible. Parameter reassignment is not visible to the caller.

What is selected at compile time versus runtime?

Overload resolution and static method selection use compile-time types. For a selected overridable instance signature, runtime dispatch chooses the override associated with the actual receiver class.

Why does primitive widening beat boxing?

Invocation resolution first searches fixed-arity methods applicable without boxing. Only if none is suitable does it search with boxing, followed later by varargs. This phased design preserves older method-selection behavior.

What does `@SafeVarargs` guarantee?

It is an assertion by the method author that operations on the generic varargs parameter are type safe. The compiler trusts it for warnings; neither the annotation nor runtime verifies the implementation. Apply it only to eligible non-overridable methods and audit the body.

Can an override throw broader checked exceptions?

No. It may declare the same checked exceptions or narrower subclasses, and it may omit them. Unchecked exceptions are not restricted in the same way. This preserves the caller's compile-time exception contract.

TRY IT | 14.12 Exercises

- 1 Create overloads for `f(long)`, `f(Integer)`, and `f(int...)`; predict calls with `1`, `Integer.valueOf(1)`, and an `int[]`.
- 2 Trace a base-reference/subclass-object example containing one overload and one override.
- 3 Rewrite a `swap(Object a, Object b)` attempt and explain why it cannot swap caller variables. Design a return-value alternative.
- 4 Audit a generic varargs method for array writes, escaping references, and unsafe casts.
- 5 Refactor a method with four boolean parameters into a clear options type.
- 6 Design tests proving that a method does not retain or mutate a caller's input array.

RECAP | 14.13 Chapter summary

Methods establish behavioral boundaries. Java initializes every parameter with a copied argument value, including copied references. Overload resolution is compile-time and phased; overriding is runtime dispatch of a previously selected instance signature. Varargs are arrays with allocation, ownership, and generic safety implications. Strong APIs minimize ambiguous overloads, document mutation and failure, and funnel convenience entry points into one canonical implementation.

TRY IT | 14.14 Revision checklist

- ☐ I can identify what is and is not part of a method signature.
- ☐ I understand the widening, boxing, and varargs resolution phases.
- ☐ I separate overload selection from override dispatch.
- ☐ I can demonstrate pass-by-value using a mutable object and reassignment.
- ☐ I treat varargs arrays as ordinary arrays with ownership concerns.
- ☐ I know why generic varargs can cause heap pollution.
- ☐ I design overload sets that remain clear with `null` and future evolution.
- ☐ I document method contracts for mutation, nullability, cost, and failure.

SDE-2 CORE CHAPTER

Chapter 15 - Arrays, Strings, Text Blocks, and Unicode

15.1 Learning objectives

By the end of this chapter, you should be able to:

- model Java arrays as fixed-length, reified reference objects;
- distinguish shallow copying, deep copying, aliasing, and covariance hazards;
- explain `String` immutability, pooling, concatenation, and comparison;
- write and reason about text blocks; and
- process Unicode code points instead of assuming one `char` equals one user-visible character.

WHY IT WORKS | 15.2 Why this matters at SDE-2

Arrays and strings sit on almost every backend boundary: network buffers, database values, identifiers, JSON, logs, and security checks. Errors here become data corruption, accidental quadratic work, or Unicode vulnerabilities. A senior engineer must recognize ownership and encoding assumptions as part of an API contract.

Interviews use arrays and strings heavily because their syntax is simple while their semantics expose fundamentals: references, mutability, indexing, complexity, equality, and representation.

WHY IT WORKS | 15.3 First-principles model

An array is an object with a runtime component type and immutable length. Its elements are mutable unless prevented by external design. A variable such as `int[] values` contains a reference, not the elements themselves. Multidimensional array syntax represents arrays whose elements are array references; rows may have different lengths or be null.

A `String` is an immutable sequence of UTF-16 code units. A `char` is one 16-bit code unit. Some Unicode code points use one code unit, others use a surrogate pair, and a visible grapheme can combine several code points. Therefore `length()`, code-point count, and what a user sees are different concepts.

A text block is source syntax for a `String`. It improves multiline readability but does not introduce a different runtime type.

Specification boundary: Java specifies array bounds checks, runtime store checks, `String` behavior, and UTF-16-based APIs. It does not guarantee a particular internal `String` field layout or byte encoding used inside a JVM implementation.

15.4 Core terminology

- **Component type:** type of an array's elements.
- **Reified:** runtime retains enough component-type information to check array stores.
- **Covariance:** `Sub[]` is a subtype of `Super[]` when `Sub` is a subtype of `Super`.
- **Aliasing:** multiple references reach the same mutable object.
- **Shallow copy:** copies top-level values or references, not referenced nested objects.
- **Interning:** canonicalizing equal strings in a JVM-managed pool.
- **Code unit:** unit used by an encoding; Java `char` is a UTF-16 code unit.
- **Code point:** Unicode numeric value such as U+1F680.
- **Grapheme cluster:** sequence perceived as one user-visible character.
- **Incidental indentation:** whitespace text blocks remove based on delimiter and content placement.

15.5 Detailed mechanics

Array creation and initialization

JAVA

```
int[] counts = new int[4];           // [0, 0, 0, 0]
String[] names = {"Ada", "Linus"};
int[][] matrix = new int[3][];
matrix[0] = new int[] {1, 2};
matrix[1] = new int[] {3};
// matrix[2] remains null
```

Every array element is default-initialized. Length is available through the final-like `length` field and cannot change. Indexes range from zero through `length - 1`; any other index throws `ArrayIndexOutOfBoundsException`.

Array declarations allow brackets after the type or name, but placing them with the type avoids mixed declarations. Generic array creation such as `new List<String>[10]` is illegal because erased generic element types cannot support the array's runtime store check.

Covariance is checked dynamically:

JAVA

```
Number[] numbers = new Integer[2];
numbers[0] = 10;
// numbers[1] = 2.5; // ArrayStoreException
```

The variable permits a `Double` by its static component type, but the actual object is `Integer[]`, so the store fails. Generic collections use invariant static checking and usually make this class of error impossible.

Copying and comparison

Assignment copies an array reference. `clone`, `Arrays.copyOf`, and `System.arraycopy` create or populate top-level arrays, but nested references remain shared.

JAVA

```
int[][] original = {{1, 2}, {3, 4}};
int[][] shallow = original.clone();
shallow[0][0] = 99;
System.out.println(original[0][0]); // 99
```

Arrays inherit identity-based `equals` and `hashCode` from `Object`. Use `Arrays.equals` for one-dimensional content, `Arrays.deepEquals` for nested arrays, and matching hash helpers. Printing an array directly does not format its elements; use `Arrays.toString` or `deepToString`.

String identity, value, and pool

String literals and constant string expressions are interned. Runtime-created strings need not share identity.

JAVA

```
String a = "java";
String b = "ja" + "va";           // compile-time constant
String part = "ja";
String c = part + "va";           // runtime concatenation
System.out.println(a == b);       // true
System.out.println(a == c);       // generally false; do not rely on identity
System.out.println(a.equals(c));  // true
```

Use `equals` for exact content and `equalsIgnoreCase` only when its locale-independent comparison matches the domain. For natural-language case conversion, specify a `Locale`. For protocol identifiers, often use `Locale.ROOT`. Unicode normalization is separate from case folding; visually equivalent strings can have different code-point sequences.

Because strings are immutable, operations such as `substring`, `replace`, and `toUpperCase` return a string and do not modify the receiver. `StringBuilder` provides a mutable sequence for single-threaded assembly. `StringBuffer` synchronizes individual operations but rarely solves a higher-level concurrency design.

Modern compilers use `invokedynamic`-based concatenation strategies for many `+` expressions, so simple one-shot concatenation is readable and efficient. Repeated `result += item` in a loop can still create work proportional to all accumulated content; use a builder or joining collector.

Text blocks

Text blocks became permanent in Java 15 and are available in Java 17 and 21. The opening `"""` must be followed by a line terminator. The compiler removes incidental indentation, normalizes line terminators, and processes escape sequences.

JAVA

```
String json = """
    {
        "name": "Ada",
        "active": true
    }
    """;
```

This value normally ends with a newline because content ends before the closing delimiter. Place the closing delimiter immediately after content, or use a line-continuation escape, when no terminal newline is desired. The `\s` escape preserves a space explicitly. `String.stripIndent()` and `translateEscapes()` expose related runtime transformations, but a compiled text block has already undergone compiler processing.

Never use text blocks as a substitute for SQL parameter binding or HTML/JSON escaping. They improve source presentation, not data safety.

Unicode processing

`String.length()` returns UTF-16 code units. `codePointCount` counts Unicode code points. Iterate safely with code-point-aware methods:

JAVA

```
String value = "A\uD83D\uDE80"; // A plus rocket
System.out.println(value.length()); // 3 code units
System.out.println(value.codePointCount(0, value.length())); // 2

value.codePoints().forEach(cp ->
    System.out.printf("U+%04X%n", cp));
```

`codePointAt` combines a valid surrogate pair. If input contains an unpaired surrogate, it returns that surrogate value; Java strings can contain ill-formed UTF-16 sequences. Encoders must decide whether to replace, report, or ignore malformed input.

Unicode escapes such as `\u000A` are processed very early in Java source translation, even in comments. A Unicode escape that becomes a line terminator can change tokenization or make source illegal. Prefer ordinary escapes like `\n` for controls inside literals.

TRACE IT | 15.6 Worked Java example

This function reverses code points rather than UTF-16 code units and preserves supplementary characters.

JAVA

```
public class UnicodeReverse {
    static String reverseCodePoints(String input) {
        int[] points = input.codePoints().toArray();
        StringBuilder result = new StringBuilder(input.length());
        for (int i = points.length - 1; i >= 0; i--) {
            result.appendCodePoint(points[i]);
        }
        return result.toString();
    }

    public static void main(String[] args) {
        String input = "A\uD83D\uDE80B";
        System.out.println(reverseCodePoints(input)); // B, rocket, A
    }
}
```

This is code-point correct, not grapheme-cluster correct. Reversing combining marks separately can still produce surprising visible output. User-perceived text segmentation requires a boundary algorithm suitable for the Unicode version and locale.

TRACE IT | 15.7 Execution or memory walkthrough

The input contains four UTF-16 code units: **A**, a high surrogate, a low surrogate, and **B**. `codePoints()` combines the surrogate pair and emits three `int` values. `toArray` allocates an `int[3]`.

The loop begins at index two and appends **B**. It then calls `appendCodePoint` for the rocket value, which emits two UTF-16 code units, followed by **A**. `toString` creates the immutable result. The original string is unchanged.

No bounds error occurs because the loop starts at `points.length - 1`, continues while nonnegative, and decrements. For an empty input it starts at -1 and performs zero iterations.

15.8 Complexity and performance

Array indexing is $O(1)$; traversal and copying are $O(n)$. `System.arraycopy` is still $O(n)$, though JVMs optimize its constant factors. A rectangular r by c traversal is $O(rc)$; a jagged traversal is proportional to actual elements.

Most string searches and transformations are $O(n)$ in code units, with algorithm-specific exceptions. Concatenating into an immutable accumulator inside a loop can be $O(n^2)$ in total copied content. A pre-sized `StringBuilder` usually makes assembly $O(n)$.

The reverse example is $O(n)$ time and $O(n)$ additional space. A more intricate backward code-point traversal can avoid the `int[]`, but clarity is often worth the allocation unless profiling identifies the path.

HotSpot note: Current HotSpot implementations may store strings compactly using a byte array plus an encoding marker when contents permit. This is not a public `String` contract and should not influence correctness assumptions.

WATCH OUT | 15.9 Edge cases and common mistakes

- Array assignment aliases; it does not copy elements.
- `clone` on a nested or object array is shallow.
- Covariant array stores can fail at runtime.
- `Arrays.asList(primitiveArray)` creates a one-element list containing that array, not a list of boxed primitives.
- `array.length`, `string.length()`, and `collection.size()` use different syntax.
- `==` compares string references; `equals` compares content.
- Calling `toString` on a null reference fails; `String.valueOf` renders it as `"null"`, which may hide missing data.
- Splitting on a regex metacharacter requires regex escaping, and `split` discards trailing empty strings by default unless a negative limit is used.
- A supplementary code point occupies two `char` values.
- Code-point correctness does not guarantee grapheme or locale correctness.
- Default-charset conversions vary by environment; specify `StandardCharsets.UTF_8` at byte boundaries.
- Text blocks still process escapes and normally include a trailing newline based on delimiter placement.

15.10 Production engineering notes

Specify ownership for arrays and mutable builders. Defensive-copy untrusted mutable input when retaining it, and do not expose internal arrays directly. For secrets, a mutable `char[]` can be cleared, but copies may still exist in libraries and memory; do not overstate that protection.

Define charset at every byte/text boundary. Validate malformed input intentionally. Normalize only when the domain requires it, and decide where canonicalization occurs so database keys, caches, signatures, and authorization checks agree.

Use locale-aware libraries for human language and locale-neutral rules for protocols. Length limits must state whether they mean bytes, UTF-16 units, code points, or grapheme clusters. Database column limits and API validators often count differently.

TRY IT | 15.11 Interview questions and model answers

Why are arrays covariant while generics are invariant?

Arrays predate generics and retain runtime component types, so Java permits covariance and protects stores dynamically. Generic type arguments are normally erased and checked statically; invariance prevents inserting the wrong subtype without waiting for a runtime failure.

Does `String.length()` count characters?

It counts UTF-16 code units. It may differ from Unicode code points and from user-perceived grapheme clusters. The correct metric depends on the requirement.

Why is `String` immutable?

Immutability enables safe sharing, stable hashing, pooling, and simpler security boundaries. It does not mean every string operation is allocation-free, and sensitive values can remain in memory beyond application control.

What does a text block produce?

An ordinary `String`. The compiler determines content through indentation removal, line-ending normalization, and escape processing. Delimiter placement controls whether a final newline remains.

Is `StringBuilder` always required for concatenation?

No. One expression using `+` is clear and compilers optimize it effectively. Use a builder when incrementally accumulating in loops or when capacity and append behavior need explicit control.

TRY IT | 15.12 Exercises

- 1 Deep-copy a jagged `int[][]` while preserving null rows.
- 2 Demonstrate an `ArrayStoreException`, then replace the API with a generic collection.
- 3 Write tests showing trailing-newline and indentation behavior for three text blocks.
- 4 Count code units, code points, and expected grapheme clusters in strings containing an emoji and a combining mark.
- 5 Implement UTF-8 encode/decode with a `CharsetEncoder` configured to report malformed input.
- 6 Replace quadratic loop concatenation with a pre-sized `StringBuilder` and justify the capacity estimate.

RECAP | 15.13 Chapter summary

Arrays are fixed-length, reified, mutable objects; assignment aliases them, covariance can defer type errors, and top-level copy operations are shallow for reference elements. Strings are immutable UTF-16 code-unit sequences and require value comparison. Text blocks are readable source syntax for ordinary strings. Correct text handling separates bytes, code units, code points, graphemes, normalization, locale, and user-visible length.

TRY IT | 15.14 Revision checklist

- ☐ I can explain array reification, covariance, and store checks.
- ☐ I distinguish aliasing, shallow copying, and deep copying.
- ☐ I use content helpers to compare and print arrays.
- ☐ I compare strings by value and understand interning boundaries.
- ☐ I know when repeated concatenation becomes expensive.
- ☐ I can predict text-block indentation and final-newline behavior.
- ☐ I distinguish UTF-8 bytes, UTF-16 units, code points, and graphemes.
- ☐ I specify charset, locale, normalization, and length units at boundaries.

SDE-2 CORE CHAPTER

Chapter 16 - Classes, Objects, Access Control, and Packages

16.1 Learning objectives

By the end of this chapter, you should be able to:

- separate class definitions, objects, references, identity, and state;
- predict field, initializer, and constructor execution order;
- apply `public`, `protected`, package, and `private` access precisely;
- use static and instance members without confusing ownership; and
- design cohesive classes and package boundaries that preserve invariants.

WHY IT WORKS | 16.2 Why this matters at SDE-2

Backend design lives in classes and package boundaries. Weak encapsulation lets invalid states spread, couples services to implementation details, and makes concurrent or evolutionary changes dangerous. Strong modeling converts business rules into construction and method contracts.

Java access control also contains interview-worthy subtleties, especially `protected` across packages and the difference between class visibility and member visibility. Experienced engineers should reason about these rules rather than repeatedly widening access until code compiles.

WHY IT WORKS | 16.3 First-principles model

A class declares a reference type and defines members: fields, methods, constructors, initializers, and nested types. An object is a runtime class instance with identity and field state. A reference value can designate that object or be `null`. Multiple references may designate one object.

Instance members belong conceptually to each object; static members belong to the class's shared namespace and initialization lifecycle. Encapsulation means an object controls how its valid state is created and changed. Access modifiers are compiler- and runtime-enforced visibility rules, not a complete security boundary.

A package groups types and contributes to access control and naming. `com.example.api` and `com.example.api.internal` are distinct packages; package names do not create inheritance of access.

Specification boundary: The Java language specifies initialization order and access checks. It does not promise a field's physical offset, an object's header format, or one object allocation per `new` expression after optimization, provided observable behavior is preserved.

16.4 Core terminology

- **Class:** declaration of a reference type and its members.
- **Object:** runtime instance of a class.
- **Identity:** distinction between object instances even when their content is equal.
- **State:** values reachable through an object's instance fields.
- **Invariant:** condition that valid instances maintain.
- **Constructor:** special declaration that initializes a newly allocated instance.
- **Initializer:** static or instance block executed during class or object initialization.
- **Receiver:** object denoted by `this` for an instance method call.
- **Package-private:** access when no modifier is written.
- **Qualified name:** package plus type name, such as `java.time.Instant`.

16.5 Detailed mechanics

Fields, methods, and receivers

JAVA

```
package example.account;

public final class Counter {
    private static int instances;
    private int value;

    public Counter(int initialValue) {
        this.value = initialValue;
        instances++;
    }

    public int increment() {
        return ++value;
    }

    public static int instancesCreated() {
        return instances;
    }
}
```

Each `Counter` has a separate `value`, while all calls observe one static `instances` field per defining class loader. `this` is the current receiver and is unavailable in a static context. A static method should be called through the class name; calling it through an expression is legal in some forms but misleading because dispatch is not based on that object's runtime class.

The example's shared count is not thread-safe. `++` is a read-modify-write sequence, and `private` says nothing about concurrency.

Construction and initialization order

Object creation conceptually allocates storage, initializes all instance fields to defaults, invokes a selected constructor, and returns a reference if construction completes. Before a constructor body runs, it invokes another constructor in the same class with `this(...)` or a superclass constructor with `super(...)`; if neither is explicit, the compiler inserts an accessible no-argument `super()`.

For each class in the hierarchy, instance field initializers and instance initializer blocks run in textual order after the superclass portion and before that class's constructor body.

JAVA

```
class Base {
    Base() { System.out.println("Base body"); }
}

class Report extends Base {
    private final String title = initializeTitle();
    { System.out.println("Report initializer"); }

    Report() {
        System.out.println("Report body");
    }

    private String initializeTitle() {
        System.out.println("title initializer");
        return "daily";
    }
}
```

Creating `Report` prints Base body, title initializer, Report initializer, then Report body. Calling an overridable instance method from a constructor is dangerous: dynamic dispatch can reach subclass code before subclass fields have been initialized.

Constructors have no return type and are not inherited. If a class declares no constructor, the compiler supplies a default constructor whose accessibility matches the class. Once any constructor is declared, no default is generated.

Static initialization

Static fields first receive defaults, then static field initializers and static initializer blocks execute in textual order when the class is initialized. Initialization occurs at most once per class or interface per defining class loader, synchronized by the JVM. A failure can leave the class erroneous for subsequent use.

Compile-time constant static fields may be inlined into clients and accessed without triggering initialization of the declaring class. This matters for side effects and binary evolution.

Access control

At top level, a class may be **public** or package-private. A public top-level class is conventionally, and under ordinary file-based compilation, stored in a same-named source file.

Member access proceeds from narrowest to broadest:

- **private**: within the top-level nest that declares it, including its nested nestmates under language access rules;
- package-private: within the same package;
- **protected**: same-package access plus subclass access under an additional cross-package receiver rule;
- **public**: wherever the declaring type and module/package exposure permit.

The cross-package **protected** rule is commonly misunderstood. Subclass code may access the inherited protected member through **this**, **super**, or a reference whose qualifying type is the subclass or its subclass, not through an arbitrary base-class instance.

JAVA

```
// package library;
public class Base { protected int code; }

// package app;
class Derived extends library.Base {
    void ok(Derived other) { other.code = 1; }
    // void bad(library.Base other) { other.code = 1; }
}
```

private members are not inherited as directly accessible members, although an object still contains superclass state and superclass methods can use it. Modern Java nest-based access allows nested classes in the same nest to access one another's private members according to language rules without treating private as public.

Packages and imports

A package declaration must precede imports and type declarations, aside from comments and whitespace. Imports shorten names at compile time; they do not load classes or add runtime dependencies by themselves. **java.lang** is implicitly imported, as are types in the current package. Wildcard imports import types from one package, not subpackages, and do not harm runtime performance.

Static imports can shorten constants or utility calls but may obscure origin. Avoid the unnamed package for production code because named-package code cannot import from it and tooling conventions assume named packages.

Java Platform Module System exports can further restrict whether a public package is accessible to another module. Reflection may also require an **opens** directive; public at the language level is not necessarily globally reflectable in a modular runtime.

TRACE IT | 16.6 Worked Java example

This class establishes a nonnegative balance invariant and returns new immutable values instead of exposing mutation.

JAVA

```
package example.money;

import java.util.Objects;

public final class Wallet {
    private final String ownerId;
    private final long cents;

    private Wallet(String ownerId, long cents) {
        this.ownerId = Objects.requireNonNull(ownerId, "ownerId");
        if (ownerId.isBlank()) {
            throw new IllegalArgumentException("blank ownerId");
        }
        if (cents < 0) {
            throw new IllegalArgumentException("negative balance");
        }
        this.cents = cents;
    }

    public static Wallet empty(String ownerId) {
        return new Wallet(ownerId, 0);
    }

    public Wallet deposit(long amountCents) {
        if (amountCents <= 0) {
            throw new IllegalArgumentException("deposit must be positive");
        }
        return new Wallet(ownerId, Math.addExact(cents, amountCents));
    }

    public long balanceCents() {
        return cents;
    }
}
```

The constructor is private so all construction flows remain under the class's control. A factory names the initial state. Final fields plus immutable field values make each instance safe to share after proper construction.

TRACE IT | 16.7 Execution or memory walkthrough

`Wallet.empty("u-7")` invokes a static method; no receiver object is required. `new Wallet` allocates a new instance whose reference fields initially hold null and whose `long` field holds zero. The constructor validates the copied `ownerId` reference, then assigns both final fields. It returns normally and the factory returns the reference.

Calling `wallet.deposit(500)` supplies the original wallet as receiver and copies `500L` into the parameter. The existing object is not modified. `addExact` computes the new balance and construction validates a new wallet. The caller may retain both versions without aliasing mutable state.

If validation throws, no reference to the incompletely constructed object is returned by the expression. Publishing `this` from a constructor, however, could allow another component to observe incomplete state and must be avoided.

16.8 Complexity and performance

Field access and ordinary method dispatch are $O(1)$, excluding method work. Object construction is generally proportional to initialization and reachable defensive copies, not merely the number of fields. The wallet operations are $O(1)$ time and allocate one new object per successful state change.

Immutability can increase allocation but enables safe sharing, caching, and simpler concurrency. A JIT may eliminate non-escaping allocations. Do not replace clear value semantics with mutable pooling unless profiling demonstrates a meaningful bottleneck and ownership remains tractable.

HotSpot note: HotSpot object headers, compressed references, field layout, escape analysis, and allocation in thread-local buffers are implementation choices. Use measurement tools when physical footprint matters; do not derive it only from field widths.

WATCH OUT | 16.9 Edge cases and common mistakes

- A reference declaration does not create an object.
- `final` on a reference prevents reassignment, not mutation of the object.
- The generated default constructor disappears after any explicit constructor is declared.
- A superclass must be initialized before subclass field initializers run.
- Overridable calls from constructors can observe default-valued subclass fields.
- Leaking `this` during construction can violate invariants and safe publication.
- `private` does not imply thread-safe, immutable, or inaccessible through all reflection configurations.
- Package hierarchy is naming convention, not access inheritance.
- `protected` is not simply package-or-world-visible-to-subclasses; the cross-package qualifying reference matters.
- A public member is unusable if its declaring type is inaccessible.
- Static mutable state is global per class loader, complicates tests, and needs concurrency control.
- Static initialization cycles can expose default values and produce fragile startup failures.

16.10 Production engineering notes

Organize packages around stable domain capabilities, not arbitrary technical buckets alone. Expose a small public surface and keep implementation types package-private. If tests need broad access, prefer testing through behavior or narrowly scoped test support over making internals public.

Construct valid objects atomically. Validate required state, use factories or builders when construction has meaningful variants, and avoid setters that create transiently invalid combinations. Keep dependency injection and serialization frameworks from bypassing invariants without explicit adapters.

Treat static state as process-wide infrastructure. Make it immutable where possible; otherwise define lifecycle, synchronization, reset behavior, and class-loader implications. Static constants that may change across versions should not be compile-time constants if clients must observe updates without recompilation.

TRY IT | 16.11 Interview questions and model answers

What is the difference between a class, an object, and a reference?

A class declares a type and behavior. An object is a runtime instance with identity and state. A reference is a value that designates an object or is null; several references can designate one object.

What is the object initialization order?

Fields receive defaults first. Constructor chaining initializes the superclass portion. For the current class, instance field initializers and instance initializer blocks run in textual order, then its constructor body runs. This repeats down the hierarchy.

How does protected access work across packages?

A subclass may access the protected member as part of its inherited view, but through a qualifying expression whose type is that subclass or a subtype, not an arbitrary instance of the superclass. Same-package code has ordinary package access to it.

Is a private constructor only for singletons?

No. It can enforce factories, canonical instances, validated creation, utility classes, or controlled subclassing. A singleton also needs lifecycle, concurrency, serialization, and testability decisions.

Does importing a type load it?

No. An import is a compile-time name-resolution convenience. Loading and initialization follow runtime use rules, not the presence of an import.

TRY IT | 16.12 Exercises

- 1 Predict the print order for a three-level hierarchy with static fields, instance fields, initializer blocks, and constructor chaining.
- 2 Build a class that cannot represent an invalid date range and has no setters.
- 3 Create two packages demonstrating legal and illegal protected access through differently typed references.
- 4 Refactor a public implementation class into a package-private type behind a public interface.
- 5 Identify all ways `this` could escape from a constructor through callbacks, collections, or threads.
- 6 Make the `Counter` instance count thread-safe, then discuss whether the global metric belongs in the domain class.

RECAP | 16.13 Chapter summary

Classes define types; objects carry identity and state; references designate objects. Construction initializes superclass state before subclass initializers and bodies. Access control combines member modifiers, declaring-type accessibility, packages, subclass rules, and possibly modules. Good class design centralizes invariants, limits public surface, avoids construction leaks, and treats static mutable state as an explicit architectural concern.

TRY IT | 16.14 Revision checklist

- ☐ I distinguish a class, object, reference, and receiver.
- ☐ I can trace static and instance initialization order.
- ☐ I know when the compiler supplies a default constructor.
- ☐ I apply all four member access levels precisely.
- ☐ I can explain the cross-package protected receiver rule.
- ☐ I know that packages and subpackages have no special access relationship.
- ☐ I avoid overridable calls and `this` escape during construction.
- ☐ I design classes whose constructors and methods preserve invariants.

SDE-2 CORE CHAPTER

Chapter 17 - Inheritance, Polymorphism, and Composition

17.1 Learning objectives

By the end of this chapter, you should be able to:

- distinguish subtype polymorphism from code reuse and object containment;
- predict overriding, hiding, field access, casts, and constructor behavior;
- apply substitutability when designing and reviewing hierarchies;
- choose composition or inheritance based on semantic ownership; and
- evolve polymorphic APIs without fragile downcasts or superclass coupling.

WHY IT WORKS | 17.2 Why this matters at SDE-2

SDE-2 design discussions often reduce to one question: where should variation live? A hierarchy can make variation explicit and open to new behavior, but a weak hierarchy spreads superclass assumptions throughout the system. Composition localizes collaboration and runtime configuration, but excessive forwarding can obscure the domain.

Interviewers expect more than definitions of "is-a" and "has-a." They look for dispatch reasoning, substitutability, constructor hazards, and pragmatic trade-offs in service and library design.

WHY IT WORKS | 17.3 First-principles model

Class inheritance creates a subtype relationship and allows a subclass object to contain a superclass portion. A variable of a supertype can hold a reference to a subtype object. When an overridable instance method is invoked, the object's runtime class chooses the implementation. This is subtype polymorphism.

Inheritance is also a reuse mechanism, but reuse alone is not sufficient justification. A subtype promises that code written against its supertype remains correct. It must preserve the supertype's stated preconditions, postconditions, invariants, and behavioral expectations.

Composition means one object holds or receives another object and delegates part of its work. It creates collaboration without claiming substitutability. Dependencies can often be replaced, combined, or decorated at runtime.

Specification boundary: Java specifies single class inheritance, multiple interface inheritance, method overriding, and dynamic dispatch. Object layout, virtual method tables, inline caches, and devirtualization are JVM implementation strategies, not Java language guarantees.

17.4 Core terminology

- **Superclass/subclass:** class being extended and class that extends it.
- **Subtype:** type whose values can be used where a supertype is expected.
- **Upcast:** safe conversion from subtype reference to supertype.
- **Downcast:** checked conversion from supertype reference to a more specific type.
- **Override:** instance method implementation replacing inherited behavior for dispatch.
- **Hide:** static method or field declaration shadows an inherited same-named member.
- **Dynamic dispatch:** runtime selection of an overriding instance method.
- **Substitutability:** subtype preserves contracts expected through the supertype.
- **Composition:** object delegates to contained or injected collaborators.
- **Fragile base class:** superclass changes unexpectedly affect subclasses coupled to internals.

17.5 Detailed mechanics

Extending classes

Every class except `Object` has one direct superclass. If `extends` is omitted, it extends `Object`. Constructors are not inherited, and subclass construction must invoke an accessible superclass constructor first.

Inherited accessible members become part of the subclass's behavior. Private superclass state remains present but can be manipulated only through accessible superclass operations. A class declared `final` cannot be subclassed. A method declared `final` cannot be overridden.

Overriding rules

An overriding method has the same subsignature, a compatible covariant reference return, no more restrictive access, and no broader checked exception declaration. `@Override` asks the compiler to verify intent and should be used consistently.

JAVA

```
class Message {
    CharSequence render() { return "message"; }
}

class Alert extends Message {
    @Override
    String render() { return "alert"; } // covariant return
}
```

Private methods are not overridden. Static methods are hidden and selected by compile-time type. Fields are hidden rather than polymorphic. Instance method calls dispatch dynamically even when called from a superclass method, except for private, static, constructor, or explicit `super` behavior.

JAVA

```
class Base {
    String name = "base field";
    static String kind() { return "base static"; }
    String describe() { return "base virtual"; }
}

class Derived extends Base {
    String name = "derived field";
    static String kind() { return "derived static"; }
    @Override String describe() { return "derived virtual"; }
}

Base value = new Derived();
System.out.println(value.name);           // base field
System.out.println(Base.kind());          // base static
System.out.println(value.describe());      // derived virtual
```

Avoid same-named fields and static hiding because they invite mistaken dispatch assumptions.

super and construction

`super(...)` invokes a direct superclass constructor and must be the first constructor invocation statement, unless another same-class constructor is invoked with `this(...)`. `super.method()` invokes the superclass implementation directly for the current object; it is not a call on a separate superclass object.

During base construction, virtual calls can dispatch to the subclass before subclass initializers run. The result may observe null or zero state and violates the expectation that methods see a fully formed object. Constructors should call private or final helpers when they need internal decomposition.

Casts and instanceof

An upcast needs no explicit syntax and never fails for a non-null compatible reference. A downcast asks the runtime whether the object is an instance of the target type; otherwise it throws `ClassCastException`. Casting null succeeds and produces null.

JAVA

```
Object candidate = "java";
if (candidate instanceof String text) {
    System.out.println(text.length());
}
```

Pattern variables are in scope where the compiler proves the match succeeded. A design requiring frequent type tests may be missing a polymorphic operation, although boundary adapters and closed algebraic models legitimately inspect variants.

Substitutability

Suppose a base method accepts any integer and promises a nonnegative result. An override cannot reject negative inputs or return negative results without breaking callers. More generally, a subtype should not strengthen preconditions, weaken postconditions, violate invariants, or change important side effects and timing without the contract allowing it.

The classic rectangle/square modeling problem demonstrates that mathematical subset relationships do not automatically make good mutable object subtypes. If a mutable rectangle allows width and height to change independently, a square cannot preserve both that behavior and its equal-sides invariant.

Composition and delegation

Composition represents roles explicitly:

JAVA

```
interface DiscountPolicy {
    long discountCents(long subtotalCents);
}

final class Checkout {
    private final DiscountPolicy policy;

    Checkout(DiscountPolicy policy) {
        this.policy = java.util.Objects.requireNonNull(policy);
    }

    long totalCents(long subtotalCents) {
        long discount = policy.discountCents(subtotalCents);
        if (discount < 0 || discount > subtotalCents) {
            throw new IllegalStateException("invalid discount");
        }
        return subtotalCents - discount;
    }
}
```

Checkout does not claim to be a discount policy. It owns orchestration and validates the collaborator's result. Different policies can be injected without subclasses inheriting checkout internals.

TRACE IT | 17.6 Worked Java example

This example combines subtype polymorphism at a narrow interface with composition in the service.

JAVA (continued 1/2)

```
import java.util.List;
import java.util.Objects;

public class NotificationDemo {
    interface Channel {
        Delivery send(String recipient, String body);
    }

    record Delivery(boolean accepted, String providerId) {}

    static final class EmailChannel implements Channel {
        @Override
        public Delivery send(String recipient, String body) {
            return new Delivery(true, "email-42");
        }
    }

    static final class Notifier {
        private final Channel channel;
```

JAVA (continued 2/2)

```
        Notifier(Channel channel) {
            this.channel = Objects.requireNonNull(channel);
        }

        Delivery notify(String recipient, List<String> lines) {
            String body = String.join(System.lineSeparator(), lines);
            if (body.isBlank()) {
                throw new IllegalArgumentException("empty body");
            }
            return channel.send(recipient, body);
        }
    }

    public static void main(String[] args) {
        Channel channel = new EmailChannel();
        Notifier notifier = new Notifier(channel);
        System.out.println(notifier.notify("a@example.test", List.of("Hello")));
    }
}
```

Notifier depends on the role it needs, not the concrete provider. Its validation and message construction stay independent from channel implementations.

TRACE IT | 17.7 Execution or memory walkthrough

`new EmailChannel()` creates an object referenced through the `Channel` interface. This upcast loses no object information; it narrows which operations are visible at compile time. `Notifier` stores a copied reference to the same channel object.

During `notify`, `String.join` builds the body. The service validates it, then invokes the interface signature `Channel.send`. Runtime dispatch selects `EmailChannel.send`. That method creates and returns a `Delivery` record. No downcast is required anywhere.

If another implementation is injected, the service bytecode still targets the interface operation. Only runtime dispatch changes. The interface contract must therefore specify acceptance meaning, null behavior, failures, and whether retries are safe.

17.8 Complexity and performance

Inheritance and composition do not determine algorithmic complexity. Dynamic dispatch and delegation each add conceptually constant overhead. In the example, joining lines takes $O(n)$ time in total character content and $O(n)$ output space; channel work dominates the remaining cost.

Deep inheritance raises cognitive cost even when runtime cost is small. Each override may require understanding implicit superclass state and hooks. Composition adds objects and calls but makes dependency graphs and test seams explicit.

HotSpot note: HotSpot often inlines interface and virtual calls when profiling shows one or a few receiver classes. It can deoptimize if assumptions change. Do not mark classes final solely for speculative call speed without measurement.

WATCH OUT | 17.9 Edge cases and common mistakes

- Overloading is not overriding; different parameter types create a new overload.
- Fields and static methods do not dispatch on runtime type.
- A downcast changes the reference's static view, not the object, and may fail.
- `instanceof` returns false for null.
- An override cannot reduce accessibility or broaden checked exceptions.
- Calling overridable methods during construction can reach uninitialized subclass state.
- Extending a concrete collection merely to reuse methods often exposes a larger, misleading contract.
- Inheriting from a class with overridable internal hooks creates fragile coupling to call order.
- A base class with `equals` rules that subclasses cannot preserve can break symmetry or transitivity.
- Composition still needs ownership decisions: whether collaborators are thread-safe, mutable, shared, or lifecycle-managed.
- A decorator must preserve the wrapped contract, including exceptions and identity expectations, not only method names.

17.10 Production engineering notes

Use inheritance for a genuine, stable subtype relationship with a documented extension contract. Prefer narrow interfaces for roles and composition for configurable behavior, infrastructure clients, and policies. If subclasses are expected, document which methods are hooks, when they run, and what state is valid.

Keep base classes small and protect invariants with private state and final template methods. Avoid exposing protected mutable fields. A protected operation can offer controlled extension without handing subclasses the representation.

Downcasts at system boundaries should fail with useful diagnostics. Within core domain logic, repeated `instanceof` chains often indicate behavior living in the wrong place. For a fixed set of variants, sealed types and pattern matching may make inspection intentional and exhaustive.

TRY IT | 17.11 Interview questions and model answers

What is runtime polymorphism in Java?

A call to an overridable instance method is dispatched according to the receiver object's runtime class. The compiler has already selected the method signature using static types; runtime chooses the most specific override of that signature.

Why prefer composition over inheritance?

Composition avoids claiming substitutability, limits coupling to a collaborator's public contract, allows runtime replacement, and does not expose superclass hooks or state. Inheritance remains appropriate when values truly are subtypes and the base is designed for extension.

What is the difference between overriding and hiding?

Instance methods override and dispatch dynamically. Static methods and fields can be hidden; access is determined from the compile-time qualifying type. Same-named fields are separate state.

What makes a subtype substitutable?

It honors the supertype's behavioral contract: it accepts at least the promised inputs, returns results meeting the promised postconditions, maintains invariants, and preserves documented failure and side-effect expectations.

Why are constructor calls to overridable methods unsafe?

Dynamic dispatch reaches the subclass implementation before the subclass's field initializers and constructor body have run. The method can observe default state or leak the incomplete object.

TRY IT | 17.12 Exercises

- 1 Predict field, static method, and instance method outputs through base and derived references.
- 2 Design a subclass that violates a base precondition contract, then refactor it into composition.
- 3 Replace a three-branch `instanceof` chain with a polymorphic operation.
- 4 Implement a timing decorator for `Channel` that preserves exceptions and return values.
- 5 Analyze whether `Stack` extends `ArrayList` would be substitutable and identify inherited operations that violate stack intent.
- 6 Write a base constructor that accidentally dispatches to subclass state, demonstrate the failure, and repair it with a private helper.

RECAP | 17.13 Chapter summary

Inheritance creates a subtype promise, not merely a reuse opportunity. Compile-time member selection and runtime overriding must be considered separately: fields and static methods hide, while overridable instance methods dispatch. Subtypes must preserve contracts. Composition expresses collaboration without exposing superclass internals and is usually safer for configurable behavior. Choose the relationship that tells the domain truth and keeps invariants local.

TRY IT | 17.14 Revision checklist

- ☐ I distinguish subtype polymorphism from implementation reuse.
- ☐ I can trace overriding, field hiding, and static hiding.
- ☐ I understand upcasts, downcasts, and pattern `instanceof`.
- ☐ I can state substitutability in terms of behavioral contracts.
- ☐ I avoid virtual calls from constructors.
- ☐ I recognize fragile base-class coupling.
- ☐ I choose composition for roles and runtime-configurable policies.
- ☐ I document extension points and collaborator ownership explicitly.

SDE-2 CORE CHAPTER

Chapter 18 - Interfaces, Abstract Classes, Sealed Types, and Pattern Matching

18.1 Learning objectives

By the end of this chapter, you should be able to:

- choose among interfaces, abstract classes, sealed hierarchies, and composition;
- resolve default-method inheritance and understand interface member rules;
- design and enforce a closed subtype family with `sealed`, `non-sealed`, and `final`;
- use Java 21 type patterns, record patterns, and pattern switch exhaustively; and
- identify which pattern features were preview versus permanent in Java 17 and 21.

WHY IT WORKS | 18.2 Why this matters at SDE-2

Backend systems repeatedly model variants: payment results, commands, events, failures, and protocol messages. An open interface supports third-party implementations; a sealed algebra supports a known set of cases. Choosing incorrectly either blocks extension or makes exhaustive reasoning impossible.

Java 21 pattern matching makes closed models concise, but concision is not the primary benefit. The real gain is aligning domain completeness with compiler checks. SDE-2 interviews increasingly test this modern type-system reasoning while still expecting knowledge of default methods and abstract base classes.

WHY IT WORKS | 18.3 First-principles model

An interface specifies a reference type whose instances are objects of implementing classes. It supports multiple inheritance of type and behavior but not per-instance interface state. An abstract class is a partially implemented class with constructors and instance fields, and a subclass can extend only one class.

A sealed type restricts which classes or interfaces may directly extend or implement it. Each permitted direct subtype must explicitly continue the policy: `final` closes it, `sealed` declares another restricted level, or `non-sealed` reopens it.

A pattern combines a test with conditional variable extraction. Pattern matching does not bypass the type system; it moves a cast and its proof into one construct. Exhaustive pattern switch is especially valuable when the selector is a closed hierarchy.

Specification boundary: Exhaustiveness, dominance, permitted subclasses, and pattern-variable scope are compile-time language rules. The JVM also records permitted subclasses in class-file metadata and rejects unauthorized direct extension; it does not guarantee that a sealed hierarchy has only a fixed number of runtime object instances.

18.4 Core terminology

- **Interface:** contract type supporting multiple implementation inheritance.
- **Default method:** interface instance method with an implementation.
- **Abstract class:** non-instantiable class that may mix abstract behavior, state, and implementation.
- **Sealed type:** type with an enumerated set of permitted direct subtypes.
- **Non-sealed type:** permitted subtype that reopens unrestricted inheritance.
- **Pattern:** test that conditionally binds variables from a value.
- **Type pattern:** tests a runtime type and binds the narrowed value.
- **Record pattern:** deconstructs record components recursively.
- **Dominance:** an earlier case makes a later case unreachable.
- **Exhaustiveness:** cases cover every possible selector value allowed by the type rules.

18.5 Detailed mechanics

Interface members and evolution

Interface fields are implicitly `public static final` and must have initializers. Interface methods can be:

- implicitly `public abstract` when declared without a body;
- `public default` instance methods with bodies;
- `public static` methods; or
- `private` instance or static helpers, available since Java 9.

Implementations cannot reduce the accessibility of an interface's public method. Static interface methods are called through the interface and are not inherited as instance behavior. An interface has no constructor and cannot hold per-object instance fields.

Default methods allow compatible interface evolution, but conflicts follow rules:

- 1 a concrete class method wins over an interface default;
- 2 a more specific subinterface default wins over a less specific one; and
- 3 unrelated inherited defaults with the same signature require an explicit override.

JAVA

```
interface JsonForm { default String format() { return "json"; } }
interface TextForm { default String format() { return "text"; } }

final class Output implements JsonForm, TextForm {
    @Override
    public String format() {
        return JsonForm.super.format();
    }
}
```

`InterfaceName.super.method()` resolves a direct superinterface default. An abstract declaration in a more specific interface can also remove an inherited default and force implementers to provide behavior.

Abstract classes

An abstract class may declare abstract methods and concrete members. It can have constructors even though it cannot be instantiated directly; subclass constructors use them to initialize the base portion. A class containing an abstract method must be abstract.

Use an abstract class when implementations share protected invariants, instance state, or a controlled template algorithm. Keep extension contracts explicit and avoid protected mutable fields. Use an interface when the important concept is a role that unrelated classes can implement or when multiple roles must compose.

JAVA

```
abstract class BatchJob {
    public final void run() {
        validate();
        execute();
    }

    protected void validate() {}
    protected abstract void execute();
}
```

The final template fixes sequencing while hooks provide limited variation. It still inherits constructor and fragile-base-class risks, so document whether hooks may throw, block, or mutate shared state.

Sealed hierarchies

Sealed classes and interfaces became permanent in Java 17. A declaration names permitted direct subtypes with `permits`, unless the compiler can infer them from declarations in the same source file.

JAVA

```
sealed interface Command permits Create, Cancel {}
record Create(String id) implements Command {}
record Cancel(String id) implements Command {}
```

Records are implicitly final, so they satisfy the continuation requirement. A permitted ordinary class must declare `final`, `sealed`, or `non-sealed`.

Permitted direct subtypes must be accessible and compiled in the same named module as the sealed type. In an unnamed module, they must be in the same package. Sealing controls direct inheritance, not visibility: a permitted subtype can itself expose behavior according to its modifiers.

Sealed types are ideal for closed domain alternatives maintained together. They are a poor public extension point when independent consumers must add implementations. `non-sealed` deliberately creates an open branch, which can reduce switch exhaustiveness over deeper runtime variants even though the direct branch is known.

Type patterns

Pattern matching for `instanceof` became permanent in Java 16 and is therefore permanent in both Java 17 and 21.

JAVA

```
static int length(Object value) {
    if (value instanceof String text && !text.isEmpty()) {
        return text.length();
    }
    return 0;
}
```

The pattern variable is in scope only where flow analysis proves the match. With `&&`, it is available on the right. With a negated test followed by an abrupt exit, it can be available afterward:

JAVA

```
if (!(value instanceof String text)) {
    throw new IllegalArgumentException();
}
System.out.println(text.length());
```

Patterns do not match null. A type pattern that would be unconditionally true from the static type is generally rejected as pointless, while casts have a different compatibility history.

Pattern switch in Java 21

Pattern matching for `switch` was preview in Java 17 under JEP 406 and required `--enable-preview`. It evolved across releases and became permanent in Java 21 under JEP 441. Java 17 preview syntax and semantics should not be treated as a stable Java 21 source contract.

JAVA

```
static String describe(Object value) {
    return switch (value) {
        case null -> "missing";
        case String text when text.isBlank() -> "blank text";
        case String text -> "text of length " + text.length();
        case Integer number -> "integer " + number;
        default -> "other";
    };
}
```


Java 21 uses **when** guards. Cases are tried in source order subject to compiler dominance rules. **case Object ignored** dominates subtype cases after it and must therefore appear last. A guarded pattern generally does not dominate the same unguarded pattern unless its guard is a constant true expression.

Pattern switch supports explicit **case null**; without a matching null label, switching on null throws **NullPointerException**. Type and record patterns do not match null, so null handling must be explicit when it is part of the domain.

An exhaustive switch expression over a sealed selector can omit **default** when all permitted alternatives are covered. Omitting a broad default is often desirable: when the hierarchy changes and code is recompiled, the compiler points to every incomplete switch. The compiler still emits a defensive failure path for incompatible binary evolution.

Record patterns in Java 21

Record patterns became permanent in Java 21 under JEP 440. They deconstruct records and can nest:

JAVA

```
record Point(int x, int y) {}
record Segment(Point start, Point end) {}

static int horizontalLength(Object value) {
    return switch (value) {
        case Segment(Point(int x1, int y1), Point(int x2, int y2))
            when y1 == y2 -> Math.abs(x2 - x1);
        default -> 0;
    };
}
```

Record patterns were not part of Java 17. Java 21 also permits **var** in record component patterns where inference is useful. A record pattern checks and extracts component values; accessor invocation and nested matching can still fail only according to ordinary execution and pattern rules, so keep record accessors pure.

TRACE IT | 18.6 Worked Java example

This closed result type turns success, rejection, and transient failure into explicit cases.

JAVA

```
public class PaymentResultDemo {
    sealed interface PaymentResult
        permits Approved, Rejected, RetryLater {}

    record Approved(String authorizationId) implements PaymentResult {}
    record Rejected(String reason) implements PaymentResult {}
    record RetryLater(long delayMillis) implements PaymentResult {}

    static String clientMessage(PaymentResult result) {
        return switch (result) {
            case Approved(String id) -> "approved: " + id;
            case Rejected(String reason) -> "rejected: " + reason;
            case RetryLater(long delay) when delay <= 1_000 -> "retry soon";
            case RetryLater(long delay) -> "retry in " + delay + " ms";
        };
    }

    public static void main(String[] args) {
        System.out.println(clientMessage(new Approved("auth-7")));
        System.out.println(clientMessage(new RetryLater(500)));
    }
}
```

This source requires Java 21 because it uses record patterns and a guarded pattern switch. It needs no preview flag in Java 21.

TRACE IT | 18.7 Execution or memory walkthrough

`new RetryLater(500)` creates a final record instance referenced as `PaymentResult`. `clientMessage` first checks alternatives according to the selector's runtime class. The two unrelated record types do not match. The `RetryLater(long delay)` pattern matches, extracts the primitive component into `delay`, and evaluates the guard.

Because 500 is at most 1000, the guarded arm produces `"retry soon"`; the unguarded `RetryLater` arm is not evaluated. Exhaustiveness follows from all three direct permitted record implementations, with the two `RetryLater` arms collectively covering that variant.

If a fourth permitted subtype is added and this code is recompiled, the compiler rejects the incomplete switch. If separately compiled binaries become inconsistent, a generated defensive path prevents silently treating the new value as an old case.

18.8 Complexity and performance

Interface calls, virtual calls, type tests, component extraction, and ordinary switch selection are conceptually $O(1)$, excluding method bodies. Nested patterns add work proportional to the number of tested components and guards. Sealed metadata can help tools and optimizers reason about possible subtypes, but its primary benefit is semantic completeness.

An abstract template can avoid repeated orchestration code; an interface-plus-composition design may add delegation. These constant costs are typically irrelevant beside I/O. Optimize the model for comprehensibility and profile before specializing dispatch.

HotSpot note: HotSpot may devirtualize interface calls and optimize type tests using class profiling. Sealing can provide additional hierarchy facts, but no source-level latency or allocation guarantee follows from declaring a type sealed.

WATCH OUT | 18.9 Edge cases and common mistakes

- Interface fields are static constants, not per-implementation state.
- A class method wins over an interface default, even if inherited from a superclass.
- Unrelated same-signature defaults require an explicit resolution.
- Abstract classes may have constructors; interfaces do not.
- A sealed type's permitted direct subclasses must satisfy module or package placement rules.
- Every permitted direct subtype must be final, sealed, or non-sealed.
- `non-sealed` is a deliberate reopening, not the absence of a decision.
- Patterns do not behave like unchecked destructuring; type compatibility and dominance are enforced.
- A broad case before a narrow case can make the latter dominated and illegal.
- Null handling in pattern switch must be intentional.
- Java 17 pattern switch was preview and changed before finalization; compile Java 21 examples as Java 21 source.
- Record patterns are final in Java 21 but unavailable in Java 17.
- An exhaustive switch without default is often better for closed models, but binary evolution still needs operational compatibility planning.

18.10 Production engineering notes

Use open interfaces for provider ecosystems and test seams. Use sealed types for centrally owned outcomes, commands, or syntax trees whose variants must be exhaustively understood. Avoid sealing across teams when deployments and extension ownership are independent.

Keep default methods small and contract-preserving. Adding a default method can still conflict with another interface a consumer already implements. Library evolution requires downstream compatibility testing, not only successful compilation of the library.

Pattern switches should translate or interpret variants at clear architectural boundaries. If every consumer repeats a large switch, behavior may belong on the variants or in a visitor-like service. Conversely, putting every external concern into domain variants can create coupling; exhaustive external interpreters are often appropriate.

TRY IT | 18.11 Interview questions and model answers

When should you use an interface instead of an abstract class?

Use an interface for a role that unrelated classes can implement, multiple inheritance of type, or an open provider contract. Use an abstract class when closely related implementations share state, construction rules, and a controlled implementation skeleton. Composition can combine either.

How are conflicting default methods resolved?

A class implementation wins over interface defaults. A more specific subinterface wins over its ancestor. Unrelated defaults require the implementing class to override and optionally delegate with `InterfaceName.super.method()`.

What does sealing guarantee?

Only listed permitted types may directly extend or implement the sealed type, subject to module or package rules. Each permitted subtype states whether its own branch closes, stays sealed, or reopens. It does not imply immutability or a fixed object count.

Which pattern features are final in Java 17?

Type patterns for `instanceof` are final, and sealed classes are final features in Java 17. Pattern matching for switch is preview in Java 17. Record patterns are not available there.

Which features became final in Java 21?

Pattern matching for switch and record patterns are permanent in Java 21. Java 21 guarded case labels use `when`. Neither feature requires `--enable-preview` in Java 21.

TRY IT | 18.12 Exercises

- 1 Resolve a diamond of two unrelated default methods while calling both implementations deliberately.
- 2 Model a sealed file-operation result and write an exhaustive Java 21 switch without default.
- 3 Add a non-sealed branch and explain what remains known to the compiler.
- 4 Convert nested casts for two records into a nested record pattern.
- 5 Write a pattern switch whose cases are initially dominated, then reorder or guard them correctly.
- 6 Decide whether a plugin API should be sealed; document extension ownership, module boundaries, and compatibility consequences.

RECAP | 18.13 Chapter summary

Interfaces model roles and support multiple inheritance of contract and default behavior. Abstract classes provide single-inheritance state and controlled templates. Sealed types make a directly permitted family explicit, while each branch chooses whether to close or reopen. Java 21's final pattern switch and record patterns let code test, extract, guard, and exhaustively handle those models. Java 17 supports final sealed types and `instanceof` patterns, but its pattern switch is preview.

TRY IT | 18.14 Revision checklist

- ☐ I can choose between an interface, abstract class, sealed type, and composition.
- ☐ I know interface field, static, default, and private-method rules.
- ☐ I can resolve default-method conflicts.
- ☐ I understand permits placement and final/sealed/non-sealed continuation.
- ☐ I use flow-scoped `instanceof` pattern variables safely.
- ☐ I can order pattern switch cases without dominance errors.
- ☐ I handle null and exhaustiveness deliberately.
- ☐ I can label Java 17 preview and Java 21 permanent pattern features accurately.

SDE-2 CORE CHAPTER

Chapter 19 - Equality, Hashing, Immutability, and Records

19.1 Learning objectives

By the end of this chapter, you should be able to:

- distinguish reference identity, logical equality, ordering, and domain identity;
- implement the `equals` and `hashCode` contracts together;
- explain why mutable hash keys and subclass equality are dangerous;
- design deeply immutable value objects with safe publication; and
- use records while understanding their generated members and shallow immutability.

WHY IT WORKS | 19.2 Why this matters at SDE-2

Equality is infrastructure for collections, caches, deduplication, persistence, tests, and distributed idempotency. A broken contract does not always fail immediately; it can make an entry unreachable in a map or create asymmetric behavior that changes with argument order.

Records reduce value-carrier boilerplate, but they do not choose a domain model for you. An SDE-2 engineer must decide which fields define equality, whether data is deeply immutable, and whether an ORM entity, request DTO, or domain value should be a record at all.

WHY IT WORKS | 19.3 First-principles model

Every object has identity. Reference `==` asks whether two reference values designate the same object, with null handled as an ordinary reference value. `Object.equals` initially implements identity equality, but classes can override it to define logical value equality.

Hashing compresses equality-relevant state into an integer used to choose candidate buckets. A hash is not a unique identifier. Equal objects must have equal hash codes; unequal objects may collide. Hash-based collections use both hash and equality.

Immutability means an object's observable state cannot change after construction. It requires control of the entire reachable representation, not only final top-level fields. A record is a transparent nominal product of its components with generated accessors and value-oriented members. Its component fields are final, but referenced objects may be mutable.

Specification boundary: Java specifies the `Object.equals` and `hashCode` contracts and record member semantics. It does not specify a `HashMap` bucket layout here, a stable hash code across JVM runs, or that `Object.hashCode` is a memory address.

19.4 Core terminology

- **Identity equality:** same runtime object, tested with reference `==`.
- **Logical equality:** class-defined equivalence, tested by `equals`.
- **Domain identity:** business identity, such as an immutable account identifier.
- **Value object:** object defined by its values rather than lifecycle identity.
- **Hash collision:** unequal values have the same hash code.
- **Consistency:** repeated equality or hash calls return compatible results while relevant state is unchanged.
- **Deep immutability:** no observable reachable mutable state can change through any alias.
- **Defensive copy:** independent representation used to prevent outside mutation.
- **Record component:** declared state item that generates a field, accessor, and participation in generated members.
- **Canonical constructor:** record constructor whose parameters correspond to all components.

19.5 Detailed mechanics

The equals contract

For non-null references, a correct equivalence relation is:

- reflexive: `x.equals(x)` is true;
- symmetric: `x.equals(y)` equals `y.equals(x)`;
- transitive: if `x` equals `y` and `y` equals `z`, `x` equals `z`;
- consistent while equality-relevant information is unchanged; and
- false for `x.equals(null)`.

An implementation normally checks identity, compatible type, and relevant fields:

JAVA

```
@Override
public boolean equals(Object other) {
    if (this == other) return true;
    if (!(other instanceof Coordinate that)) return false;
    return x == that.x && y == that.y;
}
```

`getClass()` versus `instanceof` is a design choice. Exact-class equality is easier to keep symmetric in extensible hierarchies but means a base and subclass never compare equal. `instanceof` can support equality across implementations only if every participant shares the same semantics. Making value classes final or records avoids many subclass traps.

The hashCode contract

If `x.equals(y)`, then `x.hashCode() == y.hashCode()` must hold. The reverse is not required. The result must be consistent while equality state is unchanged. Overriding one of `equals` and `hashCode` without the other is almost always a defect.

JAVA

```
@Override
public int hashCode() {
    int result = Integer.hashCode(x);
    result = 31 * result + Integer.hashCode(y);
    return result;
}
```

`Objects.hash(x, y)` is concise but uses varargs and boxing, which can matter on a hot path. Hand-written accumulation or generated code avoids that allocation. Quality matters because clustered hashes degrade hash-table performance, but correctness still relies on equality.

Never base logical hash solely on a mutable field if objects can be keys. If a key changes after insertion, lookup uses its new hash or equality state while the entry remains located according to its old state.

Equality across common types

Arrays retain identity `equals`; use `Arrays.equals` or `deepEquals`. `BigDecimal.equals` considers value and scale, so `1.0` is not equal to `1.00`; `compareTo` considers them numerically equal. This makes `BigDecimal` behavior differ between hash-based and sorted collections if the chosen semantics are not normalized.

Enums use identity safely because each declared constant is a canonical instance in its defining class loader context, and `Enum.equals` is final. Collections define content equality, usually including iteration order for lists and membership for sets.

Entity equality is difficult when database-generated IDs begin as null and proxies introduce subclasses. Do not mechanically include every mutable field. Choose a stable business key, controlled persistent identity policy, or explicit reference identity based on lifecycle requirements.

Building immutable classes

A typical immutable class is final, has private final fields, validates fully during construction, exposes no mutators, and copies mutable inputs and outputs. Final-field semantics aid safe publication when construction is correct, but final references alone do not freeze mutable referents.

JAVA

```
public final class Tags {
    private final java.util.List<String> values;

    public Tags(java.util.Collection<String> values) {
        this.values = java.util.List.copyOf(values);
    }

    public java.util.List<String> values() {
        return values;
    }
}
```

`List.copyOf` rejects nulls and returns an unmodifiable snapshot when needed. If elements are mutable, copy or transform them too for deep immutability. Unmodifiable wrappers alone are views: another alias may still mutate the backing collection.

Avoid letting `this` escape during construction. Safe final-field visibility assumes the object is not observed before constructor completion.

Records

Records became permanent in Java 16 and are available in Java 17 and 21. A declaration such as:

JAVA

```
record Coordinate(int x, int y) {}
```

implicitly defines a final class extending `java.lang.Record`, private final component fields, public component accessors `x()` and `y()`, a canonical constructor, and generated `equals`, `hashCode`, and `toString`. It can implement interfaces and declare static or instance methods, static fields, and additional constructors, but cannot declare extra instance fields or extend another class.

Generated record equality uses the same record class and component values. Its exact hash combination should not be treated as a stable serialization or partitioning algorithm. Record `toString` is convenient for diagnostics but can expose sensitive components if logged.

A compact canonical constructor validates or normalizes components. Assignments to component fields occur implicitly after its body using the possibly reassigned parameter values.

JAVA

```
record EmailAddress(String value) {
    EmailAddress {
        value = java.util.Objects.requireNonNull(value).strip();
        if (!value.contains("@")) {
            throw new IllegalArgumentException("invalid email");
        }
    }
}
```

Explicit canonical constructors cannot reduce accessibility below the record's accessibility. Records are not automatically deeply immutable, and their accessors expose component references directly.

TRACE IT | 19.6 Worked Java example

This record normalizes permission spelling and takes an immutable snapshot, producing stable equality and hashing.

JAVA

```
import java.util.Set;
import java.util.TreeSet;

public record PermissionSet(String principalId, Set<String> permissions) {
    public PermissionSet {
        if (principalId == null || principalId.isBlank()) {
            throw new IllegalArgumentException("invalid principalId");
        }
        if (permissions == null) {
            throw new IllegalArgumentException("permissions is null");
        }
        TreeSet<String> normalized = new TreeSet<>();
        for (String permission : permissions) {
            if (permission == null || permission.isBlank()) {
                throw new IllegalArgumentException("invalid permission");
            }
            normalized.add(permission.strip().toLowerCase(java.util.Locale.ROOT));
        }
        permissions = Set.copyOf(normalized);
    }

    public boolean allows(String permission) {
        return permissions.contains(permission.toLowerCase(java.util.Locale.ROOT));
    }

    public static void main(String[] args) {
        PermissionSet a = new PermissionSet("u1", Set.of("READ", "write"));
        PermissionSet b = new PermissionSet("u1", Set.of("write", "read"));
        System.out.println(a.equals(b)); // true
        System.out.println(a.allows("READ")); // true
    }
}
```

Because set equality is order-independent and the canonical constructor normalizes content, different input order and case lead to the same value. Whether permission identifiers should be case-insensitive is a domain decision, not a universal rule.

TRACE IT | 19.7 Execution or memory walkthrough

Construction receives copied references to the principal string and caller set. It validates the identifier, creates a new `TreeSet`, normalizes each permission, and then snapshots that set using `Set.copyOf`. Reassigning the constructor parameter changes the value that the compiler assigns to the final component field.

The caller's original set is not retained. The record is shallowly immutable by structure, and this particular representation is deeply immutable because strings are immutable and the set cannot be changed. Generated `equals` compares the same record type and then corresponding component values. Both instances contain equal strings and equal sets, so their hash codes must also match.

`allows` performs normalization without mutating the record. Passing null currently throws `NullPointerException`; a public API should document that or validate with a clearer message.

19.8 Complexity and performance

`equals` is proportional to compared state until a difference is found. Hash computation is similarly proportional unless a safely cached hash is used. Hash-table lookup is expected $O(1)$ with well-distributed hashes and controlled load, but collisions and adversarial inputs can change costs.

Immutable values enable sharing and memoization but defensive copies cost $O(n)$ time and space. `PermissionSet` normalization is $O(n \log n)$ because of `TreeSet`; if sorted construction is unnecessary, a hash set can offer expected $O(n)$. The final `Set.copyOf` may add another pass.

HotSpot note: Generated record methods are ordinary methods that the JVM may inline. Object allocation or field reads may be optimized, but records are still reference objects and do not imply stack allocation or flattened storage.

WATCH OUT | 19.9 Edge cases and common mistakes

- `==` on references tests identity, not logical value.
- Equal objects must share a hash, but equal hashes do not imply equal objects.
- Mutable map keys can become unreachable after insertion.
- `getClass` and `instanceof` strategies can break symmetry when mixed across a hierarchy.
- Including an array field with `Objects.equals` compares array identity; use the matching `Arrays` helper.
- `BigDecimal.equals` includes scale while `compareTo` does not.
- Unmodifiable collection wrappers do not copy their backing collections.
- Final fields do not make mutable elements immutable.
- Records can contain arrays, lists, or date objects that mutate.
- A compact record constructor can reassign parameters for normalization but should not leak partially constructed state.
- Generated `toString` can leak credentials or personal data.
- Hash codes are not stable persistence keys, signatures, or security hashes.

19.10 Production engineering notes

Write equality from domain identity, not from whatever fields happen to exist today. Value objects usually include all normalized value components. Entities need a lifecycle-aware identity policy that behaves before and after persistence and with framework proxies.

If a type is used as a cache or map key, make equality-relevant state immutable. Add contract tests for reflexivity, symmetry, transitivity, equal hashes, null, and collection behavior. Property-based tests can explore combinations better than a few examples.

Use records for transparent carriers when exposing components matches the API. Prefer a conventional class when representation must stay hidden, construction requires staged mutable machinery, or framework constraints conflict. Redact sensitive values explicitly rather than trusting generated output.

TRY IT | 19.11 Interview questions and model answers

What is the relationship between `equals` and `hashCode`?

If two objects are equal, their hash codes must be equal. Unequal objects may share a hash. Both results must stay consistent while equality state is unchanged. Therefore a class that overrides logical equality must provide a compatible hash implementation.

Why is a mutable `HashMap` key dangerous?

The map places it according to its hash at insertion. If equality-relevant state changes, lookup computes a different hash or comparison and may not find the existing entry. Immutability is the simplest key policy.

Are records immutable?

Their component fields are final and the record class is final, so component references cannot be reassigned after construction. The objects referenced by components can still mutate. Defensive copies are required for deep immutability.

Should entity equality include every field?

Usually not. Mutable fields make hashing unstable and two lifecycle instances may represent the same entity. Choose a stable business identity or a carefully specified persistent-identity strategy that handles transient instances and proxies.

Can a record extend a base class?

No. Every record implicitly extends `java.lang.Record` and is final. It may implement interfaces and define behavior.

TRY IT | 19.12 Exercises

- 1 Implement a final `Money` value with currency and minor units, including contract tests.
- 2 Demonstrate a map lookup failure after mutating a key, then repair the key design.
- 3 Compare `BigDecimal("1.0")` and `BigDecimal("1.00")` in `HashSet` and `TreeSet`; explain the result.
- 4 Make a record with a `byte[]` component deeply immutable, including accessor behavior.
- 5 Construct a base/subclass equality implementation that breaks symmetry, then redesign it.
- 6 Decide whether a JPA entity, API DTO, and domain identifier should each be a record, with reasons.

RECAP | 19.13 Chapter summary

Identity and equality answer different questions. Logical equality must be an equivalence relation, and equal values must have equal hashes. Mutable equality state is unsafe for hash keys. Immutability requires control of reachable mutable data and careful publication. Records generate transparent value-oriented structure but remain shallowly immutable reference objects; domain normalization, validation, secrecy, and equality meaning are still design responsibilities.

TRY IT | 19.14 Revision checklist

- ☐ I can state every equals contract property.
- ☐ I implement equals and hashCode together from the same state.
- ☐ I do not treat hashes as unique or stable identifiers.
- ☐ I avoid mutable equality state in collection keys.
- ☐ I can explain exact-class versus cross-class equality trade-offs.
- ☐ I distinguish final fields, unmodifiable views, and deep immutability.
- ☐ I know what members records generate and what they do not guarantee.
- ☐ I validate, normalize, copy, and redact record components as required.

SDE-2 CORE CHAPTER

Chapter 20 - Exceptions and Resource Management

20.1 Learning objectives

By the end of this chapter, you should be able to:

- distinguish checked exceptions, unchecked exceptions, and errors by contract;
- trace matching, propagation, rethrow, `finally`, and suppressed exceptions;
- use try-with-resources with correct ownership and close order;
- translate failures without losing cause or actionable context; and
- design retryable, observable, and non-leaking production error boundaries.

WHY IT WORKS | 20.2 Why this matters at SDE-2

Failure handling determines whether a service preserves data and recovers predictably. A lost cause obscures incidents, an overbroad catch turns corruption into a fake success, and a leaked connection can exhaust a pool. SDE-2 engineers must design failure semantics, not merely make the compiler accept `throws` clauses.

Interviews commonly ask about checked versus unchecked exceptions, `finally` behavior, close ordering, and what happens when both work and cleanup fail. The strongest answers connect the mechanics to API ownership and operational policy.

WHY IT WORKS | 20.3 First-principles model

An exception is an object representing abrupt completion. `throw` transfers control up dynamically nested execution until a compatible `catch` is found. If no application frame catches it, the thread terminates after its uncaught-exception handling.

Java divides `Throwable` into `Error` and `Exception`. `RuntimeException` and its subclasses are unchecked, as are `Error` subclasses. Other `Exception` subclasses are checked: a potentially escaping checked exception must be caught or declared.

A resource is something with a lifecycle that must be closed regardless of normal or abrupt completion. Try-with-resources gives that lifecycle structured scope. Ownership answers who closes; it is separate from which variable can access the resource.

Specification boundary: Java specifies exception search, `finally`, try-with-resources translation, close order, and suppression. It does not guarantee that a process can recover from every `Error`, that every termination path runs cleanup, or that stack-trace collection has a fixed cost.

20.4 Core terminology

- **Checked exception:** non-`RuntimeException` subclass of `Exception`, enforced by compile-time handling rules.
- **Unchecked exception:** `RuntimeException`, `Error`, or subclass.
- **Cause:** lower-level failure wrapped by another exception.
- **Stack trace:** recorded execution frames associated with a throwable.
- **Suppressed exception:** secondary failure retained when another failure is primary.
- **Try-with-resources:** statement that closes `AutoCloseable` resources automatically.
- **Exception translation:** mapping an implementation failure to a boundary-appropriate abstraction.
- **Precise rethrow:** compiler infers narrower exceptions when rethrowing an effectively final catch parameter.
- **Idempotency:** repeated operation has an effect compatible with safe retry.
- **Ownership:** responsibility for closing or releasing a resource.

20.5 Detailed mechanics

Throwing and declaring

`throw` requires a `Throwable` reference; throwing null causes `NullPointerException`. A `throws` clause declares possible checked failures for callers but does not itself throw anything. Unchecked failures may be documented without being declared.

Checked exceptions can communicate a recoverable condition a caller is expected to consider. Unchecked exceptions commonly represent violated preconditions, programming defects, or failures handled at a wider boundary. This is an API design choice, not a guarantee that checked failures are recoverable or unchecked failures are fatal.

JAVA

```
static int parsePort(String text) {
    int port;
    try {
        port = Integer.parseInt(text);
    } catch (NumberFormatException ex) {
        throw new IllegalArgumentException("port is not numeric: " + text, ex);
    }
    if (port < 1 || port > 65_535) {
        throw new IllegalArgumentException("port out of range: " + port);
    }
    return port;
}
```

Translation keeps the original exception as cause and adds domain context. Avoid putting secrets in messages.

Catch selection and propagation

Catch clauses are tested in source order, and the first assignment-compatible type handles the throwable. Therefore a subtype catch must precede a supertype catch or the latter makes it unreachable. After the catch runs normally, execution continues after the whole try statement.

Multi-catch uses alternatives such as `catch (IOException | SQLException ex)`. Alternatives cannot be related by subclassing, and the catch parameter is implicitly final because assigning it would make its type unclear.

Precise rethrow lets this pattern declare the specific possible checked types rather than broad `Exception` when the caught parameter is not reassigned:

JAVA

```
static void invoke() throws java.io.IOException, java.sql.SQLException {
    try {
        callThatThrowsEither();
    } catch (Exception ex) {
        audit(ex);
        throw ex;
    }
}
```

This should not justify catching broad types routinely; it is useful at cross-cutting boundaries that must observe and rethrow unchanged.

Finally and abrupt completion

A `finally` block executes after its associated try/catch completes normally or abruptly, before control continues outward. If `finally` itself returns or throws, it replaces the pending return value or exception. That can silently lose the true failure.

JAVA

```
static int misleading() {
    try {
        return 1;
    } finally {
        return 2; // replaces the pending return; avoid this
    }
}
```

Cleanup may not run if the process halts, is externally killed, the JVM crashes, or execution cannot proceed. Resource safety is scoped, not a substitute for durable recovery protocols.

Try-with-resources

A resource type implements `AutoCloseable`; `Closeable` is a narrower I/O-oriented subtype whose `close` declares `IOException`. Resources initialize left to right and close in reverse order.

JAVA

```
try (var input = java.nio.file.Files.newBufferedReader(path);
    var output = java.nio.file.Files.newBufferedWriter(target)) {
    output.write(input.readLine());
}
```

If initialization of a later resource fails, already initialized earlier resources are closed. If the body throws and closing also throws, the body exception remains primary and close failures are attached through `getSuppressed()`. If the body succeeds but close fails, the close exception propagates. Multiple close failures retain the first propagating close failure as primary and later ones as suppressed according to reverse close order.

Since Java 9, an existing final or effectively final resource variable can appear directly in the resource specification:

JAVA

```
var reader = java.nio.file.Files.newBufferedReader(path);
try (reader) {
    System.out.println(reader.readLine());
}
```

The statement assumes ownership and closes it. The variable remains in scope afterward but denotes a closed resource; do not use it.

The conceptual compiler translation uses nested try/finally logic and `addSuppressed`. Writing that translation manually is error-prone, so use try-with-resources whenever ownership is lexical.

Rethrow, wrapping, and interruption

Use `throw ex;` to rethrow the same object and preserve its trace. Wrapping with `new DomainException(message, ex)` creates an abstraction boundary with a cause. `throw new DomainException(ex.getMessage())` discards the cause and should usually be rejected in review.

Do not catch `Throwable` to continue normal service; it includes serious `Error` conditions. Catching `Exception` at a request or job boundary can be appropriate to log, translate, and isolate one unit of work, provided fatal conditions and cancellation semantics remain intact.

`InterruptedException` communicates cooperative cancellation. Code that cannot propagate it should normally restore the status with `Thread.currentThread().interrupt()` and stop its work. Swallowing it can make shutdown and timeouts fail.

TRACE IT | 20.6 Worked Java example

This example demonstrates primary and suppressed failures with two resources.

JAVA

```
public class SuppressionDemo {
    static final class Resource implements AutoCloseable {
        private final String name;

        Resource(String name) {
            this.name = name;
            System.out.println("open " + name);
        }

        @Override
        public void close() {
            System.out.println("close " + name);
            throw new IllegalStateException("close failed: " + name);
        }
    }

    public static void main(String[] args) {
        try (Resource first = new Resource("first");
            Resource second = new Resource("second")) {
            throw new IllegalArgumentException("body failed");
        } catch (Exception ex) {
            System.out.println("primary: " + ex.getMessage());
            for (Throwable suppressed : ex.getSuppressed()) {
                System.out.println("suppressed: " + suppressed.getMessage());
            }
        }
    }
}
```

The body failure stays primary, preserving the operation's causal event. Both cleanup failures remain discoverable for diagnosis.

TRACE IT | 20.7 Execution or memory walkthrough

`first` initializes, then `second`. The body constructs and throws `IllegalArgumentException`. Before catch selection, resources close in reverse order. Closing `second` throws; because a body exception is already pending, this failure is added to the body's suppressed list. Closing `first` also throws and is added next.

The `catch (Exception ex)` matches the primary `IllegalArgumentException`. It prints `body failed`, followed by `close failed: second` and `close failed: first`. The close exceptions are not causes; they are independent secondary failures during cleanup.

Had resource construction for `second` thrown, the body would never begin, `first` would still close, and any close failure would be suppressed under the second-construction failure.

20.8 Complexity and performance

Normal try blocks do not inherently change algorithmic complexity. Throwing and capturing a stack trace can be expensive relative to ordinary branches, and exceptions should not represent expected high-volume control flow. Catch matching scales with propagation depth and handlers but is not usually the dominant production concern.

Try-with-resources adds close work that was required anyway. Its safety far outweighs tiny structural overhead. Stack traces and retained causes consume memory, especially if exceptions are accumulated rather than logged and released.

HotSpot note: HotSpot optimizes normal paths through exception regions and may omit or change some diagnostic detail for certain repeated VM-thrown exceptions. Do not depend on such implementation behavior; preserve useful application context explicitly.

WATCH OUT | 20.9 Edge cases and common mistakes

- Catching a superclass before its subclass is unreachable code.
- Throwing from `finally` masks a pending exception; returning from `finally` masks both exceptions and returns.
- A `throws Exception` declaration exports implementation uncertainty and burdens every caller.
- Logging and rethrowing at every layer produces duplicate noise; log where context and ownership meet.
- Wrapping without a cause destroys the diagnostic chain.
- Catching and ignoring `InterruptedException` breaks cancellation.
- Retrying after an unknown partial side effect can duplicate writes.
- `AutoCloseable.close` is allowed to throw broad `Exception` and is not universally idempotent.
- Resource variables close in reverse declaration order.
- A caller-supplied stream should not be closed by a callee unless ownership transfer is explicit.
- `getCause()` and `getSuppressed()` represent different relationships.
- An exception message can leak tokens, SQL, customer data, or filesystem paths.

20.10 Production engineering notes

Define exceptions at architectural boundaries. Translate driver errors into repository failures and domain failures into stable API responses, while retaining the original cause internally. Give clients machine-readable error codes rather than parsing messages.

Retries require classification, idempotency, deadlines, bounded attempts, backoff, and jitter. Never classify only by broad exception type when the provider exposes more specific status. Preserve interruption and cancellation.

Use try-with-resources for files, JDBC statements/results, streams, locks represented by closeable guards, and telemetry scopes. Declare ownership in method names or documentation. At top-level request and worker boundaries, record exception class, safe context, trace or correlation ID, and suppressed failures without logging secrets.

TRY IT | 20.11 Interview questions and model answers

What is the difference between checked and unchecked exceptions?

Checked exceptions are `Exception` subclasses other than `RuntimeException`; Java requires callers to catch or declare them. Runtime exceptions and errors are unchecked. The hierarchy affects compile-time handling, not an absolute recoverability classification.

What happens if both a try body and close throw?

In try-with-resources, the body exception remains primary. Close exceptions are attached as suppressed throwables in reverse close order. This preserves both the causal operation failure and cleanup evidence.

Why should you avoid return in finally?

It replaces a pending return or exception, silently changing behavior and losing diagnostics. `finally` should perform limited cleanup that does not determine the method outcome.

When should an exception be translated?

At an abstraction boundary where the lower-level type is not part of the caller's contract. The translated exception should add safe, actionable context and retain the original as its cause.

How should `InterruptedException` be handled?

Prefer propagating it so an owning layer can cancel. If an API cannot declare it, restore interrupt status, stop the current operation, and translate only if the boundary requires it. Do not swallow and continue.

TRY IT | 20.12 Exercises

- 1 Predict primary and suppressed exceptions for three resources whose body and close methods fail.
- 2 Refactor manual file cleanup into try-with-resources and test initialization failure.
- 3 Design repository exception translation that preserves SQL cause without exposing SQL to clients.
- 4 Find all ways a return or throw in `finally` can alter a pending outcome.
- 5 Implement a retry loop that stops on interruption, deadline, permanent failure, and maximum attempts.
- 6 Document ownership for a method accepting an `InputStream`, then offer borrowing and ownership-transfer variants.

RECAP | 20.13 Chapter summary

Exceptions represent abrupt completion and propagate to the first compatible handler. Checked status is a compile-time contract choice. `finally` participates in abrupt completion and can dangerously replace outcomes. Try-with-resources initializes left to right, closes right to left, and preserves secondary failures through suppression. Production failure handling retains causes, respects cancellation and ownership, translates at abstraction boundaries, and retries only with bounded, idempotent policy.

TRY IT | 20.14 Revision checklist

- ☐ I can classify checked, runtime, and error types.
- ☐ I can trace catch selection and propagation.
- ☐ I distinguish causes from suppressed exceptions.
- ☐ I know try-with-resources initialization and reverse close order.
- ☐ I never return from `finally` and avoid throwing from cleanup.
- ☐ I preserve causes when translating failures.
- ☐ I propagate or restore interruption.
- ☐ I make resource ownership, retries, and client error contracts explicit.

SDE-2 CORE CHAPTER

Chapter 21 - Nested Types, Enums, Annotations, and Reflection

21.1 Learning objectives

By the end of this chapter, you should be able to:

- distinguish static nested, inner, local, and anonymous classes and their capture rules;
- design enums with behavior, stable external representation, and collection support;
- define annotations with appropriate targets, retention, defaults, and repeatability;
- inspect and invoke types reflectively while respecting access and module boundaries; and
- decide when metadata-driven runtime behavior is worth its safety and complexity costs.

WHY IT WORKS | 21.2 Why this matters at SDE-2

Framework-heavy Java uses all four topics. Builders and implementation details use nested types, domain state uses enums, dependency injection and serialization use annotations, and frameworks connect metadata to behavior through reflection. An SDE-2 engineer must understand what the framework is doing when scanning fails, an enum evolves, or reflective access is denied.

These features are also design tools. Used carefully, they localize helper types and declarative metadata. Used casually, they create hidden control flow, memory retention, brittle names, and startup surprises.

WHY IT WORKS | 21.3 First-principles model

A nested type is declared inside another declaration or block. A static nested class has no implicit enclosing object. A non-static member inner class is associated with an enclosing instance. Local and anonymous classes live inside executable code and can capture final or effectively final local values.

An enum is a special class with a fixed declared set of named instances. Each constant is constructed during enum class initialization. An annotation is structured metadata attached to supported program elements or type uses. Its retention policy determines whether it reaches source tools, class files, or runtime reflection.

Reflection treats types and members as data through `Class`, `Method`, `Field`, `Constructor`, and related APIs. It can discover and invoke behavior dynamically, but it shifts checks from compilation to runtime and remains constrained by access control and modules.

Specification boundary: Java defines nested-class semantics, enum identity and initialization, annotation metadata, and reflection APIs. Compiler-generated class names, synthetic capture-field names, framework scan order, and reflective inflation or accessor implementation are not stable language contracts.

21.4 Core terminology

- **Static nested class:** nested class with no implicit outer instance.
- **Inner class:** non-static nested class associated with an enclosing instance.
- **Local class:** named class declared inside a block.
- **Anonymous class:** unnamed class expression that extends or implements one type.
- **Capture:** retaining values from an enclosing lexical scope.
- **Enum constant:** canonical instance declared in an enum body.
- **Annotation element:** parameterless metadata member with a restricted return type.
- **Retention:** source-only, class-file, or runtime availability.
- **Type-use annotation:** annotation applicable wherever a type is used.
- **Reflective access:** discovering or operating on program structure at runtime.

21.5 Detailed mechanics

Member nested and inner classes

```
JAVA
class Cache {
    private final String region;

    Cache(String region) { this.region = region; }

    static final class Key {
        private final String value;
        Key(String value) { this.value = value; }
    }

    final class Entry {
        String description() { return region + " entry"; }
    }
}
```

`Cache.Key` can be constructed without a `Cache`: `new Cache.Key("k")`. It cannot directly access an instance field because it has no implicit receiver. `Entry` requires an enclosing instance: `cache.new Entry()`. Its methods can access outer private state.

Use a static nested class unless the semantic relationship truly requires an outer instance. An inner instance retains its enclosing instance, which can unintentionally keep a large object graph alive. Static declarations

inside inner classes have become more permissive in modern Java, but an explicit outer relationship is still the defining semantic distinction.

Nested types follow access control and can access private members of their nest under language rules. At the JVM level, modern class files use nestmate metadata to support this without exposing those members publicly.

Local and anonymous classes

A local class is visible only in its declaring block. An anonymous class combines allocation with an unnamed subclass or interface implementation:

JAVA

```
Runnable task = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("running");  
    }  
};
```

Anonymous classes can declare fields and methods but no explicit constructor, and their unique runtime type is awkward to name. Lambdas are usually clearer for functional interfaces, but anonymous classes differ: `this` refers to the anonymous object, they can hold extra state, and they create an actual class instance with class-style semantics.

Local variables captured by local/anonymous classes or lambdas must be final or effectively final. The code captures a value, not a mutable local variable slot. Mutable objects referenced by the captured value can still change.

Enums

Every enum implicitly extends `Enum<E>`, cannot extend another class, and may implement interfaces. Constants are public static final instances. Enum constructors are never public or protected and run during class initialization.

JAVA

```
enum Severity {  
    INFO(1), WARNING(2), CRITICAL(3);  
  
    private final int rank;  
  
    Severity(int rank) { this.rank = rank; }  
    int rank() { return rank; }  
}
```

Compare enum constants with `==`; identity is the intended semantics and null-safe when the known constant is on the left. `name()` is the declared source name, `ordinal()` is its position, and `toString()` may be overridden. Never persist ordinal because reordering or inserting constants changes it. Persist an explicit stable code if external compatibility matters.

`Enum.valueOf` matches the exact declared name and throws for unknown text. `values()` returns a new array each call. `EnumSet` and `EnumMap` are specialized, type-safe collections and are preferable to manual bit masks or general-purpose maps for enum domains.

Constants may have constant-specific class bodies that override behavior, but a large behavioral enum can become a closed service locator. Keep behavior cohesive and consider strategies when implementations need independent deployment.

Switches over enums should account for evolution. An exhaustive switch expression without default helps recompilation reveal a new constant; separately compiled old code may still encounter a defensive runtime failure with a newer enum.

Declaring annotations

JAVA

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Operation {
    String value();
    boolean idempotent() default false;
}
```

Annotation elements may return primitives, `String`, `Class`, enum, annotation, or one-dimensional arrays of these. They have no parameters or throws clauses. Values must be compile-time-compatible annotation values; null is not allowed. Defaults belong to the annotation method declaration and are applied when read, so changing a default can change behavior of already compiled annotated code at runtime.

Important meta-annotations include:

- `@Target` for legal declaration or type-use sites;
- `@Retention` with `SOURCE`, `CLASS`, or `RUNTIME`;
- `@Documented` for API documentation inclusion;
- `@Inherited`, which applies only to class-level annotation lookup through superclass inheritance, not interfaces, members, or general meta-annotation composition; and
- `@Repeatable`, whose value names a container annotation.

Runtime processors need `RUNTIME` retention. Compile-time annotation processors usually work from source/model APIs and do not require runtime retention. `TYPE_USE` supports nullness and other type-system qualifiers, while `TYPE` targets a type declaration.

Reflection

Obtain a `Class<?>` from a literal (`Order.class`), object (`value.getClass()`), primitive literal (`int.class`), array class, or `Class.forName`. A class literal normally does not initialize the class; certain reflective uses and `Class.forName(String)` default behavior can initialize it.

`getMethods()` returns public methods including inherited ones. `getDeclaredMethods()` returns methods declared directly regardless of access, excluding inherited methods. Similar distinctions apply to fields and constructors.

JAVA

```
Operation metadata = method.getAnnotation(Operation.class);
Object result = method.invoke(target, argument);
```

Reflective invocation wraps an exception thrown by the target in `InvocationTargetException`; inspect its cause. Argument mismatch and access failures are runtime exceptions or checked reflective failures rather than compile-time errors.

Calling `setAccessible(true)` or `trySetAccessible()` does not grant unlimited authority. In Java 17 and 21, strong module encapsulation can deny deep reflection when the declaring package is not opened to the caller's module. Public access may require package export; deep reflective access usually requires `opens`, command-line configuration, or an API designed for it.

Method handles in `java.lang.invoke` offer a typed, composable alternative and often integrate better with JVM optimization, but lookup access rules still apply. Reflection is appropriate for frameworks and tools, not as a replacement for ordinary polymorphism when types are known.

TRACE IT | 21.6 Worked Java example

This small registry discovers annotated methods on an explicitly supplied handler. It avoids uncontrolled classpath scanning.

JAVA (continued 1/2)

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

public class OperationRegistry {
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.METHOD)
    @interface Operation {
        String value();
    }

    static final class Handler {
        @Operation("health")
        public String health() { return "ok"; }
    }

    static Map<String, Method> discover(Class<?> type) {
        Map<String, Method> result = new HashMap<>();
        for (Method method : type.getDeclaredMethods()) {
```

JAVA (continued 2/2)

```

        Operation operation = method.getAnnotation(Operation.class);
        if (operation == null) continue;
        if (method.getParameterCount() != 0 || method.getReturnType() != String.class) {
            throw new IllegalArgumentException("invalid operation method: " + method);
        }
        if (result.putIfAbsent(operation.value(), method) != null) {
            throw new IllegalArgumentException("duplicate operation: " + operation.value());
        }
    }
    return Map.copyOf(result);
}

public static void main(String[] args) throws ReflectiveOperationException {
    Handler handler = new Handler();
    Method method = discover(Handler.class).get("health");
    System.out.println(method.invoke(handler));
}
}

```

The nested annotation and handler are scoped to the example. Production registries should validate public accessibility, inherited-method policy, duplicate names, return contracts, and target exceptions deliberately.

TRACE IT | 21.7 Execution or memory walkthrough

Loading `OperationRegistry` makes its nested class metadata available. `Handler.class` supplies a class object without constructing a handler. `getDeclaredMethods` creates reflection descriptors for directly declared methods; their order is unspecified, so the registry must not depend on iteration order.

`getAnnotation` returns a runtime annotation representation because retention is `RUNTIME`. The registry validates the signature and stores the `Method` under `health`, then creates an unmodifiable map.

`method.invoke(handler)` checks receiver and arguments and dispatches to `health`, which returns "ok".

If `health` threw, `invoke` would throw `InvocationTargetException` with the application exception as cause. If the method were inaccessible across a module boundary, discovery might still see a declared descriptor, but invocation could fail unless access was legitimately available.

21.8 Complexity and performance

Enum comparison and ordinal-based specialized collection operations are generally $O(1)$. `values()` copies $O(n)$ constants. Reflection discovery is $O(m)$ in inspected members plus annotation and validation work. Cache validated descriptors rather than scanning on each request.

Reflective invocation adds access checks, argument adaptation, boxing, and indirection compared with a direct call. For startup wiring or administrative paths this is usually irrelevant. For a hot serializer loop, cache method handles or generated accessors and benchmark realistic data.

HotSpot note: Reflection and method-handle implementations can optimize repeated access, and the JIT may inline adapted method-handle chains. Exact thresholds and generated accessor strategies are implementation-specific and version-dependent.

WATCH OUT | 21.9 Edge cases and common mistakes

- A non-static inner instance retains an enclosing instance.
- Captured locals must be effectively final, but referenced mutable state can still race.
- Anonymous-class `this` differs from lambda `this`.
- Persisting enum ordinal breaks when constants are reordered.
- Persisting `toString()` breaks when presentation changes; use an explicit stable code.
- `Enum.valueOf` is case-sensitive and rejects unknown future values.
- Runtime reflection cannot see annotations with `SOURCE` or `CLASS` retention through ordinary annotation APIs.
- `@Inherited` does not propagate method annotations or interface annotations.
- Reflection member order is unspecified.
- `getMethod` and `getDeclaredMethod` differ in inheritance and access scope.
- Target exceptions arrive wrapped by `InvocationTargetException`.
- Strong module encapsulation can reject deep reflection despite `setAccessible`.
- Annotation defaults can change runtime interpretation without recompiling clients.

21.10 Production engineering notes

Prefer static nested types for builders and implementation helpers. Make inner ownership explicit and avoid registering long-lived callbacks that accidentally retain an outer service. Captured state used concurrently needs the same synchronization as any other shared state.

Give externally serialized enums stable codes and an unknown-value policy. Adding a constant is source-compatible in many locations but can break exhaustive consumers, database constraints, or generated clients. Roll out readers before writers when protocols evolve.

Treat annotations as an API. Specify valid placement, inheritance/merge behavior, duplicates, defaults, and validation timing. Fail fast at startup rather than on the first request. Limit reflection to known packages or explicit registrations, cache results, respect modules, and never invoke untrusted member names without authorization and validation.

TRY IT | 21.11 Interview questions and model answers

What is the difference between a static nested class and an inner class?

A static nested class has no implicit enclosing instance and cannot directly use outer instance state. A member inner class is associated with a particular outer object and can access it, which also means it retains that object.

Why are captured local variables effectively final?

The nested object may outlive the method activation, so it captures the local's value rather than sharing its stack variable slot. Reassignment would create ambiguous value semantics. The captured reference can still designate mutable data.

Why should enum ordinal not be persisted?

Ordinal is declaration position, not a stable domain code. Inserting or reordering constants changes it. Persist an explicit immutable code and map unknown values intentionally.

What does runtime retention mean?

The annotation is recorded in the class file and exposed through runtime reflection APIs. CLASS retention records it but ordinary runtime reflection does not expose it; SOURCE retention need not enter the class file.

Can reflection access every private field?

No. Access depends on the caller, language access, security context, and module openness. Strong encapsulation in Java 17 and 21 can prevent deep reflection unless the package is opened appropriately.

TRY IT | 21.12 Exercises

- 1 Demonstrate an inner callback retaining its outer object, then convert it to a static nested class with explicit state.
- 2 Define an enum with stable external codes and safe unknown-code parsing.
- 3 Create a repeatable runtime method annotation and inspect both repeated instances.
- 4 Extend the registry to unwrap `InvocationTargetException` while preserving its cause.
- 5 Compare `getMethods` and `getDeclaredMethods` on a subclass with public and private members.
- 6 Design a startup validation policy for annotated handlers covering duplicates, visibility, signatures, and module access.

RECAP | 21.13 Chapter summary

Nested types control lexical scope and object relationships; only inner forms carry an implicit enclosing instance. Enums are canonical typed instances and need explicit external codes for evolution. Annotations provide constrained metadata whose target and retention define its reach. Reflection connects metadata to runtime behavior but moves checks later and remains subject to access and modules. Use explicit registration, validation, caching, and narrow authority to keep dynamic systems understandable.

TRY IT | 21.14 Revision checklist

- ☐ I distinguish static nested, inner, local, and anonymous classes.
- ☐ I understand capture, effectively final locals, and outer-instance retention.
- ☐ I use enum identity safely and never persist ordinal.
- ☐ I choose annotation targets, retention, defaults, and repeatability deliberately.
- ☐ I know the limits of `@Inherited`.
- ☐ I distinguish declared from inherited reflection queries.
- ☐ I unwrap target failures and respect module encapsulation.
- ☐ I validate and cache reflective metadata outside hot request paths.

SDE-2 CORE CHAPTER

Chapter 22 - Generics, Variance, Type Erasure, and Heap Pollution

22.1 Learning objectives

By the end of this chapter, you should be able to:

- design generic classes and methods with useful bounds;
- explain invariance and apply upper- and lower-bounded wildcards;
- reason about wildcard capture and type inference;
- describe erasure, bridge methods, reifiability, and generic restrictions; and
- prevent raw-type leaks, unsafe generic arrays, and heap pollution.

WHY IT WORKS | 22.2 Why this matters at SDE-2

Generics make collection and framework APIs reusable while preserving type safety. Weak generic design forces casts onto callers; overly clever signatures become unreadable. SDE-2 engineers must balance precision and usability, especially in repositories, event buses, serializers, and reusable algorithms.

Interview questions often focus on why `List<Integer>` is not a `List<Number>`, what `? super T` permits, and how erasure causes bridge methods or heap pollution. The production version is harder: warnings appear far from the eventual `ClassCastException`.

WHY IT WORKS | 22.3 First-principles model

A type parameter is a compile-time variable ranging over reference types. A parameterized type such as `List<String>` states that operations observe a consistent element type. Java implements generics primarily through erasure: most type arguments do not exist as independently testable runtime types.

Generic classes are invariant by default. Even though `Integer` is a subtype of `Number`, `List<Integer>` is not a subtype of `List<Number>`, because the latter would allow insertion of a `Double`. Wildcards express use-site variance: `? extends Number` is an unknown specific subtype useful for producing numbers, and `? super Integer` is an unknown specific supertype useful for consuming integers.

Heap pollution occurs when a variable of a parameterized type refers to an object that does not satisfy that parameterization. It usually begins with an unchecked operation and fails later when a compiler-inserted cast encounters the wrong value.

Specification boundary: Java specifies generic typing, erasure translation, casts, and bridge behavior. It does not preserve arbitrary type arguments for runtime `instanceof`, and reflective generic signatures are metadata rather than a fully reified runtime type system.

22.4 Core terminology

- **Type parameter:** declared type variable such as `<T>`.
- **Type argument:** supplied type such as `String` in `List<String>`.
- **Parameterized type:** generic declaration instantiated with arguments.
- **Bound:** restriction such as `<T extends Comparable<? super T>>`.
- **Invariance:** no subtype relation follows between parameterizations merely from their arguments.
- **Wildcard:** unknown type argument written `?` with optional upper or lower bound.
- **Capture conversion:** compiler treats a wildcard as a fresh unknown type.
- **Erasure:** mapping generic types to non-generic runtime representations with casts as needed.
- **Reifiable type:** type whose runtime representation retains enough information for checks.
- **Heap pollution:** runtime value violates a parameterized variable's promised type.

22.5 Detailed mechanics

Generic declarations and methods

JAVA

```
final class Box<T> {
    private final T value;

    Box(T value) { this.value = value; }
    T value() { return value; }
}

static <T> T first(java.util.List<T> values) {
    if (values.isEmpty()) throw new IllegalArgumentException("empty");
    return values.get(0);
}
```

`T` is in scope for instance members of `Box`, but static members cannot use the class's `T` because static state is shared across all parameterizations. A static method may declare its own `<T>` before the return type.

Primitive types cannot be type arguments, so use wrappers or primitive-specialized APIs. `Box<String>` and `Box<Integer>` share the same raw runtime class object under erasure.

Type inference uses argument context, target type, and bounds. Explicit type witnesses such as `Collections.<String>emptyList()` are occasionally useful when inference lacks context, but modern Java resolves most ordinary calls.

Bounds

An upper bound exposes operations supported by the bound:

JAVA

```
static <T extends Number> double total(java.util.List<T> values) {
    double sum = 0;
    for (T value : values) sum += value.doubleValue();
    return sum;
}
```

Multiple bounds use one class first, followed by interfaces: `<T extends Base & Comparable<T> & Serializable>`. Erasure of a type variable is its leftmost bound, which makes bound order relevant to generated binary signatures.

Recursive or F-bounds describe operations in terms of the implementing type. A common ordering bound is `<T extends Comparable<? super T>>`, which accepts a type comparable to itself or one of its supertypes. This is more flexible than `Comparable<T>` for inherited comparison definitions.

Invariance and wildcards

Given `List<Integer> integers`, assignment to `List<Number>` is illegal. Both can, however, be viewed through `List<? extends Number>`:

JAVA

```
java.util.List<? extends Number> source = java.util.List.of(1, 2, 3);
Number number = source.get(0);
// source.add(4); // rejected; exact captured subtype is unknown
```

You cannot add any non-null value because the actual list might be `List<Double>`. Null is type-compatible but adding it is rarely useful.

For a consumer:

JAVA

```
java.util.List<? super Integer> target = new java.util.ArrayList<Number>();
target.add(10);
Object value = target.get(0);
```

The target is known to accept `Integer`, but reading yields only `Object` because its exact element type might be `Object` or `Number`. The mnemonic PECS is useful: producer extends, consumer super. It describes the role of a parameter, not an absolute law. A mutable parameter used for both reading and writing often needs an exact `List<T>`.

An unbounded `List<?>` means a list of some one unknown type, not a list that can contain any types. It is safer than raw `List`: reads yield `Object`, and arbitrary writes are rejected.

API variance examples

JAVA

```
static <T> void copy(
    java.util.List<? extends T> source,
    java.util.List<? super T> destination) {
    for (T element : source) {
        destination.add(element);
    }
}
```

This accepts `List<Integer>` into `List<Number>` or `List<Object>`. If both parameters were `List<T>`, invariant inference would reject useful combinations.

Return types usually should not expose wildcards when the method can state a concrete parameterization. Returning `List<? extends Animal>` makes callers handle an unknown type; `List<Animal>` or a method type variable often gives a clearer ownership contract.

Wildcard capture

This seemingly simple method fails if written directly because two calls to `list.get` and `list.set` must agree on the captured unknown type. A helper captures it:

JAVA

```
static void reverseFirstTwo(java.util.List<?> list) {
    reverseFirstTwoCaptured(list);
}

private static <T> void reverseFirstTwoCaptured(java.util.List<T> list) {
    if (list.size() >= 2) {
        T first = list.get(0);
        list.set(0, list.get(1));
        list.set(1, first);
    }
}
```

The public API correctly accepts a list of any single element type. The private method gives that captured type a name so read values can be written back safely.

Erasure and bridge methods

Erasure maps an unbounded type variable to `Object`, a bounded variable to its leftmost bound, and a parameterized type to its raw declaration. The compiler inserts casts where a caller expects a more specific result.

JAVA

```
interface Provider<T> { T get(); }
final class TextProvider implements Provider<String> {
    public String get() { return "text"; }
}
```

After erasure, `Provider.get` returns `Object`, while the implementation returns `String`. The compiler emits a synthetic bridge method equivalent in effect to `Object get()` delegating to `String get()`, preserving polymorphism. Reflection and stack traces can expose bridge methods; tools should check `Method.isBridge()` and `isSynthetic()` when appropriate.

Two overloads whose signatures erase identically cannot coexist, such as `process(List<String>)` and `process(List<Integer>)`. Neither can code test `value instanceof List<String>` because runtime sees only `List`; `value instanceof List<?>` is legal.

Reifiability and arrays

Primitive types, non-generic classes, raw types, unbounded-wildcard parameterizations, and certain arrays of reifiable types are reifiable. `List<String>` is not. Arrays are reified and covariant, which conflicts with erased invariant generics. Therefore `new T[10]` and `new List<String>[10]` are illegal.

A generic data structure may allocate `Object[]` internally and cast at a controlled boundary, accept an array constructor such as `IntFunction<T[]>`, or prefer collections. Any unchecked cast must be locally justified and representation-safe.

Raw types and heap pollution

Raw types exist for backward compatibility and erase parameter checks:

JAVA

```
java.util.List<String> names = new java.util.ArrayList<>();
java.util.List raw = names;
raw.add(42); // unchecked warning
// String first = names.get(0); // ClassCastException here
```

The list object now violates the promise `List<String>`. The failure appears at the later compiler-inserted cast, not at the unsafe write. Treat unchecked warnings as defects unless a narrow adapter proves safety.

Generic varargs add an array boundary. A `T...` parameter has an array runtime representation that cannot fully represent non-reifiable `T`. `@SafeVarargs` may be applied to eligible methods and constructors only when the body neither corrupts nor unsafely exposes the array.

TRACE IT | 22.6 Worked Java example

This generic maximum works for types comparable to a supertype and accepts any producing collection.

JAVA

```
import java.util.Collection;
import java.util.List;
import java.util.NoSuchElementException;

public class GenericMaximum {
    static <T extends Comparable<? super T>> T max(
        Collection<? extends T> values) {
        var iterator = values.iterator();
        if (!iterator.hasNext()) {
            throw new NoSuchElementException("empty input");
        }
        T best = iterator.next();
        while (iterator.hasNext()) {
            T candidate = iterator.next();
            if (candidate.compareTo(best) > 0) {
                best = candidate;
            }
        }
        return best;
    }

    public static void main(String[] args) {
        List<Integer> values = List.of(4, 9, 2);
        Number result = max(values);
        System.out.println(result); // 9
    }
}
```

`Collection<? extends T>` permits a collection of a subtype of the inferred result type. The comparison bound ensures every candidate can compare itself with a compatible supertype.

TRACE IT | 22.7 Execution or memory walkthrough

For `max(values)`, inference can choose `Integer` for `T`; the target assignment to `Number` also accepts the returned `Integer`. The iterator has type compatible with the captured producer, and each `next` result can be widened to `T`.

`best` starts as 4. Candidate 9 compares greater and replaces it; candidate 2 does not. The returned reference designates the same immutable `Integer` object held by the list, not a copy of an object.

At runtime, erasure represents much of this using `Comparable`, `Collection`, and casts inserted at use sites. The static proof prevents unrelated elements from entering through this API. No runtime object representing the type variable `T` is created.

22.8 Complexity and performance

Generics do not change the algorithmic complexity of operations. `max` is $O(n)$ time and $O(1)$ additional space. Erasure normally avoids per-parameterization class generation, but wrapper types can introduce boxing and allocation when primitives pass through generic collections.

Bridge calls and casts are small constant costs and are often optimized. Wildcards are compile-time constructs and do not allocate. Prefer primitive-specialized structures only after data volume and profiling justify complexity.

HotSpot note: HotSpot can inline generic erased code, eliminate casts proven redundant, and optimize some boxing. Java generics do not promise specialization for each type argument, and current optimization should not be treated as a no-boxing guarantee.

WATCH OUT | 22.9 Edge cases and common mistakes

- `List<Dog>` is not a subtype of `List<Animal>`.
- `? extends T` supports safe reads as `T` but not arbitrary writes.
- `? super T` supports writes of `T` but reads only as `Object`.
- `List<?>` is type-safe unknown, while raw `List` disables checks.
- Primitive types cannot be generic arguments.
- Static members cannot use a class's type parameter.
- Overloads cannot differ only by generic arguments with the same erasure.
- `instanceof List<String>` and `new T[]` are illegal because types are not reified.
- An unchecked cast can defer a failure far away from its source.
- `@SuppressWarnings("unchecked")` documents suppression, not proof; keep its scope tiny and comment the invariant.
- `@SafeVarargs` is unsafe if the method writes through an alias or returns its varargs array.
- A wildcard in every return type burdens callers and can indicate an over-general API.
- Reflection's `getGenericType` reads signatures but does not make runtime values generically reified.

22.10 Production engineering notes

Expose variance at API boundaries and concrete type parameters within implementations. Accept the least restrictive producer or consumer shape the method truly supports. Keep public signatures readable; a named domain interface is sometimes better than a page of bounds.

Eliminate raw types in modern code. At legacy or reflection boundaries, validate runtime values once, copy into a correctly typed structure, and isolate a justified unchecked cast. Compile with unchecked warnings enabled and fail builds on unexplained new warnings.

Type tokens such as `Class<T>` reify only a raw class and cannot represent `List<String>`. Frameworks use richer type descriptors or anonymous-superclass capture for nested generic types. Understand whether their metadata survives proxies, serialization, and ahead-of-time compilation.

TRY IT | 22.11 Interview questions and model answers

Why is `List` not a subtype of `List`?

If it were, a method receiving `List<Number>` could add a `Double` to an actual integer list. Invariance prevents that. Use `List<? extends Number>` for a producer view or `List<? super Integer>` for a consumer view.

What does PECS mean?

Producer extends, consumer super. An input that only produces `T` can use `? extends T`; one that only consumes `T` can use `? super T`. A structure both read and written as the same exact type often uses `T` directly.

What is type erasure?

The compiler checks generic types, then maps most parameterized uses to raw runtime representations, inserting casts and sometimes bridge methods. This supports binary compatibility but prevents runtime tests of arbitrary type arguments.

What is heap pollution?

A parameterized variable refers to data that violates its claimed type, typically because of a raw type, unchecked cast, or unsafe generic varargs operation. A later generated cast then throws, far from the unsafe operation.

Why does the compiler generate bridge methods?

Erase can make an overriding method's erased descriptor differ from the generic supertype descriptor. A synthetic bridge with the erased signature delegates to the specific implementation, preserving runtime polymorphism.

TRY IT | 22.12 Exercises

- 1 Implement `copy` first with invariant lists, observe rejected calls, then add correct producer/consumer wildcards.
- 2 Write and invoke a helper that captures `List<?>` to swap two positions.
- 3 Explain the bound `<T extends Comparable<? super T>>` using a superclass that implements `Comparable`.
- 4 Create heap pollution through a raw list and locate the later compiler-inserted cast.
- 5 Inspect `TextProvider` reflection output and identify its bridge method.
- 6 Design a type-safe heterogeneous registry using `Class<T>` keys and isolate any checked cast.

RECAP | 22.13 Chapter summary

Generics provide compile-time consistency over reference types. Parameterizations are invariant, while wildcards express producer and consumer variance. Bounds expose operations and capture helpers name unknown types. Erasure preserves a shared runtime representation, requiring casts and bridge methods and preventing tests of most type arguments. Raw types, unchecked casts, and unsafe generic varargs can pollute the heap; confine unavoidable unchecked boundaries and prove their invariants.

TRY IT | 22.14 Revision checklist

- ☐ I can declare generic classes, methods, and multiple bounds.
- ☐ I can explain invariance with the `unsafe-insertion` argument.
- ☐ I apply `extends` for producers and `super` for consumers.
- ☐ I understand unbounded wildcard versus raw type.
- ☐ I can use a helper to capture a wildcard.
- ☐ I know erasure, reifiability, and bridge-method consequences.
- ☐ I can identify every common source of heap pollution.
- ☐ I isolate and justify unavoidable unchecked operations.

SDE-2 CORE CHAPTER

Chapter 23 - Lambdas, Method References, and Functional Interfaces

23.1 Learning objectives

By the end of this chapter, you should be able to:

- define and recognize functional interfaces, including generic primitive variants;
- explain lambda target typing, capture, scope, and exception compatibility;
- distinguish the four principal method-reference forms;
- compose behavior while controlling side effects and concurrency assumptions; and
- design functional APIs that remain readable, testable, and allocation-aware.

WHY IT WORKS | 23.2 Why this matters at SDE-2

Modern Java APIs use functions for streams, asynchronous stages, retries, transaction callbacks, and configuration. A lambda can make variation concise, but it can also hide blocking work, capture mutable state, or make exception handling opaque.

SDE-2 interviews test syntax less often than semantics: whether a lambda is an object, which overload it targets, how `this` behaves, and why a captured counter will not compile. Production reviews add ownership, threading, observability, and idempotency.

WHY IT WORKS | 23.3 First-principles model

A lambda expression is source syntax for supplying an implementation of a functional interface's single abstract method. It has no standalone type. Its target type comes from assignment, invocation, cast, or return context, and that context determines parameter and return compatibility.

A method reference is another compact implementation form that delegates the functional method to an existing static method, bound receiver, unbound receiver, or constructor. Neither syntax promises a particular generated class, allocation, or object identity.

Captured local values must be final or effectively final. The lambda captures their values, not mutable stack slots. A captured reference may still reach mutable state, so type legality does not imply thread safety.

Specification boundary: Java specifies lambda behavior through target functional interfaces, including capture and `this` semantics. The runtime representation, caching, generated class shape, and use of `invokedynamic` linkage are implementation details and must not be observed through identity assumptions.

23.4 Core terminology

- **Functional interface:** interface with one abstract method after inheritance and override-equivalence rules.
- **SAM:** single abstract method and its function type.
- **Target typing:** surrounding context determines a lambda or method-reference type.
- **Capture:** lambda retains values from its lexical scope.
- **Effectively final:** local assigned once even without `final` modifier.
- **Bound method reference:** receiver object is captured, such as `printer::print`.
- **Unbound method reference:** receiver becomes the first function parameter, such as `String::length`.
- **Constructor reference:** creates through `Type::new` or an array constructor.
- **Higher-order function:** accepts or returns behavior.
- **Non-interference:** operation does not modify the source or shared state in a way that invalidates processing.

23.5 Detailed mechanics

Functional interface rules

`@FunctionalInterface` makes the compiler validate intent. The annotation is optional but useful. Default, static, and private methods do not count as abstract methods. Public methods corresponding to `Object` methods do not prevent functional status in the relevant interface rules.

JAVA

```
@FunctionalInterface
interface CheckedSupplier<T> {
    T get() throws Exception;

    default T getOrElse(T fallback) {
        try {
            return get();
        } catch (Exception ex) {
            return fallback;
        }
    }
}
```

Generic interface methods can prevent lambda targeting in cases where no single concrete function type can be inferred. Keep custom interfaces focused and reuse `java.util.function` when its naming and exception contract fit.

Common interfaces include:

- `Predicate<T>`: T to boolean;
- `Function<T,R>`: T to R;
- `UnaryOperator<T>`: T to T;
- `BinaryOperator<T>`: two Ts to T;
- `Consumer<T>`: T to void;
- `Supplier<T>`: no arguments to T; and
- primitive specializations such as `IntPredicate`, `ToLongFunction<T>`, and `LongSupplier` to reduce boxing.

Lambda syntax and target typing

JAVA

```
java.util.function.Predicate<String> nonBlank = s -> !s.isBlank();
java.util.function.BinaryOperator<Integer> add = (left, right) -> left + right;
java.util.function.Supplier<String> empty = () -> "";
```

One inferred parameter can omit parentheses. Multiple or zero parameters require them. A block body uses statements and must return on every normal path when the functional result is non-void. Parameter types must be all inferred, all `var`, or all explicit; annotations can be placed on `var` parameters.

The same lambda text can target different interfaces. Overloads taking unrelated functional interfaces with identical-looking shapes can therefore be ambiguous:

JAVA

```
static void use(java.util.function.Function<String, Integer> f) {}
static void use(java.util.function.ToIntFunction<String> f) {}
// use(s -> s.length()); // ambiguous
use((java.util.function.ToIntFunction<String>) String::length);
```

Avoid public overload sets distinguished only by similar function types. Named methods or differently named operations are easier for callers.

Scope, capture, and this

JAVA

```
int limit = 10;
java.util.function.IntPredicate small = value -> value < limit;
// limit = 20; // would make limit not effectively final
```

The value 10 is captured. Instance lambdas can access fields, whose values may change because the lambda captures `this`, not a snapshot of each field. Local variables cannot be redeclared with the same name inside a lambda parameter or body when lexical scope forbids shadowing.

Lambda `this` and `super` are lexically scoped: they refer to the enclosing instance. In an anonymous class, `this` refers to the anonymous object. This distinction matters for listeners and recursion.

Capture can extend object lifetimes. Storing a lambda that references a large service or request object retains that graph. A stateless lambda may be reused by an implementation, but code must not depend on reuse or synchronize on a lambda object.

Method references

Four common forms are:

JAVA

```
java.util.function.IntBinaryOperator max = Math::max;           // static
var prefix = "id:";
java.util.function.Function<String, String> add = prefix::concat; // bound
java.util.function.ToIntFunction<String> length = String::length; // unbound
java.util.function.Supplier<java.util.ArrayList<String>> list =
    java.util.ArrayList::new;                                     // constructor
```

For `String::length`, the function argument is the receiver. An unbound reference can also have additional parameters, as `String::indexOf` matching `(String receiver, String search) -> int`, subject to overload resolution.

A bound receiver expression is evaluated when the method reference is created, not each time it is invoked. If that expression is null, creation can throw `NullPointerException`. A lambda `x -> receiver.method(x)` evaluates its captured receiver according to its own expression semantics and may fail later, so the forms are not always timing-equivalent.

Array constructor references such as `String[]::new` match `IntFunction<String[]>` and preserve a reified component type.

Composition

Standard interfaces provide combinators. Predicates support `and`, `or`, and `negate`; functions support `compose` and `andThen`; consumers support sequential `andThen`.

JAVA

```
java.util.function.Function<String, String> strip = String::strip;
java.util.function.Function<String, String> lowercase =
    text -> text.toLowerCase(java.util.Locale.ROOT);
var normalize = strip.andThen(lowercase);
```

`strip.andThen(lowercase)` applies `strip` first. `strip.compose(lowercase)` would apply `lowercase` first. Composition preserves thrown runtime exceptions and does not introduce rollback; if the first consumer has a side effect and the second fails, the first effect remains.

Standard function interfaces do not declare checked exceptions. Options are to handle within the lambda, adapt from a custom checked interface at a boundary, or redesign the surrounding API. A generic "sneaky throw" erodes declared contracts and should not be a default strategy.

Behavior and concurrency

A callback's contract must say whether it may run zero, one, or many times; synchronously or asynchronously; sequentially or concurrently; and on which thread or executor. Lambdas do not make captured mutable objects safe. Side effects in parallel streams or retry callbacks can race or repeat.

Do not assume function instances have meaningful `equals`, `hashCode`, serialization, or stable class names. If behavior needs persistent identity, represent it as explicit data or a named strategy type.

TRACE IT | 23.6 Worked Java example

This validator composes named predicates and returns useful failures rather than a bare boolean.

JAVA

```
import java.util.List;
import java.util.function.Predicate;

public class ValidationDemo {
    record Rule<T>(String message, Predicate<? super T> test) {}

    static <T> List<String> validate(T value, List<Rule<T>> rules) {
        return rules.stream()
            .filter(rule -> !rule.test().test(value))
            .map(Rule::message)
            .toList();
    }

    public static void main(String[] args) {
        List<Rule<String>> rules = List.of(
            new Rule<>("must not be blank", text -> !text.isBlank()),
            new Rule<>("must be at most 12 characters", text -> text.length() <= 12));

        System.out.println(validate("a very long value", rules));
    }
}
```

`Predicate<? super T>` lets a rule consume `T` using a predicate defined for `T` or a supertype. Each rule also carries durable descriptive data rather than requiring introspection of the lambda.

TRACE IT | 23.7 Execution or memory walkthrough

The two lambda expressions are target-typed as `Predicate<? super String>` through record construction and list inference. The list stores references to two immutable rule records, each of which holds a message and predicate reference.

`validate` creates a lazy stream pipeline. At terminal `toList`, each rule enters the filter. The blank rule passes its test, so negation makes the filter false and its message is excluded. The length rule returns false; negation includes it. `Rule::message` is an unbound instance reference whose input is a Rule receiver. `toList` returns an unmodifiable result list containing the one failure message.

The implementation may allocate or reuse lambda objects, but no correctness depends on their identity. The captured lambdas here are stateless.

23.8 Complexity and performance

Creating or invoking functional behavior is conceptually constant overhead; body cost dominates. Validation is $O(r)$ predicate invocations and $O(f)$ result space for r rules and f failures. Predicate order can short-circuit when explicitly composed, while this implementation intentionally evaluates every rule to collect all messages.

Generic standard interfaces can box primitive arguments and results. Primitive specializations reduce that cost on numeric hot paths. Deep chains may affect stack traces and debugging clarity even if the JIT inlines them.

HotSpot note: Java compilers commonly emit `invokedynamic` lambda factories, and HotSpot can reuse stateless instances, inline bodies, and remove allocations. No identity, singleton, or allocation guarantee follows from those optimizations.

WATCH OUT | 23.9 Edge cases and common mistakes

- A lambda has no standalone type; it needs a target functional interface.
- Similar functional-interface overloads can be ambiguous.
- Captured locals must be effectively final; captured objects may still mutate.
- Lambda `this` is the enclosing `this`, unlike an anonymous class.
- A bound method-reference receiver is evaluated immediately.
- Method references can become unclear when overloads or parameter reordering are involved; use a lambda for clarity.
- Standard function interfaces do not declare checked exceptions.
- Side-effecting callbacks may be invoked multiple times by retries or stream behavior.
- Parallel callbacks can race on captured collections or counters.
- Function-object identity, class names, and serialization are not portable contracts.
- `Consumer.andThen` does not run the second action if the first throws and does not undo completed effects.
- Using `Function<T, Boolean>` instead of `Predicate<T>` introduces boxing and weakens intent.

23.10 Production engineering notes

Name nontrivial behavior. A local variable such as `isEligible` or a small strategy class gives stack traces and reviews more meaning than a large inline lambda. Put blocking, transactional, and retry semantics in callback documentation.

Pass immutable captured data where possible. When callbacks run concurrently, use thread-safe collaborators or confine state per invocation. Do not capture request-scoped objects in application-lifetime registries.

Functional interfaces are public APIs. Consider checked failures, variance, invocation count, ordering, nullability, thread, cancellation, and reentrancy. If callers need to log, persist, or configure a policy, pair executable behavior with explicit metadata or use a named domain type.

TRY IT | 23.11 Interview questions and model answers

Is a lambda an anonymous inner class?

No. Both can implement a functional role, but lambda `this` is lexical, capture and typing rules differ, and the runtime representation is unspecified. An anonymous class introduces a distinct class body and its own `this`.

What is target typing?

The assignment, parameter, cast, or return context supplies the functional interface whose single abstract method determines lambda parameter and result types. Without a target, a lambda expression cannot stand alone.

Why must captured locals be effectively final?

The lambda captures their values and may outlive the method frame. Preventing reassignment preserves clear value capture. It does not freeze objects reached through captured references.

How does `String::length` work as a function?

It is an unbound instance method reference. The function's first and only `String` argument becomes the receiver, and the result is that receiver's `length`.

Are lambdas thread-safe?

Not inherently. A stateless lambda can be safely invoked concurrently if its called operations are safe. A lambda capturing mutable state has the same synchronization and race concerns as any other code.

TRY IT | 23.12 Exercises

- 1 Write examples of static, bound, unbound, constructor, and array-constructor references.
- 2 Demonstrate the different meaning of `this` in a lambda and anonymous class.
- 3 Refactor ambiguous functional overloads into unambiguous method names.
- 4 Adapt a checked `IOException`-throwing supplier to a domain result without losing cause.
- 5 Find a lambda capture that retains a request object in a singleton registry and redesign it.
- 6 Add fail-fast and collect-all modes to `ValidationDemo`, documenting invocation counts.

RECAP | 23.13 Chapter summary

Lambdas and method references implement target functional interfaces; they are not standalone dynamically typed functions. Capture copies effectively final local values, while object mutability and thread safety remain ordinary concerns. Method references cover static, bound, unbound, and constructor forms, with bound receivers evaluated at creation. Functional API contracts must define exceptions, invocation count, ordering, threading, side effects, and ownership, not only parameter types.

TRY IT | 23.14 Revision checklist

- ☐ I can determine whether an interface is functional.
- ☐ I understand lambda target typing and overload ambiguity.
- ☐ I can explain local capture and lexical `this`.
- ☐ I know all principal method-reference forms.
- ☐ I use standard and primitive functional interfaces appropriately.
- ☐ I handle checked failures at an explicit boundary.
- ☐ I do not depend on lambda identity, class shape, or serialization.
- ☐ I document callback side effects, repetition, threading, and cancellation.

SDE-2 CORE CHAPTER

Chapter 24 - Java 17 and Java 21 Language and Platform Features

24.1 Learning objectives

By the end of this chapter, you should be able to:

- identify the high-impact permanent features introduced in Java 17 and Java 21;
- distinguish permanent, preview, and incubator status as of each release;
- compile and run preview or incubator code with appropriate controls;
- evaluate virtual threads, sequenced collections, pattern matching, and platform changes; and
- plan a Java 17 to Java 21 migration without confusing language compatibility with operational readiness.

WHY IT WORKS | 24.2 Why this matters at SDE-2

Java 17 and Java 21 are common long-term-support deployment baselines, although support duration is a vendor distribution policy rather than a Java language property. SDE-2 engineers are expected to know which newer constructs improve a service and which are experimental in a given release.

A migration is not complete because source compiles. Strong encapsulation can break reflection, virtual threads can change workload shape, and preview class files require runtime flags.

WHY IT WORKS | 24.3 First-principles model

Java evolves through language, library, JVM, tooling, security, and platform changes. A feature has a status within a specific release:

- **Permanent:** standard in that release and does not require preview flags.
- **Preview:** fully specified for feedback but not permanent; source and class files require opt-in and may change or disappear.
- **Incubator:** non-final API delivered in a `jdk.incubator.*` module for experimentation; compatibility is not promised.

Preview in one release does not imply the same syntax or API in another. Code must be judged against its exact source release. A feature finalized in Java 21 is ordinary Java 21 even if its ancestors were preview in earlier releases.

Specification boundary: "LTS" is not a Java Language Specification status. Vendors decide support offerings and timelines. Permanent, preview, and incubator status is release-specific; never describe a preview API as standardized merely because a production JDK contains it.

24.4 Core terminology

- **Feature release:** numbered Java platform release with its own source/class-file level.
- **LTS:** vendor commitment to extended maintenance for selected releases.
- **Preview feature:** opt-in language or VM feature intended for real-world feedback.
- **Incubator module:** non-final API module shipped for experimentation.
- **Strong encapsulation:** enforcement of module boundaries around JDK internals.
- **Virtual thread:** lightweight Java thread scheduled by the runtime rather than permanently bound one-to-one to an OS thread.
- **Sequenced collection:** collection with defined encounter order and first/last/reversed operations.
- **Pattern matching:** combined testing and extraction based on value shape or type.
- **Generational ZGC:** Z Garbage Collector mode exploiting object-age behavior.
- **Migration baseline:** source, target bytecode, runtime, dependencies, tools, and operations agreed for deployment.

24.5 Detailed mechanics

Java 17 permanent features

Java 17 made sealed classes and interfaces permanent. They use `sealed`, `permits`, `final`, and `non-sealed` to define controlled direct inheritance. Pattern matching for `instanceof`, records, text blocks, and switch expressions were already permanent before 17 and are part of a practical Java 17 language baseline.

Java 17 restored always-strict floating-point evaluation. The older `strictfp` distinction became unnecessary for ordinary source semantics: floating-point expressions use consistently strict behavior. Existing `strictfp` code remains source-compatible but the modifier is no longer needed for this purpose.

Java 17 strongly encapsulated JDK internals, with limited critical exceptions. Applications depending on `sun.*` members or illegal deep reflection may fail and should migrate to supported APIs. `--add-opens` can be a temporary operational bridge, not a durable application API.

Java 17 preview and incubator status

As of Java 17:

- Pattern matching for `switch` was **preview**. It required preview flags, and its guarded-pattern syntax and null semantics evolved before finalization.
- The Foreign Function and Memory API was **incubator**, in `jdk.incubator.foreign`.
- The Vector API was in its **second incubator** round, in `jdk.incubator.vector`.

Do not write Java 21 `when` guards and call them Java 17 syntax. Do not assume Java 17 incubator foreign-memory source migrates unchanged to the later `java.lang.foreign` API.

Preview compilation and execution typically use matching JDKs:

BASH

```
javac --enable-preview --release 17 Example.java
java --enable-preview Example
```

Incubator modules require explicit module resolution, for example:

BASH

```
javac --add-modules jdk.incubator.vector VectorExample.java
java --add-modules jdk.incubator.vector VectorExample
```

Build tools, tests, IDEs, packaging, and runtime launchers all need consistent configuration. Preview class files are marked so a runtime does not silently run them without preview opt-in.

Java 21 permanent language and collection features

Java 21 finalized pattern matching for `switch`. Type patterns, explicit null cases, exhaustive sealed-hierarchy coverage, and `when` guards allow declarative variant handling. It also finalized record patterns, including nested deconstruction. Neither requires `--enable-preview` on Java 21.

JAVA

```
sealed interface Event permits Login, Logout {}
record Login(String user) implements Event {}
record Logout(String user) implements Event {}

static String user(Event event) {
    return switch (event) {
        case Login(String name) -> name;
        case Logout(String name) -> name;
    };
}
```

Java 21 introduced sequenced collection interfaces: `SequencedCollection`, `SequencedSet`, and `SequencedMap`. They provide uniform first, last, and reversed operations for ordered collections. Existing types such as lists, deques, linked hash sets, sorted sets, linked hash maps, and sorted maps participate according to their contracts.

JAVA

```
java.util.List<String> queue = new java.util.ArrayList<>();
queue.addLast("second");
queue.addFirst("first");
System.out.println(queue.getFirst());      // first
System.out.println(queue.reversed());      // [second, first]
```

`reversed()` is generally a reverse-ordered view, not an independent deep copy. Mutability and write-through behavior depend on the backing collection's contract.

Virtual threads in Java 21

Virtual threads became permanent in Java 21 after previews in Java 19 and 20. They implement the `Thread` API and are well suited to high-concurrency workloads that spend substantial time blocking on I/O. They preserve a straightforward thread-per-task programming model.

Virtual threads are not automatically faster for CPU-bound work and do not remove downstream capacity limits. Database pools, HTTP peers, file descriptors, memory, and rate limits still need bounds. Do not pool virtual threads merely to reduce thread count; use one per task, then bound the scarce resource itself.

As of Java 21, blocking while holding a monitor in certain operations or executing native/foreign calls can pin a virtual thread to its carrier, reducing scalability. Pinning is a performance condition, not a correctness failure. Measure with Java Flight Recorder and relevant diagnostics, keep synchronized regions short, and avoid holding locks across blocking I/O.

Thread-local values work, but creating millions of virtual threads can make large or mutable thread-local state expensive. Prefer explicit context propagation where practical; scoped values were preview in Java 21 as a structured alternative.

Java 21 JVM and operational changes

Generational ZGC became available in Java 21. It separates young and old generations to exploit the observation that most objects die young. In Java 21 it is selected with ZGC plus the generational option; collector defaults and flag evolution are release-specific, so validate exact deployment commands against the target JDK.

Virtual-thread observability requires tools that understand large thread populations. Do not assume one platform thread per request in dashboards or capacity formulas.

Java 21 preview and incubator status

As of Java 21, the following were **preview**, not permanent:

- String Templates, first preview;
- Unnamed Patterns and Variables, first preview;
- Unnamed Classes and Instance Main Methods, first preview;
- Scoped Values, preview;
- Structured Concurrency, preview; and
- Foreign Function and Memory API, third preview.

The Vector API was in its **sixth incubator** round, not preview or permanent.

String templates combined literal text, expressions, and a processor; preview syntax such as `STR."Hello \{name}"` did not make SQL safe automatically. Unnamed patterns use `_` for intentionally unused bindings. Scoped values provide bounded context, while structured concurrency groups related subtasks and their cancellation. FFM moved to the `java.lang.foreign` API family in Java 21 and was not source-compatible with Java 17's incubator API. All remain subject to their status above.

Migration and compilation boundaries

`--release 17` compiles against the documented Java 17 API and emits compatible class files even when the compiler itself is newer. Merely using `-source 17 -target 17` can accidentally compile against newer boot APIs in some workflows; prefer `--release` when targeting an older platform.

Upgrade in layers:

- 1 inventory JDK distributions, CPU/OS support, containers, agents, flags, build tools, and libraries;
- 2 run the existing Java 17 source and bytecode on Java 21 in staging;
- 3 remove illegal internal API/reflection dependencies and obsolete JVM flags;
- 4 update CI, static analysis, test frameworks, profilers, and runtime images;
- 5 establish behavioral and performance baselines; then
- 6 adopt Java 21 source features deliberately.

Runtime and source-language upgrades need not occur in one deployment.

TRACE IT | 24.6 Worked Java example

This Java 21 program combines permanent record patterns, pattern switch, and virtual threads. It uses no preview feature.

JAVA

```
import java.util.List;
import java.util.concurrent.Executors;

public class Java21Demo {
    sealed interface Request permits Read, Write {}
    record Read(String key) implements Request {}
    record Write(String key, String value) implements Request {}

    static String execute(Request request) {
        return switch (request) {
            case Read(String key) -> "read " + key;
            case Write(String key, String value) when value.isBlank() ->
                "reject blank write to " + key;
            case Write(String key, String value) -> "write " + key + "=" + value;
        };
    }

    public static void main(String[] args) throws Exception {
        List<Request> requests = List.of(new Read("a"), new Write("b", "2"));
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            var futures = requests.stream()
                .map(request -> executor.submit(() -> execute(request)))
                .toList();
            for (var future : futures) {
                System.out.println(future.get());
            }
        }
    }
}
```

Compile it with `javac --release 21 Java21Demo.java`; no `--enable-preview` is needed.

TRACE IT | 24.7 Execution or memory walkthrough

The two records are final permitted implementations. The request list preserves encounter order. Each submitted callable is assigned its own virtual thread by the executor. The runtime schedules virtual threads onto carrier platform threads and can unmount them around supported blocking operations.

The Read task matches and deconstructs its record arm. The Write task first tests the guarded blank arm; the guard is false, so it matches the unguarded Write arm. The main thread obtains futures in list order, so printing is deterministic here even if task completion order differs.

Closing the executor waits for submitted tasks under its `ExecutorService` close contract. A production workflow still needs deadlines, cancellation, partial-failure policy, and downstream concurrency bounds.

24.8 Complexity and performance

New syntax does not change underlying algorithmic complexity. Pattern matching replaces explicit tests and casts with compiler-checked forms. Sequenced view operations are typically $O(1)$, but exact operation complexity follows each collection implementation.

Virtual threads reduce the resource cost of large numbers of blocking threads; they do not reduce CPU work or remote latency. Total memory still scales with live task state. Generational ZGC trades CPU, throughput, footprint, and pause characteristics differently from other collectors and must be load-tested with the service's allocation profile.

HotSpot note: Carrier scheduling, virtual-thread continuation representation, JIT decisions, ZGC implementation, and diagnostic event details are HotSpot behavior. Code should rely on Thread and API contracts, while capacity plans should measure the exact JDK build and flags deployed.

WATCH OUT | 24.9 Edge cases and common mistakes

- LTS is a vendor support designation, not a different language edition.
- Preview source and runtime must use matching release configuration.
- Java 17 pattern switch is preview; Java 21 pattern switch is permanent and uses final `when` guard syntax.
- Record patterns are permanent in 21 and unavailable in 17.
- String templates, scoped values, structured concurrency, unnamed features, and FFM are preview in 21.
- The Vector API is incubator in both releases discussed, at different iterations.
- Virtual threads do not justify unbounded database or network load.
- Long blocking operations while holding monitors can pin carriers in Java 21.
- `reversed()` commonly returns a view, so backing mutations can be visible.
- Strong encapsulation means a successful `--add-opens` workaround is migration debt, not a public contract.
- New JDK execution does not guarantee old agents, flags, or profilers remain compatible.

24.10 Production engineering notes

Define one approved JDK distribution and patch policy per environment. Record runtime version, flags, container limits, collector, and agents in deploy metadata. Test upgrades with production-like traffic and compare latency percentiles, CPU, allocation, GC, thread pinning, connection pools, and error rates.

Use virtual threads where blocking code dominates and simpler thread-per-request structure helps. Keep concurrency controls at scarce resources, propagate cancellation, and test every dependency for blocking and thread-local assumptions. Avoid a wholesale asynchronous-to-virtual rewrite without comparative evidence.

Keep preview features behind experiments or internal modules unless the organization accepts upgrade churn and flags across the full toolchain. Never publish a reusable Java 21 library that silently requires preview class files. For permanent features, adopt style guidance so patterns and record deconstruction improve clarity rather than compressing complex business rules.

TRY IT | 24.11 Interview questions and model answers

Which major language features are permanent in Java 17?

Sealed classes became permanent in 17. Records, text blocks, switch expressions, and pattern matching for `instanceof` had finalized earlier and are available in the Java 17 baseline. Pattern switch itself is only preview in 17.

What became permanent in Java 21?

Pattern matching for switch, record patterns, virtual threads, and sequenced collections are permanent Java 21 features or APIs. They require no preview flag.

Are virtual threads faster than platform threads?

Not inherently. They improve scalability and simplify code for many concurrent blocking tasks by reducing per-thread resource cost. CPU-bound throughput remains bounded by cores, and downstream capacity still needs limits.

What is the difference between preview and incubator?

Preview features are platform language, VM, or API features that require explicit opt-in and may evolve. Incubator APIs live in `jdk.incubator` modules, require module resolution, and are also non-final with intentionally weak compatibility promises.

How would you migrate from 17 to 21 safely?

First run existing source on a Java 21 runtime after updating dependencies, tools, agents, flags, and encapsulation workarounds. Establish functional and performance baselines. Then adopt source features and virtual threads incrementally, with concurrency limits, diagnostics, and rollback.

TRY IT | 24.12 Exercises

- 1 Classify every feature in this chapter as permanent, preview, or incubator in Java 17 and Java 21.
- 2 Compile the Java 21 demo without preview flags and verify its class-file target through `javap -verbose`.
- 3 Convert an executor-based blocking service to virtual threads while retaining a semaphore around a 20-connection database pool.
- 4 Replace first/last list indexing utilities with sequenced operations and test empty behavior and reversed-view mutation.
- 5 Inventory a Java 17 service for `--add-opens`, internal JDK APIs, agents, removed flags, and old build plugins.
- 6 Design a benchmark and load-test plan comparing platform-thread pools, virtual threads, and existing async code for one real I/O workload.

RECAP | 24.13 Chapter summary

Java 17 provides a mature baseline with permanent sealed types and strong encapsulation, while its pattern switch is preview and FFM and Vector APIs are incubating. Java 21 permanently adds pattern switch, record patterns, sequenced collections, and virtual threads, alongside operational advances such as generational ZGC. Several Java 21 features remain preview, and Vector remains incubator. Safe adoption separates runtime migration from source migration and validates dependencies, flags, concurrency, observability, and workload performance.

TRY IT | 24.14 Revision checklist

- ☐ I distinguish vendor LTS policy from Java feature status.
- ☐ I can name permanent Java 17 and Java 21 language features.
- ☐ I accurately label preview and incubator features in each release.
- ☐ I know the compile and runtime controls for preview and incubator code.
- ☐ I understand virtual-thread strengths, limits, pinning, and resource bounds.
- ☐ I understand sequenced collection views and Java 21 pattern syntax.
- ☐ I can explain strong encapsulation and migration debt from `add-opens`.
- ☐ I can plan a measured Java 17 to 21 rollout before adopting new syntax.

JAVA FOUNDATIONS TO ADVANCED ENGINEERING

Part IV - Collections, Streams, and I/O

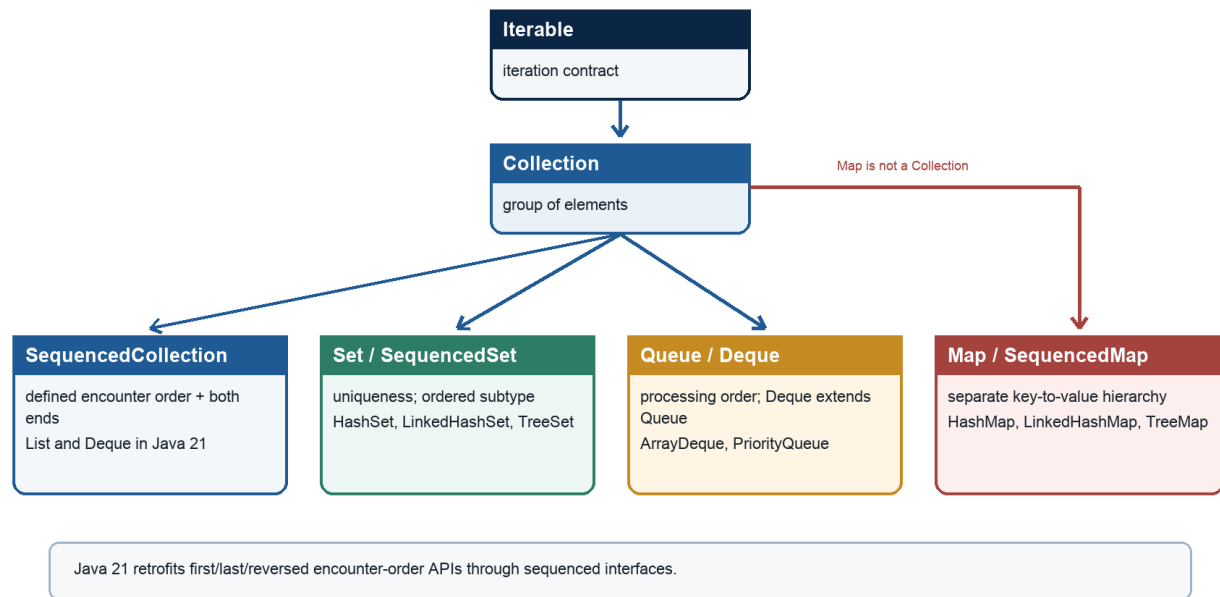
A focused part of the complete SDE-2 preparation guide

SDE-2 CORE CHAPTER

Chapter 25 - Collections Framework Architecture

Collections framework map

Java 21 contracts: select semantics first, then workload and implementation



Java Foundations to Advanced Engineering

Figure 25.1 - Collections framework semantic map

25.1 Learning objectives

By the end of this chapter, you should be able to:

- distinguish collection interfaces, implementations, algorithms, and views;
- choose between `List`, `Set`, `Queue`, `Deque`, and `Map` from behavioral requirements;
- explain optional operations, iteration order, mutability, and fail-fast iteration;
- reason about structural modification, backed views, equality, and bulk operations;
- state complexity as an implementation property, not an interface promise; and
- design collection-facing APIs that preserve invariants and ownership boundaries.

WHY IT WORKS | 25.2 Why this matters at SDE-2

Collections sit in nearly every backend path: request aggregation, caches, indexes, batching, deduplication, scheduling, and persistence mapping. At SDE-2, naming a familiar class is not enough. You are expected to identify the required semantics first, defend complexity claims, understand aliasing, and notice when an innocent view or mutable key creates a correctness defect.

An interview question such as "Which collection would you use?" is usually testing a decision process. Does order matter? Are duplicates allowed? Is lookup by key required? Are nulls valid? Is mutation concurrent? Is a sorted range query needed? The framework gives a vocabulary for those constraints.

WHY IT WORKS | 25.3 First-principles model

A collection is an object that owns or exposes a group of references. The Java Collections Framework separates four ideas:

- 1 Interfaces define behavioral contracts, such as uniqueness or positional access.
- 2 Implementations choose a data structure, memory layout, and performance profile.
- 3 Algorithms operate through interfaces, such as sorting or binary search.
- 4 Views expose a live projection of another object, such as a map's key set.

The principal hierarchy is:

TEXT

```
Iterable
  Collection
    List
    Set
      SortedSet
      NavigableSet
    Queue
    Deque

Map                                (not a Collection)
  SortedMap
  NavigableMap
```

`Map` is separate because it models key-value associations rather than individual elements. Its `keySet()`, `values()`, and `entrySet()` methods bridge into the `Collection` hierarchy through views.

Specification boundary: Interface contracts specify observable behavior. They generally do not promise array storage, linked nodes, hashing strategy, tree shape, amortized cost, or fail-fast detection. Complexity claims belong to the documentation of a concrete implementation.

25.4 Core terminology

- **Element:** A reference stored by a collection. Collections do not contain primitive values directly; boxing creates wrapper objects when needed.
- **Structural modification:** A change that can alter iteration structure, usually adding or removing elements. Replacing a value may or may not be structural for a particular implementation.
- **Encounter order:** The order in which an iteration mechanism presents elements. Some collections define it; others do not.
- **Natural ordering:** Ordering defined by `Comparable`.
- **Optional operation:** An interface method that an implementation may reject with `UnsupportedOperationException`.
- **View:** A projection backed by another object. Changes may be visible in both directions.
- **Snapshot:** An independent representation of state at a point in time.
- **Unmodifiable:** Mutation through that reference is rejected. It does not necessarily mean the underlying data or elements are immutable.
- **Immutable:** State cannot change after construction, including through aliases permitted by the abstraction.
- **Fail-fast iterator:** An iterator that attempts to detect unsupported concurrent structural modification and throw `ConcurrentModificationException`.

25.5 Detailed mechanics

Selecting the abstraction

Use a `List` when position, encounter order, or duplicates are meaningful. Use a `Set` for membership and uniqueness. Use a `Queue` when processing order matters and elements enter and leave through queue operations. Use a `Deque` for both ends or stack behavior. Use a `Map` when a unique key identifies a value.

Program parameters and return types to the narrowest useful interface. A method that only iterates can accept `Iterable<T>` or `Collection<T>` rather than `ArrayList<T>`. Do not hide an important semantic property, however: accepting `Set<T>` communicates uniqueness, while accepting `Collection<T>` does not.

Optional operations and capability

`Collection` includes methods such as `add`, `remove`, and `clear`, but implementations can reject them. `List.of(...)` produces an unmodifiable list. `Arrays.asList(array)` is fixed-size: `set` works, but size-changing methods do not. This design lets algorithms use a common interface, but it means the static type alone does not prove mutability.

Java has no standard type-level distinction between mutable and read-only collections. Document ownership and capability explicitly. Prefer immutable copies at public boundaries when callers should not mutate state:

JAVA

```
final class RoutingTable {
    private final List<String> backends;

    RoutingTable(Collection<String> backends) {
        this.backends = List.copyOf(backends);
    }

    List<String> backends() {
        return backends;
    }
}
```

`List.copyOf` also rejects null elements. That is useful only if null rejection matches the domain contract.

Iterators and structural modification

An `Iterator` is a cursor with `hasNext`, `next`, and optional `remove`. Enhanced `for` normally uses it. Removing directly from a typical collection during iteration can invalidate cursor state:

JAVA

```
for (String id : ids) {
    if (id.isBlank()) {
        ids.remove(id); // usually wrong
    }
}
```

Use `Iterator.remove`, `removeIf`, or a separate result collection. The iterator's own removal operation updates both the structure and its bookkeeping.

HotSpot note: Common OpenJDK collections maintain a modification counter and compare it with an iterator's expected value. This is implementation detail and detection is best-effort. `ConcurrentModificationException` is a bug signal, not a synchronization mechanism. Exact fields and detection points are version-sensitive.

Views and aliasing

`map.keySet()`, `map.values()`, and `map.entrySet()` are normally backed views. Removing a key from the key set removes the mapping. An entry's `setValue` can update the map when supported.

`list.subList(from, to)` is also a backed view. A structural change to the parent outside the sublist can make later sublist operations fail.

Wrappers from `Collections.unmodifiableList(source)` are read-only views, not copies. If another alias mutates `source`, the wrapper reflects it. By contrast, `List.copyOf(source)` returns an unmodifiable list whose membership is not backed by later source changes. Neither operation freezes mutable element objects.

Equality and bulk operations

`List.equals` is order-sensitive. `Set.equals` compares membership independent of order. `Map.equals` compares mappings. These contracts enable equality across implementations: an `ArrayList` can equal a `LinkedList` with the same sequence.

Bulk methods include `addAll`, `removeAll`, `retainAll`, `containsAll`, `removeIf`, and `replaceAll`. Their apparent single-call form does not imply constant time. Cost depends on both receiver and argument. For example, removing every element found in an `ArrayList` argument can be quadratic when membership checks are linear. Converting the lookup side to a `HashSet` can improve the expected bound.

Null policy and element validity

Null support varies. Some general-purpose collections allow null; many queues, concurrent collections, sorted structures with natural ordering, and factory-created immutable collections reject it. A robust API validates domain values before selecting an implementation rather than relying on an incidental null policy.

TRACE IT | 25.6 Worked Java example

The following method groups unique order IDs by customer while preserving first-seen customer order and first-seen order ID order:

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

record Order(String id, String customerId) {}

final class OrderIndex {
    static Map<String, List<String>> build(Collection<Order> orders) {
        Map<String, Set<String>> working = new LinkedHashMap<>();

        for (Order order : orders) {
            if (order == null || order.id() == null || order.customerId() == null) {
                throw new IllegalArgumentException("order fields must be non-null");
            }
            working.computeIfAbsent(
                order.customerId(), ignored -> new LinkedHashSet<>())
```

JAVA (continued 2/2)

```
                .add(order.id());
            }

            Map<String, List<String>> result = new LinkedHashMap<>();
            working.forEach((customer, ids) ->
                result.put(customer, List.copyOf(ids)));
            return Map.copyOf(result);
        }

        public static void main(String[] args) {
            List<Order> input = new ArrayList<>();
            input.add(new Order("o-1", "c-7"));
            input.add(new Order("o-2", "c-8"));
            input.add(new Order("o-1", "c-7"));
            input.add(new Order("o-3", "c-7"));

            System.out.println(build(input));
        }
    }
}
```

One subtlety: `Map.copyOf` guarantees an unmodifiable map, but its iteration order is not specified to preserve the insertion order of the supplied `LinkedHashMap`. If public iteration order is part of the result contract, return an unmodifiable copy with an order-preserving implementation, for example `Collections.unmodifiableMap(new LinkedHashMap<>(result))`, and document the guarantee.

TRACE IT | 25.7 Execution or memory walkthrough

For the four inputs above:

- 1 `o-1/c-7` creates the first map entry and a set containing `o-1`.
- 2 `o-2/c-8` creates a second entry. The map's encounter order is now `c-7`, `c-8`.
- 3 The duplicate `o-1/c-7` reaches the existing set. `Set.add` returns `false`; membership remains unchanged.
- 4 `o-3/c-7` appends a second unique ID to the first customer's insertion-ordered set.
- 5 The conversion creates new lists, so clients cannot add or remove IDs through the result.

The working representation uses a map object, map entries, set objects, set entries, and references to existing strings. The result adds list storage and map entries. Peak memory includes both representations until the method returns and the working map becomes unreachable. The code copies collection structure, not `String` content; strings are immutable, so sharing them is safe.

25.8 Complexity and performance

Let n be the number of orders and u the number of unique customer-order pairs. With typical `LinkedHashMap` and `LinkedHashSet` behavior, building has expected $O(n)$ time and $O(u)$ space; producing the result costs $O(u)$. Hash collisions, resizing, and key methods affect constants and worst cases.

Interface-only complexity should be stated cautiously:

Operation	Interface guarantee	Common implementation example
<code>List.get(i)</code>	No bound	<code>ArrayList</code> : $O(1)$; <code>LinkedList</code> : $O(n)$
<code>Set.contains(x)</code>	No bound	<code>HashSet</code> : expected $O(1)$; <code>TreeSet</code> : $O(\log n)$
<code>Map.get(k)</code>	No bound	<code>HashMap</code> : expected $O(1)$; <code>TreeMap</code> : $O(\log n)$
iteration	Semantic order may vary	Usually $O(n)$, but hash-table capacity can affect traversal
<code>containsAll</code>	No bound	Depends on sizes and membership cost

Memory is often as important as asymptotic time. Node-based structures add per-element objects and pointers. Array-based structures may reserve unused capacity but provide locality. Measure representative workloads rather than extrapolating from Big-O alone.

WATCH OUT | 25.9 Edge cases and common mistakes

- Choosing a concrete type before defining uniqueness, ordering, and access semantics.
- Returning a mutable internal list and unintentionally granting write access.
- Assuming an unmodifiable wrapper is a snapshot or that immutable membership makes elements immutable.
- Depending on `HashMap` or `HashSet` iteration order.
- Modifying a collection directly while iterating over it.
- Treating `ConcurrentModificationException` as guaranteed detection of a race.
- Forgetting that `subList` and map collection views are backed by their owner.
- Passing a collection to itself in a bulk operation whose behavior is undefined or surprising.
- Using mutable objects whose `equals`, `hashCode`, or ordering fields change while stored.
- Assuming every implementation permits null or mutation.
- Calling `size()` repeatedly on a nonstandard collection without checking whether it is cheap.
- Confusing `Collection` with `Collections`; the latter is an algorithm and wrapper utility class.

25.10 Production engineering notes

Define collection contracts in API documentation: order, duplicates, null policy, mutability, snapshot versus live view, thread safety, and expected scale. A return type of `List<T>` communicates sequence but not ownership.

Defensively copy inputs when the component must own stable membership. For very large inputs, copying can be costly; an explicit ownership-transfer convention or immutable persistent data structure may be more appropriate. Never expose a lazily changing view across layers unless that behavior is deliberate.

Avoid sharing ordinary mutable collections across threads without a synchronization policy. A synchronized wrapper makes individual calls mutually exclusive, but compound actions such as "check then add" still need one lock scope. Concurrent collections have different iteration and atomicity contracts and should be selected explicitly.

Prefer domain operations over raw collection exposure. `routingTable.chooseBackend()` protects invariants better than `routingTable.backends().remove(0)`. Validate capacity and input size where untrusted requests can cause large allocations. Instrument unusually large collection sizes and expensive conversions in latency-sensitive paths.

TRY IT | 25.11 Interview questions and model answers

Why is `Map` not a subtype of `Collection`?

A collection models individual elements, while a map models key-value mappings with key uniqueness. `Map` exposes collection views for keys, values, and entries, but operations such as adding one arbitrary element do not fit its contract.

What does fail-fast mean?

It means an iterator may detect an unsupported structural modification and throw `ConcurrentModificationException` promptly. It is best-effort behavior in common implementations, not a thread-safety guarantee or a correctness mechanism.

What is the difference between an unmodifiable view and an immutable copy?

An unmodifiable view rejects mutation through that reference but can reflect mutations through another alias. An immutable copy has independent, unchangeable membership. In both cases, referenced element objects can still be mutable unless separately constrained.

Why accept an interface rather than `ArrayList`?

It minimizes coupling and admits any implementation satisfying the required behavior. The chosen interface should still express semantics: `Set` is preferable to `Collection` when uniqueness is required.

Can interface type tell you complexity?

Usually no. `List.get` can be constant or linear depending on implementation. State the concrete type and assumptions whenever making a complexity claim.

How do backed views affect correctness?

They create aliases. Mutating the owner can change the view, and supported mutations through the view can change the owner. They are useful for efficient projections but dangerous if callers assume snapshots.

TRY IT | 25.12 Exercises

- 1 Design a method signature for returning active feature flags in deterministic order without allowing callers to mutate membership. State every contract not captured by the type.
- 2 Predict the result of removing an entry through `map.entrySet().iterator().remove()` and explain why it differs from mutating the map directly during iteration.
- 3 Compare `Collections.unmodifiableList(source)`, `List.copyOf(source)`, and `new ArrayList<>(source)` for mutability, null handling, and aliasing.
- 4 Rewrite a quadratic `removeAll` workflow so membership tests use a set. Give time and space bounds.
- 5 Create a small program that demonstrates a `subList` backed view. Then structurally modify the parent outside the view and record the observed behavior without treating it as a portable synchronization technique.

RECAP | 25.13 Chapter summary

The Collections Framework separates behavioral interfaces from data-structure implementations, reusable algorithms, and backed views. A sound choice begins with semantics: order, uniqueness, lookup, mutation, concurrency, and ownership. Complexity belongs to concrete implementations. Iterators, views, and optional operations make the framework flexible, but they also create important capability and aliasing boundaries. Production APIs should state those boundaries explicitly and copy or wrap data according to an intentional ownership model.

TRY IT | 25.14 Revision checklist

- ☐ I can sketch the `Collection` hierarchy and explain why `Map` is separate.
- ☐ I can choose among `List`, `Set`, `Queue`, `Deque`, and `Map` from requirements.
- ☐ I distinguish interface contracts from implementation complexity.
- ☐ I understand optional operations and common fixed-size or unmodifiable collections.
- ☐ I can explain structural modification and correct iterator removal.
- ☐ I can distinguish a view, an unmodifiable wrapper, a shallow copy, and deep immutability.
- ☐ I know the equality semantics of lists, sets, and maps.
- ☐ I document order, nulls, ownership, thread safety, and expected scale in APIs.

SDE-2 CORE CHAPTER

Chapter 26 - ArrayList, LinkedList, and List Trade-offs

26.1 Learning objectives

By the end of this chapter, you should be able to:

- explain the `List` contract and its positional model;
- derive the invariants of dynamic arrays and doubly linked lists;
- analyze access, insertion, removal, iteration, and memory costs;
- explain amortized growth without claiming a fixed growth formula;
- use list iterators, sublists, immutable factories, and array conversions safely; and
- select a list implementation from workload and locality, not folklore.

WHY IT WORKS | 26.2 Why this matters at SDE-2

Lists look simple, which makes them good interview probes. A candidate who says "linked lists are better for insertion" without discussing how the insertion position is found misses the important cost. Backend systems usually favor `ArrayList` because indexed storage, cache locality, low allocation count, and predictable iteration dominate. `LinkedList` remains useful for learning pointer invariants and can serve as a deque, but it is rarely the best default list.

At SDE-2 you should reason from operation distribution and data size. A batch assembled once and scanned many times has different needs from a cursor that repeatedly inserts at a known position. You should also recognize that a list can be modifiable, fixed-size, unmodifiable, or a backed range view despite sharing the same `List` type.

WHY IT WORKS | 26.3 First-principles model

A `List<E>` represents an ordered sequence indexed from `0` through `size - 1`. It permits duplicates and, depending on implementation, may permit null. Equality compares corresponding elements in sequence.

An array-backed list maintains conceptually:

TEXT

```
elements: [e0, e1, e2, e3, null, null, null, null]
size:      4
capacity:  8
```

```
Invariant: 0 <= size <= capacity
Valid logical elements occupy indexes [0, size).
```

A doubly linked list maintains nodes:

TEXT

```

null <- [prev|A|next] <-> [prev|B|next] <-> [prev|C|next] -> null
      first                                last

```

Invariant: `first.prev == null`, `last.next == null`,
and adjacent next/prev links agree.

The dynamic array makes location computation cheap: address is derived from base plus index. The linked representation must follow references to find an index. Once a linked node is already known, relinking neighbors is constant work, but the public `List` API usually supplies an index or search value, not a node handle.

Specification boundary: `List` promises positional behavior, not storage representation or complexity. `ArrayList` documents constant-time positional access and amortized constant-time append. Its exact capacity-growth policy is not a public contract.

26.4 Core terminology

- **Logical size:** Number of elements visible through the list.
- **Capacity:** Number of slots currently available in an array-backed representation.
- **Amortized cost:** Average cost per operation over a sequence, including occasional expensive operations.
- **Random access:** Efficient access by arbitrary index. `RandomAccess` is a marker used by some algorithms to adapt behavior.
- **Locality:** Tendency for nearby data to be located near each other in memory and used together.
- **Node:** A linked-list record containing an element and neighbor references.
- **Cursor:** A position between elements, represented by `ListIterator`.
- **Range view:** A live portion of a list, typically from `subList`.
- **Fixed-size list:** Element replacement is allowed, but size-changing operations are rejected.

26.5 Detailed mechanics

ArrayList operations

`get(i)` and `set(i, value)` validate the index and access one slot. Appending writes into the next slot if capacity remains. When full, the implementation allocates a larger array and copies references. Inserting at index `i` shifts the suffix `[i, size)` one slot right. Removing at `i` shifts the later suffix left and clears the obsolete final slot so the removed object is not retained through that array reference.

Suppose size equals capacity and append triggers a copy of n references. One append is $O(n)$, but if capacity grows geometrically, many preceding appends were cheap. Across a long append sequence, total copied references form a geometric series bounded by a constant multiple of n , yielding amortized $O(1)$ append.

`ensureCapacity` can reduce repeated resizing when a reasonable size estimate exists. `trimToSize` can reduce spare storage but may cost $O(n)$ and cause future regrowth. Neither should be called reflexively.

HotSpot note: OpenJDK has historically grown `ArrayList` capacity by roughly one-half of the old capacity in common cases, with boundary handling. This is version-sensitive implementation detail. Never write logic that depends on a particular capacity sequence.

LinkedList operations

Each element is stored in a node with references to predecessor and successor. Adding at either end updates a small number of links and endpoints. Finding index i walks from the nearer end in common implementations, still $O(n)$ asymptotically. Removing by value also searches before unlinking.

`LinkedList` implements both `List` and `Deque`. Its strongest use is often end-based queue behavior, but `ArrayDeque` usually offers better locality and lower allocation overhead for that role. A linked list does not reserve unused array capacity, yet it usually uses much more memory per element because every element has a node object and two link fields.

Iteration and ListIterator

For both structures, one full iterator traversal is $O(n)$. Indexed traversal can be disastrous for a linked list:

JAVA

```
for (int i = 0; i < list.size(); i++) {
    consume(list.get(i));
}
```

If `list` is a `LinkedList`, repeated `get` makes the loop $O(n^2)$. An enhanced `for` loop or iterator is $O(n)$.

`ListIterator` supports forward and backward traversal, reports neighboring indexes, and can add, remove, or replace at its cursor when supported. Its state rules matter: `remove` or `set` must follow a successful `next` or `previous`, and cannot immediately follow `add` or another `remove`.

Typical general-purpose list iterators are fail-fast when they detect structural mutation outside the iterator. Detection is best-effort and must not be used for synchronization or program control. Use the iterator's mutation methods or establish exclusive ownership.

List construction variants

- `new ArrayList<>()` is mutable and resizable.
- `new ArrayList<>(source)` is a shallow mutable copy.
- `Arrays.asList(array)` is a fixed-size view backed by the array. `set` changes the array.
- `List.of(elements...)` is unmodifiable and rejects null.
- `List.copyOf(source)` is unmodifiable and rejects null; it may reuse an already suitable immutable instance.
- `Collections.unmodifiableList(source)` is an unmodifiable view backed by `source`.
- `stream.toList()` returns an unmodifiable list, but code should not assume a concrete implementation.

Sublists

`list.subList(from, to)` uses a half-open range: `from` is included and `to` is excluded. It is normally backed by the parent. Clearing a sublist can efficiently remove a range from the parent. Structural modification of the backing list outside the view invalidates assumptions; subsequent behavior is generally not useful to depend upon and commonly produces `ConcurrentModificationException`.

If a stable independent range is needed, use `new ArrayList<>(list.subList(from, to))`.

Conversion to arrays

`toArray()` returns `Object[]`. `toArray(new String[0])` and `toArray(String[]::new)` produce a typed array. Modern JVMs optimize common zero-length-array idioms, so prefer clarity and measure if this conversion is actually hot. Conversion copies references and costs $O(n)$ space and time.

TRACE IT | 26.6 Worked Java example

This example keeps a sorted timeline in an `ArrayList` and inserts a new event using binary search:

JAVA (continued 1/2)

```
import java.time.Instant;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

record Event(Instant at, String id) {}

final class Timeline {
    private static final Comparator<Event> ORDER =
        Comparator.comparing(Event::at).thenComparing(Event::id);

    private final ArrayList<Event> events = new ArrayList<>();

    void add(Event event) {
        int result = java.util.Collections.binarySearch(events, event, ORDER);
        int insertionPoint = result >= 0 ? result : -result - 1;
        events.add(insertionPoint, event);
    }
}
```

JAVA (continued 2/2)

```
List<Event> between(Instant startInclusive, Instant endExclusive) {
    ArrayList<Event> result = new ArrayList<>();
    for (Event event : events) {
        if (event.at().isBefore(startInclusive)) {
            continue;
        }
        if (!event.at().isBefore(endExclusive)) {
            break;
        }
        result.add(event);
    }
    return List.copyOf(result);
}

List<Event> snapshot() {
    return List.copyOf(events);
}
}
```

The list invariant is that `events` is sorted by `ORDER`. The class, not callers, owns mutation. A real system with frequent arbitrary insertion may choose a tree index or append plus periodic sort; the example illustrates how search cost and movement cost differ.

TRACE IT | 26.7 Execution or memory walkthrough

Assume the list contains events at (10:00, a), (10:05, b), and (10:20, c). Adding (10:12, d) performs binary search:

TEXT

```
low=0 high=2, mid=1 -> 10:05 is smaller, low=2
low=2 high=2, mid=2 -> 10:20 is larger, high=1
search ends; insertion point=2
```

```
before: [10:00/a, 10:05/b, 10:20/c, empty]
shift:  [10:00/a, 10:05/b, 10:20/c, 10:20/c]
write:  [10:00/a, 10:05/b, 10:12/d, 10:20/c]
```

Binary search uses $O(\log n)$ comparisons, but insertion shifts $O(n)$ references in the worst case. Overall insertion remains $O(n)$. This distinction is a common interview trap.

The array holds references, not inline `Event` record payloads. `snapshot()` creates another list storage object containing the same immutable `Event` references. Because `Event` contains immutable components, shallow sharing is safe. If elements were mutable, the snapshot would protect membership only.

26.8 Complexity and performance

Operation	<code>ArrayList</code>	<code>LinkedList</code>
get or set by index	$O(1)$	$O(n)$
append	amortized $O(1)$	$O(1)$
add or remove at front	$O(n)$	$O(1)$
insert/remove at arbitrary index	$O(n)$ shift	$O(n)$ search plus $O(1)$ relink
insert/remove through positioned iterator	$O(n)$ shift	$O(1)$ relink
search by value	$O(n)$	$O(n)$
full iteration	$O(n)$	$O(n)$
auxiliary structure per element	array slot	node plus links

The table hides hardware effects. `ArrayList` traversal is usually faster because reference slots are contiguous, copying can use optimized bulk operations, and there are fewer allocations and indirections. `LinkedList` nodes increase garbage collection work and can miss CPU caches.

For small lists, constants dominate. For very large lists, the `int` index and array-size limits matter. Neither implementation should be treated as a disk-scale container. Benchmark representative data with the intended JDK and hardware.

WATCH OUT | 26.9 Edge cases and common mistakes

- Using `LinkedList` for frequent indexed reads, creating accidental quadratic behavior.
- Claiming linked insertion is $O(1)$ while ignoring the $O(n)$ search for a position.
- Using `remove(1)` on `List<Integer>` when intending to remove the value `1`; it removes index 1. Use `remove(Integer.valueOf(1))` for value removal.
- Assuming `Arrays.asList` is a normal resizable `ArrayList`.
- Exposing `subList` as a durable independent result.
- Forgetting half-open range boundaries and accepting `from > to` or invalid indexes.
- Retaining a tiny sublist backed by a very large custom or older representation; make an independent copy when retention is uncertain.
- Sorting a list and later mutating fields used by ordering without restoring the invariant.
- Treating `List.of(array)` as a list of array elements when a primitive array is passed; a primitive array is one reference element.
- Forgetting null rejection in immutable factories.
- Relying on exact `ArrayList` capacity growth.
- Using copy-on-write lists for write-heavy workloads; that concurrent implementation copies on mutation.

26.10 Production engineering notes

Default to `ArrayList` for general mutable sequence storage. Provide an expected capacity when reliable batch metadata exists, but cap estimates derived from untrusted input to avoid oversized allocation. Prefer `ArrayDeque` for stack and queue operations. Use `LinkedList` only when its measured workload and cursor or end operations justify its overhead.

Preserve invariants behind an API. If a list must remain sorted, do not return the mutable list. If duplicates are invalid, a `Set` may express the model better. If lookups by ID dominate, maintain a map rather than repeatedly scanning a list.

Bulk construction is frequently better than incremental middle insertion: append, validate, sort once, and publish an immutable snapshot. Keep lists local to a request when possible. Shared mutation requires synchronization or a concurrent design; an unmodifiable wrapper does not make concurrent reads safe while another alias mutates the backing list.

Watch memory retention in queues or batch buffers. Clear references when custom array structures remove elements. Standard implementations do this, but application-level lists can still retain large element graphs simply because a long-lived owner retains the list.

TRY IT | 26.11 Interview questions and model answers

Why is `ArrayList` usually faster than `LinkedList` for iteration?

Its reference slots are contiguous, so it uses fewer objects and indirections and has better cache locality. Both are $O(n)$, but their constants and memory behavior differ greatly.

Is insertion into `LinkedList` constant time?

Relinking is constant time once the node or iterator position is known. `add(index, value)` must locate the index and is therefore $O(n)$ overall.

How can append be both $O(n)$ and amortized $O(1)$?

An append that triggers resize copies existing references and costs $O(n)$. Geometric capacity growth makes resizes infrequent, so total work across many appends is linear and average cost per append is constant.

What does `subList` return?

A backed range view, not an independent copy. Supported mutations affect the parent, and external structural changes to the parent can invalidate the view.

What is the difference between size and capacity?

Size is the number of logical elements. Capacity is allocated slot count in an array-backed implementation. Capacity is not part of the `List` contract.

When would you deliberately choose `LinkedList`?

Only when a workload relies on constant-time end operations or insert/remove at a maintained iterator position and measurement shows it is appropriate. Even for deque use, I would compare `ArrayDeque` first.

TRY IT | 26.12 Exercises

- 1 Dry-run removal at index 2 from an array list of six elements. Count moved references and identify the slot that must be cleared.
- 2 Prove that indexed traversal of a linked list is $O(n^2)$ by summing traversal distances.
- 3 Implement a sorted list insertion method that places a duplicate after all equal values. State why binary search alone does not promise which equal item it finds.
- 4 Demonstrate aliasing between an array and `Arrays.asList(array)`. Contrast it with `new ArrayList<>(...)`.
- 5 Design a benchmark hypothesis comparing `ArrayList`, `LinkedList`, and `ArrayDeque` for removing from the front. Include allocation and warmup considerations.
- 6 Refactor a class that returns its mutable internal list into an ownership-safe API while preserving deterministic order.

RECAP | 26.13 Chapter summary

`List` defines ordered positional semantics, while implementations determine representation and cost.

`ArrayList` uses a resizable reference array, offering constant-time indexed access, amortized constant-time append, efficient traversal, and suffix movement for middle changes. `LinkedList` uses doubly linked nodes, offering constant-time relinking at known positions but linear positional access and high allocation overhead.

Views, factories, and iterators introduce mutability and aliasing differences that must be part of API design.

TRY IT | 26.14 Revision checklist

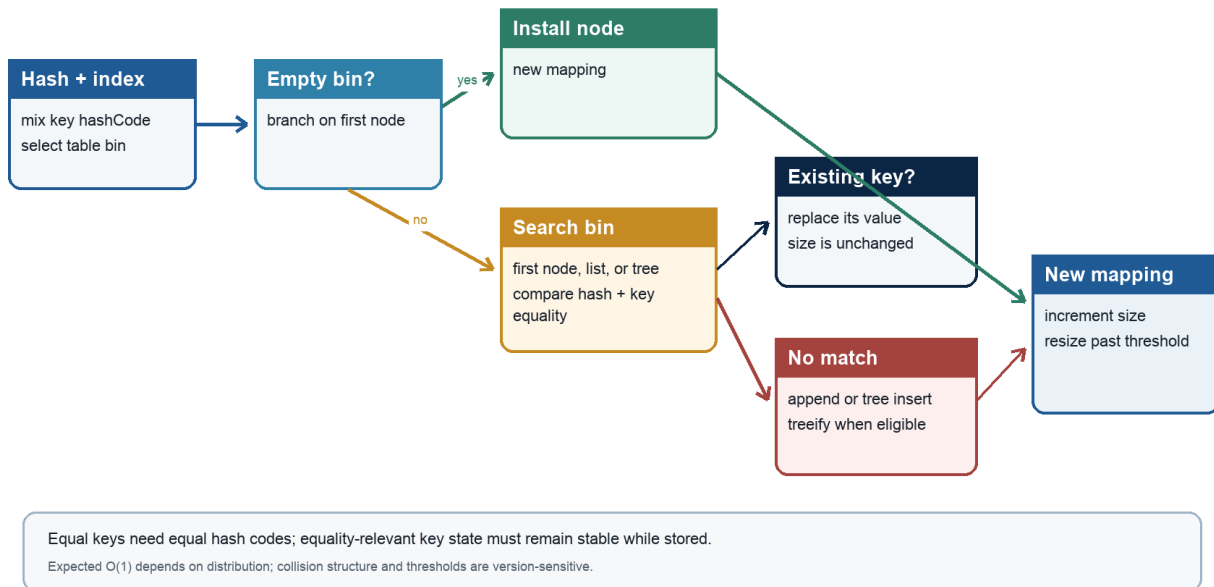
- ☐ I can state dynamic-array and doubly linked-list invariants.
- ☐ I can derive amortized append cost without relying on a growth factor.
- ☐ I include position-finding cost when analyzing linked insertion.
- ☐ I avoid indexed traversal for sequential-access lists.
- ☐ I know the behavior of `Arrays.asList`, `List.of`, `List.copyOf`, and unmodifiable wrappers.
- ☐ I can use `ListIterator` statefully and safely.
- ☐ I understand that `subList` is a backed half-open range.
- ☐ I can choose a list using access pattern, locality, allocation, and ownership.

SDE-2 CORE CHAPTER

Chapter 27 - HashMap, HashSet, and Hashing Internals

HashMap put path

Representative OpenJDK-style mechanics; not a full API contract



Java Foundations to Advanced Engineering

Figure 27.1 - Representative HashMap insertion path

27.1 Learning objectives

By the end of this chapter, you should be able to:

- derive hash-table lookup from equality and bucket selection;
- state the `equals` and `hashCode` contract and key-stability invariant;
- explain collisions, load factor, resizing, and expected complexity;
- describe a typical OpenJDK `HashMap` without presenting it as a platform guarantee;
- use `compute`, `merge`, views, and iteration safely; and
- identify security, concurrency, memory, and API-design risks around hash tables.

WHY IT WORKS | 27.2 Why this matters at SDE-2

Hash maps support indexes, caches, joins, deduplication, frequency counts, and graph representations. They are also a rich interview subject because correctness spans language-level equality, data-structure invariants, amortized analysis, and mutation discipline.

At SDE-2, "lookup is $O(1)$ " is incomplete. The useful answer is expected $O(1)$ under a suitable hash distribution and stable key behavior, with resizing and collision caveats. You should be able to diagnose a missing entry caused by a mutated key, distinguish absence from a null value, and choose an ordered, identity-based, weak-key, concurrent, or sorted alternative when semantics demand it.

WHY IT WORKS | 27.3 First-principles model

A hash table transforms a key into a candidate region rather than comparing it with every stored key. Conceptually:

TEXT

```
hashCode(key) -> mixed hash -> bucket index -> inspect candidates -> equals
```

The hash narrows the search; `equals` determines logical key identity. Different keys can have the same hash and must coexist as a collision. Equal keys must be directed to the same search region.

The central invariant is:

TEXT

```
For every stored entry (k, v), lookup using a key equal to k  
must search the bucket containing that entry.
```

That requires equal objects to have equal hash codes and requires fields used by equality and hashing to remain stable while the key is stored.

`HashSet<E>` uses the same membership idea without an application value. A typical implementation is backed by a hash map whose keys are set elements and whose values are a private marker.

Specification boundary: `Map` specifies unique keys and mapping behavior. `HashMap` specifies null support and general performance expectations in its API documentation, but bucket array shape, hash mixing, resize thresholds, tree bins, and constants are implementation details.

27.4 Core terminology

- **Hash code:** An `int` computed for an object and used to group possible matches.
- **Collision:** Distinct, non-equal keys assigned to the same bucket.
- **Bucket/bin:** A table position containing zero or more entries.
- **Capacity:** Number of table buckets in a typical hash-table representation.
- **Load factor:** Ratio controlling how full the table may become before resizing.
- **Threshold:** Entry count that triggers resizing, often capacity times load factor.
- **Rehash/resizing:** Allocating a larger table and redistributing entries.
- **Hash flooding:** Many keys deliberately or accidentally colliding.
- **Tree bin:** A collision bucket represented by a balanced tree in some implementations.
- **Key stability:** Requirement that equality and hash behavior not change while a key is stored.
- **Canonical key:** Stable value representation used as a map key, such as a normalized immutable identifier.

27.5 Detailed mechanics

Equality and hashing contract

`Object` establishes these practical rules:

- 1 If `a.equals(b)` is true, `a.hashCode() == b.hashCode()` must be true.
- 2 Unequal objects may share a hash code.
- 3 Repeated hash calls during one execution should be consistent while equality-relevant state is unchanged.

A record automatically derives value-based `equals` and `hashCode` from components, making records useful keys when components are themselves stable:

```
JAVA
```

```
record TenantUser(String tenantId, String userId) {}
```

If a mutable class uses `tenantId` and `userId` for hashing, changing either after insertion can strand the entry. The object remains physically in its old bucket while lookup computes a new bucket.

Lookup, insertion, and collision resolution

A typical `get(key)` computes a hash, maps it to an index, then checks candidate entries. It first compares hash values and then key equality. Correct comparisons account for identical references and null according to implementation policy.

On `put(key, value)`, a matching key replaces its value and returns the old value. If no matching key exists, a new entry is linked or otherwise inserted into the bucket. Map size changes only for a new key. A

`HashSet.add` similarly returns `false` when an equal element is already present.

Many hash tables use separate chaining: each bucket holds multiple entry nodes. Other hash tables use open addressing, but ordinary OpenJDK `HashMap` has traditionally used bucket chains with tree conversion under certain conditions.

Capacity, load factor, and resizing

A low load factor uses more buckets and generally shortens collision searches. A high load factor conserves table space but increases collisions. The default is intended as a general compromise, not a universal optimum.

When size crosses a threshold, a typical implementation allocates a larger bucket array and relocates or splits entries. The resize is $O(n)$, but geometric growth makes a long sequence of inserts expected amortized $O(1)$ each. Pre-sizing can avoid repeated growth when an estimate is reliable. Excessive pre-sizing wastes memory and can slow full iteration because iteration may scan buckets as well as entries.

HotSpot note: In current OpenJDK implementations, capacities are generally powers of two, index selection uses masked hash bits, and resizing often doubles capacity. Hash spreading mixes high bits into low bits. Details and helper methods can change by JDK release.

Tree bins

Long collision chains degrade lookup toward $O(n)$. Modern OpenJDK versions can transform a sufficiently populated bin into a red-black tree when the table is also large enough, giving $O(\log n)$ comparison depth for that bin under suitable conditions. Bins can later return to list form.

HotSpot note: Treeification and untreeification thresholds, minimum table capacity, tie-breaking, and node layouts are version-sensitive. The Java API does not guarantee tree bins or worst-case logarithmic lookup. Treat them as resilience in a particular implementation, not as permission to use poor hashes.

Null, absence, and defaulting

`HashMap` permits one null key and multiple null values. Therefore `get(key) == null` can mean no mapping or a mapping to null. Use `containsKey` when the distinction matters. Prefer avoiding null values in domain maps when absence has a clear meaning.

`getOrDefault` returns the mapped value even if that value is null; it uses the default only when no mapping exists. `putIfAbsent` treats a null mapping as absent for its operation. Read each method's contract carefully.

Compute and merge methods

`computeIfAbsent` is ideal for multimap assembly:

JAVA

```
index.computeIfAbsent(customerId, ignored -> new ArrayList<>()).add(order);
```

The mapping function should be short and should not structurally mutate the same map recursively. If it returns null, no mapping is recorded. `computeIfPresent` runs only for a present non-null mapping. `compute` considers both presence and absence; a null result removes the mapping. `merge(k, incoming, remapper)` inserts `incoming` when absent or combines it with the current non-null value; a null remapping result removes the key.

These methods make a compound update concise on an ordinary map but do not make that map thread-safe. Concurrent map implementations define stronger per-key atomic behavior for their overrides.

Views and iteration

`keySet`, `values`, and `entrySet` are backed views. Iterate `entrySet` when both key and value are needed; repeated `map.get(key)` is redundant. Removing through a view removes mappings. A `HashMap` defines no stable iteration order. Even if a run appears consistent, a resize, JDK change, input change, or hash change can alter it.

Typical iterators are fail-fast under detected unsupported structural modification. Replacing the value for an existing key is often nonstructural, but this is not a general concurrency contract. `Map.Entry` objects from iteration may be tied to the map and should not be retained as permanent independent records; use `Map.entry(k, v)` or a domain record for a snapshot pair.

TRACE IT | 27.6 Worked Java example

This frequency index uses an immutable composite key and `merge`:

JAVA

```
import java.util.HashMap;
import java.util.List;
import java.util.Map;

record Endpoint(String method, String path) {
    public Endpoint {
        if (method == null || path == null) {
            throw new IllegalArgumentException("endpoint fields are required");
        }
        method = method.toUpperCase(java.util.Locale.ROOT);
    }
}

record Request(String method, String path) {}

final class RequestCounter {
    static Map<Endpoint, Long> count(List<Request> requests) {
        Map<Endpoint, Long> counts = new HashMap<>();
        for (Request request : requests) {
            Endpoint key = new Endpoint(request.method(), request.path());
            counts.merge(key, 1L, Long::sum);
        }
        return Map.copyOf(counts);
    }

    public static void main(String[] args) {
        var requests = List.of(
            new Request("get", "/health"),
            new Request("GET", "/orders"),
            new Request("GET", "/health"));
        System.out.println(count(requests));
    }
}
```

Normalization is part of key construction, so equality semantics match the domain rule that HTTP method case is irrelevant here. The path is intentionally not normalized; whether `/orders/` equals `/orders` is an application decision, not a hash-table concern.

`Map.copyOf` prevents result membership changes and rejects null keys or values. It does not promise the iteration order of the source hash map, so output formatting and tests must not depend on order.

TRACE IT | 27.7 Execution or memory walkthrough

For the sample input:

- 1 `Endpoint("get", "/health")` becomes `("GET", "/health")`. Its hash selects a bucket. No equal key exists, so `merge` inserts count `1`.
- 2 `("GET", "/orders")` selects some bucket and is inserted with `1`.
- 3 A new `Endpoint("GET", "/health")` is a different reference but is record-equal to the first key and has the same hash. Lookup reaches the same bucket, `equals` succeeds, and `Long::sum` produces `2`.

Conceptually, with capacity eight:

TEXT

```
bucket 0: empty
bucket 1: [Endpoint(GET,/orders) -> 1]
bucket 2: empty
bucket 3: [Endpoint(GET,/health) -> 2]
bucket 4: empty
...
```

Actual indexes are deliberately omitted because hash spreading and table state are implementation-sensitive. The map stores one entry per unique endpoint, not one per request. Temporary equal key objects created for repeated endpoints become unreachable after lookup. The boxed `Long` values are replaced as counts grow because `Long` is immutable.

Now consider a broken mutable key:

JAVA

```
final class MutableKey {
    String id;
    MutableKey(String id) { this.id = id; }
    public boolean equals(Object other) {
        return other instanceof MutableKey k && id.equals(k.id);
    }
    public int hashCode() { return id.hashCode(); }
}
```

After `map.put(key, value)`, changing `key.id` changes its lookup hash. `map.get(key)` can return null even though iteration still reveals the same reference. This violates the table's key-stability assumption, not the map's implementation.

27.8 Complexity and performance

For n entries and a reasonable hash distribution:

Operation	Expected	Collision-degraded	Notes
get, put, remove	$O(1)$	up to $O(n)$ abstractly	tree bins may improve a particular implementation
containsKey	$O(1)$	up to $O(n)$	invokes hash/equality work
full iteration	$O(n + \text{capacity})$ typical	same form	oversized tables can hurt
resize	$O(n)$	$O(n)$	amortized across insertions
HashSet membership	same as backing hash table	same caveat	value is only a marker

Hash computation itself may not be constant in key size. `String.hashCode` is proportional to string length on first computation in many implementations, though caching and compact-string representation are implementation concerns. Expensive `equals` also increases collision cost.

Space is $O(n + \text{capacity})$ plus entry/node overhead. Load factor changes the trade-off between empty buckets and collision depth. `LinkedHashMap` adds ordering links. `IdentityHashMap` uses reference identity and a specialized representation. `EnumMap` uses enum ordinals and is typically compact. Choose semantics first, then measure.

WATCH OUT | 27.9 Edge cases and common mistakes

- Overriding `equals` without a consistent `hashCode`.
- Mutating equality-relevant key state after insertion.
- Assuming unequal keys require different hash codes.
- Assuming iteration order or using printed `HashMap` order in golden tests.
- Using `get(k) == null` to prove absence when null values are permitted.
- Calling `containsValue` as though it were a hashed lookup; it generally scans values.
- Performing expensive or recursive side effects in `computeIfAbsent`.
- Assuming ordinary `HashMap.compute` is atomic across threads.
- Pre-sizing from an untrusted claimed count and allocating excessive memory.
- Using arrays as value keys without content-based wrappers; arrays inherit identity equality.
- Returning mutable lists stored as map values and calling the outer map immutable.
- Expecting tree bins to fix adversarial equality or expensive hash functions.
- Sharing a `HashMap` for concurrent mutation without synchronization.
- Forgetting that `HashSet` uniqueness is exactly the equality definition of its elements.

27.10 Production engineering notes

Prefer immutable, small, domain-specific keys. Normalize once at the boundary and make normalization rules explicit. Avoid concatenated string keys when components can be represented by a record; concatenation risks ambiguity and repeated allocation.

Size maps from credible estimates, accounting for load factor rather than setting initial capacity equal to expected entries and assuming no resize. Avoid massive speculative capacities. Monitor cache/index cardinality, eviction, and skew; an unbounded map is often a memory leak by design.

For user-controlled keys, use current supported JDKs and validate request sizes. Hash flooding can convert an expected constant-time endpoint into CPU exhaustion. Tree bins help in common OpenJDK versions but do not replace input limits, timeouts, and observability.

Use `LinkedHashMap` when encounter or access order is a contract, `TreeMap` for ordered range queries, `ConcurrentHashMap` for shared concurrent mutation, `EnumMap` for enum keys, and identity maps only when reference identity is truly the model. A synchronized wrapper still requires external locking around compound iteration or multi-call invariants.

When publishing maps, specify whether values are deeply immutable. `Map.copyOf` freezes mappings, not mutable objects reachable from values. Be cautious with cached negative or null results; explicit wrapper types often make state clearer.

TRY IT | 27.11 Interview questions and model answers

How does `HashMap.get` work?

It computes a key hash, derives a candidate bucket, then checks entries in that bucket using hash and equality. Expected lookup is constant with good distribution; collisions require additional comparisons. Exact bucket and tree mechanics are implementation-specific.

Why must equal objects have equal hash codes?

The table uses the hash to choose where to search. If equal keys could select different regions, lookup could miss a logically identical stored key.

What happens when a key changes after insertion?

If equality or hashing changes, lookup searches using the new hash while the entry remains placed according to the old hash. Retrieval and removal can fail. Use immutable keys or stable identity fields.

What is a collision? Is it an error?

A collision is two unequal keys sharing a bucket or hash result. It is normal and must be resolved by equality checks. Excessive collisions hurt performance.

Why is `HashMap` lookup not simply guaranteed $O(1)$?

The API does not guarantee collision distribution, hash cost, or a particular bucket representation. Expected $O(1)$ assumes suitable hashing and load. Worst-case abstract lookup can be linear.

How is `HashSet` commonly implemented?

It is commonly backed by a `HashMap`, storing set elements as keys and a shared private marker as the map value. That is an OpenJDK-style implementation fact, while the set contract only guarantees uniqueness.

When would you use `computeIfAbsent`?

For lazy per-key initialization such as grouping values into lists. The function should be short, return the desired initial value, and avoid structurally modifying the same map.

TRY IT | 27.12 Exercises

- 1 Implement a correct immutable `Coordinate` key manually with `equals` and `hashCode`, then compare it with a record.
- 2 Dry-run three colliding keys through insert, replacement, lookup, and removal.
- 3 Write a program that demonstrates the mutable-key failure. Repair it by making the key immutable.
- 4 Build a word-frequency map using `merge`, then return entries sorted by frequency without relying on hash iteration order.
- 5 Given one million expected entries, explain how load factor, key size, entry overhead, and capacity affect memory. Identify what must be measured rather than guessed.
- 6 Compare an outer `Map.copyOf` containing mutable lists with a deeply unmodifiable result.

RECAP | 27.13 Chapter summary

Hash tables narrow key search through a hash and confirm identity through `equals`. Their correctness depends on consistent equality and hashing plus stable keys. Collision handling, capacity, load factor, and resizing produce expected constant-time operations under ordinary assumptions, not an unconditional guarantee. OpenJDK's power-of-two tables and tree bins are useful implementation knowledge but are version-sensitive. Production design requires bounded cardinality, deliberate key semantics, explicit order and null policies, and a concurrency strategy.

TRY IT | 27.14 Revision checklist

- ☐ I can state the `equals` and `hashCode` contract.
- ☐ I can derive lookup, insertion, collision handling, and resizing.
- ☐ I explain expected and amortized costs with assumptions.
- ☐ I know why mutable keys become unreachable by normal lookup.
- ☐ I distinguish absence from a null mapping.
- ☐ I can use `merge` and compute methods with their null semantics.
- ☐ I do not depend on `HashMap` or `HashSet` iteration order.
- ☐ I label tree bins, thresholds, hash spreading, and capacity shape as implementation details.
- ☐ I can select ordered, concurrent, enum, identity, or sorted alternatives by semantics.

SDE-2 CORE CHAPTER

Chapter 28 - TreeMap, TreeSet, Ordering, and Navigable Collections

28.1 Learning objectives

By the end of this chapter, you should be able to:

- distinguish sorted, encounter-ordered, and unordered collections;
- explain binary-search-tree and red-black-tree invariants;
- use `TreeMap`, `TreeSet`, `NavigableMap`, and `NavigableSet` operations correctly;
- reason about comparator consistency, mutable keys, ranges, and backed views;
- analyze logarithmic operations and ordered traversal; and
- choose a tree-based collection when navigation is required, not merely for deterministic output.

WHY IT WORKS | 28.2 Why this matters at SDE-2

Ordered maps and sets support time-window indexes, floor and ceiling lookup, leaderboards, routing ranges, and nearest-neighbor decisions. Interviews often ask for "the next event," "all keys in a range," or "the largest value no greater than x." A hash table cannot answer those efficiently without an additional scan or sort.

The difficult part is not recalling `TreeMap`. It is preserving an ordering that is total, stable, and consistent with the application's notion of identity. A comparator that treats two distinct keys as equal silently changes map membership. A range view that escapes its intended lifetime introduces aliasing. An SDE-2 engineer recognizes these as correctness contracts.

WHY IT WORKS | 28.3 First-principles model

A binary search tree stores one entry per node and maintains:

TEXT

```
all keys in left subtree < node key  
all keys in right subtree > node key
```

Here `<` is defined by a comparator or natural ordering. An ordinary binary search tree can become a chain under sorted insertion, producing linear operations. A balanced tree adds structural invariants that keep height logarithmic.

A red-black tree conceptually enforces:

- 1 Every node has a color, red or black.
- 2 The root is black.
- 3 Missing leaf sentinels are black.
- 4 A red node has no red child.
- 5 Every path from a node to a missing leaf contains the same number of black nodes.

These rules limit the longest root-to-leaf path to at most about twice the shortest, so height is $O(\log n)$. Insertions and removals restore invariants using recoloring and rotations.

Specification boundary: `TreeMap` and `TreeSet` promise sorted and navigable behavior with documented logarithmic basic operations. The API does not require a red-black tree specifically. OpenJDK's representation, node fields, and balancing code are implementation details.

28.4 Core terminology

- **Natural ordering:** Ordering provided by a type's `Comparable.compareTo`.
- **Comparator:** Object defining an external ordering through `compare(a, b)`.
- **Total order:** Every pair can be compared consistently, with transitivity and antisymmetry-like sign rules.
- **Ordering-equivalent:** `compare(a, b) == 0`.
- **Consistent with equals:** Ordering-equivalent exactly when `a.equals(b)` for relevant values.
- **Floor:** Greatest element less than or equal to a target.
- **Lower:** Greatest element strictly less than a target.
- **Ceiling:** Least element greater than or equal to a target.
- **Higher:** Least element strictly greater than a target.
- **Range view:** Backed portion bounded by keys or elements.
- **Rotation:** Local tree transformation preserving in-order sequence while changing shape.

28.5 Detailed mechanics

Ordering is identity inside a tree collection

For `TreeMap`, two keys are treated as the same map key when comparison returns zero. `equals` need not be consulted. For `TreeSet`, comparison zero means duplicate membership. Therefore a comparator inconsistent with `equals` can make the collection violate the general expectations of `Map` or `Set` equality, even while operating according to its own ordering.

This comparator collapses all people with the same age:

JAVA

```
Comparator<Person> byAgeOnly = Comparator.comparingInt(Person::age);
```

If distinct people of one age must coexist, add a stable tie-breaker:

JAVA

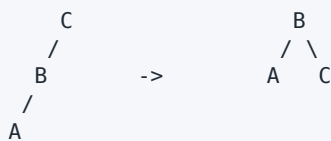
```
Comparator<Person> byAgeThenId = Comparator
    .comparingInt(Person::age)
    .thenComparing(Person::id);
```

The tie-breaker must reflect the intended identity and remain stable while stored.

Search and insertion

Lookup starts at the root. A negative comparison descends left, positive descends right, and zero finds the key. Insertion follows the same path, attaches a new node, and restores balance. A rotation changes parent-child links without changing in-order traversal:

TEXT



Removal has extra cases. A leaf can be detached. A node with one child can be replaced by that child. A node with two children is logically replaced with its successor or predecessor, after which a simpler node is removed. Balanced implementations then repair color or height invariants.

HotSpot note: Current OpenJDK `TreeMap` uses a red-black tree and parent-linked entry nodes. Exact deletion strategy, color representation, and helper methods are version-sensitive.

Navigational operations

`NavigableMap` offers `lowerEntry`, `floorEntry`, `ceilingEntry`, and `higherEntry`, along with key-returning forms. Entry-returning navigation methods produce snapshots of mappings rather than entries supporting `setValue`. `firstEntry` and `lastEntry` inspect endpoints; `pollFirstEntry` and `pollLastEntry` remove endpoints.

`NavigableSet` supplies equivalent element operations. These methods avoid the off-by-one logic that often appears in hand-written range code.

For key 20 in {10, 20, 30}:

TEXT

```
lower(20)  = 10
floor(20)  = 20
ceiling(20) = 20
higher(20) = 30
```

Range and descending views

`subMap`, `headMap`, and `tailMap` expose backed views. Navigable overloads make endpoints explicit:

JAVA

```
map.subMap(from, true, to, false) // [from, to)
```

Mutating a range view mutates the backing map. Inserting a key outside the range throws

`IllegalArgumentException`. `descendingMap` and `descendingSet` are reverse-order views, not full copied reversals. Reversing again produces an equivalent view of the original order.

Iterators over ordinary tree maps, sets, and their views commonly detect unsupported structural mutation and fail fast. Detection is best-effort, not a concurrency guarantee; synchronize access or use a collection designed for concurrent navigation.

A stable snapshot requires copying. Copy into another `TreeMap` if the comparator and sorted navigation must be retained, or into a list if only the current ordered sequence is needed.

Natural order, null, and heterogeneous keys

Natural ordering requires keys implementing mutually compatible `Comparable`. Mixing unrelated types can cause `ClassCastException`. Natural-order `TreeMap` rejects null keys because they cannot be compared. A custom comparator could define null placement, although null keys usually make domain modeling weaker.

Construct comparator chains carefully. `Comparator.comparing` can accept a key comparator, including `nullsFirst` or `nullsLast`. Do not implement numeric comparison with subtraction:

JAVA

```
// Wrong: overflow can reverse the sign.
(a, b) -> a.score() - b.score()

// Correct:
Comparator.comparingInt(Player::score)
```

TreeSet relationship

A typical `TreeSet` is backed by a `NavigableMap` and stores elements as keys with a marker value. Its comparator, range behavior, navigation, and logarithmic costs follow the ordered map. The set API exposes only elements, preserving uniqueness under comparison.

HotSpot note: Backing `TreeSet` with a `TreeMap` is the familiar OpenJDK design, not a requirement that every Java implementation use the same fields.

TRACE IT | 28.6 Worked Java example

The following index finds the configuration effective at a given time:

JAVA (continued 1/2)

```
import java.time.Instant;
import java.util.NavigableMap;
import java.util.TreeMap;

record Config(String revision, int timeoutMillis) {
    public Config {
        if (timeoutMillis <= 0) {
            throw new IllegalArgumentException("timeout must be positive");
        }
    }
}

final class ConfigTimeline {
    private final NavigableMap<Instant, Config> byStart = new TreeMap<>();

    void publish(Instant startsAt, Config config) {
        if (startsAt == null || config == null) {
            throw new IllegalArgumentException("arguments must be non-null");
        }
        Config previous = byStart.putIfAbsent(startsAt, config);
    }
}
```

JAVA (continued 2/2)

```

        if (previous != null) {
            throw new IllegalStateException("a revision already starts at " + startsAt);
        }
    }

    Config effectiveAt(Instant instant) {
        var entry = byStart.floorEntry(instant);
        if (entry == null) {
            throw new IllegalStateException("no configuration yet");
        }
        return entry.getValue();
    }

    NavigableMap<Instant, Config> changes(
        Instant fromInclusive, Instant toExclusive) {
        return java.util.Collections.unmodifiableNavigableMap(
            new TreeMap<>(byStart.subMap(
                fromInclusive, true, toExclusive, false)));
    }
}

```

The copy inside `changes` is deliberate. Returning only an unmodifiable wrapper around `subMap` would still expose later updates from the backing timeline. Here callers receive a membership snapshot that retains sorted navigation.

TRACE IT | 28.7 Execution or memory walkthrough

Suppose revisions start at 09:00, 12:00, and 18:00. Searching at 15:30 asks for `floorEntry(15:30)`. The tree follows comparisons until it either finds the key or reaches a missing child. During descent it remembers the best key seen that is below the target. The result is the 12:00 revision.

For a possible balanced shape:

TEXT

```

      12:00(B)
     /      \
  09:00(R)  18:00(R)

```

Adding 20:00 may initially attach under 18:00. If red-black rules are violated, recoloring or rotations restore them while preserving this in-order sequence:

TEXT

```
09:00, 12:00, 18:00, 20:00
```

The exact resulting shape and colors are not API-visible. Code should observe only ordering and mappings.

Each map entry typically carries key, value, left, right, parent, and balancing metadata. That is more per-entry memory than many hash-table entries and much more than flat arrays. A copied range allocates new tree entries but shares immutable `Instant` and `Config` references.

28.8 Complexity and performance

For n entries in a balanced ordered tree:

Operation	Time	Additional space
get, put, remove	$O(\log n)$	$O(1)$ excluding inserted node
floor/ceiling/lower/higher	$O(\log n)$	$O(1)$
first/last	$O(\log n)$ typical/document basic bound	$O(1)$
ordered traversal	$O(n)$	iterator state usually $O(1)$ with parent links
range traversal returning k entries	$O(\log n + k)$ conceptual	view $O(1)$, copy $O(k)$ nodes

Comparator cost multiplies tree depth. Comparing long strings or extracting expensive sort keys can dominate. Precompute immutable comparison fields when justified.

HashMap usually provides lower expected point-lookup cost and lower comparison overhead. **TreeMap** earns its cost when sorted traversal, bounded ranges, or neighbor queries are first-class. If data changes rarely and is read often, a sorted array or list plus binary search may offer better locality and memory use.

WATCH OUT | 28.9 Edge cases and common mistakes

- Supplying a comparator that is not transitive, leading to unpredictable placement and lookup.
- Returning zero for distinct keys that must coexist.
- Mutating a field used by comparison while an element is stored.
- Using subtraction in integer comparators and overflowing.
- Confusing **lower** with **floor** or **higher** with **ceiling**.
- Forgetting that range bounds can be inclusive or exclusive.
- Returning a mutable backed range to an untrusted caller.
- Assuming a descending view is an independent reversed copy.
- Expecting null keys to work with natural ordering.
- Comparing heterogeneous keys without a comparator capable of handling both.
- Using **TreeMap** solely to print deterministic test output when sorting once would be simpler.
- Assuming the precise red-black tree shape or choosing behavior based on it.

28.10 Production engineering notes

Centralize comparators as named, tested constants. Test sign symmetry, transitivity, zero behavior, and tie-breakers with generated data. Use stable immutable identifiers as final tie-breakers. When comparator equality intentionally differs from object equality, document that the tree collection defines uniqueness by ordering.

Range views are excellent within a short operation because they avoid copies. At service boundaries, prefer snapshots unless live coupling is a deliberate API feature. Treat `pollFirstEntry` and `pollLastEntry` as mutations and protect shared structures with an appropriate concurrency design; `TreeMap` is not thread-safe.

If a time index receives duplicate timestamps, decide whether one mapping, a list per timestamp, or a composite key is correct. Do not let an accidental comparator-zero replacement make that business decision. For very high read concurrency, consider publishing immutable sorted snapshots. For write-heavy concurrent navigation, evaluate purpose-built concurrent structures such as `ConcurrentSkipListMap` and its weaker snapshot semantics.

Measure retained memory and comparator CPU. Ordered indexes can duplicate data already held elsewhere. Decide which structure owns values, and remove stale entries consistently to avoid unbounded retention.

TRY IT | 28.11 Interview questions and model answers

How does `TreeMap` differ from `HashMap`?

`TreeMap` maintains keys in comparator or natural order and supports range and neighbor queries in $O(\log n)$. `HashMap` provides expected $O(1)$ point operations but no ordering contract. The right choice follows required semantics.

What happens when a comparator returns zero?

The tree treats the keys as the same key or set element. A map insertion replaces the value for that ordering-equivalent key. Therefore tie-breakers are necessary when distinct objects must coexist.

Why are tree operations logarithmic?

A balancing invariant keeps height $O(\log n)$. Each search follows one root-to-leaf path, and rebalancing uses a bounded amount of local work per level.

What is the difference between `floorKey` and `lowerKey`?

`floorKey(x)` may return `x` itself and finds the greatest key $\leq x$. `lowerKey(x)` requires a strictly smaller key.

Is `subMap` a copy?

No. It is a backed, bounded view. Changes are reflected in the owner, and out-of-range insertions are rejected. Copy it when independent lifetime is needed.

Must `TreeMap` be a red-black tree?

No. The public contract promises behavior and complexity, not a specific balancing algorithm. Red-black structure describes current common OpenJDK implementation.

TRY IT | 28.12 Exercises

- 1 Design a comparator for orders sorted by descending priority, then creation time, then immutable ID. Explain why every tie-breaker is needed.
- 2 For keys `{10, 20, 30}`, compute lower, floor, ceiling, and higher results for targets 5, 20, 25, and 35.
- 3 Implement an interval lookup where each key is a range start. State what invariant is needed to avoid overlapping ranges.
- 4 Demonstrate how a comparator by string length causes `TreeSet` to collapse distinct same-length strings. Repair it.
- 5 Compare a sorted `ArrayList` plus binary search with `TreeMap` for one bulk build followed by one million reads.
- 6 Return an immutable snapshot of a descending range while preserving its comparator.

RECAP | 28.13 Chapter summary

Navigable tree collections maintain elements under a total ordering and support point, range, endpoint, and neighbor operations. Balanced-tree invariants keep height logarithmic; OpenJDK commonly uses red-black trees, but this is not the API contract. Comparison defines key identity inside the tree, so consistency, tie-breakers, and key stability are correctness requirements. Range and descending collections are backed views, making ownership choices as important as algorithmic complexity.

TRY IT | 28.14 Revision checklist

- ☐ I can state binary-search-tree and red-black-tree invariants.
- ☐ I understand that comparison zero defines uniqueness in tree collections.
- ☐ I can build safe comparator chains without subtraction overflow.
- ☐ I know lower, floor, ceiling, and higher semantics.
- ☐ I can use inclusive and exclusive range views correctly.
- ☐ I distinguish backed views from sorted snapshots.
- ☐ I can compare tree maps with hash maps and sorted arrays.
- ☐ I label red-black representation details as version-sensitive.

SDE-2 CORE CHAPTER

Chapter 29 - Queues, Deques, PriorityQueue, and Heaps

29.1 Learning objectives

By the end of this chapter, you should be able to:

- select FIFO queues, double-ended queues, and priority queues by removal policy;
- use exception-throwing and special-value queue methods correctly;
- derive circular-buffer and binary-heap invariants;
- analyze enqueue, dequeue, heap maintenance, and iteration costs;
- avoid assumptions about priority-queue traversal, stability, and thread safety; and
- apply queues safely to scheduling, buffering, breadth-first search, and top-k problems.

WHY IT WORKS | 29.2 Why this matters at SDE-2

Queues define work order. They appear in request buffering, retries, graph traversal, schedulers, rate-limited pipelines, and event loops. Choosing the wrong removal policy can violate fairness or priority rules even when all operations compile.

Interviewers expect more than "a heap gives $O(\log n)$." You should know which operation is logarithmic, why peeking is constant, why heap iteration is not sorted, and why finding an arbitrary element is linear. In production, you must also address capacity, overload, thread ownership, and tie-breaking. An unbounded in-memory queue can convert downstream slowness into an out-of-memory incident.

WHY IT WORKS | 29.3 First-principles model

A queue separates insertion policy from removal policy:

- A FIFO queue removes the oldest eligible element.
- A deque permits insertion and removal at both ends.
- A priority queue removes the least or greatest element under an ordering, not necessarily the oldest.

An array deque can use a circular logical sequence over a physical array:

TEXT

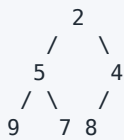
```
physical slots: [D, E, empty, empty, A, B, C]
logical order:  A, B, C, D, E
head index:     4
tail index:     2   (next insertion position)
```

Indices wrap around. The invariant is that logical elements occupy the circular range from head to tail according to the representation's empty/full convention.

A binary min-heap uses a complete binary tree whose parent is no greater than either child:

TEXT

```
array: [2, 5, 4, 9, 7, 8]
```



For zero-based index i , children are commonly $2i + 1$ and $2i + 2$; parent is $(i - 1) / 2$. Completeness enables array storage without node links.

Specification boundary: Queue, Deque, and PriorityQueue define behavior. PriorityQueue documents heap-like complexity but does not promise a particular arity, array layout, growth policy, or stable ordering among equal-priority elements.

29.4 Core terminology

- **Head:** Element selected for the next removal or inspection.
- **Tail:** End normally used for FIFO insertion.
- **FIFO:** First in, first out.
- **LIFO:** Last in, first out, or stack order.
- **Bounded queue:** Queue with a maximum capacity.
- **Backpressure:** Mechanism that slows, rejects, or redirects producers when consumers cannot keep up.
- **Heap:** Complete tree satisfying a local parent-child ordering invariant.
- **Sift up:** Move an inserted heap element toward the root until the invariant holds.
- **Sift down:** Move a displaced root toward leaves until the invariant holds.
- **Stable ordering:** Equal-priority elements retain their original relative order.
- **Tie-breaker:** Secondary comparison field that makes priority deterministic.

29.5 Detailed mechanics

Queue method pairs

The queue API offers two styles:

Intent	Throws on failure	Returns special value
insert	<code>add(e)</code>	<code>offer(e)</code> returns <code>false</code>
remove head	<code>remove()</code>	<code>poll()</code> returns <code>null</code>
inspect head	<code>element()</code>	<code>peek()</code> returns <code>null</code>

For capacity-restricted queues, `offer` is usually the clearer insertion method because full capacity is an expected condition. `poll` and `peek` use `null` to mean empty, which is one reason queue implementations commonly reject `null` elements.

Deque method families

Deque generalizes both ends. It provides `addFirst/offerFirst`, `addLast/offerLast`, `removeFirst/pollFirst`, and corresponding last methods. The queue aliases generally target the last for insertion and first for removal. Stack usage should prefer `push`, `pop`, and `peek` on a deque rather than the legacy `Stack` class.

`ArrayDeque` is usually the default general-purpose deque. It avoids a node allocation per element and offers good locality. It rejects `null`. `LinkedList` also implements `Deque`, but its per-node overhead and locality are usually worse.

HotSpot note: Current OpenJDK `ArrayDeque` uses a resizable circular array and head/tail indexes. Exact array-length conventions, growth increments, and wrap logic have changed across releases and are not public contracts.

PriorityQueue heap mechanics

In a min-priority queue, the root is the minimum under natural ordering or a supplied comparator. Insertion appends at the next array position, then sifts upward while smaller than its parent. Removal saves the root, moves the last element to the root, decreases size, and sifts down by exchanging with the smaller child.

The heap invariant is local. It guarantees every parent is no greater than its children, which implies the root is globally minimal. It does not imply siblings or array positions are globally sorted. Thus iteration order is unspecified rather than priority order.

Ordinary deque and priority-queue iterators commonly fail fast after detected structural modification. This is diagnostic, best-effort behavior, not thread coordination. Iterator traversal is not a snapshot and still does not expose priority order.

For max-priority behavior, reverse the comparator. Be careful with arithmetic comparators; use `comparingInt`, `comparingLong`, or safe comparison methods instead of subtraction.

Equal priorities and mutation

`PriorityQueue` does not guarantee FIFO order for equal elements. Add a monotonically increasing sequence as a final comparator field when stable tie-breaking is required. Consider sequence overflow and lifecycle; a `long` is ample for many process lifetimes but still deserves an explicit policy.

Never mutate an enqueued element's priority fields. The queue does not automatically locate and reposition it, so the heap can become logically invalid. Remove and reinsert the element, or enqueue immutable task descriptors and keep changing state elsewhere.

Arbitrary operations and bulk heap construction

`contains` and `remove(Object)` generally scan the backing representation, costing $O(n)$, because the heap orders only by priority and has no key index. After an arbitrary removal is found, restoring the heap costs $O(\log n)$.

Building a heap by repeated insertion costs $O(n \log n)$. Bottom-up heap construction can heapify an array in $O(n)$ because most nodes are near leaves and move only a short distance. A `PriorityQueue` constructor from a suitable collection may take advantage of bulk heapification; exact constructor paths remain implementation-specific.

Bounded and concurrent queues

General-purpose `ArrayDeque` and `PriorityQueue` are unbounded in the API sense and not thread-safe. Capacity-aware and blocking behavior lives in concurrent queue types. A bounded blocking queue can wait, time out, or reject when full. A concurrent priority queue provides thread-safe priority access but still does not make a multi-step workflow atomic.

Queue selection must include who produces, who consumes, whether operations can block, and what happens under overload. Those are system semantics, not merely collection details.

TRACE IT | 29.6 Worked Java example

This retry scheduler orders tasks by due time and preserves insertion order among equal due times:

JAVA

```
import java.time.Instant;
import java.util.Comparator;
import java.util.PriorityQueue;

record Retry(long sequence, Instant dueAt, String jobId, int attempt) {}

final class RetrySchedule {
    private static final Comparator<Retry> ORDER = Comparator
        .comparing(Retry::dueAt)
        .thenComparingLong(Retry::sequence);

    private final PriorityQueue<Retry> queue = new PriorityQueue<>(ORDER);
    private long nextSequence;

    void schedule(Instant dueAt, String jobId, int attempt) {
        if (dueAt == null || jobId == null || attempt < 1) {
            throw new IllegalArgumentException("invalid retry");
        }
        queue.offer(new Retry(nextSequence++, dueAt, jobId, attempt));
    }

    Retry pollReady(Instant now) {
        Retry next = queue.peek();
        if (next == null || next.dueAt().isAfter(now)) {
            return null;
        }
        return queue.poll();
    }

    int pending() {
        return queue.size();
    }
}
```

This class assumes single-threaded ownership. A lock would be needed if multiple threads share it; using a thread-safe queue alone would not make the `peek` followed by conditional `poll` atomic. A production retry service also persists tasks, caps capacity, supports cancellation through an index, and handles sequence lifecycle.

TRACE IT | 29.7 Execution or memory walkthrough

Suppose the comparator sees tasks (12:00, seq 0), (12:10, seq 1), and (12:05, seq 2).

TEXT

```
insert 12:00/0: [12:00/0]
insert 12:10/1: [12:00/0, 12:10/1]
insert 12:05/2: append then compare with root
                [12:00/0, 12:10/1, 12:05/2]
```

The array is a valid heap even though positions 1 and 2 are not sorted with each other. Polling removes 12:00/0, moves 12:05/2 to the root, and sifts down:

TEXT

```
before removal: [12:00/0, 12:10/1, 12:05/2]
move last:      [12:05/2, 12:10/1]
result:         12:00/0
```

If another task due at 12:05 has sequence 3, sequence 2 sorts first. The tie-breaker turns an unspecified equal-priority outcome into a domain guarantee.

The priority queue stores references in an array and `Retry` record objects separately. Removed slots are cleared by normal implementations so they do not retain tasks. Exact capacity may exceed size. A deque similarly keeps spare slots to make end operations cheap.

29.8 Complexity and performance

Operation	ArrayDeque	PriorityQueue
add/remove at supported end	amortized $O(1)$	add $O(\log n)$, poll $O(\log n)$
peek head	$O(1)$	$O(1)$
remove arbitrary object	$O(n)$	$O(n)$ search plus repair
contains	$O(n)$	$O(n)$
iterate all	$O(n)$	$O(n)$, not sorted
copy then ordered drain	$O(n)$ copy	$O(n \log n)$ drain
bottom-up heap construction	not applicable	$O(n)$ algorithmically

Deque resize is occasionally $O(n)$, giving amortized constant end operations under geometric growth. Heap operations follow height $O(\log n)$ because a complete binary tree with n nodes has logarithmic height.

For top- k selection from n values, maintain a heap of size k : $O(n \log k)$ time and $O(k)$ space. Sorting everything costs $O(n \log n)$ and $O(n)$ input storage or sorting space depending on representation. If k is close to n , constants and required output order can make sorting preferable.

WATCH OUT | 29.9 Edge cases and common mistakes

- Assuming iteration over `PriorityQueue` returns sorted order.
- Assuming equal-priority tasks are stable without a tie-breaker.
- Mutating an element's comparator fields while it is enqueued.
- Calling `remove(Object)` or `contains` frequently and expecting logarithmic behavior.
- Using null as a legitimate queue element when `poll` uses null for emptiness.
- Choosing exception-throwing `add` when full capacity is expected control flow.
- Using a queue that grows without bound under slower consumers.
- Sharing `ArrayDeque` or `PriorityQueue` across threads without protection.
- Separating `peek` and `poll` in concurrent code without one atomic policy.
- Using `PriorityQueue` for scheduling but ignoring wall-clock changes, durability, or wakeup coordination.
- Implementing a max heap by negating an integer and overflowing at `Integer.MIN_VALUE`.
- Assuming a heap is a binary search tree or that arbitrary search follows one branch.
- Forgetting stale duplicate entries in algorithms that reinsert improved priorities.

29.10 Production engineering notes

Capacity is a reliability contract. Establish maximum depth, enqueue timeout or rejection behavior, retry/drop policy, and metrics for depth, age, throughput, and rejection. FIFO does not guarantee fairness if tasks have dramatically different service times. Priority scheduling can starve low-priority work; aging or quotas may be necessary.

Use immutable queue elements. When cancellation or priority updates are frequent, pair a heap with a map from ID to state, accept lazy deletion, or use a specialized indexed heap. Lazy deletion leaves stale entries until they reach the head, so bound and observe the overhead.

For breadth-first search, mark a node visited when enqueueing, not when dequeuing, to prevent duplicate queue growth. For stacks, prefer `ArrayDeque.push/pop` and never use null sentinels. For thread handoff, choose a concurrent or blocking queue whose ordering, capacity, and memory-consistency contract match the workflow.

Do not persist only an in-memory retry heap if process restart must not lose jobs. Keep durable source-of-truth state and rebuild the heap, or use an external scheduler. Collection choice solves in-process ordering, not distributed delivery guarantees.

TRY IT | 29.11 Interview questions and model answers

Why is priority-queue iteration not sorted?

The heap invariant only orders each parent relative to its children. The backing array is a level-order heap representation, not a sorted sequence. Repeated polling, preferably from a copy, produces priority order.

What are offer/poll/peek for?

They use special return values for expected failure: `offer` can report full capacity, while `poll` and `peek` return null for an empty queue. The paired methods `add/remove/element` throw exceptions.

How does heap insertion work?

Append at the next complete-tree position, then sift upward while the new element precedes its parent. At most one root-to-leaf height is traversed, so cost is $O(\log n)$.

Why is removing an arbitrary heap element $O(n)$?

The heap has no global search ordering for arbitrary values. It may scan all entries to locate the value, then needs only logarithmic repair.

How do you make equal-priority scheduling deterministic?

Add a stable secondary comparator field, typically a monotonic sequence number or immutable ID, matching the desired business rule.

Why prefer `ArrayDeque` over `Stack` or often `LinkedList`?

It directly implements deque and stack operations, avoids legacy synchronized `Vector` behavior, uses fewer objects than a linked list, and usually has better locality.

TRY IT | 29.12 Exercises

- 1 Dry-run heap insertion of 7, 3, 9, 1, 4 and then two polls. Draw the array after each step.
- 2 Use an `ArrayDeque` to implement breadth-first traversal and explain when each node is marked visited.
- 3 Find the largest five values in a stream using a min-heap of size five. State time and space bounds.
- 4 Design overload behavior for a bounded email-work queue. Include observability and retry policy.
- 5 Modify the retry scheduler to support lazy cancellation through a set of canceled IDs. Analyze stale-entry memory.
- 6 Demonstrate that printing or iterating a priority queue is not a sorted-output algorithm.

RECAP | 29.13 Chapter summary

Queues are defined by removal policy. Deques efficiently support both ends, while priority queues expose the minimum or maximum under an ordering. Circular arrays provide amortized constant deque operations; complete binary heaps provide constant-time peek and logarithmic insertion and head removal. Their local invariant does not sort iteration or accelerate arbitrary search. Production use adds bounded capacity, overload behavior, immutability, concurrency ownership, fairness, and durability.

TRY IT | 29.14 Revision checklist

- ☐ I know the exception and special-value queue method pairs.
- ☐ I can map every **Deque** operation to an end and use it as a stack.
- ☐ I can state circular-buffer and min-heap invariants.
- ☐ I can dry-run sift-up and sift-down.
- ☐ I do not assume priority-queue iteration or equal-priority stability.
- ☐ I know why arbitrary heap search and removal are linear.
- ☐ I can derive $O(n \log k)$ top-k selection.
- ☐ I include capacity, backpressure, concurrency, fairness, and durability in production designs.

SDE-2 CORE CHAPTER

Chapter 30 - Comparable, Comparator, Sorting, and Selection

30.1 Learning objectives

By the end of this chapter, you should be able to:

- define natural and external orderings with valid comparison contracts;
- compose null-safe, deterministic comparators without overflow;
- explain stable sorting, in-place sorting, and comparison lower bounds;
- choose among object sort, primitive sort, parallel sort, heap selection, and quickselect;
- use binary search only under its ordering precondition; and
- discuss current OpenJDK algorithms without confusing them with API guarantees.

WHY IT WORKS | 30.2 Why this matters at SDE-2

Sorting is often the boundary between raw data and useful output: ranked results, merge pipelines, pagination, reconciliation, and deduplication all depend on ordering. Comparison defects can be intermittent and severe. A non-transitive comparator may make sorting fail at runtime, corrupt a tree collection's model, or create unstable pagination.

At SDE-2, you should translate a product rule into a total order, handle ties explicitly, and select an algorithm based on whether all values or only the top *k* are needed. You should distinguish specification promises, such as stability for an object sort, from OpenJDK's current TimSort or primitive-array algorithms.

WHY IT WORKS | 30.3 First-principles model

An ordering comparator maps two values to a negative number, zero, or a positive number:

TEXT

```
compare(a, b) < 0  means a precedes b
compare(a, b) = 0  means a and b are ordering-equivalent
compare(a, b) > 0  means a follows b
```

The magnitude is irrelevant. A comparator must behave like a total order over the accepted domain. Important laws are:

- sign symmetry: `sign(compare(a,b)) == -sign(compare(b,a));`

- transitivity: if $a > b$ and $b > c$, then $a > c$;
- zero consistency: if a compares equal to b , comparisons of each against c have the same sign; and
- repeatability while participating fields remain unchanged.

Comparison sorting uses pairwise questions to distinguish possible permutations. There are $n!$ permutations, so a decision tree requires height $\Omega(\log(n!))$, which is $\Omega(n \log n)$. General comparison sorting cannot asymptotically beat that bound. Algorithms using restricted keys, such as counting sort, operate under different assumptions.

Specification boundary: `Comparable` and `Comparator` define ordering contracts. Sorting APIs document properties such as stability and permitted mutation. Their exact algorithm, run detection, pivot selection, temporary storage, and thresholds can vary by JDK implementation and version.

30.4 Core terminology

- **Natural order:** A type's canonical order implemented by `Comparable`.
- **External order:** A purpose-specific `Comparator` supplied by a client.
- **Stable sort:** Equal-order elements retain input relative order.
- **In-place:** Uses only bounded or logarithmic auxiliary storage under a stated model; common library documentation may use looser wording.
- **Adaptive sort:** Exploits existing order or runs in the input.
- **Total order:** Consistent ordering for every accepted pair.
- **Partial selection:** Find a rank or top subset without fully sorting.
- **Quickselect:** Partition-based expected linear-time selection.
- **Partition:** Rearrange elements around a pivot by comparison.
- **Tie-breaker:** Additional field distinguishing otherwise equal rankings.
- **Schwartzian transform/decorate-sort-undecorate:** Precompute expensive sort keys, sort decorated values, then extract originals.

30.5 Detailed mechanics

Comparable versus Comparator

Implement `Comparable<T>` when a type has one unsurprising natural order used broadly:

JAVA

```
record Version(int major, int minor, int patch)
    implements Comparable<Version> {
    @Override
    public int compareTo(Version other) {
        int result = Integer.compare(major, other.major);
        if (result != 0) return result;
        result = Integer.compare(minor, other.minor);
        if (result != 0) return result;
        return Integer.compare(patch, other.patch);
    }
}
```

Use comparators for alternative views: orders by creation time, priority, customer, or amount. A natural order becomes part of a public type's long-lived meaning, so do not add one merely for a single screen.

Natural ordering should usually be consistent with `equals`, especially when values enter sorted sets or maps. `BigDecimal` is a notable counterexample: values such as `1.0` and `1.00` compare as zero but are not equal because scale affects `equals`. A `TreeSet<BigDecimal>` can therefore have different uniqueness behavior from a `HashSet<BigDecimal>`.

Comparator composition

Comparator factories make intent explicit:

JAVA

```
Comparator<Order> order = Comparator
    .comparingInt(Order::priority).reversed()
    .thenComparing(Order::createdAt)
    .thenComparing(Order::id);
```

Apply reversal carefully. Calling `reversed()` at the end reverses the entire chain. Calling it after the primary comparator reverses only that comparator before adding subsequent ascending tie-breakers.

Use primitive factories to avoid boxing in comparison: `comparingInt`, `comparingLong`, and `comparingDouble`. Use `Comparator.nullsFirst` or `nullsLast` only if null is valid domain data. Do not compare integers with `a - b`; overflow can violate the sign rule. Floats and doubles also require library comparison because NaN and signed zero need a consistent total order.

Stability and tie-breaking

Stable sort preserves input order when comparator returns zero. This permits multi-pass sorting: stable-sort by a secondary key, then by primary key. A single comparator chain is usually clearer and performs fewer sorts.

Stability is not a substitute for a deterministic total presentation order. If input arrives from a hash map or concurrent source with unspecified encounter order, stable sorting equal ranks preserves an unspecified order. Add a unique immutable ID as a tie-breaker for reproducible pagination and tests.

Library sorting APIs

`List.sort(comparator)` mutates the list and is specified as stable. `Collections.sort` delegates to the list's sorting facility in modern APIs. The list must support the required replacement operation; an unmodifiable list rejects sorting.

`Arrays.sort(Object[])` is stable. Primitive overloads are not required to be stable because primitive values have no separately observable identity when equal. Primitive sorts avoid boxing and are normally much more memory-efficient.

`Arrays.parallelSort` may use the common fork-join pool and temporary storage. Parallel overhead can outweigh gains for small arrays or contended services. Measure on the deployment shape, and remember that comparator work must be thread-safe and side-effect-free.

HotSpot note: Current OpenJDK releases commonly use TimSort-derived logic for object arrays and lists, dual-pivot quicksort and specialized paths for several primitive types, and parallel merge/sort strategies above thresholds. Algorithms and thresholds are version-sensitive.

TimSort intuition and comparator failures

TimSort is adaptive: it discovers ascending or descending runs, extends short runs, and merges runs while maintaining stack invariants. Nearly sorted input can therefore be efficient. It needs temporary storage for merging.

Inconsistent comparators can trigger exceptions such as "comparison method violates its general contract" in some sort paths, but detection is not guaranteed. A sort that appears to complete does not prove comparator correctness.

Selection instead of full sorting

If only the smallest or largest k items are needed:

- sort all: $O(n \log n)$ time, straightforward, produces complete order;
- maintain a heap of size k : $O(n \log k)$ time and $O(k)$ extra space;
- quickselect: expected $O(n)$ to place the k th rank, then optionally sort the selected k ; worst case $O(n^2)$ without robust pivot strategy;
- counting/bucket techniques: potentially $O(n + \text{range})$ when integer key range is small and bounded.

Quickselect partitions around a pivot. After partition, if the pivot lands at rank k , selection is complete; otherwise recurse or iterate only on the side containing rank k . It mutates the array unless implemented on a copy.

Binary search

`Collections.binarySearch` and `Arrays.binarySearch` require data sorted according to the same ordering. A nonnegative result is a matching index. A negative result encodes insertion point $-(\text{result}) - 1$. When duplicates exist, no promise should be inferred about which equal occurrence is returned unless the API states one. Use explicit lower-bound and upper-bound searches for ranges of duplicates.

TRACE IT | 30.6 Worked Java example

This method returns the top k candidates while bounding intermediate storage:

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

record Candidate(String id, int score, long completedAtEpochMillis) {}

final class Ranking {
    // Best first: higher score, earlier completion, lexicographically smaller ID.
    private static final Comparator<Candidate> BEST_FIRST = Comparator
        .comparingInt(Candidate::score).reversed()
        .thenComparingLong(Candidate::completedAtEpochMillis)
        .thenComparing(Candidate::id);

    static List<Candidate> topK(Collection<Candidate> input, int k) {
        if (k < 0) throw new IllegalArgumentException("k must be non-negative");
        if (k == 0) return List.of();
    }
}
```

JAVA (continued 2/2)

```
    // Worst retained candidate is at the head.
    PriorityQueue<Candidate> retained =
        new PriorityQueue<>(k, BEST_FIRST.reversed());

    for (Candidate candidate : input) {
        if (candidate == null) throw new IllegalArgumentException("null candidate");
        if (retained.size() < k) {
            retained.offer(candidate);
        } else if (BEST_FIRST.compare(candidate, retained.peek()) < 0) {
            retained.poll();
            retained.offer(candidate);
        }
    }

    ArrayList<Candidate> result = new ArrayList<>(retained);
    result.sort(BEST_FIRST);
    return List.copyOf(result);
}
```

The ID tie-breaker gives a deterministic total ranking. If IDs are unique, no two distinct candidates compare as zero. The heap uses reverse order so its head is the worst candidate currently retained; a better incoming candidate replaces it.

TRACE IT | 30.7 Execution or memory walkthrough

For $k = 3$ and candidates with scores 70/A, 90/B, 80/C, 85/D, and 80/E, assume times and IDs do not change the score comparisons:

- 1 Retain A, B, C. The heap head is A, the worst of those three.
- 2 D outranks A, so poll A and add D. Retained set is B, C, D; C is now worst.
- 3 E ties C on score. Completion time and then ID decide whether E is better. Replace only if the full comparator ranks E before C.
- 4 Copy the heap. Its iteration order is not ranked.
- 5 Sort the copy best-first and publish an unmodifiable result.

At most k candidate references reside in the heap, plus the final list of at most k references. Candidate records are shared rather than copied. Comparator calls extract primitive fields without boxing for score and time.

Quickselect would store the entire mutable input or a copy but could reduce asymptotic selection time. The heap supports one-pass input and is appropriate when the collection is streamed or k is small.

30.8 Complexity and performance

For **topK**, each of n inputs performs constant comparison plus at most one heap replacement costing $O(\log k)$. Time is $O(n \log k + k \log k)$ and auxiliary reference storage is $O(k)$. When $k \geq n$, the behavior approaches $O(n \log n)$ and sorting all may be simpler and faster.

Common sorting bounds:

Technique	Time	Extra space	Stable	Best use
comparison sort	$O(n \log n)$ typical/guaranteed by chosen algorithm	varies	API-dependent	full order
insertion sort	$O(n^2)$, near $O(n)$ when nearly sorted	$O(1)$	yes when implemented conventionally	tiny/nearly sorted ranges
merge sort	$O(n \log n)$	$O(n)$	yes	stable predictable sorting
heap sort	$O(n \log n)$	$O(1)$ array model	no	worst-case bound, low extra space
quicksort	average $O(n \log n)$, worst $O(n^2)$	recursion/stack varies	usually no	fast primitive/in-place-style sorting
heap top-k	$O(n \log k)$	$O(k)$	only with explicit tie order	small top subset
quickselect	expected $O(n)$, worst $O(n^2)$	often $O(1)$ iterative	no	one rank or unsorted partition

Comparator cost can dominate $n \log n$. Avoid network calls, database access, mutable clocks, locale creation, or repeated expensive parsing inside comparison. Precompute keys when profiling justifies it.

WATCH OUT | 30.9 Edge cases and common mistakes

- Implementing compare with subtraction and overflowing.
- Omitting a stable tie-breaker for pagination or distributed merge results.
- Assuming stable sort makes unspecified input order deterministic.
- Using a comparator that changes based on mutable state, current time, or side effects.
- Mixing natural and external ordering between sort and binary search.
- Expecting binary search to return the first duplicate.
- Sorting an unmodifiable or fixed-capability list without understanding supported operations.
- Mutating comparator fields while objects reside in a sorted collection.
- Assuming primitive and object sort stability are identical.
- Parallelizing a small sort or using a non-thread-safe comparator with parallel sort.
- Fully sorting millions of items to return a tiny top-k result.
- Forgetting that `reversed()` placement can reverse an entire comparator chain.
- Treating current OpenJDK algorithm names or thresholds as portable guarantees.

30.10 Production engineering notes

Define ordering once and reuse it across database queries, in-memory merge, API pagination, and tests. Any mismatch can cause duplicates or gaps between pages. Cursor pagination needs a unique final tie-breaker included in both ordering and cursor encoding.

Comparator functions should be pure, cheap, null-explicit, and tested with property-style checks for symmetry and transitivity. Normalize text before sorting if case, Unicode, or locale rules matter. Human-language collation is not equivalent to `String` code-unit order and may depend on locale/version; isolate it from identity ordering.

Avoid sorting shared mutable lists in place. Copy, sort, and publish when readers need snapshots. For large results, push sorting and limiting toward an indexed data source when possible. Enforce input caps for user-controlled sorts to prevent CPU and memory abuse.

Measure sequential versus parallel sorting in realistic service contention. A common pool is shared process capacity, not free compute. Primitive arrays avoid wrapper allocation and should be preferred for numerical kernels when the surrounding design permits them.

TRY IT | 30.11 Interview questions and model answers**When should a class implement `Comparable`?**

When it has one canonical, unsurprising natural order that is meaningful across uses. Purpose-specific orders belong in named comparators.

What properties must a comparator satisfy?

It must provide consistent sign symmetry, transitivity, and zero behavior over its accepted domain. It should be repeatable while compared state is unchanged and preferably consistent with equals for collection use.

What does stable sort mean?

Elements that compare as equal retain their original relative order. It does not create determinism if original encounter order is unspecified.

How would you find the top 100 of ten million values?

Use a size-100 min-heap while scanning: $O(n \log 100)$ time and $O(100)$ memory, then sort the retained items for final output. Discuss database pushdown or distributed merge if data is external.

Why can quickselect be faster than sorting?

It explores only the partition containing the target rank, giving expected linear work. It does not fully order the data and has quadratic worst cases without pivot safeguards.

Can I rely on TimSort for every Java object sort?

Rely on the documented stability and behavior of the API, not an algorithm name. TimSort is a common current OpenJDK implementation and can change.

TRY IT | 30.12 Exercises

- 1 Write a comparator for nullable invoices ordered by status, descending amount, due date, and unique ID. State null policy.
- 2 Produce a three-value counterexample showing how a bad cyclic comparator violates transitivity.
- 3 Implement lower-bound binary search that returns the first index whose element is not less than a target.
- 4 Compare full sort, heap top-k, and quickselect for $n = 1,000,000$ and $k = 10$, then for $k = 900,000$.
- 5 Repair a comparator written as `(a, b) -> a.timestamp() > b.timestamp() ? 1 : -1`.
- 6 Explain why stable sorting entries from a `HashMap` by value alone can still produce changing output.

RECAP | 30.13 Chapter summary

Ordering is a correctness contract before it is an algorithm. `Comparable` defines a canonical natural order; `Comparator` defines external orders that can be composed with safe tie-breakers. Stable full sorting solves complete ordering in $O(n \log n)$, while heaps and selection algorithms avoid unnecessary work for small subsets. Java APIs specify observable properties, whereas TimSort, primitive quicksort variants, and thresholds are current implementation choices. Production ordering must align across layers and remain deterministic, pure, and bounded.

TRY IT | 30.14 Revision checklist

- ☐ I can state comparator laws and consistency-with-equals concerns.
- ☐ I use safe primitive comparisons rather than subtraction.
- ☐ I understand reversal placement and comparator chaining.
- ☐ I distinguish stability from deterministic total ordering.
- ☐ I know object, primitive, and parallel sort trade-offs.
- ☐ I can choose full sort, heap top-k, or quickselect from requirements.
- ☐ I use binary search only with the same ordering used for sorting.
- ☐ I label OpenJDK sorting algorithms and thresholds as version-sensitive.

SDE-2 CORE CHAPTER

Chapter 31 - Streams, Collectors, Optional, and Spliterators

31.1 Learning objectives

By the end of this chapter, you should be able to:

- explain lazy stream pipelines, encounter order, and single-use traversal;
- distinguish stateless, stateful, short-circuiting, intermediate, and terminal operations;
- write side-effect-free transformations and associative reductions;
- compose collectors and state their mutability, duplicate, null, and ordering behavior;
- use `Optional` at absence-return boundaries without turning it into a universal container;
- interpret spliterator characteristics and splitting; and
- evaluate parallel streams using correctness laws, source shape, workload, and deployment context.

WHY IT WORKS | 31.2 Why this matters at SDE-2

Streams compress collection processing into declarative pipelines, but concise syntax can hide multiple traversals, large buffers, unsafe side effects, and ambiguous merge behavior. At SDE-2, you should be able to review a pipeline as rigorously as a loop: identify its source, cardinality changes, ordering, terminal result, allocation, and parallel assumptions.

Collectors appear in grouping, aggregation, indexing, and reporting. `Optional` appears at repository and lookup boundaries. Spliterators connect collection data structures to sequential and parallel traversal. Understanding all three prevents accidental $O(n^2)$ work, data races, and APIs that are harder to use than a simple null check or domain result type.

WHY IT WORKS | 31.3 First-principles model

A stream is not a container. It is a one-shot description of a traversal and transformation. A pipeline has:

TEXT

```
source -> intermediate operations -> terminal operation
```

Intermediate operations return another stream and are generally lazy. The terminal operation pulls elements through the pipeline. A fused sequential pipeline can process one element through several stages before requesting the next.

TEXT

```
input:  [1, 2, 3, 4, 5]
filter even -> map square -> findFirst

1 rejected
2 accepted -> 4 -> terminal completes
3, 4, 5 may never be requested
```

A collector is a mutable-reduction recipe with four conceptual functions:

- 1 supplier creates an accumulator;
- 2 accumulator incorporates one input;
- 3 combiner merges partial accumulators; and
- 4 finisher converts the accumulator to the result, unless identity finishing applies.

A spliterator is a traversal cursor that can also partition remaining work. Its characteristics describe properties a caller may exploit.

Specification boundary: Stream semantics define laziness, operation contracts, encounter-order rules, reduction requirements, and single use. They do not guarantee stage fusion strategy, thread count, partition sizes, common-pool scheduling, vectorization, or that parallel execution will be faster.

31.4 Core terminology

- **Pipeline:** Source, intermediate operations, and one terminal operation.
- **Lazy:** Work is deferred until demanded by a terminal operation.
- **Encounter order:** Order in which a stream logically presents elements when its source or operations define one.
- **Stateless operation:** Result for one element does not depend on previously seen elements, such as `map`.
- **Stateful operation:** May buffer or track elements, such as `sorted` or `distinct`.
- **Short-circuiting:** May complete without processing all input, such as `findFirst` or `limit`.
- **Non-interference:** Pipeline behavior does not modify or depend on unsafe mutation of its source during execution.
- **Associative:** Grouping operands differently produces an equivalent result.
- **Mutable reduction:** Accumulate into a mutable result container, as with `collect`.
- **Downstream collector:** Collector applied to values within each group or partition.
- **Characteristic:** Spliterator or collector property clients can use for execution decisions.
- **Primitive stream:** `IntStream`, `LongStream`, or `DoubleStream`, avoiding wrapper elements for many operations.

31.5 Detailed mechanics

Laziness, fusion, and single use

`filter`, `map`, `flatMap`, `peek`, `distinct`, `sorted`, `limit`, and `skip` are intermediate. `forEach`, `reduce`, `collect`, `count`, `findFirst`, and matching operations are terminal. Without a terminal operation, a pipeline performs no traversal.

A stream can be consumed once. Reusing it after a terminal operation throws `IllegalStateException`. Store a source or a `Supplier<Stream<T>>` when repeated traversal is intended.

Laziness enables fusion and short-circuiting, but stateful operations can create barriers. `sorted` usually needs all relevant input before emitting the first sorted result. `distinct` tracks seen values. On ordered parallel pipelines, `limit`, `skip`, and `distinct` can require coordination and buffering.

Operation classes and ordering

`map` preserves cardinality one-for-one; `filter` can reduce it; `flatMap` maps one input to zero or more outputs and closes each nested stream after use. `flatMapMulti`, available in modern Java, can emit zero or more outputs without constructing a stream per source item.

Ordered sources include lists and sorted collections. Hash-based collections generally do not promise encounter order. `forEachOrdered` respects encounter order when one exists, even in a parallel pipeline, at a coordination cost. `findFirst` is order-sensitive; `findAny` permits more freedom and can be useful in parallel.

Calling `unordered()` does not randomly shuffle data. It removes the obligation to preserve encounter order, allowing optimizations for operations whose result does not require it.

Side effects and non-interference

Behavioral parameters should be stateless and non-interfering:

JAVA

```
// Wrong for parallel use and difficult even sequentially.
List<String> output = new ArrayList<>();
input.parallelStream()
    .filter(this::valid)
    .forEach(output::add);

// Correct mutable reduction.
List<String> output = input.parallelStream()
    .filter(this::valid)
    .toList();
```

Concurrent mutation of the source during traversal may throw, produce weakly consistent observations for a concurrent source, or otherwise follow that source's contract. Streams do not add isolation.

`peek` exists mainly for debugging and carefully controlled observation. It may not execute for every conceptual element because short-circuiting and implementation optimizations can avoid traversal. Do not put required business effects in `peek`.

Reduction laws

For `reduce(identity, accumulator, combiner)`, the identity must be neutral, the reduction must be associative, and accumulator/combiner behavior must be compatible. Integer addition satisfies this ignoring overflow as Java-defined wraparound grouping effects for fixed values; subtraction does not:

TEXT

```
(10 - 3) - 2 = 5
10 - (3 - 2) = 9
```

A sequential left fold with subtraction may look predictable, but parallel partitioning changes grouping. Floating-point addition is mathematically associative but not bitwise associative due to rounding, so parallel sums can differ slightly.

Do not mutate the identity object in the three-argument `reduce`. Multiple partitions may share assumptions incompatible with mutable state. Use `collect` for mutable containers.

Collector composition

Common collectors include:

- `toList`, `toSet`, and `toCollection`;
- `joining` for character sequences;
- `counting`, `summingInt`, `averagingLong`, and `summarizingDouble`;
- `groupingBy` and `groupingByConcurrent`;
- `partitioningBy` for exactly two Boolean groups;
- `mapping`, `filtering`, and `flatMap` downstream;
- `collectingAndThen` for a finishing transformation; and
- `teeing` to run two downstream aggregations and merge their results.

When collecting to a map, duplicate keys need policy:

JAVA

```
Map<String, User> byEmail = users.stream().collect(
    java.util.stream.Collectors.toMap(
        User::email,
        java.util.function.Function.identity(),
        (left, right) -> chooseNewer(left, right),
        java.util.LinkedHashMap::new));
```

Without a merge function, duplicate keys throw `IllegalStateException`. That can be the right invariant check. The map supplier determines a concrete result representation when an overload accepts it.

`Collectors.toList()` does not promise mutability, unmodifiability, or a concrete list type.

`Stream.toList()` returns an unmodifiable list and may have different null behavior from `List.copyOf`; do

not substitute methods solely because names look similar. `Collectors.toUnmodifiableList()` rejects null elements.

Collector characteristics include `CONCURRENT`, `UNORDERED`, and `IDENTITY_FINISH`. `CONCURRENT` does not mean every use updates one shared result concurrently; source ordering and collector characteristics influence the strategy. Custom collectors must make accumulator, combiner, and finisher laws correct before seeking performance.

Optional as an absence result

`Optional<T>` models either one non-null value or absence. It works well as a return type for lookups:

JAVA

```
Optional<User> findById(UserId id)
```

Useful methods include `map`, `flatMap`, `filter`, `or`, `orElseGet`, `ifPresentOrElse`, and `stream.map` wraps a non-null mapping result and yields empty for null. `flatMap` is for functions already returning `Optional`.

`orElse(fallback())` evaluates `fallback()` eagerly. `orElseGet(this::fallback)` invokes it only when empty. `orElseThrow` is appropriate when absence violates the current layer's precondition.

Avoid `Optional` fields, collection elements, and method parameters by default. They often add a second wrapper state without improving the domain. A list already represents zero or more values; return an empty list instead of `Optional<List<T>>` unless absence and empty are genuinely different. `Optional` is value-based: do not synchronize on instances or rely on identity.

Specialized `OptionalInt`, `OptionalLong`, and `OptionalDouble` avoid boxing. `Optional` is not a substitute for a rich failure result containing reason, retryability, or validation errors.

Spliterator mechanics

A spliterator supports `tryAdvance(action)` for one element, `forEachRemaining`, `trySplit`, `estimateSize`, and `characteristics`. Common characteristics are:

- **ORDERED**: defined encounter order;
- **DISTINCT**: elements are distinct under relevant equality;
- **SORTED**: encounter order is sorted and a comparator may be available;
- **SIZED**: exact remaining size is known;
- **SUBSIZED**: split children also report exact sizes;
- **NONNULL**: source guarantees non-null elements;
- **IMMUTABLE**: source cannot be structurally modified;
- **CONCURRENT**: source supports concurrent structural modification under its contract.

`trySplit` returns a prefix or partition and leaves remaining work in the original. Balanced, cheap splitting enables useful parallelism. Array and range sources split well. Pointer-chasing, unknown-size, or I/O-backed sources may split poorly.

Characteristics are promises, not hints. A custom spliterator that falsely reports `SORTED`, `SIZED`, or `DISTINCT` can cause wrong optimizations or contract violations. Late-binding and fail-fast behavior depend on the source implementation.

HotSpot note: OpenJDK stream execution uses internal pipeline stages, fork-join tasks for many parallel pipelines, and heuristics for target partition sizes. These details and optimization shortcuts are version-sensitive.

Parallel stream decision model

Parallel streams are most plausible when input is large, CPU work per element is substantial, operations are associative and independent, splitting is balanced, and no blocking or shared mutable state is involved. They are often poor for small inputs, ordered stateful stages, blocking I/O, request-thread latency isolation, or environments where the common pool is already busy.

Measure end-to-end behavior under concurrent service load. A microbenchmark showing one pipeline speedup does not prove better tail latency or capacity in a server.

TRACE IT | 31.6 Worked Java example

The following aggregation produces immutable per-region statistics from completed orders:

JAVA

```
import java.math.BigDecimal;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

enum Status { PENDING, COMPLETED, CANCELED }
record Order(String id, String region, Status status, BigDecimal amount) {}
record RegionSummary(long count, BigDecimal total, List<String> orderIds) {}

final class OrderReport {
    static Map<String, RegionSummary> summarize(List<Order> orders) {
        return orders.stream()
            .filter(order -> order.status() == Status.COMPLETED)
            .collect(Collectors.groupingBy(
                Order::region,
                java.util.TreeMap::new,
                Collectors.collectingAndThen(
                    Collectors.toList(),
                    OrderReport::summarizeGroup)));
    }

    private static RegionSummary summarizeGroup(List<Order> group) {
        BigDecimal total = group.stream()
            .map(Order::amount)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        List<String> ids = group.stream()
            .sorted(Comparator.comparing(Order::id))
            .map(Order::id)
            .toList();

        return new RegionSummary(group.size(), total, ids);
    }
}
```

The outer `TreeMap` guarantees region order. Each list is an intermediate mutable accumulator local to collection, and the finisher converts it to an immutable summary. `BigDecimal.add` returns a new value, so there is no shared mutable numeric accumulator.

For a very large group, retaining its entire list may be unnecessary. A custom accumulator could track count, total, and IDs directly, but its combiner must merge all fields correctly. If sorted IDs are mandatory, memory proportional to group cardinality remains necessary unless output can stream to another sorted source.

TRACE IT | 31.7 Execution or memory walkthrough

Given completed orders `east/e2/$5`, `west/w1/$7`, and `east/e1/$3`, plus one canceled east order:

- 1 `filter` rejects the canceled order.
- 2 `groupingBy` asks the `TreeMap` for an accumulator list for each region.
- 3 East's list receives `e2` and `e1`; west's receives `w1`.
- 4 The downstream finisher visits each group. East total becomes $0 + 5 + 3 = 8$; IDs sort to `[e1, e2]`.
- 5 The final map is sorted by region, but it remains mutable because `groupingBy` with `TreeMap::new` returns that mutable map. If an unmodifiable outer map is required, wrap the complete collector with `collectingAndThen` and an order-preserving unmodifiable snapshot strategy.

In a parallel execution, partitions can build separate maps and lists, then combiners merge them. Encounter order for downstream lists follows the collector and source contracts; code must not assume one shared accumulator. The example remains mathematically correct because filters and mappings are pure and `BigDecimal` addition is associative in value for these exact decimal inputs, subject to any chosen math context.

Memory includes group lists containing references to orders, final ID lists containing references to existing strings, summary records, and tree map entries. Streams themselves add pipeline objects and possibly tasks/buffers, but they do not automatically duplicate every input.

31.8 Complexity and performance

Let n be input size, g number of regions, and m_i each group size. Filtering and grouping perform $O(n)$ element work plus `TreeMap` group lookup $O(\log g)$ per accepted element, so $O(n \log g)$ under this chosen map. Summing is $O(n)$. Sorting IDs costs:

TEXT

sum over groups of $O(m_i \log m_i)$, bounded by $O(n \log n)$

Space is $O(n + g)$ because group lists and final ID lists retain per-order references during finishing. A one-pass custom collector could reduce temporary duplication but not the final list storage.

Pipeline complexity composes by stages. `map` and `filter` are normally linear; `sorted` is $O(n \log n)$; `distinct` is expected $O(n)$ with hashing but depends on source/order and equality cost; `limit(k)` can be $O(k)$ sequentially with a favorable source but may coordinate more work in parallel.

Boxing matters in numerical pipelines. `stream.mapToLong(...).sum()` avoids one wrapper per mapped value. It does not fix an expensive source, I/O, or algorithm. Benchmark with JMH-style discipline and realistic cardinality before replacing a clear loop.

WATCH OUT | 31.9 Edge cases and common mistakes

- Reusing a stream after a terminal operation.
- Mutating a non-thread-safe collection from `parallel().forEach`.
- Using `peek` for required persistence or audit effects.
- Supplying a non-associative reduction and observing parallel differences.
- Mutating a shared identity in `reduce` instead of using `collect`.
- Forgetting duplicate-key behavior in `toMap`.
- Assuming collector-produced collection types, mutability, null policy, or order without a contract.
- Calling `orElse(expensive())` when lazy `orElseGet` is intended.
- Calling `optional.get()` without proving presence, recreating a less clear null check.
- Using `Optional<List<T>>` when empty list already expresses no results.
- Flat-mapping infinite streams without a termination argument.
- Placing `sorted` before a selective `filter` and sorting unnecessary elements.
- Parallelizing blocking I/O on the common pool.
- Reporting false characteristics in a custom spliterator.
- Depending on fail-fast traversal to make source mutation safe.
- Assuming `unordered()` shuffles or that parallel automatically ignores encounter order.

31.10 Production engineering notes

Prefer pipelines whose stages read like a data-flow specification and whose cardinality remains bounded. Break a complex pipeline into named methods or intermediate results when it improves observability and debugging. A loop is often better when error handling, early exits, multiple stateful outputs, or checked resource lifetimes dominate.

Keep lambdas pure. Move database calls and remote requests outside element-wise stream operations; otherwise a compact line can hide $N+1$ I/O. Batch external work explicitly and use concurrency mechanisms with controlled executors, timeouts, and backpressure.

Define output map/list order and mutability. Use explicit collection suppliers when representation matters. Treat `groupingBy` on unbounded cardinality as a memory risk. Validate keys, cap results, and avoid `distinct` on attacker-controlled unbounded streams.

Use `Optional` for an immediate maybe-one return, then unwrap or transform near that boundary. Domain-specific sealed results are clearer when absence has multiple causes. Avoid `Optional` in serialization entities unless the serializer and schema deliberately support it.

For parallel pipelines, validate algebraic correctness sequentially first. Then benchmark using the target JDK, CPU quota, container limits, and concurrent traffic. Consider a dedicated executor or structured concurrency design when work needs isolation; the convenient common pool may couple unrelated requests.

TRY IT | 31.11 Interview questions and model answers

Why are streams lazy?

Laziness lets the terminal operation pull only needed elements, fuse stateless stages, and short-circuit. Stateful operations may still buffer substantial input.

What is the difference between `map` and `flatMap`?

`map` produces one result value per input. `flatMap` maps each input to a stream and flattens all nested elements, supporting zero-to-many output.

What makes a reduction safe in parallel?

Its operation is associative, its identity is neutral, accumulator and combiner are compatible, and behavioral functions do not share mutable state.

Why should mutable reduction use `collect` rather than `reduce`?

Collector contracts explicitly provide separate accumulators and a combiner for mutable containers. Mutating a reduce identity can share state incorrectly and violates reduction assumptions.

When is `parallelStream` appropriate?

For sufficiently large, splittable, CPU-bound data with independent operations and associative aggregation, after measurement under actual deployment load. It is not a general fix for blocking I/O.

What does a spliterator contribute?

It traverses elements, can split remaining work into partitions, estimates size, and reports characteristics such as ordering, size, or distinctness that stream execution can exploit.

Why prefer `orElseGet` sometimes?

`orElse` evaluates its argument eagerly even when the optional has a value. `orElseGet` evaluates its supplier only when empty.

TRY IT | 31.12 Exercises

- 1 Rewrite a parallel `forEach` that appends to an `ArrayList` as a safe collector. State order behavior.
- 2 Show that subtraction is not associative by partitioning four integers in two different ways.
- 3 Build a nested map of department to status to count using downstream collectors.
- 4 Implement a custom collector for count and `BigDecimal` total. Prove supplier, combiner, and finisher behavior.
- 5 Compare `findFirst` and `findAny` on ordered and unordered parallel sources.
- 6 Design a spliterator for an immutable array slice. State valid characteristics and a balanced `trySplit` rule.
- 7 Refactor code using `isPresent` plus `get` into `map`, `flatMap`, `or`, or `orElseThrow` where appropriate.
- 8 Analyze memory for `sorted().limit(10)` versus a size-10 heap on ten million records.

RECAP | 31.13 Chapter summary

Streams are lazy, single-use traversal pipelines rather than containers. Correct pipelines preserve non-interference, encounter-order requirements, and associative reduction laws. Collectors formalize mutable reduction and composition but require explicit duplicate, null, order, and mutability policies. `Optional` is a focused maybe-one return type, not a universal field or parameter wrapper. Spliterators expose traversal and partition properties; parallel speed depends on honest characteristics, balanced splitting, pure CPU work, and deployment context.

TRY IT | 31.14 Revision checklist

- ☐ I can classify stream operations as intermediate, terminal, stateful, or short-circuiting.
- ☐ I understand encounter order, `forEachOrdered`, `findFirst`, and `findAny`.
- ☐ I keep stream lambdas non-interfering and side-effect-free.
- ☐ I can state associativity and identity requirements for reduction.
- ☐ I can compose grouping and downstream collectors with explicit map policies.
- ☐ I know the mutability and null guarantees I may and may not infer from collection terminals.
- ☐ I use `Optional` lazily and at appropriate absence boundaries.
- ☐ I can explain spliterator splitting and characteristics.
- ☐ I evaluate parallel streams with correctness, source shape, workload, pool contention, and measurement.

SDE-2 CORE CHAPTER

Chapter 32 - Java I/O, NIO.2, Files, Buffers, and Serialization Boundaries

32.1 Learning objectives

By the end of this chapter, you should be able to:

- choose byte, character, buffered, channel, and file APIs by data semantics;
- manage resources, partial operations, exceptions, charsets, and durability explicitly;
- use `Path` and `Files` without confusing lexical paths with filesystem identity;
- derive the `Buffer` position-limit-capacity state machine;
- explain blocking channels, selectors, direct buffers, and memory mapping at a contract boundary;
- perform safer temporary-file publication and directory traversal; and
- treat Java native serialization and all external formats as security boundaries.

WHY IT WORKS | 32.2 Why this matters at SDE-2

I/O connects Java's managed object world to files, sockets, pipes, and external data. Many production incidents originate at that boundary: leaked descriptors, default-charset corruption, partial writes, non-atomic publication, unbounded reads, path traversal, symlink races, or unsafe deserialization.

At SDE-2, you should reason beyond method names. What owns the resource? Can a read return fewer bytes? Is the output durable after `close`? Does moving a file replace atomically? Can input be trusted? Is text decoded incrementally? A robust design answers these questions and preserves them in tests and operational limits.

WHY IT WORKS | 32.3 First-principles model

I/O transfers finite sequences of bytes between a program and an external endpoint. Text is not bytes by itself; a charset maps characters to encoded bytes and back.

TEXT

```
Java characters --CharsetEncoder--> bytes --external storage/network
Java characters <--CharsetDecoder-- bytes <--external storage/network
```

Classic I/O exposes sequential streams:

- `InputStream` and `OutputStream` transfer bytes.
- `Reader` and `Writer` transfer characters.

- decorators add buffering, data conversion, compression, or other behavior.

NIO exposes buffers and channels:

TEXT

```
channel <-> ByteBuffer <-> application
```

A buffer has three core indexes satisfying:

TEXT

```
0 <= position <= limit <= capacity
```

Write into a fresh buffer from `position` toward `limit`. After filling, `flip()` sets `limit` to the amount written and `position` to zero, preparing for reads. After consuming, `clear()` prepares the entire storage for overwrite; `compact()` preserves unread bytes and prepares the remaining suffix for more input.

Specification boundary: Java APIs define resource, buffer, path, channel, and serialization behavior. Operating-system caches, filesystem atomicity support, descriptor limits, direct-buffer allocation, selector implementation, and durability details vary by platform and JDK.

32.4 Core terminology

- **Byte stream:** Sequence of raw 8-bit units.
- **Character stream:** Sequence decoded or encoded using a charset.
- **Decorator:** Wrapper that adds behavior while delegating to another stream.
- **Buffering:** Accumulating data to reduce calls and improve transfer efficiency.
- **Channel:** I/O connection supporting buffer-based operations.
- **Path:** Filesystem path representation; it may identify a nonexistent entry.
- **Partial read/write:** Successful operation transferring fewer units than requested.
- **Atomic move:** Move observed as one indivisible namespace update when supported.
- **Durability:** Extent to which data survives process or machine failure.
- **Direct buffer:** Buffer whose storage is intended to support efficient native I/O outside the ordinary Java heap array model.
- **Memory mapping:** Mapping a file region into virtual memory through a buffer-like view.
- **Serialization:** Encoding an object or data model into a transportable representation.

32.5 Detailed mechanics

Bytes, characters, and charsets

Use byte APIs for images, compressed data, protocols, hashes, and already encoded content. Use readers and writers for text. Always specify the charset at persistent or network boundaries, commonly `StandardCharsets.UTF_8`. The platform default can differ across environments or be configured, and relying on it makes formats ambiguous.

Unicode characters may require multiple bytes, and a buffer boundary can split one encoded character.

`InputStreamReader` and `OutputStreamWriter` maintain encoding state correctly. Converting arbitrary chunks independently with `new String(chunk, UTF_8)` can corrupt a multibyte character split across chunks.

Malformed or unmappable input has a policy. Convenience decoding may replace invalid sequences. When strict validation matters, configure a `CharsetDecoder` with `CodingErrorAction.REPORT`.

Stream layering and buffering

Streams compose:

JAVA

```
try (var in = new java.io.BufferedInputStream(
    new java.util.zip.GZIPInputStream(
        java.nio.file.Files.newInputStream(path))) ) {
    // consume decompressed bytes
}
```

The outermost close normally closes delegated resources, but check third-party wrappers. Buffering reduces calls across Java/native or network boundaries. It does not change asymptotic byte count, and wrapping an already buffered API repeatedly can waste memory.

`read()` returns an unsigned byte value `0..255` or `-1` for end-of-stream. Bulk `read(byte[])` can return any positive count up to array length, `-1` at end, and in some APIs zero under specified circumstances. Never assume one read fills a requested buffer. Similarly, a channel write can consume fewer remaining bytes, so loop until completion or until a nonblocking policy says to wait.

Resource ownership and try-with-resources

Open streams, channels, directory streams, and file walks must be closed. Try-with-resources closes in reverse declaration order even when work throws. If both work and close throw, the work exception is primary and close exceptions are suppressed:

JAVA

```
catch (IOException ex) {
    for (Throwable suppressed : ex.getSuppressed()) {
        log(suppressed);
    }
}
```

Do not close a resource you merely borrow unless the API transfers ownership. A method accepting an `OutputStream` often should not close it; document whether it flushes. Conversely, a method that opens a path internally should normally own and close the opened resource.

`flush` asks buffered layers to push data downstream. It does not necessarily make bytes durable on physical storage. File-channel `force` requests stronger synchronization, but filesystem, device, and platform semantics still matter. Durable transactional publication may require syncing both file data and directory metadata according to platform-specific design.

Path and Files

`Path` is a value-like representation of path components. It can be relative or absolute and does not imply existence. Useful distinctions:

- `normalize()` removes redundant lexical `.` and resolvable `..` segments without accessing the filesystem.
- `toAbsolutePath()` anchors a relative path but does not resolve symbolic links.
- `toRealPath()` accesses the filesystem, resolves links by default, and requires existence.
- `resolve` joins paths; if the argument is absolute, it may replace the base.
- `relativize` computes a lexical relative path between compatible paths.

Files methods offer creation, copying, moving, metadata, directory streams, buffered readers, and attributes. Check options precisely: for example, `CREATE_NEW` fails if the target already exists, while `TRUNCATE_EXISTING` destroys previous content once opening succeeds.

`Files.move` with `ATOMIC_MOVE` requests atomicity; providers may throw `AtomicMoveNotSupportedException`. Cross-filesystem moves generally cannot be one atomic rename. `REPLACE_EXISTING` controls collision behavior, but replacement details can vary. Design and test a fallback explicitly rather than silently weakening guarantees.

Safe path containment and symbolic links

This check is only lexical:

JAVA

```
Path candidate = root.resolve(userInput).normalize();
if (!candidate.startsWith(root.normalize())) reject();
```

It blocks simple `..` traversal but does not defeat symlinks or validation-use races. Security-sensitive access needs a trusted root, link-aware operations, and a platform-specific threat model.

Directory traversal methods such as `Files.list`, `Files.walk`, and `Files.find` return streams backed by open resources and must be closed. Avoid following symbolic links unless cycle and trust behavior is deliberate. Never read an untrusted entire file with `readAllBytes` without a size limit; file size can change after checking.

Buffer state transitions

For `ByteBuffer.allocate(8)`:

TEXT

```
initial: position=0, limit=8, capacity=8
put 3 bytes: position=3, limit=8
flip: position=0, limit=3
get 2 bytes: position=2, limit=3, remaining=1
compact: unread byte moves to index 0; position=1, limit=8
```

`clear()` does not erase bytes; it resets indexes so storage can be overwritten. `rewind()` sets position to zero without changing limit, allowing rereading current content. `mark` records a position and `reset` returns to it when the mark remains valid. Relative gets advance position; absolute indexed gets do not.

Buffer overflow and underflow indicate state-machine errors: writing beyond remaining space or reading beyond the limit. `slice` and `duplicate` create buffers with independent position/limit state but shared underlying content. Mutation is visible through aliases. Read-only buffers reject writes but may still reflect shared storage changes.

Byte order matters for multi-byte numeric values. Network protocols often specify big-endian order. Set and document `ByteOrder` rather than relying on defaults or native order.

Channels, nonblocking I/O, and selectors

File channels support positioned reads/writes, transfers, locking, mapping, and forcing. Socket channels can be blocking or nonblocking. In nonblocking mode, zero-byte progress can be a normal result, so a readiness loop registers channels with a `Selector` and reacts to acceptable, connectable, readable, or writable keys.

Readiness means an operation is likely to make progress, not that an entire message is available. Protocol framing must accumulate partial bytes across events. A selected key can become stale; handle cancellation, closure, and exceptions without spinning. Interest in write readiness should generally be enabled only while pending output exists, or a loop may wake continuously.

Heap, direct, and mapped buffers

Heap buffers use managed memory and may expose an array. Direct buffers can reduce copying for native transfers, but allocation and reclamation are costlier and memory sits outside normal heap sizing. Mapped buffers expose file regions through virtual memory; page faults, truncation, flush, and unmapping are platform-sensitive. Neither `force` nor mapping creates an application transaction.

HotSpot note: Direct-buffer cleanup, native memory accounting, zero-copy transfer paths, selector providers, and mapped-buffer unmapping are implementation and OS sensitive. Do not depend on internal cleaner APIs.

Serialization boundaries

Java native serialization reconstructs object graphs for types implementing `Serializable`. It captures private implementation shape, uses `serialVersionUID` for compatibility checks, supports custom hooks, and can invoke code during reconstruction. This makes it brittle for long-lived schemas and dangerous for untrusted data.

Never deserialize an untrusted native object stream merely because classes appear familiar. Apply process and protocol boundaries, strict allow-list filters where legacy serialization is unavoidable, size/depth/reference limits, and current security guidance. Prefer explicit schemas such as validated JSON, protocol buffers, or another designed wire format for service boundaries. Those formats also require input limits and safe parser configuration.

`transient` excludes a field from default serialization; it is not encryption. Constructors of ordinary serializable classes are not invoked in the usual way during reconstruction, so validate invariants in controlled reconstruction paths. A stable `serialVersionUID` prevents only certain version mismatch failures; it does not make schema evolution automatically correct or secure.

Specification boundary: Native serialization defines a protocol and hooks, but class evolution has limited compatibility rules. It is not a general-purpose safe RPC format and should be treated as a legacy trust boundary.

TRACE IT | 32.6 Worked Java example

This method writes UTF-8 text to a sibling temporary file, forces file content, and attempts atomic publication:

JAVA (continued 1/2)

```
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.charset.StandardCharsets;
import java.nio.file.AtomicMoveNotSupportedException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.nio.file.StandardOpenOption;

final class AtomicTextFile {
    static void replace(Path target, Iterable<String> lines) throws IOException {
        Path absolute = target.toAbsolutePath();
        Path parent = absolute.getParent();
        if (parent == null) throw new IOException("target has no parent");
        Files.createDirectories(parent);

        Path temp = Files.createTempFile(parent, ".pending-", ".tmp");
        boolean published = false;
        try {
            try (FileChannel channel = FileChannel.open(
                temp, StandardOpenOption.WRITE,
                StandardOpenOption.TRUNCATE_EXISTING)) {
                for (String line : lines) {
                    if (line.indexOf('\n') >= 0 || line.indexOf('\r') >= 0) {
                        throw new IllegalArgumentException("line contains a newline");
                    }
                }
            }
            published = true;
        } catch (IOException e) {
            // ... (rest of the method logic)
        }
    }
}
```

JAVA (continued 2/2)

```

        }
        byte[] encoded = (line + "\n").getBytes(StandardCharsets.UTF_8);
        ByteBuffer buffer = ByteBuffer.wrap(encoded);
        while (buffer.hasRemaining()) {
            channel.write(buffer);
        }
        channel.force(true);
    }

    try {
        Files.move(temp, absolute,
            StandardCopyOption.ATOMIC_MOVE,
            StandardCopyOption.REPLACE_EXISTING);
    } catch (AtomicMoveNotSupportedException unsupported) {
        // This fallback preserves complete-file publication but not atomic replacement.
        Files.move(temp, absolute, StandardCopyOption.REPLACE_EXISTING);
    }
    published = true;
} finally {
    if (!published) {
        Files.deleteIfExists(temp);
    }
}
}
}

```

Creating the temporary file in the target directory improves the chance that source and target share a filesystem, which is usually necessary for atomic rename. The fallback is an explicit weakening; a system that requires atomicity should propagate failure instead. The example does not claim full crash durability of directory metadata on every platform.

TRACE IT | 32.7 Execution or memory walkthrough

For lines `alpha` and `beta`:

- 1 A unique temp file is created in the target directory.
- 2 `alpha\n` encodes to six UTF-8 bytes and `ByteBuffer.wrap` starts with `position=0`, `limit=6`, `capacity=6`.
- 3 Each channel write advances position by the actual count. The loop handles partial writes until `position == limit`.
- 4 The same process writes `beta\n`. Each encoded byte array becomes unreachable after its iteration.
- 5 `force(true)` requests synchronization of content and metadata for the file channel.
- 6 Closing the channel releases its descriptor.
- 7 Atomic move is attempted. Before the namespace update, readers see the old target; after it, they see the complete new target under supporting provider semantics.
- 8 If writing or moving fails, the `finally` block attempts to remove the temporary file. The original target is not truncated by the write phase.

This design uses memory proportional to the largest encoded line, not the whole file. It still permits an arbitrarily large single line and an infinite iterable; callers may need explicit byte and line limits.

32.8 Complexity and performance

Writing `b` encoded bytes is $O(b)$ time and uses $O(\text{maxLineBytes})$ temporary memory in the example. Filesystem and device latency dominate CPU complexity. Buffering changes call counts and constants, not the amount of data.

Task	Typical time	Important caveat
sequential read/write	$O(\text{bytes})$	partial operations and device latency
path <code>normalize</code>	$O(\text{components})$	lexical only
directory traversal	$O(\text{entries})$	metadata calls, symlinks, open resources
copy or move across filesystems	$O(\text{bytes})$	cannot usually be atomic
same-filesystem rename	often near $O(1)$ namespace work	provider-dependent atomicity/durability
buffer flip/clear	$O(1)$	clear does not erase content
buffer compact	$O(\text{unread bytes})$	moves retained content
memory map access	$O(\text{bytes touched})$ logical	page faults and OS caching dominate

Many tiny reads and writes pay syscall, locking, TLS, or device overhead repeatedly. Choose buffer sizes empirically. Huge buffers increase latency to first processing, direct-memory use, and concurrent footprint. Zero-copy channel transfers can help large file movement on supported systems but can fall back internally.

WATCH OUT | 32.9 Edge cases and common mistakes

- Relying on the platform default charset for stored or transmitted text.
- Assuming one read fills an array or one write drains a buffer.
- Forgetting that `read` uses `-1` for end-of-stream.
- Leaking streams returned by `Files.list` or `Files.walk`.
- Calling `readAllBytes` on unbounded input.
- Confusing `normalize`, absolute path, and real path.
- Treating a lexical `startsWith` containment check as symlink-safe authorization.
- Assuming `ATOMIC_MOVE` is supported or silently weakening a required guarantee.
- Believing `flush` or close alone guarantees crash durability.
- Calling `clear()` and assuming sensitive bytes were erased.
- Forgetting `flip()` before reading data just written to a buffer.
- Losing partial protocol frames between nonblocking reads.
- Enabling selector write interest continuously and causing a busy loop.
- Allocating unbounded direct buffers and watching only Java heap metrics.
- Mutating shared storage through a sliced or duplicated buffer alias.
- Deserializing native Java objects from untrusted input.
- Treating `transient` as confidentiality or `serialVersionUID` as safe evolution.

32.10 Production engineering notes

Every I/O entry point needs limits: maximum bytes, lines, nesting, files, traversal depth, open handles, concurrency, and time. Use streaming parsing rather than whole-file loading for large content. Reject overlong records before allocating proportional memory.

Make ownership explicit. The layer that opens a resource should normally close it. When APIs accept caller-owned streams, state close and flush behavior. Propagate interruption and cancellation through blocking workflows; configure socket connect/read deadlines and handle stalled peers.

Use temp-write plus rename for complete-file publication, with a documented policy when atomic move is unavailable. Separate atomic visibility from crash durability; use a storage engine when durable transactions are required.

Protect endpoints with least-privilege permissions, trusted roots, generated names, and symlink-aware operations. Validate archive paths and decompression ratios.

Measure bytes, latency, open handles, failures, direct memory, and rejected input. Avoid logging payloads. Isolate legacy native serialization with narrow filters, graph limits, an allow-list, and a migration plan.

TRY IT | 32.11 Interview questions and model answers

When do you use a Reader instead of an InputStream?

Use a reader when the data is text and must be decoded with a known charset. Use an input stream for raw bytes. An `InputStreamReader` bridges them while handling multibyte boundaries.

Why must channel writes be in a loop?

A successful write may consume fewer than all remaining bytes, especially for sockets or nonblocking channels. Advance is reflected in buffer position, so loop while `hasRemaining`, subject to readiness and timeout policy.

Explain flip, clear, and compact.

`flip` changes from filling to draining by setting limit to current position and position to zero. `clear` prepares all storage for overwrite without erasing it. `compact` preserves unread bytes, moves them to the front, and prepares the remaining area for more input.

Is `Path.normalize()` sufficient to prevent path traversal?

No. It is a lexical operation. It can reject simple `..` escapes but does not resolve symlinks or remove validation-use races. Security-sensitive access needs a trusted-root and filesystem-aware strategy.

Does try-with-resources lose close exceptions?

If work throws, close exceptions are attached as suppressed exceptions to the primary failure. Resources close in reverse declaration order.

Why is Java native deserialization dangerous?

It reconstructs graphs and can invoke class-specific code from attacker-controlled stream content. Crafted graphs can abuse gadget paths or exhaust resources. Do not use it for untrusted data; tightly filter and bound unavoidable legacy uses.

Do virtual threads replace NIO?

They make blocking-style concurrency practical for many workloads, but do not remove protocol framing, timeouts, capacity, partial I/O, or the value of event loops in specialized systems.

TRY IT | 32.12 Exercises

- 1 Write a byte-copy loop that handles partial reads and writes with a reusable `ByteBuffer`. Dry-run its indexes for a split read.
- 2 Decode UTF-8 using a strict `CharsetDecoder` and reject malformed input.
- 3 Compare `normalize`, `toAbsolutePath`, and `toRealPath` for a path containing `..` and a symbolic link.
- 4 Implement a bounded line reader that rejects more than one megabyte or ten thousand lines.
- 5 Modify the worked example so atomic move support is mandatory. Explain the operational consequence.
- 6 Design a nonblocking length-prefixed message decoder that preserves incomplete header and body bytes across reads.
- 7 Threat-model archive extraction, including traversal, symlinks, entry count, and decompression bombs.
- 8 Propose a migration plan from a native serialized cache to a versioned explicit schema.

RECAP | 32.13 Chapter summary

Java I/O begins with byte semantics, explicit character encoding, partial transfer, and resource ownership. Classic streams compose decorators; NIO channels move data through buffers governed by position, limit, and capacity. `Path` is a representation, while filesystem identity, symlinks, atomic moves, and durability are provider concerns. Direct and mapped buffers trade heap copying for native lifecycle complexity. Every external format is a trust boundary, and native Java deserialization is especially risky. Production I/O must be bounded, observable, charset-explicit, and honest about atomicity and durability.

TRY IT | 32.14 Revision checklist

- ☐ I choose byte versus character APIs and always define external charsets.
- ☐ I handle partial reads/writes, EOF, buffering, and close ownership.
- ☐ I understand suppressed exceptions in try-with-resources.
- ☐ I distinguish lexical, absolute, and real paths.
- ☐ I can explain atomic visibility versus crash durability.
- ☐ I can dry-run `position`, `limit`, and `capacity` through flip, clear, rewind, and compact.
- ☐ I understand shared content in buffer slices and duplicates.
- ☐ I can explain selector readiness and partial protocol framing.
- ☐ I monitor direct/native memory as well as heap.
- ☐ I reject unbounded input and unsafe native deserialization.

JAVA FOUNDATIONS TO ADVANCED ENGINEERING

Part V - Concurrency and Multithreading

A focused part of the complete SDE-2 preparation guide

SDE-2 CORE CHAPTER

Chapter 33 - Threads, Lifecycle, Interruption, and Cancellation

33.1 Learning objectives

- Explain Java thread states as observable lifecycle categories rather than scheduler commands.
- Distinguish `start`, `run`, `sleep`, `waiting`, `blocking`, `joining`, `daemon` status, and `termination`.
- Use interruption as a cooperative cancellation protocol.
- Propagate or restore `InterruptedException` correctly.
- Design thread-owning components with explicit startup, bounded shutdown, and failure reporting.

WHY IT WORKS | 33.2 Why this matters at SDE-2

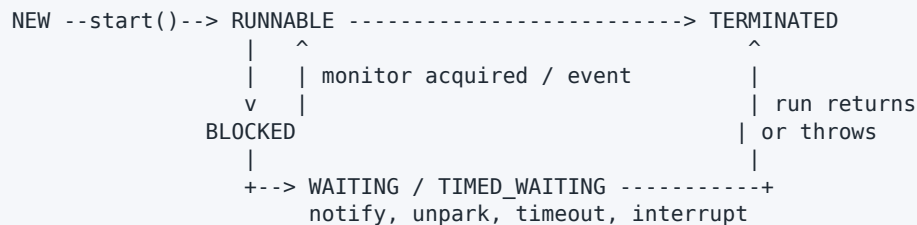
Services fail during shutdown as often as during steady state. A worker that swallows interruption can delay deployment, lose a partition lease, or leave a process to be killed after its grace period. A thread created without ownership can outlive the component that created it. A daemon thread can disappear before flushing critical work. An uncaught exception can silently remove capacity if no one observes it.

At SDE-2, "interrupt stops a thread" is not acceptable. Interruption requests cooperation. The target decides when and how to stop, blocking methods define how they react, and ownership code must wait for termination or report that it could not. These rules apply later to executors, futures, virtual threads, and structured concurrency designs.

WHY IT WORKS | 33.3 First-principles model

A thread is an independently scheduled sequence of Java actions sharing process resources with other threads. Creating a `Thread` object does not begin concurrent execution. Calling `start()` asks the runtime to schedule a new execution that invokes `run()` once.

TEXT



RUNNABLE includes threads actually executing and threads ready but waiting for CPU or certain native operations. Java's state enum is a diagnostic snapshot. It is not a complete OS scheduler model, and code should not coordinate correctness by polling another thread's state.

Cancellation is a protocol:

TEXT

```
owner requests cancellation
-> publish cancellation state and/or interrupt
-> worker notices at a cancellation point
-> worker preserves invariants and releases resources
-> worker terminates
-> owner joins or otherwise observes completion
```

33.4 Core terminology

- **Platform thread:** A traditional Java thread commonly mapped closely to an operating-system thread.
- **Virtual thread:** A lightweight Java thread scheduled by the runtime; covered deeply in Chapter 37.
- **Interrupt status:** Per-thread boolean cancellation signal manipulated by `interrupt`, `isInterrupted`, and `interrupted`.
- **Cancellation point:** A place where code checks status or calls an interruptible blocking API.
- **Daemon thread:** Thread that does not by itself keep the JVM alive.
- **Join:** Waiting for another thread to terminate.
- **Uncaught exception handler:** Last-resort observer invoked when a thread exits because `run` threw an uncaught exception.
- **Cooperative cancellation:** Target code participates in stopping at safe boundaries.
- **Thread confinement:** Mutable state accessed by only one thread.

33.5 Detailed mechanics

`Thread.start()` has two important properties. It creates a new concurrent execution of `run`, and actions before `start` happen-before actions in the started thread. Calling `run()` directly is an ordinary synchronous method call on the current thread. A `Thread` instance can be started at most once; a second `start()` throws `IllegalThreadStateException` even after termination.

The six `Thread.State` values are:

- **NEW**: constructed but not started.
- **RUNNABLE**: executing or eligible to execute in the JVM/OS view.
- **BLOCKED**: waiting to enter a `synchronized` monitor.
- **WAITING**: waiting indefinitely through operations such as untimed `join`, `wait`, or `park`.
- **TIMED_WAITING**: waiting with a deadline, including `sleep`, timed `join`, timed `wait`, or timed `park`.
- **TERMINATED**: `run` completed normally or abruptly.

State can change immediately after observation. It is useful for thread dumps, not for writing `if (thread.getState() == WAITING) ...` protocols.

Calling `interrupt()` normally sets the target's interrupt status. If the target is in an interruptible blocking method such as `sleep`, `wait`, `join`, or many `java.util.concurrent` waits, that method usually throws `InterruptedException` and clears the status. The exception is not an error to log and ignore. It is a request arriving through the method's control flow.

There are two standard handling choices:

- 1 **Propagate**: A method that can abandon its work declares `throws InterruptedException`, performs required local cleanup, and lets its caller choose policy.
- 2 **Restore and stop/return**: Code at a boundary that cannot throw restores status with `Thread.currentThread().interrupt()` and exits or reports cancellation.

Swallowing the exception loses the request. Restoring status and blindly continuing the same interruptible loop can cause immediate repeated failures, so restoration must accompany a deliberate boundary decision.

`Thread.interrupted()` returns and clears the current thread's status.

`currentThread().isInterrupted()` observes without clearing. This naming difference causes bugs. A CPU-bound loop should check status at a reasonable frequency:

JAVA

```
while (!Thread.currentThread().isInterrupted()) {  
    computeOneChunk();  
}
```

Not every block responds to interruption. Monitor entry for `synchronized` is not interruptible. Some legacy or native I/O may require closing a resource or using an API-specific cancellation mechanism. `interrupt()` cannot make arbitrary code safe to stop.

`join()` waits for termination and establishes a memory edge: all actions in the terminated thread happen-before another thread successfully returns from `join`. A timeout only bounds the wait; it does not terminate the target. Code must check `isAlive()` or use a higher-level completion result after a timed join.

Daemon status must be set before start. The JVM can exit when only daemon threads remain; it does not promise daemon `finally` blocks complete. Critical persistence, commits, lease release, and telemetry flushes need explicit lifecycle coordination rather than daemon reliance.

An uncaught exception terminates that thread. The JVM consults the thread's handler, thread group behavior, or default handler. Logging is useful, but a robust component also converts worker loss into health state or supervisor action. Restarting blindly can create a crash loop or duplicate unsafe work.

Specification boundary: The JLS and Java APIs define thread actions, lifecycle methods, interruption status, and happens-before rules. They do not promise fair scheduling, immediate execution after `start`, immediate response to interruption, an OS thread mapping, or a maximum shutdown time.

TRACE IT | 33.6 Worked Java example

JAVA (continued 1/2)

```
import java.time.Duration;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.TimeUnit;

public final class OwnedWorker implements AutoCloseable {
    private final BlockingQueue<String> jobs = new LinkedBlockingQueue<>();
    private final Thread thread = new Thread(this::runLoop, "owned-worker");
    private volatile boolean stopping;
    private boolean started;

    public synchronized void start() {
        if (started) throw new IllegalStateException("already started");
        if (stopping) throw new IllegalStateException("already stopped");
        started = true;
        thread.start();
    }

    public synchronized void submit(String job) {
        if (stopping) throw new IllegalStateException("stopping");
        jobs.add(job);
    }

    private void runLoop() {
        try {
            while (!stopping) {
                String job = jobs.poll(1, TimeUnit.SECONDS);
                if (job != null) process(job);
            }
        } catch (InterruptedException interrupted) {
            Thread.currentThread().interrupt();
        }
    }
}
```

JAVA (continued 2/2)

```
        } finally {
            releaseWorkerResources();
        }
    }

    private void process(String job) {
        System.out.println(job);
    }

    private void releaseWorkerResources() {}

    public void stop(Duration timeout) throws InterruptedException {
        if (timeout.isNegative() || timeout.isZero()) {
            throw new IllegalArgumentException("positive timeout required");
        }
        synchronized (this) {
            stopping = true;
        }
        thread.interrupt();
        thread.join(Math.max(1, timeout.toMillis()));
        if (thread.isAlive()) {
            throw new IllegalStateException("worker did not stop in " + timeout);
        }
    }

    @Override
    public void close() throws InterruptedException {
        stop(Duration.ofSeconds(5));
    }
}
```

The object has explicit ownership. Construction does not leak `this` by starting a thread. `start()` is one-shot. Shutdown publishes `stopping`, interrupts the queue wait, joins with a bound, and reports failure instead of pretending the worker died.

TRACE IT | 33.7 Execution or memory walkthrough

- 1 The owner constructs `OwnedWorker`. The internal thread is `NEW`; no concurrent access begins.
- 2 Synchronized `start` marks the component started and calls `Thread.start`. Prior construction actions are visible to the worker through the start happens-before edge.
- 3 `runLoop` polls the thread-safe queue. With no work, the worker is generally `TIMED_WAITING` in the queue implementation.
- 4 `submit("A")` publishes the element through `BlockingQueue`'s memory-consistency contract. The worker obtains it and calls `process`.
- 5 The owner calls `stop`. The volatile write `stopping=true` becomes visible to later volatile reads, and `interrupt` wakes an interruptible poll promptly.
- 6 `poll` throws `InterruptedException`, clearing status. The catch restores it because this boundary is terminating rather than propagating.
- 7 `finally` releases resources. `runLoop` returns, and the thread becomes `TERMINATED`.
- 8 Successful `join` lets the owner observe all worker actions. If timeout expires first, `isAlive()` causes a visible shutdown failure.

There is a deliberate policy question: queued jobs may remain unprocessed. A drain-before-stop service would reject new jobs, process the existing queue, then interrupt only after a grace deadline. Cancellation semantics must be documented rather than accidental.

33.8 Complexity and performance

Thread creation and platform-thread stacks consume nontrivial native/runtime resources. Starting one platform thread per tiny task can spend more time scheduling than executing. A blocking queue offers $O(1)$ expected enqueue/dequeue for typical linked implementations, but contention and wakeups dominate constants.

Polling every second is not needed for visibility because `interrupt` wakes the wait; an untimed `take()` could reduce periodic wakeups. The timed poll in the example also provides a natural health/cancellation checkpoint. A busy loop would offer lower theoretical response latency but waste CPU.

Cancellation check frequency trades overhead for response time. Check between bounded chunks, not once per trivial arithmetic operation and not only after an unbounded task. Blocking operations should have deadlines when external systems can hang independently of interruption.

HotSpot note: Platform-thread stack reservation, native mapping, scheduling transitions, safepoint interaction, and diagnostic state details are HotSpot/OS dependent. Thread priorities are hints mapped differently across operating systems and should not enforce correctness.

WATCH OUT | 33.9 Edge cases and common mistakes

- Calling `run()` when concurrency was intended.
- Starting a thread in a constructor, allowing `this` to escape before construction completes.
- Reusing a terminated `Thread` object.
- Assuming `interrupt()` kills the target or closes every I/O operation.
- Catching `InterruptedException`, logging it, and continuing.
- Restoring interrupt status but continuing an interruptible call in a tight loop.
- Using `Thread.stop`, `suspend`, or `resume`; these unsafe/deprecated mechanisms can expose broken invariants or deadlock.
- Depending on thread state polling for coordination.
- Marking critical workers daemon and expecting guaranteed cleanup.
- Waiting forever during shutdown without a deadline or diagnostics.
- Treating an uncaught-exception log as sufficient supervision.

`sleep` does not release monitors a thread owns. `wait` releases the particular monitor as part of its condition-wait protocol. Confusing them can create long lock stalls.

33.10 Production engineering notes

Every thread needs an owner, name, failure policy, cancellation mechanism, and termination observation. Prefer executors or structured task scopes where available over scattered raw threads, but the same ownership questions remain. Name threads with subsystem purpose rather than per-request secrets.

A shutdown sequence should normally:

- 1 Mark the component not ready and stop admitting work.
- 2 Request cooperative cancellation or graceful draining.
- 3 Close resources that unblock non-interruptible operations.
- 4 Wait with a deadline derived from the platform's termination grace period.
- 5 Capture thread stacks and report remaining work if the deadline expires.
- 6 Let process-level orchestration escalate only after evidence and policy.

Production incident example: a consumer deployment takes the full 30-second grace period. A thread dump shows the worker blocked in `BlockingQueue.take`; shutdown set a plain `boolean` but never interrupted. Because no item arrives, the loop cannot recheck the flag. Fix the protocol by safe publication plus `interrupt`/queue signal, then join and expose shutdown duration.

Do not use interruption as a generic business error. Preserve its meaning as cancellation. If a library translates it, include the original cause and restore status unless the API contract has transferred cancellation through another explicit result.

TRY IT | 33.11 Interview questions and model answers

What is the difference between `start()` and `run()`?

`start()` arranges a new thread of execution that invokes `run` once and creates a happens-before edge from earlier actions. Calling `run()` directly is a normal synchronous call on the current thread.

How does interruption work?

It is a cooperative request represented by interrupt status. Interruptible blocking APIs often throw `InterruptedException` and clear status. Code should propagate the exception or restore status and exit at a boundary. It cannot safely force arbitrary code to stop.

Why restore interrupt status?

If a method cannot propagate `InterruptedException`, restoring preserves the cancellation signal for higher-level code. Restoration should be paired with return, termination, or another explicit policy, not swallowed by continued work.

What does `join()` guarantee?

It waits for termination. When it returns successfully because the target terminated, all actions in that thread happen-before actions after the join in the waiting thread. A timed join can return while the target remains alive.

Daemon versus non-daemon?

Non-daemon threads keep the JVM alive. The JVM may exit when only daemon threads remain, without guaranteeing daemon cleanup. Daemon status is therefore unsuitable as the sole lifecycle policy for critical work.

TRY IT | 33.12 Exercises

- 1 Change the worker to graceful drain semantics and define what happens at the deadline.
- 2 Write a CPU-bound search that responds to interruption every bounded chunk.
- 3 Demonstrate `Thread.interrupted()` clearing status and `isInterrupted()` preserving it.
- 4 Add an uncaught-exception handler that marks component health unhealthy.
- 5 Diagnose a shutdown where a worker is blocked in socket I/O that ignores interruption.
- 6 Explain the happens-before paths created by `start`, queue handoff, volatile stop, and `join`.

RECAP | 33.13 Chapter summary

Java threads move through diagnostic lifecycle states, but correctness comes from explicit synchronization and ownership, not state polling. Interruption is a cooperative cancellation request. Interruptible waits convert it to `InterruptedException`, which must be propagated or restored with a deliberate exit policy. Reliable components stop admission, request cancellation, unblock waits, release resources, join with a deadline, and report nontermination.

TRY IT | 33.14 Revision checklist

- ☐ I can explain all `Thread.State` values without treating them as scheduler guarantees.
- ☐ I know why `start` differs from `run` and cannot be repeated.
- ☐ I can propagate or restore interruption correctly.
- ☐ I can identify APIs that may require cancellation beyond interruption.
- ☐ I can design owned startup and bounded shutdown.
- ☐ I understand daemon and uncaught-exception risks.
- ☐ I can state the `start` and `join` happens-before rules.

SDE-2 CORE CHAPTER

Chapter 34 - Synchronization, Intrinsic Locks, and Explicit Locks

34.1 Learning objectives

- Explain mutual exclusion and memory visibility provided by Java monitors.
- Choose the correct lock object and scope for a shared invariant.
- Use `wait`, `notifyAll`, `ReentrantLock`, and `Condition` with condition predicates.
- Handle interruption and exceptions without leaking locks.
- Compare intrinsic locks, explicit locks, read-write strategies, and lock-free alternatives.

WHY IT WORKS | 34.2 Why this matters at SDE-2

Most lock bugs are design bugs, not syntax bugs. Synchronizing a setter while leaving a reader unguarded does not protect an invariant. Calling external code while holding a lock can turn one slow dependency into global request serialization. A missed signal or `if` around `wait` can hang only under rare timing. An explicit lock without `finally` can permanently block a subsystem after one exception.

An SDE-2 should state what data a lock guards, which operations must be atomic together, how waiting threads are signaled, and how shutdown interrupts those waits. The goal is not "make the method thread-safe" but preserve a named invariant under all interleavings.

WHY IT WORKS | 34.3 First-principles model

A lock establishes ownership of a critical section. Mutual exclusion ensures at most one holder executes code guarded by that lock. The Java Memory Model also orders state:

TEXT

Thread A	Thread B
lock L	lock L (later)
mutate guarded state	
unlock L ----- happens-before ---->	read guarded state

Condition waiting adds another idea. A thread cannot make progress until a predicate over guarded state becomes true. It must atomically release the lock and wait, then reacquire before rechecking:

TEXT

```
lock
while (!predicate) {
    await: release lock + wait + reacquire lock
}
change guarded state
unlock
```

Signals are hints that state may have changed. The predicate, not the notification, determines permission to proceed.

34.4 Core terminology

- **Monitor/intrinsic lock:** Lock and wait-set behavior associated with every Java object.
- **Critical section:** Code that accesses a shared invariant under its guarding lock.
- **Reentrancy:** A thread holding a lock can acquire it again, with a hold count.
- **Contention:** Multiple threads compete for the same lock.
- **Condition predicate:** Boolean expression over guarded state required for progress.
- **Wait set:** Threads waiting through `Object.wait` on a monitor.
- **Condition:** Explicit wait queue associated with a `Lock`.
- **Fair lock:** Lock configured to favor longer-waiting contenders under documented limitations.
- **Read-write lock:** Lock with shared read and exclusive write modes.
- **Optimistic read:** Read attempted without exclusive ownership, followed by validation.

34.5 Detailed mechanics

Every object can serve as a monitor. Entering a `synchronized (lock)` statement acquires that monitor; normal or abrupt exit releases it. An instance `synchronized` method locks `this`. A static `synchronized` method locks the corresponding `Class` object. These are different locks, so mixing instance and static synchronization does not serialize access unless that is the intended design.

Intrinsic locks are reentrant. If method A holds `this` and calls `synchronized` method B on the same object, the thread proceeds and increments its logical hold count. It must exit the matching number of acquisitions before another thread can enter.

Monitor exit happens-before a later monitor entry on the same object. This supplies visibility for all earlier actions, not just fields declared inside the `synchronized` block. The discipline still requires every conflicting access to participate through the same lock or another valid ordering mechanism.

`Object.wait()` requires the caller to own that object's monitor. It releases that monitor and waits; upon wakeup it must reacquire before returning. It can return due to notification, interruption, timeout, or spurious wakeup. Therefore test the predicate in a loop. `notify()` wakes one arbitrary waiter; using it when different

predicates share a wait set can wake an ineligible thread and strand the eligible one. `notifyAll()` is safer for correctness, though potentially more expensive.

`sleep()` does not release owned monitors. `yield()` is only a scheduling hint. Neither creates a happens-before relationship suitable for publication.

`ReentrantLock` provides monitor-like exclusion and JMM effects with additional operations:

- `tryLock()` avoids indefinite acquisition and supports timed attempts.
- `lockInterruptibly()` lets cancellation abort lock acquisition.
- multiple `Condition` instances separate wait sets for different predicates;
- optional fairness influences admission policy;
- lock state and queue inspection support diagnostics, with snapshot limitations.

Always unlock in `finally` after a successful acquisition:

JAVA

```
lock.lock();
try {
    changeState();
} finally {
    lock.unlock();
}
```

Do not place `lock()` inside the `try` if acquisition can fail/interrupt and code might then unlock a lock it never acquired. With timed `tryLock`, branch on the boolean.

A `Condition` belongs to one lock. `await` releases that lock and reacquires before returning, with interruptible, uninterruptible, and timed variants. `signal/signalAll` require lock ownership. As with monitors, use predicate loops.

`ReentrantReadWriteLock` can improve throughput when read sections are long, writes rare, and contention material. For tiny state, bookkeeping and writer starvation concerns can make a normal lock faster. Upgrade from read to write is hazardous; release/read then acquire/write and revalidate, or use a documented conversion mechanism.

`StampedLock` offers write, read, and optimistic-read modes but is not reentrant. Optimistic reads must validate and must not expose inconsistent intermediate data before validation. It has different interruption and ownership semantics from `Lock`, so it is an advanced measured optimization, not a default.

Specification boundary: Java APIs and the JMM define monitor/lock exclusion, waiting contracts, and memory-consistency effects. They do not promise fair scheduling for intrinsic locks, a particular queue algorithm, adaptive spinning, or an exact mapping to OS mutexes.

TRACE IT | 34.6 Worked Java example

JAVA (continued 1/2)

```

import java.util.ArrayDeque;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public final class BoundedBuffer<E> {
    private final ArrayDeque<E> elements = new ArrayDeque<>();
    private final int capacity;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notEmpty = lock.newCondition();
    private final Condition notFull = lock.newCondition();

    public BoundedBuffer(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity");
        this.capacity = capacity;
    }

    public void put(E element) throws InterruptedException {
        if (element == null) throw new NullPointerException("element");
        lock.lockInterruptibly();
        try {
            while (elements.size() == capacity) {
                notFull.await();
            }
            elements.addLast(element);
            notEmpty.signal();
        } finally {
            lock.unlock();
        }
    }

```

JAVA (continued 2/2)

```

    }
}

public E take() throws InterruptedException {
    lock.lockInterruptibly();
    try {
        while (elements.isEmpty()) {
            notEmpty.await();
        }
        E result = elements.removeFirst();
        notFull.signal();
        return result;
    } finally {
        lock.unlock();
    }
}

public int size() {
    lock.lock();
    try {
        return elements.size();
    } finally {
        lock.unlock();
    }
}
}

```


Two conditions avoid waking producers when only consumers can proceed and vice versa. The queue, its size, and capacity invariant are always accessed under one lock.

TRACE IT | 34.7 Execution or memory walkthrough

Assume capacity one and an empty buffer:

- 1 Consumer C calls `take`, acquires `lock`, observes empty, and calls `notEmpty.await`.
- 2 `await` atomically places C in the condition wait protocol and releases `lock`. C is not consuming CPU.
- 3 Producer P acquires `lock`, observes space, appends A, and calls `notEmpty.signal`.
- 4 Signal makes one waiter eligible to transfer/reacquire; it does not hand the lock directly to C. P still owns the lock.
- 5 P exits `finally` and unlocks. Its state changes happen-before C's successful later acquisition through lock semantics.
- 6 C reacquires, returns from `await`, and rechecks `elements.isEmpty()` in the loop. It removes A and signals `notFull`.
- 7 If another consumer acquired first and removed A, C would find empty again and await. This is why `if` is incorrect even without a spurious wakeup.

If C is interrupted during `await`, the call throws after reacquiring/processing according to the condition contract. `finally` releases the lock, and `InterruptedException` propagates. The queue invariant remains intact.

The memory shape is simple: the `BoundedBuffer` owns one deque and lock. Waiting threads are managed through condition/lock implementation nodes outside the domain deque; they do not need application-level polling flags.

34.8 Complexity and performance

Deque insertion/removal is amortized $O(1)$; lock acquisition is $O(1)$ algorithmically but can wait without a finite bound under contention. A condition wait releases CPU resources but park/unpark and scheduling add latency. `size` is exact at the linearization point under the lock and can be stale immediately after return.

Lock performance depends on critical-section duration, arrival rate, core count, preemption, and fairness. The utilization principle matters: as time inside a serialized region approaches capacity, queueing delay rises sharply. Reduce shared state and critical-section work before replacing the lock primitive.

Measure both acquisition wait and hold duration; either can dominate tail latency.

Fair `ReentrantLock` can reduce barging/starvation in some workloads but often lowers throughput and does not guarantee fair thread scheduling. `tryLock()` can barge even on a fair lock under documented behavior. Measure the workload whose property matters.

HotSpot note: HotSpot has evolved monitor representations, fast paths, spinning, and inflation/deflation policies across JDK releases. Source-level performance should not rely on a particular object-header lock encoding. Uncontended `synchronized` can be highly optimized.

WATCH OUT | 34.9 Edge cases and common mistakes

- Locking only writes while ordinary reads race.
- Guarding one invariant with multiple unrelated lock objects.
- Synchronizing on an externally accessible string, boxed value, collection, or replaceable field.
- Calling `wait`, `notify`, or `notifyAll` without owning the same object's monitor.
- Using `if` instead of `while` around a condition wait.
- Assuming a signal transfers lock ownership or proves the predicate true.
- Forgetting `unlock` in `finally`.
- Calling slow network/database/user callbacks while holding a shared lock.
- Returning a mutable guarded collection reference to callers.
- Choosing a fair/read-write/stamped lock without measured need.
- Acquiring locks in inconsistent order; analyzed deeply in Chapter 38.

An intrinsic lock acquisition cannot be interrupted or timed. If bounded lock wait is a requirement, an explicit lock may be necessary. Conversely, replacing a simple `synchronized` block with `ReentrantLock` only for perceived speed adds failure modes without guaranteed gain.

34.10 Production engineering notes

Document each invariant and its guard:

JAVA

```
// Guarded by lock: elements.size() <= capacity and all deque access.
```

Keep critical sections small in elapsed time, not merely line count. Copy needed state under lock, release, then perform logging, serialization, callbacks, or remote I/O. If the operation must be atomic with an external system, a Java lock alone cannot provide distributed atomicity; use transactions, idempotency, or a workflow protocol.

Incident example: request p99 jumps from 50 ms to 8 seconds while CPU is low. Thread dumps show 180 threads `BLOCKED` entering a `synchronized` cache method; the owner is performing a remote refresh inside the monitor. One slow upstream serialized the service. Redesign with an immutable snapshot, single-flight refresh outside the publication lock, a timeout, and stale-value policy.

Thread dumps show monitor owner and contenders for intrinsic locks and often explicit-lock parking stacks. JFR lock events and profiles reveal duration and hot call sites. One dump is a snapshot; collect a short series and correlate with request/CPU timelines.

For shutdown, waiting methods should propagate interruption. A condition-based component can also set a closed predicate under lock and `signalAll` so every waiter can distinguish closure from temporary empty/full state.

TRY IT | 34.11 Interview questions and model answers

What does `synchronized` guarantee?

Mutual exclusion for code using the same monitor and memory ordering: an unlock happens-before a subsequent lock of that monitor. It is reentrant and releases on normal or abrupt block exit. It does not make unrelated unsynchronized accesses safe.

Why call `wait` in a loop?

Wakeups can be spurious, a notification can target a thread whose predicate is false, or another thread can consume the state before this waiter reacquires the lock. The condition predicate must be rechecked while holding the lock.

When would you use `ReentrantLock` instead of `synchronized`?

When requirements include timed or interruptible acquisition, multiple conditions, or a deliberately evaluated fairness/inspection feature. For simple scoped exclusion, `synchronized` is safer and concise.

Does `notify` wake the most appropriate waiter?

No such selection is guaranteed. With multiple predicates on one monitor, `notify` can wake an ineligible waiter. `notifyAll` plus predicate loops is safer, or separate explicit `Condition` queues.

Read-write lock versus normal lock?

A read-write lock can help with long, frequent reads and rare writes under real contention. It adds coordination, can complicate upgrades, and may reduce performance for short sections. Benchmark the actual read/write mix.

TRY IT | 34.12 Exercises

- 1 Add `close()` to `BoundedBuffer` so all waiting producers/consumers terminate with defined behavior.
- 2 Implement the same buffer with `synchronized`, `wait`, and `notifyAll`.
- 3 Demonstrate why replacing each `while` with `if` is incorrect using a two-consumer trace.
- 4 Refactor code that performs a callback under lock into snapshot-then-callback form.
- 5 Compare `lock`, `lockInterruptibly`, and timed `tryLock` cancellation behavior.
- 6 Write the invariant and lock-order documentation for a two-account transfer.

RECAP | 34.13 Chapter summary

Locks protect named invariants by combining mutual exclusion with happens-before ordering. Intrinsic monitors are reentrant and support one wait set; explicit locks add timed/interruptible acquisition and multiple conditions. Condition waits always require a predicate loop, and every successful explicit acquisition requires **finally**-based release. Performance comes mainly from reducing contention and critical-section duration, not selecting a fashionable lock.

TRY IT | 34.14 Revision checklist

- ☐ I can identify the exact invariant and lock object.
- ☐ I can explain monitor unlock-to-lock visibility.
- ☐ I know why wait/await uses a loop and reacquires the lock.
- ☐ I can use `ReentrantLock` and `Condition` exception-safely.
- ☐ I can choose among intrinsic, explicit, read-write, and optimistic locking.
- ☐ I avoid external calls and mutable-state escape under a shared lock.
- ☐ I can investigate contention using dumps, JFR, and a timeline.

SDE-2 CORE CHAPTER

Chapter 35 - Volatile, Atomics, CAS, and Happens-Before in Practice

35.1 Learning objectives

- Prove publication and visibility using happens-before rather than timing intuition.
- Identify when volatile is sufficient and when an invariant needs an atomic transition or lock.
- Explain compare-and-set, retry loops, linearization points, and progress properties.
- Recognize ABA and choose versioning, stamped references, or a different design.
- Use atomic counters, immutable state snapshots, and contention-aware accumulators correctly.

WHY IT WORKS | 35.2 Why this matters at SDE-2

Lock-free code can be wrong while looking sophisticated. Two atomic fields do not make a two-field invariant atomic. A volatile flag can publish prior writes but cannot prevent duplicate check-then-act. A CAS loop can burn a core under contention. A `LongAdder` can improve metrics throughput but is unsuitable for an exact bank balance or sequence number.

SDE-2 interviews expect both correctness and judgment. You should locate the linearization point, name the happens-before edge, state whether the operation is lock-free or wait-free, and explain why a simpler lock might be better. Production design additionally needs overflow, observability, retry, and shutdown behavior.

WHY IT WORKS | 35.3 First-principles model

Shared-state correctness has two layers:

- 1 **Ordering/visibility:** Which writes must a reader observe? Use happens-before edges.
- 2 **Atomic state transition:** Which read-decision-write sequence must appear indivisible? Use a lock, CAS, or a higher-level concurrent operation.

Volatile solves the first layer for a variable and surrounding actions:

TEXT

```
writer                                reader
snapshot = fullyBuilt;
volatile current = snapshot; --HB--> read volatile current
                                   use fully initialized snapshot
```

CAS can solve a single-location transition:

TEXT

```
loop:
  observed = state
  proposed = f(observed)
  if compareAndSet(observed, proposed): success
  else: another transition won; retry from new state
```

The successful CAS is the linearization point: the instant the operation appears to take effect.

35.4 Core terminology

- **Volatile variable:** Field whose accesses participate in the JMM volatile synchronization order and visibility rules.
- **Atomic variable:** Library wrapper such as `AtomicInteger` or `AtomicReference` supporting indivisible operations on one location.
- **CAS:** Compare-and-set; update only if the current value equals an expected value under the operation's comparison semantics.
- **Linearization point:** Single conceptual instant at which a concurrent operation takes effect.
- **Lock-free:** System-wide progress is guaranteed under continued scheduling, though one thread may starve.
- **Wait-free:** Each operation completes in a bounded number of its own steps.
- **Obstruction-free:** A thread completes if it eventually runs alone long enough.
- **ABA problem:** A location changes A to B and back to A, making equality alone hide intervening history.
- **Contention:** Concurrent attempts compete to update the same state.
- **False sharing:** Independent hot values trigger cache-coherence traffic because of physical proximity.

35.5 Detailed mechanics

A volatile write happens-before every subsequent read of that volatile in synchronization order. Program order and transitivity publish ordinary writes before the volatile write to code after the volatile read. Volatile access is individually atomic, including volatile `long` and `double`.

Volatile does not combine operations. `volatile int count; count++;` remains a read, calculation, and write. Two threads can read 5 and both write 6. Similarly, this is unsafe:

JAVA

```
if (!initialized) {
    initialized = true;
    initialize();
}
```

Even if `initialized` is volatile, two threads can both read false before either writes true, and setting it before initialization can expose the wrong protocol. Use class initialization, a lock, a future, or correctly designed CAS state.

Volatile is a good match for an independent cancellation flag, one-writer status, or reference to an immutable snapshot. Updating an immutable `Config` and publishing its single reference makes all fields move together. Publishing separate volatile host and port fields permits mixed-version snapshots.

Atomic classes offer operations such as `getAndIncrement`, `compareAndSet`, `getAndUpdate`, and `accumulateAndGet`. These are more than volatile wrappers because read-modify-write is indivisible for that atomic location. Atomic object methods have documented memory effects broadly aligned with their access modes. Newer APIs such as `VarHandle` expose a range from plain/opaque/acquire/release to volatile access; use weaker modes only with a written proof and measured need.

A CAS loop must recompute from the newly observed value after failure. The update function can execute multiple times, so it must be side-effect free. Do not charge a card, append to an external log, or increment a separate metric inside a function passed to `updateAndGet`; retries can duplicate side effects.

CAS is optimistic. Under low contention it avoids parking and often performs well. Under high contention many threads calculate losing updates, retry, and generate cache-line traffic. Lock-free means some operation makes progress; it does not mean every caller has bounded latency or that no scheduling support is involved.

ABA matters when equality is used as evidence that no relevant change occurred. In a lock-free stack, thread T1 reads head A and next C, pauses, while T2 pops A, changes the stack, and later pushes the same A object back. T1's CAS from A to C can succeed even though history changed and C may no longer represent the intended successor. Solutions include immutable/non-reused nodes under GC-specific reasoning, version counters, `AtomicStampedReference`, `AtomicMarkableReference`, hazard/epoch schemes in native memory, or a lock.

Java garbage collection prevents a freed Java object address from being manually reallocated behind a live reference in ordinary code, removing one classic native ABA route. It does not eliminate logical ABA when the same reference/value is deliberately restored.

`AtomicStampedReference` atomically compares reference plus integer stamp. Stamp overflow is theoretically possible, so the stamp width and operation lifetime matter. A monotonic version embedded in an immutable state object often makes domain history clearer.

`LongAdder` distributes updates across internal cells under contention, then sums them. It is excellent for high-rate statistics where a momentarily non-atomic aggregate is acceptable. `sum()` is not an atomic snapshot relative to concurrent updates, and `sumThenReset()` can be inappropriate when exact accounting is required. Use `AtomicLong` or locking for exact sequences/balances.

Specification boundary: The JMM and `java.util.concurrent.atomic` APIs define visibility and atomic-operation contracts. They do not require one CPU instruction, a particular cache protocol, wait-free progress for all atomic methods, or a fixed physical layout.

TRACE IT | 35.6 Worked Java example

JAVA

```
import java.util.concurrent.atomic.AtomicReference;

public final class Inventory {
    private record State(int available, long version) {
        State {
            if (available < 0) throw new IllegalArgumentException("negative");
        }
    }

    private final AtomicReference<State> state;

    public Inventory(int initial) {
        state = new AtomicReference<>(new State(initial, 0));
    }

    public boolean reserve(int units) {
        if (units <= 0) throw new IllegalArgumentException("units");
        while (true) {
            State observed = state.get();
            if (observed.available() < units) return false;
            State proposed = new State(
                observed.available() - units,
                observed.version() + 1);
            if (state.compareAndSet(observed, proposed)) return true;
            Thread.onSpinWait();
        }
    }

    public int available() {
        return state.get().available();
    }
}
```

Availability and version move in one immutable object referenced by one atomic location. No observer can see new availability with an old version. The successful CAS linearizes each reservation.

This in-memory reservation is not a substitute for a database transaction across processes. Its scope is one JVM object instance.

TRACE IT | 35.7 Execution or memory walkthrough

Start with `State(5, 0)`. Threads A and B both call `reserve(4)`:

- 1 A reads reference S0 -> (5, 0).
- 2 B reads the same reference S0.
- 3 A allocates proposed S1 -> (1, 1).
- 4 B allocates proposed S2 -> (1, 1); equal content does not make it the same object reference.
- 5 A performs CAS expected S0, update S1. It succeeds and atomically changes the location.
- 6 B performs CAS expected S0, update S2. Current reference is S1, so it fails.
- 7 B retries and reads S1. Available 1 is less than 4, so B returns false.

Exactly one reservation succeeds; availability never becomes negative. A lock-free system-progress claim applies because a failed CAS indicates another update won. B itself was not guaranteed to win.

The immutable objects are ordinary heap objects. Failed proposed states become unreachable and add allocation/GC pressure. A packed `AtomicLong` could encode fields without allocation, but packing introduces bit widths, overflow, readability, and migration constraints. Optimize only after evidence.

The atomic read that observes S1 also gives the documented visibility needed to read that fully constructed record. Final record fields further support immutable construction, but publication still uses the atomic reference.

35.8 Complexity and performance

In the absence of contention, `reserve` is $O(1)$ time and allocates one small state. Under contention, one call can retry an unbounded number of times, so it is not wait-free. Total useful updates still progress if threads continue running and the atomic implementation provides the expected progress behavior.

CAS loops work best for small, fast, independent transformations. If validation is expensive, contention high, or the invariant spans several mutable structures, a lock can reduce wasted work and produce better tail latency. Backoff may reduce contention but complicates fairness and tuning.

Atomics create hot memory locations. An `AtomicLong` incremented by every request can become a coherence bottleneck. `LongAdder` trades exact instantaneous state and memory for scalable distributed updates. Sharding domain state can also improve locality, but requires aggregation semantics.

`Thread.onSpinWait()` is only a performance hint and does not add ordering or guarantee progress. A long spin should yield to a blocking design or explicit backoff to avoid wasting CPU.

HotSpot note: HotSpot intrinsifies many atomic operations into CPU-specific instructions and barriers when possible. CAS instruction costs, spurious failure behavior of weak variants, padding, and fence mappings differ across x86-64, AArch64, and JVM versions.

WATCH OUT | 35.9 Edge cases and common mistakes

- Believing volatile makes `++`, check-then-act, or multi-field invariants atomic.
- Updating two atomics separately and calling the pair transactionally consistent.
- Performing side effects inside a retryable atomic update function.
- Ignoring counter overflow or stamp wraparound.
- Treating `LongAdder.sum()` as an exact linearizable balance.
- Assuming lock-free means starvation-free, bounded latency, or always faster.
- Retrying CAS without rereading and recomputing proposed state.
- Using reference CAS when logical equality, not identity, was intended, or vice versa.
- Dismissing ABA merely because Java has GC.
- Publishing a mutable object through volatile and then mutating it without synchronization.
- Using several volatile fields when one immutable snapshot is the actual invariant.

A volatile collection reference does not make collection operations thread-safe. Volatile orders replacement of the reference; concurrent mutation still requires a concurrent collection or other synchronization.

35.10 Production engineering notes

Write down each operation's linearization point and scope. `Inventory.reserve` is atomic only inside one process and instance; a replicated service requires a database conditional update, consensus-backed store, partition ownership, or idempotent distributed workflow.

Monitor CAS failure/retry rates, CPU, allocation, and p99 latency. A regression after traffic growth may be coherence contention rather than application computation. Profiles can show atomic intrinsics and spin hot spots; JFR/OS tools can correlate CPU saturation.

Incident example: a metrics endpoint occasionally reports fewer completed requests than the previous scrape. The code uses `LongAdder.sumThenReset()` while updates continue, assuming an exact interval transfer. Some updates race with cell summation/reset. Fix by accepting approximate monotonic cumulative counters, using a metrics library's supported model, or serializing exact interval accounting if the business truly requires it.

For configuration, construct an immutable snapshot, validate it completely, then publish one volatile/atomic reference. Keep the previous snapshot on validation failure. Readers perform one read and use that local snapshot for the whole operation, avoiding mixed versions if a refresh occurs midway.

TRY IT | 35.11 Interview questions and model answers

What does volatile provide?

Volatile access is atomic and ordered in the synchronization order. A volatile write happens-before subsequent reads of that variable, publishing prior actions to code after the read. It does not make compound operations or a multi-variable invariant atomic.

How does CAS work?

It atomically compares the current value with an expected value and installs an update only on match. A CAS loop reads state, computes a side-effect-free proposal, attempts CAS, and recomputes after failure. The successful CAS is the linearization point.

What does lock-free mean?

Under continued execution, the system as a whole makes progress; individual threads may repeatedly lose and starve. It does not mean no coordination, no blocking anywhere in the runtime, or better latency under all contention.

Explain ABA.

A location changes from A to B and back to A, so a comparison with A succeeds despite relevant intermediate history. Use a version/stamp, a design that prevents reuse/restoration, a safe reclamation scheme, or a lock when history matters.

AtomicLong versus LongAdder?

AtomicLong gives linearizable exact single-location updates and reads. **LongAdder** spreads writes for throughput but its aggregate is not one atomic snapshot. Use it for statistics, not exact balances or IDs.

TRY IT | 35.12 Exercises

- 1 Add **restock** to **Inventory**, including overflow handling and a linearization-point explanation.
- 2 Implement the inventory with **ReentrantLock** and compare correctness proof and contention behavior.
- 3 Show a two-atomic invariant that produces a mixed snapshot; replace it with one immutable atomic state.
- 4 Construct a logical ABA trace for a lock-free stack and add a stamp.
- 5 Benchmark **AtomicLong** and **LongAdder** across thread counts, then state what correctness each benchmark ignores.
- 6 Review an **updateAndGet** lambda containing a log or external call and redesign it.

RECAP | 35.13 Chapter summary

Volatile supplies ordered visibility, not compound atomicity. Atomic classes make one-location transitions linearizable, and CAS loops turn failed competition into retry. Correct lock-free design requires side-effect-free proposals, explicit progress expectations, ABA analysis, overflow handling, and contention measurement. Immutable snapshots published through one volatile or atomic reference often provide the clearest practical design.

TRY IT | 35.14 Revision checklist

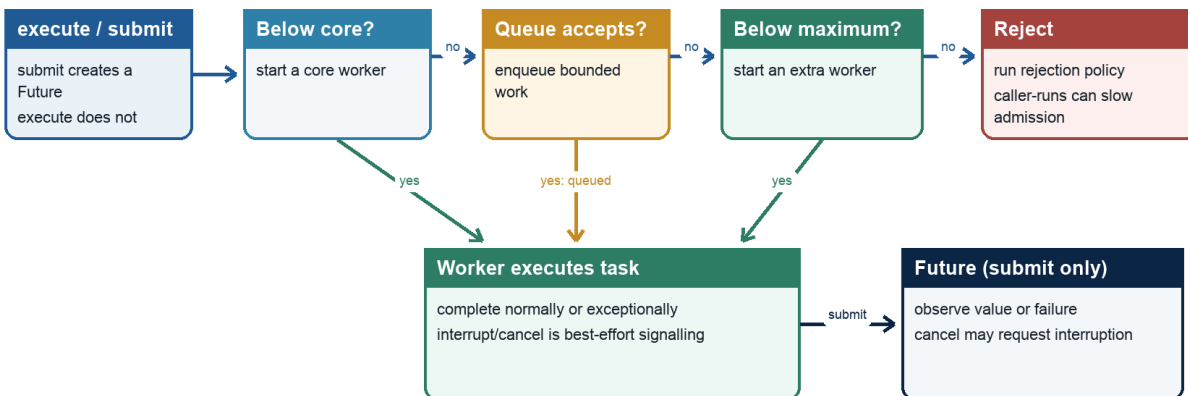
- ☐ I can prove volatile publication using happens-before.
- ☐ I know which invariants volatile cannot protect.
- ☐ I can identify a CAS loop's linearization point.
- ☐ I distinguish lock-free, wait-free, and starvation-free behavior.
- ☐ I can explain logical ABA and versioned solutions.
- ☐ I choose `AtomicLong` versus `LongAdder` by correctness needs.
- ☐ I avoid side effects in retryable update functions.

SDE-2 CORE CHAPTER

Chapter 36 - Executors, Futures, CompletableFuture, and Work Scheduling

ThreadPoolExecutor admission

Workers, queue capacity, rejection, and Future semantics



A bounded queue enables an admission decision; the executor is not automatic backpressure.

Sizing and rejection policy must match workload, latency targets, and downstream capacity.

Java Foundations to Advanced Engineering

Figure 36.1 - Executor submission, queueing, work, and completion

36.1 Learning objectives

- Separate task submission, execution policy, completion, and shutdown.
- Configure thread pools with intentional sizing, bounded queues, and saturation behavior.
- Use `Future` timeouts and cancellation without confusing them with task termination.
- Compose `CompletableFuture` stages and handle failures precisely.
- Avoid common-pool blocking, unbounded admission, hidden exceptions, and orphaned work.

WHY IT WORKS | 36.2 Why this matters at SDE-2

Executors turn a concurrency problem into a capacity system. An unbounded queue can preserve acceptance while latency and memory grow until failure. An oversized blocking pool can overwhelm a database. A `Future.get` timeout can return while the task continues consuming resources. A `CompletableFuture` callback can run on a request thread, pool worker, or common pool depending on completion timing and method choice.

An SDE-2 must design ownership and overload behavior, not just call `submit`. Interviews expect differences among `execute`, `submit`, `thenApply`, `thenCompose`, `handle`, and `exceptionally`. Production expects queue depth, rejection, task age, and shutdown behavior to be observable.

WHY IT WORKS | 36.3 First-principles model

An executor separates **what** should run from **where and when** it runs:

TEXT

```
producer -> admission -> work queue -> workers -> completion
           |               |
           reject/backpressure result/failure
```

Capacity is finite even when a data structure is unbounded. If arrival rate exceeds service rate for long enough, queued work, latency, and memory grow. A correct policy bounds one or more of: accepted concurrency, queue length, wait time, work per request, or upstream arrival.

A completion object represents a state machine:

TEXT

```
incomplete -> completed normally(value)
            -> completed exceptionally(cause)
            -> cancelled(cancellation outcome)
```

Composition transforms state without blocking a worker merely to wait for another stage.

36.4 Core terminology

- **Executor:** Interface accepting tasks for execution.
- **ExecutorService:** Executor with lifecycle, submission, futures, and shutdown operations.
- **ThreadPoolExecutor:** Configurable platform-thread pool with core/max sizes, queue, factory, and rejection policy.
- **Future:** Handle for completion, result retrieval, cancellation request, and status.
- **Saturation policy:** Behavior when an executor cannot accept more work.
- **Backpressure:** Propagating limited capacity to producers rather than buffering without bound.
- **Work stealing:** Workers take tasks from other workers' queues to balance computation.
- **Completion stage:** Dependent asynchronous computation triggered by another completion.
- **Fan-out/fan-in:** Start multiple tasks and combine their completions.
- **Deadline:** Absolute remaining-time budget propagated across operations.
- **ThreadLocal:** Per-thread variable whose lifetime is the thread's, not the task's.

36.5 Detailed mechanics

`execute(Runnable)` submits work with no result handle. If the task throws, uncaught-exception behavior may expose it through the worker/thread mechanism. `submit` returns a `Future`; thrown exceptions are captured and later wrapped by `Future.get` in `ExecutionException`. Ignoring the returned future can hide failure. This distinction often explains why changing `execute` to `submit` made logs disappear.

`ThreadPoolExecutor` admission is frequently misunderstood. In the typical policy:

- 1 If fewer than core threads exist, create one for the task.
- 2 Otherwise offer to the work queue.
- 3 If queue offer fails and thread count is below maximum, add a worker.
- 4 Otherwise reject.

With an unbounded queue, step 2 nearly always succeeds, so `maximumPoolSize` may never affect load handling. A bounded `ArrayBlockingQueue` makes overload explicit. Rejection choices include aborting with `RejectedExecutionException`, running in the caller, discarding, or discarding oldest. Silent discard is dangerous unless loss is deliberate and measured. `CallerRunsPolicy` can slow producers but may also run expensive work on an event loop or latency-sensitive request thread, so it is not universally safe.

Pool sizing begins with workload, not a magic number. For CPU-bound independent work, a starting point near available processors avoids excessive runnable threads. For blocking work, the rough formula `threads = cores / (1 - blockingFraction)` can guide an experiment, but external capacity and memory usually impose stricter limits. A database with 40 connections cannot benefit from 400 concurrent queries. Measure service time, blocking fraction, utilization, queue delay, downstream limits, and tail latency.

A `Future` result becomes available through `get`. Actions in the asynchronous computation happen-before actions after another thread successfully retrieves the result through `Future.get`, per the API memory-consistency contract. `get(timeout)` bounds how long the caller waits; `TimeoutException` does not cancel the task. `cancel(true)` requests interruption if running, but success means the `Future` transitioned to cancelled, not proof that arbitrary target code stopped. `cancel(false)` prevents execution when possible or records cancellation without interruption depending on state.

Correct executor shutdown is two-phase. `shutdown()` rejects new tasks and allows accepted work to finish. Await for a bounded period. If time expires, `shutdownNow()` requests interruption and returns tasks that never commenced. Await again and report remaining nontermination. If the shutdown thread is interrupted, request immediate shutdown, restore status, and return/propagate according to the boundary.

Scheduled executors distinguish fixed rate from fixed delay. Fixed-rate scheduling targets a cadence relative to the initial schedule and can run successive executions close together when behind, but does not concurrently overlap executions of the same periodic task through that scheduling contract. Fixed delay waits a delay after one run completes. If a periodic task throws an uncaught exception, subsequent executions are suppressed. Wrap/report failures deliberately.

`CompletableFuture` combines a writable completion with `CompletionStage` transformations:

- `thenApply`: map one result synchronously when the stage completes.
- `thenCompose`: map to another stage and flatten it, avoiding nested futures.
- `thenCombine`: combine two independent successful results.
- `allOf`: complete when all supplied stages complete; it does not collect typed results.
- `exceptionally`: recover from failure with a value.
- `handle`: transform both success and failure into a result.
- `whenComplete`: observe success/failure while generally preserving the original outcome.

Non-`Async` dependent actions can run in the thread that completes the previous stage or the thread attaching the action if already complete. `Async` variants without an executor use the default asynchronous facility, commonly the common `ForkJoinPool`. Supply an explicit executor for isolation, capacity, naming, and blocking behavior.

`thenApply` is wrong when the function returns a future: it creates `CompletableFuture<CompletableFuture<T>>`. Use `thenCompose`. A dependent success stage is skipped if its prerequisite fails. `join()` throws unchecked `CompletionException`; `get()` uses checked `ExecutionException/InterruptedException`. Unwrap deliberately without discarding context.

`CompletableFuture.cancel` completes the future exceptionally with `CancellationException`; the `mayInterruptIfRunning` parameter has no interrupt effect for its asynchronous processing according to this class's contract. Cancellation must be designed into the underlying task/resource, not assumed from the graph handle.

Per-thread state on a pooled executor

`ThreadLocal` gives each thread its own copy of a variable. On a pool this creates a lifetime mismatch that is the source of two distinct production failures, and it is worth stating precisely because the fix for one is not the fix for the other.

The value's lifetime is the *thread's*, but the meaning you assigned it is the *task's*. A pooled worker outlives every task it runs, so whatever a task leaves behind is still there when the next task arrives.

JAVA

```
private static final ThreadLocal<String> TENANT = new ThreadLocal<>();

// Wrong: the value survives this task and leaks into the next one.
executor.submit(() -> {
    TENANT.set(request.tenantId());
    handle(request);
});

// Correct: the task owns the binding for exactly its own duration.
executor.submit(() -> {
    TENANT.set(request.tenantId());
    try {
        handle(request);
    } finally {
        TENANT.remove();
    }
});
```

Failure one is correctness, and it is the dangerous one. Without the `remove`, task N+1 observes task N's tenant, user, or trace id. It does not throw. It silently attributes work to the wrong principal, and in a multi-tenant service that is a data-disclosure bug that no test with a single request will find.

Failure two is memory. `Thread` holds a `ThreadLocalMap`; entries have weak keys but **strong values**. When the `ThreadLocal` object itself becomes unreachable the key clears, yet the value stays reachable from the live pooled thread until an incidental map operation happens to purge that slot. A discarded large buffer can be retained for the life of the pool. Chapter 41 walks the root path.

`InheritableThreadLocal` copies the parent's map into each new thread at construction. On a pool that is nearly useless, because workers were constructed long before your request arrived and inherit from whichever thread created the pool. With virtual threads it is worse than useless: a per-thread map copy does not scale to millions of threads.

This is the design pressure that produced `ScopedValue` (Chapter 58). A `ScopedValue` is immutable, its lifetime is exactly the dynamic extent of the `run` call so there is nothing to remove, and a structured scope shares the binding with child threads rather than copying it. Where you control the code, prefer it on Java 25. Where you do not - an MDC in a logging framework, a security context in a framework filter - `ThreadLocal` remains, and the `try/finally` discipline above is not optional.

Specification boundary: `java.util.concurrent` specifies lifecycle, completion, cancellation requests, and memory-consistency effects. It does not guarantee task start order unless an executor says so, fair worker scheduling, task termination after cancel, or one thread per task. `ThreadLocal` specifies per-thread storage and weak key references; it does not specify when a stale entry's value is reclaimed, so leak timing is an implementation behavior rather than a contract.

TRACE IT | 36.6 Worked Java example

JAVA (continued 1/2)

```
import java.time.Duration;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public final class PricePipeline implements AutoCloseable {
    private final ThreadPoolExecutor io = new ThreadPoolExecutor(
        8, 8, 0, TimeUnit.MILLISECONDS,
        new ArrayBlockingQueue<>(100),
        new ThreadPoolExecutor.AbortPolicy());

    CompletableFuture<Integer> basePrice(String sku) {
        return CompletableFuture.supplyAsync(() -> loadPrice(sku), io);
    }

    CompletableFuture<Integer> discount(int base) {
        return CompletableFuture.supplyAsync(() -> loadDiscount(base), io);
    }

    CompletableFuture<Integer> finalPrice(String sku) {
        return basePrice(sku)
            .thenCompose(this::discount)
            .thenApply(value -> Math.max(0, value))
            .orTimeout(500, TimeUnit.MILLISECONDS)
            .exceptionally(failure -> 0);
    }
}
```

JAVA (continued 2/2)

```

    }

    private int loadPrice(String sku) { return 100; }
    private int loadDiscount(int base) { return base - 10; }

    @Override
    public void close() {
        shutdown(io, Duration.ofSeconds(5));
    }

    static void shutdown(ThreadPoolExecutor executor, Duration timeout) {
        executor.shutdown();
        try {
            if (!executor.awaitTermination(timeout.toMillis(), TimeUnit.MILLISECONDS)) {
                executor.shutdownNow();
                if (!executor.awaitTermination(timeout.toMillis(), TimeUnit.MILLISECONDS)) {
                    System.err.println("executor did not terminate");
                }
            }
        } catch (InterruptedException interrupted) {
            executor.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}

```

The bounded queue makes overload observable through rejection. `thenCompose` starts the dependent asynchronous discount only after obtaining the base. The fallback of zero is intentionally visible but would be a dangerous business default unless the domain defines it; production code should classify timeout, rejection, and data failures separately.

TRACE IT | 36.7 Execution or memory walkthrough

Successful path:

- 1 Caller invokes `finalPrice("A").basePrice` submits a task and returns an incomplete future F1.
- 2 The task enters a worker or bounded queue. If both worker capacity and queue are unavailable, submission throws rejection before a usable pipeline is returned unless handled at that boundary.
- 3 A worker loads 100 and completes F1. Its completion triggers the `thenCompose` function, which calls `discount(100)` and returns F2.
- 4 `thenCompose` links rather than nests F2. Another IO task computes 90 and completes F2.
- 5 `thenApply` maps 90 to 90. The timeout stage races normal completion against its timer semantics; normal completion wins here.
- 6 `exceptionally` sees no failure and passes the value. The returned future completes with 90.

Failure path: if `loadDiscount` throws, F2 completes exceptionally. `thenApply` is skipped. `orTimeout` does not replace an already completed failure. `exceptionally` receives a completion-wrapped cause and returns 0, converting the final stage to normal success. Metrics and logs must occur before or inside recovery if the failure must remain observable.

Timeout path: the pipeline future completes exceptionally after the configured duration, but underlying I/O may continue. `orTimeout` is not resource cancellation. The I/O client also needs connect/read/request deadlines, and cancellation hooks where supported.

36.8 Complexity and performance

Submission is generally $O(1)$, while queueing delay is unbounded in time unless capacity and deadlines constrain it. A queue of capacity `q` uses $O(q)$ references plus retained task graphs. Each task can retain request payloads, traces, and closures, so a "small" queue can hold significant heap.

Little's Law relates average concurrency `L`, arrival rate `lambda`, and average time `W`: $L = \text{lambda} * W$ in a stable system. If 500 requests/second each occupy an IO operation for 200 ms, average in-flight demand is about 100 before bursts. Downstream limits and desired utilization determine admission below saturation.

Blocking on one future from a task in the same small executor can create thread starvation deadlock: every worker waits for subtasks queued behind them. Compose asynchronously or use separate capacity domains. Work-stealing pools excel at many nonblocking fork/join tasks; unmanaged blocking can reduce parallelism and starve unrelated common-pool users.

WATCH OUT | 36.9 Edge cases and common mistakes

- Using an unbounded queue and assuming `maximumPoolSize` controls overload.
- Creating an executor per request or forgetting to close an owned executor.
- Ignoring a `Future` returned by `submit`, hiding task exceptions.
- Treating `get(timeout)` as cancellation.
- Assuming `cancel(true)` proves target termination.
- Blocking common-pool workers on slow I/O.
- Using non-`Async` completion actions while assuming a dedicated thread.
- Using `thenApply` for a future-returning function and creating nested futures.
- Recovering every failure to a default and losing incident visibility.
- Scheduling a periodic task whose first exception silently suppresses future runs.
- Letting a queue hold work past caller deadlines.
- Using `CallerRunsPolicy` on an event loop or lock-holding producer without analysis.

36.10 Production engineering notes

Treat each executor as a named resource pool with an owner and SLO. Export active workers, pool size, queue size/capacity, oldest task age if feasible, completed count, rejection count, execution duration, queue delay, cancellation, and shutdown duration. Thread names should identify the capacity domain.

Separate pools only when they represent different blocking behavior, priorities, or failure domains; too many pools make total concurrency uncontrollable. The strongest limit is often a semaphore or client pool aligned with a downstream resource, regardless of executor size.

Incident example: heap rises while CPU and database usage remain moderate. The service uses `newFixedThreadPool(32)`, whose factory commonly uses an unbounded queue. A dependency slows, incoming requests continue submitting closures that retain 200 KB payloads, and queue depth reaches 50,000. Fix with a bounded queue, rejection/backpressure, end-to-end deadlines, limited payload retention, and upstream shedding. Increasing heap only delays failure.

Propagate deadlines, not independent full timeouts at every layer. If an incoming request has 800 ms remaining, a downstream stage should not start a new 2-second timeout. When a stage times out, cancel or close the underlying client operation if its API supports it, and ensure late completion cannot commit unwanted side effects.

HotSpot note: The common asynchronous facility normally uses `ForkJoinPool.commonPool()` in mainstream JDKs, with behavior influenced by system properties and environment. Worker implementation, work-stealing queues, compensation for managed blocking, and timer mechanics are library/runtime details, not language guarantees.

TRY IT | 36.11 Interview questions and model answers

How does `ThreadPoolExecutor` use core size, maximum size, and queue?

It normally creates workers up to core, then queues; only when queueing fails does it grow toward maximum, then reject. Therefore an unbounded queue often makes maximum size irrelevant. The queue and rejection policy are central overload decisions.

How do you size a pool?

Start near processor count for CPU-bound work. For blocking work, estimate blocking fraction and concurrency demand, but cap by downstream capacity, memory, and latency. Validate queue delay, utilization, and tail latency under representative load rather than trust a formula.

`thenApply` versus `thenCompose`?

`thenApply` maps `T` to `U`. If the function maps `T` to `CompletionStage<U>`, use `thenCompose` to flatten the asynchronous dependency rather than produce a nested stage.

How do `CompletableFuture` exceptions work?

A failed prerequisite skips normal dependent stages. `exceptionally` recovers failures, `handle` maps both outcomes, and `whenComplete` observes while preserving outcome in the ordinary case. `join` reports failure through `CompletionException`; `get` uses `ExecutionException` and is interruptible.

What is correct shutdown?

Stop admission with `shutdown`, await a bounded grace period, request interruption with `shutdownNow` if needed, await again, report survivors, and restore caller interrupt status if shutdown itself is interrupted.

TRY IT | 36.12 Exercises

- 1 Add explicit handling for rejection so `finalPrice` returns a failed future rather than throwing synchronously.
- 2 Replace the default fallback with typed classification for timeout, rejection, and domain failure.
- 3 Size a pool and queue for a stated arrival rate, service time, downstream connection limit, and burst budget.
- 4 Create and then fix a thread-starvation deadlock caused by waiting on same-pool subtasks.
- 5 Compare fixed-rate and fixed-delay execution when one run exceeds the period.
- 6 Implement fan-out to two independent futures and combine with `thenCombine` under one deadline.

RECAP | 36.13 Chapter summary

Executors encapsulate scheduling and capacity; futures represent completion, not guaranteed task termination. Bounded queues and explicit rejection make overload controllable. Pool sizing follows computation, blocking, downstream limits, and measured queue latency. `CompletableFuture` composition avoids blocking when `thenCompose` and combination stages express dependencies, but execution context and exception recovery must be explicit. Every executor needs ownership and bounded shutdown.

TRY IT | 36.14 Revision checklist

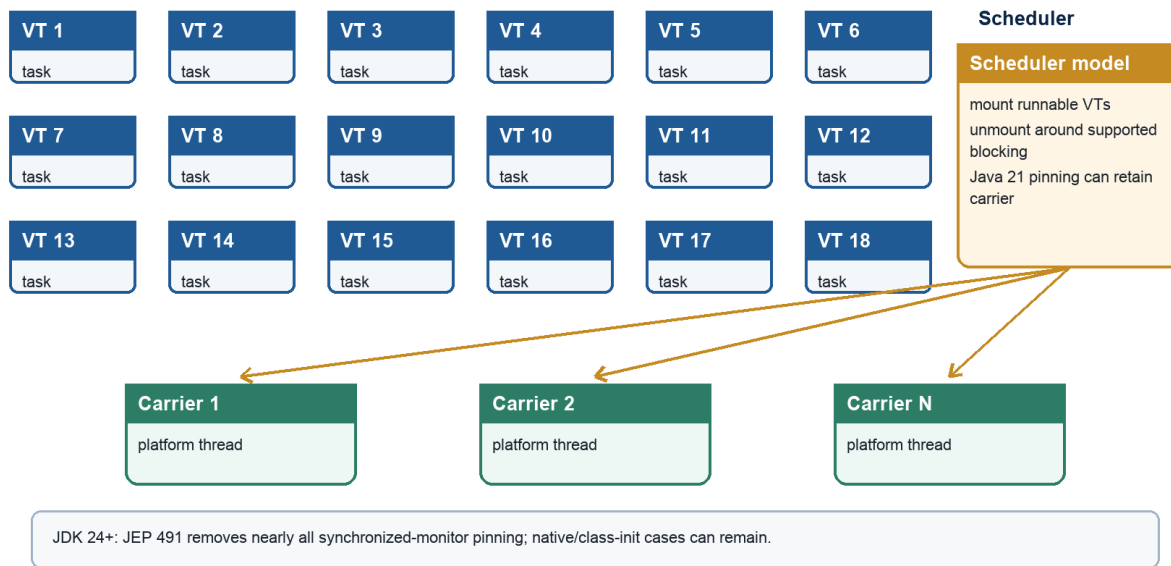
- ☐ I understand executor admission order and unbounded-queue risk.
- ☐ I can size pools from workload and downstream constraints.
- ☐ I know `get(timeout)` and cancellation limitations.
- ☐ I can implement two-phase executor shutdown.
- ☐ I can choose `thenApply`, `thenCompose`, `thenCombine`, and error stages.
- ☐ I specify callback execution context rather than assume it.
- ☐ I can diagnose queue growth, hidden exceptions, and same-pool starvation.

SDE-2 CORE CHAPTER

Chapter 37 - Concurrent Collections and Virtual Threads

Virtual thread scheduling

Java 21/OpenJDK model: many virtual threads mount on fewer carrier threads



Java Foundations to Advanced Engineering

Figure 37.1 - Virtual threads mounted on carrier threads

37.1 Learning objectives

- Choose a concurrent collection by access pattern, ordering, consistency, and capacity needs.
- Use atomic map operations instead of unsafe compound check-then-act sequences.
- Explain weakly consistent and snapshot iteration without expecting fail-fast behavior.
- Design Java 21 virtual-thread-per-task services without pooling virtual threads.
- Recognize pinning and bound scarce downstream resources independently of thread count.

WHY IT WORKS | 37.2 Why this matters at SDE-2

Replacing `HashMap` with `ConcurrentHashMap` prevents structural corruption but does not automatically preserve a business invariant across several calls. An unbounded "concurrent" queue can still exhaust memory. Snapshot iteration can be perfect for configuration listeners and disastrous for a write-heavy list.

Virtual threads change the cost of blocking concurrency, not the capacity of databases, remote APIs, CPU cores, or memory. A service can move from 200 pooled platform threads to 20,000 virtual requests and overload its database in seconds. SDE-2 design therefore separates task representation from resource admission.

WHY IT WORKS | 37.3 First-principles model

A concurrent collection defines synchronization at the operation boundary. The key question is which compound action is one operation:

TEXT

<pre>unsafe composition if (!map.containsKey(k)) { map.put(k, load(k)); }</pre>	<pre>atomic collection operation map.computeIfAbsent(k, loader)</pre>
---	---

Thread safety of each call does not make the left pair atomic. Another thread can change the map between calls.

Virtual threads address a different boundary. They let one Java thread represent one task while the runtime schedules many such threads over fewer carrier platform threads:

TEXT

```
virtual tasks:  V1 V2 V3 V4 V5 ... V10000
                \ | /   \ | /
carrier threads: P1  P2  P3  ...
                  |
                  CPU cores
```

When supported blocking operations park a virtual thread, its continuation can be unmounted so the carrier runs another. The application keeps straightforward blocking code without dedicating one platform stack/thread to every wait.

37.4 Core terminology

- **Linearizable operation:** Appears to take effect at one point between invocation and response.
- **Weakly consistent iterator:** Tolerates concurrent updates, does not throw `ConcurrentModificationException`, and may reflect some updates under its contract.
- **Snapshot iterator:** Iterates an immutable array/version captured when the iterator was created.
- **Blocking queue:** Queue whose operations can wait for capacity or elements.
- **Nonblocking collection:** Collection whose core operations use progress techniques without ordinary mutual-exclusion blocking.
- **Virtual thread:** Lightweight `Thread` implementation finalized in Java 21 for high-throughput blocking tasks.
- **Carrier:** Platform thread on which a virtual thread is mounted for execution.
- **Mount/unmount:** Runtime scheduling of virtual-thread execution state onto/off a carrier.
- **Pinning:** Condition in which a blocked virtual thread cannot unmount from its carrier in the relevant JDK.
- **Bulkhead:** Independent capacity limit preventing one workload/resource from consuming all concurrency.

37.5 Detailed mechanics

`ConcurrentHashMap` supports concurrent retrievals and high update concurrency. It rejects null keys and values, which avoids ambiguity between absent and mapped-to-null in concurrent reads. Individual operations such as `putIfAbsent`, conditional `remove`, `replace`, `compute`, `computeIfAbsent`, and `merge` provide atomic compound behavior under their documented contracts.

Use the operation that matches the invariant. A cache loader usually needs `computeIfAbsent`; a state machine may need `compute`; compare-and-remove can use `remove(key, expectedValue)`. Do not split an atomic transition into `get`, decide, `put`. Mapping/remapping functions should be short, nonblocking where possible, and must not recursively update the same map in ways forbidden by the API. Other map keys can change while one key is computed, so map-wide invariants still need another design.

`ConcurrentHashMap` iterators, spliterators, and enumerations are weakly consistent. They do not freeze the map, do not throw fail-fast exceptions, and may observe updates occurring during traversal according to the documented consistency. Aggregate methods such as `size`, `isEmpty`, and bulk reductions can be transient under concurrent mutation; use them for monitoring or estimates, not authorization decisions requiring a global snapshot.

HotSpot note: Mainstream JDK implementations use CAS, bins, selective locking, resizing cooperation, and tree-shaped bins under collision for `ConcurrentHashMap`. These structures and thresholds are implementation details, not the Map or concurrency specification.

`CopyOnWriteArrayList` stores an array snapshot. Reads and traversal avoid mutation interference; an iterator sees the array version from iterator creation and does not support mutating iterator operations. Every

structural write copies the backing array, making writes $O(n)$ and producing garbage. It fits small, read-mostly listener/config lists with rare changes, not event queues or frequently updated registries.

`ConcurrentLinkedQueue` is an unbounded nonblocking FIFO queue. `offer` and `poll` are efficient under concurrency, while `size()` typically traverses and can be expensive/inexact as a control signal under mutation. Because it is unbounded, it supplies no backpressure.

Blocking queues combine storage with wait/signal protocols:

- `ArrayBlockingQueue`: fixed capacity array, optional fairness, predictable bound.
- `LinkedBlockingQueue`: optionally bounded; default effectively large capacity is an overload risk.
- `SynchronousQueue`: no element capacity; each handoff pairs producer and consumer.
- `PriorityBlockingQueue`: priority ordering but logically unbounded capacity.
- `DelayQueue`: elements become available after delay expiration and is logically unbounded.

Methods express overload policy: `add` throws on full, `offer` reports immediately, timed `offer` waits to a deadline, and `put` waits interruptibly. Consumers similarly choose `poll`, timed `poll`, or `take`. Queue memory-consistency effects publish actions before insertion to a thread that subsequently removes/accesses the element through the specified handoff.

`ConcurrentSkipListMap` and `Set` provide sorted concurrent navigation with expected $O(\log n)$ search/update. They are useful when range queries and order are requirements; a hash-based structure is usually simpler/faster for unordered exact lookup. `ConcurrentSkipListMap` also disallows null.

Virtual threads are ordinary Java `Thread` instances in API semantics: they have names/IDs, interruption, thread locals, stack traces, and happens-before lifecycle rules. Java 21 creation includes:

JAVA

```
Thread.startVirtualThread(task);
Thread.ofVirtual().name("fetch-", 0).start(task);
Executors.newVirtualThreadPerTaskExecutor();
```

The executor creates a new virtual thread for each task; it is not a fixed pool of reusable virtual threads. Pooling cheap task threads reintroduces queueing and thread-local leakage without protecting a scarce resource. Bound the scarce resource directly with a connection pool, semaphore, bounded queue, rate limiter, or downstream concurrency control.

Virtual threads improve scalability for workloads with many blocking waits and a natural thread-per-request style. They do not increase CPU parallelism. CPU-bound work still competes for cores and may need an executor sized for CPU scheduling or a semaphore limiting expensive sections.

In Java 21, a virtual thread can be pinned to its carrier while executing inside a `synchronized` block/method or native/foreign call. If it blocks while pinned, the carrier cannot run another virtual thread. Brief uncontended synchronized sections are not inherently a problem; long/blocking operations while holding a monitor are.

`ReentrantLock` waits integrate with parking and can be an alternative after measurement, but mechanical replacement can introduce bugs.

JDK 24 delivered [JEP 491](#), which removes nearly all pinning caused by `synchronized` monitors. Selected native/foreign-call and class-loading or class-initialization situations can still retain a carrier. Code, diagnosis, and interview answers must therefore name the deployed release. Pinning affects scalability, not monitor correctness.

Thread locals work with virtual threads but can multiply memory by enormous thread counts. Large per-thread caches and pool-era assumptions should be removed. Scoped values and structured concurrency appear as preview APIs in Java 21 and must be compiled/run with preview enabled; they should not be presented as final Java 21 contracts.

Specification boundary: Java 21 specifies virtual-thread API behavior and concurrent collection contracts. Carrier scheduling, continuation representation, default scheduler parallelism, `ConcurrentHashMap` layout, and specific pinning diagnostics are implementation details.

TRACE IT | 37.6 Worked Java example

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.Semaphore;

public final class VirtualThreadQueries {
    private final Semaphore databaseSlots = new Semaphore(20);

    String query(int id) throws InterruptedException {
        databaseSlots.acquire();
        try {
            Thread.sleep(25); // Simulated interruptible blocking database call.
            return "row-" + id;
        } finally {
            databaseSlots.release();
        }
    }
}
```

JAVA (continued 2/2)

```

    }

    List<String> loadAll(int count) throws InterruptedException, ExecutionException {
        try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<String>> futures = new ArrayList<>();
            for (int i = 0; i < count; i++) {
                int id = i;
                futures.add(executor.submit(() -> query(id)));
            }

            List<String> results = new ArrayList<>(count);
            for (Future<String> future : futures) {
                results.add(future.get());
            }
            return results;
        }
    }
}

```

The executor can represent hundreds or thousands of waiting tasks without an equally large platform-thread pool. The semaphore preserves the real database concurrency budget at 20. In production, the connection pool may itself enforce a bound, but an earlier application bulkhead can bound waiters and deadline handling more intentionally.

TRACE IT | 37.7 Execution or memory walkthrough

For `loadAll(100)`:

- 1 The loop creates 100 task submissions; the executor starts one virtual thread per task rather than queuing behind a fixed virtual-thread pool.
- 2 The first 20 acquire semaphore permits. The remaining 80 park waiting for permits. Parking through the concurrency framework normally allows their carriers to run other virtual threads.
- 3 Each admitted thread calls the simulated blocking operation. `Thread.sleep` unmounts a non-pinned virtual thread in the Java 21 runtime, freeing its carrier.
- 4 When one sleep completes, the virtual thread is made runnable, remounts on an available carrier, returns its row, and releases a permit in `finally`.
- 5 A waiting virtual thread acquires that permit and proceeds. At no point does the simulated database have more than 20 operations.
- 6 `Future.get` consumes results in submission order. Later futures can already be complete, but one slow early future delays result assembly. A completion-order design could reduce head-of-line waiting if ordering is unnecessary.
- 7 If `get` is interrupted, try-with-resources closes the owned executor during unwinding under its API behavior, but production code still needs clear cancellation/deadline policy for outstanding tasks and external calls.

Each virtual thread has stack/task state, so 100 is not free. It is substantially lighter than reserving 100 large platform stacks, but captured request objects, thread locals, futures, and queue nodes still consume heap.

37.8 Complexity and performance

Submitting and collecting n tasks uses $O(n)$ futures/result references in the example. Each semaphore operation is $O(1)$ algorithmically but can wait. Overall elapsed time is approximately $\text{ceil}(n/20) * \text{serviceTime}$ under the simplified equal-duration model, plus scheduling overhead.

Collection choices:

Structure	Typical read	Typical update	Iteration model	Capacity
<code>ConcurrentHashMap</code>	expected $O(1)$	expected $O(1)$	weakly consistent	unbounded by API
<code>ConcurrentSkipListMap</code>	expected $O(\log n)$	expected $O(\log n)$	weakly consistent, ordered	unbounded
<code>CopyOnWriteArrayList</code>	$O(1)$ index	$O(n)$ copy	snapshot	unbounded
<code>ConcurrentLinkedQueue</code>	$O(1)$ offer/poll	$O(1)$ expected	weakly consistent	unbounded
<code>ArrayBlockingQueue</code>	$O(1)$	$O(1)$	weakly consistent	fixed

Virtual-thread throughput benefits appear when tasks spend substantial time blocking. For pure computation, more runnable virtual threads increase scheduling overhead without adding cores. Pinning or synchronized native libraries can reduce carrier availability; JFR and load tests should verify.

WATCH OUT | 37.9 Edge cases and common mistakes

- Composing individually thread-safe map calls into a racy compound operation.
- Expecting a concurrent iterator to produce an atomic whole-map snapshot.
- Calling `size()` repeatedly on `ConcurrentLinkedQueue` for admission.
- Using `CopyOnWriteArrayList` for frequent writes or large lists.
- Treating default `LinkedBlockingQueue` or priority/delay queues as safely bounded.
- Performing slow recursive work inside `ConcurrentHashMap.compute`.
- Pooling virtual threads instead of limiting the real resource.
- Assuming virtual threads make database connections or CPU unlimited.
- Holding a Java 21 monitor across slow blocking I/O and ignoring pinning.
- Carrying large thread-local caches into millions of virtual threads.
- Assuming virtual threads remove the need for interruption, deadlines, or shutdown.
- Using preview structured-concurrency/scoped-value APIs without labeling and enabling preview.

37.10 Production engineering notes

Choose a collection with an explicit consistency statement. If a dashboard may tolerate a weakly consistent traversal, document it. If billing requires a snapshot, build an immutable version, take a lock, use a database snapshot, or design an event log; do not infer snapshot semantics from "concurrent."

Incident example: a service migrates from a 100-thread JDBC executor to virtual-thread-per-request. Throughput rises briefly, then database latency and connection wait explode because 8,000 requests concurrently reach the 100-connection pool. Virtual threads made waiting cheap but did not make the database faster. Add admission aligned with connection/query capacity, propagate request deadlines, shed overload, and measure semaphore/connection wait separately from query time.

For Java 21 pinning, JFR can record virtual-thread pinning events, and HotSpot offers version-specific diagnostics such as pinned-thread tracing properties. Capture stack context and duration; prioritize blocking while pinned, not every short monitor. JDK 24's JEP 491 removes nearly all monitor pinning and changes the diagnostic picture, so confirm the exact JDK before applying a Java 21 runbook.

Monitor virtual-thread count, runnable/parked distribution, carrier utilization, task latency, downstream permit/connection waits, rejections, memory per task, and thread-local growth. A massive thread dump is hard to read; JFR and aggregate tooling are designed to group virtual-thread behavior.

TRY IT | 37.11 Interview questions and model answers

Is `ConcurrentHashMap` enough for `containsKey` then `put`?

No. Each call is safe but the pair is not atomic. Use `putIfAbsent`, `computeIfAbsent`, `compute`, or another operation matching the transition. A multi-key invariant may still require locking or a different state model.

What does weakly consistent iteration mean?

Iteration tolerates concurrent modification and does not fail fast. It traverses elements according to the collection's documented consistency and may reflect some concurrent updates, but it is not an atomic snapshot.

Why are virtual threads useful?

They make thread-per-task blocking designs scalable by allowing blocked virtual threads to unmount from carrier platform threads. They improve concurrency for waiting-heavy workloads, not CPU parallelism or downstream capacity.

Should virtual threads be pooled?

Usually no. They are intended to be cheap per-task threads. Pooling them creates an artificial queue. Bound scarce resources such as connections, CPU-heavy sections, or partner calls directly.

What is pinning in Java 21?

A virtual thread can remain attached to a carrier while inside a monitor or native/foreign call. If it blocks then, the carrier cannot run another virtual thread, reducing scalability. Correctness remains, and later JDK behavior must be checked separately.

TRY IT | 37.12 Exercises

- 1 Replace a `get/put` cache race with `computeIfAbsent`; specify loader failure behavior.
- 2 Compare weakly consistent and copy-on-write snapshot iteration in a listener registry.
- 3 Choose a blocking queue and overload method for a lossless worker, best-effort telemetry, and direct handoff.
- 4 Add a deadline-aware semaphore acquisition to the worked example.
- 5 Produce completion-order results instead of submission-order waits.
- 6 Create a Java 21 pinning demonstration safely, record it with JFR, then shorten the monitor scope.

RECAP | 37.13 Chapter summary

Concurrent collections make documented individual and compound operations safe; they do not grant transactions across arbitrary call sequences. Iteration consistency and capacity are first-class selection criteria. Java 21 virtual threads let blocking tasks use a thread-per-task style by multiplexing execution over carriers, but CPU and downstream resources remain finite. Do not pool virtual threads; bound the scarce resource, monitor task memory, and diagnose Java 21 pinning where blocking occurs under monitors or native calls.

TRY IT | 37.14 Revision checklist

- ☐ I can replace racy map compositions with an atomic operation.
- ☐ I distinguish weakly consistent and snapshot iteration.
- ☐ I know capacity/backpressure properties of major concurrent queues.
- ☐ I can explain virtual threads, carriers, mounting, and blocking.
- ☐ I do not equate concurrency with parallelism or downstream capacity.
- ☐ I understand Java 21 pinning and version dependence.
- ☐ I can design a virtual-thread service with explicit bulkheads and deadlines.

SDE-2 CORE CHAPTER

Chapter 38 - Concurrency Failure Modes, Testing, and Design Patterns

38.1 Learning objectives

- Classify races, deadlocks, livelocks, starvation, missed signals, leaks, and performance collapse.
- Diagnose blocking and progress failures from timelines, thread dumps, JFR, and capacity metrics.
- Test concurrent code with coordinated starts, invariants, histories, stress harnesses, and bounded completion.
- Apply immutability, confinement, message passing, lock ordering, open calls, and bulkheads.
- Separate safety, liveness, and performance claims in design and interviews.

WHY IT WORKS | 38.2 Why this matters at SDE-2

Concurrent failures are nondeterministic only from the observer's perspective. They follow a protocol and a schedule. Adding sleeps may hide the schedule; adding logging may alter it. A test that passed one million times increases confidence but does not prove correctness. A thread dump showing every worker waiting may represent healthy idleness, a deadlock, downstream saturation, or same-pool starvation.

SDE-2 engineers turn symptoms into a graph: who owns which resource, who waits for whom, what state transition was expected, which happens-before edge is missing, and what progress guarantee was intended. They also prefer designs whose proof is short.

WHY IT WORKS | 38.3 First-principles model

Evaluate concurrency across three independent properties:

- **Safety:** Nothing bad happens. Examples: no negative balance, no duplicate lease owner, linearizable queue operations.
- **Liveness:** Something good eventually happens. Examples: every accepted task completes or cancels; locks are eventually acquired under stated assumptions.
- **Performance:** It happens within resource and latency goals.

A design can be safe but deadlocked, live but too slow, or fast in a benchmark but racy.

Wait-for graphs make deadlock concrete:

TEXT

Thread T1 owns Lock A and waits for Lock B
 Thread T2 owns Lock B and waits for Lock A

T1 -> B -> T2 -> A -> T1 cycle means no participant can proceed

Removing one necessary condition prevents classic lock deadlock: mutual exclusion, hold-and-wait, no preemption, or circular wait. Lock ordering targets circular wait.

38.4 Core terminology

- **Race condition:** Result depends incorrectly on relative timing/interleaving.
- **Data race:** Conflicting accesses not ordered by happens-before.
- **Lost update:** Two read-modify-write operations overwrite one effect.
- **Deadlock:** Participants wait in a cycle or condition that cannot be satisfied.
- **Livelock:** Participants keep changing/retrying but make no useful progress.
- **Starvation:** One participant is indefinitely denied progress while others proceed.
- **Priority inversion:** Higher-priority work waits behind lower-priority work holding a needed resource.
- **Thread starvation deadlock:** All workers wait for tasks queued to the same exhausted executor.
- **Linearizability:** Concurrent history behaves as if each operation took effect atomically between call and response.
- **Stress test:** Repeated/concurrent exploration intended to expose allowed schedules.

38.5 Detailed mechanics

Data races create visibility and ordering uncertainty. Race conditions are broader: two correctly atomic operations can still violate a check-then-act protocol. Lost updates, duplicate initialization, stale snapshots, and unsafe publication belong here. The fix is to make the intended transition atomic and ordered, not merely mark every field volatile.

Missed-signal bugs occur when condition state and notification are not protected by one protocol. If a producer signals before a consumer enters an unrelated wait, the consumer can sleep forever despite available work. A correct condition variable changes/checks the predicate under one lock and always waits in a loop. Semaphores, latches, futures, and blocking queues retain state and are often safer than hand-written notifications.

Classic lock deadlock often meets four conditions:

- 1 Resources have exclusive ownership.
- 2 A participant holds one while waiting for another.
- 3 Ownership cannot be forcibly taken safely.

4 The wait-for graph contains a cycle.

Prevent it with a global lock order, one coarser lock, atomic database operation, avoiding nested ownership, or timed/interruptible acquisition plus rollback. Timeouts detect/escape some waits but do not make a protocol correct: repeated partial work can create livelock or data inconsistency.

Livelock example: two polite transfer workers detect conflict, both release, sleep for the same fixed delay, and retry in sync forever. Randomized/exponential backoff can reduce synchronization, but a deterministic owner/order is preferable when possible. Starvation can arise from unfair locks, endless high-priority tasks, read-heavy read/write locks, CAS losers, or a scheduler/resource admission policy.

Thread starvation deadlock does not require two locks. In a two-thread executor, two parent tasks submit child tasks to the same executor and block on their futures. Both workers are occupied by parents; children remain queued. Compose without blocking, run child work directly, allocate a separately justified executor, or redesign ownership.

Cancellation failures are liveness/resource bugs. A timeout on a caller does not necessarily stop downstream work. Orphan tasks can later mutate state, retain memory, hold connections, or consume the result of an expired request. Define cancellation propagation, idempotency, deadlines, and what commits are still legal after caller departure.

Thread leakage occurs when threads/executors are created repeatedly and never terminated, or when blocked tasks never receive cancellation. Thread-local leakage occurs when pooled threads retain per-request data after completion. Always remove manually managed thread locals in `finally`; prefer explicit context passing.

Testing must control setup without pretending to control the runtime schedule. Use `CountDownLatch`, `CyclicBarrier`, or `Phaser` to align starts and increase overlap. Use timeouts so a deadlock fails the test rather than hangs the suite. Avoid sleeps as correctness synchronization; a sleep can be a workload delay but should not prove that another thread finished.

Assert invariants and histories, not just final happy values. For a transfer system, total balance and nonnegative balances matter after every legal transition. For a concurrent object, record invocation/response times and results, then check whether a legal sequential ordering exists between them. This is linearizability testing; exhaustive checking can be expensive, so histories are bounded.

The OpenJDK `jctstress` project embodies JVM concurrency stress-testing concepts: actors perform operations concurrently, an arbiter observes outcomes, and results are classified as acceptable, interesting, or forbidden. It is especially useful for JMM litmus tests. Repeating a JUnit method manually is less rigorous because compiler warm-up, actor placement, outcome collection, and fork isolation matter. Stress tools find counterexamples; they do not prove all large programs correct.

Model checking or deterministic scheduler frameworks can explore controlled interleavings for abstractions they support. Static analysis can find inconsistent locking and unsafe publication patterns. Code review remains essential: list shared mutable variables, all accesses, guards, linearization points, cancellation points, and progress assumptions.

Design patterns reduce states:

- **Immutability plus atomic publication:** replace one validated snapshot.
- **Thread confinement/single writer:** one owner mutates state; others send messages.

- **Producer-consumer:** bounded queue separates producers and workers with backpressure.
- **Bulkhead:** independent concurrency limit per dependency/workload.
- **Lock ordering:** acquire multiple locks by a stable global key.
- **Open call:** copy/decide under lock, invoke unknown or slow code after releasing it.
- **Split phase:** begin asynchronous work, release resources, resume on completion.
- **Idempotent command:** retries/cancellation races do not duplicate business effects.
- **Structured lifetime:** child work does not silently outlive its owning operation.

Structured concurrency is a preview API in Java 21, not a finalized Java 21 feature. Its design idea remains useful: task lifetimes form a tree, sibling failure/cancellation is coordinated, and the owner joins before leaving scope.

Specification boundary: Java defines monitor, volatile, atomic, thread lifecycle, and concurrent-library contracts. It does not guarantee fair scheduling or that stress tests explore every legal execution. Deadlock freedom and linearizability are properties the program must establish.

TRACE IT | 38.6 Worked Java example

JAVA (continued 1/2)

```
import java.util.concurrent.locks.ReentrantLock;

public final class BankTransfers {
    static final class Account {
        final long id;
        final ReentrantLock lock = new ReentrantLock();
        long cents;

        Account(long id, long cents) {
            if (cents < 0) throw new IllegalArgumentException("cents");
            this.id = id;
            this.cents = cents;
        }

        long balance() {
            lock.lock();
            try {
                return cents;
            } finally {
                lock.unlock();
            }
        }
    }

    static boolean transfer(Account from, Account to, long cents)
```

JAVA (continued 2/2)

```

        throws InterruptedException {
    if (from == to) return true;
    if (cents <= 0) throw new IllegalArgumentException("cents");
    if (from.id == to.id) throw new IllegalArgumentException("duplicate id");

    Account first = from.id < to.id ? from : to;
    Account second = from.id < to.id ? to : from;

    first.lock.lockInterruptibly();
    try {
        second.lock.lockInterruptibly();
        try {
            if (from.cents < cents) return false;
            from.cents -= cents;
            to.cents += cents;
            return true;
        } finally {
            second.lock.unlock();
        }
    } finally {
        first.lock.unlock();
    }
}

```

Every transfer acquires accounts by ascending stable ID regardless of direction, eliminating a two-account circular wait. Both balance changes occur while both locks are held, so the in-process invariant moves atomically relative to code following the same discipline.

TRACE IT | 38.7 Execution or memory walkthrough

Let account A have ID 1 and B have ID 2. T1 transfers A to B while T2 transfers B to A:

- 1 Both compute `first=A, second=B` because ordering depends on IDs, not direction.
- 2 Suppose T1 acquires A. T2 waits interruptibly for A and owns no other account lock.
- 3 T1 acquires B, validates funds, subtracts/adds, releases B then A.
- 4 Lock release/acquisition orders the updated balances for T2.
- 5 T2 acquires A then B and performs its transition.

No state has T1 holding A waiting for B while T2 holds B waiting for A. The wait-for graph cannot contain that two-lock cycle if every operation respects the same total order.

If T1 is interrupted while waiting for B, `lockInterruptibly` throws. T1 never acquired B, so the inner `try/finally` did not begin. The outer finally releases A. No balances changed because mutation follows both acquisitions.

Overflow is not handled in this educational code. `to.cents += cents` should use exact arithmetic or validated limits in a financial implementation. Across multiple JVMs/database rows, these Java locks do not coordinate other processes; a database transaction or distributed protocol is required.

38.8 Complexity and performance

Transfer does $O(1)$ domain work but lock wait is unbounded without stronger scheduling assumptions. Contention is per account: unrelated account pairs can proceed concurrently. A global bank lock simplifies proof but serializes all transfers; fine-grained account locks increase parallelism and lock-order complexity.

Deadlock detection from a wait-for graph is $O(V + E)$ for cycle detection, but production evidence can omit application-level resources such as database locks or message acknowledgments. Tail latency increases nonlinearly near resource saturation even without deadlock.

Stress testing cost grows with threads, operations, histories, and possible interleavings. Complete schedule exploration is generally exponential. Target small state machines, boundary races, and known weak points, then combine formal reasoning, static checks, unit tests, stress, integration load, and production observability.

WATCH OUT | 38.9 Edge cases and common mistakes

- Concluding "no race" because a test passed repeatedly.
- Fixing a race by adding sleep, logging, or `yield`.
- Taking one thread dump and declaring deadlock without an ownership cycle.
- Applying lock ordering in one method while another path violates it.
- Generating lock-order IDs that can collide or change.
- Timing out lock acquisition but leaving partial side effects before retry.
- Retrying conflicts with identical delay, creating livelock.
- Using fair locks as a complete starvation guarantee.
- Blocking executor workers on child tasks queued to the same executor.
- Letting timed-out work commit non-idempotent effects later.
- Forgetting to shut down executors or clear thread locals.
- Treating a synchronized in-memory invariant as distributed consistency.
- Testing only final count and missing illegal intermediate states.

Priority changes are not a portable correctness fix for priority inversion. OS/JVM mappings vary, and application priorities can worsen starvation. Reduce lock duration, separate workloads, and use explicit admission.

38.10 Production engineering notes

Diagnose from a synchronized timeline:

- 1 Determine whether the symptom is wrong state, no progress, or slow progress.
- 2 Capture several thread dumps, not just one; include timestamps and CPU/load.
- 3 Identify RUNNABLE CPU consumers, monitor owners/contenders, parked explicit-lock waiters, executor queues, and downstream waits.
- 4 Use JVM deadlock detection where available, but add database lock graphs, connection pools, queue lag, and remote dependency state.
- 5 Correlate JFR lock/park events, CPU profiles, GC/safepoints, and request traces.
- 6 Preserve the triggering workload and deploy a protocol fix with a regression stress test.

Incident example: all 32 pool threads show `FutureTask.get`, CPU is near zero, and queue depth grows. Each task submitted an enrichment subtask to the same pool and waited. No Java monitor cycle appears, so automatic deadlock detection reports nothing. This is thread-starvation deadlock. Replace blocking nested submission with completion composition or reserve/avoid the dependency structure; adding workers only moves the threshold.

Livelock evidence looks different: CPU and retry metrics are high, state changes repeatedly, but completed operations remain near zero. Add correlation IDs and attempt counts with rate-limited logs, then establish ownership/order or randomized bounded backoff.

HotSpot note: HotSpot thread dumps and `jcmd Thread.print` can identify owned/waited monitors and some ownable synchronizers; JFR can capture Java monitor enter/wait, park, virtual-thread, and scheduling-related evidence depending on JDK configuration. Tool visibility is not a proof that no application or external deadlock exists.

TRY IT | 38.11 Interview questions and model answers

Deadlock versus livelock versus starvation?

In deadlock, participants are blocked in a cycle or unsatisfiable wait. In livelock, they keep reacting/retrying but complete no useful work. In starvation, the system progresses while one participant is indefinitely denied resources. Diagnosis and remedies differ.

How do you prevent deadlock?

Remove a necessary condition: avoid nested locks, use one lock, acquire all locks in a stable global order, use timed/interruptible acquisition with safe rollback, or redesign around message passing/transactions. Then verify every acquisition path follows the protocol.

How do you test concurrent code?

First prove invariants and happens-before/linearization points. Use latches or barriers to align actors, bounded timeouts to catch hangs, record histories, and stress across many forks/architectures with a harness such as jctstress for JMM cases. Tests find counterexamples but do not replace proof.

What is thread-starvation deadlock?

All executor workers block waiting for tasks that are queued to the same executor, leaving no worker to run them. It may not appear as a monitor cycle. Avoid blocking same-pool dependency chains and compose work or redesign capacity.

Which concurrency design patterns do you prefer?

Start with immutability, confinement/single ownership, immutable snapshot publication, bounded message passing, and high-level concurrent operations. Use locks for explicit multi-field invariants, with stable ordering and open calls. The best design minimizes shared mutable states.

TRY IT | 38.12 Exercises

- 1 Write a two-lock deadlock, capture it safely, then impose a global order.
- 2 Build a same-executor parent/child starvation example and replace blocking gets with composition.
- 3 Design a barrier-based lost-update test with acceptable and forbidden outcomes.
- 4 Define a linearizable history checker for a tiny concurrent stack with at most four operations.
- 5 Refactor a callback-under-lock component using the open-call pattern.
- 6 Design cancellation propagation for an HTTP request, database query, and message publish with an idempotency key.
- 7 List safety, liveness, and performance requirements for a bounded work queue separately.

RECAP | 38.13 Chapter summary

Concurrency correctness requires separate safety, liveness, and performance arguments. Races violate atomicity or ordering; deadlocks form unsatisfiable waits; livelocks spend work without progress; starvation denies one participant. Tests should coordinate actors, assert invariants/histories, use deadlines, and stress with purpose, while recognizing that absence of a failure is not proof. Immutability, confinement, bounded message passing, lock ordering, open calls, bulkheads, and structured lifetimes reduce the state space and operational risk.

TRY IT | 38.14 Revision checklist

- ☐ I can distinguish data races, race conditions, lost updates, and missed signals.
- ☐ I can draw a wait-for graph and apply a stable lock order.
- ☐ I can identify deadlock, livelock, starvation, and same-pool starvation evidence.
- ☐ I use barriers/latches and bounded timeouts rather than sleeps for tests.
- ☐ I understand linearizability histories and jctstress-style outcome testing.
- ☐ I can design cancellation and shutdown without orphan work.
- ☐ I prefer immutability, confinement, message passing, and explicit ownership.
- ☐ I separate in-process synchronization from distributed consistency.

JAVA FOUNDATIONS TO ADVANCED ENGINEERING

Part VI - Performance, Diagnostics, and Reliability

A focused part of the complete SDE-2 preparation guide

SDE-2 CORE CHAPTER

Chapter 39 - Performance Methodology and JMH Benchmarking

39.1 Learning objectives

By the end of this chapter, you should be able to:

- turn a vague performance concern into a falsifiable question and a measurable objective;
- distinguish latency distributions, throughput, utilization, allocation, and asymptotic cost;
- explain warmup, tiered compilation, dead-code elimination, constant folding, and benchmark contamination;
- build a disciplined JMH benchmark with state, forks, warmup, measurement, and parameters;
- interpret measurements with uncertainty instead of selecting one attractive number; and
- connect microbenchmark evidence to profiling, load tests, production traces, and business constraints.

WHY IT WORKS | 39.2 Why this matters at SDE-2

Performance work at SDE-2 is decision work. The engineer must identify which user-visible objective is failing, collect evidence at the correct layer, and make the smallest change that improves the limiting resource without violating correctness. "This collection is faster" or "the JVM will optimize it" is not an engineering argument.

Java makes naive measurement particularly misleading. Code begins interpreted or lightly compiled, hot methods may be recompiled, unused work can disappear, object allocations can be scalar-replaced, garbage collection can interrupt samples, and the operating system can move the process between CPUs. JMH handles many mechanics, but it cannot repair a meaningless workload or prove that a micro-level improvement changes end-to-end behavior.

WHY IT WORKS | 39.3 First-principles model

Performance is work completed per constrained resource and time. Begin with an observable target:

TEXT

Question: Why did checkout p99 latency rise from 180 ms to 420 ms?
Scope: production requests in region A after release R
Constraints: error rate $\leq 0.1\%$, same durability and validation
Evidence: latency histogram, traces, CPU, allocation, GC, downstream timings
Hypothesis: repeated JSON materialization increased allocation and GC pauses
Experiment: remove one copy, compare controlled canary and allocation profile

The feedback loop is:

TEXT

```
define -> measure baseline -> form hypothesis -> design experiment  
      -> change one variable -> validate -> deploy safely -> observe
```

Latency is a distribution, not one scalar. Throughput and latency interact through queueing: as utilization approaches capacity, waiting time can rise sharply. A faster isolated operation may not improve the system if a database, lock, network call, or admission limit remains the bottleneck.

Specification boundary: Java SE does not specify JIT compilation thresholds, escape analysis outcomes, code layout, garbage collector ergonomics, or JMH behavior. These are implementation and tool concerns. Preserve Java semantics, but measure optimization behavior on the actual JDK, collector, hardware, and configuration.

39.4 Core terminology

- **Throughput:** Completed operations per unit time.
- **Latency:** Time for one operation; report percentiles and the measurement window.
- **Tail latency:** High-percentile latency such as p95, p99, or p99.9.
- **Utilization:** Fraction of a resource's available capacity in use.
- **Service time:** Active processing time excluding some or all queue waiting, depending on definition.
- **Warmup:** Execution before measurement so runtime state approaches the intended regime.
- **Fork:** Fresh JVM process used as an independent benchmark run.
- **Steady state:** Period whose relevant runtime behavior is sufficiently stable for the question.
- **Dead-code elimination:** Removal of computations whose results cannot affect observable behavior.
- **Constant folding:** Compile-time or JIT-time evaluation of expressions known to be constant.
- **Escape analysis:** Reasoning that can enable allocation elimination or synchronization optimization.
- **Benchmark state:** Data whose lifecycle and sharing are controlled by the harness.
- **Confidence interval/error estimate:** Range describing measurement uncertainty under a statistical model.
- **Coordinated omission:** Missing slow observations because the load generator waits before issuing the next request.

39.5 Detailed mechanics

Define the metric before the experiment

State the operation, population, units, percentile, traffic shape, input distribution, and correctness constraints. "Average endpoint time" can hide a damaging tail. CPU time excludes blocking. Wall time includes scheduling and waiting. Allocation rate does not directly equal retained heap. Pick metrics tied to the hypothesis.

Record a baseline and a control. Change one primary variable. Preserve source data, JVM arguments, dependency versions, CPU limits, and test duration when possible. Run enough independent trials to see variance. If a result changes sign across forks, the honest conclusion is uncertainty, not the best-looking fork.

Why stopwatch loops fail

`System.nanoTime` is suitable for elapsed intervals but does not create a correct harness. A hand-written loop often measures compilation transitions, loop machinery, a constant expression, timer calls, or shared setup. It can also omit the computed result, allowing the optimizer to remove the work.

The JIT optimizes the program it sees. If benchmark inputs are constants, it may precompute results. If an allocated object does not escape, it may never become a heap object. These are valid production optimizations, but a benchmark intended to measure allocation or general inputs must prevent accidental specialization without preventing realistic optimization.

JMH execution model

JMH, the Java Microbenchmark Harness, is a separate tool rather than a Java SE API. Its generated harness controls invocation, warmup, measurement, result consumption, and process forks.

Important annotations include:

- `@Benchmark` marks measured methods.
- `@BenchmarkMode` selects throughput, average time, sample time, or single-shot behavior.
- `@OutputTimeUnit` chooses presentation units.
- `@Warmup` and `@Measurement` configure iteration phases.
- `@Fork` requests fresh JVM processes.
- `@State` defines state sharing: per thread, benchmark instance, or group.
- `@Param` expands controlled input cases.
- `@Setup` and `@TearDown` manage state outside timed invocations at trial, iteration, or invocation level.

Use multiple forks for JVM-level independence. Warmup is not "always five iterations"; it must be long enough for the workload and question. Inspect per-iteration results for drift. Single-shot mode is appropriate for cold or one-off operations only when that is explicitly the target.

Preventing invalid optimization without disabling the JVM

Return the result from the benchmark or pass it to JMH's `Blackhole`. Do not add arbitrary logging, volatile writes, or synchronization merely to keep work alive; they can dominate the measurement. Supply varied state through parameters or setup when constants are unrealistic.

Move construction out of timed code unless construction is the operation under test. If each invocation mutates state, reset at a deliberate lifecycle level and include or exclude reset cost according to the real question. Invocation-level setup can be expensive enough to distort very small operations.

Benchmark modes and profilers

Throughput mode answers operations per time. Average time reports mean operation time under the harness model. Sample time records a sample distribution; it is not a replacement for a production latency histogram. Single-shot measures individual invocations and is sensitive to noise.

JMH profilers can add allocation, GC, stack, compiler, or platform-counter evidence, but profilers perturb execution. Compare profiled runs with unprofiled runs. Hardware counters and assembly views depend on OS permissions, architecture, installed tools, and JVM support.

HotSpot note: Tiered compilation, inlining budgets, on-stack replacement, escape analysis, and code-cache behavior are HotSpot implementation details and version-sensitive. A benchmark result from Java 17 is evidence for that setup, not a promise for Java 21 or another JVM.

From micro to macro

A microbenchmark isolates mechanism. A component benchmark includes serialization, pools, and local dependencies. A load test checks queueing and capacity. Production observation validates actual inputs, contention, and downstream behavior. Use the lowest layer that can answer the question, then validate upward.

Do not extrapolate nanoseconds saved per isolated call by multiplying by request count unless the operation is actually on the critical path at that frequency. Profiling should confirm contribution to CPU or latency. Optimization can shift cost to memory, startup, code size, or maintainability.

TRACE IT | 39.6 Worked Java example

This JMH benchmark compares miss lookup in a list and a set. It isolates lookup by constructing state once per trial:

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.concurrent.TimeUnit;
import java.util.stream.IntStream;
import org.openjdk.jmh.annotations.Benchmark;
import org.openjdk.jmh.annotations.BenchmarkMode;
import org.openjdk.jmh.annotations.Fork;
import org.openjdk.jmh.annotations.Level;
import org.openjdk.jmh.annotations.Measurement;
import org.openjdk.jmh.annotations.Mode;
import org.openjdk.jmh.annotations.OutputTimeUnit;
import org.openjdk.jmh.annotations.Param;
import org.openjdk.jmh.annotations.Scope;
import org.openjdk.jmh.annotations.Setup;
import org.openjdk.jmh.annotations.State;
import org.openjdk.jmh.annotations.Warmup;

@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 5, time = 1)
@Measurement(iterations = 7, time = 1)
@Fork(3)
public class MembershipBenchmark {
    @State(Scope.Thread)
    public static class Data {
        @Param({"16", "1024", "65536"})
```

JAVA (continued 2/2)

```

    int size;

    List<Integer> list;
    Set<Integer> set;
    Integer missing;

    @Setup(Level.Trial)
    public void setup() {
        ArrayList<Integer> values = IntStream.range(0, size)
            .boxed()
            .collect(java.util.stream.Collectors.toCollection(ArrayList::new));
        list = values;
        set = new HashSet<>(values);
        missing = -1;
    }

    @Benchmark
    public boolean listMiss(Data data) {
        return data.list.contains(data.missing);
    }

    @Benchmark
    public boolean setMiss(Data data) {
        return data.set.contains(data.missing);
    }
}

```

The JMH annotation processor and runtime must be configured in the build. Pin their version rather than copying an unversioned command from another project. A typical generated benchmark artifact can be filtered by benchmark name and optionally run with a GC profiler, but exact command-line options belong to the selected JMH version.

TRACE IT | 39.7 Execution or memory walkthrough

For each fork, JMH starts a new JVM. For each parameter value it creates thread-scoped `Data`, runs trial setup, then executes warmup iterations whose results are not reported as measurements. Compilation and profile information evolve during warmup. Measurement iterations follow and JMH aggregates fork-level observations.

For `size = 16`, `list.contains(-1)` checks 16 boxed integer references/equality operations. The hash set computes a hash, selects a bucket, and usually confirms absence after a small amount of work, but it has a larger structure and less locality. At tiny sizes, constants may make the list competitive. At 65,536 elements, the list miss is linear while expected set lookup remains constant under good hashing.

The benchmark returns a Boolean result to the harness, preventing trivial removal. It does not time collection construction, iteration, memory footprint, cache warmness across production requests, successful lookups at varied positions, or concurrent access. Those are separate experiments.

39.8 Complexity and performance

The underlying algorithm predicts trends:

Operation	List	Hash set, typical expectation
miss lookup	$O(n)$	expected $O(1)$
build from n values	$O(n)$	expected $O(n)$
storage	$O(n)$ references	$O(n + \text{capacity})$ plus entry overhead
ordered iteration	insertion sequence for list	no general hash-set order

Big-O does not predict the crossover size. Hash computation, equality, allocation, cache locality, table load, JVM optimization, and hardware determine constants. The benchmark measures one point in this design space.

Measurement cost also matters. More forks and longer iterations improve confidence but consume time. Prefer enough independent evidence to make the decision, not maximal duration by ritual. Report configuration, sample counts, central estimates, uncertainty, outliers, and all tested parameter values. Never report more precision than noise supports.

WATCH OUT | 39.9 Edge cases and common mistakes

- Optimizing without a baseline, SLO, profile, or explicit hypothesis.
- Reporting only averages and hiding tail behavior or errors.
- Timing startup while claiming steady-state throughput, or warming up while claiming cold startup.
- Leaving benchmark results unused and measuring code eliminated by the optimizer.
- Putting data generation, logging, assertions, or random seeding in the timed operation unintentionally.
- Sharing mutable benchmark state across threads without matching production semantics.
- Running one fork and treating one score as universal.
- Comparing runs with different CPU quotas, power modes, thermal states, JVM flags, or background load.
- Using unrealistic constant inputs that enable specialization.
- Treating a profiler run as unperturbed timing evidence.
- Assuming a statistically significant micro improvement is operationally meaningful.
- Running load generators that suffer coordinated omission and under-report queue delay.
- Claiming causality from correlation in one production graph.

39.10 Production engineering notes

Start with production-safe telemetry: request histograms, traces, error rate, CPU, allocation, GC, pool saturation, database timing, and queue depth. Align timestamps and release markers. Sampling profilers are usually safer than ad hoc high-frequency instrumentation, but every diagnostic has cost.

Benchmark on the same major JDK, collector, architecture, and container limits as production. Pin benchmark code and dependencies in source control. Capture JVM arguments and environment metadata with results. Run noisy-neighbor and concurrency tests when shared infrastructure is part of the question.

Optimize one confirmed hot path, add correctness and performance regression tests at the appropriate layer, and canary the change. A microbenchmark threshold in CI can be flaky; compare distributions on controlled runners and alert on substantial, repeated regressions rather than nanosecond drift.

Stop when the SLO and capacity target are met with margin. Further optimization spends complexity budget and can reduce reliability. Preserve a written experiment log so the next incident does not repeat disproven hypotheses.

TRY IT | 39.11 Interview questions and model answers

Why is a loop around `System.nanoTime` not enough?

It does not control warmup, forks, dead-code elimination, setup, or statistical independence. Timer and loop overhead can dominate. JMH supplies a generated harness, but the workload still needs a valid question and realistic inputs.

Why does JMH use warmup and forks?

Warmup lets runtime compilation and profiles approach the target regime. Forks create fresh JVM processes, reducing dependence on one process's compilation, heap, and code-cache history.

How do you prevent dead-code elimination?

Return the computed result or consume it through JMH's `Blackhole`. Also avoid fully constant inputs if the production operation receives varied data.

When is a microbenchmark the wrong tool?

When the question depends on I/O, queueing, thread-pool saturation, distributed calls, or user-visible tail latency. Use component, load, or production evidence for those effects.

Can parallel code have higher throughput but worse latency?

Yes. It can increase contention, coordination, queueing, allocation, and tail variation while completing more aggregate work. Measure both under the intended concurrency.

What would you report with a benchmark result?

Question, code revision, JDK and arguments, hardware and limits, JMH configuration, inputs, forks, score and uncertainty, profiler observations, correctness checks, and the scope of conclusions.

TRY IT | 39.12 Exercises

- 1 Design a JMH benchmark comparing string concatenation strategies for 10, 100, and 10,000 fragments. Decide whether construction belongs inside the timed method.
- 2 Find three ways a benchmark of `Math.log(42.0)` could measure constant folding rather than general logarithm cost.
- 3 Write separate experiments for cold startup and warmed request throughput. Explain why one configuration cannot answer both.
- 4 Given p50 20 ms, p99 900 ms, low CPU, and a saturated connection pool, form three hypotheses and choose evidence for each.
- 5 Add successful first, middle, and last list lookups to the worked example without introducing random-number generation into timed code.
- 6 Review a claimed 3 percent speedup whose fork error intervals overlap substantially. Write the appropriate conclusion and next experiment.

RECAP | 39.13 Chapter summary

Performance engineering begins with a user-visible objective, a baseline, and a falsifiable hypothesis. Java runtime optimization makes naive timing unreliable. JMH controls microbenchmark lifecycle, result consumption, warmup, measurement, and forks, but it cannot supply realistic semantics. Use complexity to predict trends, profilers to locate mechanisms, higher-level tests to expose queueing and integration effects, and production canaries to validate impact. Report uncertainty and stop when measured objectives are met.

TRY IT | 39.14 Revision checklist

- ☐ I define the operation, distribution, metric, load, and correctness constraints before measuring.
- ☐ I distinguish throughput, latency percentiles, service time, utilization, allocation, and retention.
- ☐ I can explain warmup, tiered compilation, dead-code elimination, constant folding, and escape analysis.
- ☐ I use JMH state, setup, parameters, measurement iterations, and forks intentionally.
- ☐ I consume benchmark results and isolate setup according to the question.
- ☐ I report environment, uncertainty, outliers, and scope of conclusions.
- ☐ I validate microbenchmark findings with profiles and the appropriate higher-level experiment.
- ☐ I label HotSpot and tool behavior as version-sensitive.

SDE-2 CORE CHAPTER

Chapter 40 - JVM Diagnostics with jcmd, jstack, jmap, JFR, and Mission Control

40.1 Learning objectives

By the end of this chapter, you should be able to:

- build a timestamped incident timeline before changing JVM state;
- select low-risk diagnostics for CPU, latency, deadlock, allocation, heap, and native-memory symptoms;
- use `jcmd`, thread dumps, Java Flight Recorder, and heap tools with explicit cost and data-handling controls;
- interpret thread states, repeated stacks, JFR events, histograms, and heap dumps without overclaiming causality;
- explain when `jstack`, `jmap`, and Mission Control are appropriate; and
- write a safe evidence-capture runbook for Java 17 and Java 21 deployments.

WHY IT WORKS | 40.2 Why this matters at SDE-2

During an incident, diagnostic commands are production changes. Some briefly stop application threads, some walk the heap, some write files as large as the heap, and many capture secrets. An SDE-2 engineer must gather enough evidence to distinguish CPU saturation, lock contention, garbage collection, downstream waiting, native-memory growth, and deadlock while avoiding a second outage caused by diagnosis.

The strongest incident report correlates sources. A thread dump is a moment, a JFR recording is a timeline, service metrics show impact, and a heap dump shows one graph. None alone proves root cause. Preserve timestamps, process identity, container identity, release version, JVM configuration, host pressure, and command results so observations can be aligned.

WHY IT WORKS | 40.3 First-principles model

Diagnostics answer three questions:

TEXT

What is the symptom?
Which resource or dependency is limiting progress?
What evidence would disprove each plausible cause?

Use an escalation ladder:

TEXT

```
existing metrics/logs/traces
  -> process identity and JVM metadata
  -> short JFR or repeated thread dumps
  -> targeted histograms/native-memory summaries
  -> heap dump or invasive tooling only with capacity and approval
```

Start with evidence already being collected because it adds no incident-time perturbation. Prefer time-bounded, reversible capture. Do not tune, restart, force GC, or clear caches before obtaining a baseline unless immediate recovery takes priority over diagnosis. If recovery is necessary, say which evidence was lost.

Specification boundary: These tools are JDK and vendor facilities, not Java Language Specification guarantees. Command names, accepted arguments, attach behavior, event fields, virtual-thread representation, and overhead can change between JDK builds. Inspect `jcmd <pid> help` on the target runtime and test runbooks on the exact distribution.

40.4 Core terminology

- **Attach:** Mechanism by which a diagnostic process connects to a running JVM.
- **Safepoint:** Runtime state where selected VM operations can safely inspect or modify global state.
- **Thread dump:** Snapshot of Java threads, states, stacks, and optionally owned synchronizers.
- **JFR:** Java Flight Recorder, an event recording facility integrated with supported JDKs.
- **JMC:** Java Mission Control, a graphical analysis application commonly used with JFR recordings.
- **Class histogram:** Counts and shallow bytes grouped by class.
- **Heap dump:** Snapshot of heap objects and references, commonly in HPROF form.
- **Native memory:** Process memory outside ordinary Java heap, including metaspace, code, threads, direct buffers, and native libraries.
- **NMT:** HotSpot Native Memory Tracking.
- **Deadlock:** Cycle in which participants wait indefinitely for resources held by one another.
- **Perturbation:** Change in application behavior caused by measurement.
- **Evidence custody:** Secure storage, access control, retention, and deletion of diagnostic artifacts.

40.5 Detailed mechanics

Establish identity and context

Confirm the operating-system PID inside the relevant PID namespace. In containers, a host PID and container PID can differ. Record current time with timezone, pod or host, deployment revision, traffic, and whether the symptom is active. Avoid relying solely on `jps`; not every Java process is discoverable in every namespace or permission setup.

`jcmd <pid> VM.version, VM.command_line`, and `VM.flags` can capture runtime identity and configuration. System properties and command lines may contain credentials, tokens, paths, or customer identifiers, so collect them only into restricted storage. `jcmd <pid> help` lists commands actually available in that JVM.

Thread dumps as repeated snapshots

`jcmd <pid> Thread.print -l` is the preferred general HotSpot route in many modern deployments. `jstack -l <pid>` is a useful fallback when available and supported. Capture several dumps separated by a meaningful interval. A repeated identical stack on an on-CPU method is stronger evidence than one appearance.

Common thread states require context:

- **RUNNABLE** means eligible/running from the JVM perspective; it can include native I/O and does not prove CPU consumption.
- **BLOCKED** means waiting to enter an intrinsic monitor.
- **WAITING** can be normal parking in a pool, queue, or `join`.
- **TIMED_WAITING** includes timed sleep, park, and wait.
- terminated threads do not appear as live work.

Look for deadlock reports, many threads blocked behind the same owner, pool threads waiting on downstream calls, and stacks that remain active across captures. Correlate native thread IDs with OS CPU tools where supported. Thread names are operational API: name pools and critical workers meaningfully.

Virtual threads change scale. Dump formats and commands capable of representing large virtual-thread sets have evolved across JDK releases. Do not assume one platform-thread-style dump will list or interpret every virtual-thread task identically; validate Java 21 tooling and prefer JFR events or supported virtual-thread dumps for the target build.

Java Flight Recorder

JFR records timestamped JVM and application events with configurable settings. Useful event families include execution samples, allocation samples, garbage collection, monitor contention, thread parking, file/socket I/O, exceptions, class loading, and JVM configuration. Availability and fields are version-dependent.

A continuously running, size- and age-bounded recording is often more valuable than starting after an event. It preserves the lead-up. If continuous recording is not configured, `jcmd` can start a named, duration-bounded recording and write it to a restricted path. The `default` configuration targets lower overhead; `profile` commonly collects more detail at higher cost. Validate both overhead and disk behavior under load.

Typical command shapes are:

BASH

```
jcmd 12345 JFR.check  
jcmd 12345 JFR.start name=incident settings=default duration=120s filename=/restricted/incident.jfr  
jcmd 12345 JFR.dump name=incident filename=/restricted/snapshot.jfr  
jcmd 12345 JFR.stop name=incident
```

Do not paste these blindly. Confirm PID, available help, output directory, free space, permissions, recording name, and the JDK's syntax. JFR can contain class names, paths, stack traces, thread names, URLs, and application event fields.

Mission Control opens recordings for timeline analysis, event browsing, automated rules, flame-like views, lock analysis, and GC inspection. It is an analyzer, not an oracle. Automated warnings are leads to correlate with service impact. JMC versions are distributed separately from some JDKs and should be compatible with the recording format in use.

Class histograms and heap dumps

A class histogram answers "which classes account for many live or heap-visible objects and shallow bytes?" It does not show why they are retained. `jcmd <pid> GC.class_histogram` and `jmap -histo` variants can be costly because implementations may require a safepoint and heap traversal; live-only variants can trigger additional GC work. Syntax and impact vary.

A heap dump preserves objects and reference edges for dominator and path-to-root analysis. It can pause the process, perform substantial I/O, consume disk near heap size or more, expose secrets, and worsen a memory-starved host. Verify disk headroom, storage performance, encryption/access controls, and whether a replica can be removed from traffic. Prefer `jcmd` heap-dump facilities over older `jmap` paths when the target JDK recommends them.

Configure `-XX:+HeapDumpOnOutOfMemoryError` and a controlled dump path only after validating disk, permissions, retention, and restart behavior. The JVM may be too impaired to complete a dump. Never make heap-dump success the only OOM evidence plan.

Native memory and process evidence

Resident-set growth with stable heap may come from thread stacks, direct buffers, metaspace, code cache, JNI, memory mapping, allocators, or kernel accounting. Process metrics and OS maps provide evidence. On HotSpot, NMT can categorize tracked native allocations, but it must be enabled at process startup and adds overhead. Summary tracking is normally less costly than detail tracking.

With NMT enabled, `jcmd <pid> VM.native_memory baseline` followed later by `summary.diff` or the target version's equivalent can expose category growth. NMT does not track every native allocation and its totals need not equal RSS. Treat categories as a model to correlate, not exact process accounting.

HotSpot note: Attach implementation, safepoint behavior, NMT categories, JFR event sets, compiler names, and diagnostic-command side effects are HotSpot and build specific. Container security policies can disable attach even when commands are present.

Tool failure is evidence, not permission to escalate blindly

Attach can fail because of permissions, namespaces, disabled mechanisms, a hung JVM, or incompatible tools. Use tools from the same JDK family as the target. Do not reach immediately for forced serviceability agents or debuggers on production; they can suspend or destabilize the process. Follow an approved runbook and decide whether replica failover, core dump, or restart is safer.

TRACE IT | 40.6 Worked Java example

This bounded program creates observable intrinsic-lock contention without deadlocking forever:

JAVA

```
import java.util.ArrayList;
import java.util.List;

public final class ContentionDemo {
    private static final Object LOCK = new Object();

    private static void work() {
        for (int i = 0; i < 100; i++) {
            synchronized (LOCK) {
                try {
                    Thread.sleep(20);
                } catch (InterruptedException interrupted) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {
        List<Thread> workers = new ArrayList<>();
        for (int i = 0; i < 4; i++) {
            Thread worker = new Thread(ContentionDemo::work, "worker-" + i);
            workers.add(worker);
            worker.start();
        }
        for (Thread worker : workers) {
            worker.join();
        }
    }
}
```

In a disposable local environment, run the process, find its PID safely, and capture two thread dumps while it is active. A short JFR recording can then show monitor-blocked events. Do not run an artificial contention program on a shared production host.

TRACE IT | 40.7 Execution or memory walkthrough

At most one worker owns `LOCK`. That worker calls `sleep` while still inside the synchronized block, so it appears `TIMED_WAITING` while retaining the monitor. Other workers attempting entry appear `BLOCKED`, and the dump identifies the monitor owner when lock detail is available.

Twenty milliseconds later, the owner wakes and exits. Scheduling and monitor acquisition determine the next owner; intrinsic locks do not promise strict FIFO fairness. Across several dumps, different workers may own the monitor, but the structural pattern remains one sleeper and multiple blocked contenders.

A JFR timeline adds duration: it can reveal how long monitor entry was blocked and which stack requested it. OS CPU may remain low because most threads wait. This combination disproves the hypothesis that high latency necessarily means CPU saturation.

No deadlock exists because the owner eventually releases one lock and there is no dependency cycle. A single dump with blocked threads is therefore not a deadlock diagnosis.

40.8 Complexity and performance

Diagnostic cost scales with captured scope:

Evidence	Typical relative impact	Scaling concern
existing metrics/logs	already paid	cardinality and logging volume
thread dump	usually brief	number of threads and stack depth
low-overhead JFR	designed for continuous use	event rate, stack traces, disk limits
profile JFR	higher	sampling/event configuration and workload
class histogram	moderate to high	heap object count and safepoint work
heap dump	high	live/used heap, pause, disk and I/O bandwidth
detailed NMT	startup-enabled overhead	native allocation rate and tracking detail

These are not universal rankings. A JVM with millions of virtual threads, a nearly full disk, or a stalled safepoint can change risk. Estimate artifact size and latency, test on a realistic replica, and define an abort condition.

Analysis complexity also matters. A histogram is small enough for quick comparison but lacks edges. A heap dump provides retention paths but can take significant offline processing memory. JFR duration and event settings trade history against file size and detail.

WATCH OUT | 40.9 Edge cases and common mistakes

- Attaching to the wrong PID or wrong container namespace.
- Collecting credentials from command lines, properties, heap, or JFR into world-readable files.
- Taking one thread dump and labeling every waiting thread a problem.
- Treating **RUNNABLE** as proof that a thread consumes CPU.
- Calling blocked threads a deadlock without a wait cycle.
- Forcing GC before preserving the heap behavior under investigation.
- Requesting a heap dump on a nearly full filesystem or latency-critical sole instance.
- Assuming a histogram's shallow bytes identify the retaining owner.
- Comparing NMT totals directly with RSS and declaring the difference a leak.
- Starting a high-detail recording without duration and size controls.
- Using diagnostic tools from an incompatible JDK distribution or version.
- Restarting before recording JVM arguments, timeline, and at least low-cost evidence.
- Uploading diagnostic artifacts to unapproved analysis services.
- Treating a Mission Control rule as proven causality.

40.10 Production engineering notes

Prepare diagnostics before incidents. Bake compatible tools into an approved debug image or host path, configure secure artifact storage, define who may attach, and test commands against Java 17 and Java 21 services. Keep commands parameterized by verified PID and never use broad kill or file targets.

Enable bounded continuous JFR where overhead tests permit. Give recordings unique incident IDs and UTC timestamps. Apply least privilege, encryption, short retention, and an audit trail. Heap dumps usually require the strongest classification because they can contain entire request bodies and credentials.

Capture three synchronized views: application impact, JVM behavior, and host/container limits. CPU throttling, memory limits, file descriptors, DNS, storage, and downstream pools can imitate JVM problems. Preserve deployment events and feature-flag changes.

Separate recovery from root-cause analysis. Traffic shedding, failover, or restart may be the correct first action. Record that decision, collect evidence from another replica or continuous recording, and reproduce later. Never delay recovery solely to obtain a perfect dump.

TRY IT | 40.11 Interview questions and model answers

What would you collect for a Java latency spike?

First correlate request histograms, errors, traces, CPU/throttling, GC, allocation, pools, and downstream latency. Then verify process metadata and capture a short JFR or repeated thread dumps. Escalate to heap evidence only if the hypothesis requires it and impact is acceptable.

Why take multiple thread dumps?

One dump is a snapshot. Repeated dumps show whether the same stacks make no progress, which owners change, and whether contention or downstream waiting persists.

What is the difference between a histogram and a heap dump?

A histogram aggregates class counts and shallow bytes. A heap dump preserves individual objects and reference edges, enabling dominator and root-path analysis at much greater cost and sensitivity.

How do JFR and JMC relate?

JFR records timestamped events in the JVM. Mission Control analyzes recordings and presents timelines and rules. It helps form hypotheses but does not automatically prove root cause.

Can NMT prove a native-memory leak?

It can show growth in tracked HotSpot categories and guide a hypothesis. It omits some allocations and differs from RSS, so correlate it with process maps, direct-buffer metrics, thread counts, and repeated baselines.

Why can a diagnostic command be dangerous?

It can safepoint or pause the JVM, traverse a huge heap, consume disk and I/O, expose sensitive data, or fail under attach restrictions. Validate impact and storage before running it.

TRY IT | 40.12 Exercises

- 1 Given high CPU and stable GC, design a capture sequence using OS per-thread CPU, repeated dumps, and JFR samples.
- 2 Given rising RSS but flat committed heap, list evidence for thread stacks, direct buffers, metaspace, and native libraries.
- 3 Write a runbook precheck for heap dumping a 24 GB service. Include replica status, disk, access, pause, and deletion.
- 4 Run the contention example locally and explain `BLOCKED`, `TIMED_WAITING`, monitor ownership, and why it is not deadlock.
- 5 Design a continuous JFR retention policy for a service with strict customer-data controls.
- 6 Review a thread dump with 200 idle pool workers in `WAITING`. Explain what additional evidence is required before tuning the pool.

RECAP | 40.13 Chapter summary

JVM diagnosis is controlled evidence collection. Begin with timelines and existing telemetry, verify process identity, and escalate from repeated thread snapshots or bounded JFR to histograms and heap dumps only when the hypothesis justifies their cost. Thread state needs temporal and OS context. JFR provides event history; Mission Control supports analysis; heap and native-memory tools answer narrower retention questions. Every artifact is sensitive, every command is version-dependent, and recovery may take priority over perfect evidence.

TRY IT | 40.14 Revision checklist

- ☐ I verify PID, namespace, JDK, timestamp, release, and symptom before attachment.
- ☐ I can interpret thread states without equating **RUNNABLE** with CPU or waiting with failure.
- ☐ I capture repeated dumps and correlate them with OS and request evidence.
- ☐ I can start, dump, and stop a bounded JFR recording after checking target help.
- ☐ I distinguish JFR recording from Mission Control analysis.
- ☐ I know the cost and information difference among histogram, heap dump, and NMT.
- ☐ I secure artifacts and check disk, replica capacity, and abort conditions.
- ☐ I label HotSpot, vendor, container, and JDK-version behavior explicitly.

SDE-2 CORE CHAPTER

Chapter 41 - Memory Leaks, GC Incidents, and Tuning Playbooks

41.1 Learning objectives

By the end of this chapter, you should be able to:

- distinguish high allocation, intended live data, heap retention leaks, native growth, and resource leaks;
- read heap occupancy, allocation, pause, CPU, promotion, and process-memory signals together;
- investigate a retention path from GC root to dominated objects;
- execute a staged memory or GC incident playbook without destroying evidence;
- tune only after defining latency, throughput, footprint, and headroom goals; and
- prevent common leaks through bounded ownership, lifecycle APIs, and observability.

WHY IT WORKS | 41.2 Why this matters at SDE-2

An out-of-memory failure is an endpoint, not a root cause. A growing cache, an overloaded queue, a class-loader leak, direct-buffer growth, too many threads, or an undersized container can all end in memory failure and require different fixes. Increasing `-Xmx` can buy recovery time, hide an unbounded invariant, or make pauses and heap dumps larger.

SDE-2 engineers are expected to stabilize service, classify the memory domain, preserve evidence, and separate allocation pressure from unwanted retention. They must also resist tuning folklore. A collector flag is not a substitute for controlling cardinality, closing resources, removing listeners, bounding concurrency, and respecting the process memory limit.

WHY IT WORKS | 41.3 First-principles model

Garbage collection reclaims objects that are not reachable from GC roots. A heap leak in a managed runtime is therefore unwanted reachability, not forgotten manual deallocation:

TEXT

GC root -> long-lived owner -> collection -> entry -> large object graph

The key quantities are:

TEXT

```
allocation rate = bytes allocated per unit time
live set       = bytes still reachable after an effective collection
headroom      = usable memory above live set for allocation and GC work
```

High allocation with a stable post-collection baseline is pressure, not necessarily a leak. A rising post-collection baseline under comparable load suggests increasing live data. Stable Java heap with rising process resident memory suggests a non-heap or native domain.

The investigation loop is:

TEXT

```
stabilize -> classify memory domain -> preserve timeline
           -> compare repeated evidence -> find owner/path
           -> fix lifecycle or capacity -> load-test -> canary -> monitor
```

Specification boundary: Java specifies reachability concepts and reference types, not a collector algorithm, generation layout, region size, pause target, object age threshold, or out-of-memory message. Collector behavior and diagnostic counters are JVM/vendor/version specific.

41.4 Core terminology

- **GC root:** Starting point for reachability, such as selected thread, static, class-loader, JNI, or VM references.
- **Live set:** Objects that remain reachable after collection under the chosen observation.
- **Shallow size:** Memory occupied directly by one object.
- **Retained size:** Memory that could become reclaimable if an object were removed, under dominator analysis.
- **Dominator:** Object through which every root path to another object passes.
- **Allocation pressure:** High creation rate that forces frequent collection even if objects die quickly.
- **Promotion:** Movement or classification of surviving objects into longer-lived storage in a generational collector.
- **Fragmentation:** Free memory exists but not in a form suitable for requested allocation or collector progress.
- **Humongous/large object:** Collector-specific category for unusually large allocation.
- **Metaspace:** HotSpot native memory commonly used for class metadata.
- **Direct memory:** Native storage used by direct byte buffers and related I/O.
- **Leak slope:** Rate at which a stable-comparison memory baseline grows.
- **Backlog:** Work retained because production exceeds consumption.

41.5 Detailed mechanics

Classify before collecting expensive evidence

Start with container or host memory, Java heap used/committed/max, post-GC occupancy, allocation rate, GC pause and CPU, thread count, loaded classes, direct-buffer pool metrics, file descriptors, and queue/cache cardinality. Align them with traffic and deployment changes.

Useful patterns include:

- high allocation, frequent collections, stable baseline: allocation pressure;
- rising post-collection old/live occupancy: growing retained set;
- heap near max, low reclaim, repeated long collections: exhaustion or oversized live set;
- stable heap, rising RSS: direct/native/thread/metaspaces/mapping hypothesis;
- rising loaded classes and metaspaces after redeploy-like activity: class-loader retention hypothesis;
- queue depth and memory rising together: downstream throughput or backpressure failure.

A single sawtooth graph cannot identify an owner. Compare the same metric under comparable load and collector phase. Concurrent collectors can report occupancy at phases that do not correspond to a stop-the-world full collection.

Allocation pressure versus retention

Allocation pressure is often caused by repeated parsing, boxing, temporary collections, copying, oversized buffers, logging arguments, or retry amplification. JFR allocation samples and an allocation profiler identify hot allocation stacks. Optimize only allocations that materially affect GC CPU, pause, or capacity.

Retention analysis asks why objects remain reachable. Class histograms taken at comparable lifecycle points show class growth. A heap dump enables dominator trees and paths to roots. Look for the first unexpected long-lived owner, not merely the largest byte array. A million value objects may be correctly owned by one accidental map.

Histogram growth is a lead. Class counts can rise because traffic or legitimate state rises. Heap-dump retained sizes depend on the snapshot and tool model. Confirm the suspected owner in code and with a reproducible lifecycle test.

Common heap-retention patterns

- unbounded maps, caches, deduplication sets, queues, and retry histories;
- entries with expiration timestamps but no active eviction;
- `ThreadLocal` values not removed on pooled threads;
- listeners, callbacks, observers, and scheduled tasks never deregistered;
- static registries retaining application or class-loader objects;
- keys whose equality mutation prevents normal removal;
- a small view or lambda retaining a large backing object graph;
- completed futures or request contexts retained by diagnostics or metrics labels;
- class loaders retained by threads, drivers, caches, or shutdown hooks;
- weak-reference designs whose values retain keys indirectly.

Weak references are not a general cache policy. Their reclamation follows reachability and collector timing, not a service-level capacity contract. Prefer explicit maximum size, weight, expiry, admission, and observability.

Non-heap and native growth

Each platform thread consumes native stack and VM metadata; thread explosion can exhaust address space or native allocation before heap fills. Direct buffers use native memory and can be retained by Java references even though their bytes are outside the heap. Metaspace grows with classes and class loaders. JIT code cache, JNI libraries, memory maps, TLS/native libraries, and allocator fragmentation also contribute to RSS.

Correlate thread counts, direct-buffer pools, class loading, HotSpot NMT categories when pre-enabled, OS mappings, and library metrics. NMT does not cover every native allocation and RSS includes shared/resident accounting that does not match NMT totals.

OutOfMemoryError is a family of symptoms

Messages can point toward Java heap, GC overhead, metaspace, direct-buffer reservation, native-thread creation, or array-size limits. Message wording and triggering policy are implementation-specific. Preserve the exact exception, JVM logs, process limit, heap/native metrics, thread count, and allocation context.

Do not assume catch-and-continue is safe. The process may lack memory to log, allocate a response, or restore invariants. Keep emergency handling minimal, avoid large allocations, shed traffic, and use supervisor restart/failover policy. `-XX:+HeapDumpOnOutOfMemoryError` is a HotSpot option that may aid analysis but needs secure disk capacity and may fail.

Collector-neutral GC diagnosis

Ask whether GC is the cause of impact or reacting to allocation/load. Correlate pause windows with request latency, GC CPU with application CPU, allocation with traffic, and post-cycle live set with heap capacity. A pause at the same time as latency is evidence; its duration and affected requests establish contribution.

Current HotSpot distributions commonly offer throughput-oriented, region-based, and low-pause collectors. Defaults and availability depend on JDK/vendor/platform. Collector selection changes trade-offs among throughput, pause, footprint, CPU, and headroom. It cannot collect reachable objects or make an unbounded queue bounded.

HotSpot note: G1 is a common default in mainstream 64-bit HotSpot server configurations for Java 17 and 21, but verify `VM.flags` or startup logs. ZGC, Generational ZGC modes, Shenandoah availability, region/humongous thresholds, and logging fields are release and vendor sensitive.

A safe incident playbook

- 1 Protect users: shed load, stop nonessential producers, fail over, or restart a replica according to policy.
- 2 Record timeline, release, traffic, limits, JVM/collector flags, exact OOM or pause symptoms.
- 3 Classify heap versus native using existing metrics.
- 4 Capture bounded JFR and GC logs if already available; take repeated histograms at comparable points.
- 5 Take a heap dump only after disk, pause, replica, permissions, and sensitive-data checks.
- 6 Analyze dominators and shortest/meaningful paths to roots; map the owner to code and lifecycle.
- 7 Reproduce with a bounded load test and assert cardinality or baseline recovery.
- 8 Fix ownership/capacity before tuning, then canary with rollback criteria.

Never invoke full GC solely to make a graph look clean without recording pre-GC evidence. `System.gc()` is a request whose treatment depends on JVM flags and collector; it is not a portable diagnostic barrier.

Tuning in the right order

First reduce unwanted retention and overload. Next set a realistic process/container budget that includes heap, metaspace, direct/native memory, thread stacks, code cache, agents, and safety margin. Then choose heap size and collector from measured objectives. Finally adjust a small number of documented flags, one hypothesis at a time.

`-Xms`, `-Xmx`, percentage-based container sizing, pause targets, and collector flags are HotSpot/vendor options. Large heaps provide burst headroom but increase footprint and some diagnostic costs. Small heaps collect more frequently. Setting minimum equal to maximum can improve predictability but commits a capacity choice and is not universally best.

TRACE IT | 41.6 Worked Java example

This event bus makes listener lifetime explicit:

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;

record Event(String name) {}

final class RequestContext {
    private final byte[] scratch = new byte[1_000_000];
    private int eventsSeen;

    void onEvent(Event event) {
        scratch[Math.floorMod(eventsSeen++, scratch.length)] =
            (byte) event.name().length();
    }
}

final class EventBus {
    interface Registration extends AutoCloseable {
        @Override void close();
    }

    private final List<Consumer<Event>> listeners = new ArrayList<>();

    Registration subscribe(Consumer<Event> listener) {
        java.util.Objects.requireNonNull(listener);
        listeners.add(listener);
        return new Registration() {
```

JAVA (continued 2/2)

```
        private boolean closed;

        @Override
        public void close() {
            if (!closed) {
                listeners.remove(listener);
                closed = true;
            }
        }
    };
}

void publish(Event event) {
    List.copyOf(listeners).forEach(listener -> listener.accept(event));
}

public final class ListenerLifecycleDemo {
    public static void main(String[] args) {
        EventBus bus = new EventBus();
        RequestContext context = new RequestContext();
        try (EventBus.Registration ignored = bus.subscribe(context::onEvent)) {
            bus.publish(new Event("request-complete"));
        }
    }
}
```

The example is single-threaded. A production bus needs a concurrency policy, exception isolation, and documented behavior when a listener removes itself during publication. The lifecycle idea remains: subscription returns an idempotent handle that owners must close.

TRACE IT | 41.7 Execution or memory walkthrough

During subscription, the method reference refers to `context`. The event bus stores that listener in its list:

TEXT

```
live bus -> listeners -> method-reference object -> RequestContext -> 1 MB byte[]
```

If the bus is application-scoped and registration is never removed, the request context remains reachable after the request ends. Repeating this pattern creates an approximately linear retained-set slope. The byte array is not the root cause; the unexpected listener ownership is.

The try-with-resources block closes the registration. `close` removes the same listener reference, breaking the path. After `main` no longer uses `context` and a future collection occurs, the context and byte array are eligible for reclamation. Eligibility does not promise immediate collection or memory return to the operating system.

`List.copyOf` in `publish` creates a snapshot of listener references so a callback cannot directly disrupt that iteration through the original list. It adds allocation and is not thread-safety. Alternative policies have different costs.

41.8 Complexity and performance

For n listeners, publication is $O(n)$ time and the snapshot is $O(n)$ temporary references. Registration append is amortized $O(1)$; removal from an array list is $O(n)$. If registration churn or listener count is high, a different representation may be justified, but it must preserve lifecycle and concurrency semantics.

Memory incident costs scale differently:

Action	Time/space tendency	Primary risk
allocation sample	sampled event cost	attribution precision
histogram	heap-object traversal/aggregation	pause and shallow-only view
heap dump	$O(\text{heap objects} + \text{data})$	pause, disk, secrets
offline dominator analysis	roughly graph-scale, tool-dependent	analyzer memory and time
increasing heap	more headroom	larger footprint, delayed failure
bounding a queue/cache	bounded steady storage	eviction/rejection semantics

An unbounded collection is $O(n)$ space where n may be lifetime traffic. A capacity limit turns it into $O(\text{capacity})$ but forces an explicit behavior at the boundary: block, reject, evict, spill, or degrade.

WATCH OUT | 41.9 Edge cases and common mistakes

- Calling any high memory usage a leak without comparing post-collection baselines.
- Looking only at heap while process RSS or native threads grow.
- Treating a large class histogram row as the retaining owner.
- Taking one heap dump after traffic changed and inferring a slope.
- Increasing heap repeatedly while leaving cache or queue cardinality unbounded.
- Forcing full GC during the incident before preserving allocation and occupancy evidence.
- Assuming weak keys solve retention when values reference keys or cleanup lags.
- Forgetting `ThreadLocal.remove` on pooled threads.
- Adding eviction without defining what happens to correctness on eviction.
- Tuning a pause target without measuring throughput, CPU, and headroom effects.
- Using old collector flags that are ignored, deprecated, or removed on the target JDK.
- Ignoring diagnostic artifact sensitivity and disk consumption.
- Expecting memory to return immediately to the OS after objects become unreachable.
- Catching `OutOfMemoryError` and continuing normal request processing.

41.10 Production engineering notes

Put bounds and ownership on every long-lived collection, executor queue, cache, registry, and per-tenant metric label. Emit current size, maximum, eviction/rejection, oldest age, and key cardinality. Expiration needs an active maintenance policy and tests using controllable time.

Close registrations, scheduled tasks, streams, class-loader-owned executors, and thread-local state at lifecycle boundaries. Use load tests that repeat create/use/destroy cycles and assert that post-cycle cardinality or heap baselines stabilize. A one-request unit test will not reveal lifetime leaks.

Maintain memory margin below the container limit. Account for heap plus non-heap components and traffic bursts. Monitor both JVM and cgroup/host views. Configure OOM handling, secure diagnostic paths, restart policy, and traffic failover before failure.

Keep GC logging/JFR settings and parsers compatible with each supported JDK. Compare collector changes with the same workload and objectives. Roll out canaries and retain rollback capacity; a lower pause percentile may cost enough CPU to reduce total service capacity.

TRY IT | 41.11 Interview questions and model answers

How can Java have a memory leak with garbage collection?

The collector removes unreachable objects. A leak occurs when objects remain reachable through an unintended long-lived path, such as an unbounded cache, listener registry, thread local, or class loader.

How do you distinguish high allocation from a leak?

High allocation shows rapid object creation and frequent collection, but post-collection live occupancy stabilizes. A retention leak tends to show a rising comparable post-collection baseline and growing owners across repeated histograms or dumps.

What would you inspect in a heap dump?

Dominators, retained size, growing classes, and paths from suspect objects to GC roots. I seek the first unexpected owner and verify its lifecycle in code rather than blaming the largest leaf array.

Why might RSS grow while Java heap is flat?

Thread stacks, direct buffers, metaspace/class loaders, code cache, JNI or native libraries, mappings, or allocator behavior. Correlate NMT where enabled with OS maps and subsystem metrics.

Should you increase `-Xmx` during an incident?

It can provide temporary headroom if process limits allow, but may delay an unbounded failure and increase footprint or diagnostic cost. Preserve evidence, define the budget, and fix ownership or capacity when that is the cause.

How do you tune GC?

Define pause, throughput, footprint, and CPU goals; fix retention and overload; size the whole process; establish a baseline; then change one supported setting or collector and test representative load before canarying.

TRY IT | 41.12 Exercises

- 1 Draw the root path for a pooled thread retaining a `ThreadLocal` request buffer. Show the correct `try/finally` cleanup.
- 2 Given stable heap, growing RSS, and increasing thread count, design an evidence and mitigation plan.
- 3 Design bounds for a retry queue, including rejection, durability, metrics, and tenant fairness.
- 4 Extend the event bus with thread safety without holding a lock while callbacks execute. State snapshot and removal semantics.
- 5 Compare two histograms five minutes apart and list reasons class growth might be legitimate.
- 6 Build a canary plan for changing collectors, including latency, throughput, CPU, footprint, and rollback thresholds.

RECAP | 41.13 Chapter summary

Managed-memory leaks are unwanted reachability. Diagnose them by distinguishing allocation rate, live-set growth, and non-heap process memory, then following retention paths to an unexpected owner. Stabilize service and preserve low-cost evidence before invasive dumps. Bounded collections and explicit lifecycle handles prevent more failures than collector flags. GC tuning begins only after workload, ownership, whole-process memory, and objectives are understood, and every result is specific to the tested JVM and deployment.

TRY IT | 41.14 Revision checklist

- ☐ I distinguish allocation pressure, intended live data, heap leak, native growth, and resource leak.
- ☐ I correlate post-collection occupancy, allocation, GC CPU/pause, RSS, threads, classes, and queues.
- ☐ I understand shallow size, retained size, dominators, roots, and comparable snapshots.
- ☐ I can execute a staged incident playbook with safe dump criteria.
- ☐ I can identify listener, thread-local, cache, queue, key, and class-loader retention patterns.
- ☐ I budget heap and native components below process/container limits.
- ☐ I tune one measured hypothesis at a time and validate with representative load.
- ☐ I treat collector behavior, flags, OOM messages, and counters as vendor/version specific.

JAVA FOUNDATIONS TO ADVANCED ENGINEERING

Part VII - Data Structures and Algorithms in Java

A focused part of the complete SDE-2 preparation guide

SDE-2 CORE CHAPTER

Chapter 42 - Complexity and the SDE-2 Problem-Solving Method

42.1 Learning objectives

By the end of this chapter, you should be able to:

- derive time and space complexity from executed work rather than loop appearance;
- distinguish worst-case, expected, amortized, and output-sensitive costs;
- state preconditions, postconditions, loop invariants, and progress measures;
- move from a correct baseline to an optimized algorithm systematically; and
- communicate a complete SDE-2 solution, including trade-offs, proof, tests, and production limits.

WHY IT WORKS | 42.2 Why this matters at SDE-2

Coding interviews at this level do not reward pattern recall alone. The interviewer is looking for controlled reasoning under incomplete requirements. Can you expose an ambiguity before it becomes a bug? Can you justify discarding part of the search space? Can you separate an algorithmic bound from Java implementation costs? Can you recover when the first approach is too slow?

The same skill appears in production. A service incident might require estimating whether a loop is linear in records, quadratic in tenant pairs, or bounded by output. A design review might hinge on whether memory is $O(n)$ per request or $O(n)$ for the whole process. Complexity vocabulary is useful only when tied to named inputs and an execution model.

Focused series path: Use Number Systems (DSA-02) when powers of two, repeated halving, logarithms, integer square roots, or overflow boundaries are not yet automatic. Use Time and Space Complexity (DSA-01), Java-SDE2-DSA-01-Time-and-Space-Complexity.pdf, for a compact complexity-first study path.

WHY IT WORKS | 42.3 First-principles model

An algorithm transforms inputs satisfying preconditions into an output satisfying postconditions. Correctness has two parts: partial correctness, meaning the promised result holds if the algorithm terminates, and termination, meaning it eventually stops for every valid input.

An invariant is a statement true at a defined program point before and after each iteration or recursive expansion. A progress measure moves toward a bound. Together they turn intuition into a proof. For example, binary search does not work because it "checks the middle." It works because the answer remains inside a maintained interval and each comparison safely removes a region that cannot contain it.

Complexity describes resource growth as input dimensions grow. It ignores many machine constants to compare scalability, but it does not erase them from engineering. First derive the asymptotic model. Then discuss allocations, cache locality, hashing behavior, boxing, and expected data sizes.

Specification boundary: Big-O is a mathematical bound, not a Java or JVM guarantee about elapsed time. Java specifies observable program behavior; it does not promise a particular instruction count, cache behavior, object size, or JIT optimization for a source-level operation.

42.4 Core terminology

- **Input dimension:** independent size variable such as n records, m queries, or V vertices and E edges.
- **Big-O:** asymptotic upper bound.
- **Big-Omega:** asymptotic lower bound.
- **Big-Theta:** matching upper and lower bound.
- **Worst case:** maximum cost among inputs of a given size.
- **Expected case:** average over a stated randomness or input distribution.
- **Amortized cost:** total cost of an operation sequence divided across that sequence, without assuming random inputs.
- **Auxiliary space:** extra working storage excluding input and usually output.
- **Output-sensitive:** cost includes the size of the result, such as $O(n + k)$ for k matches.
- **Invariant:** property preserved at a chosen control point.
- **Progress measure:** quantity that moves monotonically toward termination.
- **Baseline:** simplest clearly correct approach used to expose structure and constraints.

42.5 Detailed mechanics

A defensible complexity analysis

Start by naming dimensions. If one loop reads **users** and another reads **orders**, call the sizes **u** and **o**; do not collapse them into **n** unless the relationship is known. Count a meaningful dominant operation, form a sum, and simplify afterward.

Sequential phases add. Sorting and then scanning is $O(n \log n + n)$, which simplifies to $O(n \log n)$. Independent nested loops multiply: comparing every user with every role is $O(ur)$. Dependent loops require a sum. This loop is $O(n)$, not $O(n^2)$, because **right** only advances:

JAVA

```
for (int left = 0, right = 0; right < values.length; right++) {
    while (left <= right && windowIsValid(left, right)) {
        left++;
    }
}
```

Across the whole execution, each pointer moves at most n times. Aggregate pointer movement is often the right unit for windows, monotonic structures, and union-find analyses.

Halving a remaining range produces logarithmic depth: after k halvings, size is approximately $n / 2^k$, so reaching one requires k near $\log_2(n)$. A balanced divide-and-conquer algorithm that does linear work at each of $\log n$ levels is $O(n \log n)$. Recursive analysis must include both number of calls and work per call; $T(n) = 2T(n/2) + O(n)$ differs from $T(n) = T(n/2) + O(1)$.

Space analysis separates several categories:

Category	Question	Example
Input	What storage already exists?	Supplied <code>int[]</code>
Auxiliary	What new working state is needed?	Hash map of n entries
Call stack	How deep can recursion become?	$O(h)$ tree height
Output	How large must the result be?	$O(k)$ returned matches
Hidden library work	Does an API copy or box?	Sorting objects, <code>substring</code> , streams

An "in-place" recursive algorithm may still use $O(n)$ call-stack space. Returning all pairs cannot use less than $O(k)$ output storage if k pairs are materialized.

Worst, expected, and amortized claims

State the qualifier. Hash-table operations are commonly expected $O(1)$ under suitable hashing and load, not an unconditional mathematical constant for every possible collision pattern. Dynamic-array append is amortized $O(1)$: rare $O(n)$ resize copies are paid for across many cheap appends. Randomized quickselect is expected $O(n)$ with an appropriate pivot strategy but has a quadratic worst case.

Do not call amortized analysis "average case." Amortization holds across any operation sequence covered by its potential or aggregate proof; expected analysis depends on probability.

The SDE-2 control loop

Use the following repeatable sequence:

- 1 **Contract:** restate inputs, outputs, mutability, null policy, duplicates, ordering, ranges, and failure behavior.
- 2 **Examples:** create a normal case and an adversarial boundary. Calculate expected output manually.
- 3 **Baseline:** describe a correct direct solution and its cost. This establishes a fallback.
- 4 **Constraint pressure:** compare baseline cost with input bounds. Name the repeated or unnecessary work.
- 5 **Representation:** choose the state needed to avoid that work: hash map, pointer boundary, prefix state, heap, graph frontier, or DP table.
- 6 **Invariant:** say what every variable or data structure means at a stable point.
- 7 **Progress:** show why each iteration or recursive call approaches completion.
- 8 **Implementation:** use names that encode the proof and keep the main path visible.
- 9 **Trace:** execute the code on a boundary and a representative case.
- 10 **Close:** state time, auxiliary space, output space, mutation, and relevant alternatives.

This is not ceremony. Each step catches a different failure class. The contract catches wrong problems, the baseline catches unjustified optimization, the invariant catches logic errors, and the trace catches boundary defects.

Invariant proof template

For a loop, answer three questions:

- **Initialization:** why is the invariant true before the first iteration?
- **Maintenance:** assuming it is true, why does one iteration preserve it?
- **Termination:** when the loop stops, how does the invariant imply the postcondition?

Then give a variant or progress measure that is bounded and changes every continuing iteration. This proof style works for partitioning, two pointers, BFS layers, heap selection, and DP fill order.

Binary search as a reasoning template

Binary search applies to a monotone predicate, not only to equality in a sorted array. Define a search interval consistently. A half-open interval `[low, high)` is often easiest because its size is `high - low`, empty means `low == high`, and `high` can be `n` for insertion at the end.

For lower bound, maintain:

- every index before `low` has value less than target;
- every index at or after `high` has value at least target; and
- the first qualifying index remains in `[low, high)`.

Use `mid = low + (high - low) / 2` and update one boundary to exclude `mid`; otherwise the interval may not shrink.

TRACE IT | 42.6 Worked Java example

The following Java 21 program returns the first index whose value is at least the target. It returns `values.length` when no element qualifies.

JAVA

```
import java.util.Arrays;

public final class LowerBound {
    public static int lowerBound(int[] values, int target) {
        int low = 0;
        int high = values.length;

        while (low < high) {
            int mid = low + (high - low) / 2;
            if (values[mid] < target) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return low;
    }

    private static void requireSorted(int[] values) {
        for (int i = 1; i < values.length; i++) {
            if (values[i - 1] > values[i]) {
                throw new IllegalArgumentException("array must be sorted");
            }
        }
    }

    public static void main(String[] args) {
        int[] values = {1, 3, 3, 7, 9};
        requireSorted(values);
        for (int target : new int[] {0, 3, 4, 10}) {
            System.out.printf("target=%d index=%d\n",
                              target, lowerBound(values, target));
        }
        System.out.println(Arrays.toString(values));
    }
}
```

The sortedness check is useful at an external boundary but would change a single search from $O(\log n)$ to $O(n)$. In an interview, state whether sorted input is a trusted precondition instead of silently validating it.

TRACE IT | 42.7 Execution or memory walkthrough

Trace `lowerBound([1, 3, 3, 7, 9], 3)`:

Step	low	high	mid	values[mid]	Update
Start	0	5	-	-	Candidate interval [0, 5)
1	0	5	2	3	high = 2
2	0	2	1	3	high = 1
3	0	1	0	1	low = 1
End	1	1	-	-	Return 1

At each start, indices below `low` are proven too small, indices at or above `high` are proven qualifying, and the boundary lies between them. Each iteration strictly reduces `high - low`. At termination, no candidate interval remains; `low` is the unique partition point.

The method stores four primitive locals and uses no recursion, so auxiliary space is $O(1)$. It does not mutate the input. The JVM may place locals in registers after compilation, but that does not change the source-level space analysis.

42.8 Complexity and performance

Pattern	Time	Auxiliary space	Important qualifier
Full scan	$O(n)$	$O(1)$	May stop early, worst case remains n
Compare all pairs	$O(n^2)$	$O(1)$	Output may itself be quadratic
Sort then scan	$O(n \log n)$	Depends on sort	Mutation and stability matter
Binary search	$O(\log n)$	$O(1)$ iterative	Requires monotone search space
Hash lookup sequence	Expected $O(n)$	$O(n)$	Hash quality and boxing matter
Balanced recursion	Often $O(\log n)$ depth	$O(\log n)$ stack	Skew can make depth $O(n)$
Dynamic array append	Amortized $O(1)$	Capacity overhead	Individual resize is $O(n)$

`lowerBound` is $O(\log n)$ time and $O(1)$ auxiliary space when sortedness is a precondition. Calling `requireSorted` first makes the combined operation $O(n)$. For one query on untrusted data, a linear scan may be simpler. For many queries, validating once and reusing the ordered representation can be worthwhile.

Constants re-enter after the asymptotic choice. A linear scan over a small contiguous primitive array can beat a more elaborate structure. Benchmark representative workloads rather than treating the notation as a stopwatch.

HotSpot note: HotSpot may inline methods, eliminate range checks, vectorize loops, and compile hot branches using profile data. These optimizations can change constants but not the algorithm's asymptotic operation count or correctness proof.

WATCH OUT | 42.9 Edge cases and common mistakes

- Naming only `n` when the algorithm depends independently on two inputs.
- Multiplying every pair of nested loops without checking total pointer movement.
- Dropping output space or recursive stack space from the analysis.
- Claiming hash operations are worst-case $O(1)$ without qualification.
- Confusing expected with amortized complexity.
- Optimizing before clarifying whether mutation, sorting, or extra memory is allowed.
- Choosing a pattern from keywords without proving its invariant.
- Mixing closed `[low, high]` and half-open `[low, high)` binary-search rules.
- Computing `(low + high) / 2` when indexes or search values could overflow.
- Updating `low = mid` in a loop where `mid` can equal `low`, causing nontermination.
- Giving average-case complexity when the interviewer asked for a service limit or worst-case bound.
- Hiding assumptions about integer overflow, Unicode, duplicates, or invalid input.

42.10 Production engineering notes

Production inputs have distributions, limits, and adversaries. Record all three. Hash-flooding, enormous result sets, recursion depth, and a caller-controlled sort key can turn a textbook choice into a reliability problem. Put hard bounds around memory and output, use `long` for counts that may exceed `int`, and fail before partial mutation when possible.

Complexity is end-to-end. Reading `n` rows through an ORM may perform `n` extra queries; an $O(n)$ Java loop can sit on top of $O(n)$ network round trips. A theoretically better algorithm can lose if it allocates millions of wrappers or destroys locality. Instrument the actual bottleneck and preserve a clear correctness model while optimizing.

During review, demand qualifiers: `n` means what, hash time under what assumption, recursion depth under which shape, and whether output is included. This precision transfers directly from interviews to capacity planning.

TRY IT | 42.11 Interview questions and model answers**Are two nested loops always $O(n^2)$?**

No. Derive total executions. In a sliding window, two nested-looking pointer loops may advance each pointer at most n times, yielding $O(n)$. Independent full-range loops do produce a product.

What is the difference between expected and amortized $O(1)$?

Expected $O(1)$ relies on a probability model, such as suitable hash distribution. Amortized $O(1)$ bounds the average cost across an operation sequence, such as dynamic-array appends including occasional resizes, without assuming random inputs.

How do you prove binary search correct?

Define a monotone predicate and an interval invariant that retains the boundary or answer. Show initialization, show each comparison discards only impossible values, show the interval shrinks, and use the termination condition to derive the returned boundary.

Why start with a brute-force baseline?

It validates understanding, gives a correctness oracle for small tests, identifies repeated work, and provides a fallback. The optimized solution should be motivated by constraints, not pattern guessing.

What space should be reported?

State auxiliary working space, recursion stack, and output space separately. Also mention whether input is mutated. This avoids calling a recursive or output-heavy method "constant space."

When can $O(n)$ be worse than $O(n \log n)$?

Asymptotic notation describes growth, not every concrete runtime. The $O(n)$ approach may have much larger constants, random memory access, boxing, or remote operations. For sufficiently large n its growth is better, but production decisions need representative measurements.

TRY IT | 42.12 Exercises

- 1 Analyze a loop with two pointers that each only move forward. Write the aggregate sum that proves $O(n)$.
- 2 Implement upper bound, returning the first index whose value is greater than a target, using a half-open interval.
- 3 Analyze $T(n) = 2T(n/2) + n$ with a recursion tree and account for stack depth separately.
- 4 Give a potential or aggregate argument for amortized dynamic-array append.
- 5 Take a quadratic duplicate-detection baseline and derive hashing and sorting alternatives, including mutation and worst-case qualifiers.
- 6 Write preconditions, postconditions, an invariant, and a progress measure for stable in-place compaction.
- 7 Estimate memory for one million boxed map entries, then explain why asymptotic $O(n)$ alone is insufficient for a capacity decision.

RECAP | 42.13 Chapter summary

Strong problem solving begins with a contract and ends with a proof, a cost model, and tests. Name independent input dimensions, count total work, qualify expected and amortized claims, and separate auxiliary, stack, and output space. Use a correct baseline to expose repeated work. Then choose state, state an invariant, prove progress, and trace boundaries. Binary search is the canonical example: its power comes from a monotone predicate and a shrinking candidate interval, not from memorized syntax.

TRY IT | 42.14 Revision checklist

- ☐ I name every independent input dimension before analyzing cost.
- ☐ I sum dependent work and multiply only independent repeated work.
- ☐ I distinguish O, Omega, Theta, worst, expected, and amortized bounds.
- ☐ I report auxiliary, call-stack, output, and mutation costs separately.
- ☐ I clarify the contract and produce a correct baseline before optimizing.
- ☐ I state invariants with initialization, maintenance, and termination.
- ☐ I identify a bounded progress measure.
- ☐ I can implement lower and upper bound without mixing interval conventions.
- ☐ I discuss constants and production constraints only after the asymptotic model is sound.

SDE-2 CORE CHAPTER

Chapter 43 - Arrays, Strings, Hashing, Two Pointers, Sliding Windows, and Prefix Sums

43.1 Learning objectives

By the end of this chapter, you should be able to:

- select a sequence pattern from constraints and eliminated work;
- maintain correct two-pointer and sliding-window invariants;
- transform subarray sums into prefix-state lookups;
- use hashing with explicit equality, range, and complexity assumptions;
- apply binary search to sorted data or a monotone answer predicate; and
- implement Java 21 solutions that handle overflow, duplicates, empty input, and Unicode boundaries deliberately.

WHY IT WORKS | 43.2 Why this matters at SDE-2

Arrays and strings dominate coding screens because they expose reasoning without requiring much setup. The same small set of ideas solves a wide range of tasks: deduplication, rate-window analysis, contiguous aggregates, text scanning, sorted pair search, and event correlation.

At SDE-2, naming "sliding window" is not sufficient. You must explain why shrinking restores validity, why no candidate is skipped, whether negative values break monotonicity, and whether a map stores the earliest index, latest index, frequency, or count of prefix states. Those details are the algorithm.

Focused prerequisites: Number Systems (DSA-02) establishes numeric boundaries, prefix sums under modulo, and safe index arithmetic. Continue through DSA-05 to DSA-08 for loops and indices, arrays, strings, and hashing. Use the series index PDF for the ordered filenames.

WHY IT WORKS | 43.3 First-principles model

An array offers indexed, fixed-length storage. A Java `String` is an immutable sequence of UTF-16 code units. Both make a sequence available in order. Most interview patterns compress information about a processed prefix so the algorithm does not rescan it.

Two pointers replace repeated pair or range exploration when order lets one pointer movement eliminate a set of candidates. A sliding window maintains state for a contiguous interval. Prefix sums answer a range query by

subtracting two summaries. Hashing turns a previously seen value or state into an expected constant-time lookup.

The common proof shape is a partition:

- a processed region whose answer-relevant information is summarized;
- a current boundary or window with a precisely stated meaning; and
- an unprocessed region that will be considered exactly once or safely skipped.

Specification boundary: Java guarantees array indexing and `String` UTF-16 semantics. It does not guarantee worst-case $O(1)$ for every `HashMap` operation, a particular hash-table layout, or one Unicode code point per `char`.

43.4 Core terminology

- **Subarray/substring:** contiguous interval.
- **Subsequence:** elements retain relative order but need not be contiguous.
- **Frequency map:** key to occurrence count.
- **Index map:** key to an earliest, latest, or otherwise selected position.
- **Opposing pointers:** boundaries move inward, commonly on sorted data.
- **Same-direction pointers:** read/write or left/right positions move forward.
- **Fixed window:** interval length is constant.
- **Variable window:** boundary moves to preserve a validity predicate.
- **Prefix sum:** sum of elements strictly before an index in the common `prefix[0] = 0` convention.
- **Difference array:** boundary updates later reconstructed by a prefix sum.
- **Monotone predicate:** once true (or false), remains so across an ordered search domain.

43.5 Detailed mechanics

Pattern-selection map

Signal	Candidate technique	Required proof
Need membership, duplicates, complement, or frequency	Hash set/map	Stored state answers future query
Sorted pairs or symmetric comparison	Opposing pointers	Pointer move discards impossible pairs
In-place filtering or compaction	Read/write pointers	Prefix contains exactly accepted elements
Best contiguous range with monotone validity	Variable window	Shrink restores validity and no start is revisited
Aggregate for many ranges	Prefix sum	Range equals difference of prefix states
Count subarrays with exact aggregate	Prefix plus hash frequencies	Earlier required prefix is counted
Sorted boundary or monotone feasibility	Binary search	Answer remains in candidate interval

Do not select only from a noun in the prompt. "Subarray" suggests a window, but exact sums with negative values often require prefix hashing because adding a value does not move the sum monotonically.

Arrays and strings as state spaces

Indexes are half-open boundaries whenever possible. Interval `[left, right)` has length `right - left`, is empty when equal, and composes cleanly. Java APIs vary, so translate carefully: `Arrays.copyOfRange` is half-open, while some problem statements use inclusive endpoints.

For read/write compaction, maintain: `values[0..write)` contains exactly the accepted elements from `values[0..read)` in stable order. When the read value qualifies, write it and increment `write`. Every input is examined once, giving $O(n)$ time and $O(1)$ auxiliary space.

String indexing needs a declared unit. A `char` solution counts UTF-16 code units and may split supplementary code points. If the task means Unicode code points, convert with `text.codePoints().toArray()` or advance using `codePointAt` and `Character.charCount`. If it means user-perceived graphemes, code-point processing is still insufficient and a Unicode boundary implementation is needed.

Hashing patterns

Hashing stores the minimum past state necessary for future decisions:

- **set:** has this key appeared?
- **frequency map:** how many times has it appeared?
- **index map:** which position should be used?
- **grouping map:** which values share a canonical signature?
- **prefix-frequency map:** how many earlier prefixes enable the current range?

For two-sum on unsorted data, before inserting `x`, look for `target - x`. This prevents pairing an element with itself unless an earlier equal value exists. If original indices matter, sorting loses them unless pairs of value and index are retained.

Map update order and duplicate policy matter. Longest-substring algorithms often need the latest occurrence so `left` jumps forward. Longest-subarray prefix algorithms often need the earliest index to maximize length. Counting algorithms need every occurrence as a frequency, not one index.

Use `long` for sums and complements when `n` and element magnitudes can overflow `int`. Boxed `Long` and `Integer` keys allocate or reuse wrappers according to implementation; the asymptotic map model remains expected $O(1)$, but memory can be significant.

Opposing and same-direction pointers

For a sorted two-sum search, compare `values[left] + values[right]` with target. If the sum is too small, every pair using the current left with an index no larger than right is also too small, so incrementing left is safe. The symmetric argument justifies decrementing right when the sum is too large.

This elimination proof depends on sorted order. Without it, moving a pointer has no such implication. Sorting first costs $O(n \log n)$, may mutate input, and complicates original-index output. A hash map offers expected $O(n)$ time and $O(n)$ space instead.

Same-direction fast/slow pointers appear in compaction and linked-list cycle detection. In arrays, name them `read` and `write` when that reflects meaning; semantic names make invariants easier to state.

Sliding windows

A fixed-size window updates incrementally: add the entering element, remove the leaving element, and evaluate when length reaches `k`. This replaces $O(k)$ work for each start with $O(1)$ state updates, producing $O(n)$ total time.

A variable window uses a validity condition:

TEXT

```
for each right:
    include right in state
    while window is invalid:
        remove left from state
        advance left
    evaluate the valid window
```

The invariant is that state describes exactly `[left, right]` and the window is valid after shrinking. Whether evaluating the current window finds an optimum depends on the problem. For "longest window with at most `k` distinct values," after restoring validity, the current left is the earliest valid start for that right, so it gives the longest valid window ending there.

Variable sum windows usually require nonnegative elements. With negatives, extending can reduce a sum and shrinking can increase it, so the needed monotonicity disappears. Prefix hashing, a monotonic deque over prefix sums, or another technique may apply instead.

Prefix sums and difference arrays

Define `prefix[i]` as the sum of the first `i` elements:

TEXT

```
prefix[0] = 0
prefix[i + 1] = prefix[i] + values[i]
sum(left, rightExclusive) = prefix[rightExclusive] - prefix[left]
```

The leading zero removes special cases for ranges beginning at index zero. Building costs $O(n)$; each range-sum query is $O(1)$.

For an exact target subarray ending at current index with prefix `p`, an earlier prefix must equal `p - target`. Store counts of earlier prefixes and seed frequency zero with one, representing the empty prefix. Query before incrementing the current prefix frequency if zero-length ranges are not allowed.

A difference array reverses the idea. To add delta to inclusive range `[left, right]`, apply `difference[left] += delta` and, if it exists, `difference[right + 1] -= delta`. One final prefix pass materializes all values. This is ideal for many offline range updates, not arbitrary online queries.

Binary search in sequence problems

Use binary search for a sorted boundary, rotated-array partition with careful duplicate handling, or a monotone answer such as minimum feasible capacity. Do not binary-search a condition that can flip repeatedly. State whether the interval is closed or half-open and whether you seek any match, first true, last false, lower bound, or upper bound.

TRACE IT | 43.6 Worked Java example

This Java 21 class provides three complementary templates: sorted two pointers, a variable character window, and prefix sum plus hashing.

JAVA (continued 1/2)

```
import java.util.HashMap;
import java.util.Map;

public final class SequencePatterns {
    public static int[] twoSumSorted(int[] values, long target) {
        int left = 0;
        int right = values.length - 1;
        while (left < right) {
            long sum = (long) values[left] + values[right];
            if (sum == target) return new int[] {left, right};
            if (sum < target) left++;
            else right--;
        }
        return new int[] {-1, -1};
    }

    public static int longestAtMostKDistinct(String text, int k) {
        if (k <= 0 || text.isEmpty()) return 0;
        Map<Character, Integer> frequencies = new HashMap<>();
        int left = 0;
        int best = 0;

        for (int right = 0; right < text.length(); right++) {
            char added = text.charAt(right);
            frequencies.merge(added, 1, Integer::sum);

            while (frequencies.size() > k) {
                char removed = text.charAt(left++);
                int remaining = frequencies.get(removed) - 1;
                if (remaining == 0) frequencies.remove(removed);
            }
        }
    }
}
```

JAVA (continued 2/2)

```

        else frequencies.put(removed, remaining);
    }
    best = Math.max(best, right - left + 1);
}
return best;
}

public static long countSubarraysWithSum(int[] values, long target) {
    Map<Long, Integer> prefixFrequency = new HashMap<>();
    prefixFrequency.put(0L, 1);
    long prefix = 0;
    long count = 0;

    for (int value : values) {
        prefix += value;
        count += prefixFrequency.getOrDefault(prefix - target, 0);
        prefixFrequency.merge(prefix, 1, Integer::sum);
    }
    return count;
}

public static void main(String[] args) {
    int[] pair = twoSumSorted(new int[] {1, 2, 4, 7, 11}, 9);
    System.out.println(pair[0] + "," + pair[1]); // 1,3
    System.out.println(longestAtMostKDistinct("eceba", 2)); // 3
    System.out.println(countSubarraysWithSum(
        new int[] {1, 2, 1, 2}, 3)); // 3
}
}

```

The string method intentionally defines length in UTF-16 code units. For a code-point contract, operate on an `int[]` of code points and use `Map<Integer, Integer>`.

TRACE IT | 43.7 Execution or memory walkthrough

For `longestAtMostKDistinct("eceba", 2)`:

<code>right</code>	Added	Window after shrink	Frequencies	Best
0	e	e	e:1	1
1	c	ec	e:1,c:1	2
2	e	ece	e:2,c:1	3
3	b	eb	e:1,b:1	3
4	a	ba	b:1,a:1	3

At right 3, adding b creates three distinct characters. Shrinking removes e at index 0 but e remains, then removes c at index 1, restoring two distinct characters with left at 2. Each code unit enters once and leaves at most once.

For prefix counting on `[1, 2, 1, 2]` with target 3, prefix values are 1, 3, 4, 6. Required earlier prefixes are -2, 0, 1, 3. They occur zero, one, one, and one times, so the count becomes three: ranges `[0, 2)`, `[1, 3)`, and `[2, 4)`. Seeding prefix zero is what counts the first range.

For sorted two-sum target 9, `(1, 11)` is too large so right moves; `(1, 7)` is too small so left moves; `(2, 7)` matches. Every move eliminates all pairs involving the discarded boundary under sorted order.

43.8 Complexity and performance

Technique	Time	Auxiliary space	Preconditions or caveats
Linear scan	$O(n)$	$O(1)$	Constant summary state
Hash membership/frequency	Expected $O(n)$	$O(n)$	Equality and hash quality
Sorted opposing pointers	$O(n)$	$O(1)$	Input already sorted
Sort plus two pointers	$O(n \log n)$	Sort-dependent	Mutation and original indices
Fixed/variable window	$O(n)$	$O(k)$ or alphabet size	Incremental state; validity monotonicity
Prefix array plus q queries	$O(n + q)$	$O(n)$	Static aggregate
Prefix-frequency counting	Expected $O(n)$	$O(n)$	Use frequency, not one index
Binary search	$O(\log n)$	$O(1)$	Sorted or monotone domain

The worked two-pointer method is $O(n)$ time and $O(1)$ auxiliary space. The window is expected $O(n)$ time because each pointer moves at most n and map operations are expected $O(1)$; space is $O(\min(n, \text{alphabet diversity}))$. Prefix counting is expected $O(n)$ time and $O(n)$ space.

Primitive arrays are compact and locality-friendly. Maps add nodes or table storage, wrapper objects, hashing, and indirection. If the key domain is small and dense, an `int[]` frequency table is usually simpler and faster than `HashMap<Character, Integer>`.

HotSpot note: HotSpot can eliminate some array bounds checks and optimize tight primitive loops. It cannot make an asymptotically quadratic rescan linear, and it does not turn boxed hash-map state into a primitive array contract.

WATCH OUT | 43.9 Edge cases and common mistakes

- Empty input, one element, no solution, and all-equal data.
- Confusing a contiguous subarray with a subsequence.
- Overflowing an `int` sum before assigning it to `long`; cast an operand first.
- Sorting when input mutation or original indices are forbidden.
- Using `left <= right` when a pair must use two distinct elements.
- Moving a pointer on unsorted data without an elimination proof.
- Forgetting to decrement or remove a leaving window frequency.
- Updating the best window before restoring validity.
- Applying a sum window to negative values without monotonicity.
- Forgetting `prefixFrequency.put(0L, 1)`.
- Storing one prefix index when the question asks for a count.
- Using latest prefix index when longest range requires earliest.
- Mixing inclusive range endpoints with half-open prefix formulas.
- Treating `String.length()` as code-point or grapheme count.
- Claiming deterministic $O(1)$ map operations.
- Binary-searching a predicate that is not monotone or failing to shrink the interval.

43.10 Production engineering notes

Define the unit of text and the maximum input size at the API boundary. For ASCII protocol tokens, a fixed frequency array is appropriate. For human names, code points, normalization, locale, and grapheme behavior may matter. Do not silently apply interview-style `char` assumptions to security identifiers.

For streaming data, an unbounded prefix map or window can become a memory leak. Add time or count retention, eviction, and late-event policy. A sliding time window over out-of-order events is not the same as a pointer window over a sorted array.

Avoid hidden mutation. If sorting is chosen, copy when ownership requires it and account for $O(n)$ space. For large counts, use `long` and define what happens if even `long` overflows. Hash keys influenced by untrusted input need robust platform defaults and resource limits.

In production code, favor standard collection operations and readable loops. In interviews, explain when a dense array can replace a map, when preprocessing is amortized over many queries, and when the output size dominates any possible algorithm.

TRY IT | 43.11 Interview questions and model answers**When is a variable sliding window valid?**

When state can be updated as boundaries move and validity changes monotonically enough that shrinking safely restores it. For nonnegative sums, increasing right cannot lower the sum. Negative values break that property, so exact-sum problems often need prefix hashing instead.

Why can a loop containing a while loop still be $O(n)$?

Analyze pointer movement over the whole run. If right advances n times and left only advances, at most n times, the combined work is $O(2n)$, which is $O(n)$.

Hashing or sorting for two-sum?

Hashing offers expected $O(n)$ time and $O(n)$ space on unsorted data and preserves original indices. Sorting plus two pointers is $O(n \log n)$, may use less auxiliary space, and changes index handling. If data is already sorted, pointers give $O(n)$ time and $O(1)$ space.

Why use a prefix-frequency map for subarray counts?

For current prefix p , every earlier prefix p minus target defines a target-sum subarray. Multiple equal earlier prefixes produce different starts, so a frequency is required. Seed zero for ranges starting at index zero.

What does a window invariant contain?

It states the exact interval represented by the auxiliary state, the validity condition after shrinking, and why the retained boundary is optimal or sufficient for the current right endpoint.

How do duplicates affect binary search?

An arbitrary equality search can return any duplicate. If the requirement is first or last occurrence, search for a boundary using lower or upper bound and keep the half-open interval invariant explicit.

TRY IT | 43.12 Exercises

- 1 Implement stable removal of a chosen value in place and state the read/write invariant.
- 2 Find the longest substring without repeated Unicode code points, not `char` values.
- 3 Count subarrays whose sum is divisible by k using normalized prefix remainders; handle negative values.
- 4 Implement minimum-length subarray with sum at least target for positive values, then explain why negatives break it.
- 5 Apply a difference array to range increments and use `long` for reconstructed values.
- 6 Return original indices for two-sum using both hashing and sort-plus-pairs approaches.
- 7 Implement binary search on the minimum feasible capacity for partitioning workloads into at most d days.
- 8 Create property tests comparing optimized subarray counting against an $O(n^2)$ baseline on small random arrays.

RECAP | 43.13 Chapter summary

Sequence algorithms avoid rescanning by compressing processed state. Hash maps answer questions about earlier values, two pointers exploit ordering to eliminate candidates, windows maintain an incrementally updated interval, and prefix sums convert ranges into differences. Binary search applies when a boundary predicate is monotone. Every technique depends on a specific invariant and precondition; sortedness, nonnegative values, duplicate policy, overflow, and text unit must be explicit.

TRY IT | 43.14 Revision checklist

- ☐ I distinguish subarrays, substrings, and subsequences.
- ☐ I choose map contents based on membership, index, or frequency needs.
- ☐ I can prove every two-pointer movement eliminates only impossible candidates.
- ☐ I state the exact window interval and validity invariant.
- ☐ I know why negative values can invalidate sum windows.
- ☐ I use the `prefix[0] = 0` convention and seed prefix frequency correctly.
- ☐ I use `long` before arithmetic can overflow.
- ☐ I distinguish UTF-16 units, code points, and graphemes.
- ☐ I apply binary search only to sorted data or a monotone predicate.
- ☐ I can compare hashing, sorting, and dense-array state in Java.

SDE-2 CORE CHAPTER

Chapter 44 - Linked Lists, Stacks, Queues, and Monotonic Structures

44.1 Learning objectives

By the end of this chapter, you should be able to:

- mutate singly linked lists without losing nodes or creating accidental cycles;
- use sentinels, fast/slow pointers, and merge as reusable list techniques;
- choose stack, queue, or deque semantics from dependency order;
- derive monotonic stack and deque algorithms from domination rules; and
- prove linear time using aggregate pushes, pops, and pointer movement.

WHY IT WORKS | 44.2 Why this matters at SDE-2

These structures test whether a candidate can control mutable state. Linked-list bugs are often one incorrect assignment; monotonic algorithms require explaining why removed candidates can never matter again. Stack and queue choices reveal whether traversal order is understood rather than memorized.

Production Java rarely uses a hand-written linked list for ordinary storage, but the pointer discipline transfers to trees, caches, lock-free structures, and intrusive queues. Deques power schedulers, buffers, parsers, graph traversals, and rolling analytics. SDE-2 answers should connect the abstract behavior to memory locality, capacity, concurrency, and ownership.

WHY IT WORKS | 44.3 First-principles model

A singly linked node stores a value and a reference to the next node. The list is reachable from a head reference; changing one link can make an entire suffix unreachable or can introduce a cycle. Safe mutation therefore preserves a reference to future work before redirecting the current link.

A stack exposes the most recently added item first. A queue exposes the earliest added item first. A deque supports both ends. These are ordering contracts, not particular storage layouts.

A monotonic structure keeps candidates ordered by value while positions arrive in sequence. When a new candidate dominates an older one for every possible future query, the older one is removed permanently. This deletion rule is what makes the structure useful and what gives an amortized linear bound.

Specification boundary: Java references are not source-level memory pointers, and Java does not expose pointer arithmetic. `Deque` defines endpoint behavior; a particular implementation's internal array or node layout is not part of that contract. `ArrayDeque` rejects null elements.

44.4 Core terminology

- **Head/tail:** first and last reachable list nodes.
- **Successor:** node referenced by `next`.
- **Sentinel/dummy:** extra node that unifies boundary mutations.
- **Fast/slow pointers:** references advancing at different rates or from offset positions.
- **LIFO:** last in, first out stack order.
- **FIFO:** first in, first out queue order.
- **Frontier:** discovered work waiting for processing.
- **Monotonic stack:** stack whose relevant values are maintained increasing or decreasing.
- **Monotonic deque:** deque retaining ordered candidates, often with expiry at the front.
- **Domination:** proof that one candidate can never beat another in a future answer.
- **Amortized analysis:** bounds total operations across a sequence rather than one step.

44.5 Detailed mechanics

Linked-list mutation discipline

For iterative reversal, before changing `current.next`:

```

1  save next = current.next;
2  redirect current.next = previous;
3  advance previous = current; and
4  advance current = next.
```

The loop invariant is:

- `previous` heads a correctly reversed prefix;
- `current` heads the original-order unprocessed suffix; and
- every original node is reachable from exactly one of those two roots.

Initialization uses an empty prefix (`previous = null`) and the full suffix. One step moves exactly one node from suffix to prefix. At termination the suffix is empty and `previous` heads the complete reversal.

Assignment order matters. Writing `current.next = previous` before saving the old successor loses access to the remaining suffix. Drawing three boxes is not childish; it externalizes an aliasing problem that is difficult to repair after code is written.

Sentinels and boundary normalization

Insertion or deletion at the head often needs a special branch because there is no predecessor. A sentinel node placed before the real head makes every real node have a predecessor. For "remove the *n*th node from the end," a sentinel also permits removal of the original head using the same `previous.next = previous.next.next` operation.

The sentinel is algorithmic state and is not returned as data. Its value is irrelevant. In generic production code, prefer a dedicated node shape or explicit absence over inventing a value that might collide with valid input.

Fast and slow pointers

Fast/slow pointers compress distance information:

- move fast two and slow one to detect a cycle;
- after a meeting, reset one pointer to head and move both one to find cycle entry;
- start fast *k* nodes ahead to locate the *k*th node from the end;
- move fast two and slow one to find a midpoint.

Floyd cycle detection works because, inside a cycle, the relative distance changes by one modulo the cycle length each iteration, so the pointers must meet. It uses $O(1)$ space. A visited-node set is easier to generalize and can report more information but costs $O(n)$ space.

Null checks must match the dereference: before `fast.next.next`, require both `fast != null` and `fast.next != null`. Decide which middle to return for even length; initial positions and loop condition control the answer.

Merge as a primitive

Merging two sorted lists uses a sentinel tail. At each step, attach the smaller current node, advance that source, and advance the output tail. The invariant says the output prefix is sorted and contains exactly the consumed input nodes. Once one input ends, the other suffix is already sorted and can be attached.

This $O(n + m)$ primitive appears in merge sort, *k*-way merging with a heap, and interval or stream processing. Clarify whether nodes may be relinked or a new list is required. Relinking is $O(1)$ auxiliary space but mutates the inputs and changes ownership.

Stack semantics

Use a stack when unresolved work must finish in reverse order:

- matching nested delimiters;
- evaluating expressions;
- iterative depth-first traversal;
- simulating recursive frames; and
- finding next/previous greater or smaller elements.

In Java, prefer `ArrayDeque` through the `Deque` interface over legacy `Stack`. Use `push`, `pop`, and `peek` consistently for stack intent. Decide whether malformed input returns a result or throws; `pop` and `removeFirst` throw on emptiness, while `poll` forms return null, which `ArrayDeque` can use as an absence signal because it disallows null elements.

Delimiter matching needs both kind and order. A count of openings is insufficient for `( [ ) ]`; the most recent unmatched opening must match the current closing delimiter.

Queue and deque semantics

Use a queue when work must be processed in discovery or arrival order. BFS relies on FIFO order to process all states at distance d before distance $d + 1$ in an unweighted graph. Mark a state visited when enqueueing, not when dequeuing, to avoid repeated queue entries.

Level-order processing can store `(node, distance)` pairs, use a fixed `levelSize = queue.size()` before a level loop, or place a sentinel marker. Size-based boundaries avoid null markers and make the invariant explicit.

A circular buffer represents a bounded FIFO with an array, head index, tail index, and size or one deliberately unused slot to distinguish full from empty. Every representation needs one unambiguous convention. Bounded capacity is a production feature: it establishes backpressure instead of allowing memory to grow without limit.

Monotonic stacks

For next greater value to the right, scan from right to left. Maintain a decreasing stack of values that could answer a future position. Before answering current x , pop every value less than or equal to x ; those values are both closer to x than earlier positions and no larger than x , so x dominates them for all future elements to the left. The remaining top, if present, is the nearest greater value. Then push x .

Many variants need indexes instead of values:

- distances to next greater temperature;
- histogram rectangle widths;
- previous smaller boundaries; and
- span calculations.

Choose strict versus non-strict popping from duplicate semantics. $\leq$ and $<$ produce different boundaries. State the desired relation before coding.

Monotonic deques

For maximum in every size- k window, keep indexes in decreasing value order. For each index i :

- 1 remove front indexes $\leq i - k$ because they expired;
- 2 remove back indexes whose values are $\leq \text{values}[i]$ because the new value is newer and at least as good;
- 3 append i ; and
- 4 once a full window exists, the front is its maximum.

The deque maintains two dimensions: indexes increase from front to back, while values decrease. Expiry happens only at the front; domination happens at the back.

Although one iteration can pop many entries, each index is appended once and removed at most once from each relevant end. Total deque work is $O(n)$, an aggregate proof that is more informative than inspecting the nested while loops.

TRACE IT | 44.6 Worked Java example

This Java 21 class contains canonical linked-list and monotonic templates.

JAVA

```
import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Deque;

public final class LinearStructures {
    static final class Node {
        final int value;
        Node next;

        Node(int value) {
            this.value = value;
        }
    }

    static Node reverse(Node head) {
        Node previous = null;
        Node current = head;
        while (current != null) {
            Node next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        return previous;
    }

    static boolean hasCycle(Node head) {
        Node slow = head;
        Node fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

The monotonic-stack and monotonic-deque methods continue the same `LinearStructures` class:

JAVA (continued 1/2)

```

static int[] nextGreater(int[] values) {
    int[] answer = new int[values.length];
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = values.length - 1; i >= 0; i--) {
        while (!stack.isEmpty() && stack.peek() <= values[i]) {
            stack.pop();
        }
        answer[i] = stack.isEmpty() ? -1 : stack.peek();
        stack.push(values[i]);
    }
    return answer;
}

static int[] slidingWindowMaximum(int[] values, int k) {
    if (k < 1 || k > values.length) {
        throw new IllegalArgumentException("invalid window size");
    }
    int[] answer = new int[values.length - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();

    for (int i = 0; i < values.length; i++) {
        while (!deque.isEmpty() && deque.peekFirst() <= i - k) {
            deque.removeFirst();

```

JAVA (continued 2/2)

```

        }
        while (!deque.isEmpty()
            && values[deque.peekLast()] <= values[i]) {
            deque.removeLast();
        }
        deque.addLast(i);
        if (i >= k - 1) answer[i - k + 1] = values[deque.peekFirst()];
    }
    return answer;
}

public static void main(String[] args) {
    Node one = new Node(1);
    one.next = new Node(2);
    one.next.next = new Node(3);
    Node reversed = reverse(one);
    System.out.println(reversed.value + " " + reversed.next.value); // 3 2
    System.out.println(hasCycle(reversed)); // false
    System.out.println(Arrays.toString(nextGreater(
        new int[] {2, 1, 2, 4, 3}))); // [4,2,4,-1,-1]
    System.out.println(Arrays.toString(slidingWindowMaximum(
        new int[] {1, 3, -1, -3, 5, 3, 6, 7}, 3))); // [3,3,5,5,6,7]
}
}

```

The list methods compare node references by identity. Defining logical node equality is unnecessary and could make cycle logic incorrect if equal values appeared in distinct nodes.

TRACE IT | 44.7 Execution or memory walkthrough

For reversal of 1 -> 2 -> 3, state evolves as follows:

Step	previous reversed prefix	current suffix
Start	null	1 -> 2 -> 3
Move 1	1 -> null	2 -> 3
Move 2	2 -> 1	3
Move 3	3 -> 2 -> 1	null

The saved `next` reference preserves the suffix before redirection. No new nodes are allocated.

For window maxima over [1, 3, -1, -3, 5] with k = 3, deque indexes evolve:

i	Value	Deque after expiry/domination	Output
0	1	[0]	-
1	3	[1]	-
2	-1	[1, 2]	3
3	-3	[1, 2, 3]	3
4	5	[4]	5

At i = 4, index 1 first expires, then value 5 dominates indexes 3 and 2. The deque front is always inside the window and holds its maximum.

44.8 Complexity and performance

Operation or algorithm	Time	Auxiliary space
Access kth singly linked node	$O(k)$	$O(1)$
Insert after known node	$O(1)$	$O(1)$
Reverse list	$O(n)$	$O(1)$ iterative
Cycle detection	$O(n)$	$O(1)$
Merge two sorted lists	$O(n + m)$	$O(1)$ if relinking
Stack delimiter scan	$O(n)$	$O(n)$ worst case
BFS frontier traversal	$O(V + E)$	$O(V)$
Next greater monotonic stack	$O(n)$ amortized	$O(n)$
Sliding maximum deque	$O(n)$ amortized	$O(k)$

The worked algorithms are linear because each node or index is processed a bounded number of times.

ArrayDeque endpoint operations are amortized constant time under its documented collection behavior, while an individual growth can copy internal storage.

Linked nodes have poor spatial locality and per-node object overhead. An array-based representation often outperforms a linked structure even when both have the same asymptotic scan cost. Conversely, relinking known nodes can avoid moving large payloads.

HotSpot note: HotSpot may scalar-replace short-lived iterator or node-like objects that do not escape, but linked structures generally involve pointer chasing and allocations. Exact object layout and **ArrayDeque** growth strategy are implementation details.

WATCH OUT | 44.9 Edge cases and common mistakes

- Empty list, one node, and a cycle beginning at the head.
- Losing the suffix during reversal by redirecting before saving `next`.
- Returning the sentinel rather than `sentinel.next`.
- Using value equality when an algorithm depends on node identity.
- Advancing fast without checking both fast and `fast.next`.
- Failing to define which middle an even-length list returns.
- Relinking caller-owned lists without declaring mutation.
- Using legacy `Stack` or mixing queue and stack method names on a deque.
- Popping an empty structure because malformed input was not validated.
- Marking BFS visited when dequeued, creating duplicate work.
- Storing monotonic values when the answer requires distance or expiry indexes.
- Using `<` where duplicates require `<=`, or the reverse.
- Expiring after reading the maximum, allowing an out-of-window answer.
- Calling a monotonic structure $O(n)$ without the push/pop aggregate proof.
- Treating a concurrent queue as a complete atomic workflow.

44.10 Production engineering notes

Prefer `ArrayList` or arrays for most sequential data; use `LinkedList` only when its semantics and measured access pattern justify node overhead. For custom linked structures, define ownership, whether nodes may be shared, and whether mutation must be synchronized. Never expose partially relinked state to concurrent readers.

Queues need capacity and overload behavior. An unbounded producer-consumer queue converts load spikes into memory pressure and latency. Specify whether producers block, fail, shed, or persist. Queue thread safety does not make a multi-step check-then-act protocol atomic.

Monotonic deques are valuable for rolling telemetry, but real time windows may include out-of-order events, late data, and expiry based on timestamps rather than indexes. Extend the invariant accordingly. For parser stacks, place limits on nesting depth to resist malicious input.

TRY IT | 44.11 Interview questions and model answers**How do you prove iterative list reversal?**

Maintain a reversed processed prefix headed by `previous` and an untouched suffix headed by `current`. Save the successor, move one node to the prefix, and preserve reachability. When the suffix is empty, the prefix is the whole reversed list.

Why does Floyd's cycle algorithm use constant space?

It stores two references. Once both are inside a cycle, their relative offset advances modulo cycle length and must become zero. No visited set is needed.

Why prefer a sentinel node?

It gives the original head a predecessor, so insertion or deletion at the head uses the same link update as an interior position. This reduces branches and proof cases.

Why is a monotonic stack linear despite nested loops?

Every index is pushed once and, once popped, never returns. Across the full algorithm there are at most n pushes and n pops, so total stack operations are $O(n)$.

Why store indexes in a monotonic deque?

Indexes identify when candidates leave the window and allow distance results. Values alone cannot distinguish duplicate occurrences or expiry.

Does BFS always find a shortest path?

It finds a minimum-edge path in an unweighted graph, or when all edges have equal cost, because FIFO processing explores by distance layers. Arbitrary positive weights require a weighted shortest-path algorithm.

TRY IT | 44.12 Exercises

- 1 Reverse nodes between positions left and right using a sentinel and state the local invariant.
- 2 Find a cycle entry with Floyd's algorithm and derive why the reset step works.
- 3 Merge k sorted linked lists using a heap; analyze n total nodes and k lists.
- 4 Implement a delimiter validator with `ArrayDeque<Character>` and report the first error index.
- 5 Build a bounded circular queue using an array and a `size` field; test wraparound.
- 6 Return days until a warmer temperature using a monotonic stack of indexes.
- 7 Solve largest rectangle in a histogram, specifying duplicate-height behavior.
- 8 Compare the deque maximum template against a heap with lazy expiry on random inputs.

RECAP | 44.13 Chapter summary

Linked-list correctness depends on preserving reachability while changing links. Sentinels normalize boundaries, fast/slow pointers encode relative position, and merge is a reusable sorted primitive. Stacks and queues express reverse-dependency and discovery order. Monotonic stacks and deques retain only candidates that can still win; expiry and domination rules define their invariants. Their linear bounds come from counting each push and permanent removal across the whole run.

TRY IT | 44.14 Revision checklist

- ☐ I save a successor before redirecting a list link.
- ☐ I can prove reversal with prefix and suffix reachability.
- ☐ I use sentinels to remove head special cases.
- ☐ I understand fast/slow cycle, midpoint, and offset patterns.
- ☐ I choose LIFO, FIFO, or double-ended semantics intentionally.
- ☐ I use `ArrayDeque` consistently and handle empty operations.
- ☐ I state monotonic order, expiry rule, and domination rule.
- ☐ I store indexes when distance, duplicate identity, or expiry matters.
- ☐ I prove linear time by aggregate pushes, pops, and pointer advances.
- ☐ I discuss locality, ownership, capacity, and concurrency in production.

SDE-2 CORE CHAPTER

Chapter 45 - Trees, BSTs, Heaps, and Tries

45.1 Learning objectives

By the end of this chapter, you should be able to:

- choose preorder, inorder, postorder, or level order from data dependencies;
- write recursive tree contracts and translate them to explicit-stack traversals;
- validate and query binary search trees using inherited bounds;
- use heaps for top-k and frontier selection with correct comparator semantics;
- design tries around an explicit alphabet and terminal-state model; and
- analyze time and space in terms of node count, height, branching, and key length.

WHY IT WORKS | 45.2 Why this matters at SDE-2

Trees force candidates to reason about hierarchical dependencies. The best traversal is determined by when a node's result depends on ancestors, children, or peers. Heaps test whether only an extreme candidate is needed rather than global ordering. Tries expose representation trade-offs between query speed and memory.

Production systems use related ideas in syntax trees, indexes, schedulers, routing tables, autocomplete, dependency models, and hierarchical configuration. The textbook structure is simplified, but its invariants transfer. An SDE-2 answer should separate a logical binary search tree from a database index or a Java library implementation.

WHY IT WORKS | 45.3 First-principles model

A rooted tree is a connected acyclic hierarchy. Every node except the root has one parent, and each child roots a disjoint subtree. This self-similarity makes recursion natural: solve each subtree under a clear contract, then combine results.

A binary search tree adds an ordering invariant over whole subtrees, not just direct children. A heap adds only an extreme-at-root invariant and a complete-tree shape; it does not sort all elements. A trie organizes keys by prefixes, making each edge represent part of a key rather than a comparison against a whole key.

Algorithm choice follows the question:

- information flowing from ancestors suggests preorder or parameters carrying context;
- information flowing from descendants suggests postorder;
- sorted BST output suggests inorder; and

- minimum unweighted depth or per-level output suggests BFS.

Specification boundary: Java does not provide a built-in general tree node contract. `PriorityQueue` guarantees heap-based queue behavior and head ordering according to natural order or a comparator; it does not expose a sorted iteration order or stable ordering among ties.

45.4 Core terminology

- **Root/leaf:** top node and node with no children.
- **Depth:** edges from root to a node under the common convention.
- **Height:** longest downward edge path from a node to a leaf; conventions for empty trees must be stated.
- **Subtree:** a node and all descendants.
- **Preorder/inorder/postorder:** node-before, node-between, or node-after child traversal for binary trees.
- **Level order:** breadth-first traversal by depth.
- **BST invariant:** all keys in one subtree precede the node and all in the other follow it under a duplicate policy.
- **Heap:** complete tree with parent-child priority invariant.
- **Top-k:** retain only k best candidates seen so far.
- **Trie:** prefix tree with edges labeled by key units.
- **Terminal marker:** indicates that a trie path forms a complete key.

45.5 Detailed mechanics

Recursive contracts and traversal selection

Before writing recursion, finish this sentence: "For node x, this function returns..." Examples include subtree height, whether the subtree is balanced, maximum downward path, or a pair of summaries. Then define the null result and show how child results combine.

For height in nodes, one possible contract is:

TEXT

```
height(null) = 0
height(node) = 1 + max(height(node.left), height(node.right))
```

For balance, a naive method recomputes heights at each node and can become $O(n^2)$ on a chain. A postorder method can return height for a balanced subtree or a sentinel such as -1 for imbalance, computing both properties in one $O(n)$ pass.

Traversal choice table:

Requirement	Natural traversal	Reason
Copy tree, emit prefix notation	Preorder	Process node before descendants
Sorted keys from a valid BST	Inorder	Left, node, right follows ordering
Delete tree, height, balance, diameter	Postorder	Need child results before parent
Minimum depth, nearest target, levels	BFS	Processes increasing edge distance
Root-to-leaf path constraints	DFS with carried state	Context follows one active path

A recursive DFS uses $O(h)$ call-stack space for height h , which is $O(\log n)$ for a balanced tree and $O(n)$ for a skewed tree. Java does not guarantee tail-call elimination. An iterative traversal uses an explicit stack with the same asymptotic worst-case depth but avoids thread-stack overflow and can store explicit frame state.

State ownership in DFS

Distinguish three categories:

- **path state:** belongs only to the active root-to-current path and must be undone when backtracking;
- **subtree result:** returned upward and summarizes descendants; and
- **global result:** best answer across all visited nodes.

Tree diameter illustrates the split. Each call returns the longest downward height usable by its parent. A separate maximum records a path that may connect the left and right heights through the current node. Returning diameter instead of height would violate the parent's needed contract.

Prefer returning a small record such as `record Result(boolean balanced, int height) {}` over mutable global state when it keeps the contract local and supports reentrant calls.

Binary search tree reasoning

The BST invariant is transitive. Checking only `node.left.value < node.value < node.right.value` misses a deep value that violates an ancestor. Carry an allowed range into each recursive call:

TEXT

```
validate(node, lowerExclusive, upperExclusive)
left gets (lowerExclusive, node.value)
right gets (node.value, upperExclusive)
```

Use `long` bounds for `int` keys or nullable bounds so `Integer.MIN_VALUE` and `MAX_VALUE` remain valid. Define duplicates: forbidden, always left, always right, or counted in a node. The inequality must match that contract.

BST search, insertion, and deletion cost $O(h)$. They are $O(\log n)$ only when height is logarithmic; an unbalanced insertion order can create a chain and $O(n)$ operations. Self-balancing trees enforce height bounds, but implementing rotations is usually outside a basic interview unless requested.

Inorder traversal of a valid BST yields sorted keys. The k th smallest element can stop after k visits, use an explicit stack, or use stored subtree sizes for order-statistic queries. Augmented metadata must be updated on every mutation.

Lowest common ancestor and path problems

For a general binary tree, a postorder LCA contract can return whether a target was found and an ancestor candidate. The familiar method that returns a node when it equals p or q , then returns the current node when left and right both return non-null, assumes both targets exist unless additional found-state is tracked.

For a BST with distinct known keys, ordering permits directed descent: if both are smaller go left; if both larger go right; otherwise the current node splits them and is the LCA. This is $O(h)$ and avoids exploring both subtrees.

Root-to-leaf sums use path state. Arbitrary paths may require combining child summaries. Always define whether an empty path is legal, whether endpoints must be leaves, and whether node or edge weights are summed.

Heaps and priority queues

A binary heap uses a complete-tree shape, commonly stored in an array. For zero-based index i :

TEXT

```
parent = (i - 1) / 2
left   = 2 * i + 1
right  = 2 * i + 2
```

Insertion appends and sifts up; removal swaps in the last value and sifts down. Both are $O(\log n)$. Peeking at the minimum in Java's default `PriorityQueue` is $O(1)$. Building a heap from n values with bottom-up heapify is $O(n)$, not $O(n \log n)$, because most nodes are near the leaves and move little.

For k largest values, retain a min-heap of size k . The root is the weakest retained candidate. For each new value, insert until size k ; afterward replace the root only if the new value is larger. The invariant is that the heap contains the k largest values of the processed prefix, and its root is the k th largest among them.

For k smallest values, use a max-heap of size k . Avoid $b - a$ comparators because subtraction can overflow; use `Comparator.reverseOrder()` or `Integer.compare(b, a)`. A priority queue does not support efficient arbitrary removal or membership; combine it with a map and lazy deletion when updates are frequent.

Tries

A trie node stores outgoing edges and whether the path ending there is a complete key. Search cost is $O(L)$ for key length L , independent of the number of stored keys under the chosen edge-access model. Prefix search reaches the prefix node and then enumerates descendants; total cost must include output.

Representation depends on alphabet:

- fixed array: fast and simple for a small dense alphabet, but allocates many null slots;
- hash map: sparse and flexible, with hashing and object overhead;
- sorted map or compact edge vector: supports ordered enumeration at additional lookup cost;
- compressed/radix trie: stores strings on edges to collapse single-child chains.

The terminal flag distinguishes "app" from the prefix of "apple". Deletion clears a terminal marker and prunes nodes only when they are nonterminal and childless. Normalize case or Unicode only if the domain requires it, and make normalization consistent between insertion and lookup.

TRACE IT | 45.6 Worked Java example

This Java 21 program demonstrates global-bound BST validation, heap-based top-k selection, and an ASCII lowercase trie.

JAVA (continued 1/2)

```
import java.util.Arrays;
import java.util.PriorityQueue;

public final class TreeToolkit {
    static final class Node {
        final int value;
        Node left;
        Node right;

        Node(int value) {
            this.value = value;
        }
    }

    static boolean isValidBst(Node root) {
        return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private static boolean validate(Node node, long lower, long upper) {
        if (node == null) return true;
        if (node.value <= lower || node.value >= upper) return false;
```

JAVA (continued 2/2)

```

        return validate(node.left, lower, node.value)
            && validate(node.right, node.value, upper);
    }

    static int[] largestK(int[] values, int k) {
        if (k < 0 || k > values.length) {
            throw new IllegalArgumentException("invalid k");
        }
        PriorityQueue<Integer> selected = new PriorityQueue<>();
        for (int value : values) {
            if (selected.size() < k) selected.add(value);
            else if (k > 0 && value > selected.peek()) {
                selected.remove();
                selected.add(value);
            }
        }
        int[] answer = new int[k];
        for (int i = 0; i < k; i++) answer[i] = selected.remove();
        return answer;
    }

```

The trie implementation continues inside the same `TreeToolkit` class:

JAVA (continued 1/2)

```

    static final class Trie {
        private static final class TrieNode {
            final TrieNode[] children = new TrieNode[26];
            boolean terminal;
        }

        private final TrieNode root = new TrieNode();

        void insert(String word) {
            TrieNode node = root;
            for (int i = 0; i < word.length(); i++) {
                int edge = edge(word.charAt(i));
                if (node.children[edge] == null) {
                    node.children[edge] = new TrieNode();
                }
                node = node.children[edge];
            }
            node.terminal = true;
        }

        boolean contains(String word) {
            TrieNode node = find(word);

```

JAVA (continued 2/2)

```

        return node != null && node.terminal;
    }

    boolean hasPrefix(String prefix) {
        return find(prefix) != null;
    }

    private TrieNode find(String text) {
        TrieNode node = root;
        for (int i = 0; i < text.length(); i++) {
            node = node.children[edge(text.charAt(i))];
            if (node == null) return null;
        }
        return node;
    }

    private static int edge(char value) {
        if (value < 'a' || value > 'z') {
            throw new IllegalArgumentException("only a-z is supported");
        }
        return value - 'a';
    }
}

```

The entry point completes `TreeToolkit` and exercises all three structures:

JAVA

```

public static void main(String[] args) {
    Node root = new Node(8);
    root.left = new Node(3);
    root.right = new Node(10);
    root.left.right = new Node(6);
    System.out.println(isValidBst(root)); // true

    System.out.println(Arrays.toString(largestK(
        new int[] {5, 1, 9, 3, 7, 8}, 3))); // [7, 8, 9]

    Trie trie = new Trie();
    trie.insert("app");
    trie.insert("apple");
    System.out.println(trie.contains("app")); // true
    System.out.println(trie.contains("ap")); // false
    System.out.println(trie.hasPrefix("ap")); // true
}
}

```

`largestK` returns the selected values in ascending order because repeated removal from a min-heap yields its head order. The top-k problem itself may not require sorted output; if it does not, draining the heap is still a simple deterministic choice.

TRACE IT | 45.7 Execution or memory walkthrough

BST validation begins at 8 with allowed range (`Long.MIN_VALUE`, `Long.MAX_VALUE`). Node 3 receives the upper bound 8. Its right child 6 receives (3, 8), so it passes. Node 10 receives (8, `Long.MAX_VALUE`). A hidden node 9 under the left subtree of 3 would fail because it inherits upper bound 8 even if it is greater than its immediate parent.

For `largestK([5, 1, 9, 3, 7, 8], 3)`, the heap first becomes [1, 5, 9] conceptually, with 1 at the root. Value 3 replaces 1; 7 replaces 3; 8 replaces 5. The final heap contains {7, 8, 9}. Every removed root is proven unable to belong to the largest three of the processed prefix.

Trie insertion of `app` creates edges a, p, p and marks the last node terminal. Inserting `apple` reuses that path, then creates l and e and marks a different terminal. Thus `ap` is a reachable prefix but not a stored key.

45.8 Complexity and performance

Operation	Time	Auxiliary space
DFS traversal	$O(n)$	$O(h)$ stack
BFS traversal	$O(n)$	$O(w)$ frontier, w max width
BST search/update	$O(h)$	$O(1)$ iterative
BST validation	$O(n)$	$O(h)$
Heap peek	$O(1)$	$O(1)$
Heap insert/remove head	$O(\log n)$	$O(1)$ beyond heap
Largest k of n	$O(n \log k)$	$O(k)$
Trie insert/search	$O(L)$	$O(L)$ new nodes for insert
Enumerate prefix results	$O(P + \text{output})$	Traversal-dependent

The worked BST validator is $O(n)$ time and $O(h)$ call-stack space. Top-k is $O(n \log k)$ time and $O(k)$ space, plus $O(k \log k)$ to drain in sorted order. Trie lookup is $O(L)$, assuming fixed-array edge access, but the constant memory per node is 26 references.

For small k , a heap beats sorting all n values asymptotically. For k near n and sorted output, sorting can be simpler and competitive. A trie can outperform repeated full-key comparison for prefix workloads but may consume far more memory than a sorted array of strings.

HotSpot note: Tree node layout, reference compression, recursion compilation, and `PriorityQueue`'s backing-array details are HotSpot/library implementation concerns. Pointer-heavy trees can have poor cache locality even when asymptotic bounds match array-based structures.

WATCH OUT | 45.9 Edge cases and common mistakes

- Inconsistent height definitions for null, leaf, edges, and nodes.
- Recomputing subtree height at every node and accidentally creating $O(n^2)$ work.
- Using recursion on an unbounded skewed tree and overflowing the Java stack.
- Validating a BST only against immediate children.
- Using `int` sentinels that reject legitimate minimum or maximum keys.
- Leaving duplicate-key placement unspecified.
- Assuming a BST is balanced because it is a BST.
- Assuming `PriorityQueue` iteration is sorted or tie order is stable.
- Reversing a comparator with subtraction and overflowing.
- Keeping k largest values in a max-heap, which exposes the wrong eviction candidate.
- Calling top- k $O(n \log k)$ without including required sorted-output work.
- Forgetting a trie terminal marker, making every prefix appear to be a word.
- Allocating a dense child array for a huge sparse alphabet.
- Applying ASCII `char` trie logic to arbitrary Unicode without a stated key unit.
- Mutating augmented tree metadata on some update paths but not others.

45.10 Production engineering notes

Real database B-trees and LSM systems are designed around pages, storage hierarchy, concurrency, and durability; do not equate them with a pointer-based interview BST. Similarly, `TreeMap` is a balanced library map with a contract broader than a hand-written node exercise. Prefer these libraries unless implementing the structure is itself the task.

Use heaps for bounded candidate sets, timers, schedulers, and graph frontiers, but define stale-entry handling and tie breakers. A queue entry that references mutable priority can violate heap ordering; insert immutable snapshots or remove/reinsert through a controlled API.

Trie deployments need an alphabet, normalization, memory budget, update policy, and output cap. Autocomplete can return enormous subtrees, so cache or rank top suggestions and paginate. Concurrent readers and writers require immutability, copy-on-write, locking, or a specialized structure; ordinary node arrays are not thread-safe.

TRY IT | 45.11 Interview questions and model answers**How do you choose a tree traversal?**

Follow dependency order. Process a node before children when passing ancestor state, after children when combining subtree results, between children for BST order, and by levels when distance from the root matters.

Why is checking only BST children insufficient?

Every descendant must satisfy all ancestor bounds. A value can be less than its parent yet still be greater than an ancestor whose left subtree contains it. Carry the valid range or verify strict inorder order.

Why is bottom-up heap construction $O(n)$?

Although one sift-down can cost $O(\log n)$, most nodes are near leaves and move zero or one levels. Summing nodes by height yields a convergent weighted series proportional to n .

Which heap finds the k largest elements?

A min-heap of size k . Its root is the smallest retained value and therefore the correct candidate to evict when a larger value arrives.

What does trie search complexity omit?

$O(L)$ covers reaching the node for a length- L key under an edge lookup assumption. Returning all completions also costs at least the visited subtree and output size. Memory depends strongly on child representation and alphabet.

Recursive or iterative DFS?

Both are $O(n)$ time and $O(h)$ state. Recursion is concise and mirrors the proof, but Java stack depth is bounded and not tail-call optimized. Use an explicit stack for adversarial depth or when frame phases need direct control.

TRY IT | 45.12 Exercises

- 1 Return balance and height in one postorder pass without global mutable state.
- 2 Implement iterative inorder traversal and use it to find the k th smallest BST key.
- 3 Validate a BST under an "equal keys go right" duplicate policy.
- 4 Compute tree diameter while clearly separating returned downward state from global path state.
- 5 Merge k sorted iterators with a priority queue and a stable tie breaker.
- 6 Find the k closest points without comparator overflow and discuss output ordering.
- 7 Add deletion to the trie, pruning only safe nodes.
- 8 Replace the fixed trie child array with a map and compare memory and lookup trade-offs.

RECAP | 45.13 Chapter summary

Trees turn hierarchy into recursive subproblems. Choose traversal from the direction of information flow, define the null result, and separate path, subtree, and global state. BST correctness requires inherited ordering bounds and has $O(h)$, not automatically $O(\log n)$, operations. Heaps maintain only an extreme and support bounded top-k selection. Tries exchange memory for prefix-directed lookup and require explicit alphabet and terminal semantics.

TRY IT | 45.14 Revision checklist

- ☐ I define depth and height conventions before using them.
- ☐ I choose traversal from dependency order rather than habit.
- ☐ I state a recursive return contract and null base result.
- ☐ I account for $O(h)$ recursion or explicit-stack space.
- ☐ I validate BSTs with global bounds and a duplicate policy.
- ☐ I never assume an arbitrary BST is balanced.
- ☐ I choose the heap whose root is the eviction candidate.
- ☐ I use safe comparators and do not assume priority-queue iteration order.
- ☐ I distinguish trie prefix reachability from terminal keys.
- ☐ I include output size and representation memory in trie analysis.

SDE-2 CORE CHAPTER

Chapter 46 - Graphs, Topological Sort, Shortest Paths, and Union-Find

46.1 Learning objectives

By the end of this chapter, you should be able to:

- model directed, undirected, weighted, and state-space graphs explicitly;
- implement BFS and DFS with correct visited timing and disconnected coverage;
- detect cycles and produce a topological order for a directed acyclic graph;
- choose a shortest-path algorithm from edge-weight constraints;
- implement Dijkstra with stale-entry handling and safe distance types; and
- use union-find with path compression and union by size for incremental connectivity.

WHY IT WORKS | 46.2 Why this matters at SDE-2

Many backend problems are graphs even when no vertex is named: service dependencies, build order, account relationships, workflows, routes, permissions, and state transitions. The hard part is often the model. Is an edge directed? Can there be parallel edges? Is cost uniform? Is the state only a location, or location plus remaining budget?

At SDE-2, "use BFS" is incomplete. You must state why BFS order proves minimum distance, when a node becomes visited, what happens in disconnected components, and which resource bounds make the representation viable. Weighted paths and dependency cycles are common follow-ups.

WHY IT WORKS | 46.3 First-principles model

A graph is a set of vertices and edges. An edge can be directed or undirected and may carry weight or metadata. Traversal maintains a frontier of discovered but unfinished states. BFS and DFS differ mainly in frontier policy: FIFO versus LIFO.

The visited state records what future work is redundant. Sometimes it is one boolean per vertex. Sometimes the true vertex is a compound state such as `(cell, keysHeld)`, `(service, retriesUsed)`, or `(airport, stops)`. Marking only the visible location would merge states that have different legal futures.

A topological order linearizes precedence constraints. It exists exactly when a directed graph is acyclic. A shortest-path algorithm repeatedly establishes that certain distances cannot later improve, but the proof depends on edge weights. Union-find answers a different question: whether undirected elements are already in the same connected component as edges are added.

Specification boundary: Java collections supply list, deque, map, and priority-queue behavior; they do not define a graph abstraction or algorithm. `PriorityQueue` does not guarantee stable tie order or sorted iteration, so graph correctness must rely only on head priority.

46.4 Core terminology

- **Vertex/edge:** graph entity and relationship.
- **Directed/undirected:** edge has one direction or is traversable both ways.
- **Weighted/unweighted:** edge costs differ or each edge counts equally.
- **Adjacency list:** outgoing neighbors stored per vertex.
- **Adjacency matrix:** V by V edge table.
- **Frontier:** discovered states waiting to be expanded.
- **Visited/finalized:** discovered state or state whose optimal result is proven, depending on algorithm.
- **DAG:** directed acyclic graph.
- **Indegree:** number of incoming directed edges.
- **Relaxation:** improve a tentative distance through an edge.
- **Connected component:** maximal mutually connected set in an undirected graph.
- **Disjoint-set union (DSU):** union-find representation of evolving components.

46.5 Detailed mechanics

Model before traversal

Ask these questions first:

- 1 What uniquely identifies a vertex?
- 2 Is each relationship directed, undirected, or asymmetric in cost?
- 3 Are self-loops or parallel edges legal?
- 4 Are vertices dense integers or arbitrary domain keys?
- 5 Is the graph static, streaming, or generated on demand?
- 6 What state affects future legal moves?
- 7 Is the output reachability, a path, cost, ordering, components, or all solutions?

For dense vertex IDs $0 \dots V-1$, `List<List<Edge>>` or primitive arrays are convenient. Arbitrary keys may use a map from key to integer ID, retaining a reverse list for output. This often improves memory and makes `boolean[]`, `int[]`, and DSU possible.

Representation trade-offs:

Representation	Space	Edge lookup	Neighbor iteration
Adjacency list	$O(V + E)$	$O(\text{outdegree})$ typical	$O(\text{outdegree})$
Adjacency matrix	$O(V^2)$	$O(1)$	$O(V)$
Edge list	$O(E)$	$O(E)$	$O(E)$ unless indexed
Generated neighbors	State-dependent	Computed	No stored edge graph

An undirected edge must normally be added in both adjacency directions. Parallel edges can affect indegree, shortest paths, and output; deduplicate only when semantics allow it.

BFS

BFS for an unweighted graph:

TEXT

```
mark source visited and enqueue it
while queue not empty:
    remove front vertex
    for every neighbor:
        if unseen:
            mark visited
            record distance/parent
            enqueue
```

Mark on enqueue. If marking waits until dequeue, several parents can enqueue the same vertex, increasing memory and work. FIFO order processes all distance- d vertices before distance $d + 1$, so the first discovery gives a minimum number of edges from the source.

Store `parent[neighbor] = current` to reconstruct a path. Distance alone cannot recover the route. For multi-source BFS, enqueue all sources with distance zero before traversal; it computes distance to the nearest source.

Grid problems are implicit graphs. Bounds, blocked cells, movement directions, and whether diagonal moves exist define the edges. Mutating a grid as visited can save space only when input ownership permits it.

DFS and cycle state

DFS explores one path deeply using recursion or an explicit stack. It is useful for components, reachability, postorder, backtracking, and cycle detection. A single visited boolean detects redundant exploration but not every directed cycle. Use three colors or two booleans:

- unseen;
- active on the current recursion path; and
- complete.

An edge to an active vertex is a directed back edge and proves a cycle. An edge to a complete vertex does not. In an undirected graph, the immediate parent edge is expected and must be excluded; an edge to another visited vertex proves a cycle.

For a disconnected graph, wrap traversal in an outer loop over all vertices. Starting once answers only the source component.

Recursive DFS can overflow the Java stack on a long chain. An iterative postorder needs a frame phase or a (vertex, exiting) marker; simply pushing neighbors does not automatically reproduce recursive finish order.

Topological sort

Kahn's algorithm uses indegrees:

- 1 count each vertex's incoming edges;
- 2 enqueue all zero-indegree vertices;
- 3 remove one, append it to the order, and decrement its outgoing neighbors;
- 4 enqueue a neighbor when its indegree reaches zero; and
- 5 if fewer than V vertices are emitted, a directed cycle exists.

The invariant is that emitted vertices have all prerequisites emitted, and current indegree counts incoming edges from the un-emitted subgraph. Removing a zero-indegree vertex cannot violate precedence.

Topological orders are generally not unique. If lexicographically smallest order is required, use a priority queue for zero-indegree choices, changing complexity to $O((V + E) \log V)$ in the broad bound. If all orders are required, the output may be exponential.

DFS finish-time reversal also yields a topological order if cycle detection succeeds. Kahn's algorithm often makes cycle detection and scheduling layers easier to explain.

Shortest-path decision table

Edge model	Algorithm	Typical complexity
Unweighted/equal weight	BFS	$O(V + E)$
Weights 0 or 1	0-1 BFS with deque	$O(V + E)$
Nonnegative weights	Dijkstra with binary heap	$O((V + E) \log V)$ in the standard simple-graph model
Negative edges, no reachable negative cycle	Bellman-Ford	$O(VE)$
DAG, any weights	Topological relaxation	$O(V + E)$
All pairs, dense/small	Floyd-Warshall	$O(V^3)$, $O(V^2)$ space

Clarify whether a negative cycle reachable from the source makes the shortest value undefined because cost can decrease without bound.

Dijkstra's algorithm

Dijkstra maintains tentative distances and repeatedly expands the vertex with minimum tentative distance. With nonnegative edges, when a minimum-distance entry is current, no later route through unprocessed vertices can produce a smaller value: any extension adds a nonnegative amount.

Java's `PriorityQueue` has no efficient decrease-key operation. Insert a new state whenever a distance improves. Old entries become stale. When polling `(vertex, distance)`, skip it if `distance != distances[vertex]`. Each successful relaxation can add an entry, producing $O(E)$ queue entries and $O((V + E) \log E)$ time in the broad multigraph analysis. For a simple graph, $E \leq V^2$, so this is conventionally written $O((V + E) \log V)$.

Use `long` distances. Do not add to a sentinel `Long.MAX_VALUE`; only expand finite queued states. Reject negative weights before relying on the proof. If a predecessor is needed, update it in the same successful relaxation.

Dijkstra does not work with a negative edge because a vertex considered final can later improve through that edge. This is a proof failure, not merely a library limitation.

Union-find

DSU stores each element's parent; roots represent components. `find(x)` follows parents to a root. `union(a, b)` connects roots. Two optimizations make sequences nearly constant time:

- **path compression:** make nodes visited by `find` point closer or directly to the root;
- **union by size/rank:** attach the smaller or shallower tree beneath the larger.

Over m operations on n elements, the cost is $O(m \alpha(n))$, where α is the inverse Ackermann function and is tiny for practical n . This is an amortized bound.

DSU is excellent for incremental undirected connectivity, redundant-edge detection, Kruskal's minimum spanning tree, and grouping equivalences. It does not naturally support deletions, shortest paths, directed reachability, or listing a route.

The invariant is that every element reaches one root, and two elements have the same root exactly when processed unions connect them.

TRACE IT | 46.6 Worked Java example

This Java 21 class includes Kahn topological sort, Dijkstra, and an array-backed DSU.

JAVA (continued 1/2)

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.Deque;
import java.util.List;
import java.util.PriorityQueue;

public final class GraphAlgorithms {
    record Edge(int to, int weight) {}
    record State(int vertex, long distance) {}

    static List<Integer> topologicalOrder(int vertices, int[][] edges) {
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < vertices; i++) graph.add(new ArrayList<>());
        int[] indegree = new int[vertices];
        for (int[] edge : edges) {
            int from = edge[0];
            int to = edge[1];
```

JAVA (continued 2/2)

```
            graph.get(from).add(to);
            indegree[to]++;
        }

        Deque<Integer> ready = new ArrayDeque<>();
        for (int v = 0; v < vertices; v++) {
            if (indegree[v] == 0) ready.addLast(v);
        }

        List<Integer> order = new ArrayList<>(vertices);
        while (!ready.isEmpty()) {
            int current = ready.removeFirst();
            order.add(current);
            for (int next : graph.get(current)) {
                if (--indegree[next] == 0) ready.addLast(next);
            }
        }
        return order.size() == vertices ? List.copyOf(order) : List.of();
    }
}
```

Dijkstra's algorithm continues inside the same `GraphAlgorithms` class:

JAVA

```

static long[] dijkstra(List<List<Edge>> graph, int source) {
    long[] distance = new long[graph.size()];
    Arrays.fill(distance, Long.MAX_VALUE);
    distance[source] = 0;

    PriorityQueue<State> frontier = new PriorityQueue<>(
        Comparator.comparingLong(State::distance));
    frontier.add(new State(source, 0));

    while (!frontier.isEmpty()) {
        State current = frontier.remove();
        if (current.distance() != distance[current.vertex()]) continue;

        for (Edge edge : graph.get(current.vertex())) {
            if (edge.weight() < 0) {
                throw new IllegalArgumentException("negative edge");
            }
            long candidate = current.distance() + edge.weight();
            if (candidate < distance[edge.to()]) {
                distance[edge.to()] = candidate;
                frontier.add(new State(edge.to(), candidate));
            }
        }
    }
    return distance;
}

```

The disjoint-set implementation is the next member of `GraphAlgorithms`:

JAVA (continued 1/2)

```

static final class DisjointSet {
    private final int[] parent;
    private final int[] size;

    DisjointSet(int count) {
        parent = new int[count];
        size = new int[count];
        for (int i = 0; i < count; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int find(int value) {
        int root = value;
        while (root != parent[root]) root = parent[root];
        while (value != root) {
            int next = parent[value];
            parent[value] = root;
        }
    }
}

```

JAVA (continued 2/2)

```

        value = next;
    }
    return root;
}

boolean union(int left, int right) {
    int rootLeft = find(left);
    int rootRight = find(right);
    if (rootLeft == rootRight) return false;
    if (size[rootLeft] < size[rootRight]) {
        int temporary = rootLeft;
        rootLeft = rootRight;
        rootRight = temporary;
    }
    parent[rootRight] = rootLeft;
    size[rootLeft] += size[rootRight];
    return true;
}
}

```

The entry point completes the class and demonstrates each algorithm:

JAVA

```

public static void main(String[] args) {
    System.out.println(topologicalOrder(4,
        new int[][] {{0, 1}, {0, 2}, {1, 3}, {2, 3}}));

    List<List<Edge>> graph = List.of(
        List.of(new Edge(1, 4), new Edge(2, 1)),
        List.of(new Edge(3, 1)),
        List.of(new Edge(1, 2), new Edge(3, 5)),
        List.of());
    System.out.println(Arrays.toString(dijkstra(graph, 0))); // [0,3,1,4]

    DisjointSet sets = new DisjointSet(4);
    sets.union(0, 1);
    sets.union(2, 3);
    System.out.println(sets.find(0) == sets.find(2)); // false
    sets.union(1, 2);
    System.out.println(sets.find(0) == sets.find(3)); // true
}
}

```

The topological method returns an empty list for a cycle. That API is compact but ambiguous when the graph itself has zero vertices; a production API could return a record containing both status and order.

TRACE IT | 46.7 Execution or memory walkthrough

For topological edges $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$, initial indegrees are $[0, 1, 1, 2]$. Queue contains 0. Emitting 0 reduces 1 and 2 to zero and enqueues them. Emitting 1 reduces 3 to one; emitting 2 reduces it to zero; then 3 is emitted. Depending on neighbor and queue order, another valid DAG can have several correct outputs.

For Dijkstra, source 0 begins at distance 0. Expanding 0 proposes distance 4 to vertex 1 and 1 to vertex 2. Vertex 2 is next; it improves vertex 1 to 3 and proposes 6 to vertex 3. Vertex 1 at distance 3 improves vertex 3 to 4. The old queue states $(1, 4)$ and $(3, 6)$ later fail the equality check and are skipped. Final distances are $[0, 3, 1, 4]$.

For DSU, unions create components $\{0, 1\}$ and $\{2, 3\}$. Union of 1 and 2 attaches the smaller/equal root under the chosen larger root. Later finds compress traversed paths, so all four elements reach one root.

46.8 Complexity and performance

Algorithm	Time	Auxiliary space
BFS/DFS adjacency list	$O(V + E)$	$O(V)$
Kahn topological sort	$O(V + E)$	$O(V)$ plus graph
Lazy Dijkstra, binary heap	$O((V + E) \log E)$, or $O((V + E) \log V)$ for simple graphs	$O(V + E)$ queue worst case
Bellman-Ford	$O(VE)$	$O(V)$
DAG shortest paths	$O(V + E)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$
DSU operation sequence	$O(m \alpha(n))$	$O(n)$

Building an adjacency list is itself $O(V + E)$ time and space. If the caller already supplies one, say whether representation construction is included. BFS path output also costs $O(\text{path length})$.

The worked Dijkstra can hold stale entries, so queue memory may exceed V . An indexed heap with decrease-key can bound entries more tightly but is harder to implement correctly. For interviews, lazy stale-entry skipping is usually the right trade-off.

HotSpot note: Object-heavy adjacency lists and record queue states create allocation and indirection that primitive graph libraries can reduce. HotSpot may optimize some short-lived objects, but no elimination is guaranteed; model large production graphs with measured bytes per vertex and edge.

WATCH OUT | 46.9 Edge cases and common mistakes

- Reversing a directed edge or adding only one direction for an undirected graph.
- Ignoring self-loops, duplicate edges, or vertices with no edges.
- Starting one traversal and missing disconnected components.
- Marking BFS visited on dequeue and enqueueing duplicates.
- Using one visited boolean for directed cycle detection instead of active/complete state.
- Treating the parent edge as an undirected cycle.
- Returning a partial topological order as if it were valid.
- Assuming a topological order is unique.
- Using BFS with arbitrary positive weights.
- Running Dijkstra with a negative edge.
- Omitting stale-entry checks from lazy Dijkstra.
- Storing distance in `int` and overflowing path sums.
- Confusing an unreachable sentinel with a very large reachable distance.
- Using DSU for directed reachability or shortest paths.
- Forgetting that DSU does not support arbitrary edge deletion naturally.
- Modeling visited only by location when budget or mode changes future transitions.

46.10 Production engineering notes

Graph data is often larger than heap-friendly interview examples. Map external IDs to dense integers, choose primitive storage when memory dominates, stream edge lists when possible, and cap path/output materialization. Avoid recursively traversing untrusted dependency chains.

Dependency scheduling needs more than a topological order: failed prerequisites, optional edges, concurrency limits, retries, and versioned graph snapshots matter. Detect cycles with diagnostic paths, not only a boolean, so operators can repair configuration.

Routing weights can change while a path is computed. Define snapshot consistency and whether stale results are acceptable. For currency, latency, or risk, validate nonnegativity and overflow. Dijkstra's mathematical model does not include network calls made while expanding neighbors; cache or batch appropriately.

DSU is useful in offline batch grouping and incremental connectivity, but concurrent mutation requires synchronization or partitioning. Parent-array updates are not thread-safe merely because each entry is an `int`.

TRY IT | 46.11 Interview questions and model answers**Adjacency list or matrix?**

Use a list for sparse graphs: $O(V + E)$ space and efficient neighbor traversal. Use a matrix when the graph is dense or constant-time arbitrary edge lookup dominates and $O(V^2)$ memory is acceptable.

Why does BFS find a shortest unweighted path?

FIFO order expands vertices by nondecreasing edge distance. Every neighbor first discovered from a distance- d vertex has distance $d + 1$, and no later discovery can use fewer edges.

How does directed cycle detection differ from undirected?

Directed DFS needs active-path state; an edge to active is a back edge. Undirected DFS ignores the edge to the immediate parent and treats another visited neighbor as a cycle. Kahn's emitted count is another directed-cycle test.

Why does Dijkstra require nonnegative weights?

Its finalization proof assumes extending any path cannot reduce cost. A negative edge can create a later cheaper route to a vertex already removed as minimum, invalidating the greedy choice.

Why are duplicate priority-queue entries acceptable?

Java's queue lacks decrease-key. Inserting improved states preserves correctness if a polled state is processed only when its distance equals the current best. Stale entries increase constants and memory but keep the implementation simple.

When should union-find be chosen?

For repeated undirected connectivity and component merging as edges are added, or for Kruskal's cycle checks. It is not a traversal, cannot provide shortest paths, and does not naturally handle deletions.

TRY IT | 46.12 Exercises

- 1 Implement BFS path reconstruction and distinguish unreachable from source-to-source paths.
- 2 Detect a directed cycle and return one concrete cycle, not only an empty topological order.
- 3 Produce lexicographically smallest topological order and update the complexity.
- 4 Implement 0-1 BFS and prove why deque front/back placement preserves distance order.
- 5 Extend Dijkstra to return predecessors and reconstruct the path; guard distance overflow.
- 6 Implement Bellman-Ford with reachable negative-cycle detection.
- 7 Use DSU to find the first redundant undirected edge and report component sizes.
- 8 Model a grid where state includes remaining obstacle eliminations; explain why cell-only visited state is wrong.

RECAP | 46.13 Chapter summary

Graph algorithms begin with the correct vertex and edge model. BFS and DFS are frontier policies with different order guarantees; visited state must represent the full future-relevant state. Kahn's algorithm removes zero-indegree vertices and detects directed cycles by emitted count. Shortest-path choice follows weight constraints: BFS, 0-1 BFS, Dijkstra, Bellman-Ford, or DAG relaxation. Union-find efficiently maintains incremental undirected components through path compression and union by size.

TRY IT | 46.14 Revision checklist

- ☐ I define direction, weights, identity, duplicates, and state before traversal.
- ☐ I choose an adjacency representation from V, E, and query needs.
- ☐ I mark BFS states when enqueueing and can reconstruct paths.
- ☐ I cover disconnected graphs with an outer loop when required.
- ☐ I use active/complete state for directed DFS cycles.
- ☐ I can prove Kahn's indegree invariant and detect incomplete output.
- ☐ I select shortest-path algorithms from weight constraints.
- ☐ I implement Dijkstra with **long** distances and stale-entry skipping.
- ☐ I know DSU's invariant, amortized bound, and limitations.
- ☐ I include representation construction, output, and memory in complexity.

SDE-2 CORE CHAPTER

Chapter 47 - Recursion, Backtracking, Greedy Reasoning, and Dynamic Programming

47.1 Learning objectives

By the end of this chapter, you should be able to:

- define recursive contracts with base cases, progress, and combination logic;
- implement backtracking with explicit choose, explore, and unchoose phases;
- prune duplicates and infeasible branches without removing valid solutions;
- justify a greedy choice with an exchange, staying-ahead, or cut argument;
- specify dynamic programming by state, transition, base, order, and answer; and
- choose among brute force, memoization, tabulation, greedy, and space compression.

WHY IT WORKS | 47.2 Why this matters at SDE-2

These techniques address large search spaces. Recursion mirrors decomposable structure. Backtracking explores alternatives while reusing one mutable path. Greedy algorithms commit early when a proof says the choice cannot hurt. Dynamic programming stores repeated subproblem results when no safe local commitment exists.

The senior-level signal is not recognizing "this is DP." It is defining enough state to make the future independent of the past, proving a transition covers all choices, and explaining why a proposed greedy shortcut is valid or offering a counterexample. Production relevance includes bounded search, scheduling, resource allocation, parsing, and configuration exploration.

WHY IT WORKS | 47.3 First-principles model

Imagine a directed acyclic state graph. A state summarizes everything needed to solve the remaining problem. Transitions represent legal choices. Plain recursion may visit the same state repeatedly. Backtracking traverses a decision tree while maintaining a current partial solution. Memoization caches state results. Tabulation evaluates states in dependency order. A greedy algorithm follows one outgoing transition and discards the rest only when a proof makes them unnecessary.

The central question is information sufficiency: if two histories map to the same state, do they truly have identical legal futures and objective consequences? If not, the state is under-specified and caching it is incorrect.

Specification boundary: Java uses ordinary method invocation for recursion and does not guarantee tail-call elimination. Maximum safe recursion depth is not specified and depends on frame shape, JVM, options, and thread stack. Algorithmic proofs must not rely on recursive calls being converted to loops.

47.4 Core terminology

- **Recursive contract:** meaning of one call for its parameters.
- **Base case:** smallest state solved directly.
- **Progress:** argument moves toward a base case.
- **Decision tree:** nodes are partial choices; edges are candidate decisions.
- **Backtracking:** depth-first exploration that restores mutable path state.
- **Pruning:** skip a branch proven infeasible, duplicate, or unable to improve the answer.
- **Greedy choice property:** an optimal solution exists that begins with the selected local choice.
- **Optimal substructure:** optimal solution can be composed from optimal subproblem solutions.
- **Overlapping subproblems:** same state is reached by multiple histories.
- **Memoization/tabulation:** top-down cached recursion and bottom-up ordered evaluation.
- **State compression:** discard table dimensions or history not needed by future transitions.

47.5 Detailed mechanics

Recursion from a contract

Write a recursive method in four steps:

- 1 Define exactly what `solve(state)` returns.
- 2 Handle all minimal or invalid states.
- 3 Generate strictly smaller or closer states.
- 4 Combine recursive results into the promised result.

For binary-tree height: `height(node)` returns the number of nodes on the longest downward path in that subtree. Null returns zero. Non-null returns one plus the maximum child height. The structure guarantees progress because calls move to proper subtrees.

For index recursion, prove the index increases or remaining amount decreases. Beware zero-value choices: a recursive coin call with unchanged amount never reaches the base case.

Recursive time is number of distinct calls only when memoized. Without caching, count all nodes in the recursion tree. Fibonacci-style branching can be exponential even though there are only $O(n)$ unique argument values.

Recursive space is maximum active depth times frame state, not total calls. A branching tree may execute exponentially many calls but hold only $O(\text{depth})$ simultaneously.

Backtracking discipline

Backtracking is DFS over choices:

TEXT

```
search(state, path):
    if state is complete:
        record a copy of path
        return
    for each legal candidate:
        choose candidate
        search(next state, path)
        undo candidate
```

The path invariant is that it represents exactly the decisions from the root to the current state. The unchoose step restores the caller's path before trying its next candidate. If results store the mutable path reference instead of a copy, all output entries can later appear identical.

Candidate-generation strategy prevents duplicate output:

- subsets/combinations use a start index so order does not create permutations;
- permutations use a `used[]` set or in-place swaps;
- sorted candidates allow same-depth duplicate skipping;
- repeated reuse keeps the same candidate index, while single use advances it.

For sorted duplicate permutation input, the rule `i > 0 && values[i] == values[i-1] && !used[i-1]` skips choosing the later equal value before its identical predecessor at the same decision level. It removes symmetric branches, not distinct value sequences.

Prune only with proof. A partial sum exceeding target is safe to prune when all remaining values are nonnegative. With negative values, the branch might later recover. A branch-and-bound estimate must be optimistic enough that a pruned branch truly cannot beat the incumbent.

Greedy reasoning

A greedy algorithm makes an irrevocable local choice. Fast code is not evidence of correctness. Common proof styles are:

- **Exchange:** transform an optimal solution to use the greedy first choice without worsening it.
- **Staying ahead:** after every prefix, greedy is at least as good as any competitor under a useful measure.
- **Cut property:** the lightest or safest edge crossing a partition can belong to an optimum.
- **Invariant:** processed choices can be extended to some optimal solution.

Interval scheduling by earliest finish is a classic exchange proof. Let g be the compatible interval finishing first. Take any optimal schedule whose first interval is o . Replacing o with g cannot reduce remaining room because g ends no later. Therefore an optimum exists beginning with g ; repeat on intervals starting after g .

Choosing earliest start, shortest duration, or greatest value is not equivalent. Construct a counterexample before trusting a heuristic.

Greedy coin selection also depends on denomination structure. For coins $[1, 3, 4]$ and amount 6, largest-first chooses $4+1+1$ (three coins), while $3+3$ uses two. The local choice lacks a general exchange proof, so dynamic programming is appropriate.

Dynamic programming specification

A rigorous DP has five parts:

- 1 **State:** what does `dp[ . . . ]` mean?
- 2 **Transition:** which final or next choice connects states?
- 3 **Base:** which states are known directly?
- 4 **Order:** are dependencies computed before use?
- 5 **Answer:** which state or aggregate is returned?

For minimum coins, `dp[a]` is the minimum number of coins needed to total exactly a , or impossible. Base `dp[0] = 0`. For every amount a and usable coin c :

TEXT

```
dp[a] = min(dp[a], dp[a - c] + 1)
```

Ascending amounts permit unlimited reuse. For 0/1 selection with a one-dimensional table, amounts often iterate downward so the current item is not reused in the same iteration. Loop direction is part of the state dependency proof, not a micro-optimization.

Memoization versus tabulation

Memoization follows only reachable states and often mirrors a recurrence. It carries recursion depth and cache lookup overhead. Tabulation avoids recursion and controls iteration order, but may fill unreachable states. Under the same state graph, both are roughly $O(\text{number of states times transitions per state})$; actual constants and reachability differ.

Use a distinct marker for uncomputed versus computed-impossible. `null`, a parallel boolean array, or a value outside the result domain can work. A numeric zero is often a valid answer and cannot double as "not computed."

Designing state dimensions

Common dimensions include index, remaining budget, last choice, mask of used items, transaction count, and whether an action is currently allowed. Every extra dimension multiplies state count. State (`index`, `remaining`) with n indexes and budget B has $O(nB)$ states before transitions.

If a memo key includes a mutable list or unordered representation, equality and hashing can be expensive or unstable. Prefer dense integer indexes, records with immutable components, or bit masks for small sets.

Pseudo-polynomial DP such as $O(nB)$ is polynomial in numeric value B , not in the number of bits needed to encode B . It can be infeasible when B is one billion even if n is small.

Space compression and reconstruction

Compress only after identifying dependencies. If row i uses only row $i-1$, two rows may suffice. If each cell uses the updated value to its left and the previous-row value above, update direction determines which versions remain available.

Compression can destroy information needed to reconstruct chosen actions. To return the solution, retain parent decisions, recompute portions, or use a divide-and-conquer reconstruction. State whether the problem asks only for optimal value or also the actual choices.

Use safe sentinels. Adding one to `Integer.MAX_VALUE` overflows. Either skip impossible predecessors or choose a bounded sentinel whose arithmetic cannot overflow under constraints.

TRACE IT | 47.6 Worked Java example

This Java 21 program contrasts a proven greedy interval algorithm, dynamic programming for coin change, and duplicate-aware permutation backtracking.

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public final class OptimizationPatterns {
    record Interval(int start, int end) {
        Interval {
            if (end < start) throw new IllegalArgumentException("negative interval");
        }
    }

    static int maxNonOverlapping(Interval[] input) {
        Interval[] intervals = input.clone();
        Arrays.sort(intervals, Comparator
            .comparingInt(Interval::end)
            .thenComparingInt(Interval::start));

        int selected = 0;
        int previousEnd = Integer.MIN_VALUE;
        for (Interval interval : intervals) {
            if (interval.start() >= previousEnd) {
                selected++;
                previousEnd = interval.end();
            }
        }
    }
}
```

JAVA (continued 2/2)

```
    }
    return selected;
}

static int minCoins(int[] coins, int amount) {
    if (amount < 0) throw new IllegalArgumentException("negative amount");
    for (int coin : coins) {
        if (coin <= 0) throw new IllegalArgumentException("nonpositive coin");
    }

    int impossible = Integer.MAX_VALUE / 4;
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, impossible);
    dp[0] = 0;

    for (int subtotal = 1; subtotal <= amount; subtotal++) {
        for (int coin : coins) {
            if (coin <= subtotal && dp[subtotal - coin] != impossible) {
                dp[subtotal] = Math.min(
                    dp[subtotal], dp[subtotal - coin] + 1);
            }
        }
    }
    return dp[amount] == impossible ? -1 : dp[amount];
}
```

Duplicate-aware permutation generation continues inside the same `OptimizationPatterns` class:

JAVA (continued 1/2)

```

static List<List<Integer>> uniquePermutations(int[] input) {
    int[] values = input.clone();
    Arrays.sort(values);
    List<List<Integer>> answer = new ArrayList<>();
    backtrack(values, new boolean[values.length],
              new ArrayList<>(), answer);
    return List.copyOf(answer);
}

private static void backtrack(int[] values, boolean[] used,
    List<Integer> path, List<List<Integer>> answer) {
    if (path.size() == values.length) {
        answer.add(List.copyOf(path));
        return;
    }

    for (int i = 0; i < values.length; i++) {
        if (used[i]) continue;

```

JAVA (continued 2/2)

```

        if (i > 0 && values[i] == values[i - 1] && !used[i - 1]) continue;

        used[i] = true;
        path.add(values[i]);
        backtrack(values, used, path, answer);
        path.remove(path.size() - 1);
        used[i] = false;
    }
}

public static void main(String[] args) {
    Interval[] meetings = {
        new Interval(1, 4), new Interval(2, 3),
        new Interval(3, 5), new Interval(5, 7)};
    System.out.println(maxNonOverlapping(meetings)); // 3
    System.out.println(minCoins(new int[] {1, 3, 4}, 6)); // 2
    System.out.println(uniquePermutations(new int[] {1, 1, 2}));
}
}

```

The interval method treats touching half-open intervals as compatible (`start >= previousEnd`). If endpoints are closed and sharing an endpoint conflicts, the comparison must be strict.

TRACE IT | 47.7 Execution or memory walkthrough

For interval scheduling, sorting by end gives (2,3), (1,4), (3,5), (5,7). Select (2,3). Reject (1,4) because it starts before 3. Select (3,5) and (5,7), producing three. The exchange proof says choosing (2,3) cannot leave less room than any other first selection.

For coins [1,3,4], amount 6, the table becomes:

Amount	0	1	2	3	4	5	6
Minimum coins	0	1	2	1	1	2	2

At amount 6, coin 3 uses $dp[3] + 1 = 2$; coin 4 uses $dp[2] + 1 = 3$. The optimal recurrence considers every possible final coin, so the better 3+3 result is retained.

For permutations of $[1, 1, 2]$, sorting makes equal choices adjacent. At an empty path, index 1 is skipped while identical index 0 is unused. This prevents duplicate root branches. Once index 0 is used, index 1 can be chosen later, so valid sequences with both ones remain. Each recorded path is copied before undo.

47.8 Complexity and performance

Technique	Time	Auxiliary space	Output consideration
Linear recursion	$O(n)$	$O(n)$ call stack	None
Binary choice enumeration	$O(2^n)$	$O(n)$ path	Up to $O(2^n)$ outputs
Permutations	$O(n * n!)$	$O(n)$ excluding output	$n!$ lists of length n
Interval scheduling	$O(n \log n)$	Sort-dependent	$O(1)$ for count
Memoized states S , transitions T	$O(S * T)$	$O(S)$ plus stack	Reachable states only
Tabulated states S , transitions T	$O(S * T)$	$O(S)$	Iteration order required
Coin DP	$O(\text{amount} * \text{coinCount})$	$O(\text{amount})$	Pseudo-polynomial

The backtracking output itself dominates: for distinct values there are $n!$ permutations, each copied in $O(n)$. No algorithm materializing all permutations can be sublinear in that output.

The interval method clones input before sorting, using $O(n)$ extra references and $O(n \log n)$ time.

`Arrays.sort` details vary by element type; the algorithmic decision is sort then scan. Coin DP allocates $\text{amount} + 1$ integers and can fail due to memory long before integer indexing reaches its theoretical maximum.

HotSpot note: HotSpot may inline recursive calls but does not promise tail-call elimination. Deep recursion still consumes thread stack. Boxing path integers and allocating output lists remain material costs even when recursive control is optimized.

WATCH OUT | 47.9 Edge cases and common mistakes

- Missing base case or a recursive transition that does not make progress.
- Calculating recursion complexity from depth while ignoring branching.
- Treating total calls as simultaneous stack space.
- Recording the same mutable path reference instead of a snapshot.
- Forgetting the unchoose phase after a recursive call.
- Applying positive-value pruning when negative choices exist.
- Skipping duplicates globally instead of only at the relevant decision level.
- Claiming a greedy rule because it works on sample inputs without a proof.
- Using a canonical coin-system intuition for arbitrary denominations.
- Defining DP state without enough history to determine the future.
- Using zero as both a valid result and an uncomputed marker.
- Filling table states before their dependencies.
- Iterating a compressed 0/1 DP in the direction that reuses an item.
- Adding to an impossible sentinel and overflowing.
- Calling $O(nB)$ polynomial without noting that B is numeric magnitude.
- Compressing state before considering path reconstruction.

47.10 Production engineering notes

Exponential search requires explicit limits: maximum depth, candidates, outputs, deadline, and cancellation. Return an iterator or visitor when materializing all solutions is unnecessary. Order outputs deterministically when tests, caches, or users depend on it.

Recursion over untrusted depth can exhaust a worker thread. Convert to an explicit stack or reject excessive nesting. Memo tables need bounded keys and lifecycle; caching a high-dimensional state across requests can become a memory leak.

Greedy scheduling models must match operational constraints. Earliest finish is optimal for maximizing count of unweighted compatible intervals, not for weighted value, setup time, multiple machines, or fairness. Each added constraint can invalidate the exchange proof and require DP, flow, or approximation.

DP tables can expose a denial-of-service surface when dimensions come directly from client numeric values. Validate budgets before allocation. For monetary or large objectives, choose numeric types and infinity sentinels deliberately.

TRY IT | 47.11 Interview questions and model answers**How do you derive a recursive solution?**

Define the exact return contract for one state, identify direct base states, express larger states through strictly progressing smaller states, and combine their results. Then count the recursion tree and maximum active depth separately.

What is the backtracking invariant?

The mutable path contains exactly the choices from the root to the current search state, and auxiliary used-state matches that path. After each recursive call, undo restores the caller's state before the next candidate.

How do you prove a greedy algorithm?

Show an optimal solution can adopt the greedy choice without becoming worse, show greedy stays ahead after every prefix, or use a structural cut property. Samples and intuition are not proofs; actively seek a counterexample.

How do you recognize DP?

There are alternative choices, optimal substructure, and repeated states whose future depends only on a compact summary. Define the state first. If a safe greedy proof exists, DP may be unnecessary; if states do not overlap, plain recursion or divide-and-conquer may suffice.

Memoization or tabulation?

Memoization is natural for sparse reachable state spaces and recursive recurrences but uses call stack. Tabulation controls order and avoids recursion but may fill unused states. With the same visited states and transitions their asymptotic work is similar.

When is DP space compression safe?

When future transitions depend only on a limited subset of already computed layers and the update order preserves the old values still needed. It may be unsafe when reconstruction or same-row dependencies require discarded information.

TRY IT | 47.12 Exercises

- 1 Generate all subsets, then all fixed-size combinations, and state how candidate indexing avoids duplicates.
- 2 Solve N-Queens with column and diagonal occupancy arrays; derive output-sensitive complexity.
- 3 Produce a counterexample for shortest-duration-first interval scheduling.
- 4 Extend interval scheduling to weighted intervals with binary search plus DP.
- 5 Implement top-down and bottom-up edit distance and compare state definitions.
- 6 Solve 0/1 knapsack with one row; explain why capacity iterates downward.
- 7 Reconstruct the actual minimum-coin selection rather than only its count.
- 8 Add a deadline and output cap to permutation generation without leaving mutable state corrupted.

RECAP | 47.13 Chapter summary

Recursion begins with a return contract and progress toward base cases. Backtracking explores choices while preserving and restoring one path state. Greedy algorithms discard alternatives only when an exchange, staying-ahead, cut, or invariant proof permits it. Dynamic programming caches a sufficient state over overlapping subproblems; its specification includes state, transition, base, order, and answer. Space compression and pruning are correctness transformations that require proof, not automatic optimizations.

TRY IT | 47.14 Revision checklist

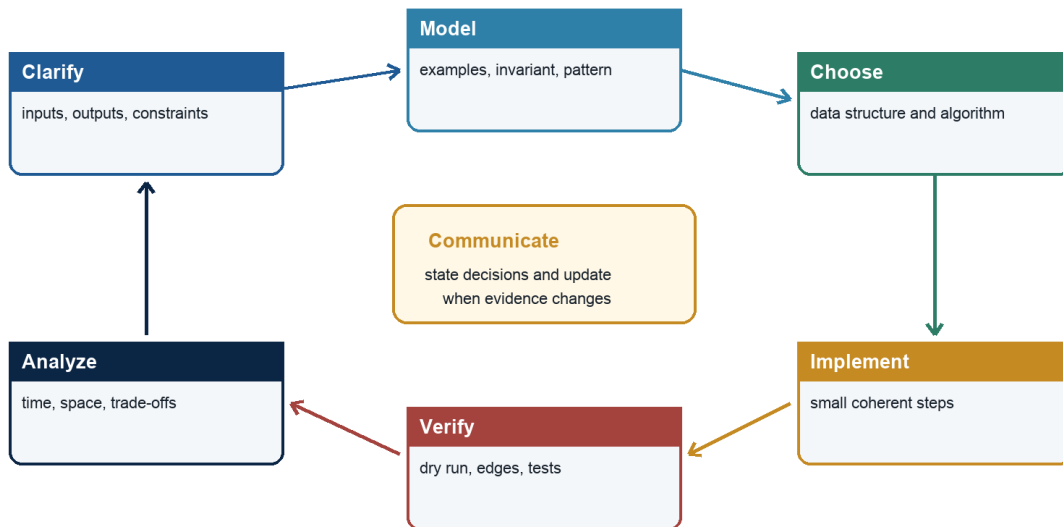
- ☐ I define a recursive contract, base cases, progress, and combination.
- ☐ I separate recursion-tree time from maximum stack depth.
- ☐ I implement choose, explore, and unchoose symmetrically.
- ☐ I copy mutable paths when recording results.
- ☐ I prove duplicate and feasibility pruning is safe.
- ☐ I demand a greedy proof and search for counterexamples.
- ☐ I specify DP state, transition, base, order, and answer.
- ☐ I distinguish uncomputed, impossible, and valid zero states.
- ☐ I explain pseudo-polynomial dimensions and sentinel safety.
- ☐ I compress DP space only after proving dependency and reconstruction needs.

SDE-2 CORE CHAPTER

Chapter 48 - The Java Coding Interview Playbook

Coding interview control loop

A repeatable process makes reasoning visible and reduces avoidable errors



Java Foundations to Advanced Engineering

Figure 48.1 - Repeatable coding interview control loop

48.1 Learning objectives

By the end of this chapter, you should be able to:

- run a repeatable control loop from clarification through final complexity;
- convert examples and constraints into a baseline, pattern, invariant, and proof;
- communicate while coding without narrating every keystroke;
- write compact, compilable Java 21 with deliberate numeric and collection choices;
- test with a boundary matrix and recover systematically from defects; and
- handle optimization, follow-ups, and production questions at SDE-2 depth.

WHY IT WORKS | 48.2 Why this matters at SDE-2

Knowledge does not automatically become interview performance. Time pressure causes candidates to skip the contract, silently assume input properties, start coding a half-remembered pattern, and discover an off-by-one error near the end. A playbook preserves reasoning bandwidth.

The interviewer is evaluating several channels at once: correctness, problem decomposition, complexity, communication, code quality, testing, and response to feedback. A candidate who reaches code slightly later but keeps the solution controlled often performs better than one who types immediately and debugs by instinct.

This chapter is not a script to recite. It is a feedback loop. Each artifact - contract, example, baseline, invariant, dry run, and complexity statement - makes the next decision safer and gives the interviewer a chance to correct a misunderstanding early.

Focused practice path: The 40-book companion series turns this playbook into shorter topic loops. Begin with the first book whose completion check is not yet automatic, and use `JAVA-09` for the question bank, mock loops, and final revision.

WHY IT WORKS | 48.3 First-principles model

A coding interview is a constrained engineering session. The problem statement defines an incomplete contract. Input bounds create a performance budget. The solution is a claim that an implementation satisfies the clarified contract within that budget. Evidence consists of an invariant or recurrence, a dry run, tests, and complexity analysis.

Treat the session as a control system:

TEXT

```
observe requirements -> propose model -> receive feedback -> implement
      ^                               |
      +----- trace and verify <-----+
```

At every stage, maintain a usable fallback. The brute-force baseline is not wasted time: it validates the target behavior, reveals repeated work, and can become a test oracle. If optimization fails, a correct partial solution with honest analysis is better than an unverified sophisticated one.

Specification boundary: Java guarantees defined language and library behavior, but not the judge's memory limit, stack size, default input constraints, or expected method signature. Those belong to the interview contract and must be clarified or stated as assumptions.

48.4 Core terminology

- **Clarifying question:** resolves an ambiguity that can change correctness or design.
- **Constraint pressure:** reason the baseline fails at stated input scale.
- **Pattern signal:** structural property suggesting a technique, not proof by keyword.
- **Invariant:** statement connecting variables to the processed and unprocessed state.
- **Oracle:** trusted implementation or property used to verify another result.
- **Boundary matrix:** compact set of tests covering shape, value, and behavioral extremes.
- **Counterexample:** input disproving a proposed rule or invariant.
- **Follow-up:** changed constraint requiring adaptation or trade-off discussion.
- **Recovery loop:** reproduce, localize invariant violation, repair, and retrace.
- **Closeout:** final proof, complexity, mutation, edge cases, and alternative summary.

48.5 Detailed mechanics

A 45-minute operating rhythm

Exact interview lengths vary, but a time budget prevents uncontrolled drift:

Phase	Approximate time	Deliverable
Clarify and examples	4-6 minutes	Contract and expected cases
Baseline and optimization	5-8 minutes	Costed alternatives and selected approach
Invariant and pseudocode	3-5 minutes	Proof skeleton
Java implementation	15-20 minutes	Compilable solution
Dry run and tests	5-7 minutes	Corrected boundary behavior
Follow-ups and close	Remaining	Complexity and trade-offs

Do not obey the table mechanically. A familiar problem can move faster; a modeling-heavy graph problem deserves more clarification. The important discipline is noticing when ten silent minutes have passed without a verified artifact.

Step 1: clarify the contract

Ask questions that can change the algorithm:

- Can input be null or empty?
- What are n and value ranges?
- Are values sorted, unique, positive, or immutable?
- Do duplicates represent separate items?
- May the method mutate or reorder input?
- Which output is required when several are valid?
- Is there always a solution? What represents absence?
- Are indexes, values, paths, counts, or only an optimum required?
- What does "character," interval overlap, or distance mean?

Avoid spending time on irrelevant possibilities. If the interviewer says to choose reasonable assumptions, state them aloud and continue: "I will treat input as non-null, allow an empty array, and return -1 when infeasible."

Restate the contract in one sentence. This creates an early synchronization point.

Step 2: construct examples

Use one normal example, one smallest case, and one adversarial case tied to a likely mistake. For a window, include a case that shrinks multiple times. For a BST, include a deep ancestor violation. For Dijkstra, include a stale priority-queue entry. For duplicates, include equal values on both sides of a boundary.

Calculate expected output manually before coding. If that is difficult, the contract is not yet understood. Do not rely exclusively on the sample supplied by the interviewer; samples often avoid exactly the boundary that breaks a memorized template.

Step 3: establish and pressure-test a baseline

Describe the direct correct approach in enough detail to cost it. For example: enumerate every subarray, calculate each sum incrementally, and track the best, $O(n^2)$ time and $O(1)$ space. Then compare with constraints. At $n = 200$, it may be fine. At $n = 200,000$, it is not.

Name eliminated work:

- repeated membership test -> hash set;
- repeated range aggregate -> prefix sum;
- repeated valid-range rescan -> sliding window;
- repeated minimum selection -> heap;
- repeated subproblem -> memoization/DP;
- exhaustive ordered boundary -> binary search;
- repeated connectivity traversal -> DSU.

This explanation is stronger than saying "I know this pattern." It shows why the representation pays for itself.

Step 4: select a pattern and state its preconditions

Use a decision map, then check its proof obligation:

Need	Candidate	Must be true
Complement/frequency/history	Hashing	Equality and memory acceptable
Eliminate ordered candidates	Two pointers	Pointer movement is monotone and safe
Maintain contiguous valid region	Window	State updates and validity permit shrinking
Ordered boundary/answer	Binary search	Predicate is monotone
Nearest in edge count	BFS	Edges have equal effective weight
Next greater/smaller	Monotonic stack	Popped candidate is permanently dominated
Best k/frontier minimum	Heap	Only priority head is required
Repeated optimal states	DP	State is sufficient; transitions complete

If a precondition fails, do not force the pattern. Negative numbers can invalidate a sum window. Negative edges invalidate Dijkstra. A non-monotone feasibility test invalidates binary search.

Step 5: articulate the invariant before code

An interview invariant should be operational, not philosophical:

- "At loop start, indexes below `low` are infeasible and indexes at or above `high` are feasible."
- "The map counts prefix sums strictly before the current prefix."
- "The deque contains unexpired indexes in increasing index and decreasing value order."
- "`dp[a]` is the minimum number of coins for exact amount `a`, or impossible."

Also state progress: right advances, candidate interval shrinks, one node moves from suffix to prefix, or every recursive call reduces remaining work.

Write short pseudocode when it helps validate phase order. For mutation-heavy logic, draw state. For recursion, write the return contract and base case before the method body.

Step 6: implement interview-grade Java

Prefer Java that makes the proof visible:

- semantic names such as `left`, `rightExclusive`, `indegree`, and `prefixFrequency`;
- half-open intervals where they simplify boundaries;
- `long` before potentially overflowing arithmetic;
- `ArrayDeque` for stacks and queues;
- `PriorityQueue` with `Integer.compare`, `Long.compare`, or comparator factories;
- records for small immutable state pairs;
- arrays for dense integer domains and maps for sparse domains;
- helper methods where they isolate a predicate or traversal contract.

Do not build a production framework during a 40-minute exercise. Avoid streams when a stateful loop is easier to trace. Avoid clever one-liners, raw types, mutable static fields, and subtraction comparators. A local nested record is often clearer than parallel arrays, but check what the interview environment supports.

Compilation is a separate correctness layer. Scan imports, generic types, return paths, method names, and static context. If no IDE is available, mentally compile one declaration at a time.

Step 7: dry run by invariant

Do not narrate only values. At each iteration, verify the invariant and boundary updates. Use a small table with the variables that prove correctness. For recursive code, draw the first few frames and show state restoration. For a heap or deque, show contents in logical order, not internal library array order.

Select a case that activates the difficult branch. Testing a sliding window that never shrinks or a graph without a cycle proves little.

Step 8: use a boundary matrix

Cover categories rather than random anecdotes:

Dimension	Cases
Size	empty, one, two, typical, maximum shape
Values	zero, negative, duplicates, min/max numeric
Position	answer at start, middle, end, absent
Structure	sorted/reversed, skewed tree, disconnected graph, cycle
Behavior	impossible, multiple valid answers, all qualify, none qualify

Trace two or three high-yield cases aloud. Mention additional cases you would automate. If time permits, compare optimized output against the baseline on small generated inputs; this property-based oracle catches many pointer and DP defects.

Step 9: recover from a bug

Do not patch randomly. Use this loop:

- 1 Freeze a minimal failing input.
- 2 Identify the first step where actual state violates the invariant.
- 3 Classify the cause: initialization, update order, boundary, stale state, overflow, or contract mismatch.
- 4 Make the smallest coherent correction.
- 5 Restart the trace from initialization.
- 6 Recheck complexity and neighboring boundary cases.

Say what you found. "I am expiring the deque after reading its front, so an old maximum leaks into the answer. I will expire before reporting." This demonstrates control rather than panic.

Step 10: close and handle follow-ups

End with:

- time in named dimensions and any expected/amortized qualifier;
- auxiliary, recursion-stack, and output space;
- whether input is mutated;
- the critical precondition and invariant;
- one alternative and when it is preferable; and
- production constraints if asked.

When a follow-up changes one constraint, identify which proof breaks before changing code. If input becomes streaming, random access disappears. If memory becomes $O(1)$, hashing may no longer fit. If negatives are allowed, window monotonicity can fail. This approach prevents cargo-cult adaptation.

TRACE IT | 48.6 Worked Java example

Problem: each positive workload takes `ceil(work / rate)` hours at integer processing rate. Workloads are handled sequentially, and each nonempty workload consumes at least one hour. Return the minimum rate that completes all work within `hourLimit`, or -1 when impossible.

JAVA (continued 1/2)

```
public final class MinimumProcessingRate {
    public static int minimumRate(int[] workloads, int hourLimit) {
        if (workloads == null) throw new IllegalArgumentException("null workloads");
        if (workloads.length == 0) return 0;
        if (hourLimit < workloads.length) return -1;

        int high = 0;
        for (int workload : workloads) {
            if (workload <= 0) {
                throw new IllegalArgumentException("workloads must be positive");
            }
            high = Math.max(high, workload);
        }

        int low = 1;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (finishesWithin(workloads, hourLimit, mid)) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }
    }
}
```

JAVA (continued 2/2)

```
    }
    return low;
}

private static boolean finishesWithin(
    int[] workloads, int hourLimit, int rate) {
    long hours = 0;
    for (int workload : workloads) {
        hours += (workload + (long) rate - 1) / rate;
        if (hours > hourLimit) return false;
    }
    return true;
}

public static void main(String[] args) {
    System.out.println(minimumRate(new int[] {3, 6, 7, 11}, 8)); // 4
    System.out.println(minimumRate(new int[] {5}, 1));           // 5
    System.out.println(minimumRate(new int[] {2, 3}, 1));        // -1
    System.out.println(minimumRate(new int[] {}, 0));             // 0
}
}
```

The feasibility predicate is monotone: if rate r finishes in time, every larger rate also finishes in time. The answer is therefore the first true rate in $[1, \text{maxWorkload}]$.

TRACE IT | 48.7 Execution or memory walkthrough

For workloads $[3, 6, 7, 11]$ and limit 8, initialize candidate range $[1, 11]$ with both endpoints represented by `low` and `high`. Rate 11 is feasible because each workload takes one hour.

Step	<code>low</code>	<code>high</code>	<code>mid</code>	Hours needed	Update
1	1	11	6	6	<code>high = 6</code>
2	1	6	3	10	<code>low = 4</code>
3	4	6	5	8	<code>high = 5</code>
4	4	5	4	8	<code>high = 4</code>
End	4	4	-	-	Return 4

Invariant: every rate below `low` is infeasible, and `high` is feasible; the first feasible rate remains in $[\text{low}, \text{high}]$. A feasible `mid` can still be larger than minimum, so retain it by setting `high = mid`. An infeasible `mid` is excluded with `low = mid + 1`. The interval strictly shrinks.

The ceiling formula promotes before addition to avoid `int` overflow. Early exit in the predicate preserves correctness because hours only increase. The empty-input behavior and impossible case are explicit contract choices rather than accidental loop results.

48.8 Complexity and performance

Let n be workload count and M the maximum workload. Feasibility is $O(n)$ time and $O(1)$ space. Binary search performs $O(\log M)$ predicate calls, so total time is $O(n \log M)$ and auxiliary space is $O(1)$. The value-domain logarithm is distinct from $O(\log n)$.

A brute-force scan of rates is $O(nM)$ and is a useful conceptual oracle for small M . Sorting does not help because every feasibility check needs the aggregate work across all workloads. Precomputing cannot remove dependence on rate without a more specialized constraint set.

Interview choice	Benefit	Cost or risk
Baseline first	Correctness anchor and oracle	Uses a few minutes
Helper predicate	Isolates monotonic proof	Extra method boundary
<code>long</code> aggregate	Prevents realistic overflow	Must promote before arithmetic
Early predicate exit	Less work on infeasible rates	Does not change worst-case bound
Half-open/closed convention	Prevents boundary drift	Must remain consistent

HotSpot note: HotSpot may inline the feasibility helper and optimize the primitive loop. Such optimization changes constants, not the $O(n \log M)$ algorithm or the need for overflow-safe arithmetic.

WATCH OUT | 48.9 Edge cases and common mistakes

- Coding before deciding null, empty, impossible, and mutation behavior.
- Asking many questions that cannot affect the algorithm while missing value ranges.
- Quoting a pattern without stating its preconditions.
- Giving an optimized idea without a correct baseline or proof.
- Using sample input as the only test.
- Mixing `int` and `long` so overflow occurs before widening.
- Writing ceiling division as floating point and introducing precision or conversion issues.
- Failing to establish that the binary-search predicate is monotone.
- Mixing first-true updates with any-match loop conditions.
- Narrating syntax instead of decisions and invariants.
- Going silent after finding a bug and applying unexplained patches.
- Claiming success without tracing the branch that mutates or shrinks state.
- Reporting $O(\log n)$ when the search dimension is a value M .
- Forgetting stack or output space.
- Overengineering interfaces and classes that do not help solve or verify the problem.
- Treating interviewer feedback as a command to patch rather than evidence to re-evaluate the model.

48.10 Production engineering notes

Interview code optimizes for a short, isolated demonstration. Production code needs input limits, observability, cancellation, API error types, test suites, documentation, and ownership. Do not paste a coding solution directly into a service boundary.

Numeric models deserve special attention. A workload might exceed `int`, processing might be parallel rather than sequential, or hour limits might use time zones and partial units. Clarify units and use domain types where appropriate. The worked predicate assumes deterministic integer work and no setup cost.

For large inputs, benchmark the actual representation. A map-heavy expected $O(n)$ solution may exceed memory. A recursive DFS may overflow a worker stack. A returned list may be the dominant allocation. State these constraints in an SDE-2 follow-up without derailing the initial interview solution.

Interview practice should be measurable. Track failure categories: contract, recognition, proof, coding, boundary, complexity, and communication. Re-solving ten problems without classifying mistakes is less useful than repairing one repeated invariant failure.

TRY IT | 48.11 Interview questions and model answers

Should I start coding immediately if I recognize the problem?

No. Spend a short, bounded period confirming the contract, constraints, and expected output. Then state the approach and invariant. Recognition accelerates reasoning but does not replace proof or boundary checks.

How much should I talk while coding?

Explain decisions, invariant-changing updates, and trade-offs. Do not narrate punctuation. Brief silence while implementing a stated plan is fine; resynchronize when entering a tricky branch or after changing the plan.

What if I cannot find the optimal solution?

Present a correct baseline, analyze its bottleneck, and explore one optimization at a time. Ask for a constraint hint if appropriate. A verified suboptimal solution plus clear progress is stronger than speculative code.

What should I do when the interviewer finds a counterexample?

Restate the failing case, find the first invariant violation, explain which assumption failed, repair the model, and retrace. Do not defend the old approach or patch only the observed output.

How do I know binary search on the answer is valid?

Define a totally ordered candidate domain and a feasibility predicate. Prove all candidates on one side are false and all on the other side true. Then search explicitly for the first true or last false boundary.

What distinguishes an SDE-2 closeout?

It includes named dimensions, qualified time, auxiliary/stack/output space, mutation, critical invariant, edge cases, and a credible alternative. It can also discuss scale, overflow, concurrency, or API contracts when prompted.

TRY IT | 48.12 Exercises

- 1 Run the ten-step playbook on two-sum, recording no more than one sentence per step.
- 2 Solve a variable-window problem and design a test that forces several left-pointer moves in one iteration.
- 3 Use an $O(n^2)$ oracle to property-test an $O(n)$ prefix-hash solution on small random arrays.
- 4 Conduct a 45-minute graph mock and record time spent in each phase.
- 5 Deliberately inject an off-by-one error into lower bound, then practice the recovery loop aloud.
- 6 Adapt `minimumRate` when workloads may be zero and when work can be split across workers; identify which assumptions change.
- 7 Take one recent failed problem and classify the root cause as contract, model, invariant, implementation, or testing.
- 8 Practice three follow-ups: forbid extra space, make input streaming, and require all valid outputs.

RECAP | 48.13 Chapter summary

A reliable coding interview is a controlled engineering loop. Clarify the contract, compute examples, establish a costed baseline, name eliminated work, verify pattern preconditions, state an invariant and progress measure, implement readable Java, trace a difficult branch, test a boundary matrix, and close with complete complexity. When a defect appears, localize the first invariant violation rather than patching randomly. Follow-ups change constraints; identify which proof breaks before adapting code.

TRY IT | 48.14 Revision checklist

- ☐ I can restate null, empty, duplicate, mutation, and absence behavior.
- ☐ I create a normal, smallest, and adversarial example before coding.
- ☐ I state a correct baseline and the repeated work being eliminated.
- ☐ I verify pattern preconditions rather than matching keywords.
- ☐ I articulate an operational invariant and progress measure.
- ☐ I use semantic Java names, safe arithmetic, and appropriate collections.
- ☐ I trace the difficult branch and use a boundary matrix.
- ☐ I recover by finding the first invariant violation.
- ☐ I report time, auxiliary space, stack, output, and mutation.
- ☐ I identify which assumption a follow-up changes before editing code.

JAVA FOUNDATIONS TO ADVANCED ENGINEERING

Part VIII - Engineering Practice and Interview Readiness

A focused part of the complete SDE-2 preparation guide

SDE-2 CORE CHAPTER

Chapter 49 - Clean Java APIs, SOLID Design, and Low-Level Design Patterns

49.1 Learning objectives

By the end of this chapter, you should be able to:

- derive classes and interfaces from domain invariants, use cases, and ownership;
- design small Java APIs with explicit null, mutation, failure, ordering, blocking, and concurrency contracts;
- apply SOLID principles as trade-off questions rather than slogans;
- recognize and implement Strategy, Factory, Adapter, Decorator, Builder, State, Command, and Observer where justified;
- evaluate substitutability, cohesion, coupling, and extension cost; and
- communicate a low-level design from requirements through objects, flows, edge cases, and tests.

WHY IT WORKS | 49.2 Why this matters at SDE-2

SDE-2 design interviews test whether you can turn ambiguous behavior into a maintainable object model. Production code tests the same skill over years: callers depend on names, defaults, failures, side effects, and timing as much as on types. A clean implementation behind an unclear API still spreads defects.

Good design is not the maximum number of interfaces or patterns. It places volatile decisions behind narrow boundaries, keeps invariants close to state, and makes the common call correct by construction. It also acknowledges operational behavior: an innocent-looking getter that performs network I/O is not clean simply because its name is short.

WHY IT WORKS | 49.3 First-principles model

Start with behavior and invariants:

TEXT

```
actors and use cases
-> commands and queries
-> state and invariants
-> ownership and lifecycle
-> failure and concurrency policy
-> interfaces at real variation boundaries
```

An object is useful when it combines state with operations that preserve that state's valid region. If callers must remember five checks before every mutation, the abstraction has leaked its invariant.

For every public operation, define:

TEXT

```
preconditions -> state transition or result -> postconditions  
              -> failure mode -> side effects -> ownership
```

Prefer a small number of domain concepts with high cohesion. Coupling is unavoidable; the goal is to couple stable policy to explicit abstractions and keep unstable infrastructure at replaceable edges.

Specification boundary: Java types express some constraints, such as access, generic relationships, and checked exceptions, but they do not express nullness, units, thread safety, blocking, ownership, idempotency, or most value invariants. These are application API contracts and must be encoded through types, validation, tests, and documentation.

49.4 Core terminology

- **Invariant:** Condition that must hold for every externally observable valid object state.
- **Precondition:** Requirement a caller must satisfy before an operation.
- **Postcondition:** Guarantee after successful completion.
- **Cohesion:** Degree to which a component's responsibilities belong together.
- **Coupling:** Dependencies between components or knowledge of one another's details.
- **Encapsulation:** Protecting a model by controlling access to representation and transitions.
- **Composition:** Building behavior by containing and delegating to collaborators.
- **Policy:** Business decision or rule.
- **Mechanism:** Technical means used to carry out policy.
- **Dependency inversion:** High-level policy depends on an abstraction rather than a low-level detail.
- **Behavioral subtyping:** A subtype can replace its supertype without violating caller expectations.
- **Pattern:** Named reusable structure for a recurring design pressure, not a mandatory template.
- **Seam:** Boundary where behavior can vary or be tested independently.

49.5 Detailed mechanics

Clean API contracts

Names should communicate units and effects: `timeout(Duration)` is stronger than `timeout(int)`, and `loadFromDatabase` is more honest than `get` when I/O occurs. Prefer domain types such as `OrderId`, `Money`, or `PageToken` when primitives would be confused or invalid states are common.

Validate at construction and mutation boundaries. Fail fast with the most specific useful exception when a programming precondition is violated. Domain rejection may deserve a domain result or exception rather than `IllegalArgumentException`. Do not partially mutate an object and then validate.

Return empty collections instead of null for zero results. Use `Optional` for a maybe-one return when absence is normal, not for every field or parameter. State whether collections preserve order, permit duplicates/nulls, are snapshots, or are live views. `List.copyOf` protects membership but does not make mutable elements immutable.

Keep command-query separation as a useful default: queries report state without changing domain state; commands perform transitions and return the minimum needed result. It is acceptable for a command to return an identifier or outcome. Avoid setters that bypass rules. `order.cancel(reason, now)` communicates and enforces more than `setStatus(CANCELED)`.

Checked versus unchecked exceptions is an API decision. Checked exceptions can force handling of recoverable boundary conditions but propagate coupling. Unchecked exceptions suit violated programming contracts and failures callers cannot meaningfully recover from locally. Never discard context; wrap with causal chains and safe domain metadata.

SOLID as five review questions

Single Responsibility Principle: Does this unit have one cohesive reason to change? "One method" is not the rule. A pricing component can contain several methods if all preserve pricing policy. A class mixing pricing, SQL, JSON, retries, and email has unrelated change drivers.

Open/Closed Principle: Can a likely new variant be added behind a deliberate seam without editing a fragile conditional everywhere? Do not create extension points for imagined futures. A sealed hierarchy can intentionally close variants; openness is not universally desirable.

Liskov Substitution Principle: Can every implementation honor the abstraction's preconditions, postconditions, invariants, and failure semantics? A subtype that rejects inputs the base accepts, returns weaker results, or changes a nonblocking call into unbounded blocking is not substitutable even if signatures match.

Interface Segregation Principle: Do clients depend only on capabilities they use? Split read, write, administration, and lifecycle interfaces when those capabilities differ. Avoid one giant service interface, but also avoid dozens of one-method interfaces with no independent variation.

Dependency Inversion Principle: Does high-level policy own the abstraction it needs, while adapters translate database, HTTP, clock, or vendor details? Depending on interfaces is not sufficient if the interface merely mirrors one vendor SDK.

Composition and inheritance

Prefer composition for optional or combinable behavior. Inheritance exposes protected state, constructor ordering, override interactions, and a strong "is-a" behavioral promise. Use it for genuine substitutable families with stable shared semantics. Mark classes or methods final when extension would bypass invariants.

Records suit transparent immutable data aggregates, not automatically rich entities. Their components are shallowly final; contained collections and objects may mutate. Sealed types model a known set of variants and enable exhaustive reasoning. Interfaces model capabilities across unrelated implementations.

HotSpot note: Whether a composed call, interface call, or small value object allocates at runtime depends on profiling, inlining, and escape analysis. Do not distort an API solely to predict HotSpot optimization. Measure a confirmed hot path on the target JDK.

Patterns and their design pressure

Pattern	Pressure it addresses	Common misuse
Strategy	Select one interchangeable policy	interface for behavior that never varies
Factory	Centralize valid creation and implementation selection	hiding arbitrary service location
Adapter	Translate an external or legacy contract	leaking vendor types through the adapter
Decorator	Add composable behavior around one contract	changing core semantics unexpectedly
Builder	Construct many optional or staged parameters	mutable builder reused across threads
State	Behavior changes with explicit lifecycle state	replacing a small clear switch with many classes
Command	Represent an action for queueing, logging, undo, or dispatch	turning every method call into an object
Observer	Notify multiple dependents	unclear lifetime, ordering, and failure isolation
Template Method	Share an algorithm skeleton through inheritance	fragile override hooks

Patterns can combine. A factory creates a strategy; decorators add metrics or retries; an adapter implements the strategy using a vendor. Each layer must preserve the underlying contract. Retry, for example, is safe only for idempotent operations or when an idempotency mechanism exists.

Low-level design interview method

Clarify scale, actors, operations, consistency, concurrency, and out-of-scope requirements. Identify nouns but validate them through behavior; not every noun needs a class. State invariants and lifecycle transitions. Sketch public APIs and one critical sequence. Then examine extension points, storage boundaries, failure, thread safety, and test seams.

Discuss trade-offs explicitly. A map-based in-memory design may be correct for one-process scope but not durable or distributed. Do not smuggle distributed guarantees into a class diagram. Keep the initial model runnable and explain how boundaries evolve.

TRACE IT | 49.6 Worked Java example

This pricing design keeps money invariants in a value object and discount variation behind a Strategy:

JAVA (continued 1/2)

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;

record Money(BigDecimal amount, String currency) {
    public Money {
        if (amount == null || currency == null || currency.isBlank()) {
            throw new IllegalArgumentException("money fields are required");
        }
        amount = amount.setScale(2, RoundingMode.UNNECESSARY);
        if (amount.signum() < 0) {
            throw new IllegalArgumentException("amount must be non-negative");
        }
    }

    Money add(Money other) {
        requireSameCurrency(other);
        return new Money(amount.add(other.amount), currency);
    }

    Money subtract(Money other) {
        requireSameCurrency(other);
        return new Money(amount.subtract(other.amount), currency);
    }

    Money multiply(int quantity) {
        if (quantity < 0) throw new IllegalArgumentException("negative quantity");
    }
}
```

JAVA (continued 2/2)

```

        return new Money(amount.multiply(BigDecimal.valueOf(quantity)), currency);
    }

    private void requireSameCurrency(Money other) {
        if (!currency.equals(other.currency)) {
            throw new IllegalArgumentException("currency mismatch");
        }
    }
}

record LineItem(String sku, int quantity, Money unitPrice) {
    public LineItem {
        if (sku == null || sku.isBlank() || quantity <= 0 || unitPrice == null) {
            throw new IllegalArgumentException("invalid line item");
        }
    }
}

record Order(List<LineItem> items, String customerTier) {
    public Order {
        items = List.copyOf(items);
        if (items.isEmpty() || customerTier == null) {
            throw new IllegalArgumentException("invalid order");
        }
    }
}

```

The policy contract and pricing service build on those value objects:

JAVA (continued 1/2)

```

interface DiscountPolicy {
    Money discountFor(Order order, Money subtotal);
}

final class TierDiscountPolicy implements DiscountPolicy {
    @Override
    public Money discountFor(Order order, Money subtotal) {
        BigDecimal rate = switch (order.customerTier()) {
            case "GOLD" -> new BigDecimal("0.10");
            case "SILVER" -> new BigDecimal("0.05");
            default -> BigDecimal.ZERO;
        };
        BigDecimal amount = subtotal.amount().multiply(rate)
            .setScale(2, RoundingMode.HALF_EVEN);
        return new Money(amount, subtotal.currency());
    }
}

record Quote(Money subtotal, Money discount, Money total) {}

```

JAVA (continued 2/2)

```

final class PricingService {
    private final DiscountPolicy discountPolicy;

    PricingService(DiscountPolicy discountPolicy) {
        this.discountPolicy = java.util.Objects.requireNonNull(discountPolicy);
    }

    Quote quote(Order order) {
        String currency = order.items().get(0).unitPrice().currency();
        Money subtotal = new Money(BigDecimal.ZERO.setScale(2), currency);
        for (LineItem item : order.items()) {
            subtotal = subtotal.add(item.unitPrice().multiply(item.quantity()));
        }
        Money discount = discountPolicy.discountFor(order, subtotal);
        if (!discount.currency().equals(currency)
            || discount.amount().compareTo(subtotal.amount()) > 0) {
            throw new IllegalStateException("discount policy violated its contract");
        }
        return new Quote(subtotal, discount, subtotal.subtract(discount));
    }
}

```

The service owns orchestration; the policy owns discount variation; **Money** owns currency and nonnegative-value rules. A factory could select a policy from configuration, and a decorator could measure policy latency without changing **PricingService**.

TRACE IT | 49.7 Execution or memory walkthrough

For two USD items priced 10.00×2 and 5.00×1 , **PricingService** starts with USD zero, produces subtotals **20.00** and then **25.00**, and asks the policy for a discount. A GOLD order yields **2.50**, so the quote total is **22.50**.

Construction copies the item list, preventing later membership changes through the caller's list. **LineItem** and **Money** are immutable records, so this particular object graph is stable. If a component were a mutable object, **List.copyOf** alone would not deep-copy it.

The service checks the strategy's postcondition: same currency and discount no greater than subtotal. That defensive boundary makes a broken implementation fail near the cause rather than creating negative totals downstream. It does not need to know how the strategy chose the discount.

49.8 Complexity and performance

For n line items, quote computation is $O(n)$ time and $O(1)$ additional domain storage beyond immutable result objects. `BigDecimal` arithmetic cost depends on precision and magnitude, so the simple $O(n)$ model treats money operation size as bounded.

Abstraction cost should be considered at the design level first:

Choice	Benefit	Cost to evaluate
immutable values	safe sharing, simple invariants	allocation and copying
interface seam	substitution and testing	more concepts and contract surface
defensive copy	ownership isolation	$O(n)$ time and references
decorator	composable cross-cutting behavior	call depth, ordering interactions
builder	readable optional construction	extra mutable lifecycle

Avoid making every field a wrapper or every class an interface in the name of cleanliness. Complexity of understanding is a performance constraint on the engineering organization. Measure runtime concerns only after profiling identifies a hot path.

WATCH OUT | 49.9 Edge cases and common mistakes

- Starting from patterns instead of use cases and invariants.
- Creating interfaces for every class with no independent implementation or test seam.
- Using setters that permit invalid intermediate states.
- Hiding blocking I/O or mutation behind query-like names.
- Returning internal mutable collections or mutable builder state.
- Treating record components as deeply immutable.
- Applying LSP only to method signatures and ignoring failure, timing, and side effects.
- Building a base class with protected fields and many override hooks.
- Adding retry decorators to non-idempotent operations.
- Letting an adapter expose vendor DTOs, exceptions, or configuration throughout the domain.
- Using a service locator or static singleton and calling it dependency inversion.
- Splitting cohesive behavior across so many classes that one change requires navigating a graph.
- Using `double` for money or omitting currency and rounding contracts.
- Optimizing dispatch based on guessed JVM behavior.

49.10 Production engineering notes

Treat public APIs as versioned products. Remove ambiguity about nulls, blocking, timeout, idempotency, thread safety, collection ownership, ordering, and exceptions. Favor additive evolution and migration adapters over silently changing semantics.

Keep domain policy free of framework annotations and vendor DTOs where practical. Translate at controllers, messaging adapters, and repositories. This makes business behavior testable and limits dependency upgrades to boundary code. Do not create duplicate models mechanically when one stable representation genuinely serves both layers.

Lifecycle is part of design. Observers need registration handles; executors and clients need closure; caches need bounds; decorators need ordering; factories need validated configuration. Expose health and metrics through separate operational interfaces rather than mixing administrative mutation with core use cases.

Run architecture checks for forbidden dependency directions, but use reviews to evaluate semantics. Static package rules cannot prove substitutability or a useful abstraction. Record significant design decisions, alternatives, and reversal cost.

TRY IT | 49.11 Interview questions and model answers

What makes an API clean?

It has cohesive operations, explicit invariants and ownership, unsurprising names, minimal invalid states, and documented failure, blocking, concurrency, null, and ordering behavior. It is easy to use correctly and hard to misuse.

Explain dependency inversion with an example.

Pricing policy depends on a small exchange-rate abstraction owned by the domain, not a vendor HTTP client. An adapter implements that abstraction using the vendor. Policy tests use a deterministic implementation, and vendor changes stay at the edge.

How do you decide between composition and inheritance?

Use inheritance only for a true behavioral subtype with stable shared semantics. Use composition for optional, replaceable, or combinable behavior because it exposes fewer override interactions and preserves encapsulation.

Is SOLID always good?

The principles are review heuristics with trade-offs. Premature interfaces and tiny classes can increase cognitive load. Apply them at observed change boundaries and preserve cohesion.

How do Strategy and Decorator differ?

Strategy selects the core policy implementation. Decorator wraps the same contract to add behavior such as metrics, caching, or authorization while delegating. A decorator must preserve the wrapped contract.

How would you approach a low-level design interview?

Clarify use cases and scale, state invariants, model lifecycle and ownership, sketch the public API and critical sequence, then cover extension, concurrency, failure, persistence boundaries, complexity, and tests.

TRY IT | 49.12 Exercises

- 1 Replace a `setStatus` order API with valid transition methods and specify preconditions and failures.
- 2 Design a retry decorator contract that is safe only for idempotent commands. Show how callers communicate idempotency.
- 3 Refactor a report class that queries SQL, calculates totals, formats JSON, and emails output into cohesive boundaries.
- 4 Add a promotion-composition policy to the worked example. Decide whether discounts stack, cap, or select the best.
- 5 Model a vending machine with either a State pattern or an enum switch. Compare extension and readability.
- 6 Review an interface whose implementations disagree on null, timeout, and exception behavior. Write a substitutable contract.

RECAP | 49.13 Chapter summary

Clean Java design starts with use cases, invariants, ownership, and observable contracts. SOLID principles expose change, substitutability, capability, and dependency questions; they are not a class-count target. Patterns name structures that address real variation or lifecycle pressure, and composition is usually the safest default. A strong low-level design remains runnable, states trade-offs, isolates infrastructure, and makes invalid state and accidental side effects difficult.

TRY IT | 49.14 Revision checklist

- ☐ I derive types and methods from behavior and invariants rather than nouns alone.
- ☐ I document null, mutation, order, failure, blocking, idempotency, lifecycle, and thread safety.
- ☐ I can apply all five SOLID principles as concrete review questions.
- ☐ I distinguish behavioral subtyping from signature compatibility.
- ☐ I prefer composition unless a genuine stable is-a relationship exists.
- ☐ I can select and critique common low-level design patterns.
- ☐ I keep policy independent from vendor and framework details at deliberate seams.
- ☐ I can present requirements, APIs, sequence, edge cases, complexity, and tests in a design interview.

SDE-2 CORE CHAPTER

Chapter 50 - Backend Java Boundaries: JDBC, Transactions, Serialization, and Services

50.1 Learning objectives

By the end of this chapter, you should be able to:

- use JDBC resources and prepared statements with explicit ownership and transaction control;
- choose transaction boundaries from business invariants rather than repository method boundaries;
- explain isolation anomalies and why database implementations must be verified;
- avoid dual-write inconsistency with transactional outbox and idempotent processing patterns;
- design versioned, validated serialization DTOs independent from persistence entities; and
- specify timeouts, retries, idempotency, backpressure, and observability at service boundaries.

WHY IT WORKS | 50.2 Why this matters at SDE-2

Backend correctness lives between components. A Java method can be locally correct while its transaction commits half an invariant, its JSON change breaks an older consumer, or its retry charges a customer twice. JDBC, messaging, HTTP, and serialization are not plumbing details; they define consistency and failure semantics.

At SDE-2, you are expected to place a transaction around a use case, explain what it cannot include atomically, and make remote effects retry-safe. You should know that closing a pooled connection usually returns it rather than closing a socket, that `PreparedStatement` protects values rather than arbitrary SQL identifiers, and that a timeout does not prove the remote operation did not happen.

WHY IT WORKS | 50.3 First-principles model

A boundary converts one model and failure domain into another:

TEXT

```
HTTP bytes -> validated request DTO -> application command
           -> domain transition -> SQL transaction
           -> outbox record -> broker message -> remote consumer
```

Each arrow needs a contract: encoding, validation, identity, timeout, atomicity, retry, ordering, ownership, and version compatibility.

A local database transaction groups statements under ACID goals:

- **atomicity:** all transaction effects commit or none do;
- **consistency:** constraints and application rules move valid state to valid state;
- **isolation:** concurrent transactions observe behavior defined by an isolation level;
- **durability:** committed changes survive according to database guarantees and configuration.

The database transaction does not automatically include an HTTP call, email, cache, or message broker. Crossing failure domains requires protocols and recoverable state, not a larger Java `try` block.

Specification boundary: JDBC standardizes interfaces and broad transaction operations. SQL syntax, type mappings, generated keys, isolation behavior, lock semantics, timeout enforcement, batch behavior, and connection-pool reset are database, driver, and pool specific. Test the exact versions in use.

50.4 Core terminology

- **DataSource:** Factory for database connections; often backed by a pool.
- **Connection:** JDBC session and transaction context.
- **Prepared statement:** Precompiled/parameterized statement separating SQL structure from values.
- **Transaction boundary:** Scope whose database effects commit or roll back together.
- **Isolation anomaly:** Concurrent outcome such as dirty read, nonrepeatable read, phantom, lost update, or write skew.
- **Optimistic concurrency:** Detect conflict using a version or compare-and-set condition.
- **Pessimistic locking:** Acquire database locks before a conflicting transition.
- **Idempotency:** Repeating an operation with the same identity has the intended single logical effect.
- **Outbox:** Database table storing events in the same local transaction as domain changes.
- **Inbox/deduplication:** Consumer record of processed message identities.
- **DTO:** Boundary-specific data transfer object.
- **Schema evolution:** Changing a wire or storage representation while preserving compatibility policy.
- **Retry budget:** Bound on attempts, elapsed time, and load amplification.

50.5 Detailed mechanics

JDBC resource ownership

Acquire a connection as late as practical and release it promptly. `Connection`, `Statement`, and `ResultSet` are `AutoCloseable`; nested try-with-resources closes them in reverse order. Closing a connection from a pool normally returns it to the pool, but that behavior belongs to the configured `DataSource`.

Do not hold a database connection while performing slow remote I/O. It extends lock time and consumes scarce pool capacity. A pool is a concurrency bound, not a throughput generator. Monitor active, idle, wait time, timeouts, and leaked borrows.

Use prepared statements for values:

JAVA

```
try (var statement = connection.prepareStatement(
    "select id, status from orders where customer_id = ?")) {
    statement.setString(1, customerId);
    try (var rows = statement.executeQuery()) {
        while (rows.next()) {
            // Map explicitly by stable column labels.
        }
    }
}
```

Placeholders do not represent table names, column names, sort direction, or arbitrary SQL fragments. Dynamic identifiers require a fixed allow-list mapped to known SQL. Never concatenate untrusted input.

Set query, lock, transaction, and network timeouts according to driver capabilities and an end-to-end deadline. `Statement.setQueryTimeout` units and enforcement have driver limits. A canceled query may continue server-side briefly; monitor the database as well as the client.

Transaction boundaries and failure handling

Disable auto-commit before a multi-statement transaction, commit once after all invariant checks and writes, and roll back on failure. If rollback throws, preserve it as a suppressed exception rather than losing the primary cause. Never return a connection with an open transaction or altered session state; use a pool that reliably resets documented state and keep application behavior disciplined.

The boundary belongs at the application use case. Two repository methods called separately with independent transactions cannot enforce a cross-table invariant. Conversely, a transaction spanning user think time or a remote request holds resources and locks too long.

Database constraints are the final concurrent guard. Use primary keys, unique constraints, foreign keys, checks, and atomic update predicates. A Java "check then insert" can race. Map constraint violations to domain outcomes carefully using driver/database error information rather than brittle message parsing.

Isolation and concurrency control

Isolation levels are commonly described by prohibited phenomena, but real databases use locks or multiversion concurrency control with vendor-specific behavior. "Repeatable read" does not mean identical semantics everywhere. Serializable execution can abort a transaction and require retry.

For an update such as reserving inventory, useful approaches include:

- atomic conditional SQL: update where available quantity is sufficient, then inspect update count;
- optimistic versioning: update where `version = expected`, increment version, reject count zero;
- pessimistic lock: lock the row, inspect, then update within one short transaction;
- serializable transaction: let the database detect conflicting executions and retry safe failures.

Choose from contention, invariant shape, latency, and database behavior. Retrying a transaction reruns its code, so no non-idempotent external side effect should occur inside it.

Vendor boundary: Lock clauses, deadlock detection, snapshot rules, serialization failures, timestamp precision, UUID/JSON types, and DDL transaction behavior differ. Treat database documentation and integration tests as part of the contract.

The dual-write problem

Suppose the service commits an order and then publishes `OrderCreated`. A crash between those actions loses the event. Publishing first can expose an event for an order that later rolls back. Ordinary local transactions cannot atomically cover both systems.

The transactional outbox writes the order and an outbox row in one database transaction. A separate relay reads committed rows, publishes messages, and marks progress. Publication may occur more than once around failures, so consumers use stable event IDs and idempotent inbox or natural-key effects. This provides recoverable at-least-once processing, not magical exactly-once behavior across arbitrary side effects.

An outbox needs cleanup, lag metrics, partition/order policy, claim/lease semantics, and poison-message handling. Change-data-capture can relay rows but remains an operational component with its own offsets and failure modes.

Serialization boundaries

Do not serialize persistence entities or rich domain objects directly by default. Lazy-loading proxies, bidirectional graphs, internal fields, and refactors make unstable contracts. Define request/response/event DTOs and map explicitly at the boundary.

Specify charset, media type, field names, required/optional/default behavior, numeric range and precision, timestamps and timezone, enum evolution, unknown fields, null versus absent, collection limits, and maximum nesting/bytes. Validate syntactic shape at decoding and domain rules in the application/domain layer.

Backward compatibility usually means new producers avoid removing or changing fields older consumers require, while consumers tolerate documented additions. Adding an enum value can break exhaustive older consumers. Version semantics, not merely a `/v2` path, must be planned.

Native Java serialization is unsuitable for untrusted service boundaries. Prefer an explicit schema and safe parser configuration. No format is automatically safe: JSON and binary parsers still need size, depth, duplicate-field, polymorphism, and resource limits.

Remote service boundaries

Every call needs an end-to-end deadline propagated through connection acquisition, connect, request, and response phases. Layered timeouts should leave callers time to recover. Cancellation and interruption should release resources.

Retry only failures that are transient and operations that are idempotent or protected by an idempotency key. Use exponential backoff with jitter and a strict attempt/deadline budget. Retries amplify load during an outage; combine them with admission control, circuit breaking, concurrency limits, and bounded queues.

A timeout is an unknown outcome. The server may have committed after the client stopped waiting. Reconcile using an idempotency key and status lookup rather than issuing a semantically new request. Record request IDs and trace context, but do not place secrets or unbounded tenant IDs in metric labels.

Framework transaction caveats

Framework annotations often implement transactions through proxies or interception. Method visibility, self-invocation, exception type, propagation, asynchronous boundaries, and reactive execution can affect whether a transaction exists. These are framework/version rules, not Java or JDBC guarantees. Verify with integration tests that inspect committed outcomes, not only mocked repository calls.

TRACE IT | 50.6 Worked Java example

This service writes an order and outbox record atomically using standard JDBC:

JAVA (continued 1/2)

```
import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.time.Clock;
import java.time.Instant;
import java.util.UUID;
import javax.sql.DataSource;

record PlaceOrder(String customerId, BigDecimal total, String currency) {}

public final class OrderApplicationService {
    private final DataSource dataSource;
    private final Clock clock;

    public OrderApplicationService(DataSource dataSource, Clock clock) {
        this.dataSource = java.util.Objects.requireNonNull(dataSource);
        this.clock = java.util.Objects.requireNonNull(clock);
    }

    public String place(PlaceOrder command) throws SQLException {
```

JAVA (continued 2/2)

```

        validate(command);
        String orderId = UUID.randomUUID().toString();
        String eventId = UUID.randomUUID().toString();
        Instant now = clock.instant();

        try (Connection connection = dataSource.getConnection()) {
            connection.setAutoCommit(false);
            try {
                insertOrder(connection, orderId, command, now);
                insertOutbox(connection, eventId, orderId, now);
                connection.commit();
                return orderId;
            } catch (SQLException | RuntimeException failure) {
                try {
                    connection.rollback();
                } catch (SQLException rollbackFailure) {
                    failure.addSuppressed(rollbackFailure);
                }
                throw failure;
            }
        }
    }
}

```

The insert helpers and validation complete the same `OrderApplicationService` class:

JAVA (continued 1/2)

```

private static void insertOrder(Connection connection, String orderId,
    PlaceOrder command, Instant now) throws SQLException {
    String sql = "insert into orders "
        + "(id, customer_id, total, currency, created_at) "
        + "values (?, ?, ?, ?, ?)";
    try (PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setString(1, orderId);
        statement.setString(2, command.customerId());
        statement.setBigDecimal(3, command.total());
        statement.setString(4, command.currency());
        statement.setTimestamp(5, Timestamp.from(now));
        if (statement.executeUpdate() != 1) {
            throw new SQLException("order insert affected unexpected row count");
        }
    }
}

private static void insertOutbox(Connection connection, String eventId,
    String orderId, Instant now) throws SQLException {
    String sql = "insert into outbox "
        + "(event_id, event_type, aggregate_id, payload, created_at) "

```

JAVA (continued 2/2)

```

        + "values (?, ?, ?, ?, ?)";
    try (PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setString(1, eventId);
        statement.setString(2, "OrderCreated");
        statement.setString(3, orderId);
        statement.setString(4, "order-created:" + orderId);
        statement.setTimestamp(5, Timestamp.from(now));
        if (statement.executeUpdate() != 1) {
            throw new SQLException("outbox insert affected unexpected row count");
        }
    }
}

private static void validate(PlaceOrder command) {
    if (command == null || command.customerId() == null
        || command.total() == null || command.total().signum() < 0
        || command.currency() == null) {
        throw new IllegalArgumentException("invalid order command");
    }
}
}

```

The textual payload is deliberately minimal; a real event uses a versioned serializer and schema. SQL types and timestamp mappings must be integration-tested with the selected driver.

TRACE IT | 50.7 Execution or memory walkthrough

- 1 Validation runs before borrowing a scarce connection.
- 2 IDs and time are created once, making retry identity a design decision. As written, a caller retry invokes `place` again and creates a new order; a public idempotency key should be supplied for retry-safe API behavior.
- 3 Auto-commit is disabled. The order insert and outbox insert share one connection and transaction.
- 4 If both affect one row, commit makes both visible under database semantics.
- 5 If the second insert fails, rollback removes the first insert. A rollback failure is preserved as suppressed evidence.
- 6 A commit or close exception can leave the caller uncertain whether commit succeeded. With a pool, close should return and reset the connection according to pool configuration; public retry safety still requires a stable idempotency key.
- 7 Later, a relay publishes the event. A crash after broker acceptance but before marking progress can cause duplicate delivery; consumers deduplicate by `eventId`.

The method retains only command and small JDBC objects. The database, not the Java heap, owns durable state. Holding the transaction open while calling the broker would increase lock and pool occupancy without creating cross-system atomicity.

50.8 Complexity and performance

Application-side computation is $O(1)$ for fixed-size fields, but database cost depends on indexes, constraints, logging, contention, and network latency. An indexed primary-key insert is often described as $O(\log n)$ for a B-tree, yet storage engines batch, cache, split pages, and write logs; measure the actual database.

Important capacity relationships are:

TEXT

```
concurrent database work <= connection-pool capacity
queueing wait + query time + commit time <= request deadline
retry traffic <= remaining downstream capacity
outbox production rate <= sustainable relay rate over time
```

Batching can reduce round trips but changes failure attribution and memory. Large transactions retain locks/versions/log records longer and make retries more expensive. $N+1$ queries turn one logical operation into $O(n)$ round trips; fetch or batch deliberately while avoiding explosive joins.

WATCH OUT | 50.9 Edge cases and common mistakes

- Building SQL by concatenating values or untrusted sort identifiers.
- Forgetting to close a result set, statement, or connection on an exception path.
- Holding a connection across HTTP calls or user think time.
- Placing each repository call in an independent transaction when the invariant spans calls.
- Assuming application checks replace unique or check constraints.
- Retrying all SQL exceptions, including permanent constraints and unknown side effects.
- Assuming timeout means the database or remote service rolled back.
- Publishing an event after commit with no durable recovery record.
- Claiming exactly-once effects because a broker has an exactly-once feature.
- Serializing ORM entities, lazy proxies, or internal exception objects directly.
- Changing enum, timestamp, numeric, null, or unknown-field semantics without compatibility testing.
- Returning database error details or SQL in public responses.
- Using an annotation without verifying proxy, propagation, and rollback behavior.
- Making queues and retry attempts unbounded during a downstream outage.

50.10 Production engineering notes

Size connection pools from database capacity and measured service time, not application thread count. Set acquisition deadlines and expose saturation. Validate connections and pool reset behavior. Keep transactions short, but never split an invariant merely to lower a timing metric.

Use migration tooling with ordered, reviewed, backward-compatible changes. Deploy expand/migrate/contract sequences when old and new application versions overlap. Index creation and DDL locking are database-specific operational events, not harmless startup steps.

Version events and APIs, retain contract fixtures, and run consumer compatibility tests. Redact logs, cap body size, reject invalid content types/charsets, and avoid generic polymorphic deserialization from untrusted input.

For every remote dependency, publish deadline, retry, idempotency, circuit, concurrency, and fallback policy. Measure attempts separately from logical requests, and measure outbox age, retries, poison records, deduplication conflicts, and end-to-end delivery latency.

TRY IT | 50.11 Interview questions and model answers

Where should a transaction boundary be?

Around the shortest database scope that enforces one business invariant or use case. It should include all required local reads and writes, but not remote I/O that the database cannot commit atomically.

How do prepared statements prevent SQL injection?

They separate SQL structure from bound values so values are not parsed as SQL syntax. They do not parameterize identifiers or arbitrary fragments; dynamic structure must come from an allow-list.

What is the dual-write problem?

Updating a database and publishing to another system cannot be made atomic by ordinary local transaction code. A crash between operations creates inconsistency. A transactional outbox stores publish intent with the domain update and supports recoverable at-least-once relay.

How do you handle duplicate messages?

Give each logical event a stable ID and make the consumer effect idempotent using an inbox/unique constraint or natural idempotent update in the same transaction as its business effect.

What does a timeout tell you?

Only that the caller stopped waiting within its policy. The remote operation may not have started, may still run, or may have committed. Use idempotency and status reconciliation.

How do you choose an isolation level?

Start from the invariant and concurrent anomalies that must be prevented, then choose an atomic statement, optimistic version, lock, or isolation level supported by the target database. Test contention and retry behavior.

TRY IT | 50.12 Exercises

- 1 Implement an optimistic update using `where id = ? and version = ?`; interpret update count zero.
- 2 Add a caller-provided idempotency key to the worked example with a unique database constraint and replayed-result behavior.
- 3 Design an outbox relay claim algorithm and state what happens after publish succeeds but progress update fails.
- 4 Specify a versioned `OrderCreated` schema, including time, money, enum, unknown field, and size behavior.
- 5 Analyze a request whose 500 ms deadline includes a 200 ms pool wait and two retries. Construct a valid remaining-time budget.
- 6 Write an integration test that proves a framework transaction rolls back both rows when the outbox insert fails.

RECAP | 50.13 Chapter summary

Backend boundaries convert models and failure domains. JDBC code must own resources, parameterize values, keep transactions short, and rely on database constraints for concurrent truth. Isolation is an invariant decision with vendor-specific behavior. Database and broker effects require a recoverable outbox plus idempotent consumers, not a claim of universal exactly-once delivery. Serialization and remote APIs need explicit version, validation, timeout, retry, and unknown-outcome contracts.

TRY IT | 50.14 Revision checklist

- ☐ I close JDBC resources and avoid holding connections across remote I/O.
- ☐ I use prepared values and allow-list dynamic SQL structure.
- ☐ I place transactions around use-case invariants and preserve rollback failures.
- ☐ I can compare atomic updates, optimistic versions, locks, and isolation levels.
- ☐ I understand dual-write failure, outbox relay, and consumer idempotency.
- ☐ I define DTO schema, limits, timestamps, money, enum, null, and compatibility behavior.
- ☐ I treat timeout as an unknown outcome and retries as bounded load amplification.
- ☐ I verify database, driver, pool, and framework behavior with integration tests.

SDE-2 CORE CHAPTER

Chapter 51 - Testing, Build Tools, Static Analysis, and Dependency Management

51.1 Learning objectives

By the end of this chapter, you should be able to:

- choose unit, integration, contract, end-to-end, property, and performance tests from risk;
- write deterministic behavioral tests with clear fixtures, boundaries, and oracles;
- use fakes, stubs, mocks, clocks, and seeded randomness deliberately;
- configure Java toolchains and release targets for repeatable builds;
- integrate compiler checks, static analysis, formatting, coverage, and mutation evidence without metric gaming; and
- govern direct and transitive dependencies for compatibility, security, licensing, and reproducibility.

WHY IT WORKS | 51.2 Why this matters at SDE-2

SDE-2 engineers own confidence, not only code. A test suite must catch meaningful regressions quickly enough to run, a build must produce the same intended artifact in CI and release, and dependencies must be understood as executable supply-chain input.

Many teams have thousands of tests and still fear deployment because tests assert implementation details, integration behavior is mocked away, and flaky timing tests are ignored. Other teams create slow end-to-end suites for behavior that a pure unit test would prove better. The goal is a layered evidence system whose cost matches risk.

WHY IT WORKS | 51.3 First-principles model

A test is an experiment:

TEXT

```
given controlled state and inputs
when one behavior occurs
then observable results and side effects match an oracle
```

A useful suite varies the scope of that experiment:

TEXT

small pure tests	-> fast logic feedback
component/integration	-> database, serialization, framework wiring
contract tests	-> producer/consumer compatibility
end-to-end tests	-> a few critical deployed journeys
load/security tests	-> nonfunctional limits and abuse behavior
production telemetry	-> actual environment validation

The build is a function from reviewed source plus pinned inputs to an artifact and evidence. Hidden machine state, changing dependency metadata, undeclared tools, timezone, locale, or network downloads make that function unstable.

Specification boundary: The Java compiler and `--release` option define language/API targeting behavior for supported releases, but Maven, Gradle, JUnit, analysis plugins, coverage tools, and dependency resolution are separate tools. Their defaults and configuration models are version-sensitive.

51.4 Core terminology

- **Test oracle:** Rule that decides whether observed behavior is correct.
- **Fixture:** Controlled state and data used by a test.
- **Unit test:** Test of a small behavior with in-process collaborators controlled.
- **Integration test:** Test of interaction with a real boundary or multiple configured components.
- **Contract test:** Test that a provider and consumer agree on a boundary schema/behavior.
- **Test double:** Replacement collaborator, including fake, stub, spy, or mock.
- **Fake:** Working simplified implementation, such as an in-memory repository.
- **Stub:** Supplies predetermined responses.
- **Mock:** Verifies configured interactions.
- **Flaky test:** Test whose result varies without a relevant product change.
- **Hermetic build/test:** Runs from declared inputs without relying on uncontrolled external state.
- **Toolchain:** Selected JDK used to compile, test, or run build tasks.
- **Transitive dependency:** Dependency brought in by another dependency.
- **Lock/checksum:** Mechanism constraining resolved versions or artifact integrity.
- **SBOM:** Software bill of materials describing shipped components.

51.5 Detailed mechanics

Test behavior, not implementation shape

Assert public outcomes, emitted records, committed state, and meaningful collaborator contracts. Avoid asserting private method calls, exact incidental SQL order, or every getter invocation. Such tests prevent refactoring without protecting users.

Use Arrange-Act-Assert or Given-When-Then to make the causal boundary visible. One test can assert several aspects of one outcome, but avoid multiple unrelated acts. Name the scenario and expected behavior: `expired_token_is_rejected` communicates more than `testValidate2`.

Cover happy paths, boundaries, invalid input, duplicate/retry behavior, and failure recovery. Use equivalence partitions rather than enumerating random examples. Parameterized tests express a behavior table. Property-based tests generate broad input and shrink failures, but properties need meaningful invariants; "does not throw" is often weak.

Determinism

Inject `Clock` rather than calling the current time throughout domain code. Supply a seeded or fake random source when exact identity matters. Use temporary directories supplied by the test framework. Set explicit charset, locale, timezone, and ordering when they affect output.

Do not use arbitrary sleeps to coordinate concurrent tests. Use latches, barriers, futures with bounded waits, controllable schedulers, or event probes. A test that passes 1,000 times does not prove a data race absent. Use concurrency stress/model-checking tools where appropriate and retain a simple invariant test.

Isolate mutable state. Static caches, singleton clients, thread locals, environment variables, and database rows can leak between tests. Parallel test execution magnifies hidden sharing. Every test should either own state or use namespaced fixtures and cleanup that survives failure.

Test doubles and boundaries

Use a fake when stateful behavior matters, a stub for a simple response, and a mock when the interaction itself is the contract. Mocking a class you own can reveal that its API is awkward; mocking a vendor SDK throughout the domain spreads vendor details.

Do not mock the database to prove SQL, transaction, constraint, type, or isolation behavior. Run integration tests against the same database engine and compatible version. An in-memory substitute may implement different SQL and concurrency semantics. Containerized test dependencies improve realism but add image, startup, platform, and supply-chain concerns.

Consumer-driven contract tests can catch incompatible request/response changes before deployment, but they do not prove availability or semantic correctness. Maintain a small end-to-end path through authentication, network, persistence, and messaging for the most critical journeys.

Exception and async tests

Assert the exception type, safe message or error code, causal information, and state after failure. A test that only expects "some exception" accepts too much. For futures and reactive/asynchronous work, await with a bounded deadline and assert both result and cancellation/failure propagation.

Test retry logic with a fake sleeper/scheduler rather than wall-clock delay. Assert maximum attempts, backoff sequence, idempotency identity, and non-retryable failure. Test that resources close after exceptions using a small instrumented fake or integration metric.

Coverage, mutation, and static evidence

Line or branch coverage shows executed structure, not assertion quality. A high percentage can coexist with no meaningful oracle. Use coverage to find untested risk, not as a universal quality score.

Mutation testing changes operations and asks whether tests fail. Surviving meaningful mutations expose weak assertions, but equivalent mutations and runtime cost require triage. Apply it to critical pure logic rather than every generated accessor.

Static analysis complements tests by exploring paths without running inputs. Useful categories include nullness, resource leaks, ignored return values, concurrency annotations, insecure APIs, bug patterns, API compatibility, style, and architecture dependency rules. Enable compiler lint checks and treat new serious findings as failures. Suppress narrowly with a reason and scope; a global exclusion turns evidence off.

Tooling note: SpotBugs, Error Prone, NullAway, Checkstyle, PMD, JaCoCo, mutation tools, and architecture-test libraries have distinct models and false positives. Pin versions and understand whether a rule is sound, heuristic, source-level, or bytecode-level before making it a gate.

Build reproducibility and Java targeting

Use the Maven or Gradle wrapper so developers and CI invoke a reviewed tool version. Select JDK toolchains explicitly. Compiling on JDK 21 with source compatibility 17 alone can accidentally call Java 21 APIs. Use the compiler's `--release 17` mechanism, through the build tool, when the artifact must run on Java 17.

Pin plugin versions. Avoid dynamic dependency versions and mutable artifact repositories. Verify checksums/signatures according to organizational policy. Reproducible archives also require stable timestamps, file order, metadata, and generated content. Byte-for-byte reproducibility is stronger than "the same source compiled successfully."

Separate fast checks from slower integration, contract, and performance stages while keeping one release provenance chain. Generated sources, annotation processors, and code generators are build dependencies with executable power. Pin and review them like runtime libraries.

Dependency resolution and governance

Declare direct dependencies explicitly rather than relying on a transitive one by accident. Use scopes/configurations correctly so test tools do not ship in runtime artifacts and compile-only APIs are present where needed. Inspect the resolved graph, not only the build file.

Maven and Gradle resolve version conflicts differently and behavior changes with plugins/configuration. A BOM or platform aligns a tested family but does not prove compatibility with every application dependency. Locking improves repeatability; it also means update automation must deliberately refresh and validate locks.

For each shipped component consider:

- origin and artifact integrity;
- supported version and release cadence;
- known vulnerabilities and exploitability in the application;
- license and distribution obligations;
- transitive footprint and duplicate functionality;
- Java baseline, native code, reflection, and framework compatibility; and
- maintenance or replacement plan.

A vulnerability scanner match is a triage input. Confirm the actual artifact, affected version, reachable feature, deployment exposure, compensating control, and vendor fix. Do not ignore a critical issue because no public exploit is known, and do not upgrade blindly without compatibility tests.

CI pipeline design

A practical pipeline runs formatting/checks, compilation, unit tests, static analysis, packaging, integration/contract tests, dependency and secret scans, artifact signing/attestation, and selected deployment tests. Order cheap high-signal failures early. Run from a clean checkout with least-privilege credentials.

Cache immutable artifacts using keys that include relevant locks and tool versions. A cache must accelerate resolution, not alter it. Publish one promoted artifact rather than rebuilding independently for each environment.

TRACE IT | 51.6 Worked Java example

This invitation policy injects time, making expiry tests deterministic:

JAVA (continued 1/2)

```
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneOffset;
import org.junit.jupiter.api.Test;

record Invitation(Instant expiresAt) {
    boolean isValidAt(Instant instant) {
        return instant.isBefore(expiresAt);
    }
}

final class InvitationService {
    private final Clock clock;
    private final Duration lifetime;

    InvitationService(Clock clock, Duration lifetime) {
        this.clock = java.util.Objects.requireNonNull(clock);
        if (lifetime == null || lifetime.isZero() || lifetime.isNegative()) {
            throw new IllegalArgumentException("lifetime must be positive");
        }
        this.lifetime = lifetime;
    }
}
```

JAVA (continued 2/2)

```
}

    Invitation create() {
        return new Invitation(clock.instant().plus(lifetime));
    }
}

final class InvitationServiceTest {
    private static final Instant NOW = Instant.parse("2026-01-01T12:00:00Z");
    private static final Clock CLOCK = Clock.fixed(NOW, ZoneOffset.UTC);

    @Test
    void invitation_is_valid_before_expiry_but_not_at_expiry() {
        Invitation invitation = new InvitationService(CLOCK, Duration.ofMinutes(15))
            .create();

        assertTrue(invitation.isValidAt(NOW.plusSeconds(899)));
        assertFalse(invitation.isValidAt(NOW.plusSeconds(900)));
    }

    @Test
    void non_positive_lifetime_is_rejected() {
        assertThrows(IllegalArgumentException.class,
            () -> new InvitationService(CLOCK, Duration.ZERO));
    }
}
```


The snippet uses JUnit Jupiter APIs and needs a pinned JUnit dependency in the test configuration. It tests boundary semantics at exactly the expiry instant without sleeping.

TRACE IT | 51.7 Execution or memory walkthrough

The fixed clock always returns noon. `create` adds 15 minutes, producing an immutable invitation expiring at 12:15. At 12:14:59 the test instant is strictly before expiry, so validity is true. At exactly 12:15 the strict comparison is false. The test documents a half-open validity interval rather than leaving `<=` versus `<` implicit.

The second test never constructs invalid service state. It checks the public constructor boundary and exact exception type. No mock verifies that `clock.instant()` was called once; call count is an implementation detail unless the contract requires one time snapshot.

Memory and threads are local to each test. The static fixed clock and instant are immutable. If tests changed JVM default timezone instead, parallel tests could interfere. Explicit UTC data avoids that global-state dependency.

51.8 Complexity and performance

Test feedback time is roughly:

TEXT

```
sum(test execution + fixture setup + dependency startup)
divided by safe parallelism, plus scheduling and build overhead
```

Unit tests should normally be milliseconds or less, while real database startup and migration dominate integration stages. Reuse can reduce cost but risks state leakage. Prefer suite-level infrastructure with per-test transaction/schema/namespace isolation when behavior remains realistic.

Build complexity grows with modules, annotation processing, generated code, dependency graph, and cache misses. Parallelism helps independent tasks until CPU, memory, disk, ports, database connections, or test isolation becomes limiting.

Evidence	Strength	Blind spot
unit tests	precise logic and edge cases	real wiring and infrastructure
integration tests	driver/framework/database behavior	full deployment/network path
contract tests	boundary compatibility	provider correctness/availability
end-to-end	critical real journey	slow diagnosis and combinatorial coverage
static analysis	broad path/pattern scan	runtime/environment behavior
coverage	executed code map	oracle quality
mutation	assertion sensitivity	cost/equivalent changes

WATCH OUT | 51.9 Edge cases and common mistakes

- Asserting private methods or exact call sequences instead of behavior.
- Mocking the database and claiming transaction or SQL correctness.
- Using sleeps, current time, default timezone, unordered collections, or unseeded randomness.
- Sharing ports, rows, files, static caches, or environment state across parallel tests.
- Catching an exception in a test but never failing when no exception occurs.
- Treating coverage percentage as correctness.
- Quarantining flaky tests indefinitely without ownership and deadline.
- Running every test as an end-to-end test and producing slow, ambiguous failures.
- Compiling with a new JDK's APIs while claiming an older runtime target.
- Omitting plugin and annotation-processor versions.
- Depending accidentally on a transitive library.
- Trusting a BOM, lock file, or vulnerability scanner as proof of safety.
- Rebuilding different artifacts for staging and production.
- Placing production credentials or personal data in fixtures, logs, or test reports.

51.10 Production engineering notes

Give flaky tests the urgency of production defects. Capture seed, schedule, environment, and artifacts; assign an owner; fix root cause or temporarily quarantine with a deadline and visible risk. Blind retries hide nondeterminism and inflate CI cost.

Keep tests close to risk. Database migrations need forward/backward and realistic-volume tests. Serialization needs golden compatibility fixtures. Idempotent message handling needs duplicate and crash-window tests. Security controls need negative and abuse cases. Performance-sensitive changes need controlled benchmarks, not fixed nanosecond assertions on shared CI.

Use dependency update automation to open small reviewed changes with release notes, resolved graph diff, tests, and rollback plan. Generate an SBOM from the shipped artifact, not only declared dependencies. Scan container base images and build plugins as well as Java libraries.

Protect the build system: least-privilege tokens, isolated untrusted pull-request jobs, approved repositories, checksum verification, no secret echo, and signed/promoted artifacts. A compromised build dependency executes with CI authority.

TRY IT | 51.11 Interview questions and model answers

What should you unit test versus integration test?

Unit-test domain decisions and boundaries with controlled collaborators. Integration-test behavior owned by the database, driver, serializer, framework configuration, or network protocol. Use a few end-to-end tests for critical deployed journeys.

When is a mock appropriate?

When interaction with a collaborator is itself the contract or a difficult boundary must return a controlled result. Prefer state/output assertions and fakes when possible; avoid mocking implementation details.

How do you make time-dependent tests reliable?

Inject `Clock` or a domain time source, use fixed instants and explicit zones, and test just before, at, and after boundaries without sleeping.

What does `--release 17` do when compiling on JDK 21?

It selects the supported Java 17 language/API target model so code cannot accidentally link against newer standard APIs, while generating the corresponding class-file target. It does not validate third-party runtime compatibility.

How do you manage dependency vulnerabilities?

Inventory the shipped graph, verify the affected artifact/version and reachable feature, assess exposure and controls, upgrade or mitigate promptly, run compatibility tests, and document exceptions with expiration.

Why are reproducible builds important?

They make artifacts traceable to reviewed inputs, support verification and incident response, and prevent hidden machine state or mutable dependencies from changing what ships.

TRY IT | 51.12 Exercises

- 1 Refactor a test using `Thread.sleep(1_000)` into one using a fake scheduler or synchronization primitive.
- 2 Design tests for a money transfer: pure invariant tests, real database transaction tests, duplicate-command tests, and one end-to-end path.
- 3 Write a property for a sorting function that is stronger than "output has the same size."
- 4 Configure a Java 21 build toolchain that produces a Java 17-compatible library and explain third-party dependency checks still needed.
- 5 Inspect a hypothetical resolved graph with two logging API versions and propose an alignment and verification plan.
- 6 Create a CI stage order that gives fast feedback while protecting release provenance and secrets.

RECAP | 51.13 Chapter summary

Testing is a layered evidence system. Deterministic unit tests protect domain behavior, integration tests validate real boundaries, contract tests protect schemas, and a few end-to-end tests validate critical journeys. Build wrappers, toolchains, release targeting, pinned plugins, locks, checksums, and promoted artifacts make delivery reproducible. Static analysis and coverage guide risk but do not replace oracles. Dependency governance must inspect the shipped transitive graph, security, licensing, origin, and compatibility.

TRY IT | 51.14 Revision checklist

- ☐ I select test scope from the behavior and failure risk.
- ☐ I assert public outcomes rather than private implementation steps.
- ☐ I control clock, randomness, locale, timezone, files, ports, and concurrency.
- ☐ I know when to use a fake, stub, mock, real database, or contract test.
- ☐ I interpret coverage, mutation, and static findings as evidence, not proof.
- ☐ I use wrappers, pinned plugins, explicit toolchains, and `--release` targeting.
- ☐ I inspect and lock the resolved dependency graph and generate an artifact SBOM.
- ☐ I treat flaky tests and build-system security as production reliability concerns.

SDE-2 CORE CHAPTER

Chapter 52 - Secure and Reliable Java

52.1 Learning objectives

By the end of this chapter, you should be able to:

- threat-model Java service assets, actors, trust boundaries, and abuse cases;
- validate input and prevent injection, unsafe deserialization, path, XML, SSRF, and resource-exhaustion defects;
- apply authentication, authorization, secrets, cryptography, and logging at appropriate boundaries;
- design deadlines, bounded retries, idempotency, bulkheads, backpressure, and graceful degradation;
- connect security and reliability through least privilege, bounded work, safe failure, and recovery; and
- present a practical secure-development and incident-readiness checklist for Java 17 and Java 21 services.

WHY IT WORKS | 52.2 Why this matters at SDE-2

Security and reliability fail at assumptions: a field was "internal," a timeout meant rollback, a queue would stay small, a URL was trusted after parsing, or a dependency was safe because it was popular. Attackers deliberately exercise edge cases that normal traffic rarely reaches, and outages turn ordinary retries and logs into amplifiers.

SDE-2 engineers should build controls into API and system design rather than bolt them onto controllers. They must understand that validation is context-specific, authorization belongs on every protected action, cryptography requires lifecycle and key management, and graceful degradation cannot violate money, privacy, or integrity rules.

WHY IT WORKS | 52.3 First-principles model

Threat modeling begins with a data-flow view:

TEXT

```
external actor -> network boundary -> parser -> application policy
                -> database/cache -> message -> downstream service
```

At each crossing ask:

- 1 What assets and invariants matter?
- 2 Who can supply or observe data?
- 3 How can identity, confidentiality, integrity, availability, or auditability fail?

- 4 Which prevention, detection, response, and recovery controls apply?
- 5 What residual risk and operational signal remain?

Reliability uses a similar model. Work consumes finite CPU, memory, threads, connections, file descriptors, queue slots, and downstream capacity. Every admission path needs a bound and every remote effect needs an outcome model.

TEXT

```
safe service = validated authority + bounded work + explicit deadlines
              + idempotent recovery + observable state + tested rollback
```

Specification boundary: Java supplies type safety, memory safety for ordinary managed code, cryptographic and networking APIs, and runtime checks. It does not make application authorization, parsers, native code, reflection, dependencies, secrets, SQL, files, or distributed protocols automatically safe. Provider algorithms and TLS/JCA defaults vary by JDK/vendor/version.

52.4 Core terminology

- **Asset:** Data, capability, availability, money, identity, or reputation requiring protection.
- **Trust boundary:** Point where data or authority crosses between different trust assumptions.
- **Authentication:** Establishing a principal's identity.
- **Authorization:** Deciding whether that principal may perform one action on one resource.
- **Least privilege:** Granting only capabilities required for a limited purpose and duration.
- **Defense in depth:** Independent controls that limit failure of one layer.
- **Injection:** Untrusted data interpreted as commands or structure in another language.
- **SSRF:** Server-side request forgery that makes a service access unintended network resources.
- **Deserialization:** Converting external representation into in-memory values or objects.
- **Deadline:** Absolute latest completion time propagated across calls.
- **Backpressure:** Slowing, rejecting, or shedding producers when capacity is exhausted.
- **Bulkhead:** Resource partition that limits one workload's blast radius.
- **Circuit breaker:** State machine that temporarily rejects calls after evidence of dependency failure.

52.5 Detailed mechanics

Validate by context and bound work

Decode using an explicit charset and media type, enforce total bytes and nesting before expensive allocation, then validate type, range, length, format, and cross-field domain rules. Prefer allow-lists and typed parsers. A string safe as display text is not automatically safe in SQL, HTML, shell, LDAP, a file path, a regular expression, or a log.

Parameterize SQL values. Avoid executing operating-system commands; when unavoidable, pass an argument list to a fixed executable rather than building a shell string, validate every argument, set a working directory/environment deliberately, and impose timeout/output limits.

Regular expressions can consume superlinear CPU, so bound input and choose safe patterns. Compressed uploads need entry, expanded-byte, nesting, path, and ratio limits. **Content-Length** is not a sufficient bound.

Paths, URLs, XML, and serialization

Normalize paths to reject simple traversal, but remember symlinks and validation-use races. Store user content under generated names and least-privilege roots. Archive extraction validates every final destination and refuses links or special files unless explicitly supported.

For outbound URLs, strictly allow schemes, hosts, ports, and paths; account for private/link-local/loopback addresses, redirects, and DNS rebinding. Network egress controls provide an independent layer. Blocking one textual hostname is not SSRF prevention.

Disable XML external entities and external DTD/schema access unless narrowly required. Defaults vary, so test the actual parser with malicious fixtures.

Never use native Java deserialization for untrusted data. Isolate unavoidable legacy use with allow-list and graph limits. Explicit schemas still need polymorphism, size, duplicate-field, and version policies.

Authentication and authorization

Authenticate at a verified boundary, then authorize each operation and resource. Resolve tenant and ownership from trusted server-side state, not solely from request fields.

Use short-lived, narrowly scoped credentials and validate signature, issuer, audience, time, and key rotation through the protocol library. Enforce application-level authorization on user, administrative, and background paths, not only in a UI or gateway.

Secrets and cryptography

Keep secrets out of source, build logs, command lines, heap dumps, URLs, and ordinary metrics. Obtain them from an approved secret system, grant least privilege, rotate, audit access, and design clients to refresh without full outages. Environment variables can be exposed by process diagnostics and are not a universal secure store.

Use maintained protocols and approved JCA/JCE libraries. Do not invent encryption, nonce, password-hashing, signature, or certificate rules. Passwords need a salted, work-factor KDF selected by current policy. Random tokens need **SecureRandom**; UUID uniqueness does not automatically provide secret unpredictability.

Authenticated encryption requires unique nonces per key and verified tags before plaintext use. Keys need rotation, revocation, and access separation. Never install a "trust all" TLS manager or disable hostname verification in production.

Vendor boundary: Available JCA providers, algorithm names, key-store support, TLS versions/cipher defaults, certificate revocation, and hardware acceleration differ by distribution and configuration. Pin policy and test handshakes on every supported runtime.

Logging, errors, and audit

Log events and identifiers needed for diagnosis, not raw request bodies, access tokens, passwords, card data, session cookies, encryption keys, or broad object `toString` output. Sanitize control characters to prevent log forging. Bound message length and metric-label cardinality.

Return stable public errors with minimal detail and preserve restricted causal context internally. Keep access-controlled audit events separate from debug logs when required.

Deadlines and bounded retry

Propagate an absolute deadline so each layer sees remaining time. Configure connection acquisition, connect, read/write, database statement, and queue waits within it. Cancellation must release resources. A timeout is an unknown remote outcome, not proof of rollback.

Wall clocks can jump and hosts can disagree. Propagate an absolute deadline for cross-process policy, account for clock skew, and use a monotonic elapsed-time source for local budget enforcement when precision matters.

Retry only classified transient failures and idempotent operations, or reuse a stable idempotency key. Bound attempts and elapsed time, use exponential backoff with jitter, and honor server retry hints only within policy. Retrying at multiple stack layers multiplies attempts. Centralize ownership of retries and measure attempts versus logical requests.

Circuit breakers reduce repeated calls to a failing dependency but add a state machine and can reject recovery probes. Bulkheads bound concurrency per dependency or tenant. Rate limits and admission control reject before expensive work. Bounded queues force a documented response: block within a deadline, reject, spill durably, shed lower priority, or degrade safely.

Idempotency and state transitions

An idempotency key identifies one logical command, is scoped to principal/operation, and stores a request fingerprint plus completed or in-progress outcome. Reuse with a different payload must be rejected. Expiry must exceed plausible client retry windows or the old effect can be repeated later.

Database unique constraints or compare-and-set updates enforce concurrency. Consumers record message identity in the same transaction as their business effect. Exactly-once transport wording does not make an email, payment, or arbitrary database side effect exactly once.

Safe degradation and recovery

Degrade only features whose loss does not violate authorization, integrity, billing, or durability.

Recommendations can disappear; authorization normally fails closed.

Liveness should answer whether the process should be restarted; readiness should answer whether it should receive new traffic. Making liveness depend on every downstream can cause restart storms. Exact probe semantics belong to the orchestrator and service design.

On shutdown, stop admission, mark unready, allow bounded in-flight completion, checkpoint or return leased work, close clients/executors, and exit before the platform's hard deadline. Test forced termination and duplicate processing.

Backups are not recovery until restores are tested. Define recovery objectives, key recovery, schema repair, incident authority, evidence preservation, and credential rotation.

Java-specific attack surface

Reflection, agents, annotation processors, JNI, native libraries, and dynamic loading expand capability. Minimize them and treat processors as executable dependencies. The Security Manager is deprecated for removal in Java 17/21; isolate hostile code at process/container and OS boundaries.

TRACE IT | 52.6 Worked Java example

This retrier makes limits, classification, deadline, sleeping, and jitter explicit:

JAVA (continued 1/2)

```
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.TimeoutException;
import java.util.function.LongSupplier;

record RetryPolicy(int maxAttempts, Duration initialBackoff,
    Duration maxBackoff) {
    public RetryPolicy {
        if (maxAttempts < 1 || initialBackoff == null || maxBackoff == null
            || initialBackoff.isNegative() || initialBackoff.isZero()
            || maxBackoff.compareTo(initialBackoff) < 0) {
            throw new IllegalArgumentException("invalid retry policy");
        }
    }
}

@FunctionalInterface interface CheckedOperation<T> {
    T run(Instant deadline) throws Exception;
}

@FunctionalInterface interface RetryClassifier {
    boolean isRetryable(Exception failure);
}
```

JAVA (continued 2/2)

```

}

@FunctionalInterface interface Sleeper {
    void sleep(Duration duration) throws InterruptedException;
}

@FunctionalInterface interface Jitter {
    Duration apply(Duration upperBound);
}

final class BoundedRetrier {
    private final Clock clock;
    private final LongSupplier nanoTime;
    private final Sleeper sleeper;
    private final Jitter jitter;

    BoundedRetrier(Clock clock, LongSupplier nanoTime,
        Sleeper sleeper, Jitter jitter) {
        this.clock = java.util.Objects.requireNonNull(clock);
        this.nanoTime = java.util.Objects.requireNonNull(nanoTime);
        this.sleeper = java.util.Objects.requireNonNull(sleeper);
        this.jitter = java.util.Objects.requireNonNull(jitter);
    }
}

```

The bounded retry loop continues inside the same `BoundedRetrier` class:

JAVA (continued 1/2)

```

<T> T execute(Instant deadline, RetryPolicy policy,
    RetryClassifier classifier, CheckedOperation<T> operation)
    throws Exception {
    Duration backoff = policy.initialBackoff();
    Duration initialBudget = Duration.between(clock.instant(), deadline);
    long startedAtNanos = nanoTime.getAsLong();
    Exception lastFailure = null;

    for (int attempt = 1; attempt <= policy.maxAttempts(); attempt++) {
        Duration remaining = remainingBudget(
            deadline, initialBudget, startedAtNanos);
        if (remaining.isNegative() || remaining.isZero()) {
            TimeoutException timeout = new TimeoutException("deadline exceeded");
            if (lastFailure != null) timeout.addSuppressed(lastFailure);
            throw timeout;
        }
        try {
            return operation.run(deadline);
        } catch (InterruptedException interrupted) {
            Thread.currentThread().interrupt();
            throw interrupted;
        } catch (Exception failure) {
            lastFailure = failure;
            if (!classifier.isRetryable(failure)
                || attempt == policy.maxAttempts()) {

```

JAVA (continued 2/2)

```

        throw failure;
    }
}

remaining = remainingBudget(deadline, initialBudget, startedAtNanos);
Duration cap = min(backoff, remaining);
if (cap.isNegative() || cap.isZero()) {
    TimeoutException timeout = new TimeoutException("no retry budget");
    timeout.addSuppressed(lastFailure);
    throw timeout;
}
Duration delay = jitter.apply(cap);
if (delay == null || delay.isNegative() || delay.compareTo(cap) > 0) {
    throw new IllegalStateException("jitter violated its contract");
}
try {
    sleeper.sleep(delay);
} catch (InterruptedException interrupted) {
    Thread.currentThread().interrupt();
    throw interrupted;
}
backoff = doubleCapped(backoff, policy.maxBackoff());
}
throw new AssertionError("unreachable");
}

```

The duration helps complete `BoundedRetrier`:

JAVA

```

private Duration remainingBudget(Instant deadline,
    Duration initialBudget, long startedAtNanos) {
    Duration wallRemaining = Duration.between(clock.instant(), deadline);
    long elapsedNanos = nanoTime.getAsLong() - startedAtNanos;
    if (elapsedNanos < 0) {
        return Duration.ZERO;
    }
    Duration monotonicRemaining = initialBudget.minusNanos(elapsedNanos);
    return min(wallRemaining, monotonicRemaining);
}

private static Duration min(Duration left, Duration right) {
    return left.compareTo(right) <= 0 ? left : right;
}

private static Duration doubleCapped(Duration value, Duration maximum) {
    return value.compareTo(maximum.dividedBy(2)) > 0
        ? maximum : value.multipliedBy(2);
}
}

```

A production constructor can inject `System::nanoTime` as the monotonic ticker; tests inject a deterministic ticker. The wrapper stops when either the propagated wall-clock deadline or the original monotonic elapsed-time budget is exhausted, so a backward wall-clock adjustment cannot extend retries. A production jitter implementation might choose a random duration between zero and the cap; tests inject exact delays. The operation receives the absolute deadline and must apply its own per-call timeouts. The wrapper cannot interrupt an arbitrary blocking API merely because either clock reaches the deadline.

TRACE IT | 52.7 Execution or memory walkthrough

Assume `maxAttempts = 3`, initial backoff 100 ms, maximum 1 second, and five seconds remain. The first transient failure is classified retryable. Jitter returns 60 ms, so the sleeper waits 60 ms. The next cap is 200 ms. If the second call succeeds, execution returns after two attempts.

If failure is an authentication rejection, the classifier returns false and no retry occurs. If the thread is interrupted during the operation or sleep, the retrier restores the interrupt flag and propagates interruption rather than treating cancellation as transient failure.

If only 40 ms remains after a failure under either clock, the backoff cap becomes 40 ms; jitter cannot exceed it. The next operation still receives the same absolute deadline. A client with a 500 ms socket timeout would violate that deadline, so the operation must derive its timeout from remaining time.

The retrier holds only one last exception, policy values, and local durations: $O(1)$ state. The remote side still needs an idempotency key because the first timed-out attempt may have committed.

52.8 Complexity and performance

With at most a attempts, local control work is $O(a)$ and state is $O(1)$. Total worst-case delay is bounded by the deadline and attempt limit, not merely by the backoff sum. Without limits, retries can grow traffic by a multiplier at every layer:

TEXT

```
3 gateway attempts x 3 service attempts x 3 client attempts = 27 calls
```

Security bounds turn attacker-controlled dimensions into configured limits:

Dimension	Unbounded risk	Control
request bytes/nesting	heap/CPU exhaustion	parser and transport limits
regex/input length	CPU exhaustion	safe pattern plus length bound
queue/concurrency	memory and downstream collapse	admission and bulkhead
decompression	disk/heap explosion	expanded-byte and ratio limits
retries	outage amplification	classifier, jitter, attempts, deadline
metric labels/logs	memory/storage/cardinality	allow-list and truncation

Cryptographic cost is intentional. Password KDF parameters consume CPU/memory to resist guessing and must be capacity-tested. Do not lower them reactively without a security decision; protect login capacity with rate limiting and dedicated resource budgets.

WATCH OUT | 52.9 Edge cases and common mistakes

- Validating a string once and reusing it in a different output context.
- Trusting client-supplied tenant/resource IDs after authentication without authorization.
- Concatenating SQL, shell, path, URL, LDAP, or log structure from untrusted data.
- Allowing URL redirects or DNS resolution to bypass an SSRF host check.
- Enabling XML external entities or broad polymorphic deserialization.
- Treating encryption without authentication as integrity protection.
- Reusing an AEAD nonce or storing keys beside ciphertext with identical access.
- Logging tokens, passwords, request bodies, secrets, or diagnostic dumps without controls.
- Enforcing elapsed retry budgets only with an adjustable wall clock and ignoring clock skew.
- Retrying permanent failures, interruption, or non-idempotent commands.
- Implementing retries at several layers without a total budget.
- Using unbounded queues or cached thread pools to absorb overload.
- Failing open when authorization, billing, or integrity dependencies fail.
- Making liveness depend on a downstream service and causing restart storms.
- Running untrusted code under the deprecated Security Manager as a sandbox.
- Assuming a successful backup job proves restoration.

52.10 Production engineering notes

Maintain threat models and owners for controls, alerts, keys, exceptions, and recovery. Review changes involving authorization, parsing, cryptography, file/network access, native code, and serialization.

Default to denial, bounded work, safe parsers, verified TLS, and no production debug surface. Make overrides narrow, audited, and temporary.

Measure logical requests, attempts, rejection, queue age, circuit state, timeout phase, authorization decisions, and dependency latency without leaking secrets or high-cardinality IDs.

Practice restores, outages, key/certificate rotation, duplicates, overload, and shutdown. Runbooks cover containment, evidence, credentials, communication, and rollback.

TRY IT | 52.11 Interview questions and model answers**How do you secure a new endpoint?**

Define assets and actors, authenticate the principal, authorize the action on the resource, bound and validate parsing, parameterize downstream queries, set deadline and capacity, redact observability, make side effects idempotent, and test abuse and failure paths.

What is the difference between authentication and authorization?

Authentication establishes who the caller is. Authorization decides whether that caller can perform this action on this resource under current policy. A valid identity can still be unauthorized.

Why are retries dangerous?

They amplify load, increase latency, and can duplicate side effects. Retry only transient failures within one bounded owner and deadline, with jitter and idempotency or reconciliation.

How would you prevent SSRF?

Use strict URL parsing and allow-lists, validate scheme/host/port, control DNS/private addresses and redirects, enforce egress network policy, bound response/time, and avoid accepting arbitrary URLs when an identifier can be used.

Should a service fail open or closed?

It depends on the invariant. Authorization, integrity, and payment controls normally fail closed. Optional presentation features may degrade. Make the decision explicit and test dependency failure.

What is defense in depth?

Independent controls limit one another's failure: application URL policy plus network egress restrictions, prepared SQL plus least-privilege database credentials, and parser limits plus request admission.

How do you handle an unknown outcome after timeout?

Reuse a stable idempotency key and query status or retry the same logical operation safely. Do not create a new command identity and assume the first attempt rolled back.

TRY IT | 52.12 Exercises

- 1 Threat-model a file-upload endpoint: include parser, archive, path, malware, authorization, quotas, storage, and download behavior.
- 2 Design idempotency storage for a payment command, including payload fingerprint, in-progress state, response replay, and expiry.
- 3 Calculate worst-case calls when three layers each retry twice after the original attempt. Redesign with one retry owner.
- 4 Extend the retrier test design with a fake clock, sleeper, jitter, transient failure, deadline exhaustion, and interruption.
- 5 Review an outbound webhook feature for SSRF, secret signing, timeouts, retries, redirect policy, and tenant isolation.
- 6 Write graceful-shutdown steps for a message consumer that leases work and may receive duplicate delivery.

RECAP | 52.13 Chapter summary

Secure and reliable Java systems make trust and capacity explicit. Validate data for its destination context, authorize every protected resource action, use approved cryptography and secret lifecycle, and restrict dangerous parsing and dynamic capabilities. Bound bytes, time, concurrency, queues, retries, and cardinality. Treat timeouts as unknown outcomes and use idempotency for recovery. Layer preventive controls with telemetry, graceful failure, least privilege, tested restore, and incident practice.

TRY IT | 52.14 Revision checklist

- ☐ I can identify assets, actors, trust boundaries, abuse cases, and residual risk.
- ☐ I validate input by context and bound bytes, depth, CPU, concurrency, and output.
- ☐ I can prevent SQL/shell/path/XML/deserialization and SSRF classes of defects.
- ☐ I separate authentication from resource-level authorization.
- ☐ I use approved secret, password, encryption, nonce, key, and TLS practices.
- ☐ I propagate deadlines and bound one retry owner with classification, jitter, and idempotency.
- ☐ I design bulkheads, backpressure, safe degradation, and graceful shutdown.
- ☐ I protect logs, metrics, diagnostics, dependencies, builds, backups, and incident evidence.

SDE-2 CORE CHAPTER

Chapter 53 - SDE-2 Java Interview Question Bank

53.1 Learning objectives

- Answer Java questions with definition, mechanism, guarantee, trade-off, and production relevance.
- Recognize follow-up probes that distinguish memorization from working knowledge.
- Detect misconception traps involving specifications, implementation details, complexity, and concurrency.
- Practice across JVM internals, language engineering, collections, concurrency, performance, DSA, backend design, testing, and security.
- Convert weak answers into short, precise SDE-2 model answers.

WHY IT WORKS | 53.2 Why this matters at SDE-2

An SDE-2 interview rarely stops at a definition. "HashMap is $O(1)$ " invites collision, resizing, equality, and mutability probes. "Volatile gives visibility" invites a lost-update example. "The object is on the heap" invites escape analysis and the distinction between Java semantics and HotSpot optimization.

The interviewer is evaluating reasoning under uncertainty. A strong candidate states the contract first, labels implementation-specific details, tests assumptions, and connects the answer to a production decision. A weak candidate recites internal trivia without knowing whether it is guaranteed.

This bank is organized for active recall. Cover the model answer, speak for 60 to 120 seconds, then answer the follow-up before reading. Mark a question green only if the explanation is correct, bounded, and usable without prompting.

WHY IT WORKS | 53.3 First-principles model

Use a five-part answer shape:

TEXT

1. Definition: What is it?
2. Contract: What does Java guarantee?
3. Mechanism: How is it commonly implemented?
4. Trade-off: When does it help or hurt?
5. Evidence/example: How would I prove, test, or diagnose it?

Do not force all five parts into the opening sentence. Lead with the direct answer, then expand according to the probe. For comparison questions, establish dimensions before choosing: correctness, complexity, contention, ordering, memory, startup, tail latency, maintainability, and operational evidence.

For coding questions, use a parallel loop:

TEXT

clarify contract -> examples -> baseline -> invariant -> algorithm
-> complexity -> implement -> test edges -> discuss alternatives

Specification boundary: Interview answers should distinguish JLS/JVMS and Java API guarantees from HotSpot, OpenJDK library, operating-system, and hardware behavior. An implementation detail is useful only when labeled with version and purpose.

53.4 Core terminology

- **Opening answer:** Direct 20-to-40-second response before deeper probes.
- **Follow-up probe:** Question testing mechanism, edge cases, alternatives, or production application.
- **Misconception trap:** Plausible but false simplification, such as "Java passes objects by reference."
- **Invariant:** Property that must remain true throughout an algorithm or state transition.
- **Linearization point:** Instant at which a concurrent operation appears to take effect.
- **Compatibility:** Source, binary, class-file, runtime, or behavioral compatibility, depending on context.
- **Complexity qualification:** Expected, amortized, worst-case, or implementation-sensitive cost.
- **Readiness evidence:** Timed performance, explanation quality, test coverage, and corrected-error history rather than confidence alone.

53.5 Detailed mechanics

The following bank includes an opening model answer, one likely follow-up, and a trap. The answers are intentionally concise; a mock interviewer should ask for examples and counterexamples.

JVM and execution

1. What is the difference among JDK, JRE, and JVM?

Model answer: The JVM loads and executes class files. A runtime combines a JVM with standard modules, native support, and resources. A JDK contains a runtime plus tools such as `javac`, `jar`, `javadoc`, and diagnostics. Modern deployments may use a custom `jlink` runtime instead of a separately packaged historical JRE.

Follow-up: Why can code compiled on JDK 21 fail on Java 17? Trap: Saying a JRE always exists as a separate vendor download.

2. Trace Java source to CPU execution.

Model answer: `javac` lexes, parses, resolves names, type-checks, and emits class files. At runtime, loaders define classes; the JVM verifies, prepares, resolves, and initializes them. An execution engine interprets, JIT-compiles, ahead-of-time executes, or combines strategies before native instructions run.

Follow-up: Where can `NoSuchMethodError` occur? Trap: Calling Java only interpreted.

3. What is bytecode?

Model answer: Bytecode is the JVM's platform-neutral instruction representation stored in class-file method structures. It uses an operand-stack model and symbolic constant-pool references. It is not CPU machine code, and one source construct need not map to one bytecode sequence.

Follow-up: Use `javap -c` to explain `invokevirtual`. Trap: Calling the class-file constant pool the same as the string pool.

4. What happens during class loading?

Model answer: Loading finds bytes and creates a loader-scoped definition. Linking verifies safety, prepares static storage with defaults, and resolves symbolic references, possibly lazily. Initialization later executes static initializers on first required active use under a synchronized once-only protocol.

Follow-up: Does `SomeType.class` initialize the class? Trap: Saying explicit static values are assigned during preparation.

5. Why can two classes with the same name fail a cast?

Model answer: Runtime class identity includes the binary name and defining class loader. Two loaders that independently define `com.example.Message` produce different types. Plugin systems normally place shared contracts in a parent loader and implementations in isolated children.

Follow-up: Describe a class-loader leak. Trap: Assuming byte-for-byte identical definitions are assignment compatible.

6. Explain JVM runtime data areas.

Model answer: The JVM model has a shared heap for objects, shared per-class/method structures and runtime constant pools, and per-thread PCs, JVM stacks, frames, and native execution support. HotSpot structures such as Metaspace, TLABs, and code cache implement or extend this model but are not JVMs region names.

Follow-up: Why can process memory exceed `-Xmx`? Trap: Putting every static-referenced object in Metaspace.

7. What does a JIT compiler optimize?

Model answer: An adaptive JIT uses profiles to compile hot paths, inline calls, specialize receiver types, eliminate checks, analyze escape, and optimize loops. Guarded assumptions remain correct through fallback and deoptimization. Compilation consumes CPU and code-cache memory.

Follow-up: What is on-stack replacement? Trap: Assuming a source method call always has a physical frame.

8. What is a safepoint?

Model answer: It is an implementation safe state used for VM operations needing coordinated managed state. Relevant threads reach polls or transitions, then the operation runs. Time to safepoint and operation duration are separate; stop-the-world does not automatically mean full GC.

Follow-up: Name a non-GC safepoint operation. Trap: Calling safepoints a JLS feature.

Memory and garbage collection

9. What happens when `new` executes?

Model answer: The class must be ready, storage and identity are obtained, fields receive mandatory defaults, and constructor invocation runs superclass construction, instance initializers, and the body. TLAB allocation, header words, field offsets, and compressed references are HotSpot details.

Follow-up: Can allocation be eliminated? Trap: Claiming Java guarantees stack allocation.

10. Shallow size versus retained size?

Model answer: Shallow size is the storage directly occupied by one object. Retained size is the storage that becomes unreachable if that object is removed, determined by GC-root paths and dominance. Referenced objects do not automatically belong to a parent's retained set.

Follow-up: Why can a small cache entry retain megabytes? Trap: Summing declared field widths as retained size.

11. How does GC identify garbage?

Model answer: A tracing collector starts from VM-known roots and follows strong and other applicable reachability edges. Objects with no relevant root path become reclamation candidates. Unreachable cycles are collectible, unlike naive reference counting.

Follow-up: Name common GC roots. Trap: Saying an object is garbage when a local is assigned null regardless of other paths.

12. Compare strong, soft, weak, and phantom references.

Model answer: Strong references retain normally. Soft references are discretionary memory-sensitive references and poor deterministic cache bounds. Weak references do not retain weakly reachable referents. Phantom references always return null and support post-mortem cleanup coordination through a `ReferenceQueue`.

Follow-up: Why drain a reference queue? Trap: Relying on finalization or phantom references to close resources promptly.

13. Can Java have memory leaks?

Model answer: Yes. GC removes unreachable objects, not reachable objects the application no longer needs. Unbounded caches, listeners, queues, thread locals, and class loaders can retain useless graphs. Compare allocation rate with post-collection live-set and dominator evidence.

Follow-up: Heap leak versus native leak? Trap: Calling every high allocation rate a leak.

14. Compare G1, ZGC, and Parallel GC.

Model answer: Parallel GC prioritizes throughput with parallel stop-the-world work. G1 uses regions, evacuation, concurrent marking, and pause goals for a balance. ZGC performs most work concurrently for very low pauses. Exact capabilities, including generations, depend on JDK version.

Follow-up: Which would you choose for a 99.9 latency SLO? Trap: Selecting from heap size alone without workload evidence.

Java language and API design

15. Is Java pass-by-reference?

Model answer: No. Every argument value is copied into a parameter. For objects, that copied value is a reference, so caller and callee can mutate the same object. Reassigning the parameter cannot reassign the caller's variable.

Follow-up: Demonstrate with an array. Trap: Saying "objects are pass-by-reference" as shorthand.

16. Overloading versus overriding?

Model answer: Overloading selects among methods at compile time using declared types and conversions. Overriding supplies runtime instance dispatch based on the receiver's actual class. Static methods are hidden, not overridden; constructors are neither inherited nor overridden.

Follow-up: Which overload receives `null`? Trap: Treating return type alone as an overload distinction.

17. Interface versus abstract class?

Model answer: An interface defines a multiple-inheritance contract and can have default/static/private behavior but no per-instance constructor state. An abstract class shares state, protected implementation, and construction under single class inheritance. Choose by semantic relationship and evolution needs, not method count.

Follow-up: How do default-method conflicts resolve? Trap: Saying interfaces contain no implementation.

18. Composition versus inheritance?

Model answer: Inheritance creates a substitutability commitment and exposes superclass evolution. Composition delegates to a collaborator and keeps behavior replaceable. Prefer composition unless the subtype preserves the parent's behavioral contract and shared protected implementation is genuinely valuable.

Follow-up: Give a Liskov-substitution violation. Trap: Using inheritance only for code reuse.

19. What makes a class immutable?

Model answer: Its observable state cannot change after construction. Use private final fields, validate construction, prevent `this` escape, make defensive copies of mutable inputs/outputs, and ensure referenced components are immutable or encapsulated. A final reference alone is insufficient.

Follow-up: Are records deeply immutable? Trap: Equating `final` or record with deep immutability.

20. Explain `equals` and `hashCode`.

Model answer: Equal objects must produce the same hash code; unequal objects may collide. Equality should be reflexive, symmetric, transitive, consistent, and false for null. Keys must not change equality-relevant state while in hash collections.

Follow-up: Why can subclass equality break symmetry? Trap: Using database-generated IDs inconsistently across entity lifecycle.

21. Checked versus unchecked exceptions?

Model answer: Checked exceptions must be caught or declared and can express recoverable alternatives callers are expected to handle. Unchecked exceptions often represent violated contracts or failures not locally recoverable. The choice is API design, not severity; preserve causes and avoid broad catch-and-ignore.

Follow-up: When should an exception be translated? Trap: Logging and rethrowing at every layer.

22. What does try-with-resources guarantee?

Model answer: It closes initialized resources in reverse order on normal or abrupt completion. If body and close both throw, the body exception remains primary and close failures are suppressed. Resource ownership must still be clear.

Follow-up: How do you inspect suppressed exceptions? Trap: Assuming close exceptions replace the primary failure.

23. Explain generic type erasure.

Model answer: Java generics are mainly enforced at compile time; type variables are translated to bounds/Object with casts and bridge methods where needed. Parameterized types generally do not have distinct runtime classes, so `new T()` and `instanceof List<String>` are unavailable.

Follow-up: What is heap pollution? Trap: Saying all generic information disappears from metadata and reflection.

24. Explain PECS.

Model answer: For a parameterized structure that produces `T`, use `? extends T`; for one that consumes `T`, use `? super T`. It guides flexible APIs but does not mean a structure cannot both read and write in all designs.

Follow-up: Why can you add null to `List<? extends Number>` but not an Integer? Trap: Treating `List<Integer>` as a subtype of `List<Number>`.

25. What Java 17 and 21 features matter most?

Model answer: Java 17 provides a modern LTS baseline with records, sealed types, and finalized pattern matching for `instanceof`. Java 21 adds virtual threads, record patterns, switch pattern matching, and sequenced collections. Preview features must be labeled separately.

Follow-up: When not to use a record? Trap: Listing preview structured concurrency as final Java 21.

Collections, streams, and I/O

26. ArrayList versus LinkedList?

Model answer: `ArrayList` gives $O(1)$ indexed access, amortized $O(1)$ append, compact storage, and cache locality. `LinkedList` gives $O(1)$ insertion only when an existing node position is already represented by an iterator, but lookup is $O(n)$ and each node costs memory. `ArrayList` is the usual default.

Follow-up: Removing from the front? Trap: Choosing `LinkedList` for arbitrary middle insertion while ignoring traversal.

27. How does HashMap work?

Model answer: It spreads a key hash to choose a table bin, then uses `equals` within collisions. Expected lookup is $O(1)$ under good distribution; resize is $O(n)$, and collision behavior can use trees in current implementations. Capacity and load factor trade memory against collision/resizing.

Follow-up: Why are mutable keys dangerous? Trap: Claiming `hashCode` must be unique.

28. TreeMap versus HashMap?

Model answer: `TreeMap` maintains key order and navigation with $O(\log n)$ operations. `HashMap` offers expected $O(1)$ lookup without ordering. `TreeMap` equality for keys is effectively based on comparator/compare-to zero, so ordering consistency with `equals` matters.

Follow-up: How do range queries work? Trap: Using subtraction in a comparator and risking overflow.

29. What does fail-fast mean?

Model answer: Many ordinary collection iterators detect some unsupported structural modifications and may throw `ConcurrentModificationException` on a best-effort basis. It is a bug detector, not a synchronization guarantee. Concurrent collections define weak or snapshot iteration instead.

Follow-up: Is iteration safe if no exception occurred? Trap: Catching the exception and retrying as concurrency control.

30. Comparable versus Comparator?

Model answer: `Comparable` defines a type's natural order. `Comparator` is an external strategy, allowing multiple orderings and composition. Both must satisfy ordering contracts; sorted sets/maps treat comparison zero as equivalent for membership/key uniqueness.

Follow-up: Handle nulls? Trap: `Comparator` inconsistent/transitivity violations.

31. Stream versus collection?

Model answer: A collection stores elements; a stream is a single-use computation pipeline over a source. Intermediate operations are lazy, terminal operations trigger traversal, and side-effect-free stateless operations compose safely. Parallel streams need splittable work, sufficient size, associative reduction, and execution-context care.

Follow-up: `map` versus `flatMap`? Trap: Reusing a consumed stream.

32. Why must reduction be associative?

Model answer: Parallel partitioning can group operands differently. An associative accumulator/combiner produces the same logical result under regrouping. Floating-point addition is not mathematically associative due to rounding, so exact reproducibility may require sequential or specialized algorithms.

Follow-up: Identity requirements? Trap: Mutating one non-thread-safe container in `forEach` on a parallel stream.

33. Optional best practices?

Model answer: `Optional` is primarily a return type for possibly absent values. Use `map`, `flatMap`, `orElseGet`, and explicit absence semantics. Avoid `Optional` fields/parameters in ordinary domain models unless a framework/contract justifies them; never use `Optional` to hide errors.

Follow-up: `orElse` versus `orElseGet`? Trap: Calling `get()` without proving presence.

34. Buffered I/O versus NIO?

Model answer: Buffering amortizes system calls for stream/channel I/O. NIO.2 adds paths, file operations, channels, buffers, selectors, and asynchronous facilities. A `ByteBuffer` has capacity, position, and limit; `flip` changes from writing into the buffer to reading from it.

Follow-up: Direct buffer trade-offs? Trap: Assuming one `read` or `write` transfers the entire requested amount.

Concurrency

35. What is happens-before?

Model answer: It is the JMM relation ordering memory effects. It comes from program order, volatile write/read, monitor unlock/lock, thread start/join, class initialization, and concurrent-library contracts plus transitivity. Wall-clock ordering or sleep is not enough.

Follow-up: Prove safe publication with volatile. Trap: Explaining only CPU caches.

36. What does synchronized guarantee?

Model answer: Mutual exclusion for the same monitor and memory ordering: unlock happens-before a later lock. It is reentrant and releases on block exit, including exceptions. All conflicting accesses must follow the same guard or another valid protocol.

Follow-up: `wait` versus `sleep`? Trap: Synchronizing readers and writers on different objects.

37. What does volatile guarantee?

Model answer: Volatile access is atomic and globally ordered for that variable; a write happens-before subsequent reads, publishing prior actions. It does not make read-modify-write or multi-field invariants atomic. It fits flags and immutable snapshot references.

Follow-up: Why is `volatile count++` unsafe? Trap: Calling volatile a lightweight universal lock.

38. How does interruption work?

Model answer: Interruption is cooperative cancellation through status. Interruptible waits often throw `InterruptedException` and clear status. Propagate it, or restore status and exit at a boundary. It cannot safely force arbitrary code or every I/O call to stop.

Follow-up: Why restore status? Trap: Catch, log, and continue.

39. ReentrantLock versus synchronized?

Model answer: Both provide exclusion and visibility. `ReentrantLock` adds timed/interruptible acquisition, multiple conditions, and optional fairness but requires `finally` unlock. Use `synchronized` for simple scoped locking; choose explicit features from a requirement, not presumed speed.

Follow-up: Why await in a loop? Trap: Unlocking outside finally.

40. Explain CAS and ABA.

Model answer: CAS installs an update only if current state matches expected; a successful CAS is a linearization point. Retry loops must recompute without side effects. ABA occurs when state changes A-B-A and equality hides history; use a version/stamp or different design.

Follow-up: Is lock-free starvation-free? Trap: Assuming GC eliminates logical ABA.

41. How do you size an executor?

Model answer: Start near cores for CPU-bound work. For blocking work, estimate blocking fraction but cap by downstream capacity, memory, and latency. Use bounded queues, explicit rejection/backpressure, deadlines, metrics, and load tests. An unbounded queue can make maximum threads irrelevant.

Follow-up: Explain same-pool starvation deadlock. Trap: Maximizing threads without considering database connections.

42. `thenApply` versus `thenCompose`?

Model answer: `thenApply` maps `T` to `U`; `thenCompose` maps `T` to a completion stage and flattens it. Non-Async callbacks may run on the completing/attaching thread. Explicit executors isolate blocking and capacity. Recovery stages can accidentally hide failures.

Follow-up: `handle` versus `whenComplete`? Trap: Assuming `CompletableFuture` cancellation interrupts work.

43. Why virtual threads?

Model answer: Java 21 virtual threads make thread-per-task blocking code scalable by unmounting blocked tasks from carrier platform threads. They do not add CPU cores or downstream capacity. Do not pool them; limit scarce resources directly. In Java 21, blocking while pinned in synchronized/native code can occupy carriers.

Follow-up: Migration incident with JDBC? Trap: Removing all concurrency limits.

44. `ConcurrentHashMap` guarantees?

Model answer: It supports thread-safe concurrent operations and atomic methods such as `putIfAbsent`, `compute`, and `merge`. Iteration is weakly consistent, not a whole-map snapshot. Multi-key invariants still require another protocol. Null keys/values are not allowed.

Follow-up: Why is `containsKey` then `put` racy? Trap: Using `size` for a precise admission decision during mutation.

Performance, DSA, design, backend, testing, and security

45. How do you benchmark Java code?

Model answer: Define the decision and metric, use representative data, separate warm-up and measurement, consume results, fork JVMs, control environment, and report distributions. Use JMH for microbenchmarks because it addresses dead-code elimination, constant folding, and harness effects.

Follow-up: Why can `nanoTime` around one loop lie? Trap: Inferring production impact from a nanobenchmark.

46. CPU is high. What do you inspect?

Model answer: Confirm scope and timeline, then separate application, GC, JIT, native, and system CPU. Capture JFR or repeated profiles, correlate hot stacks with traffic and changes, and inspect runnable threads. Thread dumps alone sample stacks but not proportional CPU reliably.

Follow-up: What if CPU is low but latency high? Trap: Tuning GC without evidence.

47. What is the SDE-2 DSA method?

Model answer: Clarify input/output and constraints, run examples, state brute force, identify the invariant/data structure, derive complexity, implement incrementally, and test boundaries. Explain why the algorithm works, not only its pattern name.

Follow-up: How do you recover when stuck? Trap: Coding before clarifying duplicates, overflow, or mutation rules.

48. BFS versus DFS?

Model answer: BFS explores by distance layers and gives shortest path in unweighted graphs, using $O(\text{width})$ queue space. DFS explores depth, supports cycle/topological/component reasoning, and uses $O(\text{depth})$ stack/explicit storage. Both are $O(V+E)$ with adjacency lists.

Follow-up: Why mark visited on enqueue? Trap: Recursive DFS on adversarial depth without stack analysis.

49. What makes a good backend API?

Model answer: Clear contract, validation, stable error model, idempotency where retries occur, pagination/bounds, authorization, observability, and compatibility. Keep transport DTOs separate from mutable persistence concerns and define timeout/cancellation behavior across dependencies.

Follow-up: Design idempotent payment creation. Trap: Treating HTTP retry as harmless without a key/state machine.

50. Explain transaction isolation and Java locks.

Model answer: A Java lock coordinates threads sharing one lock instance. A database transaction coordinates persisted operations across clients under an isolation level. In-process synchronization does not prevent another service instance from changing the row; use database constraints, locking/version checks, and idempotency.

Follow-up: Optimistic locking failure handling? Trap: Holding a Java lock across a remote transaction and calling it distributed consistency.

51. How would you test concurrent code?

Model answer: Prove invariants and happens-before/linearization points, then coordinate actor starts with barriers/latches, use bounded timeouts, record histories, and stress across forks/architectures. jctstress-style outcome tests can expose JMM behaviors. Passing tests do not prove absence of races.

Follow-up: How detect deadlock? Trap: Adding sleep to force order.

52. Unit versus integration versus contract tests?

Model answer: Unit tests isolate deterministic logic and run fast. Integration tests validate boundaries such as database, serialization, and framework wiring. Contract tests verify provider/consumer assumptions across

service evolution. Use the cheapest layer that can catch the failure, with a smaller number of realistic end-to-end tests.

Follow-up: Test transaction rollback? Trap: Mocking the boundary whose semantics are under test.

53. What security checks belong in a Java service?

Model answer: Authenticate identity, authorize the specific resource/action, validate bounded input, parameterize SQL, avoid unsafe deserialization, protect secrets, constrain outbound requests, patch dependencies/JDK, and log security events without sensitive data. Apply least privilege and fail closed.

Follow-up: Prevent SSRF in a URL-fetch endpoint. Trap: Treating validation as only regex syntax.

54. How do you prevent injection?

Model answer: Keep data separate from interpreter syntax: prepared SQL parameters, contextual output encoding, safe command APIs without shell concatenation, and allowlisted structure when identifiers cannot be parameterized. Validation helps policy but escaping alone is context-sensitive and fragile.

Follow-up: Can prepared statements parameterize table names? Trap: Building shell commands from quoted user strings.

TRACE IT | 53.6 Worked Java example

An interviewer asks: "Implement a small bounded LRU cache and discuss concurrency." A focused baseline is:

JAVA

```
import java.util.LinkedHashMap;
import java.util.Map;

public final class LruCache<K, V> {
    private final int capacity;
    private final LinkedHashMap<K, V> entries;

    public LruCache(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity");
        this.capacity = capacity;
        this.entries = new LinkedHashMap<>(16, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
                return size() > LruCache.this.capacity;
            }
        };
    }

    public synchronized V get(K key) {
        return entries.get(key);
    }

    public synchronized void put(K key, V value) {
        entries.put(key, value);
    }

    public synchronized int size() {
        return entries.size();
    }
}
```

`accessOrder=true` means a successful `get` mutates order, so even reads require synchronization. The intrinsic lock gives a simple linearization point per method. This is an in-process cache, not a distributed cache, and it deliberately omits loader coalescing, expiration, metrics, null policy, weight-based capacity, and external callback safety.

TRACE IT | 53.7 Execution or memory walkthrough

For capacity two:

- 1 `put(A, 1)` inserts A. Order is `[A]`.
- 2 `put(B, 2)` inserts B. Order is `[A, B]`, least to most recently used.
- 3 `get(A)` returns 1 and moves A: `[B, A]`.
- 4 `put(C, 3)` temporarily produces `[B, A, C]`.
- 5 `removeEldestEntry` observes size three and removes B, leaving `[A, C]`.

If T1 executes `get(A)` while T2 executes `put(C)`, the same monitor serializes the full `LinkedHashMap` operations. Without synchronization, a `get` can race because access-order maintenance changes linked structure. Returning mutable values still lets callers race on those values; cache thread safety does not make value objects immutable.

An excellent interview discussion adds that eviction callbacks must run after releasing the lock, loading should avoid duplicate work without holding the lock across I/O, and production caches need tested libraries unless the exercise explicitly requires implementation.

53.8 Complexity and performance

Expected `get` and `put` are $O(1)$; eviction is $O(1)$ for one eldest entry. Space is $O(\text{capacity})$, excluding retained key/value graphs. The single lock caps parallel operations and can create contention, but it provides a short proof. Segmenting/sharding can increase concurrency at the cost of approximate global LRU.

For the question bank, track more than answer count. Score each response from 0 to 4:

- 0: no viable answer.
- 1: memorized fragment or materially incorrect.
- 2: correct opening but weak mechanism/edge cases.
- 3: correct, structured, one follow-up handled.
- 4: precise contract, trade-off, production example, and follow-ups.

Require at least 3 on correctness-critical areas: JMM, equality/hashing, exceptions/resources, complexity, transactions, and security. Fast wrong answers are worse than slower bounded reasoning.

WATCH OUT | 53.9 Edge cases and common mistakes

- Starting with five minutes of context instead of answering the question.
- Inventing a guarantee when uncertain; state the stable contract and label the uncertainty.
- Giving Big-O without expected/amortized/worst-case qualification.
- Describing HotSpot object headers or HashMap thresholds as language guarantees.
- Using a framework slogan instead of Java mechanics.
- Optimizing before defining the invariant and constraints.
- Ignoring null, empty, duplicate, overflow, cancellation, and adversarial inputs.
- Saying "thread-safe" without naming the operation/invariant.
- Presenting an incident with no measurement, decision, or outcome.
- Arguing that a data race is safe because it worked on x86.
- Hiding a weak area behind excessive terminology.

53.10 Production engineering notes

Prepare six reusable incident stories: latency, correctness, memory/GC, concurrency, dependency failure, and security/reliability. Each should state scale, signal, hypothesis, evidence, action, trade-off, and measured result. Never disclose employer secrets; normalize identifiers and approximate scale where needed.

For JVM questions, mention the tool that would verify the claim: `javap` for class files, JFR/profile for CPU/allocation/locks, GC logs and heap analysis for memory, thread dumps for blocking, and build/runtime metadata for compatibility. Tool names without an investigation sequence are not enough.

HotSpot note: Questions about TLABs, mark words, C1/C2, safepoint polls, collector defaults, and virtual-thread diagnostics are version-specific. A strong answer says "in mainstream HotSpot on the stated JDK" before explaining them.

TRY IT | 53.11 Interview questions and model answers

Use this twelve-question rapid loop after completing the bank:

- 1 **Explain source to CPU in 90 seconds.** Model: compiler, class file, loading/linking/init, adaptive execution, native code, specification boundary.
- 2 **Diagnose process OOM with stable heap.** Model: inspect container/RSS, stacks, Metaspace, direct buffers, code cache, native libraries, and NMT where available.
- 3 **Design an immutable key.** Model: stable equality/hash, final state, defensive copies, no mutable identity fields.
- 4 **Choose ArrayList or LinkedList.** Model: operation distribution and locality; ArrayList default.
- 5 **Prove volatile publication.** Model: program order, volatile synchronizes-with, transitive happens-before.
- 6 **Fix a shutdown hang.** Model: stop admission, interrupt/close blocking resource, propagate cancellation, join/await deadline, capture survivors.
- 7 **Bound an executor.** Model: downstream capacity, queue, rejection/backpressure, deadlines, metrics.
- 8 **Explain virtual-thread migration risk.** Model: cheap waiting increases concurrency; preserve DB/API bulkheads and diagnose Java 21 pinning.
- 9 **Solve longest unique substring.** Model: sliding window with last-seen indices, $O(n)$ time and $O(\text{character-set})$ space.
- 10 **Design idempotent create.** Model: client key, atomic uniqueness, stored outcome/state machine, request-hash conflict policy.
- 11 **Investigate p99 regression.** Model: compare timeline/distributions, profile, queues/locks/GC/downstream, controlled experiment.
- 12 **Prevent unsafe deserialization.** Model: simple schema-bound formats, type allowlists, validation, dependency patches, least privilege, size/depth limits.

TRY IT | 53.12 Exercises

- 1 Record answers to all 54 questions; score them 0 to 4 and keep the first incorrect sentence for review.
- 2 For ten questions, produce a 30-second, 90-second, and five-minute answer.
- 3 Add two follow-ups and one misconception trap to every question scored below 3.
- 4 Implement and test the LRU cache, then redesign loader coalescing without holding a lock during I/O.
- 5 Run three paired mocks: JVM/concurrency, collections/DSA, and backend/performance/security.
- 6 Build a one-page error log containing the guarantee you confused, a counterexample, and the corrected rule.
- 7 Select five implementation-specific answers and attach an exact JDK/VM version label.

RECAP | 53.13 Chapter summary

SDE-2 answers begin with a direct contract, then explain mechanism, trade-offs, and evidence. The bank spans the execution pipeline, memory, language, collections, concurrency, performance, DSA, backend design, testing, and security because interviews connect these domains. Active recall, follow-up probes, misconception traps, and scored corrections convert knowledge into interview performance.

TRY IT | 53.14 Revision checklist

- ☐ I can answer every bank question in under two minutes before follow-ups.
- ☐ I distinguish Java guarantees from HotSpot and library implementation details.
- ☐ I qualify complexity as expected, amortized, or worst-case.
- ☐ I can prove concurrency answers with happens-before or a linearization point.
- ☐ I connect JVM/performance answers to diagnostic evidence.
- ☐ I have production stories with measured outcomes and clear ownership.
- ☐ I handle coding constraints, edges, complexity, implementation, and tests aloud.
- ☐ No correctness-critical bank category scores below 3.

SDE-2 CORE CHAPTER

Chapter 54 - Eight-Week Study Plan and Mock Interview Loops

54.1 Learning objectives

- Convert the book into an eight-week schedule with daily outputs and weekly readiness gates.
- Balance Java depth, DSA execution, backend design, diagnostics, and behavioral stories.
- Run realistic mock loops and score them consistently.
- Use spaced retrieval and an error log instead of passive rereading.
- Recover from missed days, weak mocks, plateaus, and burnout without abandoning the plan.

WHY IT WORKS | 54.2 Why this matters at SDE-2

Interview preparation fails when study feels productive but never tests retrieval under time. Reading concurrency twice is not the same as proving a happens-before edge aloud. Solving a problem after seeing its pattern is not the same as clarifying, deriving, coding, and testing in 40 minutes. Completing 200 random problems can coexist with weak Java depth and poor communication.

This plan treats preparation as an engineering system. Each week has inputs, outputs, measurements, and a gate. Mocks are feedback, not final exams. The schedule assumes an experienced backend engineer with work obligations and about 15 focused hours per week. It can be compressed or extended without changing the order of learning and verification.

WHY IT WORKS | 54.3 First-principles model

Use a closed feedback loop:

TEXT

```
learn a model
-> retrieve without notes
-> implement or diagnose
-> explain under a time limit
-> receive evidence-based feedback
-> record the smallest corrected rule
-> retrieve again after spacing
```

Every study block must create an artifact: solved code, a spoken answer, a diagram from memory, a scored mock, an incident timeline, or a corrected error card. Highlighting pages is input, not evidence of mastery.

The plan has four parallel tracks:

Track	Outcome
Java/JVM	Precise contracts, internals, trade-offs, and diagnostics
Coding/DSA	Correct medium problems in 35-45 minutes with explanation/tests
Design/backend	APIs, data, concurrency, reliability, and scaling decisions
Communication/behavioral	Structured answers, ownership stories, and collaborative reasoning

One track must not consume every hour. An SDE-2 loop often mixes coding, Java/concurrency, design, and behavioral evidence.

54.4 Core terminology

- **Active recall:** Producing an answer without looking at notes.
- **Spaced retrieval:** Recalling material after increasing delays, such as 1, 3, 7, and 14 days.
- **Interleaving:** Mixing related problem types so pattern selection must be inferred.
- **Error log:** Short record of an incorrect decision, root cause, corrected rule, and retest date.
- **Mock loop:** Timed simulation followed by scoring and a correction cycle.
- **Readiness gate:** Objective threshold required before increasing interview intensity.
- **Red flag:** Correctness-critical weakness that cannot be offset by a high average.
- **Maintenance set:** Small recurring review set preventing mastered topics from decaying.
- **Recovery plan:** Predefined response to schedule or performance disruption.

54.5 Detailed mechanics

Before Day 1: establish the baseline

Reserve 90 minutes and do not study first:

- 1 Answer 20 mixed questions from Chapter 53, two minutes each.
- 2 Solve one unseen medium array/hash problem in 40 minutes.
- 3 Explain one production incident in five minutes.
- 4 Draw source-to-CPU and JVM memory from memory.
- 5 Score with the rubrics later in this chapter.

Create an error log with columns: date, domain, prompt, observed error, root cause, corrected rule, smallest counterexample, retest dates, and status. Keep answers short. "Review concurrency" is not actionable; "I claimed volatile increment is atomic; retest with a two-thread lost-update trace on D+1/D+3/D+7" is.

Choose one Java 21 JDK distribution and record its exact version. Build a small scratch project with tests, compiler warnings, and commands for `javap`, JFR, and thread dumps. Interviews may use an online editor, so practice both IDE and plain-editor execution.

Specification boundary: Treat finalized Java 21 language/API behavior separately from preview features, vendor tools, and interview-platform restrictions. Preview APIs require explicit enablement and can change; a study schedule or hiring rubric is an editorial policy, not a Java platform guarantee.

Standard daily schedule

The working-engineer schedule is about 15 hours per week:

Monday through Friday, 2 hours:

- 15 minutes: spaced recall cards and one diagram.
- 35 minutes: focused chapter study.
- 45 minutes: implementation, diagnosis, or one coding problem.
- 15 minutes: speak one model answer without notes.
- 10 minutes: tests, error log, and next retrieval dates.

Saturday, 3 hours: 60-minute mock, 30-minute break/debrief, 75-minute targeted repair, 15-minute plan update.

Sunday, 2 hours: weekly retrieval test, story rehearsal, maintenance problems, and deliberate rest. Stop early if the gate is met and fatigue is high.

For a 90-minute weekday, keep 10 minutes recall, 30 minutes concept, 35 minutes code, and 15 minutes explanation/log. Do not remove retrieval or explanation; reduce scope. For a full-time search, add a second independent block after a long break, not a five-hour continuous session.

Week 1: computing model and JVM foundations

Goal: explain how Java executes and where memory/diagnostic evidence belongs.

Day	Primary work	Required output
1	Baseline assessment and environment	Scores, error log, exact JDK/runtime record
2	Why Java, JDK/JVM, compatibility, tools	90-second source-to-runtime answer
3	Compilation, bytecode, class files	Annotated <code>javap -c</code> for one class
4	JVM architecture and runtime data areas	Memory diagram from memory
5	Class loading and initialization	Failure table: CNFE, NCD FE, linkage/init errors
6	Object layout, calls, recursion	Coding mock plus reference/frame dry run
7	GC and JIT overview	Collector comparison and weekly retrieval score

Implementation tasks: compile with `--release 17`, inspect bytecode, create a static-initialization failure safely, capture a thread dump, and use a small JFR recording. Do not tune flags. The purpose is evidence literacy.

Gate 1: score at least 3/4 on source-to-CPU, class lifecycle, runtime areas, object creation, and GC reachability. Solve one medium array/hash problem in 50 minutes with correct complexity. If the JVM diagram still mixes Metaspace with heap guarantees, repeat the diagram on Days 8 and 10.

Week 2: Java language engineering

Goal: write and explain precise Java 17/21 code, equality, exceptions, generics, and object design.

Day	Primary work	Required output
8	Primitives, numeric semantics, operators	Overflow/promotion prediction test
9	Methods, overloading, varargs, pass-by-value	Five call-resolution dry runs
10	Arrays, strings, Unicode, text blocks	String/array memory and complexity explanation
11	Classes, access, inheritance, composition	Refactor inheritance to composition
12	Interfaces, sealed types, records, equality	Immutable value type with tests
13	Exceptions, resources, nested types, reflection	Suppressed-exception and reflection exercise
14	Generics, erasure, lambdas, Java 21 features	Language deep-dive mock and error repair

Complete two DSA problems this week involving strings/two pointers. For every solution, test empty input, one element, duplicates, boundary indices, and overflow where applicable. Explain when a record is inappropriate, why a final collection is still mutable, and one heap-pollution example.

Gate 2: average 3/4 across language mock dimensions with no red flags on pass-by-value, equality/hash contract, exception resource handling, or generic variance. Implement a correct immutable class in 25 minutes. If overload/generic reasoning is weak, use ten small compile-prediction snippets rather than rereading entire chapters.

Week 3: collections, streams, and I/O

Goal: select a data structure from operations and explain implementation-sensitive costs.

Day	Primary work	Required output
15	Collection hierarchy and List trade-offs	Selection table for six workloads
16	HashMap/HashSet internals	Hash/equality dry run and mutable-key failure
17	TreeMap/TreeSet and ordering	Three valid comparators with tests
18	Queue, deque, priority queue, heaps	Top-K implementation and complexity
19	Sorting, selection, Comparable/Comparator	Comparator contract review
20	Streams, collectors, Optional	Sequential pipeline plus safe collector
21	NIO.2, buffers, serialization boundaries	File-copy/buffer state exercise and mock

Coding maintenance: one sliding-window problem, one heap/top-K problem, and one interval/sorting problem. Repeat one older problem from memory after seven days; do not count immediate re-solves as mastery.

Gate 3: choose ArrayList/HashMap/TreeMap/heap/deque correctly from a new scenario, state qualified complexity, and solve two collection-heavy medium problems within 45 minutes each on separate days. Explain why fail-fast is not thread safety and why parallel streams are not a universal speed switch.

Week 4: concurrency and the Java Memory Model

Goal: prove visibility/atomicity and design cancellation, locking, scheduling, and virtual-thread capacity.

Day	Primary work	Required output
22	Threads, states, interruption, shutdown	Owned-worker shutdown protocol
23	Happens-before, volatile, safe publication	HB graph for two publication patterns
24	Intrinsic locks, wait/notify	Predicate-loop bounded buffer
25	ReentrantLock/Condition, deadlock	Lock-order transfer and wait-for graph
26	Atomics, CAS, ABA, accumulators	CAS dry run and linearization point
27	Executors, bounded queues, futures	Executor capacity plan and shutdown code
28	Concurrent collections, CompletableFuture, virtual threads	Full concurrency mock and incident diagnosis

Run one stress exercise, but pair it with a proof: a lost-update counter, safe volatile publication, or condition wait. Capture thread dumps from a deliberate safe deadlock and a healthy idle executor; learn the difference. On Java 21, create virtual threads and demonstrate a semaphore protecting a simulated downstream.

Gate 4: no score below 3 on interruption, happens-before, lock/condition loops, executor admission, and virtual-thread capacity. You must explain why `volatile count++` fails, why `Future.get(timeout)` does not cancel, and why a virtual-thread migration can overload JDBC. This is a hard gate; concurrency misconceptions should delay interviews involving deep Java.

Week 5: DSA foundations in Java

Goal: make problem-solving communication and core linear structures automatic.

Day	Primary work	Required output
29	Complexity and interview method	Verbal template plus brute-force comparison
30	Arrays, hashing, prefix sums	Two mediums, one timed
31	Two pointers and sliding windows	Invariant written before implementation
32	Linked lists	Reverse, cycle, and merge dry runs
33	Stacks, queues, monotonic structures	Next-greater/interval exercise
34	Trees and recursive reasoning	Traversal plus depth-risk discussion
35	BSTs and heaps	60-minute coding mock and repair

Problem selection should be representative, not random. For each pattern, solve one learning problem with notes, one related problem without notes after a day, and one mixed problem where the pattern is not named. Maintain Java hygiene: use `ArrayDeque` for stack/queue, avoid subtraction comparators, handle integer overflow, and choose meaningful helper methods.

Gate 5: in three unseen mediums, achieve two correct solutions within 45 minutes and one viable solution with minor correction. Every solution must include examples, invariant, complexity, and tests. If recognition is good but implementation fails, reduce new problems and do line-by-line dry runs plus typed reimplementations from a blank file.

Week 6: advanced DSA and low-level design

Goal: handle graph/state-space problems and translate requirements into maintainable Java objects.

Day	Primary work	Required output
36	Graph representation, BFS/DFS	Component/shortest-path comparison
37	Topological sort and union-find	Dependency ordering implementation
38	Weighted shortest paths	Dijkstra assumptions and stale-entry trace
39	Backtracking and pruning	State-choice-undo invariant
40	Greedy reasoning and counterexamples	Exchange argument or rejected greedy
41	Dynamic programming	State/transition/base/order derivation
42	SOLID, API design, patterns	60-minute LLD mock plus coding maintenance

Use one graph, one backtracking, and two DP problems; quality beats volume. For LLD, practice a rate limiter, parking lot, task scheduler, or notification service. State scope, concurrency, extension points, failure behavior, and what belongs outside the process.

Gate 6: solve one unseen graph medium and one DP medium with correct state/complexity; reach 3/4 on an LLD rubric for requirements, model, APIs, invariants, extensibility, and testability. A memorized pattern without a correctness explanation does not pass.

Week 7: backend, performance, reliability, and full loops

Goal: connect Java mechanics to service design and production evidence.

Day	Primary work	Required output
43	JDBC, connection pools, transactions	Transaction/isolation failure walkthrough
44	Serialization, APIs, idempotency	Idempotent create state machine
45	Performance method and JMH	Valid benchmark plan, no premature tuning
46	jcmd, JFR, dumps, GC incidents	Investigation playbook for p99/OOM/high CPU
47	Testing, build tools, dependencies	Test pyramid and reproducible build record
48	Security and reliability	Threat review for one API
49	Full coding + Java + design loop	Scores, recording, top-three repair plan

Prepare a story bank: difficult bug, performance improvement, conflict/trade-off, ownership beyond role, failed decision and learning, ambiguous project, and mentoring/collaboration. Each story should be five minutes with a 30-second summary. Include metrics but avoid confidential detail.

Gate 7: complete a full loop without a correctness red flag. Diagnose one incident using a timeline and tools, design idempotency/transactions correctly, and score at least 3 on testing/security. If system design is broad but shallow, practice one subsystem deeply rather than adding more architecture boxes.

Week 8: simulation, consolidation, and interview taper

Goal: produce stable performance, not learn an entirely new curriculum.

Day	Primary work	Required output
50	Full mock loop 1	Independent scores and error priorities
51	Repair top correctness weakness	Counterexample, implementation, retest
52	Full mock loop 2	Compare score trend and fatigue
53	Repair communication/timing	30/90-second answer set
54	Company-shaped mixed loop	Coding plus likely Java/design emphasis
55	Light maintenance and logistics	Environment, questions, story cards, sleep plan
56	Retrieval only and rest	No heavy new problems; final readiness decision

Schedule real interviews only after a gate if possible, with lower-priority companies before top choices but without treating any interviewer disrespectfully. Leave recovery time between loops. The final 48 hours should reduce volume, preserve sleep, and rehearse opening frameworks, not attempt a new advanced topic.

Mock loop formats

Coding mock, 60 minutes: 5 minutes clarification/examples, 5 minutes baseline/invariant, 35 minutes implementation, 10 minutes tests/complexity, 5 minutes feedback. The interviewer should not rescue pattern recognition immediately.

Java deep dive, 45 minutes: six rapid questions, two deep follow-up trees, one code/concurrency dry run, and five-minute summary feedback.

Low-level design, 60 minutes: requirements/scope 10, model/API 15, core flows/invariants 15, concurrency/failure/scaling 10, trade-offs/testing 10.

Backend/system design, 60 minutes: requirements/SLOs 10, estimates/interfaces 10, data/workflows 15, scaling/reliability 15, deep dive/trade-offs 10.

Behavioral, 45 minutes: four stories with follow-ups on ownership, conflict, failure, data, and learning. Feedback must identify missing context, personal action, trade-off, and outcome.

Debrief within 30 minutes: score first without notes, compare interviewer score, identify the earliest wrong decision, write one corrected rule, and schedule a retest. Do not immediately redo the same prompt from memory and call it fixed; use a transfer problem first.

TRACE IT | 54.6 Worked Java example

This small Java 21-compatible tracker encodes a readiness gate instead of relying on mood:

JAVA (continued 1/2)

```
import java.util.List;

public final class ReadinessTracker {
    public record MockScore(
        int correctness,
        int communication,
        int complexity,
        int testing,
        int javaDepth,
        int design) {

        public MockScore {
            int[] values = {
                correctness, communication, complexity,
                testing, javaDepth, design
            };
            for (int value : values) {
                if (value < 0 || value > 4) {
                    throw new IllegalArgumentException("score must be 0..4");
                }
            }
        }
    }

    boolean clearsCriticalGate() {
```

JAVA (continued 2/2)

```
        return correctness >= 3
            && communication >= 3
            && javaDepth >= 3
            && testing >= 2;
    }

    static double weighted(MockScore score) {
        return score.correctness() * 0.25
            + score.communication() * 0.15
            + score.complexity() * 0.15
            + score.testing() * 0.10
            + score.javaDepth() * 0.20
            + score.design() * 0.15;
    }

    static boolean ready(List<MockScore> scores) {
        if (scores.size() < 3) return false;
        List<MockScore> recent = scores.subList(scores.size() - 3, scores.size());
        return recent.stream().allMatch(MockScore::clearsCriticalGate)
            && recent.stream().mapToDouble(ReadinessTracker::weighted)
                .average().orElse(0.0) >= 3.2;
    }
}
```

Weights are a planning policy, not a universal hiring rubric. Adjust them for the target role, but never let a high design score average away incorrect/racy code. Keep red-flag gates.

TRACE IT | 54.7 Execution or memory walkthrough

Suppose the last three mock scores are:

TEXT

```
M1 = correctness 3, communication 3, complexity 3,  
    testing 2, javaDepth 3, design 3      weighted 2.90  
  
M2 = correctness 4, communication 3, complexity 3,  
    testing 3, javaDepth 3, design 3      weighted 3.25  
  
M3 = correctness 4, communication 4, complexity 3,  
    testing 3, javaDepth 4, design 3      weighted 3.60
```

All three clear the critical minima, but the average is 3.25, so `ready` returns true. If M3 had Java depth 2, the average might remain acceptable, but `clearsCriticalGate` would fail and readiness would be false.

The method examines only the latest three mocks so old baseline failures do not dominate forever. That also means mock selection must be representative. Three easy array problems cannot establish system-design or concurrency readiness. Store domain-specific scorecards alongside the aggregate.

At runtime, record instances and list elements are ordinary objects; no concurrency is implied. If multiple evaluators update a shared tracker, publish immutable score-list snapshots or use an appropriate thread-safe store. The example calculates a decision; it does not persist evidence.

54.8 Complexity and performance

`weighted` is $O(1)$. `ready` examines three recent records, effectively $O(1)$, with $O(1)$ auxiliary space aside from stream machinery. A generalized window of k mocks would be $O(k)$.

The standard schedule invests about 15 hours/week, or 120 hours over eight weeks. Protect those hours from context switching. Two uninterrupted 60-minute blocks usually produce more evidence than four scattered half-hours. Track completed outputs and scores, not chair time.

Spaced retrieval should be selective. A new error is recalled after approximately 1, 3, 7, and 14 days; mastered cards move to weekly maintenance. If review consumes more than 25 percent of study time, retire duplicates and keep rules with counterexamples rather than accumulating trivia.

Problem count is a weak throughput metric. A better weekly dashboard includes unseen timed correctness, median time to viable approach, implementation defect count, Java question score, mock trend, sleep/fatigue, and number of unresolved red flags.

WATCH OUT | 54.9 Edge cases and common mistakes

- Reading the entire book sequentially without retrieval or implementation.
- Solving only familiar tagged problems and overestimating pattern recognition.
- Counting a solution read as a problem solved.
- Doing mocks without written scores or repeating mocks without repairing root causes.
- Cramming missed work into one exhausted weekend.
- Letting DSA consume all Java, design, or behavioral time.
- Changing resources every few days instead of completing one feedback loop.
- Memorizing HotSpot internals without contracts or production evidence.
- Scheduling top-choice interviews before concurrency/correctness red flags clear.
- Treating a weighted average as permission to ignore a score of 1 in security or correctness.
- Sacrificing sleep; fatigue directly harms working memory, communication, and coding accuracy.
- Studying company trivia instead of role requirements and reusable fundamentals.

54.10 Production engineering notes

Treat preparation artifacts like an engineering repository: dated scorecards, runnable code, failing/passing tests, diagrams, and a changelog of corrected rules. Keep personal/employer data out of examples. Use synthetic incidents when a real story is confidential.

Before each interview, verify JDK/editor assumptions, meeting link, time zone, backup network/audio, allowed references, and whether coding occurs in a shared editor. Prepare concise questions about team ownership, reliability expectations, deployment, on-call, technical debt, mentorship, and success in the first six months.

HotSpot note: Practice implementation-specific diagnostics on the same Java 21 runtime you recorded, but rehearse answers at the portable contract level first. Tool output and defaults can differ in the interviewer's JDK.

Recovery plans

Missed one or two days: Do not double the next day. Preserve recall, the week's mock, and the gate-critical topic. Drop optional new problems and move one deep dive to Sunday.

Lost a full week: Extend the calendar by a week if possible. If the interview date cannot move, retain Weeks 1-4 correctness foundations, core DSA, one design loop, and two final mocks; reduce breadth, not verification.

Repeated coding failure: Classify: misunderstood contract, wrong pattern, broken invariant, Java implementation defect, or time management. Do two untimed derivations, one blank-file implementation, then one transfer problem timed. Avoid ten more random problems.

Weak Java deep dive: Build 30/90-second answers and draw mechanisms. For each wrong claim, attach a counterexample and specification/implementation label. Retest verbally with follow-ups.

Mock anxiety: Increase exposure gradually: record alone, peer with known questions, peer with unseen questions, then full loop. Keep the same opening framework so the first two minutes are automatic.

Plateau: Inspect the error distribution. If the same root cause repeats, change the drill. If errors are diverse, reduce cognitive load and improve rest. Seek external feedback because self-scoring may be miscalibrated.

Burnout or sleep loss: Take a full rest day, cut volume by one third for a week, preserve only recall and one mock, and restore sleep/exercise. Extending the plan is cheaper than reinforcing careless habits.

TRY IT | 54.11 Interview questions and model answers

How do I know I am ready?

Use three representative recent mocks with weighted average at least 3.2/4, no critical score below the gate, two unseen coding mediums solved correctly under time on different days, and one successful Java/concurrency plus design loop. Confidence is supporting information, not the gate.

How many coding problems should I solve?

Enough to demonstrate transfer across core patterns. Fifty deeply reviewed mixed problems can outperform 200 copied solutions. Track unseen timed correctness, explanation, tests, and delayed re-solves rather than a universal count.

Should I postpone an interview after a failed mock?

Look at failure type and timing. One difficult prompt is noise; a repeated correctness red flag in concurrency, coding, or design is signal. If rescheduling is feasible, repair and pass a transfer mock first. Do not wait for perfection.

What if I have only four weeks?

Run the baseline, combine Weeks 1-2, Weeks 3-4, Weeks 5-6, and Weeks 7-8. Keep one mock and debrief each week. Reduce problem breadth and optional internals, but retain JMM, collections, core DSA, backend correctness, and final simulations.

How should mocks be scored?

Use observable behavior: correct assumptions, invariant, implementation, tests, complexity, communication, depth, trade-offs, and recovery from hints. Score independently before discussion and record the earliest wrong decision, not only final outcome.

What should I do the day before?

Light retrieval, one easy confidence problem, logistics, story headlines, questions for interviewers, normal food/exercise, and sufficient sleep. No long mock, new advanced topic, or late-night cramming.

TRY IT | 54.12 Exercises

- 1 Run the baseline and create the first eight error cards with D+1/D+3/D+7 dates.
- 2 Customize the 56-day table to your interview format without deleting any critical gate.
- 3 Build scorecards for coding, Java deep dive, LLD, backend design, and behavioral rounds.
- 4 Implement **ReadinessTracker**, add tests for insufficient mocks, boundary scores, and one red flag.
- 5 Schedule the first four mocks now, including reviewer and debrief time.
- 6 Write recovery versions for 90 minutes/day, one missed week, and a fixed interview in 14 days.
- 7 Record one 90-second JVM answer and one five-minute incident story; score and repeat after three days.
- 8 At the end of each week, publish a private one-page review: evidence, red flags, repaired rules, next gate.

RECAP | 54.13 Chapter summary

An effective eight-week plan alternates learning with retrieval, implementation, explanation, mocks, and correction. Each week produces artifacts and ends at an objective gate. Java/JVM foundations and concurrency correctness precede high interview volume; DSA, design, backend reliability, testing, security, and behavioral evidence remain parallel tracks. Scored mock trends and critical minima determine readiness, while predefined recovery plans keep one disruption from destroying the schedule.

TRY IT | 54.14 Revision checklist

- ☐ I completed a no-study baseline and created an actionable error log.
- ☐ My calendar contains daily outputs, weekly mocks, debriefs, and rest.
- ☐ I use 1/3/7/14-day retrieval for corrected rules.
- ☐ Every week has a measurable gate and recovery action.
- ☐ My coding practice includes unseen timed transfer problems.
- ☐ I score Java, coding, design, testing/security, and communication separately.
- ☐ I have at least three representative recent mocks with no critical red flag.
- ☐ My production stories include evidence, trade-offs, actions, and measured outcomes.
- ☐ I have a reduced-volume plan for missed time or burnout.
- ☐ The final 48 hours prioritize retrieval, logistics, and sleep.

JAVA FOUNDATIONS TO ADVANCED ENGINEERING

Part IX - Core Library Depth and the Modern Java Baseline

A focused part of the complete SDE-2 preparation guide

SDE-2 CORE CHAPTER

Chapter 55 - Dates, Times, Zones, and the java.time API

55.1 Learning objectives

By the end of this chapter, you should be able to:

- choose the correct `java.time` type for an instant, a local date, a wall-clock appointment, and a duration;
- explain the difference between `Instant`, `LocalDateTime`, `OffsetDateTime`, and `ZonedDateTime`;
- reason about time zones, daylight-saving gaps and overlaps, and the tzdb update cycle;
- apply `Duration`, `Period`, and `ChronoUnit` without conflating machine time and calendar time; and
- design storage, serialization, and test boundaries so that time is an injected dependency rather than ambient state.

WHY IT WORKS | 55.2 Why this matters at SDE-2

Time bugs are rarely caught by unit tests and are almost always found in production. A scheduled job fires twice on the night the clocks go back. A subscription renews an hour early for users in one region. A report joins two tables whose timestamps mean different things, and nobody notices until the totals disagree at a quarter boundary.

The legacy `java.util.Date` and `Calendar` types encouraged these failures: `Date` is a mutable instant with a misleading name, `Calendar` months are zero-based, and neither type distinguishes a point on the timeline from a local calendar reading. `java.time` replaced them with immutable types whose names state their meaning. At SDE-2, the interview question is usually "how do you store a timestamp," and the correct answer depends on whether the value is an event that happened or an appointment that will happen.

WHY IT WORKS | 55.3 First-principles model

There are two fundamentally different kinds of time, and most defects come from using one where the other is required.

Machine time is a point on a continuous timeline, independent of any calendar or location. `Instant` models this: a count of seconds and nanoseconds from the 1970-01-01T00:00:00Z epoch. Two observers anywhere in the world agree on an `Instant`.

Calendar time is a human reading - a date and a wall-clock time as displayed by a local calendar. `LocalDate`, `LocalTime`, and `LocalDateTime` model this. They carry no zone and therefore do not identify a point on the timeline. `2026-03-08T02:30` is a valid `LocalDateTime` and, in several time zones, an instant that never existed.

A zone converts between the two. `ZoneId` names a region with a full history of offset rules ("America/New_York"); `ZoneOffset` is a fixed displacement from UTC ("-05:00"). A region is not an offset: New York is `-05:00` in January and `-04:00` in July. `ZonedDateTime` is a local date-time plus a zone plus the resolved offset, so it identifies a real instant and remembers the rules that produced it.

Specification boundary: Java specifies the ISO-8601 calendar system, the arithmetic of each `java.time` type, and the resolution strategy for daylight-saving gaps and overlaps. It does not specify the contents of the time-zone database. Zone rules are political, change several times a year, and ship as tzdb data inside the JDK or from `-Djava.time.zone.DefaultZoneRulesProvider`. Correct code can produce wrong answers on a stale runtime.

55.4 Core terminology

- **Instant:** a point on the timeline, in UTC, with nanosecond field precision.
- **LocalDate / LocalTime / LocalDateTime:** calendar readings with no zone and no offset.
- **ZoneId:** a region whose UTC offset varies over time according to tzdb rules.
- **ZoneOffset:** a fixed offset from UTC; a `ZoneId` subtype with no rule history.
- **OffsetDateTime:** a local date-time plus a fixed offset - a real instant, no rule history.
- **ZonedDateTime:** a local date-time plus a `ZoneId` plus the resolved offset.
- **Duration:** machine-time amount measured in seconds and nanoseconds.
- **Period:** calendar-time amount measured in years, months, and days.
- **Gap:** a local time that does not exist because the clocks sprang forward.
- **Overlap:** a local time that occurs twice because the clocks fell back.
- **Clock:** the injectable abstraction supplying "now" and the default zone.

55.5 Detailed mechanics

Choosing the type

The decision is driven by what the value means, not by what is convenient to store.

The value is	Use	Reason
Something that happened - a log line, an audit record, a payment capture	<code>Instant</code>	The event has one unambiguous time; local readings are a display concern.
A birthday, an invoice date, a holiday	<code>LocalDate</code>	There is no time and no zone; the date is the same fact everywhere.
A store's opening time, a recurring alarm	<code>LocalTime</code>	The wall-clock reading is the requirement, whatever the offset that day.
A future appointment in a named place	<code>ZonedDateTime</code>	The user means "9 a.m. in Chicago," which must survive a rule change.
An API or database timestamp with a known offset	<code>OffsetDateTime</code>	Unambiguous instant, and it round-trips through <code>TIMESTAMP WITH TIME ZONE</code> .
An elapsed measurement or a timeout	<code>Duration</code>	Machine time; must not be affected by calendar arithmetic.
"One month later" on a billing cycle	<code>Period</code>	Calendar time; the day count deliberately varies by month.

The two rows most often confused are the first and the fourth. Store a *past* event as an `Instant`. Store a *future* appointment as a `ZonedDateTime` or as a `LocalDateTime` plus a `ZoneId` column. If you collapse a future appointment to an `Instant` at write time and the region later changes its rules, the meeting silently moves.

Immutability and the wither pattern

Every `java.time` value is immutable and thread-safe. Mutating methods do not exist; `plusDays`, `withHour`, and `truncatedTo` return new values.

JAVA

```

LocalDate date = LocalDate.of(2026, 1, 31);
date.plusMonths(1);           // result discarded - a common bug
LocalDate next = date.plusMonths(1); // 2026-02-28

```

Note the clamping in the second call. Adding one calendar month to 31 January yields 28 February, because month arithmetic resolves to the last valid day. This is intentional and specified, but it is not reversible: `date.plusMonths(1).minusMonths(1)` is 28 January, not 31 January. Any billing logic that assumes round-tripping is wrong.

Duration versus Period

Duration counts elapsed seconds. **Period** counts calendar fields. On a daylight-saving boundary they disagree, and that disagreement is the point.

JAVA

```
ZoneId chicago = ZoneId.of("America/Chicago");
ZonedDateTime before = ZonedDateTime.of(
    LocalDate.of(2026, 3, 7), LocalTime.of(12, 0), chicago);

ZonedDateTime plusPeriod = before.plus(Period.ofDays(1)); // 2026-03-08T12:00-05:00
ZonedDateTime plusDuration = before.plus(Duration.ofDays(1)); // 2026-03-08T13:00-05:00
```

Period.ofDays(1) means "same wall-clock time tomorrow" and consumes 23 real hours across the spring-forward boundary. **Duration.ofDays(1)** means "86,400 seconds later" and lands an hour further along the clock. A daily 12:00 job must use **Period** or a **ZonedDateTime** field addition; a one-day cache expiry must use **Duration**.

Gaps and overlaps

When a local date-time does not exist, **ZonedDateTime.of** does not throw. It shifts the result forward by the size of the gap.

JAVA

```
ZoneId chicago = ZoneId.of("America/Chicago");
LocalDateTime springForward = LocalDateTime.of(2026, 3, 8, 2, 30);
ZonedDateTime resolved = ZonedDateTime.of(springForward, chicago);
// 2026-03-08T03:30-05:00 - 02:30 never occurred
```

When a local date-time occurs twice, **of** selects the **earlier** offset. **withLaterOffsetAtOverlap()** selects the other. Silently picking the earlier one is right for most displays and wrong for anything that must not fire twice; consult **ZoneRules.getValidOffsets** when the distinction matters.

JAVA

```
ZoneRules rules = chicago.getRules();
LocalDateTime fallBack = LocalDateTime.of(2026, 11, 1, 1, 30);
List<ZoneOffset> valid = rules.getValidOffsets(fallBack);
// size 0 -> gap, size 1 -> normal, size 2 -> overlap
```

A scheduler that treats "size 2" as ordinary will run the 01:30 job twice each November.

Parsing, formatting, and the locale trap

`DateTimeFormatter` is immutable and thread-safe - unlike `SimpleDateFormat`, which is not and which caused a long tail of production corruption when shared across threads.

Formatters default to the JVM's locale unless told otherwise. `DateTimeFormatter.ofPattern("MMM d")` produces "Jan 5" on one host and a localized month name on another. Always pin the locale for machine-readable output, and prefer the predefined ISO constants for wire formats.

JAVA

```
DateTimeFormatter machine = DateTimeFormatter.ISO_INSTANT;
DateTimeFormatter human = DateTimeFormatter
    .ofPattern("d MMMM yyyy", Locale.US)
    .withZone(ZoneId.of("America/Chicago"));
```

The pattern letters are a frequent source of defects: `yyyy` is the calendar year and `YYYY` is the week-based year. They differ for a few days each January, which is why "the dashboard showed 2025 for three days" is a recurring new-year incident.

Clock injection

`Instant.now()` reads the system clock and the default zone. Both are ambient global state, and code that calls them directly cannot be tested deterministically without changing the machine.

`Clock` is the seam. Inject it, default it to `Clock.systemUTC()`, and substitute a fixed clock in tests.

JAVA

```
public final class TrialService {
    private final Clock clock;

    public TrialService(Clock clock) {
        this.clock = clock;
    }

    public boolean expired(Instant startedAt, Duration trialLength) {
        return Instant.now(clock).isAfter(startedAt.plus(trialLength));
    }
}
```

`Clock.fixed(instant, zone)` freezes time; `Clock.offset(base, duration)` shifts it. Neither requires mocking a static method, which is why this design is worth the one extra constructor parameter.

TRACE IT | 55.6 Worked Java example

A billing service that renews a subscription must add a calendar month in the customer's own zone, then resolve to an instant for storage.

JAVA (continued 1/2)

```
import java.time.Clock;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;

public final class RenewalSchedule {
    private final Clock clock;

    public RenewalSchedule(Clock clock) {
        this.clock = clock;
    }

    /**
     * Next renewal at the same local billing hour, one calendar month later,
     * resolved in the customer's zone. Returned as an Instant because the
     * scheduler compares against a monotonic timeline.
     */
    public Instant nextRenewal(LocalDate anchorDate, LocalTime billingHour, ZoneId customerZone) {
        ZonedDateTime current = ZonedDateTime.of(anchorDate, billingHour, customerZone);
```

JAVA (continued 2/2)

```
        ZonedDateTime next = current.plus(Period.ofMonths(1));
        return next.toInstant();
    }

    public boolean dueNow(Instant renewalAt) {
        return !Instant.now(clock).isBefore(renewalAt);
    }

    public static void main(String[] args) {
        RenewalSchedule schedule = new RenewalSchedule(Clock.systemUTC());
        ZoneId chicago = ZoneId.of("America/Chicago");

        Instant fromJan31 = schedule.nextRenewal(
            LocalDate.of(2026, 1, 31), LocalTime.of(2, 30), chicago);
        System.out.println(fromJan31); // 2026-02-28T08:30:00Z

        Instant acrossDst = schedule.nextRenewal(
            LocalDate.of(2026, 2, 8), LocalTime.of(2, 30), chicago);
        System.out.println(acrossDst); // 2026-03-08T08:30:00Z
    }
}
```

The first call demonstrates month clamping: 31 January has no counterpart in February, so the renewal lands on the 28th. The second call crosses the spring-forward boundary; 02:30 on 8 March does not exist in Chicago, so the resolution rule shifts it to 03:30 local, which is 08:30Z.

TRACE IT | 55.7 Execution or memory walkthrough

`ZonedDateTime.of(anchorDate, billingHour, customerZone)` builds a `LocalDateTime`, asks `ZoneRules` for the valid offsets of that local value, and stores the local value, the zone, and the chosen offset as three fields. No lookup table is copied; `ZoneRules` instances are shared per zone.

`plus(Period.ofMonths(1))` operates on the **local** fields first. It adds one to the month, clamps the day to the month length, and then re-resolves the offset against the zone rules. This ordering is what makes the result "the same wall-clock time next month" rather than "a fixed number of seconds later."

`toInstant()` subtracts the resolved offset from the local date-time and converts to epoch seconds. The returned `Instant` holds two primitive fields - a long of seconds and an int of nanoseconds - and carries no zone. The zone information is deliberately discarded, which is why the *future* appointment case needs the `ZonedDateTime` retained rather than only its instant.

Every intermediate value here is a separate immutable object. The allocation is real but small and short-lived; these objects die in the young generation.

55.8 Complexity and performance

All field arithmetic is $O(1)$. Offset resolution is a binary search over the zone's transition table - $O(\log t)$ in the number of historical transitions, typically a few hundred - and the table is loaded once per zone and cached.

`ZoneId.of` and `DateTimeFormatter.ofPattern` are the expensive calls. Both parse and build structures that should be hoisted to `static final` fields rather than constructed per request. Formatting and parsing dominate any realistic profile of date-heavy code; the arithmetic does not.

`Instant` comparison is a long comparison. Prefer comparing instants over comparing formatted strings, which is both slower and wrong across offsets.

HotSpot note: `java.time` values are ordinary heap objects. They are good escape-analysis candidates - a `LocalDate` created and consumed inside one inlined method may be scalar-replaced - but this is an optimization, not a guarantee. Do not restructure clear date logic to avoid allocation without a benchmark showing the allocation matters.

WATCH OUT | 55.9 Edge cases and common mistakes

- Treating `LocalDateTime` as an instant. It has no zone; converting it assumes a default that varies by host.
- Storing a future appointment as an `Instant`, losing the intent when zone rules change.
- Using `ZoneOffset` where `ZoneId` is required, freezing an offset that is only correct half the year.
- Assuming `plusMonths(1).minusMonths(1)` is the identity. Month clamping is lossy.
- Using `Duration.ofDays` for calendar days across a DST boundary.
- Sharing a `SimpleDateFormat` across threads. Migrate to `DateTimeFormatter`, which is immutable.
- `YYYY` instead of `yyyy` in a pattern, producing a wrong year for a few days each January.
- Calling `LocalDate.now()` with no argument in domain logic, making the code untestable and host-dependent.
- Assuming a day has 86,400 seconds. Leap seconds are smeared by most infrastructure, but DST alone breaks the assumption.
- Comparing timestamps with `equals` across types. `Instant` and `OffsetDateTime` never compare equal; use `isEqual` or compare instants.
- Persisting to a database column that silently converts. MySQL `TIMESTAMP` normalizes to UTC using the session zone; `DATETIME` does not. Know which you have.
- Relying on a JDK's bundled tzdb in a long-lived container image. A container built two years ago has two-year-old political data.

55.10 Production engineering notes

Store instants in UTC and convert at the edges. The database column should be `TIMESTAMP WITH TIME ZONE` or an explicit UTC-normalized type; the application should convert to the user's zone only at rendering time. This single rule removes most cross-region reporting defects.

Keep the customer's `ZoneId` as first-class data when future scheduling is involved. A subscription row that carries `next_renewal_local` and `zone_id` survives a tzdb update correctly; one that carries only `next_renewal_utc` does not.

Update tzdb deliberately. The JDK ships it, so a base-image bump changes time behavior. Treat that as a production change with the same care as a dependency upgrade, and know that `TZUpdater` exists for patching a JDK in place when a full upgrade is not available.

Inject `Clock` in every service that reads the current time. This is the difference between a test that asserts renewal behavior across a DST boundary in milliseconds and a test that cannot be written at all.

Log in ISO-8601 with an explicit offset. `2026-03-08T08:30:00Z` is greppable, sortable, and unambiguous; `Mar 8 03:30:00` is none of those and has lost the information needed to reconstruct the event ordering.

TRY IT | 55.11 Interview questions and model answers

What is the difference between `Instant` and `LocalDateTime`?

`Instant` is a point on the timeline in UTC - an unambiguous moment every observer agrees on.

`LocalDateTime` is a calendar reading with no zone, so it does not identify a moment. Converting between them requires a zone, and choosing the wrong one is the most common time defect in a distributed system.

When would you use `ZonedDateTime` instead of `Instant`?

For future events tied to a place. "9 a.m. in Chicago next March" must remain 9 a.m. even if the offset rules change before then, so the zone must be retained rather than collapsed to an instant at write time. Past events go the other way: they already happened at one moment, so `Instant` is correct.

What is the difference between `Duration` and `Period`?

`Duration` is machine time - seconds and nanoseconds - and is unaffected by calendars. `Period` is calendar time - years, months, days - and deliberately varies in real length. Across a spring-forward boundary, `Period.ofDays(1)` advances 23 hours while `Duration.ofDays(1)` advances 24.

What happens when you construct a `ZonedDateTime` for a time that does not exist?

It does not throw. The gap resolution rule shifts the result forward by the length of the gap, so 02:30 on a spring-forward morning becomes 03:30. If you need to detect this rather than absorb it, call `ZoneRules.getValidOffsets` and check for an empty list.

Why is `java.time` preferred over `Date` and `Calendar`?

The types are immutable and thread-safe, the names state whether a value has a zone, months are one-based, and the API separates machine time from calendar time. `SimpleDateFormat`'s thread-unsafety alone caused a large class of production bugs that `DateTimeFormatter` cannot reproduce.

How do you make time-dependent code testable?

Inject a `Clock` and read the current time through it. `Clock.fixed` then makes DST boundaries, month-end clamping, and expiry edges ordinary unit tests rather than untestable behavior.

TRY IT | 55.12 Exercises

- 1 Write a method that returns every local time between 01:00 and 03:00 on a fall-back date in a zone of your choice, and mark which readings are ambiguous.
- 2 Take an existing service that calls `Instant.now()` directly and refactor it to accept a `Clock`; add a test that asserts behavior at a month boundary.
- 3 Demonstrate the `yyyy` versus `YYYY` discrepancy by formatting 29 December through 2 January.
- 4 Model a monthly subscription anchored on the 31st and produce twelve renewal dates. Explain each clamped result.
- 5 Store the same appointment as an `Instant` and as a `ZonedDateTime`, then simulate a zone-rule change and show which representation survives.
- 6 Benchmark `DateTimeFormatter.ofPattern` inside a loop against a hoisted `static final` formatter and report the difference.

RECAP | 55.13 Chapter summary

`java.time` separates machine time from calendar time, and nearly every date defect comes from confusing the two. `Instant` is a moment; `LocalDate` and `LocalDateTime` are calendar readings without a zone; `ZonedDateTime` binds a local reading to a region's rule history and therefore to a real moment. `Duration` measures elapsed seconds while `Period` measures calendar fields, and they diverge exactly where daylight-saving boundaries make the divergence matter. Every type is immutable and thread-safe, so formatters can be shared and values can cross threads freely. Store past events as UTC instants, retain the zone for future appointments, pin locales on formatters, treat tzdb as production data, and inject `Clock` so that the hard cases become testable.

TRY IT | 55.14 Revision checklist

- ☐ I can state which `java.time` type models an event, a birthday, an appointment, and a timeout.
- ☐ I can explain why `LocalDateTime` does not identify a moment.
- ☐ I know the difference between `ZoneId` and `ZoneOffset` and when each is wrong.
- ☐ I can predict the result of `Period.ofDays(1)` and `Duration.ofDays(1)` across a DST boundary.
- ☐ I can detect a gap and an overlap using `ZoneRules.getValidOffsets`.
- ☐ I know why month arithmetic does not round-trip.
- ☐ I pin the locale on every human-facing formatter and hoist formatters to constants.
- ☐ I inject `Clock` rather than calling `Instant.now()` in domain logic.
- ☐ I can explain how a stale tzdb produces correct code with wrong answers.

SDE-2 CORE CHAPTER

Chapter 56 - Regular Expressions and Text Processing

56.1 Learning objectives

By the end of this chapter, you should be able to:

- use `Pattern` and `Matcher` correctly, including their very different threading rules;
- explain greedy, reluctant, and possessive quantifiers and choose between them deliberately;
- recognize catastrophic backtracking and rewrite a pattern to eliminate it;
- apply groups, named groups, lookaround, and boundaries without over-reaching; and
- decide when a regular expression is the wrong tool.

WHY IT WORKS | 56.2 Why this matters at SDE-2

Regular expressions appear in log parsing, input validation, routing rules, data migration, and every ad-hoc extraction task that outlives its author. They are also one of the few places where a single line of application code can take down a service: a pattern that runs in microseconds on typical input can run for hours on a crafted string, and the thread is not interruptible while it does.

The SDE-2 signal is not whether you can recall the syntax for a character class. It is whether you can reason about how the engine executes the pattern, whether you know that `String.split` compiles a pattern on every call, and whether you can tell an interviewer why `(a+)+b` is a denial-of-service vector.

WHY IT WORKS | 56.3 First-principles model

Java's `java.util.regex` is a **backtracking** engine. It does not build a deterministic automaton and run the input through it once. It walks the pattern, and whenever a construct could match in more than one way, it takes one branch, continues, and - if the rest of the pattern fails - returns and tries the next alternative.

That single design fact explains almost everything that matters in practice. Backtracking is what makes backreferences and lookaround expressible. It is also what makes the worst case exponential rather than linear, because a pattern with nested ambiguous quantifiers can have exponentially many ways to divide the same substring, and a failing match forces the engine to try all of them.

A `Pattern` is a compiled, immutable, thread-safe program. A `Matcher` is the mutable execution state for one pattern against one input: current position, group boundaries, and the backtracking stack. Compiling is expensive; matching is cheap; sharing a `Matcher` across threads is a data race.

Specification boundary: Java specifies the pattern syntax, the greediness of each quantifier, group numbering, and the semantics of the flags. It does not specify a time bound. Match time is a property of the engine's backtracking strategy, not of the language, and no timeout facility is built in - a runaway match is not interruptible by `Thread.interrupt`.

56.4 Core terminology

- **Pattern:** compiled immutable regular expression; thread-safe and reusable.
- **Matcher:** mutable, single-threaded engine state for one input.
- **Greedy quantifier:** `*`, `+`, `?` - consume as much as possible, then give back on failure.
- **Reluctant quantifier:** `*?`, `+?`, `??` - consume as little as possible, then take more.
- **Possessive quantifier:** `*+`, `++`, `?+` - consume maximally and never give back.
- **Atomic group:** `(?>...)` - once matched, discards its backtracking alternatives.
- **Capturing group:** `(...)` - numbered left to right by opening parenthesis.
- **Named group:** `(?<name>...)` - retrieved by name rather than index.
- **Lookahead / lookbehind:** `(?=)`, `(?!)`, `(?<=)`, `(?<!=)` - zero-width assertions.
- **Catastrophic backtracking:** exponential match time from nested ambiguous quantifiers.
- **Region:** the sub-range of input a `Matcher` is restricted to.

56.5 Detailed mechanics

Pattern is shared; Matcher is not

This is the single most important operational rule.

JAVA

```
public final class LogParser {
    // Compiled once. Immutable. Safe to share across every thread.
    private static final Pattern LEVEL =
        Pattern.compile("(?<level>TRACE|DEBUG|INFO|WARN|ERROR)\\s+(?<rest>.*)$");

    public Optional<String> level(String line) {
        Matcher matcher = LEVEL.matcher(line);    // fresh Matcher per call
        return matcher.matches() ? Optional.of(matcher.group("level")) : Optional.empty();
    }
}
```

Hoisting the `Pattern` to a `static final` field is not micro-optimization. `Pattern.compile` parses the expression and builds a node tree; doing that per request in a hot path is often the largest single cost in a parsing routine.

The convenience methods hide this. `String.matches`, `String.split`, and `String.replaceAll` all call `Pattern.compile` internally on every invocation. In a loop, replace them with a hoisted pattern.

JAVA

```
// Recompile the pattern on every iteration.
for (String line : lines) {
    if (line.matches("\\d{4}-\\d{2}-\\d{2}")) { ... }
}

// Compiles once.
private static final Pattern ISO_DATE = Pattern.compile("\\d{4}-\\d{2}-\\d{2}");
for (String line : lines) {
    if (ISO_DATE.matcher(line).matches()) { ... }
}
```

`String.split` has one documented fast path: a single-character, non-metacharacter separator bypasses the regex engine entirely. `split(",")` is cheap; `split("\\s*,\\s*")` is not.

matches, find, and lookingAt

Three methods, three anchoring rules, and a frequent source of wrong results.

- `matches()` - the pattern must consume the **entire** input.
- `lookingAt()` - the pattern must match at the **start**, need not reach the end.
- `find()` - the pattern may match **anywhere**, and successive calls advance through the input.

JAVA

```
Pattern digits = Pattern.compile("\\d+");
Matcher m = digits.matcher("abc123def456");

m.matches(); // false - the whole string is not digits
m.reset().lookingAt(); // false - does not start with digits
m.reset();
while (m.find()) {
    System.out.println(m.group() + " at " + m.start()); // 123 at 3, 456 at 9
}
```

A validation method that uses `find()` where it meant `matches()` accepts every string containing a valid fragment. This is how "email validation" regularly accepts `not-an-email a@b.co garbage`.

Greedy, reluctant, and possessive

Consider extracting the content of the first HTML-like tag from `<a><b>`.

JAVA

```
Pattern greedy = Pattern.compile("<(.)>"); // group: "a<b"
Pattern reluctant = Pattern.compile("<(.)?>"); // group: "a"
Pattern possessive = Pattern.compile("<(.)++>"); // no match at all
```

The greedy `.+` consumes to the end, then backs off one character at a time until the trailing `>` can match - landing on the last `>`. The reluctant `.+?` starts with one character and grows only as needed, landing on the first `>`. The possessive `.++` consumes everything and refuses to give any back, so the final `>` has nothing left to match and the whole attempt fails.

Possessive quantifiers and atomic groups are the deliberate tools for bounding backtracking. They say: once this part has matched, never reconsider it. When you know a sub-expression's match is unambiguous, making it possessive converts a potential exponential blowup into a linear scan.

Catastrophic backtracking

The classic failing shape is a quantifier applied to something that is itself ambiguously quantified, followed by something that fails.

JAVA

```
Pattern bad = Pattern.compile("(a+)+b");
bad.matcher("aaaaaaaaaaaaaaaaaaaaaaaaaaaaa!").matches();
```

The input has thirty **a** characters and no **b**. The engine must prove no match exists. **(a+)+** can partition those thirty characters into groups in exponentially many ways, and the trailing **b** fails for every one of them. Match time roughly doubles per added **a**. At thirty characters this takes seconds; at forty it outlives the request timeout; the thread is pinned and cannot be interrupted.

Three fixes, in order of preference:

JAVA

```
Pattern fixed1 = Pattern.compile("a+b");           // remove the redundant nesting
Pattern fixed2 = Pattern.compile("(?>a+)+b");      // atomic group: no re-partitioning
Pattern fixed3 = Pattern.compile("(a++)+b");       // possessive: same effect
```

The first is usually available and always best - the nesting was adding nothing. Reach for atomic groups when the structure is genuinely needed.

The engineering rule: any pattern applied to untrusted input should either be provably linear or be run with a bound. Since Java offers no timeout, the practical bound is a `CharSequence` wrapper whose `charAt` throws after a budget is exhausted, or running the match on a task you can abandon while accepting the pinned thread.

Groups, named groups, and numbering

Groups are numbered by the position of their **opening** parenthesis, left to right, starting at 1. Group 0 is the entire match. Nesting does not reset the count.

JAVA

```
Pattern p = Pattern.compile("((\\d{4})-(\\d{2}))-(\\d{2})");
Matcher m = p.matcher("2026-08-02");
m.matches();
m.group(0); // 2026-08-02
m.group(1); // 2026-08
m.group(2); // 2026
m.group(3); // 08
m.group(4); // 02
```

Counting parentheses is a maintenance hazard: inserting a group anywhere shifts every later index. Named groups remove the problem.

JAVA

```
Pattern p = Pattern.compile("(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})");
Matcher m = p.matcher("2026-08-02");
if (m.matches()) {
    String year = m.group("year");
}
```

Use `(?:...)` for grouping without capturing. It expresses intent and avoids allocating group state you will not read.

An unmatched group returns `null`, not an empty string - a routine `NullPointerException` source when a group sits behind an optional branch.

Lookaround

Lookaround asserts without consuming. The position does not advance, and the assertion contributes nothing to the match text.

JAVA

```
Pattern beforeK = Pattern.compile("\\d+(?=k)"); // digits followed by k
Pattern notK    = Pattern.compile("\\d+(?!k)"); // digits not followed by k
Pattern afterUsd = Pattern.compile("(?<=\\$)\\d+"); // digits preceded by $
```

Java requires lookbehind to be **bounded** in length - `(?<=a{1,5})` is legal, `(?<=a*)` is not. Lookahead has no such restriction.

Lookahead is the honest way to express "all of these must hold" without dictating order, which is why password-policy checks read as a chain of assertions rather than one tangled alternation.

Flags and Unicode

Flags may be passed to `compile` or embedded inline as `(?i)`.

- `CASE_INSENSITIVE` is ASCII-only unless combined with `UNICODE_CASE`.
- `DOTALL` makes `.` match line terminators; by default it does not.
- `MULTILINE` makes `^` and `$` match at line boundaries rather than only input boundaries.
- `COMMENTS` allows whitespace and `#` comments inside the pattern - genuinely worth using for anything long.

The default word boundary `\b` and shorthand classes such as `\w` are ASCII-oriented.

`UNICODE_CHARACTER_CLASS` redefines them against Unicode properties, which matters the moment real names or non-English text arrive.

JAVA

```
Pattern ascii = Pattern.compile("\\w+");
Pattern unicode = Pattern.compile("\\w+", Pattern.UNICODE_CHARACTER_CLASS);
// On a string containing "naïve" spelled with a diaeresis over the i,
// the ASCII pattern finds two fragments, "na" and "ve", because the accented
// character is not an ASCII word character. The Unicode pattern finds the
// whole word. The same split happens to any name outside ASCII.
```

Because Java strings are UTF-16, a character outside the Basic Multilingual Plane is two `char` values. `.` matches a full code point, but a hand-rolled class such as `[a-zA-Z]` reasons in code units and will happily split a surrogate pair.

Replacement pitfalls

In the replacement string of `replaceAll`, `$` introduces a group reference and `\` escapes. Any user-supplied replacement text must be escaped, or a stray `$1` in the data becomes a group reference and a stray `$` throws.

JAVA

```
String replacement = Matcher.quoteReplacement(userInput);
String result = pattern.matcher(source).replaceAll(replacement);
```

Symmetrically, `Pattern.quote` escapes user input used as a *pattern*, turning it into a literal. Building a pattern by concatenating unescaped user input is regular-expression injection: it is at best a wrong result and at worst a supplied `(a+)+b` that hangs the thread.

Prefer `replaceAll(Function<MatchResult, String>)` when the replacement is computed, since it avoids the escaping rules entirely.

TRACE IT | 56.6 Worked Java example

Extracting structured fields from a log line, with the pattern hoisted, named groups, and a bounded quantifier.

JAVA (continued 1/2)

```
import java.util.Optional;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public final class AccessLogParser {

    // Anchored, no nested quantifiers, bounded status field.
    private static final Pattern ENTRY = Pattern.compile(
        "^(<ip>[0-9.]{7,15})\\s+"
        + "\\[(?<timestamp>[^\]]++\\)\\s+" // possessive: content is unambiguous
        + "\"(?<method>[A-Z]{3,7})\\s(?<path>[^\"]++\\)\"\\s+"
        + "(?<status>\\d{3})\\s+"
        + "(?<bytes>\\d++)$");

    public record Entry(String ip, String timestamp, String method,
        String path, int status, long bytes) {}

    public Optional<Entry> parse(String line) {
        Matcher m = ENTRY.matcher(line);
        if (!m.matches()) {
```

JAVA (continued 2/2)

```

        return Optional.empty();
    }
    return Optional.of(new Entry(
        m.group("ip"),
        m.group("timestamp"),
        m.group("method"),
        m.group("path"),
        Integer.parseInt(m.group("status")),
        Long.parseLong(m.group("bytes"))));
    }

    public static void main(String[] args) {
        AccessLogParser parser = new AccessLogParser();
        String line = "10.0.0.7 [02/Aug/2026:14:22:01 +0000] \"GET /api/orders\" 200 4821";
        System.out.println(parser.parse(line).orElseThrow());
        // Entry[ip=10.0.0.7, timestamp=02/Aug/2026:14:22:01 +0000,
        //      method=GET, path=/api/orders, status=200, bytes=4821]
        System.out.println(parser.parse("garbage").isPresent()); // false
    }
}

```

Three deliberate choices. `matches()` rather than `find()`, so a partially valid line is rejected rather than silently accepted. The quantifiers are possessive, because each field's extent is fixed by its delimiter - there is no legitimate reason for the engine to reconsider them. Named groups mean adding a field later cannot renumber the existing ones.

The `path` group is worth dwelling on, because the obvious spelling is wrong. Writing it as `\\S++` looks natural - a path contains no whitespace - but `\"` is also not whitespace, so the possessive quantifier consumes the closing quote and then refuses to give it back. The pattern fails on every well-formed line. Negating the actual delimiter with `[^\" ]++` is correct: a possessive quantifier is only safe when its character class genuinely cannot overlap what follows it. This is the possessive trade-off in miniature - you gain a guarantee against backtracking and you give up the engine's ability to rescue an imprecise class.

TRACE IT | 56.7 Execution or memory walkthrough

`Pattern.compile` runs once at class initialization. It parses the expression into a linked tree of node objects - one per construct - and stores the group count and name-to-index map. This tree is immutable, which is what makes the `Pattern` shareable.

`ENTRY.matcher(line)` allocates a `Matcher` holding a reference to the input `CharSequence`, an int array of group boundaries sized to the group count, and the append position. Nothing is copied from the input.

`matches()` anchors at position 0 and requires the terminal node to sit at the input end. The engine walks the node tree, each node advancing the position and calling the next. When `[A-Z]{3,7}` matches `GET` and the following node fails, a greedy node would restore the saved position and retry at a shorter length; the possessive nodes in this pattern discard that saved state immediately, so those regions are scanned exactly once.

`m.group("ip")` resolves the name to an index, reads the start and end offsets from the int array, and calls `subSequence` - the only string allocation in the whole parse, and only for groups actually read. This is why

unnecessary capturing groups cost more than `(?:...)`: the boundary bookkeeping happens whether or not you read them.

56.8 Complexity and performance

For a pattern without ambiguous nesting, matching is $O(n \cdot m)$ in input length and pattern size, and in practice close to linear. For a pattern with nested ambiguous quantifiers, the worst case is **exponential** in input length, and the bad case is reached by a failing match, not a successful one - so tests built from valid inputs never find it.

Compilation is the dominant cost in code that recompiles. A hoisted `static final Pattern` removes it entirely.

Allocation comes from group extraction and from `split`, which builds an array of substrings. Where only presence matters, `find()` without calling `group()` allocates nothing beyond the `Matcher`.

HotSpot note: the matcher's inner loop is ordinary Java and inlines well, so a simple pattern over short input is genuinely fast. JIT compilation does not change algorithmic behavior: no amount of warmup rescues a pattern whose backtracking is exponential.

WATCH OUT | 56.9 Edge cases and common mistakes

- Compiling inside a loop or a request handler instead of hoisting to `static final`.
- Using `find()` for validation where `matches()` was meant, accepting any string with a valid fragment.
- Nested ambiguous quantifiers such as `(a+)+`, `(a*)*`, or `(\\s+)+` on untrusted input.
- Assuming a runaway match can be interrupted. It cannot; `Thread.interrupt` has no effect.
- Building a pattern from unescaped user input - regex injection. Use `Pattern.quote`.
- Forgetting `Matcher.quoteReplacement` for user-supplied replacement text containing `$` or `\`.
- Reading a group that did not participate in the match and dereferencing the `null`.
- Counting group numbers by hand, then inserting a group and shifting every later index.
- Expecting `\\w` and `\\b` to handle non-ASCII without `UNICODE_CHARACTER_CLASS`.
- Expecting `.` to cross a line terminator without `DOTALL`.
- Using an unbounded lookbehind, which Java rejects at compile time.
- Sharing a `Matcher` between threads. `Pattern` is safe; `Matcher` is not.
- Reaching for a regular expression to parse HTML, JSON, or any nested structure - these are not regular languages, and no pattern parses them correctly.
- Using `split` on a single-character separator with an escaped metacharacter, losing the fast path for no reason.

56.10 Production engineering notes

Treat every pattern that touches untrusted input as a potential availability risk. Review it for nested quantifiers, prefer possessive or atomic forms where the extent is unambiguous, and fuzz it with long non-matching inputs - the failing case is the dangerous one.

Keep patterns in named constants next to the code that uses them, with `COMMENTS` mode and inline documentation once they exceed a line. A pattern nobody can read is a pattern nobody can safely change.

Prefer a real parser when the input has structure. CSV with embedded quotes, HTML, and JSON all have grammars that regular expressions cannot express, and the "almost working" pattern is worse than an obvious dependency because it fails on the rare row.

Where a regex is user-configurable - routing rules, alert filters, redaction patterns - you have accepted arbitrary code with unbounded runtime from your users. Validate on submission, keep the evaluation off the request thread, and constrain what the configuration surface allows.

Log the pattern name rather than the pattern text when reporting a match failure, and never log the input if it may contain secrets. Redaction patterns in particular tend to run over exactly the data you must not emit.

TRY IT | 56.11 Interview questions and model answers

Is `Pattern` thread-safe? Is `Matcher`?

`Pattern` is immutable and safe to share; the standard practice is one `static final` instance per pattern. `Matcher` holds mutable position and group state, so each thread must create its own via `pattern.matcher(input)`.

What is catastrophic backtracking and how do you fix it?

A pattern with nested ambiguous quantifiers, such as `(a+)+b`, can partition the same input exponentially many ways. On a failing match the engine tries all of them, so time doubles with each added character. Fix it by removing the redundant nesting, or by making the inner quantifier possessive or the group atomic so the engine cannot re-partition.

What is the difference between greedy, reluctant, and possessive quantifiers?

Greedy consumes as much as possible and gives back on failure. Reluctant consumes as little as possible and takes more on failure. Possessive consumes maximally and never gives back, which prevents backtracking and can turn an exponential pattern into a linear one - at the cost of failing some matches a greedy quantifier would find.

Why is `String.matches` a problem in a loop?

It compiles the pattern on every call. Hoist a `static final Pattern` and reuse it; compilation is far more expensive than the match for short inputs.

How does `matches()` differ from `find()`?

`matches()` requires the pattern to consume the entire input; `find()` searches for a match anywhere and can be called repeatedly to iterate. Using `find()` for validation is a common defect, since it accepts any input containing a valid substring.

Can you parse HTML with a regular expression?

No. HTML is arbitrarily nested and regular expressions cannot count nesting depth. A pattern can extract from a known fixed shape, but any general parsing needs a real parser.

TRY IT | 56.12 Exercises

- 1 Write `(a+)+b`, time it against inputs of 20, 25, and 30 `a` characters with no `b`, and plot the growth. Then fix it three ways and re-measure.
- 2 Convert a validation method that uses `find()` to one that uses `matches()`, and find an input whose acceptance changes.
- 3 Take a pattern with six numbered groups, convert it to named groups, then insert a new group at the front and confirm nothing else breaks.
- 4 Demonstrate the difference between `\\w+` with and without `UNICODE_CHARACTER_CLASS` on a string containing accented characters and an emoji.
- 5 Write a `replaceAll` whose replacement text comes from user input containing `$1`, observe the failure, then fix it with `Matcher.quoteReplacement`.
- 6 Benchmark `split(",")` against `split("\\s*,\\s*")` on a large file and explain the gap using the single-character fast path.
- 7 Rewrite the `AccessLogParser` pattern with greedy quantifiers throughout and measure the difference on ten thousand lines, including malformed ones.

RECAP | 56.13 Chapter summary

Java's regex engine backtracks, and that one fact drives everything practical about it. `Pattern` is immutable, expensive to build, and meant to be hoisted and shared; `Matcher` is mutable, cheap, and strictly per-thread. Greedy quantifiers give back on failure, reluctant ones grow on demand, and possessive quantifiers and atomic groups refuse to reconsider - which is the primary defense against catastrophic backtracking, an exponential failure mode that only appears on inputs that *fail* to match and therefore survives most test suites. Anchor validation with `matches()` rather than `find()`, prefer named groups over hand-counted indices, escape user input with `Pattern.quote` and replacements with `Matcher.quoteReplacement`, and enable Unicode classes when real-world text arrives. When the input has nested structure, use a parser: the correct regular expression does not exist.

TRY IT | 56.14 Revision checklist

- ☐ I hoist every pattern to a `static final` field and never compile in a loop.
- ☐ I know `Pattern` is thread-safe and `Matcher` is not.
- ☐ I can explain `matches()`, `find()`, and `lookingAt()` and pick the right one for validation.
- ☐ I can predict what greedy, reluctant, and possessive quantifiers extract from `<a><b>`.
- ☐ I can recognize a catastrophic pattern and fix it three ways.
- ☐ I know a runaway match cannot be interrupted.
- ☐ I use named groups and `(?:...)` rather than counting parentheses.
- ☐ I escape user input with `Pattern.quote` and replacements with `Matcher.quoteReplacement`.
- ☐ I know when `\\w` and `\\b` need `UNICODE_CHARACTER_CLASS`.
- ☐ I can explain why regular expressions cannot parse HTML.

SDE-2 CORE CHAPTER

Chapter 57 - The Module System, JPMS, and Strong Encapsulation

57.1 Learning objectives

By the end of this chapter, you should be able to:

- explain what a module declares and why `public` stopped meaning "accessible to everyone";
- distinguish the module path from the class path, and named, automatic, and unnamed modules;
- read and write a `module-info.java` including `requires transitive`, `exports ... to`, and `opens`;
- diagnose `InaccessibleObjectException` and split-package errors from the message alone; and
- decide whether modularizing a given service is worth the cost.

WHY IT WORKS | 57.2 Why this matters at SDE-2

Most Java services are not modularized, and many engineers conclude the module system is therefore irrelevant. It is not, for two reasons.

First, the JDK itself is modular and has been since Java 9. Strong encapsulation of JDK internals became the default in Java 16 and was made final in Java 17, so reflective access into `java.base` now throws rather than warns. When a serialization library, a mocking framework, or an ORM fails with `InaccessibleObjectException` after a JDK upgrade, that is JPMS enforcing a boundary - and someone has to know what `--add-opens` actually does before pasting it into a startup script.

Second, the vocabulary appears in interviews as a proxy for how well you understand accessibility, class loading, and dependency hygiene. "Why does `public` no longer mean accessible?" is a question about encapsulation design, not about syntax.

WHY IT WORKS | 57.3 First-principles model

Before Java 9, the unit of reuse was the JAR and the unit of accessibility was the package. Neither was enforced meaningfully. Any JAR on the class path could read any `public` type in any other JAR; the class path was a flat, ordered search list with no notion of dependencies; two JARs could both contain `com.example.util` and the loader would silently take whichever came first.

A **module** adds a name, an explicit dependency list, and an explicit export list to a set of packages. Three consequences follow directly:

- 1 **Accessibility becomes two-dimensional.** A type is accessible only if it is `public` *and* its package is exported by its module *and* the reading module requires that module. `public` became necessary but no longer sufficient.
- 2 **Dependencies become declared and verified.** The module graph is resolved at startup. A missing module is an error at launch rather than a `NoClassDefFoundError` on the unlucky code path six hours in.
- 3 **Packages become unique.** A package may belong to only one module in a configuration. Split packages, previously silent, are now a startup failure.

Specification boundary: Java specifies module declarations, the readability and accessibility rules, and resolution at startup. It does not specify versions. `module-info.java` has no version constraint syntax and the module system performs no version selection or conflict resolution - that remains entirely the job of Maven, Gradle, or whatever builds your module path.

57.4 Core terminology

- **Named module:** has a `module-info.class`; declares its own name, requires, and exports.
- **Automatic module:** a plain JAR placed on the module path; gets a name, reads everything, exports everything.
- **Unnamed module:** everything loaded from the class path; reads all modules, exports all its packages.
- **Module path:** `--module-path / -p`; entries are resolved as modules.
- **Class path:** `-cp`; entries land in the unnamed module with legacy behavior.
- **requires:** this module reads another.
- **requires transitive:** readers of this module also read that one - implied readability.
- **requires static:** needed at compile time, optional at run time.
- **exports:** package is accessible at compile time and run time.
- **opens:** package is available for deep reflection at run time only.
- **uses / provides:** the `ServiceLoader` declaration pair.
- **Strong encapsulation:** non-exported packages are inaccessible even to reflection.

57.5 Detailed mechanics

Reading a module declaration

`module-info.java` sits at the source root and compiles to `module-info.class`.

JAVA

```
module com.example.orders {
    requires java.sql;                // I use JDBC types internally
    requires transitive com.example.model; // my API signatures expose model types
    requires static com.example.codegen; // compile-time only annotation processor

    exports com.example.orders.api;    // public API
    exports com.example.orders.spi to com.example.admin; // qualified: one consumer only

    opens com.example.orders.entity;    // deep reflection for the ORM

    uses com.example.orders.spi.PricingRule; // I consume this service
    provides com.example.orders.spi.PricingRule
        with com.example.orders.internal.StandardPricing; // I supply this implementation
}
```

Everything not listed is inaccessible. `com.example.orders.internal` is not exported, so no other module can reference it - the compiler rejects the import and reflection throws at run time.

requires transitive and API leakage

`requires transitive` exists for one situation: when your public API's signatures mention types from another module.

JAVA

```
// In com.example.orders
public Order lookup(CustomerId id); // CustomerId comes from com.example.model
```

Any caller of `lookup` must be able to name `CustomerId`. Without `requires transitive com.example.model`, every consumer would have to add its own `requires com.example.model` - and would be baffled as to why. The rule is mechanical: if a type from module `M` appears in your exported signatures, `requires transitive M`. Otherwise use plain `requires`, which keeps the dependency an implementation detail you can change later.

This makes API leakage visible in a way the class path never did. A module needing ten `requires transitive` entries is telling you its public surface depends on ten other modules.

exports versus opens

This distinction is the one that actually bites in production.

- `exports` grants **compile-time and run-time access to public types**. It does not grant reflective access to non-public members. `setAccessible(true)` on a private field in an exported-but-not-opened package throws.
- `opens` grants **run-time deep reflection** into all members, including private ones, but grants nothing at compile time.

Frameworks that populate fields reflectively - JPA providers, Jackson, Spring, most mocking libraries - need **opens**, not **exports**. Entity packages are the canonical case: you rarely want application code importing entity internals, but Hibernate must reach private fields.

JAVA

```
opens com.example.orders.entity;           // to everyone
opens com.example.orders.entity to org.hibernate.orm.core; // qualified, preferred
```

`open module com.example.orders { ... }` opens every package at once. It is the pragmatic escape hatch when migrating a large codebase, and it discards most of the encapsulation benefit, so treat it as a transition state rather than a destination.

The three kinds of module

Understanding migration requires understanding what happens to code that has no `module-info`.

Unnamed module - everything on the class path. It reads every other module, and all its packages are exported. This is why an unmodularized application still runs on a modern JDK: the unnamed module is deliberately permissive. But no named module can **requires** the unnamed module, because it has no name. That asymmetry is the entire difficulty of incremental migration: you must modularize bottom-up, dependencies first.

Automatic module - a plain JAR on the *module* path. It gets a name (from `Automatic-Module-Name` in the manifest, or derived from the filename), reads every other module including the unnamed one, and exports all its packages. It is the bridge that lets a named module depend on a not-yet-modularized library.

Deriving a module name from a filename is fragile - the name changes if the artifact is renamed - so a library that has not modularized should at minimum publish `Automatic-Module-Name` in its manifest. That is a one-line manifest addition and it stabilizes the name for every downstream consumer.

Named module - has `module-info.class`, enforces its declaration.

Strong encapsulation of the JDK

The change most engineers actually encounter. JDK internals such as `sun.misc.Unsafe` and much of `java.lang` reflection were accessible on Java 8, warned about in 9 through 15, and denied from 16 onward.

TEXT

```
java.lang.reflect.InaccessibleObjectException: Unable to make
field private final java.lang.String java.lang.String.value accessible:
module java.base does not "opens java.lang" to unnamed module @1b6d3586
```

The message is precise and worth reading closely: it names the field, the owning module (`java.base`), the missing directive (`opens java.lang`), and the requesting module (`unnamed`, so the caller is on the class path). The corresponding flag mirrors that structure exactly:

BASH

```
java --add-opens java.base/java.lang=ALL-UNNAMED -jar app.jar
```

`--add-opens module/package=target` grants deep reflection; `--add-exports` grants only public access. `ALL-UNNAMED` targets everything on the class path.

Two warnings. First, these flags are a compatibility bridge, not a fix - they belong in a build file with a comment naming the library that needs them and the ticket to remove them. Second, they must be present at every launch, so a flag that works locally and is missing from the container entrypoint produces a failure that appears only in deployment.

Split packages

A package may exist in only one module on a given path. Two JARs both containing `com.example.util` fail at startup rather than silently shadowing:

TEXT

```
Error occurred during initialization of boot layer
java.lang.LayerInstantiationException: Package com.example.util in both
module lib.b and module lib.a
```

This surfaces genuine problems - shaded JARs, forked libraries, an old `javax.annotation` split - that the class path had been hiding.

Services

JPMS makes `ServiceLoader` a first-class, declared relationship. The consumer declares `uses`, the provider declares `provides ... with`, and resolution binds them without either module referencing the other's implementation package.

JAVA

```
// consumer module
uses com.example.orders.spi.PricingRule;

// provider module, in a different artifact
provides com.example.orders.spi.PricingRule with com.example.pricing.RegionalPricing;
```

The implementation class need not be in an exported package. This is genuinely better than the `META-INF/services` file it replaces: it is compile-checked, and a typo is a compilation error rather than a silent empty iterator.

jlink and runtime images

Because dependencies are declared, the module graph can be computed and a custom runtime image containing only the reachable modules can be produced.

BASH

```
jlink --add-modules com.example.orders --output runtime --strip-debug --no-man-pages
```

For a service using a modest slice of the JDK this can cut a runtime image from a few hundred megabytes to under 60 - a real container-image win. [jlink](#) requires every module in the graph to be a named module, which in practice is the strongest concrete incentive to modularize.

TRACE IT | 57.6 Worked Java example

A two-module layout where the boundary is the point.

JAVA

```
// ----- module com.example.model -----
// src/com.example.model/module-info.java
module com.example.model {
    exports com.example.model.api;
    // com.example.model.internal is NOT exported
}

// src/com.example.model/com/example/model/api/CustomerId.java
package com.example.model.api;

public record CustomerId(String value) {
    public CustomerId {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException("customer id required");
        }
    }
}

// src/com.example.model/com/example/model/internal/IdCodec.java
package com.example.model.internal;

public final class IdCodec {           // public, but unreachable outside the module
    public static String encode(String raw) { return raw.strip().toUpperCase(); }
}
```

JAVA

```
// ----- module com.example.orders -----
// src/com.example.orders/module-info.java
module com.example.orders {
    requires transitive com.example.model; // CustomerId appears in my API
    exports com.example.orders.api;
}

// src/com.example.orders/com/example/orders/api/OrderService.java
package com.example.orders.api;

import com.example.model.api.CustomerId;
// import com.example.model.internal.IdCodec; // compile error: not exported

public final class OrderService {
    public String describe(CustomerId id) {
        return "orders for " + id.value();
    }

    public static void main(String[] args) {
        System.out.println(new OrderService().describe(new CustomerId("c-100")));
        // orders for c-100
    }
}
```

Compile and run both modules:

BASH

```
javac -d out --module-source-path src $(find src -name '*.java')
java --module-path out --module com.example.orders/com.example.orders.api.OrderService
```

Uncommenting the `IdCodec` import produces a compile error naming the reason:

TEXT

```
error: package com.example.model.internal is not visible
(package com.example.model.internal is declared in module
com.example.model, which does not export it)
```

`IdCodec` is `public`. It is still unreachable. That is the whole idea in one error message.

TRACE IT | 57.7 Execution or memory walkthrough

At launch, the module system resolves the graph before any application class loads. It reads the root modules named by `--module` or `--add-modules`, transitively reads their `requires` edges, and fails immediately on a missing module, a duplicate module name, a split package, or a cycle. Resolution produces a **configuration**, which is instantiated as the boot **layer**.

Each module in the layer is assigned to a class loader - the built-in application loader for the module path - and the loader records which packages belong to which module. This is why a split package is detectable at all: the mapping from package to module must be a function.

At link time, a reference from `com.example.orders` to `CustomerId` is checked twice. The compiler verifies that `com.example.orders` reads `com.example.model` and that the package is exported. The JVM verifies the same facts again during resolution, because a module path can differ at run time from the one used at compile time.

An `IdCodec` reference would fail the second check with `IllegalAccessError` even if it somehow passed the first - which is the meaningful difference from the class path, where the check did not exist.

Reflection consults the same tables. `setAccessible(true)` asks whether the target's package is *open* to the caller's module; the answer is a table lookup, and a negative answer throws `InaccessibleObjectException`. No bytecode is rewritten and no proxy is generated; the enforcement is a runtime check in `AccessibleObject`.

57.8 Complexity and performance

Resolution is roughly linear in the number of modules and edges, and it happens once. For a typical service the cost is a few milliseconds against a JVM startup measured in hundreds.

Access checks are performed at resolution or first access and then cached in the constant pool, so steady-state throughput is unaffected. There is no per-call module check.

The measurable wins are startup and image size. A `jlink` image containing only reachable modules loads a smaller class-data-sharing archive and produces a much smaller container image. The measurable cost is build complexity, which is where modularization is usually paid for.

HotSpot note: module boundaries do not create optimization barriers. The JIT inlines across modules exactly as it does within one, because accessibility is resolved before compilation. Modules are a correctness and packaging mechanism, not a performance one.

WATCH OUT | 57.9 Edge cases and common mistakes

- Assuming `public` still means accessible. It requires `public` plus `exports` plus `requires`.
- Using `exports` where a framework needs `opens`, then debugging `InaccessibleObjectException`.
- Using `opens` where `exports` was meant, giving reflective access but no compile-time access.
- Omitting `requires transitive` when an exported signature mentions another module's type, forcing every consumer to add a dependency it cannot explain.
- Expecting `module-info.java` to express versions. It cannot; version selection belongs to the build tool.
- Expecting a named module to `requires` the unnamed module. It cannot - migration must go bottom-up.
- Relying on a filename-derived automatic module name, which changes when the artifact is renamed.
- Split packages from shaded or forked JARs, now a hard startup failure.
- Adding `--add-opens` at compile time only and omitting it from the runtime launch command.
- Putting the same JAR on both the class path and the module path, producing two copies with different identities.
- Treating `open module` as the finished state rather than a migration step.
- Forgetting that `requires static` supplies compile-time visibility only; the type must be absent-safe at run time.
- Assuming `jlink` works with automatic modules. It does not - every module in the graph must be named.

57.10 Production engineering notes

Be honest about whether modularization pays. For a Spring Boot service deployed as a fat JAR, the answer is usually no: the framework does extensive reflection, many dependencies are unmodularized, and you gain little over a well-structured build with enforced package rules. For a published library, a CLI, or anything shipping a runtime image, the answer is often yes.

If you do not modularize, still add `Automatic-Module-Name` to any JAR you publish. It costs one manifest line and spares every downstream consumer a name that changes when you rename a file.

Record every `--add-opens` and `--add-exports` in the build with a comment naming the library that requires it and the condition for removing it. These flags accumulate silently and become a list nobody dares touch. Re-test them on each JDK and library upgrade - most eventually become unnecessary.

Put the flags where every launch sees them. `MANIFEST.MF` supports `Add-Opens` for executable JARs, and the `JDK_JAVA_OPTIONS` environment variable is honored by the `java` launcher. A flag configured only in a local IDE run configuration is a production incident waiting for a deploy.

When a JDK upgrade breaks reflective access, prefer upgrading the offending library over adding a flag. `InaccessibleObjectException` is usually a library reaching into internals that have a supported alternative, and the maintainers have generally already fixed it.

TRY IT | 57.11 Interview questions and model answers

Why is a `public` class not always accessible in Java 9 and later?

Accessibility now has two dimensions. The type must be `public`, its package must be exported by its module, and the calling module must read that module. A `public` class in a non-exported package is visible to its own module only, which is what makes internal APIs genuinely internal.

What is the difference between `exports` and `opens`?

`exports` grants compile-time and run-time access to public members. `opens` grants run-time deep reflection into all members, including private ones, and grants nothing at compile time. Frameworks that populate private fields need `opens`; ordinary API consumers need `exports`.

When do you need `requires transitive`?

When a type from the required module appears in your own exported signatures. Consumers must be able to name those types, and `requires transitive` gives them implied readability instead of forcing each one to declare a dependency it does not obviously use.

What is an automatic module?

A plain JAR on the module path. It takes its name from `Automatic-Module-Name` or the filename, reads every other module including the unnamed one, and exports all its packages. It exists so a named module can depend on a library that has not modularized yet.

How do you fix `InaccessibleObjectException`?

Read the message - it names the module, the package, and the caller. Then either add `opens` to your own module declaration if you own the code, or pass `--add-opens module/package=ALL -UNNAMED` at launch.

Preferably upgrade the library instead, since the exception usually means it is reaching into internals that now have a supported replacement.

Why does the module system reject split packages?

Each package must map to exactly one module so that accessibility and class loading are well defined. On the class path, duplicate packages silently resolved by search order, which produced defects that depended on artifact ordering. JPMS makes it a startup error.

TRY IT | 57.12 Exercises

- 1 Build the two-module example, then uncomment the `IdCodec` import and read the compiler error carefully.
- 2 Add a third module that requires `com.example.orders` and uses `CustomerId`. Remove `transitive` and explain the resulting error.
- 3 Write a class that calls `setAccessible(true)` on a private field of a non-opened package, observe the exception, and fix it with `opens` and again with `--add-opens`.
- 4 Create two JARs sharing a package name, put both on the module path, and read the `LayerInstantiationException`.
- 5 Convert a small library to a named module and run `jlink`, comparing the runtime image size to a full JDK.
- 6 Take a JAR with no `Automatic-Module-Name`, place it on the module path, and print its derived name with `java --list-modules`. Rename the file and print again.
- 7 Declare a `uses/provides` service pair across two modules and confirm `ServiceLoader` finds the implementation without the consumer importing it.

RECAP | 57.13 Chapter summary

The module system adds a name, an explicit dependency list, and an explicit export list to a group of packages, which changes accessibility from a one-dimensional `public` check into a three-part test: public, exported, and required. `requires transitive` propagates readability for types that appear in your API; `exports` grants ordinary access while `opens` grants deep reflection, and confusing the two is the most common practical failure. Unnamed and automatic modules keep legacy code working and make bottom-up migration possible, at the cost of an asymmetry - named modules can never require the unnamed one. Strong encapsulation of JDK internals since Java 16 is where most engineers first meet JPMS, through `InaccessibleObjectException` and the `--add-opens` flag that answers it. The module system verifies structure, not versions, and modularizing is a judgment call: often unnecessary for a fat-JAR service, often worthwhile for a published library or a `jlink` runtime image.

TRY IT | 57.14 Revision checklist

- ☐ I can state the three conditions required for a type to be accessible.
- ☐ I can explain `exports` versus `opens` and which one an ORM needs.
- ☐ I know when `requires transitive` is mandatory and why.
- ☐ I can describe named, automatic, and unnamed modules and how each behaves.
- ☐ I know why a named module cannot require the unnamed module.
- ☐ I can read an `InaccessibleObjectException` and derive the exact `--add-opens` flag.
- ☐ I can explain why split packages are now a startup failure.
- ☐ I know `module-info.java` carries no version information.
- ☐ I can declare a `uses/provides` service pair.
- ☐ I can argue both sides of whether a given service should be modularized.

SDE-2 CORE CHAPTER

Chapter 58 - Java 22 to 25: The Modern LTS Baseline

58.1 Learning objectives

By the end of this chapter, you should be able to:

- state what changed between the Java 21 and Java 25 long-term-support baselines;
- use stream gatherers, the Class-File API, and the Foreign Function and Memory API where they replace older approaches;
- explain why virtual threads behave differently under `synchronized` from Java 24 onward;
- distinguish a finalized feature from a preview feature and justify a production policy for each; and
- answer "what version should we target" with reasoning rather than a number.

WHY IT WORKS | 58.2 Why this matters at SDE-2

This book's baseline is Java 21, the LTS release most teams standardized on. Java 25, released 16 September 2025, is the next LTS, and the four releases between them changed several things an SDE-2 is expected to reason about - most consequentially, the pinning behavior of virtual threads under `synchronized`.

Interviewers do not test JEP numbers. They test whether you can say what a feature is *for*, whether you know the difference between "previewed" and "shipped," and whether you have an opinion about upgrade cadence. An engineer who says "we're on 21 and we'll move to 25 once our agent vendors certify it, mainly for the `synchronized` pinning fix" is demonstrating exactly the judgment the level requires.

WHY IT WORKS | 58.3 First-principles model

Since Java 9, releases ship every six months on a fixed date. Features are not held for a release; a release takes whatever is ready. Roughly every two years a release is designated **long-term support**, which is a *vendor* commitment to backported security and bug fixes, not a technical property of the JDK. Java 8, 11, 17, 21, and 25 are the LTS line.

Language and API changes arrive through a staged process:

- **Incubator** - an API in a separate `jdk.incubator` module, expected to change.
- **Preview** - a fully specified feature, disabled by default, requiring `--enable-preview` at compile *and* run time. Preview class files are tied to the exact JDK version that produced them.
- **Final** - permanent, subject to normal compatibility rules.

The staging matters commercially. Preview features can be revised or withdrawn: String Templates previewed in Java 21 and 22, then were removed entirely before Java 23 because the design was not right. Anyone who had

shipped code depending on them had to rewrite it. That is the argument for the policy stated later in this chapter.

Specification boundary: Java specifies the language and API changes in each release, and the preview mechanism that gates them. It does not specify support duration, backport policy, or which vendor's build you run. "LTS" is a distribution commitment from Oracle, Temurin, Amazon, Red Hat, and others - their support windows differ, and so do their patch levels.

58.4 Core terminology

- **LTS:** a release vendors commit to supporting for years; 8, 11, 17, 21, 25.
- **Preview feature:** fully specified but provisional; requires `--enable-preview`.
- **Incubator module:** provisional API under `jdk.incubator.*`.
- **Pinning:** a virtual thread that cannot unmount, blocking its carrier platform thread.
- **Gatherer:** a user-defined intermediate stream operation.
- **Arena:** the lifetime scope controlling native memory in the FFM API.
- **Compact object headers:** 8-byte object headers instead of 12 or 16.
- **AOT cache:** ahead-of-time class loading and linking state captured from a training run.

58.5 Detailed mechanics

Java 22 - Foreign Function and Memory API (JEP 454, final)

The FFM API reached final status in Java 22, giving Java a supported way to call native code and access off-heap memory without JNI.

JAVA

```
import java.lang.foreign.*;
import java.lang.invoke.MethodHandle;

try (Arena arena = Arena.ofConfined()) {
    MemorySegment name = arena.allocateFrom("world");

    Linker linker = Linker.nativeLinker();
    MethodHandle strlen = linker.downcallHandle(
        linker.defaultLookup().find("strlen").orElseThrow(),
        FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS));

    long length = (long) strlen.invoke(name);    // 5
}
```

Two things make this a genuine replacement for JNI rather than a variation on it. There is no C shim to compile and ship - the binding is expressed in Java. And memory lifetime is explicit: an **Arena** is a scope, closing it frees every segment allocated from it, and touching a freed segment throws **IllegalStateException** rather than

corrupting the heap. `Arena.ofConfined()` further restricts access to the creating thread, so use-after-free and cross-thread races become exceptions instead of undefined behavior.

This is also the intended replacement for the parts of `sun.misc.Unsafe` that deal with memory access, which were deprecated for removal in Java 23 and now warn at runtime.

Unnamed variables and patterns (JEP 456, final) let `_` stand for a binding you must declare but will not use:

JAVA

```
try {
    process(order);
} catch (IOException _) {           // named but unused
    metrics.increment("io.failure");
}

for (var _ : items) { count++; }

if (obj instanceof Point(int x, _)) { ... } // ignore the second component
```

Small, but it removes a category of misleading names and lets the compiler verify that an ignored binding really is ignored.

Java 23 - Markdown in Javadoc (JEP 467, final)

Documentation comments may be written in Markdown using `///`:

JAVA

```
/// Returns the customer's outstanding balance.
///
/// - Never negative.
/// - Excludes pending authorizations.
///
/// @param id the customer identifier
public Money balance(CustomerId id) { ... }
```

The practical effect is that documentation gets written, because the friction of HTML tags in comments was real.

ZGC became generational by default, and the non-generational mode was removed in Java 24. If you tuned ZGC flags on 21, revisit them.

Java 24 - the release that matters most operationally

Synchronize virtual threads without pinning (JEP 491) is the single most important change between the two LTS releases.

In Java 21, a virtual thread that blocked inside a `synchronized` block or method could not unmount from its carrier platform thread. The carrier stayed pinned for the duration. With a default scheduler parallelism equal to the core count, a handful of virtual threads blocking inside `synchronized` on a database call could stall every other virtual thread in the JVM. The recommended workaround was to replace `synchronized` with `ReentrantLock` in any code path a virtual thread might block on - a real, invasive migration cost.

From Java 24, virtual threads unmount correctly while holding a monitor. The workaround becomes unnecessary, and the most common reason for virtual-thread adoption to disappoint goes away.

JAVA

```
// On Java 21 this pins the carrier for the duration of the call.
// From Java 24 the virtual thread unmounts and the carrier is reused.
synchronized (lock) {
    result = jdbcTemplate.query(...);
}
```

Pinning still occurs inside native frames and class initializers. But if your team evaluated virtual threads on 21, hit carrier starvation, and shelved them, the evaluation is worth repeating on 25.

Stream gatherers (JEP 485, final) finally make intermediate stream operations extensible. Before this, you could write any terminal operation you liked via [Collector](#), but the intermediate stage was a closed set.

JAVA

```
// Sliding window of 3 - impossible to express before gatherers
List<List<Integer>> windows = Stream.of(1, 2, 3, 4, 5)
    .gather(Gatherers.windowSliding(3))
    .toList();
// [[1, 2, 3], [2, 3, 4], [3, 4, 5]]

// Fixed batches, useful for chunked writes
List<List<String>> batches = ids.stream()
    .gather(Gatherers.windowFixed(500))
    .toList();
```

Built-in gatherers cover [fold](#), [scan](#), [mapConcurrent](#), [windowFixed](#), and [windowSliding](#).

[Gatherer.of](#) builds custom ones with an initializer, an integrator, an optional combiner for parallel streams, and a finisher. [mapConcurrent](#) deserves particular attention: it runs a mapping function on virtual threads with a concurrency limit, which is a clean way to fan out I/O without hand-managing an executor.

The Class-File API (JEP 484, final) gives the JDK a supported library for parsing and generating class files, replacing the widespread dependency on ASM. If you work on agents, instrumentation, or build tooling, this removes a dependency that had to be upgraded in lockstep with every JDK.

Ahead-of-time class loading and linking (JEP 483) records the loaded and linked state of classes from a training run and replays it at startup, cutting JVM startup time substantially for large applications. Java 25 adds command-line ergonomics (JEP 514) and method profiling (JEP 515) on top.

The Security Manager was permanently disabled (JEP 486). It had been deprecated since Java 17; code still calling [System.setSecurityManager](#) now fails.

Java 25 - the new LTS

Scoped values (JEP 506, final) provide immutable, inheritable, bounded data sharing - the replacement for `ThreadLocal` in a virtual-thread world.

JAVA

```
private static final ScopedValue<RequestContext> CONTEXT = ScopedValue.newInstance();

ScopedValue.where(CONTEXT, new RequestContext(traceId, tenantId))
    .run(() -> handleRequest()); // CONTEXT.get() visible to everything called here

// Outside the run block, CONTEXT.get() throws NoSuchElementException
```

`ThreadLocal` was designed for a few hundred pooled platform threads. It is mutable, unbounded in lifetime, inherited by copying, and a well-known leak source. With millions of virtual threads, copying an inheritable `ThreadLocal` map per thread is untenable. A `ScopedValue` is immutable, its lifetime is exactly the dynamic extent of the `run` call, and child threads in a structured scope share the binding rather than copying it.

Compact object headers (JEP 519) move from experimental to production-ready, reducing object headers from 12 or 16 bytes to 8. For allocation-heavy applications with many small objects this is a real heap reduction - commonly reported in the 10-20% range - and it is enabled with `-XX:+UseCompactObjectHeaders`.

Module import declarations (JEP 511, final) import every package a module exports:

JAVA

```
import module java.base; // java.util, java.io, java.nio.file, ... all visible

List<String> names = new ArrayList<>();
Path path = Path.of("orders.csv");
```

Useful in scripts and teaching material. In a production codebase, explicit imports still document dependencies better.

Compact source files and instance main methods (JEP 512, final) finish the four-release effort to shorten the first program a learner writes:

JAVA

```
void main() {
    IO.println("Hello");
}
```

No class declaration, no `static`, no `String[] args`, no `System.out`. Combined with `java Program.java` and multi-file source launching from Java 22, Java is now genuinely scriptable.

Flexible constructor bodies (JEP 513, final) allow statements before `super(...)` or `this(...)`, provided they do not reference the instance under construction:

JAVA

```
public Order(List<Item> items) {  
    if (items.isEmpty()) {                // validate BEFORE the super call  
        throw new IllegalArgumentException("order requires at least one item");  
    }  
    super(computeTotal(items));  
    this.items = List.copyOf(items);  
}
```

Previously this validation had to be smuggled into a static helper inside the `super` argument list. The change also lets you avoid the classic bug where a superclass constructor calls an overridable method before the subclass has initialized its fields.

Still preview in Java 25: structured concurrency (fifth preview), stable values, primitive types in patterns, PEM encodings. Structured concurrency in particular has been through five previews with API changes each time - a reminder that "preview" is a real warning, not a formality.

Java 26 and beyond

Java 26 reached general availability on 17 March 2026 with a frozen feature set including HTTP/3 support in the HTTP client (JEP 517), removal of the Applet API (JEP 504), ahead-of-time object caching with any GC (JEP 516), and "prepare to make final mean final" (JEP 500) - which begins closing the loophole allowing reflective mutation of `final` fields. It is not an LTS release. The next LTS is expected to be Java 29 in September 2027.

TRACE IT | 58.6 Worked Java example

A batching pipeline that would have needed hand-written iterator code before Java 24, combining gatherers with `mapConcurrent`.

JAVA

```
import java.util.List;
import java.util.stream.Gatherers;
import java.util.stream.Stream;

public final class BulkEnricher {

    record Customer(String id, String name) {}

    /**
     * Fetches customers in batches of 100, enriching up to 8 batches
     * concurrently on virtual threads, and flattens the result.
     */
    public List<Customer> enrichAll(List<String> ids) {
        return ids.stream()
            .gather(Gatherers.windowFixed(100))           // List<String> batches
            .gather(Gatherers.mapConcurrent(8, this::fetchBatch))
            .flatMap(List::stream)
            .toList();
    }

    private List<Customer> fetchBatch(List<String> batch) {
        // One network round trip per batch; blocking is fine on a virtual thread.
        return batch.stream().map(id -> new Customer(id, "name-" + id)).toList();
    }

    public static void main(String[] args) {
        List<String> ids = Stream.iterate(1, i -> i + 1).limit(250)
            .map(i -> "c-" + i).toList();
        List<Customer> all = new BulkEnricher().enrichAll(ids);
        System.out.println(all.size());           // 250
        System.out.println(all.getFirst());       // Customer[id=c-1, name=name-c-1]
    }
}
```

Three batches of 100, 100, and 50 are produced by `windowFixed`; `mapConcurrent` runs up to eight of them at a time on virtual threads while preserving encounter order in the output. Before gatherers, expressing "batch then fan out with a concurrency limit" as a stream was not possible - you dropped to a loop with an `ExecutorService` and reassembled the ordering yourself.

TRACE IT | 58.7 Execution or memory walkthrough

`windowFixed(100)` is a stateful gatherer. Its integrator accumulates elements into a private buffer, and when the buffer reaches 100 it pushes the list downstream and starts a new one. The finisher pushes the trailing partial window - the 50-element remainder. Because it holds mutable state, it declares itself unsuitable for parallel splitting; the stream stays sequential regardless of `parallel()`.

`mapConcurrent(8, fn)` maintains a bounded set of in-flight tasks, each on its own virtual thread, and a queue that preserves encounter order. When the ninth batch arrives, the gatherer blocks the calling thread until a slot frees. Blocking here is cheap precisely because the workers are virtual threads: each unmounts from its carrier while waiting on I/O, so eight concurrent network calls consume eight heap-allocated stacks rather than eight OS threads.

On Java 21 the same code would work but any `synchronized` block inside `fetchBatch` - including one inside a JDBC driver - could pin a carrier thread. On Java 25 the virtual thread unmounts and the carrier is returned to the pool.

`flatMap(List::stream).toList()` allocates the final list once, sized from the spliterator estimate where available.

58.8 Complexity and performance

Gatherers add no asymptotic cost: `windowFixed` is $O(n)$ time and $O(w)$ space for window size w ; `windowSliding` is $O(n-w)$ in output volume, which is inherent to producing overlapping windows. `mapConcurrent` bounds memory by its concurrency limit rather than the input size - the reason to prefer it over submitting every element to an executor.

FFM downcalls cost roughly what a JNI call costs for simple signatures and less for many, because there is no JNI shim and the linker can specialize the handle. Hoist `MethodHandle` construction to a `static final` field; building it per call dominates everything else.

Compact object headers reduce heap by 4 or 8 bytes per object. For a workload dominated by small objects - nodes, boxed values, short strings - this is a double-digit percentage of live heap, and it improves cache density as well as footprint.

AOT class loading and linking mainly moves work out of startup. It does not change steady-state throughput; it changes time-to-first-request, which is what matters for scale-to-zero and serverless deployments.

HotSpot note: the virtual-thread pinning fix in JEP 491 is a JVM change, not a library change. Recompilation is not required - running existing Java 21 bytecode on a Java 25 JVM gets the improved behavior. This is the general shape of runtime improvements: upgrading the JVM often delivers more than adopting new syntax.

WATCH OUT | 58.9 Edge cases and common mistakes

- Treating a preview feature as stable. String Templates previewed twice and were withdrawn; anyone who shipped on them rewrote that code.
- Forgetting that `--enable-preview` is required at both compile time and run time, and that preview class files are pinned to the exact JDK version.
- Assuming "LTS" is a JDK property. It is a vendor support commitment, and windows differ by vendor.
- Concluding virtual threads are unusable after testing on Java 21 and hitting `synchronized` pinning that Java 24 fixed.
- Believing Java 24 removed pinning entirely. Native frames and class initializers still pin.
- Using `ThreadLocal` for request context with virtual threads. Prefer `ScopedValue` in Java 25.
- Using a stateful gatherer and expecting `parallel()` to help. Stateful gatherers suppress splitting.
- Building an FFM `MethodHandle` inside the hot path instead of hoisting it.
- Letting an `Arena` escape its try-with-resources scope, then touching a freed `MemorySegment`.
- Using `Arena.ofShared()` when `ofConfined()` suffices, giving up the thread-confinement checks.
- Still calling `System.setSecurityManager`, which fails from Java 24.
- Tuning non-generational ZGC flags that no longer exist after Java 24.
- Enabling `-XX:+UseCompactObjectHeaders` without measuring; benefits depend heavily on object size distribution.
- Assuming an agent, profiler, or bytecode-manipulation library works across a major upgrade. These break most often, and they are usually what actually blocks an upgrade.

58.10 Production engineering notes

Adopt finalized features freely and preview features never - in production code. Preview is genuinely useful for evaluation and for giving feedback to the JDK team, and that work belongs in a spike branch, not in a service you are on call for.

Separate the JVM upgrade from the language-level upgrade. Running Java 21 bytecode on a Java 25 JVM delivers the pinning fix, generational ZGC, compact object headers, and the AOT startup work with no source changes. Bumping `--release` to 25 is a second, independent decision. Doing them together makes a regression twice as hard to attribute.

Inventory your agents first. Profilers, APM agents, mocking frameworks, and anything using ASM or reaching into `sun.misc.Unsafe` are the usual blockers. The Class-File API existing does not mean your dependencies have migrated to it.

Expect `sun.misc.Unsafe` memory-access warnings on 23 and later. They are deprecation notices pointing at FFM and `VarHandle` replacements. Find which dependency triggers them before removal becomes mandatory rather than after.

Re-evaluate virtual threads on 25 if you shelved them on 21. Also re-run any performance baseline: generational ZGC, compact object headers, and improved G1 synchronization all shift the numbers your capacity model was built on.

TRY IT | 58.11 Interview questions and model answers

What changed between Java 21 and Java 25 that would affect an existing service?

Operationally, the largest change is JEP 491 in Java 24: virtual threads no longer pin their carrier thread when blocking inside `synchronized`. Beyond that, ZGC became generational by default, compact object headers became production-ready, the Security Manager was permanently disabled, and ahead-of-time class loading cut startup time. Most of these arrive by upgrading the JVM alone, without recompiling.

What is a preview feature and how should a team treat it?

A fully specified but provisional feature, disabled unless `--enable-preview` is passed at both compile and run time, with class files tied to the exact JDK version. Treat it as unusable in production: String Templates previewed in Java 21 and 22 and were then withdrawn entirely.

What problem do stream gatherers solve?

They make intermediate stream operations extensible. `Collector` always allowed custom terminal operations, but intermediate stages were a fixed set, so operations like sliding windows or fixed batching required dropping out of the stream. `Gatherers.windowFixed`, `windowSliding`, `fold`, `scan`, and `mapConcurrent` cover the common cases, and `Gatherer.of` handles the rest.

When would you use `ScopedValue` instead of `ThreadLocal`?

For request-scoped context, especially with virtual threads. `ScopedValue` is immutable, bounded to the dynamic extent of a `run` call, and shared rather than copied into child threads in a structured scope. `ThreadLocal` is mutable, unbounded in lifetime, a common leak source, and its inheritable form copies per thread, which does not scale to millions of virtual threads.

Why is the Foreign Function and Memory API better than JNI?

There is no C shim to write, compile, and ship per platform. Memory lifetime is explicit through `Arena`, so freeing is deterministic and use-after-free throws `IllegalStateException` instead of corrupting memory. Confined arenas add thread-confinement checks, converting a class of undefined behavior into exceptions.

Should a team upgrade from Java 21 to Java 25?

Usually yes, but as two decisions. Upgrade the JVM first for the pinning fix, GC improvements, and startup work with no code change; then decide separately about `--release 25`. The blocker is almost always agents and bytecode-manipulation libraries, so inventory those first.

TRY IT | 58.12 Exercises

- 1 Write a program using `synchronized` around a blocking call inside a virtual thread. Run it on Java 21 and Java 25 with `-Djdk.tracePinnedThreads` and compare.
- 2 Implement a sliding-window moving average with `Gatherers.windowSliding`, then reimplement it without gatherers and compare readability.
- 3 Build a custom `Gatherer` that emits an element only when it differs from the previous one, and explain why it cannot split for parallel streams.
- 4 Call a native function through the FFM API using a confined `Arena`. Then let a `MemorySegment` escape the scope and observe the exception.
- 5 Replace a `ThreadLocal`-based request context with `ScopedValue` and describe what changes for child threads.
- 6 Measure heap usage of a workload with many small objects before and after `-XX:+UseCompactObjectHeaders`.
- 7 Compile a class using a preview feature, then run it on a different JDK 25 build and explain the failure.
- 8 Rewrite a constructor that validates arguments through a static helper inside `super(...)` using flexible constructor bodies.

RECAP | 58.13 Chapter summary

Java ships every six months and designates an LTS roughly every two years; Java 25 is the successor to the Java 21 baseline this book targets. Between them, Java 22 finalized the Foreign Function and Memory API and unnamed variables, Java 23 added Markdown Javadoc and made ZGC generational by default, Java 24 delivered the operationally decisive change - virtual threads no longer pin their carrier under `synchronized` - plus stream gatherers, the Class-File API, and AOT class loading, and Java 25 finalized scoped values, flexible constructor bodies, module imports, and compact source files while promoting compact object headers to production. Most of this value arrives from upgrading the JVM rather than the language level, which argues for separating those two decisions. Finalized features are safe; preview features are not, and String Templates' withdrawal after two previews is the evidence. The practical upgrade blocker is rarely the language - it is agents, profilers, and bytecode libraries.

TRY IT | 58.14 Revision checklist

- ☐ I can name the LTS releases and explain that LTS is a vendor commitment.
- ☐ I can explain the difference between incubator, preview, and final, and cite a withdrawn preview.
- ☐ I know that `--enable-preview` is required at compile and run time and pins the class file version.
- ☐ I can explain the Java 24 virtual-thread pinning fix and what still pins.
- ☐ I can use `windowFixed`, `windowSliding`, and `mapConcurrent`.
- ☐ I can justify `ScopedValue` over `ThreadLocal` for request context.
- ☐ I can describe FFM arenas and why they beat JNI.
- ☐ I know compact object headers, generational ZGC, and AOT class loading are JVM-level wins.
- ☐ I can argue for separating a JVM upgrade from a language-level upgrade.
- ☐ I know agents and bytecode libraries are the usual upgrade blocker.

JAVA FOUNDATIONS TO ADVANCED ENGINEERING

Appendices

A focused part of the complete SDE-2 preparation guide

REFERENCE APPENDIX

Appendix A - Java Syntax and Language Quick Reference

This appendix is a recall sheet, not a substitute for the language chapters. Every compact rule hides edge cases. When correctness depends on an exact compile-time rule, use the Java Language Specification linked in Appendix G.

After Java Foundations, continue to [Java-SDE2-DSA-01-Time-and-Space-Complexity.pdf](#), then use [Java-SDE2-DSA-02-Number-Systems-and-Math-Foundations.pdf](#) for a printable numeric quick-reference with worked algorithms. For the full focused sequence, open the [series index](#).

A.1 Source files, packages, and launch

A conventional source file begins with an optional package declaration, followed by imports and type declarations:

JAVA

```
package com.example.orders;

import java.time.Instant;
import java.util.List;

public final class OrderBatch {
    private final List<String> ids;
    private final Instant createdAt;

    public OrderBatch(List<String> ids, Instant createdAt) {
        this.ids = List.copyOf(ids);
        this.createdAt = createdAt;
    }

    public static void main(String[] args) {
        var batch = new OrderBatch(List.of("demo"), Instant.now());
        System.out.println("orders=" + batch.ids.size());
    }
}
```

At most one top-level public type appears in a source file, and its name normally matches the file name. A package name maps to a directory layout by convention and by most build tools. The unnamed package is unsuitable for maintained applications because named packages cannot import from it.

Common commands:

TEXT

```
javac --release 21 -d out src/com/example/orders/OrderBatch.java
java -cp out com.example.orders.OrderBatch
java --enable-preview --source 21 PreviewExample.java
javap -classpath out -c -v com.example.orders.OrderBatch
```

`--release 21` selects a supported language level and matching documented Java SE APIs. It is safer for cross-compilation than setting only `-source` and `-target`. Java 21 features covered in this book do not require preview mode when they were final in that release.

A.2 Primitive types, references, and default values

Java has eight primitive types:

Type	Width or model	Representative range or values	Default field value
<code>byte</code>	8-bit signed	-128 through 127	<code>0</code>
<code>short</code>	16-bit signed	-32,768 through 32,767	<code>0</code>
<code>int</code>	32-bit signed	-2^{31} through $2^{31} - 1$	<code>0</code>
<code>long</code>	64-bit signed	-2^{63} through $2^{63} - 1$	<code>0L</code>
<code>char</code>	16-bit unsigned UTF-16 code unit	<code>\u0000</code> through <code>\uffff</code>	<code>\u0000</code>
<code>float</code>	IEEE 754 binary32	finite values, infinities, NaN	<code>0.0f</code>
<code>double</code>	IEEE 754 binary64	finite values, infinities, NaN	<code>0.0d</code>
<code>boolean</code>	logical value	<code>true</code> , <code>false</code>	<code>false</code>

Reference variables hold reference values or `null`; they do not contain an object inline as a language guarantee. Fields and array elements receive defaults. Local variables must be definitely assigned before use.

Integer arithmetic on `byte`, `short`, and `char` is normally promoted to `int`. Integer overflow wraps using two's-complement arithmetic; it does not throw. Use `Math.addExact`, `subtractExact`, and related methods when overflow must be rejected. Floating-point NaN is unequal to every value, including itself; use `Double.isNaN` rather than equality.

Useful literals:

JAVA

```
int decimal = 1_000_000;
int hexadecimal = 0xFF;
int binary = 0b1010_0110;
long longValue = 9_000_000_000L;
float ratio = 0.25F;
char newline = '\n';
String textBlock = """
    line one
    line two
    """;
```

A.3 Conversions, boxing, and numeric comparison

Widening primitive conversions such as `int` to `long` are implicit, although a large integer can lose precision when converted to floating point. Narrowing conversions require a cast and can discard high bits or truncate a fractional component.

Boxing converts a primitive to a wrapper, and unboxing extracts the primitive. Unboxing `null` throws `NullPointerException`. Wrapper identity is not a numeric comparison:

JAVA

```
Integer a = 1_000;
Integer b = 1_000;
boolean wrongQuestion = a == b;           // compares references
boolean valueEquality = a.equals(b);       // compares int values
```

Do not rely on wrapper caches. Prefer primitives in hot numeric loops unless nullability, generic APIs, or object semantics are required.

For money, binary floating point is usually the wrong domain model. Use an integer minor-unit representation when scale is fixed, or use `BigDecimal` with an explicit scale and rounding policy. `new BigDecimal("0.1")` is predictable; `new BigDecimal(0.1)` captures the binary floating-point approximation.

A.4 Operators and precedence

From high to low, a practical precedence summary is:

Family	Operators
postfix	<code>expr++, expr--</code>
unary	<code>++, --, +, -, ~, !, cast</code>
multiplicative	<code>*, /, %</code>
additive	<code>+, -</code>
shift	<code><<, >>, >>></code>
relational	<code><, <=, >, >=, instanceof</code>
equality	<code>==, !=</code>
bitwise	<code>&, then ^, then ^</code>
conditional logical	<code>&&, then ^</code>
conditional	<code>condition ? yes : no</code>
assignment	<code>=, compound assignments</code>

Use parentheses when a reviewer would otherwise have to recall the table. `&&` and `||` short-circuit; `&` and `|` evaluate both operands even for booleans. Compound assignment includes an implicit conversion: `byteValue += 1` compiles where `byteValue = byteValue + 1` needs a cast.

`==` compares primitive values or reference identity. Semantic object equality is normally expressed by `equals`. Strings must not be compared with `==`.

A.5 Control flow and pattern matching

Basic forms:

JAVA

```
if (ready) {
    start();
} else {
    defer();
}

for (int i = 0; i < values.length; i++) {
    consume(values[i]);
}

for (String value : values) {
    consume(value);
}

while (condition()) {
    step();
}

do {
    step();
} while (condition());
```

A switch expression returns a value and should be exhaustive:

JAVA

```
enum State { NEW, RUNNING, DONE }

String label = switch (state) {
    case NEW -> "queued";
    case RUNNING -> "active";
    case DONE -> "complete";
};
```

The colon form retains fall-through semantics. Prefer arrow labels unless fall-through is deliberate and obvious.

Pattern variables are flow-scoped:

JAVA

```
if (candidate instanceof String text && !text.isBlank()) {
    System.out.println(text.length());
}
```

Record patterns and pattern matching for switch are final in Java 21. A sealed hierarchy lets the compiler check exhaustiveness when all permitted direct subtypes are known in the relevant compilation context.

A.6 Methods, overloads, and parameter passing

A method signature for overloading consists of the method name and parameter types. Return type alone cannot distinguish overloads. Resolution occurs at compile time using the declared argument types, applicability, conversions, and most-specific rules. Overriding is dynamic dispatch based on the runtime receiver type.

JAVA

```
static long sum(int left, long right) {
    return left + right;
}

static int sum(int... values) {
    int total = 0;
    for (int value : values) total += value;
    return total;
}
```

Varargs is an array parameter with call-site convenience. It can allocate an array and can create overload ambiguity. A caller may pass `null` as the entire array, so a public varargs method still needs a null policy.

Java is always pass-by-value. For an object argument, the copied value is a reference. The callee can use that copied reference to mutate the same mutable object, but assigning a different object to the parameter does not change the caller's variable.

A.7 Arrays, strings, and text

Arrays are reified, covariant objects with a fixed length:

JAVA

```
String[] names = new String[3];
int[][] matrix = new int[4][5];
int[] copy = java.util.Arrays.copyOf(values, values.length);
```

Covariance can fail at runtime: assigning a `String[]` to `Object[]` is legal, but storing a non-string throws `ArrayStoreException`. Generic collections are invariant and catch the analogous mismatch at compile time.

`String` is immutable. Concatenation in one expression is compiled using platform mechanisms selected by the compiler; repeated concatenation in a loop should normally use `StringBuilder`. `String.length()` counts UTF-16 code units, not user-perceived characters. A supplementary Unicode code point uses a surrogate pair. Use `codePoints()` when code points are the needed abstraction and a text library when grapheme clusters matter.

Common equality and emptiness checks:

JAVA

```
boolean same = java.util.Objects.equals(left, right);
boolean empty = text.isEmpty();
boolean blank = text.isBlank();
```

A.8 Classes, records, interfaces, and sealed hierarchies

Class skeleton:

JAVA

```
public final class Account {
    private final String id;
    private long balance;

    public Account(String id) {
        this.id = java.util.Objects.requireNonNull(id);
    }

    public synchronized void credit(long cents) {
        balance = Math.addExact(balance, cents);
    }
}
```

Access levels:

Modifier	Same class	Same package	Subclass in another package	Everywhere
private	yes	no	no	no
package-private	yes	yes	no	no
protected	yes	yes	qualified rules apply	no
public	yes	yes	yes	yes

Constructors are not inherited. A subclass constructor begins with an explicit or implicit invocation of another constructor. Static methods are hidden, not overridden. Private methods are not overridden. An overriding method may use a covariant return type and cannot broaden checked exceptions.

A record declares a nominal data carrier:

JAVA

```
public record Range(int start, int end) {
    public Range {
        if (start > end) throw new IllegalArgumentException("reversed");
    }
}
```

Records are shallowly immutable: component fields are final, but a referenced component can still be mutable. Make defensive copies when value semantics require them.

An interface defines a contract and can contain abstract, default, static, and private methods. An abstract class can hold instance state and protected implementation machinery. Prefer composition when the relationship is behavioral reuse rather than a stable subtype relationship.

Sealed types restrict direct subtypes:

JAVA

```
sealed interface Result permits Success, Failure {}
record Success(String value) implements Result {}
record Failure(String message) implements Result {}
```

Each permitted subtype must be `final`, `sealed`, or `non-sealed` unless its form, such as a record, is implicitly `final`.

A.9 Equality, hashing, and ordering

The `equals` contract requires reflexivity, symmetry, transitivity, consistency, and a false result for null. Equal objects must have equal hash codes. Unequal objects may collide. Fields used by `equals` and `hashCode` should not change while the object is a key in a hash-based collection.

`Comparable<T>` defines a natural ordering. `Comparator<T>` defines an external ordering and supports composition:

JAVA

```
Comparator<Person> order = Comparator
    .comparing(Person::lastName)
    .thenComparing(Person::firstName)
    .thenComparingInt(Person::age);
```

An ordering inconsistent with equality is legal for some APIs but can make a sorted set or map treat unequal values as the same key. Document the choice.

A.10 Exceptions and resource management

Checked exceptions are subclasses of `Exception` excluding `RuntimeException`; the compiler enforces catch-or-declare. Unchecked exceptions include `RuntimeException` and `Error`. Do not catch `Throwable` as routine application control flow.

JAVA

```
try (var input = java.nio.file.Files.newBufferedReader(path)) {
    return input.readLine();
} catch (java.io.IOException failure) {
    throw new UncheckedIOException("cannot read " + path, failure);
}
```

Try-with-resources closes initialized resources in reverse order. If both the body and close fail, the body failure is primary and close failures are suppressed. Inspect `getSuppressed()` when diagnosing layered resource failures.

Catch the narrowest useful type, preserve the cause when translating, and attach context without secrets. An interruption caught as `InterruptedException` should normally be propagated or restored with `Thread.currentThread().interrupt()`.

A.11 Generics and variance

Generic types are invariant: `List<Integer>` is not a subtype of `List<Number>`. Use bounded wildcards at API boundaries:

JAVA

```
static double total(java.util.List<? extends Number> source) {  
    return source.stream().mapToDouble(Number::doubleValue).sum();  
}  
static void addDefaults(java.util.List<? super Integer> sink) { /* write */ }
```

Mnemonic: producer extends, consumer super. A wildcard is not a declaration-site type parameter and may need capture by a helper method.

Type erasure means most type arguments are not reified at runtime. Consequently, Java prohibits `new T()`, `new T[]`, `instanceof List<String>`, and overloading methods whose signatures erase to the same form. Heap pollution occurs when a variable of a parameterized type refers to an incompatible value, often through raw types, unchecked casts, or generic varargs.

Prefer a generic method when the type relationship is local to one operation. Prefer a generic class when the type relationship belongs to the object's persistent state or contract.

A.12 Lambdas, method references, and streams

A lambda implements a functional interface, an interface with one abstract method after inheritance rules:

JAVA

```
java.util.function.Predicate<String> useful =  
    value -> value != null && !value.isBlank();  
java.util.function.Function<String, Integer> length = String::length;
```

Captured local variables must be final or effectively final. Capturing mutable state does not make access thread-safe.

Streams describe a lazy traversal pipeline. Intermediate operations are lazy; a terminal operation drives evaluation:

JAVA

```
Map<String, Long> counts = words.stream()  
    .filter(useful)  
    .map(String::toLowerCase)  
    .collect(java.util.stream.Collectors.groupingBy(  
        java.util.function.Function.identity(),  
        java.util.stream.Collectors.counting()));
```

Do not reuse a consumed stream. Avoid side effects in behavioral parameters. Parallel streams use shared runtime machinery and are not automatically faster; measure a representative, sufficiently large, CPU-oriented workload and account for blocking, ordering, splitting, and contention.

A.13 Concurrency recall sheet

Key rules:

- Starting a thread happens-before actions in that thread.
- All actions in a thread happen-before another thread detects its termination, including an untimed `join()` that returns.
- An unlock on a monitor happens-before a later lock on the same monitor.
- A volatile write happens-before a later read of that same field in synchronization order.
- Final fields receive special visibility guarantees when construction is correct and `this` does not escape.
- `volatile` supplies visibility and ordering for the field; it does not turn `count++` into one atomic action.
- Higher-level concurrent collection and executor methods have their own documented memory-consistency effects.

Prefer ownership, immutability, message passing, and well-defined concurrent components over scattered low-level synchronization. Every blocking operation needs a cancellation and shutdown story.

Java 21 virtual threads make thread-per-task practical for many blocking workloads. They do not remove resource limits, make CPU work parallel beyond available processors, or repair data races. Bound downstream resources such as connections and request quotas rather than pooling virtual threads merely to limit their count.

A.14 Common interview traps

- Java has no pass-by-reference parameter mode.
- `String` immutability does not make every object containing a string immutable.
- `final` freezes a variable binding or prevents overriding/inheritance in context; it is not a universal deep-immutability or thread-safety keyword.
- `volatile` is not mutual exclusion.
- `HashMap` has no general iteration-order contract.
- `PriorityQueue` exposes only its head ordering; iteration is not sorted.
- `Arrays.asList` returns a fixed-size list backed by the array.
- `List.subList` is a backed view whose structural validity depends on disciplined modification.
- `Stream.toList()` returns an unmodifiable list by contract; do not assume a concrete implementation.
- `Optional` is primarily a return-type tool, not a universal field, parameter, or serialization model.
- `Thread.sleep` does not release a held monitor.
- `wait` must be called while owning the monitor, releases that monitor while waiting, and belongs in a condition loop.
- A successful test run cannot prove a racy program correct.
- Big-O does not describe allocation, locality, constant costs, warm-up, contention, or tail latency.

Use this appendix for rapid recall, then return to the relevant chapter to recover the mechanism, proof obligation, trade-off, and production implication.

REFERENCE APPENDIX

Appendix B - Collection Complexity and Selection Matrix

Complexities below describe common OpenJDK implementations for ordinary workloads, not interface-level guarantees unless stated. Hash-table operations are expected-time claims under an adequate hash distribution. Adversarial keys, resizing, comparator cost, allocation, cache locality, and concurrency can dominate a real service.

B.1 List implementations

Operation	<code>ArrayList</code>	<code>LinkedList</code>	Engineering note
indexed <code>get</code> / <code>set</code>	$O(1)$	$O(n)$	Array locality usually favors <code>ArrayList</code>
append	amortized $O(1)$	$O(1)$	Array resize occasionally copies elements
prepend	$O(n)$	$O(1)$	<code>ArrayDeque</code> is usually the better queue
insert/remove by index	$O(n)$ shift	$O(n)$ traversal, $O(1)$ relink	Traversal often dominates linked-list edits
search by value	$O(n)$	$O(n)$	Equality cost is part of the constant
iteration	$O(n)$	$O(n)$	Pointer chasing can hurt locality
memory per element	reference plus spare capacity	node object plus links	Measure retained size for large collections

Default choice: `ArrayList`. Choose `LinkedList` only when its list-plus-deque contract and a measured mutation pattern justify node overhead. Even frequent insertion is not sufficient if finding the insertion point is linear.

`CopyOnWriteArrayList` makes traversal stable and unsynchronized for readers by copying the backing array on every structural mutation. It fits small, read-mostly listener/configuration sets; it is disastrous for frequent writes or large lists.

B.2 Set and map implementations

Type	Lookup / update	Ordering	Null policy	Representative use
<code>HashMap</code>	expected $O(1)$	unspecified	one null key, null values	general mutable map
<code>LinkedHashMap</code>	expected $O(1)$	insertion or access order	like <code>HashMap</code>	predictable traversal, bounded LRU building block
<code>TreeMap</code>	$O(\log n)$	comparator/natural sorted	null key generally rejected by natural ordering	range queries, nearest-key navigation
<code>EnumMap</code>	$O(1)$ array-like	enum declaration order	null key rejected	dense enum-keyed state
<code>IdentityHashMap</code>	expected $O(1)$	unspecified	permits null	graph/topology algorithms using identity semantics
<code>ConcurrentHashMap</code>	expected $O(1)$	unspecified, weakly consistent traversal	null keys/values rejected	shared concurrent lookup/update
<code>immutableMap.of()</code> / <code>Map.copyOf()</code>	implementation-dependent efficient lookup	unspecified	null rejected	fixed snapshots and safe sharing

Corresponding sets are commonly backed by maps or tree structures:

Type	Membership	Ordering	Notes
<code>HashSet</code>	expected $O(1)$	unspecified	mutable elements can become unfindable
<code>LinkedHashSet</code>	expected $O(1)$	insertion order	extra link metadata
<code>TreeSet</code>	$O(\log n)$	sorted	comparator equality controls uniqueness
<code>EnumSet</code>	$O(1)$ bit-vector-like	enum order	exceptionally compact for enum domains
<code>CopyOnWriteArraySet</code>	$O(n)$ lookup, $O(n)$ copy on write	insertion-like snapshot order	very small read-mostly sets
<code>ConcurrentHashMap.newKeySet()</code>	expected $O(1)$	weakly consistent	scalable concurrent membership set

Important `HashMap` facts:

- Capacity and load factor affect resizing, memory, and traversal cost.
- Current OpenJDK `HashMap` implementations may transform a collision-heavy bin into a tree when thresholds and table capacity permit. That is an implementation strategy, not permission to use poor keys.

- `computeIfAbsent` is not a general exactly-once side-effect facility. Read each map's contract, keep mapping functions short, and avoid recursive structural updates.
- Mutating an equality/hash field after insertion violates the lookup assumption even though the entry still occupies a bucket.
- Iteration order can look stable in a small experiment and still have no contract.

B.3 Queues, dequeues, and heaps

Type	Add/remove	Head access	Capacity / blocking	Use
<code>ArrayDeque</code>	amortized $O(1)$ at ends	$O(1)$	unbounded, nonblocking	default stack, queue, deque
<code>PriorityQueue</code>	$O(\log n)$ add/remove	$O(1)$ peek	unbounded, nonblocking	next minimum/maximum by comparator
<code>ConcurrentLinkedQueue</code>	expected $O(1)$	typically $O(1)$; may traverse removed nodes	unbounded, nonblocking	scalable multi-producer/consumer queue
<code>ArrayBlockingQueue</code>	$O(1)$	$O(1)$	bounded, blocking	fixed-capacity back pressure
<code>LinkedBlockingQueue</code>	$O(1)$	$O(1)$	optionally bounded, blocking	producer/consumer, usually explicit bound
<code>PriorityBlockingQueue</code>	$O(\log n)$	$O(1)$	unbounded, blocking take	concurrent priority scheduling without capacity control
<code>SynchronousQueue</code>	handoff	none stored	zero capacity, blocking	direct producer-consumer rendezvous
<code>DelayQueue</code>	$O(\log n)$	eligible head	unbounded	delayed/expiry work

Use `offer`, `poll`, and `peek` when absence/capacity is expected and should be represented by a result. Use `add`, `remove`, and `element` when failure should be exceptional. Blocking queues add `put` and `take`; timed variants let cancellation and service-level policy participate.

`PriorityQueue` guarantees that `peek/remove` exposes a least element according to its ordering. Its iterator is not sorted. To emit sorted order, repeatedly remove elements from a copy, or sort a collection explicitly.

B.4 Sorted and navigable operations

`NavigableMap` and `NavigableSet` support relative queries in $O(\log n)$ for tree implementations:

- **lower**: greatest element strictly less than the key.
- **floor**: greatest element less than or equal to the key.
- **ceiling**: least element greater than or equal to the key.
- **higher**: least element strictly greater than the key.
- **subMap**, **headMap**, and **tailMap**: usually backed range views.

Backed views enforce their range. An insertion outside the view range fails. Changes through either the view or the source are visible through the other, subject to the collection's synchronization and iterator contracts.

B.5 Views, wrappers, and copies

These APIs have deliberately different ownership semantics:

API	Structural mutability	Backing relationship	Null handling
<code>Arrays.asList(array)</code>	fixed size; <code>set</code> allowed	backed by array	permits null
<code>list.subList(a, b)</code>	generally mutable within range	backed by list	source policy
<code>Collections.unmodifiableList(list)</code>	wrapper rejects mutation	reflects backing-list changes	source policy
<code>List.copyOf(source)</code>	unmodifiable	snapshot-like shallow copy/reuse	rejects null
<code>List.of(values...)</code>	unmodifiable	independent fixed value	rejects null
<code>stream.toList()</code>	unmodifiable	result collection	may contain null elements produced by stream
<code>Collectors.toList()</code>	no mutability/type guarantee	new result	collector behavior

Unmodifiable is not deeply immutable. If elements are mutable, their state can change. A defensive copy protects collection structure only; copy elements or use immutable value types when deeper isolation is required.

B.6 Iterator consistency

Iterator labels describe detection and visibility, not thread safety of arbitrary compound actions:

- Fail-fast iterators on many ordinary collections may throw `ConcurrentModificationException` after an uncoordinated structural change. Detection is best-effort, not a synchronization mechanism.
- Snapshot iterators, such as those of copy-on-write collections, traverse a stable historical array and do not see later writes.
- Weakly consistent iterators on concurrent collections tolerate concurrent updates and may reflect some updates without throwing. They do not freeze a transactionally consistent snapshot.

Use external locking when multiple operations must preserve one invariant and the collection does not supply an atomic compound method. `if (!map.containsKey(k)) map.put(k, v)` is a race on a shared map; use `putIfAbsent`, `compute`, or explicit coordination as the required semantics dictate.

B.7 Stream and collector cost reminders

A stream pipeline is not a collection. It describes one traversal and is normally single-use. Complexity is the sum of traversal and operations:

Operation	Typical cost	Important qualification
<code>filter</code> , <code>map</code>	$O(n)$	function cost and boxing may dominate
<code>distinct</code>	expected $O(n)$	retains seen values; ordered mode can cost more
<code>sorted</code>	$O(n \log n)$	buffers elements; comparator cost matters
<code>limit</code>	passes at most k downstream	upstream filters or stateful stages can still perform $O(n)$ work; ordered parallel pipelines may coordinate heavily
<code>findFirst</code>	short-circuiting	encounter order constrains parallel execution
<code>grouping</code>	expected $O(n)$	retains keys and downstream accumulation state
<code>toMap</code>	expected $O(n)$	merge policy required when duplicate output keys are possible

Collector characteristics such as `CONCURRENT`, `UNORDERED`, and `IDENTITY_FINISH` are promises to the reduction framework. A mutable result container alone does not make a collector concurrent.

B.8 Selection questions

Choose a collection by answering in order:

- 1 What semantics are required: sequence, uniqueness, association, priority, range navigation, or handoff?
- 2 Is order absent, insertion-based, access-based, sorted, or merely head-priority?
- 3 Which operations dominate, and what are their input sizes?
- 4 Is mutation local, externally synchronized, or concurrent?
- 5 Must a compound invariant span several operations?
- 6 Are nulls valid domain values, and can the chosen type represent them?
- 7 Who owns the collection, and should the API expose a view, wrapper, or copy?
- 8 What bounds memory: maximum size, admission control, expiry, or back pressure?
- 9 Does iteration need a snapshot, weak consistency, or strict external coordination?
- 10 Have representative allocation, locality, contention, and tail latency been measured?

B.9 Rapid decision matrix

The strongest interview answer states the contract first, compares two plausible choices, names the dominant workload operation, and closes with ownership, concurrency, and measurement implications.

Requirement	Start with	Reconsider when
general indexed sequence	<code>ArrayList</code>	frequent measured front edits or immutable persistent structure needed
stack / FIFO / double-ended work	<code>ArrayDeque</code>	cross-thread blocking or bounded capacity required
general key lookup	<code>HashMap</code>	sorted/range operations, stable order, enum keys, or concurrency required
predictable insertion traversal	<code>LinkedHashMap</code>	sorted key semantics required
sorted map and nearest-key query	<code>TreeMap</code>	hash lookup is sufficient and ordering cost is wasted
enum membership	<code>EnumSet</code>	domain is not an enum
minimum/maximum scheduling	<code>PriorityQueue</code>	full sorted iteration or bounded blocking is required
shared concurrent map	<code>ConcurrentHashMap</code>	one invariant spans external state or several keys
bounded producer/consumer	<code>ArrayBlockingQueue</code>	transfer semantics, priority, or different throughput characteristics matter
fixed public result	<code>List.copyOf / Map.copyOf</code>	caller intentionally needs a live view
small read-mostly listener set	copy-on-write collection	writes or collection size grow

REFERENCE APPENDIX

Appendix C - JVM Tools, Flags, and Incident Commands

This appendix is a field checklist for OpenJDK HotSpot-style deployments. Tool availability, exact output, attach permissions, overhead, container behavior, and flags vary by release and distribution. Treat every command as version-specific and rehearse it in a safe environment. Prefer evidence collection with understood overhead over speculative tuning.

C.1 Identify the runtime before interpreting it

Capture the executable, release, vendor, process arguments, and container limits:

TEXT

```
java -version
java -XshowSettings:vm -version
jcmd <pid> VM.version
jcmd <pid> VM.command_line
jcmd <pid> VM.flags
jcmd <pid> VM.system_properties
jcmd <pid> VM.info
```

Also record:

- wall-clock time, timezone, host/pod identity, deployment version, and incident interval;
- CPU quota and throttling, memory limit, resident memory, swap policy, and OOM-kill evidence;
- request rate, concurrency, error rate, latency percentiles, dependency health, and recent changes;
- heap sizing, collector, thread count, file descriptors, connection-pool limits, and native-library versions.

Without runtime identity and workload context, two similar-looking tool outputs can imply different mechanisms.

C.2 Evidence-first incident sequence

A conservative sequence is:

- 1 Establish user-visible symptoms and exact time bounds.
- 2 Correlate application and infrastructure metrics before attaching a tool.
- 3 Record runtime identity, arguments, limits, and recent deployments.
- 4 Collect several lightweight thread and heap summaries rather than one isolated sample.
- 5 Start a bounded JFR recording if its overhead is acceptable and existing continuous recordings are unavailable.
- 6 Escalate to heap dumps or more invasive diagnostics only with disk, pause, privacy, and operational risks understood.
- 7 Preserve raw artifacts and command lines; analyze copies.
- 8 Change one causal control at a time, define success/rollback criteria, and keep monitoring guardrails.

Restarting may restore service but destroys in-process evidence. When safety permits, collect the smallest decisive artifact first.

C.3 jcmd command map

List local JVMs and supported diagnostic commands:

TEXT

```
jcmd -l  
jcmd <pid> help  
jcmd <pid> help <command>
```

Common commands:

Goal	Representative command	Interpretation note
thread dump	<code>jcmd <pid> Thread.print -l</code>	take several samples; one blocked stack is not a trend
class histogram	<code>jcmd <pid> GC.class_histogram</code>	count/bytes are shallow; command may affect the process
heap overview	<code>jcmd <pid> GC.heap_info</code>	collector-specific summary, not a leak proof
heap dump	<code>jcmd <pid> GC.heap_dump filename=/safe/path/heap.hprof</code>	large and potentially sensitive; check command help/version
class-loader stats	<code>jcmd <pid> VM.classloader_stats</code>	useful for loader/metaspace growth
native memory	<code>jcmd <pid> VM.native_memory summary</code>	requires NMT enabled at startup
compiler state	<code>jcmd <pid> Compiler.codecache</code>	exact command set varies
system counters	<code>jcmd <pid> PerfCounter.print</code>	implementation counters need release context
start JFR	<code>jcmd <pid> JFR.start name=incident settings=profile duration=5m filename=/safe/path/incident.jfr</code>	quote/adjust syntax for shell and release
inspect recordings	<code>jcmd <pid> JFR.check</code>	lists active recordings
dump JFR	<code>jcmd <pid> JFR.dump name=incident filename=/safe/path/snapshot.jfr</code>	preserves current data
stop JFR	<code>jcmd <pid> JFR.stop name=incident</code>	optionally specify output per command help

Some diagnostic commands are marked with impact levels in `jcmd help`. Read them. A production-safe action depends on heap size, allocation rate, storage, pause objectives, and incident severity.

C.4 Thread diagnosis

Collect repeated dumps several seconds apart:

TEXT

```
jcmd <pid> Thread.print -l > threads-01.txt
# wait while preserving the incident interval
jcmd <pid> Thread.print -l > threads-02.txt
jcmd <pid> Thread.print -l > threads-03.txt
```

`jstack -l <pid>` is a familiar alternative when available; `jcmd` is generally the preferred HotSpot diagnostic entry point.

Look for:

- the same runnable stacks consuming samples, correlated with process/thread CPU;
- many threads waiting for one monitor, lock, connection pool, queue, or future;
- deadlock reports and an actual resource cycle;
- executor queues growing while workers block downstream;
- repeated socket/database waits aligned with dependency latency;
- virtual-thread pinning or carrier scarcity only after confirming release-specific evidence;
- shutdown threads waiting on tasks that ignore interruption.

Thread state names are snapshots. **RUNNABLE** can include native I/O or code not currently executing on a CPU. **WAITING** is not automatically a problem. Ask whether the state persists and whether it explains the service symptom.

C.5 Java Flight Recorder

JFR records time-correlated JVM and application events with configurable detail. Prefer a continuous, bounded recording established before the incident when organizational policy allows it.

Command-line startup examples:

TEXT

```
java -XX:StartFlightRecording=filename=service.jfr,dumpOnExit=true,maxSize=512m,maxAge=2h ...
java -XX:FlightRecorderOptions=stackdepth=128 ...
```

Attach example:

TEXT

```
jcmd <pid> JFR.start name=incident settings=profile delay=10s duration=5m filename=/safe/path/incident.jfr
```

Inspect in JDK Mission Control or with the `jfr` command where available:

TEXT

```
jfr summary incident.jfr
jfr print --events jdk.CPULoad,jdk.GarbageCollection incident.jfr
```

Correlate events instead of ranking isolated hot methods. Useful relationships include allocation pressure -> GC work -> pause or concurrent CPU -> request latency; monitor contention -> blocked duration -> endpoint; socket reads -> remote host -> pool occupancy; and compilation/deoptimization -> phase change -> latency.

Recording settings and event names evolve. A profile recording can be more expensive than the default configuration. Measure overhead for the selected settings and workload.

C.6 GC and safepoint logging

Unified logging is the modern mechanism:

TEXT

```
-Xlog:gc*:file=/safe/path/gc.log:time,uptime,level,tags:filecount=10,filesize=50m
-Xlog:safepoint:file=/safe/path/safepoint.log:time,uptime,level,tags
-Xlog:gc+heap=debug,gc+age=trace:file=/safe/path/gc-detail.log:time,uptime,tags
```

Validate selectors for the exact JDK:

TEXT

```
java -Xlog:help
```

Questions to answer from GC evidence:

- Is allocation rate rising, or is the retained live set rising?
- Does old occupancy return to a stable baseline after a complete relevant cycle?
- Are pauses caused by evacuation work, reference processing, class unloading, humongous allocations, or another phase?
- Is concurrent collector work receiving enough CPU under the container quota?
- Is the heap undersized for the live set and allocation burst, or oversized in a way that harms objectives?
- Do latency spikes align with GC at all?

Never conclude "memory leak" merely because used heap rises between collections. A leak is unintended retained reachability. Use a time series and a retention analysis.

C.7 Heap histograms and dumps

A class histogram is a triage view:

TEXT

```
jcmd <pid> GC.class_histogram
```

Compare multiple histograms under similar collection conditions. Shallow bytes do not reveal who retains objects. A large legitimate cache can rank first without being a leak, while a small retaining root can keep a huge graph alive.

Heap dumps:

TEXT

```
jcmd <pid> GC.heap_dump filename=/safe/path/heap.hprof
```

Before dumping, verify:

- enough disk for approximately heap-scale output plus safety margin;
- write path and container persistence;
- pause and I/O impact for this release/heap;
- secrets, personal data, credentials, payloads, and retention policy;
- secure transfer and access controls;
- whether an existing dump-on-OOME policy is more appropriate.

Analyze dominator trees, retained size, paths to GC roots, duplicate values, and class-loader ownership. Compare a suspect population to expected workload state and lifecycle.

`jmap` offers historical heap commands, but prefer supported `jcmd` commands for the deployed JDK. Avoid force/SA attachment unless standard attachment is impossible, the risks are understood, and the incident justifies it.

C.8 Native memory and process RSS

The Java heap is only part of process memory. Other consumers include metaspace, code cache, thread stacks, direct/mapped buffers, GC/compiler structures, JNI/native libraries, allocator fragmentation, and observability agents.

Native Memory Tracking must be enabled at startup:

TEXT

```
-XX:NativeMemoryTracking=summary
```

Then:

TEXT

```
jcmd <pid> VM.native_memory baseline
jcmd <pid> VM.native_memory summary.diff scale=MB
jcmd <pid> VM.native_memory detail scale=MB
```

NMT has overhead, does not account for every native allocation, and its category names are implementation details. Correlate it with OS/container RSS, mapped memory, thread count, and native-library metrics.

A container OOM kill with no Java `OutOfMemoryError` often means the total process crossed the cgroup limit before the JVM could report a heap failure. Compare committed heap, live heap, native categories, stacks, direct buffers, sidecars, and limit/headroom.

C.9 Class loading and metaspace

Useful evidence:

TEXT

```
jcmd <pid> VM.classloader_stats  
jcmd <pid> VM.classloaders  
jcmd <pid> GC.class_histogram  
-Xlog:class+load=info,class+unload=info
```

Metaspace growth can follow legitimate dynamic class generation, redeployment leaks, proxy churn, scripting engines, or class loaders retained by threads, caches, logging frameworks, JDBC drivers, or static registries. A class can unload only when its defining loader and associated classes are unreachable and the JVM performs relevant collection work.

Track loaded/unloaded class counts and group suspect classes by defining loader. Fix the retaining lifecycle rather than simply raising a metaspace limit.

C.10 Compiler and code-cache evidence

Warm-up, compilation, deoptimization, and code-cache pressure can create phase-dependent latency.

Representative diagnostics include JFR compilation events, unified compiler logging selected for the release, and commands advertised by:

TEXT

```
jcmd <pid> help | grep -i compiler
```

Diagnostic flags such as detailed compilation output can be noisy and version-specific. Do not enable them globally in production without a test. A method that appears hot in interpreted execution may later be compiled; a microbenchmark that omits warm-up can measure compilation rather than steady-state work.

C.11 Heap and collector flags

Common controls:

Goal	Representative option	Caution
initial heap	<code>-Xms<size></code>	committing behavior and container headroom matter
maximum heap	<code>-Xmx<size></code>	leave room for native memory and sidecars
choose G1	<code>-XX:+UseG1GC</code>	default on many modern HotSpot configurations, but verify
choose ZGC	<code>-XX:+UseZGC</code>	capabilities and generational modes vary by release
pause target	<code>-XX:MaxGCPauseMillis=<ms></code>	a heuristic goal, not an SLA
dump on heap OOME	<code>-XX:+HeapDumpOnOutOfMemoryError</code>	secure and provision dump path
dump location	<code>-XX:HeapDumpPath=/safe/path</code>	ensure persistence and free space
exit on OOME	<code>-XX:+ExitOnOutOfMemoryError</code>	coordinate with orchestrator and evidence policy
error files	<code>-XX:ErrorFile=/safe/path/hs_err_pid%p.log</code>	ensure writable persistent location

Avoid cargo-cult flag bundles. Defaults, flag names, diagnostic status, and collector behavior change. Begin with an objective and evidence, change the smallest relevant control, and validate under a workload with production-like allocation and latency sensitivity.

C.12 Container checklist

- Confirm the JDK recognizes the actual CPU and memory limits.
- Compare `-Xmx` and total native headroom to the cgroup limit, not host RAM.
- Inspect throttled CPU time; concurrent GC and JIT work still need CPU.
- Bound direct buffers, threads, queues, caches, and pools according to the whole process budget.
- Persist crash logs, heap dumps, and JFR outside ephemeral storage when policy permits.
- Align orchestrator termination grace with Java shutdown and diagnostic time.
- Distinguish Java OOME, kernel/cgroup OOM kill, liveness restart, and operator eviction.

C.13 Symptom-to-evidence map

Symptom	First evidence	Common false lead
CPU saturation	process/thread CPU, repeated stacks, JFR execution samples	assuming any <code>RUNNABLE</code> thread is consuming CPU
long tail latency	request spans/metrics, JFR, GC/safepoint correlation, pool occupancy	blaming GC without time alignment
growing heap after GC	live-set series, histograms, heap dominators and roots	reading sawtooth growth as a leak
rising RSS with flat heap	NMT, thread/direct-buffer counts, mappings, native tools	increasing <code>-Xmx</code>
stuck shutdown	repeated thread dumps, executor/task cancellation paths	calling <code>System.exit</code> before preserving evidence
deadlock	JVM deadlock report plus resource graph	labeling ordinary lock contention deadlock
task starvation	executor active/queue metrics, worker stacks, dependency waits	adding unbounded threads
allocation storm	JFR allocation samples, GC log allocation rate, endpoint correlation	tuning pause targets before fixing churn
metaspace growth	class/loader counts, loader stats, unload logs, roots	only raising metaspace maximum
container OOM kill	cgroup event, RSS/native budget, limit history	expecting a heap dump automatically

C.14 Incident handoff template

A good incident explanation separates observation, inference, and decision. Example: "old occupancy remained above 82% after three mixed cycles" is an observation; "a retained cache is likely" is a hypothesis; "capture a bounded heap dump on one canary" is a decision with operational and privacy costs.

Record:

TEXT

```
User impact and interval:
Deployment/runtime identity:
Host or container limits:
Workload and dependency state:
Recent changes:
Thread evidence (timestamps and repetitions):
GC/safepoint evidence:
JFR recording and settings:
Heap/native evidence:
Leading hypotheses:
Evidence that supports/refutes each hypothesis:
Mitigation, risk, and rollback:
Follow-up experiment and owner:
Artifact locations and data classification:
```

REFERENCE APPENDIX

Appendix D - Java 17 and Java 21 Feature Matrix

This matrix emphasizes features that influence SDE-2 interviews and backend engineering. It uses Java 21 as the executable baseline and distinguishes final features from preview or incubating work. A feature can be introduced across several releases before becoming final; the final release is the portable baseline used here.

D.1 LTS orientation

Release	Status in this book	Selected significance
Java 8	historical baseline	lambdas, streams, default interface methods, <code>java.time</code> , <code>CompletableFuture</code>
Java 11	migration baseline	standard HTTP client, single-file launch, module-era runtime, removed bundled technologies
Java 17	supported modern baseline	records, sealed classes, pattern matching for <code>instanceof</code> , strong encapsulation context
Java 21	main code baseline	virtual threads, record patterns, pattern switch, sequenced collections, generational ZGC option
Java 25	later ecosystem context	LTS after the book's Java 21 code baseline; do not assume its APIs in examples

Long-term-support labels and commercial support schedules are vendor policy, not Java language semantics. Verify the distribution and support contract used by an employer.

D.2 Language feature timeline

Feature	Final release	Java 17	Java 21	Interview implication
local variable type inference (<code>var</code>)	10	yes	yes	local static type remains fixed; not dynamic typing
switch expressions	14	yes	yes	value-producing, exhaustive form with arrow labels
text blocks	15	yes	yes	multi-line literals with incidental indentation rules
records	16	yes	yes	nominal data carriers with generated members and shallow immutability
pattern matching for <code>instanceof</code>	16	yes	yes	flow-scoped pattern variable
sealed classes/interfaces	17	yes	yes	closed direct-subtype set; supports exhaustive reasoning
record patterns	21	no	yes	nested deconstruction patterns
pattern matching for switch	21	preview	final	type/record patterns, null handling, dominance and exhaustiveness rules

var

JAVA

```
var names = new java.util.ArrayList<String>();
```

`var` is permitted for qualifying local variables, loop variables, and lambda parameters in specified forms. It cannot declare fields, method parameters, or return types. The initializer determines a compile-time type; `names` is not dynamically typed. Use it when the initializer makes the type clear, not to hide a meaningful abstraction.

Switch expressions

JAVA

```
int priority = switch (severity) {
    case LOW -> 1;
    case MEDIUM -> 2;
    case HIGH -> 3;
};
```

The compiler checks that a switch expression produces a value for every possible input covered by the type rules. In a block arm, use `yield` to produce the value. The older colon statement form remains and can fall through.

Text blocks

JAVA

```
String json = """
    {
        "enabled": true
    }
    """;
```

Text blocks reduce escaping but still produce an ordinary `String`. Their indentation and trailing-newline behavior are defined transformations; use `stripIndent` or explicit escapes only when the contract requires it.

Records

JAVA

```
record Point(int x, int y) {}
```

A record is implicitly final and extends `java.lang.Record`. It receives private final component fields, accessors named after components, a canonical constructor, and value-oriented `equals`, `hashCode`, and `toString`. It may implement interfaces and define members, but cannot declare extra instance fields. Component objects can still be mutable.

Sealed hierarchies

JAVA

```
sealed interface Command permits Create, Delete {}
record Create(String id) implements Command {}
record Delete(String id) implements Command {}
```

The permits relationship is enforced at compile time and in class metadata. It is useful when the domain genuinely owns a closed alternative set. `non-sealed` deliberately reopens a branch.

Record patterns and pattern switch

JAVA

```
static String describe(Object value) {
    return switch (value) {
        case null -> "null";
        case Point(int x, int y) when x == y -> "diagonal " + x;
        case Point(int x, int y) -> x + "," + y;
        default -> value.toString();
    };
}
```

Case labels are checked for dominance. An earlier unconditional supertype pattern can make a later subtype case unreachable. Guards refine a matching case. Exhaustiveness rules depend on the selector type; enum and sealed hierarchies are common interview examples.

D.3 Library and platform matrix

Feature	Release	Core idea	Boundary to remember
module system	9	reliable configuration and strong encapsulation	modules complement, not replace, packages/JARs
collection factories	9/10	List.of , Set.of , Map.of , copyOf	unmodifiable and null-rejecting, not necessarily a new object
standard HTTP client	11	synchronous/asynchronous HTTP/2-capable client	application still owns timeouts, body limits, retries, and executors
helpful NPE messages	14	more precise dereference context	diagnostic behavior, not a null-safety type system
strong encapsulation progression	9-17	restrict unsupported JDK internals	migration should remove reliance on internals, not add permanent opens
random generator interfaces	17	named/splittable/jumpable algorithms	choose by statistical/concurrency need, not habit
simple web server	18	minimal local HTTP serving API	not a full production service framework
UTF-8 default charset	18	standardized default charset	explicit charsets remain best at protocol/file boundaries
sequenced collections	21	uniform first/last/reversed encounter-order APIs	only meaningful for collections with a defined encounter order
virtual threads	21	cheap JVM-managed thread-per-task	do not pool to limit threads; bound external resources
generational ZGC	21	generational mode option for ZGC	collector flags/production maturity are release-specific

D.4 Sequenced collections

Java 21 adds `SequencedCollection`, `SequencedSet`, and `SequencedMap` to express defined encounter order and operations at both ends.

Representative methods:

JAVA

```
list.getFirst();
list.getLast();
list.addFirst(value);
list.reversed();

map.firstEntry();
map.lastEntry();
map.sequencedKeySet();
map.reversed();
```


Support and mutation behavior still depend on the concrete collection. A reversed view is generally a view, not an independent copy. An unmodifiable collection remains unmodifiable through its reversed view.

D.5 Virtual threads

Creation patterns:

JAVA

```
Thread thread = Thread.ofVirtual().start(task);

try (var executor = java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor()) {
    var future = executor.submit(task);
    future.get();
}
```

Virtual threads preserve the **Thread** programming model and are most compelling for large numbers of mostly blocking tasks. They improve scalability by allowing a virtual thread to unmount from a carrier during many blocking operations. They do not make CPU-bound work cheaper, change happens-before rules, or make shared mutable state safe.

Operational considerations:

- Replace thread-pool size as admission control with explicit semaphores, connection pools, rate limits, and bounded queues at constrained resources.
- Thread-local values multiplied across very many tasks can retain surprising state; keep them deliberate and small.
- Some blocking while holding a monitor or executing native/foreign code can pin a virtual thread to a carrier in Java 21. Diagnose with release-appropriate JFR/tooling rather than guessing.
- Preserve cancellation and structured lifecycle even when threads are inexpensive.

Structured concurrency and scoped values were preview APIs in Java 21 and are not used as final Java 21 APIs in this book.

D.6 API removals, encapsulation, and migration

Modernizing from Java 8/11 to 17/21 is more than changing a compiler level:

- 1 Inventory runtime distribution, build plugins, bytecode agents, annotation processors, JNI libraries, and reflective frameworks.
- 2 Run `jdeps` and tests to find internal JDK dependencies and removed modules.
- 3 Update dependencies before using broad `--add-opens` workarounds.
- 4 Check default charset, TLS/security defaults, GC defaults, container sizing, and logging.
- 5 Compile with `--release` and treat illegal reflective access or deprecation warnings as migration work.
- 6 Benchmark representative startup, steady-state, allocation, memory, and tail latency.
- 7 Roll out with canaries and explicit rollback criteria.

`--add-opens` can be a temporary compatibility bridge. It should name a specific need and have an owner/removal plan, because it weakens encapsulation and can hide unsupported coupling.

D.7 Preview and incubator discipline

Preview language/platform features require `--enable-preview` for compilation and execution with a matching release. Class files record preview usage. Incubator modules normally require `--add-modules` and can change or disappear.

Java 21's non-final features included:

Feature	Java 21 status	JEP
String Templates	preview	430
Unnamed Patterns and Variables	preview	443
Unnamed Classes and Instance Main Methods	preview	445
Foreign Function and Memory API	third preview	442
Scoped Values	preview	446
Structured Concurrency	preview	453
Vector API	sixth incubator	448

The iteration label matters: a later preview or incubator round is still not a final Java SE contract. Verify the exact JDK release before compiling examples or making a production compatibility promise.

D.8 Later-release delta: JDK 24 and JDK 25

This is context for interviews held after the Java 21 baseline. It does not change the source level of the book's runnable examples.

Later change	Delivered status	What a Java 21 answer should add
virtual-thread monitor pinning	JEP 491 , delivered in JDK 24	Java 21 can pin a virtual thread during blocking inside synchronized ; JDK 24 removes nearly all monitor-related pinning, while selected native and class-initialization cases can remain
scoped values	JEP 506 , final in JDK 25	do not use the Java 21 preview API as a stable contract; describe the final JDK 25 API separately
module import declarations	JEP 511 , final in JDK 25	this is source convenience and does not replace module readability, exports, or reliable configuration
compact source files and instance main methods	JEP 512 , final in JDK 25	useful for small programs and learning; ordinary class-based source remains valid
flexible constructor bodies	JEP 513 , final in JDK 25	statements may precede an explicit constructor invocation under the new rules; Java 21 retains the older first-statement restriction
compact object headers	JEP 519 , delivered in JDK 25	object-header width is a VM/version/flag detail, so never present one layout as a Java guarantee
string templates	JEP 465 withdrawn	Java 21's preview syntax did not become a final feature; do not write production guidance as if STR were a current standard API
structured concurrency	fifth preview in JDK 25	the concept is useful, but preview signatures and lifecycle rules still require exact-release verification

The [OpenJDK JDK 25 release page](#) records the complete delivered feature set and its General Availability date. Long-term-support availability remains a distribution/vendor support decision even when a release is widely described as LTS.

This delta illustrates a durable interview habit: name the requested baseline first, then give a short later-release update. Do not silently answer a Java 21 question with JDK 25 behavior, and do not keep repeating a Java 21 limitation after the deployed runtime has removed it.

Interview answer pattern:

- State whether the feature is final, preview, or incubating in the named JDK.
- Describe the final Java 21 baseline separately from later evolution.
- Do not present a preview API signature as a stable production contract.
- Explain why an organization might experiment, and how it limits migration and support risk.

D.9 Feature selection checklist

Need	Relevant modern feature	Question before adoption
compact immutable-looking value carrier	record	are component values themselves safely immutable/copyable?
closed domain alternatives	sealed hierarchy	does the domain truly own and control all direct variants?
exhaustive type-based branching	pattern switch	are null, dominance, and future evolution handled?
deconstruct nested values	record pattern	is pattern logic clearer than behavior on the domain type?
readable embedded multi-line text	text block	are indentation, escaping, and untrusted input handled?
thread-per-request blocking service	virtual threads	which downstream resources still need explicit bounds?
uniform first/last/reverse access	sequenced collections	does the concrete collection define encounter order and supported mutation?
safer fixed collection boundary	factory/copy methods	is shallow immutability enough and are nulls prohibited?

Modern syntax is valuable when it sharpens the model. An SDE-2 answer also names compatibility, operability, lifecycle, and team-readability costs.

REFERENCE APPENDIX

Appendix E - Exercise Hints and Selected Solutions

E.1 How to use this appendix

These are not scripts to memorize. Each solution demonstrates an interview method:

- 1 restate the contract and assumptions;
- 2 identify the invariant;
- 3 choose a representation;
- 4 derive correctness before complexity;
- 5 name edge cases and failure behavior; and
- 6 test with a small dry run.

For each exercise, stop after the hint and attempt a solution. Compare reasoning before comparing syntax. A different solution is valid when it satisfies the stated contract and you can defend its costs. Code targets Java 17 unless a Java 21 feature is explicitly identified.

The self-check criteria are intentionally strict. An interview answer is not complete merely because the sample returns the expected output. It must state why the result generalizes and what the code promises at production boundaries.

E.2 JVM exercise: class initialization order

Problem. Predict the output. Explain which events are guaranteed by the Java language and which physical loading or compilation details remain implementation-specific.

JAVA

```
public final class InitializationOrderExercise {
    static class Parent {
        static int parentValue = mark("Parent field");

        static {
            System.out.println("Parent block");
        }

        static int mark(String text) {
            System.out.println(text);
            return 9;
        }
    }

    static class Child extends Parent {
        static final int CONSTANT = 7;
        static Integer boxed = mark("Child field");

        static {
            System.out.println("Child block");
        }
    }

    public static void main(String[] args) {
        System.out.println(Child.CONSTANT);
        System.out.println(Child.boxed);
    }
}
```

Hint. Separate loading/linking from initialization. Then ask whether each field read is an active use and whether the field is a compile-time constant variable.

Selected solution. The expected output is:

TEXT

```
7
Parent field
Parent block
Child field
Child block
9
```

`Child.CONSTANT` is a `static final` primitive initialized by a constant expression. Its value can be embedded in the caller, so reading it does not trigger `Child` initialization. Reading `Child.boxed` is an active use of `Child`; before a class initializes, its superclass initializes. Within each class, static field initializers and static blocks execute in textual order. Therefore the parent field and block precede the child field and block.

The JVM may have loaded or verified either nested class earlier, and HotSpot may interpret or compile methods at any appropriate time. Those details do not change the specified initialization output. In a real system, avoid

important side effects in static initialization: failure produces `ExceptionInInitializerError`, and later active use can fail because initialization did not complete.

Self-check. You should be able to:

- distinguish load, link, and initialize;
- identify a compile-time constant without running the code;
- state superclass-before-subclass initialization; and
- avoid claiming when JIT compilation occurs.

E.3 Language exercise: pass-by-value and aliasing

Problem. Explain why `swap` fails to swap the caller's variables while `rename` changes the shared object. Then repair the design so the caller can obtain swapped values without hidden mutation.

JAVA (continued 1/2)

```
public final class PassByValueExercise {
    static final class User {
        private String name;

        User(String name) {
            this.name = name;
        }

        void rename(String newName) {
            name = newName;
        }

        String name() {
            return name;
        }
    }

    record Pair<T>(T first, T second) {}

    static void swap(User left, User right) {
        User temporary = left;
        left = right;
        right = temporary;
    }
}
```

JAVA (continued 2/2)

```
static void rename(User user) {
    user.rename("renamed");
}

static <T> Pair<T> swapped(T left, T right) {
    return new Pair<>(right, left);
}

public static void main(String[] args) {
    User first = new User("first");
    User second = new User("second");

    swap(first, second);
    System.out.println(first.name() + "," + second.name());

    rename(first);
    System.out.println(first.name());

    Pair<User> result = swapped(first, second);
    first = result.first();
    second = result.second();
    System.out.println(first.name() + "," + second.name());
}
```

Hint. Draw caller variables and parameter variables as separate boxes. Copy references into parameter boxes. Reassignment changes one box; mutation follows a reference to an object.

Selected solution. Java always passes argument values. For reference expressions, the value is a reference. `swap` receives copies of the two references and only reassigns its local copies, so the caller prints `first, second`. `rename` follows its copied reference to the same `User`, so the caller observes `renamed`. Returning a `Pair` communicates new reference placement explicitly; the final line is `second, renamed`.

The phrase "objects are passed by reference" is misleading because it predicts that parameter reassignment can update caller variables. It cannot. The language does not expose the physical address representation of a reference.

Self-check. Your diagram should contain four variable boxes before `swap`: two caller variables and two parameters. You should predict every output and distinguish reassignment, mutation, and immutable-result design.

E.4 Generics exercise: a type-safe copy operation

Problem. Implement a method that copies values from a producer collection into a consumer collection. It must allow `List<Integer>` to be copied into `List<Number>` without raw types or casts.

JAVA

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;

public final class GenericCopyExercise {
    static <T> void copy(Collection<? extends T> source,
        Collection<? super T> destination) {
        for (T value : source) {
            destination.add(value);
        }
    }

    public static void main(String[] args) {
        List<Integer> integers = List.of(1, 2, 3);
        List<Number> numbers = new ArrayList<>();
        copy(integers, numbers);
        System.out.println(numbers);
    }
}
```

Hint. Use PECS: producer extends, consumer super. Ask what can be safely read and what can be safely written.

Selected solution. `source` has an unknown element type that is a subtype of `T`, so every value read from `? extends T` is assignable to `T`. `destination` can consume `T`, so `? super T` is safe for adding a `T`. Reading an arbitrary value from the destination only yields `Object` because its actual element type might be any supertype of `T`.

`List<Integer>` is not a subtype of `List<Number>`; generic types are invariant. If it were, a caller could add a `Double` through the `List<Number>` view and corrupt the integer list. Wildcards express use-site variance without permitting that write.

Self-check. Compile calls for `Integer -> Number`, `Integer -> Object`, and `Number -> Object`. Confirm that `Number -> Integer` is rejected. Explain the rejection without saying "the compiler is confused."

E.5 Collections exercise: stable frequency ranking

Problem. Count words case-insensitively and return entries ordered by descending frequency, then ascending normalized word. Do not depend on `HashMap` encounter order.

JAVA

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

public final class FrequencyRankingExercise {
    record Count(String word, int occurrences) {}

    static List<Count> rank(List<String> words) {
        Map<String, Integer> counts = new HashMap<>();
        for (String word : words) {
            if (word == null) {
                throw new IllegalArgumentException("null word");
            }
            String normalized = word.toLowerCase(Locale.ROOT);
            counts.merge(normalized, 1, Integer::sum);
        }

        ArrayList<Count> result = new ArrayList<>(counts.size());
        counts.forEach((word, count) -> result.add(new Count(word, count)));
        result.sort(Comparator.comparingInt(Count::occurrences).reversed()
            .thenComparing(Count::word));
        return List.copyOf(result);
    }

    public static void main(String[] args) {
        System.out.println(rank(List.of("Java", "map", "JAVA", "Map", "java")));
    }
}
```

Hint. Separate aggregation order from presentation order. Add a deterministic tie-breaker.

Selected solution. The hash map provides expected constant-time aggregation. The comparator completely defines presentation: higher counts first, then lexical word. The result is `[Count[word=java, occurrences=3], Count[word=map, occurrences=2]]`. `Locale.ROOT` avoids environment-dependent case mapping for protocol-like tokens.

Let C be the total character count and u the number of unique normalized words. Expected aggregation is $O(C)$ because normalization, hashing, and equality inspect characters. Result creation is $O(u)$. Sorting performs $O(u \log u)$ comparisons, with lexical comparison cost proportional to the compared prefixes; it is only $O(u \log u)$ under a constant-length-word assumption. Space is $O(C)$ in the worst case for retained normalized keys. If words are human-language text, normalization and collation require a domain-specific Unicode/locale policy beyond lowercase.

Self-check. Test empty input, ties, repeated case variants, and a null. Explain why stable sorting alone would not make tied output deterministic if the input to the sort came from an unordered map.

E.6 Collections design exercise: bounded LRU cache

Problem. Implement a small least-recently-used cache. State what the implementation does not guarantee.

JAVA (continued 1/2)

```
import java.util.LinkedHashMap;
import java.util.Map;

public final class LruCacheExercise<K, V> {
    private final int capacity;
    private final LinkedHashMap<K, V> entries;

    public LruCacheExercise(int capacity) {
        if (capacity < 1) {
            throw new IllegalArgumentException("capacity must be positive");
        }
        this.capacity = capacity;
        this.entries = new LinkedHashMap<>(16, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
                return size() > LruCacheExercise.this.capacity;
            }
        };
    }

    public V get(K key) {
```

JAVA (continued 2/2)

```
        return entries.get(key);
    }

    public void put(K key, V value) {
        entries.put(key, value);
    }

    public int size() {
        return entries.size();
    }

    public static void main(String[] args) {
        LruCacheExercise<String, Integer> cache = new LruCacheExercise<>(2);
        cache.put("a", 1);
        cache.put("b", 2);
        cache.get("a");
        cache.put("c", 3);
        System.out.println(cache.get("a")); // 1
        System.out.println(cache.get("b")); // null
    }
}
```

Hint. `LinkedHashMap` can maintain access order. Evict only after insertion makes size exceed capacity.

Selected solution. Accessing `a` moves it behind `b` in access order. Adding `c` makes `b` eldest, so the override removes `b`. Basic map operations are expected `O(1)`.

This is not thread-safe, does not distinguish an absent key from a cached null, uses entry count rather than byte weight, has no expiration, does not load values atomically, and is not distributed. Production cache design needs

an admission/eviction policy, metrics, concurrency, stampede control, and failure semantics. `LinkedHashMap` access-order behavior is an API contract; its node layout is not.

Self-check. Verify recency changes on both `get` and replacement. Explain how you would add synchronization without returning iterators that escape the lock. Define whether null keys/values are valid.

E.7 Concurrency exercise: safe lazy initialization

Problem. Replace a racy lazy singleton without explicit locking on every read. Explain its happens-before argument.

JAVA

```
public final class LazyInitializationExercise {
    private LazyInitializationExercise() {}

    private static final class Holder {
        static final LazyInitializationExercise INSTANCE =
            new LazyInitializationExercise();
    }

    public static LazyInitializationExercise instance() {
        return Holder.INSTANCE;
    }

    public static void main(String[] args) throws InterruptedException {
        LazyInitializationExercise[] observed = new LazyInitializationExercise[2];
        Thread first = new Thread(() -> observed[0] = instance());
        Thread second = new Thread(() -> observed[1] = instance());
        first.start();
        second.start();
        first.join();
        second.join();
        System.out.println(observed[0] == observed[1]);
    }
}
```

Hint. Class initialization is synchronized by the JVM and happens-before later use of that class by any thread.

Selected solution. `Holder` is not initialized merely because the outer class initializes. The first active read of `Holder.INSTANCE` triggers `Holder` initialization. The initialization procedure permits one thread to perform it while other threads wait, and successful class initialization happens-before subsequent active use. Final-field semantics add safety for immutable constructed state, but the key publication edge here is class initialization.

The program prints `true`. `join` also gives the main thread a happens-before edge from each worker's completed actions, so the array reads are visible. Without `join` or another synchronization edge, reading the array from main would race.

An enum singleton is another simple solution when its serialization and initialization semantics fit. Double-checked locking requires a `volatile` field and a correct local/read pattern; the holder idiom is harder to get wrong.

Self-check. Name both happens-before edges: class initialization to active use, and worker actions to successful `join` return. Explain why "the object was constructed first" is not alone a memory-visibility proof.

E.8 Concurrency exercise: bounded task fan-out and cancellation

Problem. Run several tasks with a total timeout, cancel unfinished work, preserve interruption, and avoid leaking the executor.

JAVA (continued 1/2)

```
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;

public final class BoundedFanOutExercise {
    static <T> List<T> run(List<? extends Callable<T>> tasks,
        int parallelism, Duration timeout) throws Exception {
        if (parallelism < 1 || timeout.isNegative() || timeout.isZero()) {
            throw new IllegalArgumentException("invalid limits");
        }

        ExecutorService executor = Executors.newFixedThreadPool(parallelism);
        try {
            List<Future<T>> futures;
            try {
                futures = executor.invokeAll(
                    tasks, timeout.toNanos(), TimeUnit.NANOSECONDS);
            } catch (InterruptedException interrupted) {
                Thread.currentThread().interrupt();
                throw interrupted;
            }

            ArrayList<T> results = new ArrayList<>(futures.size());
            for (Future<T> future : futures) {
```

JAVA (continued 2/2)

```

        if (future.isCancelled()) {
            throw new TimeoutException("fan-out deadline exceeded");
        }
        try {
            results.add(future.get());
        } catch (ExecutionException failed) {
            Throwable cause = failed.getCause();
            if (cause instanceof Exception exception) {
                throw exception;
            }
            throw (Error) cause;
        } catch (InterruptedException interrupted) {
            Thread.currentThread().interrupt();
            throw interrupted;
        }
    }
    return List.copyOf(results);
} finally {
    executor.shutdownNow();
}
}

public static void main(String[] args) throws Exception {
    List<Callable<String>> tasks = List.of(
        () -> "alpha",
        () -> "beta",
        () -> "gamma");
    System.out.println(run(tasks, 2, Duration.ofSeconds(1)));
}
}

```

Hint. Use the timed bulk operation, but still inspect cancellation. Treat interrupt as cancellation, not a retryable ordinary exception.

Selected solution. `invokeAll` submits all tasks, waits no longer than the timeout, and cancels unfinished futures on return. A fixed pool bounds simultaneous work, not the submitted task list; callers must also bound list size. Results retain input order because the returned future list corresponds to task order, not completion order.

`shutdownNow` requests interruption but cannot force code that ignores interruption to stop. The task contract must honor cancellation and bound its own I/O. Before timed `invokeAll` returns, each task has either completed or been canceled because the timeout expired; throwing while inspecting the first failed future does not undo work that already completed. A production design must decide fail-fast versus collect-all outcomes, attach multiple failures if needed, and use bounded `awaitTermination` handling when executor termination matters to lifecycle safety.

Java 21 virtual threads can simplify blocking task-per-request code, but scarce downstream connections still require a semaphore, pool, or admission bound. Virtual threads remove thread scarcity, not resource limits.

Self-check. Test one slow task, one throwing task, and caller interruption. Confirm no test waits forever. State which order the returned results use and why cancellation is cooperative.

E.9 Performance exercise: repair a misleading benchmark

Problem. A candidate presents this result as proof that string parsing takes two nanoseconds:

JAVA

```
long start = System.nanoTime();
for (int i = 0; i < 100_000_000; i++) {
    Integer.parseInt("123");
}
long elapsed = System.nanoTime() - start;
System.out.println(elapsed / 100_000_000.0);
```

Identify at least six validity problems and design a JMH experiment.

Hint. Ask whether the result is observable, the input is constant, compilation is warm, process history is independent, setup matches production, and uncertainty is reported.

Selected solution. Problems include:

- 1 The result is unused, so the optimizer can eliminate work.
- 2 The string is constant, so parsing can be folded or specialized.
- 3 One process mixes warmup and measurement.
- 4 One aggregate timing reports no iteration/fork variance.
- 5 The loop and division are part of a homemade harness.
- 6 There is no baseline for harness overhead.
- 7 One tiny valid input does not represent lengths, signs, failures, or varied digits.
- 8 Environment, JDK, flags, CPU limits, and thermal/background state are missing.
- 9 A microbenchmark does not establish endpoint impact.

A defensible JMH design uses `@Param` for representative strings prepared in trial state, returns the parsed value or consumes it in a **Blackhole**, uses several warmup and measurement iterations in multiple forks, and reports configuration and uncertainty. Invalid-input parsing should be a separate benchmark because exception construction changes the operation. Add allocation or compiler profiling in separate runs, then confirm through a production profile that parsing is hot enough to matter.

Do not make input unpredictability artificial. If production repeatedly parses a small fixed vocabulary, model that vocabulary. If values vary per request, prepare an array of varied inputs and advance an index without putting random generation in the timed operation.

Self-check. Your design must state the exact question, mode, inputs, state scope, setup level, result consumption, forks, warmup, measurement, and the higher-level validation needed.

E.10 DSA exercise: longest substring without repeating characters

Problem. Given an ASCII string, return the length of its longest substring containing no repeated character. Target $O(n)$ time.

JAVA

```
import java.util.Arrays;

public final class LongestUniqueWindowExercise {
    static int longestUnique(String text) {
        if (text == null) {
            throw new IllegalArgumentException("text is required");
        }
        int[] lastSeen = new int[128];
        Arrays.fill(lastSeen, -1);

        int left = 0;
        int best = 0;
        for (int right = 0; right < text.length(); right++) {
            char current = text.charAt(right);
            if (current >= 128) {
                throw new IllegalArgumentException("ASCII input required");
            }
            left = Math.max(left, lastSeen[current] + 1);
            lastSeen[current] = right;
            best = Math.max(best, right - left + 1);
        }
        return best;
    }

    public static void main(String[] args) {
        System.out.println(longestUnique("abba")); // 2
        System.out.println(longestUnique("abcabcbb")); // 3
    }
}
```

Hint. Maintain a half-open or closed window invariant and jump the left boundary past the previous occurrence. Never move left backward.

Selected solution. Before processing `right`, the window from `left` through `right - 1` contains no duplicate. If the new character was seen inside that window, `lastSeen[current] + 1` moves `left` past it. `Math.max` prevents an older occurrence outside the window from moving `left` backward.

For `abba`, windows evolve as `a`, `ab`, then the second `b` moves left from 0 to 2, producing `b`; the final `a` has an old index 0 outside the current window, so left remains 2 and `ba` has length 2.

Each index advances once and table operations are constant, so time is $O(n)$ and auxiliary space is $O(1)$ for the fixed ASCII alphabet. Java `char` represents UTF-16 code units, not every Unicode code point. A Unicode-code-point contract should iterate `text.codePoints()` and use a map or appropriately sized structure.

Self-check. Test empty, one-character, all-equal, all-unique, and `abba`. State the window invariant before writing complexity.

E.11 DSA exercise: deterministic topological ordering

Problem. Given directed edges `prerequisite -> dependent`, return a deterministic topological order or reject a cycle.

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Set;

public final class TopologicalOrderExercise {
    record Edge(String prerequisite, String dependent) {}

    static List<String> order(Set<String> nodes, List<Edge> edges) {
        Map<String, List<String>> outgoing = new HashMap<>();
        Map<String, Integer> indegree = new HashMap<>();
        for (String node : nodes) {
            outgoing.put(node, new ArrayList<>());
            indegree.put(node, 0);
        }

        Set<Edge> uniqueEdges = new HashSet<>();
        for (Edge edge : edges) {
            if (!nodes.contains(edge.prerequisite())
                || !nodes.contains(edge.dependent())) {
                throw new IllegalArgumentException("edge contains unknown node");
            }
            if (uniqueEdges.add(edge)) {
                outgoing.get(edge.prerequisite()).add(edge.dependent());
                indegree.merge(edge.dependent(), 1, Integer::sum);
            }
        }
    }
}
```

JAVA (continued 2/2)

```

    PriorityQueue<String> ready = new PriorityQueue<>();
    indegree.forEach((node, degree) -> {
        if (degree == 0) ready.offer(node);
    });

    ArrayList<String> result = new ArrayList<>(nodes.size());
    while (!ready.isEmpty()) {
        String node = ready.poll();
        result.add(node);
        for (String dependent : outgoing.get(node)) {
            int remaining = indegree.merge(dependent, -1, Integer::sum);
            if (remaining == 0) ready.offer(dependent);
        }
    }

    if (result.size() != nodes.size()) {
        throw new IllegalArgumentException("graph contains a cycle");
    }
    return List.copyOf(result);
}

public static void main(String[] args) {
    Set<String> nodes = Set.of("compile", "test", "package", "deploy");
    List<Edge> edges = List.of(
        new Edge("compile", "test"),
        new Edge("test", "package"),
        new Edge("package", "deploy"));
    System.out.println(order(nodes, edges));
}
}

```

Hint. Kahn's algorithm repeatedly removes a zero-indegree node. If nodes remain but none is ready, those remaining dependencies contain a cycle.

Selected solution. Indegree counts unmet prerequisites. Removing a ready node conceptually removes all its outgoing edges; a dependent becomes ready exactly when its count reaches zero. The priority queue provides lexical determinism among simultaneously ready nodes. Duplicate edges are removed so they do not inflate indegree.

With V nodes and E unique edges, map construction and edge processing are expected $O(V + E)$. Each node enters the heap once, adding $O(V \log V)$ for deterministic selection; total is $O(E + V \log V)$ and space $O(V + E)$. A simple deque gives $O(V + E)$ but its order depends on insertion/encounter policy.

Self-check. Test disconnected nodes, two valid choices, duplicate edges, a self-loop, a longer cycle, and an unknown endpoint. Prove that every emitted node has all prerequisites emitted first.

E.12 DSA exercise: top-k values without sorting everything

Problem. Return the largest k integers in descending order. Use $O(k)$ auxiliary storage when k is much smaller than n .

JAVA

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

public final class TopKExercise {
    static List<Integer> largest(List<Integer> values, int k) {
        if (k < 0 || k > values.size()) {
            throw new IllegalArgumentException("k out of range");
        }
        PriorityQueue<Integer> retained = new PriorityQueue<>(Math.max(1, k));
        for (Integer value : values) {
            if (value == null) throw new IllegalArgumentException("null value");
            if (retained.size() < k) {
                retained.offer(value);
            } else if (k > 0 && value > retained.peek()) {
                retained.poll();
                retained.offer(value);
            }
        }
        ArrayList<Integer> result = new ArrayList<>(retained);
        result.sort(Comparator.reverseOrder());
        return List.copyOf(result);
    }

    public static void main(String[] args) {
        System.out.println(largest(List.of(4, 1, 9, 2, 9, 7), 3));
    }
}
```

Hint. Keep the worst currently retained answer at the heap head. It can be replaced in logarithmic time.

Selected solution. A min-heap of size k stores the largest values seen so far. Once full, a value no greater than the root cannot belong to a strictly better top- k multiset; a larger value replaces the root. Duplicate values are retained because the problem asks for values, not unique values.

For $1 \leq k \leq n$, scanning costs $O(n \log k)$, sorting the result costs $O(k \log k)$, and auxiliary space is $O(k)$. For $k = 0$, this implementation still validates all inputs and therefore costs $O(n)$ time and $O(1)$ auxiliary space. For k close to n , sorting a copy may be simpler and faster. Priority-queue iteration is not sorted, so the final explicit sort is necessary.

Self-check. Test $k = 0$, $k = n$, duplicates, negative values, and invalid k . Explain why arbitrary priority-queue iteration cannot be returned directly.

E.13 Backend design exercise: idempotent order plus outbox

Problem. Design `POST /orders` so a client can safely retry after a timeout. The service must create one logical order and eventually emit `OrderCreated` without a distributed transaction.

Hint. Separate client command identity, local atomic state, relay delivery, and consumer effect identity.

Selected solution. Require an idempotency key scoped to authenticated customer and operation. In one database transaction:

- 1 use a database-tested atomic insert-if-absent or upsert for an idempotency row containing `(customer, operation, key)`, a canonical request fingerprint, status `IN_PROGRESS`, and a generated order ID under a unique constraint;
- 2 when that operation reports an existing row, lock/read it where the database permits the read in the same transaction; if a uniqueness error aborts the transaction, roll it back and read in a new transaction. Reject a different fingerprint, return the stored completed response, or report/internally coordinate an in-progress attempt;
- 3 insert the order using the reserved order ID;
- 4 insert a versioned outbox event with a stable event ID and aggregate ID;
- 5 store the completed response/status in the idempotency row; and
- 6 commit.

A relay claims committed outbox rows, publishes, and records progress. If it crashes after broker acceptance but before progress commit, it republishes. Consumers therefore record event ID in an inbox or use a naturally idempotent conditional update in the same transaction as their effect. This is recoverable at-least-once delivery; it does not promise one physical delivery.

The HTTP request has an end-to-end deadline. A timeout is an unknown outcome, so the client retries the same key or queries order status. The key record's retention exceeds the maximum retry window. Cleanup never removes an in-progress record without reconciliation. Metrics include key conflicts, in-progress age, outbox age, publish attempts, poison events, and consumer duplicates.

Security rules include binding the key to customer identity, limiting key length and cardinality, comparing request fingerprints, redacting payloads, and authorizing status lookup. Reliability rules include a unique database constraint as the concurrent truth, bounded relay batches, leases, backoff, and dead-letter/repair workflow.

Self-check. Your answer must identify these crash windows: before commit, after commit before response, after publish before relay acknowledgement, and during consumer effect. For each, state the durable record that permits recovery and the stable identity that prevents duplicate logical effects.

E.14 Low-level design exercise: notification delivery

Problem. Design a notification component supporting email and SMS today, another channel later, templates, user preferences, retries, and audit. Do not build a pattern catalog for its own sake.

Hint. Separate application orchestration, policy, rendering, channel adapter, durable command state, and delivery attempts.

Selected solution. A compact domain API might be:

JAVA (continued 1/2)

```
import java.time.Instant;
import java.util.Locale;
import java.util.Map;

enum Channel { EMAIL, SMS }

record NotificationCommand(String commandId, String userId,
    String templateId, Map<String, String> parameters, Instant createdAt) {
    public NotificationCommand {
        parameters = Map.copyOf(parameters);
    }
}

record Recipient(String address, Locale locale) {}
record RenderedMessage(String subject, String body) {}
record DeliveryResult(String providerMessageId, Instant acceptedAt) {}

interface PreferencePolicy {
    java.util.List<Channel> allowedChannels(String userId);
}

interface TemplateRenderer {
```

JAVA (continued 2/2)

```
    RenderedMessage render(String templateId, Locale locale,
        Map<String, String> parameters);
}

interface ChannelSender {
    Channel channel();
    DeliveryResult send(String idempotencyKey, Recipient recipient,
        RenderedMessage message) throws DeliveryException;
}

final class DeliveryException extends Exception {
    private final boolean retryable;

    DeliveryException(String message, boolean retryable) {
        super(message);
        this.retryable = retryable;
    }

    boolean retryable() {
        return retryable;
    }
}
```

`NotificationService` validates and durably stores the command before asynchronous processing. `PreferencePolicy` is policy; `TemplateRenderer` owns safe template expansion; each `ChannelSender` is a Strategy and vendor adapter. A factory or map selects a sender by its declared channel. Metrics can be a Decorator, but retry belongs in orchestration because it needs durable attempt state and idempotency.

The command ID is the logical idempotency identity. A delivery table records channel, recipient version, template version, attempt, next eligible time, status, and safe provider response. Workers claim bounded batches with a lease. Retryable errors receive jittered backoff under an attempt/deadline policy; permanent address or authorization errors do not retry. Provider timeouts are unknown outcomes, so the sender passes a stable idempotency key when supported or reconciles provider status.

Preferences and destination addresses can change. Decide whether the command snapshots them at acceptance or resolves them at delivery; that business rule affects audit and privacy. Templates require versioning so a retry does not silently change content. Audit records exclude secrets and sensitive body content unless retention policy explicitly permits it.

Adding push notification introduces one `Channel`, sender, address policy, and integration tests without changing existing senders. This is Open/Closed Principle applied at an observed variation boundary. The system still needs rate limits per tenant/channel, provider bulkheads, queue bounds, graceful shutdown, and deletion/redaction workflows.

Self-check. State ownership, lifecycle, idempotency, retry classification, timeout outcome, template/preference version, provider isolation, audit sensitivity, and extension path. If your design sends directly in the request transaction, explain how it recovers from every crash window.

E.15 Final self-assessment rubric

Score each selected solution from 0 to 2 on each dimension:

A score below 10 suggests revisiting the chapter before adding more problems. A score of 12 or more is interview-ready only if you can reproduce the reasoning without seeing the solution. Practice explaining each answer in three layers: a 30-second summary, a two-minute model, and a detailed defense with code and edge cases.

Dimension	0	1	2
Contract	assumptions missing	partial contract	inputs, outputs, failure, ownership explicit
Invariant	absent	named but not used	drives representation and proof
Correctness	example only	plausible explanation	general argument plus dry run
Complexity	missing/wrong	time only	time, space, assumptions, constants discussed
Edge cases	none	common cases	boundaries, invalid input, overflow/null/concurrency
Production	ignored	generic note	limits, observability, lifecycle, failure policy
Communication	code dump	understandable	answer-first, structured, trade-offs explicit

REFERENCE APPENDIX

Appendix F - Glossary

F.1 Reading the glossary

Definitions describe the Java 21 platform unless a version is named. "HotSpot" identifies common OpenJDK implementation behavior rather than a Java specification guarantee. Chapter references point to the principal treatment; many concepts recur throughout the book.

F.2 A

- **Abstract class:** A class declared `abstract` that cannot be instantiated directly and may combine implemented methods, state, constructors, and abstract methods for subclasses. See Chapter 18.
- **Abstraction:** A model exposing behavior that clients need while hiding representation or mechanism. Good abstractions state invariants, ownership, failure, and performance boundaries. See Chapter 49.
- **Access modifier:** `public`, `protected`, or `private`. Where the declaration rules permit it, omitting an access modifier gives package access; interface members and implicitly declared constructors have additional rules. Modifiers control source-level member/type accessibility, not network authorization or runtime data secrecy. See Chapter 16.
- **ACID:** Atomicity, consistency, isolation, and durability: goals used to reason about database transactions. Exact isolation and durability depend on the database and configuration. See Chapter 50.
- **Active use:** An action that can trigger class or interface initialization, such as selected static field access, static method invocation, instance creation, or reflective request. See Chapter 5.
- **Adapter:** A design pattern translating one contract into another, commonly isolating a vendor API from domain policy. A good adapter prevents vendor types from leaking inward. See Chapter 49.
- **Allocation rate:** Bytes or objects allocated per time. High allocation can drive GC work without implying a retention leak. Correlate it with live-set and latency evidence. See Chapters 39 and 41.
- **Amortized analysis:** Averaging total cost over an operation sequence. Dynamic-array append is amortized constant time even though an individual resize copies many references. See Chapters 26 and 42.
- **Annotation:** Metadata attached to declarations, types, or other supported locations. Retention and target control where it is available; behavior comes from compilers, runtimes, or frameworks. See Chapter 21.
- **API contract:** Observable promises of a type or operation: valid inputs, results, failures, side effects, ordering, mutation, blocking, ownership, and thread safety. See Chapter 49.
- **Array covariance:** Java permits a `Sub[]` where a `Super[]` is expected. Runtime store checks preserve type safety and can throw `ArrayStoreException`; generics are invariant. See Chapters 15 and 22.
- **Atomic operation:** An operation observed as indivisible under its stated concurrency model. Atomicity of one method does not make a multi-call invariant atomic. See Chapters 35 and 37.
- **Atomicity:** All-or-nothing behavior within a defined boundary. CPU atomics, Java atomic classes, and database transactions have different scopes and guarantees. See Chapters 35 and 50.
- **Autoboxing:** Compiler conversion from a primitive to its wrapper, such as `int` to `Integer`. It can add allocation, null, identity, and overload-selection surprises. See Chapter 12.

F.3 B

- **Backpressure:** A policy that slows, rejects, blocks, sheds, or durably spills production when consumers lack capacity. It prevents an unbounded queue from hiding overload. See Chapters 36 and 52.
- **Balanced tree:** A search tree with structural constraints keeping height logarithmic. Java's API need not prescribe the exact balancing algorithm of every ordered collection. See Chapters 28 and 45.
- **Behavioral subtyping:** Requirement that a subtype preserve the supertype's preconditions, postconditions, invariants, failures, and relevant timing/side-effect behavior. This is central to LSP. See Chapter 49.
- **Big-O notation:** An asymptotic upper-bound language that suppresses constant factors and lower-order terms. Always state input variable, operation, and assumptions. See Chapter 42.
- **Binary heap:** A complete tree, usually array-backed, satisfying a parent-child priority invariant. It exposes an extremum efficiently but is not globally sorted. See Chapters 29 and 45.
- **Binary search tree:** A tree whose left and right subtrees are ordered relative to each node under a comparator. Without balancing, height can become linear. See Chapters 28 and 45.
- **Blocking:** Waiting while a thread cannot make current progress, for example on I/O, a lock, or a queue. Blocking semantics need timeout, interruption, and capacity policy. See Chapters 33 and 37.
- **Boxing:** Explicit or implicit creation/use of a wrapper representation for a primitive value. Primitive streams and specialized collections/arrays can avoid some boxing overhead. See Chapters 12 and 31.

F.4 C

- **Cache locality:** Performance benefit from accessing nearby or recently used memory. Array-backed collections generally have better locality than node-heavy linked structures. See Chapters 26 and 39.
- **Capacity:** Allocated storage or admitted workload limit, distinct from current logical size. Explicit capacity is also a reliability boundary for queues, pools, and caches. See Chapters 26, 29, and 52.
- **CAS:** Compare-and-set, an atomic conditional update that succeeds only if the observed value still equals an expected value. Algorithms must handle failure and possible ABA concerns. See Chapter 35.
- **Checked exception:** A `Throwable` subtype that is neither a `RuntimeException` subtype nor an `Error` subtype. Java's compile-time checking generally requires callers to catch or declare checked exceptions. They should represent meaningful recoverable boundaries. See Chapter 20.
- **Class:** A Java reference-type declaration defining members, constructors, inheritance, and implementation. At runtime a class is associated with its defining loader. See Chapters 16 and 5.
- **Class file:** The binary format containing JVM instructions, constants, metadata, and verification information for a class or interface. It is specified by the JVM Specification. See Chapter 3.
- **Class initialization:** Execution of static field initializers and static blocks under the language's initialization procedure. Successful initialization has important inter-thread visibility guarantees. See Chapter 5.
- **Class loader:** Runtime component that creates a `Class` from binary data and establishes a namespace. The same binary name under different defining loaders denotes different runtime types. See Chapter 5.
- **Class path:** Ordered set of locations searched by class-loading tools in class-path mode. Duplicate classes and ordering can make resolution fragile; modules provide another model. See Chapters 2 and 51.
- **Class variable:** A `static` field associated with a class rather than each instance. Shared mutability requires an explicit concurrency and lifecycle policy. See Chapters 16 and 33.
- **Class-loader leak:** Retention of an otherwise obsolete loader through threads, statics, registries, drivers, or callbacks, thereby retaining all classes and related metadata it defines. See Chapter 41.
- **Closure:** Function-like behavior together with captured lexical values. A Java lambda can capture only final or effectively final local variables, though captured objects may be mutable. See Chapter 23.
- **Code cache:** HotSpot native memory storing generated machine code and related metadata. Its organization and tuning are implementation-specific. See Chapters 10 and 40.
- **Cohesion:** Degree to which a component's responsibilities belong together and change for related reasons. High cohesion makes invariants and ownership easier to locate. See Chapter 49.
- **Collision:** In hashing, distinct unequal keys mapping to the same hash region. Correct tables resolve collisions using equality; excessive collisions degrade performance. See Chapter 27.
- **Comparable:** Interface defining a type's natural ordering through `compareTo`. Natural order should normally be consistent with `equals`, especially for sorted collection use. See Chapter 30.
- **Comparator:** Object defining an external ordering through `compare`. It must satisfy sign, transitivity, and zero-consistency rules; comparison zero defines uniqueness in tree collections. See Chapter 30.

- **Composition:** Building behavior by owning and delegating to collaborators. It usually exposes fewer extension hazards than inheritance and supports Strategy or Decorator designs. See Chapter 49.
- **Concurrency:** Multiple tasks making progress during overlapping periods, whether or not they execute simultaneously. Correctness requires ordering, ownership, cancellation, and capacity reasoning. See Part V.
- **Concurrent collection:** Collection designed with specified thread-safe operations and traversal semantics. Its individual method safety does not automatically make a compound workflow atomic. See Chapter 37.
- **Constant pool:** See runtime constant pool. A class file also contains a per-class-file constant pool used to encode symbolic and literal information. See Chapters 3 and 4.
- **Constructor:** Special class member that initializes a newly created instance after allocation and superclass construction steps. It is not inherited and has no return type. See Chapter 16.
- **Contention:** Multiple threads competing for a limited resource such as a lock, CPU, connection, or cache line. It can reduce throughput and increase tail latency. See Chapters 34 and 39.
- **Covariance:** Type relationship preserving direction, such as `? extends T` for a producer view. Java arrays are covariant; ordinary generic types are invariant. See Chapter 22.
- **Critical section:** Code region whose shared-state invariant requires controlled access, commonly through a lock or atomic protocol. Keep it correct first, then minimize held time. See Chapter 34.

F.5 D

- **Data race:** Conflicting accesses to shared memory by different threads, at least one a write, without sufficient happens-before ordering. Race-free design is prerequisite to simple visibility reasoning. See Chapter 11.
- **Deadlock:** Cycle of waiting in which each participant requires a resource held by another and none can progress. Prevention can impose lock ordering or avoid hold-and-wait. See Chapter 38.
- **Defensive copy:** Independent copy made to prevent caller or callee aliases from mutating owned collection/array structure. A shallow copy still shares element objects. See Chapters 19 and 49.
- **Dependency injection:** Supplying collaborators from outside a component rather than constructing global/concrete dependencies internally. It improves explicitness and test seams when boundaries are meaningful. See Chapter 49.
- **Dependency inversion:** High-level policy depends on an abstraction it needs; low-level mechanisms implement that abstraction. An interface mirroring one vendor SDK does not necessarily invert policy. See Chapter 49.
- **Deserialization:** Reconstruction of data or objects from an external representation. It is a trust boundary requiring schema, type, size, depth, and code-execution controls. See Chapters 32 and 52.
- **Direct buffer:** NIO buffer whose storage is outside the ordinary heap-array model and intended for efficient native I/O. Allocation, cleanup, and accounting are JVM/platform-sensitive. See Chapter 32.
- **Dominator:** In heap-graph analysis, an object through which every root path to another object passes. Dominators help locate retention owners. See Chapter 41.
- **Double-checked locking:** Lazy-initialization pattern checking before and after acquiring a lock. In Java, the published field must be `volatile` and the implementation must preserve the protocol. See Chapter 35.
- **Downstream collector:** Collector applied inside another collector, such as counting values inside each `groupingBy` bucket. It enables declarative multi-level reductions. See Chapter 31.
- **DTO:** Data transfer object designed for a boundary such as HTTP, events, or persistence mapping. It should carry an explicit version/validation contract rather than expose internal entities. See Chapter 50.
- **Durability:** Degree to which committed state survives failures under a storage system's contract. Java `flush` or object reachability does not itself imply durable storage. See Chapters 32 and 50.
- **Dynamic dispatch:** Runtime selection of an overridden instance method based on the receiver's runtime class. Static, private, and constructor behavior follows different rules. See Chapter 17.
- **Dynamic programming:** Algorithm technique storing solutions to overlapping subproblems under a recurrence and state definition. Correct state and transition derivation precede table optimization. See Chapter 47.

F.6 E

- **Encapsulation:** Protecting representation and invariants through controlled operations. Private fields alone are insufficient if mutable aliases or invalid setters escape. See Chapters 16 and 49.
- **Encounter order:** Logical order in which a collection or stream presents elements. It can be defined, absent, or deliberately relaxed; it is distinct from sorted order. See Chapters 25 and 31.
- **Enum:** Special class with a fixed set of named instances. Enums can have fields, methods, and per-constant behavior and work efficiently with `EnumSet/EnumMap`. See Chapter 21.
- **Equality contract:** `equals` must be reflexive, symmetric, transitive, consistent, and false for null. Equal objects must have equal hash codes. See Chapter 19.
- **Erasure:** See type erasure.
- **Escape analysis:** JVM optimization analysis determining whether an object/reference escapes a scope, potentially enabling allocation or synchronization elimination. Results are HotSpot/version-dependent. See Chapters 7 and 39.
- **Exception:** Object representing abrupt completion. Checked/unchecked classification, causal chains, suppression, and recovery scope are API-design concerns. See Chapter 20.
- **Executor:** Abstraction separating task submission from scheduling/execution policy. Executor services also define lifecycle, rejection, queue, and shutdown behavior. See Chapter 36.

F.7 F

- **Fail-fast iterator:** Iterator that attempts to detect unsupported structural modification and throw `ConcurrentModificationException`. Detection is best-effort and is not synchronization. See Chapter 25.
- **Fairness:** Policy controlling which waiter obtains a resource. Fair locks/queues can reduce starvation but often add coordination cost and do not guarantee application-level fairness. See Chapter 34.
- **False sharing:** Performance interference when independent frequently written variables occupy the same hardware cache line. It is hardware/layout-sensitive and should be confirmed by measurement. See Chapter 39.
- **FIFO:** First in, first out removal policy. Queue ordering can still be affected by priorities, multiple consumers, retry insertion, or implementation semantics. See Chapter 29.
- **Final field:** Field assigned once by constructor/initializer rules. Correct construction gives special visibility guarantees for final state, but referenced mutable objects can still change. See Chapters 11 and 19.
- **ForkJoinPool:** Executor based on work-stealing queues, suited to recursively divisible CPU work. Blocking and use of the common pool require capacity awareness. See Chapter 36.
- **Functional interface:** A non-sealed interface whose inherited abstract methods, after excluding signatures matching public methods of `Object`, induce one function type under the JLS rules. It can be a target for lambdas and method references; default and static methods do not add abstract requirements. See Chapter 23.

F.8 G

- **G1:** Garbage-First collector, a region-based HotSpot collector and common default in mainstream Java 17/21 server configurations. It is an implementation choice, not Java semantics. See Chapter 9.
- **Garbage collection:** Automatic reclamation of storage for unreachable objects. Java does not mandate one collector, generation scheme, pause target, or immediate return of memory to the OS. See Chapter 9.
- **GC root:** Starting reference used in reachability analysis, such as selected thread, static, class-loader, JNI, or VM references. Root paths explain retention. See Chapter 41.
- **Generics:** Compile-time parameterization of types and methods. Java generics largely use erasure and are invariant unless wildcards express use-site variance. See Chapter 22.

F.9 H

- **Happens-before:** JMM ordering relation guaranteeing visibility and ordering between actions. Program order, locks, volatile, thread start/join, class initialization, and transitivity create important edges. See Chapter 11.
- **Hash code:** Integer used to choose candidate hash regions. Equal objects must have equal hash codes; unequal objects may collide. Key-relevant state must remain stable while stored. See Chapter 27.
- **Hash flooding:** Accidental or adversarial concentration of keys into collisions, increasing CPU cost. Current implementations may add resilience, but input bounds and sound hashes remain necessary. See Chapter 27.
- **HashMap:** General-purpose hash-table [Map](#) allowing one null key and null values, with expected constant-time basic operations under suitable hashing. It has no encounter-order guarantee. See Chapter 27.
- **Heap (data structure):** Priority structure satisfying a heap-order invariant between parents and descendants. A complete, array-backed binary heap is the common representation in Java priority-queue and top-k discussions. See binary heap and Chapters 29 and 45.
- **Heap (JVM):** Runtime data area from which class instances and arrays are allocated conceptually. Physical representation and collector layout are implementation-specific. See Chapter 6.
- **Heap pollution:** A parameterized variable refers to an object not compatible with its parameterized type, often through raw types, unchecked casts, arrays, or unsafe varargs. See Chapter 22.
- **HotSpot:** OpenJDK's widely used JVM implementation. Its JIT tiers, collectors, object layout, flags, and diagnostics are useful engineering details but not Java specification guarantees.

F.10 I

- **Idempotency:** Property that repeated execution under one logical identity produces the intended single logical effect. It is essential when timeouts leave remote outcomes unknown. See Chapters 50 and 52.
- **Identity:** Distinction between the same object reference and merely equal values. `==` tests reference identity for references; `equals` can define logical value equality. See Chapter 19.
- **Immutability:** Inability to change an object's observable state after construction. Final fields and unmodifiable collections help, but deep immutable graphs require controlling mutable referenced objects. See Chapter 19.
- **Inheritance:** Mechanism by which a class extends another class and inherits accessible members/behavior. Correct use requires a genuine behavioral subtype, not merely code reuse. See Chapter 17.
- **Interface:** Reference type declaring a contract and possibly default/static/private methods and constants. Interfaces support multiple capability inheritance but do not automatically guarantee substitutability. See Chapter 18.
- **Interrupt:** Cooperative thread cancellation/status mechanism. Blocking methods may throw `InterruptedException`; code should propagate or restore status unless it deliberately consumes cancellation. See Chapter 33.
- **Invariant:** Condition that remains true for every valid observable state of an object, algorithm, or system. It is the center of correctness proofs and API design. See Chapters 42 and 49.
- **Isolation:** Transaction property controlling observable interference among concurrent transactions. Named levels and anomalies have database-specific realization and must be integration-tested. See Chapter 50.
- **Iterator:** Stateful traversal cursor with `hasNext` and `next`, plus optional removal. Iterator ordering, mutation, consistency, and thread behavior come from the source contract. See Chapter 25.

F.11 J

- **JAR:** ZIP-based Java archive format containing classes, resources, and metadata. A JAR is packaging, not an isolation or dependency-resolution mechanism. See Chapters 2 and 51.
- **javac:** Reference Java compiler included with the JDK. It translates source according to selected language/release options; its diagnostics and optimization choices are tool-specific. See Chapter 3.
- **JDK:** Java Development Kit: a Java runtime plus development, packaging, documentation, and diagnostic tools. Distribution and included components vary by vendor. See Chapter 2.
- **JFR:** Java Flight Recorder, a JDK event-recording facility for timestamped runtime/application evidence. Event configuration and fields are version-sensitive. See Chapter 40.
- **JIT:** Just-in-time compilation of hot runtime code to machine code. Java permits but does not require a particular compiler, threshold, tier, or optimization. See Chapter 10.
- **JLS:** Java Language Specification, the normative definition of Java syntax, typing, evaluation, initialization, exceptions, and memory-model rules for a release.
- **JMC:** JDK Mission Control, a separately distributed analysis and monitoring application commonly used to inspect JFR recordings. Tool versions should match supported recording formats. See Chapter 40.
- **JNI:** Java Native Interface, a boundary for calling native code and interacting with JVM objects. It expands memory-safety, lifecycle, portability, and crash risk. See Chapters 4 and 52.
- **JVM:** Abstract Java Virtual Machine plus, colloquially, an implementation executing class files. Separate JVMs guarantees from HotSpot behavior. See Chapter 4.
- **JVMS:** Java Virtual Machine Specification, the normative definition of class-file format, runtime data areas, loading/linking/initialization, instructions, and verification for a release.

F.12 L

- **Lambda expression:** Java syntax producing an instance of a functional-interface target type. Capture and runtime translation do not imply a specified anonymous-class allocation. See Chapter 23.
- **Latency:** Elapsed time for an operation. It is a distribution; p50, p95, p99, maximum, errors, and load context convey more than an average. See Chapter 39.
- **Lazy evaluation:** Deferring work until a result is demanded. Stream intermediate operations are generally lazy, enabling fusion and short-circuiting but allowing stateful buffering. See Chapter 31.
- **Linearizability:** Concurrent-object correctness condition in which each completed operation appears to take effect at one instant between invocation and response, respecting real-time order. See Chapter 37.
- **Linking:** JVM process of verification, preparation, and optionally resolution after loading. Verification and preparation precede initialization, while symbolic-reference resolution may continue after initialization. See Chapter 5.
- **Live set:** Memory occupied by objects that remain reachable after an effective collection/observation point. A growing comparable live set suggests increasing intended or unwanted retention. See Chapter 41.
- **Livelock:** Threads continue taking actions in response to one another but make no useful progress. Randomized/backoff coordination or protocol redesign may resolve it. See Chapter 38.
- **Load factor:** Hash-table fullness parameter influencing resize threshold, empty buckets, and collision depth. Exact capacity policy belongs to the concrete implementation. See Chapter 27.
- **Lock:** Synchronization mechanism granting controlled access and creating memory-ordering edges. Intrinsic monitors and [Lock](#) implementations differ in API features. See Chapter 34.
- **LSP:** Liskov Substitution Principle: implementations must remain behaviorally usable wherever the abstraction is expected. See behavioral subtyping and Chapter 49.

F.13 M

- **Map:** Association from unique keys to values. **Map** is not a subtype of **Collection**; its key, value, and entry views bridge into collection APIs. See Chapter 25.
- **Memory leak:** Unwanted retention or unreleased resource that grows footprint or exhausts capacity. In Java heap analysis, the root issue is an unintended reachability path. See Chapter 41.
- **Memory model:** Rules describing legal inter-thread observations, ordering, data races, final fields, volatile, locks, and happens-before. Java's model is specified in JLS Chapter 17. See Chapter 11.
- **Metaspace:** HotSpot native-memory area commonly holding class metadata. It can grow through class loading and class-loader retention and is distinct from ordinary Java heap. See Chapters 6 and 41.
- **Method area:** JVM's shared runtime area storing per-class structures such as runtime constants and method/field data. Physical placement is not specified. See Chapter 6.
- **Method overloading:** Multiple methods share a name but have different parameter signatures; compile-time resolution selects among applicable methods. Return type alone cannot overload. See Chapter 14.
- **Method overriding:** Subclass or implementation supplies an instance method matching an inherited method contract, enabling dynamic dispatch. Visibility, return, and checked-exception rules constrain it. See Chapter 17.
- **Module:** Named unit in the Java Platform Module System declaring dependencies and exported/open packages. Modules improve reliable configuration and encapsulation but are not security sandboxes. See Chapter 2.
- **Monitor:** Intrinsic synchronization object associated conceptually with each object/class, used by **synchronized**, **wait**, **notify**, and **notifyAll**. See Chapter 34.
- **Mutable reduction:** Accumulation into a mutable container using a supplier, accumulator, combiner, and optional finisher, as modeled by stream collectors. See Chapter 31.

F.14 N

- **N+1 query:** One initial query followed by one query per result/item, causing linear database round trips. Batch/fetch design should preserve cardinality and memory bounds. See Chapter 50.
- **Natural ordering:** Canonical ordering defined by **Comparable**. Sorted maps/sets use it when no comparator is supplied; comparison zero controls their key/element uniqueness. See Chapters 28 and 30.
- **NIO:** Java APIs centered on buffers, channels, selectors, charsets, and modern filesystem paths. NIO does not mean every operation is nonblocking. See Chapter 32.
- **Non-interference:** Stream requirement that behavioral functions not improperly modify or depend on mutation of the source during pipeline execution. See Chapter 31.
- **Null:** The sole value of the null type, assignable to reference types but not primitive types. API null support is contract-specific; dereference throws **NullPointerException**. See Chapter 12.

F.15 O

- **Object:** Class instance or array at runtime. A reference variable holds a reference value, not the object inline as a language guarantee. See Chapters 7 and 16.
- **Object header:** HotSpot-specific metadata commonly associated with an object, such as class and locking/GC information. Exact bits and size depend on JVM/configuration. See Chapter 7.
- **Object monitor:** See monitor.
- **Optional:** Value-based container representing one non-null value or absence. Best used as a maybe-one return type, not universally as a field, parameter, or collection wrapper. See Chapter 31.
- **Outbox:** Durable table/record written atomically with domain changes and later relayed to messaging, addressing the database-plus-broker dual-write gap. See Chapter 50.
- **Overloading:** See method overloading.
- **Overriding:** See method overriding.

F.16 P

- **Parallel stream:** Stream pipeline eligible for partitioned execution, commonly using fork/join infrastructure. Correctness needs associative/stateless behavior; speed depends on source splitting and workload. See Chapter 31.
- **Parameterized type:** Generic type instantiated with type arguments, such as `List<String>`. It provides compile-time constraints but is mostly erased at runtime. See Chapter 22.
- **Pass-by-value:** Java argument passing copies the evaluated argument value into a parameter. For objects, the copied value is a reference; parameter reassignment does not affect caller variables. See Chapter 14.
- **Pattern matching:** Language features that test structure/type and introduce variables when a pattern matches, including `instanceof`, record patterns, and pattern `switch`. See Chapters 18 and 24.
- **Platform thread:** `Thread` typically mapped one-to-one to an operating-system thread for its lifetime. It is heavier and scarcer than a virtual thread. See Chapters 33 and 37.
- **Polymorphism:** One abstraction supports values with differing implementations, with behavior selected through overriding, interfaces, or patterns. Substitutability remains a contract requirement. See Chapter 17.
- **Prepared statement:** JDBC statement separating SQL structure from bound values. It mitigates value injection but cannot parameterize arbitrary identifiers or SQL fragments. See Chapter 50.
- **Primitive type:** One of Java's boolean or numeric non-reference types. Primitive values have defined ranges/semantics and cannot be null. See Chapter 12.
- **Priority queue:** Queue whose head is selected by ordering rather than arrival time. Java's `PriorityQueue` is not stable and its iterator is not sorted. See Chapter 29.
- **Promotion:** Collector-specific movement/classification of surviving objects into longer-lived storage. Promotion pressure can indicate survivor behavior or live-set growth, not automatically a leak. See Chapters 9 and 41.

F.17 Q

- **Queue:** Collection modeling a head chosen for inspection/removal under a policy. `offer/poll/peek` use special results, while `add/remove/element` can throw. See Chapter 29.

F.18 R

- **Race condition:** Correctness depends on uncontrolled relative timing. A race can exist without meeting the JMM's narrower definition of data race. See Chapter 38.
- **Record:** Final data-carrier class with declared components and generated accessors, constructor, equality, hash, and string form. Component references are only shallowly immutable. See Chapters 19 and 24.
- **Recursion:** Method/problem definition invoking itself on smaller subproblems with a base case. Stack depth and repeated work must be analyzed. See Chapters 8 and 47.
- **Reference type:** Class, interface, array, or type-variable category whose values are references or null. Representation of references is not fixed by the language. See Chapter 12.
- **Reflection:** Runtime inspection/invoke through metadata APIs. It can cross design boundaries, add access/configuration complexity, and should not be assumed to have ordinary call performance. See Chapter 21.
- **Region:** Collector-specific heap subdivision used by region-based collectors such as G1. Region size, roles, and humongous thresholds are implementation/configuration details. See Chapter 9.
- **Reification:** Runtime preservation of type information. Arrays are reified enough for store checks; most generic arguments are erased. See Chapter 22.
- **Retained size:** Heap-analysis estimate of memory that becomes unreachable if an object is removed, based on domination. It differs from direct shallow size. See Chapter 41.
- **Retry budget:** Explicit bound on attempts and elapsed time, ideally within one propagated deadline. It prevents retries from multiplying an outage. See Chapter 52.
- **Runtime constant pool:** Per-class/interface JVM runtime representation derived from the class-file constant pool, containing literals and symbolic references used in linking/execution. See Chapters 4 and 6.

F.19 S

- **Safepoint:** HotSpot mechanism/state permitting selected global VM operations when relevant threads are at safe locations. Reasons, polling, and pause behavior are implementation-specific. See Chapter 10.
- **Sealed class/interface:** Type restricting permitted direct subclasses/implementations, enabling controlled hierarchies and exhaustive reasoning. Permitted types must satisfy language/module/package rules. See Chapters 18 and 24.
- **Serialization:** Encoding data/object state for storage or transfer. Wire schema, compatibility, limits, validation, and trust matter more than convenient object mapping. See Chapters 32 and 50.
- **Shallow copy:** New outer object/collection whose referenced elements are shared. It protects outer structure but not mutable nested objects. See Chapter 19.
- **Shallow size:** Memory directly occupied by one heap object, excluding referenced objects. Large retained graphs can have a small shallow owner. See Chapter 41.
- **Short-circuiting:** Operation that can complete without consuming all input, such as `findFirst`, `anyMatch`, or Boolean `&&`. State/order can constrain savings. See Chapters 13 and 31.
- **SOLID:** Five object-design heuristics: single responsibility, open/closed, Liskov substitution, interface segregation, and dependency inversion. They guide trade-offs, not class-count targets. See Chapter 49.
- **Stack frame:** Per-invocation JVM frame containing local variables, operand stack, and linkage information conceptually. Frame layout is implementation-specific. See Chapters 6 and 8.
- **Starvation:** A task remains unable to acquire service/resource while others progress. Fairness, partitioning, priority aging, or capacity policy may mitigate it. See Chapter 38.
- **Static binding:** Compile-time selection not based on receiver overriding, as with static method hiding and overload resolution. Contrast with dynamic instance-method dispatch. See Chapters 14 and 17.
- **Stream:** Single-use lazy traversal pipeline, not a data container. Intermediate operations describe transformations; a terminal operation drives consumption. See Chapter 31.
- **Synchronization:** Coordination that protects invariants and establishes memory ordering. It includes locks, volatile, atomics, thread lifecycle edges, and higher-level concurrent structures. See Part V.
- **Synchronized:** Java keyword for intrinsic monitor acquisition/release around a block or method. Successful unlock happens-before a later successful lock of the same monitor. See Chapter 34.
- **System class loader:** Application class loader returned by `ClassLoader.getSystemClassLoader` in ordinary launches. Its exact implementation and hierarchy can vary. See Chapter 5.

F.20 T

- **Tail latency:** High-percentile response time, such as p99, reflecting slow requests hidden by averages. Always state traffic, window, errors, and sampling method. See Chapter 39.
- **Thread:** Java execution abstraction represented by `Thread`, with lifecycle, interruption, and memory-ordering rules. It may be a platform or virtual thread in Java 21. See Chapter 33.
- **Thread confinement:** Keeping mutable state accessible to only one thread at a time, eliminating shared-memory races for that state. Transfer still needs safe publication. See Chapter 38.
- **Thread local:** Per-thread value associated with a `ThreadLocal` key. Pooled-thread values require cleanup; massive virtual-thread use requires footprint awareness. See Chapters 33 and 41.
- **Thread pool:** Reuses a bounded/scaled set of worker threads to execute tasks, usually with a queue and rejection policy. Pool sizing follows workload and downstream capacity. See Chapter 36.
- **Thread safety:** Property that a component's documented invariants hold under allowed concurrent use. It must specify atomicity, traversal, visibility, and compound-operation boundaries. See Chapter 38.
- **Throughput:** Completed useful operations per unit time under stated load and correctness. Higher throughput can coexist with worse latency or resource cost. See Chapter 39.
- **Transaction:** Group of database operations committed or rolled back under a database's atomicity/isolation/durability contract. It does not automatically include remote services. See Chapter 50.
- **Transitive dependency:** Library pulled into a build through another dependency. Its version, license, vulnerabilities, and runtime presence remain application supply-chain concerns. See Chapter 51.
- **Treeification:** OpenJDK `HashMap` optimization that can convert a sufficiently collided bucket into a tree under thresholds. It is version-sensitive, not a `Map` contract. See Chapter 27.
- **Try-with-resources:** Statement managing `AutoCloseable` resources in reverse declaration order. Work failure remains primary and close failures become suppressed. See Chapter 20.
- **Type erasure:** Translation model removing most generic type arguments from runtime representation while adding casts/bridges as needed. It explains raw types and heap pollution. See Chapter 22.
- **Type inference:** Compiler derivation of omitted generic/lambda/local types from constraints and context. Inferred type follows language rules, not runtime value changes. See Chapters 22 and 24.

F.21 U

- **Unboxing:** Conversion from wrapper reference to primitive. Unboxing null throws `NullPointerException`; numeric overload and equality behavior can change across boxing boundaries. See Chapter 12.
- **Unicode code point:** Integer identifying a Unicode character value. Java `String` indexes UTF-16 code units, so one supplementary code point occupies two `char` values. See Chapter 15.
- **Unmodifiable view:** Wrapper rejecting mutation through that reference while often reflecting mutations through a backing alias. It is not necessarily an immutable snapshot. See Chapter 25.

F.22 V

- **Value-based class:** API-design designation for classes whose instances should be treated by value and whose identity should not be used for synchronization or identity-sensitive operations. See Chapter 19.
- **Variance:** Relationship between parameterized types when type arguments are related. Java uses invariant generic classes with `extends`/`super` wildcards for use-site covariance/contravariance. See Chapter 22.
- **Virtual thread:** Lightweight `Thread` scheduled by the Java runtime, suited to high-concurrency blocking workloads. It improves scale, not CPU speed, and does not remove downstream limits. See Chapter 37.
- **Volatile:** Field modifier providing defined visibility/order and atomic reads/writes for the field's value. Compound actions like increment remain non-atomic. See Chapters 11 and 35.

F.23 W

- **Wait set:** Conceptual monitor-associated set of threads that called `wait`. Notification permits competition to reacquire the monitor; condition loops must handle wakeups and recheck predicates. See Chapter 34.
- **Weak reference:** Reference that does not keep its referent strongly reachable and is cleared under GC reachability rules. It is not a deterministic cache eviction policy. See Chapters 9 and 41.
- **Work stealing:** Scheduling approach where idle workers take tasks from other workers' dequeues, used by fork/join execution. Blocking and task granularity affect efficiency. See Chapter 36.
- **Write skew:** Transaction anomaly where concurrent transactions read overlapping state and write disjoint rows, jointly violating an invariant under some snapshot isolation schemes. See Chapter 50.

F.24 Z

- **ZGC:** HotSpot low-latency concurrent collector. Generational mode was introduced in JDK 21; selection, flags, defaults, and behavior are release/vendor-specific. See Chapters 9 and 41.

REFERENCE APPENDIX

Appendix G - Primary References and Further Reading

This appendix is a map to primary sources, not a list of links to memorize. Use it when a statement in this book needs a precise boundary: whether code is legal Java, what bytecode must mean, what an API promises, what a particular JDK implements, or how a build and test tool behaves. Unless a link explicitly says otherwise, the Java links below target Java SE or JDK 21, the baseline used by this book.

G.1 A Source Hierarchy for Interview-Quality Answers

Java documentation is published in several forms with different authority. Treating all of them as interchangeable leads to confident but inaccurate answers.

- 1 **Normative specifications** define the language and virtual machine. The Java Language Specification (JLS) answers questions such as whether an expression compiles, which overload is selected, and what synchronization orders memory actions. The Java Virtual Machine Specification (JVMS) defines class-file structure, loading, linking, initialization, instructions, and runtime data areas.
- 2 **Platform API specifications** define public contracts for Java SE types and methods. They are the first source for collection behavior, exception conditions, thread guarantees, stream requirements, and other library semantics. Read the entire class and method contract, including implementation requirements and API notes, while remembering that an implementation note is not necessarily a portable guarantee.
- 3 **OpenJDK JEPs** record the motivation, goals, alternatives, and delivery plan for JDK changes. A JEP is excellent design context, but it is not the final language or API specification. For a shipped feature, confirm the resulting contract in the JLS, JVMS, or Java SE API.
- 4 **JDK implementation and vendor documentation** describes tools and behavior of a distribution, commonly Oracle JDK or OpenJDK HotSpot. It is authoritative for that documented tool or implementation, not automatically for every Java implementation.
- 5 **Project documentation** is authoritative for Maven, Gradle, JUnit, JMH, and similar projects. Such documentation evolves independently of JDK 21. Pin the tool version used by a real repository and consult that version's manual.
- 6 **Source code** can explain an implementation, but it is not a substitute for a public contract. An interview answer should label implementation observations as such and avoid relying on internals unless the question explicitly asks about them.

A defensible answer often takes this form: "The API guarantees X; OpenJDK 21 commonly implements it using Y; code should rely on X." This separates portable reasoning from useful implementation knowledge.

G.2 Java SE 21 Specification Entry Points

- **Java Language Specification, Java SE 21** - Normative language specification. Use it for syntax, types, conversions, declarations, generics, overload resolution, expressions, exceptions, definite assignment, records, patterns, and the Java Memory Model.
- **Java Virtual Machine Specification, Java SE 21** - Normative JVM specification. Use it for class files, runtime data areas, loading, linking, initialization, verification, bytecode instructions, and execution semantics.
- **Java SE 21 specifications hub** - Official index for Java SE specifications plus JDK-specific specifications and tool manuals. Check the classification shown on the page rather than assuming every linked document is part of the language specification.
- **Java SE 21 API specification** - Public platform API contracts. Start here whenever the question is about an interface, class, method, exception, ordering guarantee, null policy, thread-safety statement, or performance characteristic promised by an API.
- **New API list in Java SE 21** - Version-specific inventory of added APIs. It is useful when separating Java 21 capabilities from APIs introduced earlier or later.
- **Deprecated API list in Java SE 21** - Official deprecation inventory and replacement guidance. Deprecation does not by itself mean immediate removal.
- **JDK 21 tool specifications** - Oracle JDK tool manuals for commands such as `java`, `javac`, `jcmd`, and `jfr`. These describe the JDK distribution and its tools; they are not the JLS.
- **javac command specification** - Compiler options, source compatibility, class paths, module paths, annotation processing, and `--release`. Use it to make build claims precise.
- **Java Object Serialization Specification for Java SE 21** - Serialization stream format, object graph rules, versioning, hooks, and security considerations. Prefer explicit formats for many new systems, but consult this specification when legacy Java serialization is actually in scope.

G.3 Language Topics in the JLS

The JLS is dense. The following direct chapter links make it practical during study and review.

- **Chapter 4: Types, Values, and Variables** - Primitive and reference types, type variables, parameterized types, reifiable types, intersections, arrays, and the relationship between compile-time types and runtime classes.
- **Chapter 5: Conversions and Contexts** - Widening, narrowing, boxing, unboxing, capture conversion, assignment conversion, invocation contexts, casting, and numeric promotion. Use it to resolve "does this compile?" questions rather than relying on intuition.
- **Chapter 6: Names** - Scope, shadowing, hiding, name classification, accessibility, and qualified names.
- **Chapter 8: Classes** - Fields, methods, constructors, inheritance, overriding, initialization order, nested classes, enum classes, and record classes.
- **Chapter 9: Interfaces** - Interface inheritance, default methods, functional interfaces, annotations, and annotation interfaces.
- **Chapter 10: Arrays** - Array covariance, runtime component types, creation, initialization, and access.
- **Chapter 11: Exceptions** - Checked and unchecked exceptions, exception analysis, abrupt completion, and precise rethrow.
- **Chapter 12: Execution** - Loading, linking, initialization, object creation, finalization terminology, and program exit. Pair it with JVMs Chapter 5 for runtime details.
- **Chapter 14: Blocks, Statements, and Patterns** - Control flow, switch statements and expressions, pattern variables, reachability, and definite assignment interactions.
- **Chapter 15: Expressions** - Evaluation order, method invocation, overload resolution, lambdas, method references, operators, casts, switch expressions, and pattern-related expressions.
- **Chapter 17: Threads and Locks** - Synchronization actions, happens-before, volatile variables, final-field semantics, wait sets, and the formal Java Memory Model. This is the primary source for visibility and ordering claims.
- **Chapter 18: Type Inference** - Constraint formulas and inference variables behind generic method calls, lambdas, method references, and diamond inference. Use it after first explaining the practical result in plain language.

For interview preparation, read the introductory portions and the exact subsection relevant to an example. Do not quote a nearby rule without checking its exceptions and defined terms.

G.4 JVM Topics in the JVMs

- **Chapter 2: JVM Structure** - Runtime data areas, frames, stacks, heaps, method areas, run-time constant pools, native stacks, and the JVM's abstract architecture. It deliberately leaves many implementation choices open.
- **Chapter 3: Compiling for the JVM** - Illustrative mappings from Java constructs to bytecode. It helps connect source-level reasoning to operand-stack execution, but does not mandate one compiler strategy for every construct.
- **Chapter 4: Class File Format** - Class-file layout, constant pools, descriptors, attributes, stack-map frames, verification constraints, and binary representation.
- **Chapter 5: Loading, Linking, and Initializing** - Class and interface loading, verification, preparation, resolution, class-loader constraints, and initialization. Use it for class-loader identity and initialization-trigger questions.
- **Chapter 6: JVM Instruction Set** - Normative definitions of bytecode instructions, their operands, stack effects, and exceptional behavior.
- **Chapter 7: Opcode Mnemonics** - Compact opcode reference useful while reading `javap` output.
- **Oracle `javap` manual for JDK 21** - Tool documentation for disassembling class files. Use `javap -c -v` as an observation aid, then interpret output using the JVMs.

The JVMs describes an abstract machine, not HotSpot's exact memory layout or JIT algorithms. Statements about TLABs, compressed object pointers, tiered compilation, or a collector's regions belong to implementation documentation and should be labeled accordingly.

G.5 Java 21 Feature Design Records

These OpenJDK JEP pages are primary design records. They explain goals and tradeoffs well, but the final JLS and API remain the contract for shipped behavior.

- **JEP 431: Sequenced Collections** - Motivation and design for uniform first/last/reversed operations across ordered collections. Follow with the Java SE 21 `SequencedCollection`, `SequencedSet`, and `SequencedMap` API contracts.
- **JEP 439: Generational ZGC** - Rationale and goals for adding generations to ZGC in JDK 21. Use the JDK's GC documentation and measurements from the target workload before recommending a collector.
- **JEP 440: Record Patterns** - Final design for record-pattern deconstruction in Java 21. Confirm typing, applicability, and match behavior in the JLS.
- **JEP 441: Pattern Matching for `switch`** - Final design for type patterns, guarded cases, null handling, dominance, and enhanced exhaustiveness in Java 21. Use JLS Chapters 14 and 15 for normative rules.
- **JEP 444: Virtual Threads** - Goals, scheduling model, observability, and migration guidance for virtual threads finalized in JDK 21. Pair it with the `Thread` API and Oracle's virtual-thread guide.

Earlier feature records often referenced by SDE-2 interviews include [JEP 361: Switch Expressions](#), [JEP 394: Pattern Matching for instanceof](#), [JEP 395: Records](#), and [JEP 409: Sealed Classes](#). Their historical examples clarify design intent; use the Java 21 JLS for the consolidated rules.

G.6 Core Library API Maps

- [java.util package specification](#) - Collections, comparators, optionals, random generation, utilities, and common value types. Read the collection-interface contracts before relying on a concrete implementation.
- [SequencedCollection API](#), [SequencedSet API](#), and [SequencedMap API](#) - Java 21 contracts for encounter-ordered collection operations and reversed views.
- [java.util.stream package specification](#) - Stream pipelines, laziness, non-interference, statelessness, ordering, reduction, collectors, and parallel execution. It is the primary source for behavioral constraints on stream lambdas.
- [java.util.concurrent package specification](#) - Executors, queues, synchronizers, concurrent collections, atomics, futures, and package-level memory-consistency guarantees.
- [Thread API for Java 21](#) - Platform and virtual thread creation, lifecycle, interruption, joining, uncaught exceptions, and method contracts.
- [Executors API](#) - Executor factory contracts, including the virtual-thread-per-task executor. Factory convenience does not remove the need to reason about admission control and downstream capacity.
- [CompletableFuture API](#) - Completion-stage composition, execution policies, cancellation behavior, and exceptional completion.
- [java.nio package specification](#) - Buffers, channels, charsets, non-blocking I/O foundations, and related contracts.
- [Files API](#) - File operations, streams over directories and lines, copy/move behavior, attributes, and resource-closing requirements.
- [ByteBuffer API](#) - Position, limit, capacity, slicing, byte order, heap versus direct buffers, and view semantics.
- [java.io package specification](#) - Byte and character streams, readers, writers, serialization APIs, and closeable resources.
- [java.time package specification](#) - Immutable date-time types, time zones, clocks, durations, periods, parsing, and thread-safety guidance.
- [java.sql package specification](#) and [javax.sql package specification](#) - JDBC contracts, data sources, connections, statements, result sets, transactions, and row sets. Database isolation and locking behavior also require the target database's documentation.

When an API page states average or expected complexity, null handling, optional operations, encounter order, or concurrency guarantees, cite that statement. Otherwise, present complexity as an implementation-specific expectation and name the implementation.

G.7 Virtual Threads and Concurrency Guidance

- **Oracle Java 21 virtual threads guide** - JDK 21 guidance on creating virtual threads, representing tasks, scheduling, pinning, thread-local usage, and observability. This is Oracle JDK guidance, while JEP 444 and the API describe design and contract.
- **Java Memory Model in JLS 17.4** - Normative model for inter-thread actions, synchronization order, happens-before, data races, and correctly synchronized programs.
- **Lock API** - Explicit-lock semantics and memory synchronization effects. Read each implementation's fairness, interruptibility, and condition behavior separately.
- **AtomicInteger API** and **VarHandle API** - Atomic operations and explicit memory-ordering modes. Prefer the simplest ordering that is demonstrably correct; do not infer compound-operation atomicity from individual atomic calls.

For concurrency questions, give the invariant first, identify the synchronization edge that protects it, and then cite the applicable API or JLS rule. A schedule that "usually works" is not a correctness proof.

G.8 HotSpot, Garbage Collection, and Runtime Tuning

- **Java 21 HotSpot Virtual Machine guide** - Oracle documentation for the HotSpot implementation shipped with JDK 21. Use it for runtime implementation topics, not as a universal JVM specification.
- **Java 21 HotSpot Garbage Collection Tuning Guide** - Collector selection, ergonomics, generations, pause goals, throughput, footprint, and tuning workflow for HotSpot. Treat flags and defaults as JDK-version-specific.
- **G1 Garbage Collector tuning** - Oracle guidance for diagnosing and tuning G1. Begin with evidence from GC logs and workload goals before changing flags.
- **JEP 439: Generational ZGC** - JDK 21 design context for generational ZGC. A JEP's goals are not a latency guarantee for a specific service.
- **OpenJDK 21 GA source tree** - Primary implementation source for OpenJDK 21. It can answer implementation questions after the specification boundary is clear. Internal classes and algorithms may change without preserving source-level compatibility.

A tuning claim should name the JDK build, collector, heap constraints, allocation rate, traffic profile, and measured outcome. Avoid universal prescriptions such as "collector X is always faster" or "larger heaps always reduce latency."

G.9 Diagnostics, JFR, JMC, and jcmd

- **Java 21 troubleshooting diagnostic tools** - Oracle's overview of command-line and graphical diagnostic facilities. It is a good starting map during an incident.
- **jcmd manual for JDK 21** - Authoritative Oracle JDK command documentation for sending diagnostic requests to a running JVM. Obtain the commands supported by the target process rather than assuming every build exposes the same set.
- **jfr command manual for JDK 21** - Recording-file inspection, summary, printing, assembly, disassembly, and metadata commands.
- **JDK Flight Recorder API Programmer's Guide** - Oracle guide to consuming recordings and creating custom JFR events using the `jdk.jfr` API.
- **JFR recording configurations** - Guidance on predefined and custom recording settings. Choose settings based on diagnostic goals and validate overhead in the target environment.
- **JDK Mission Control documentation** - Oracle documentation and releases for JMC, a separately distributed tool for inspecting JFR data and JVM behavior. It is not part of the Java language specification and may have its own compatibility matrix.
- **jstack manual for JDK 21** and **jmap manual for JDK 21** - JDK tool references for thread and memory diagnostics. Their manuals label these tools experimental and unsupported; prefer supported diagnostic paths such as appropriate `jcmd` operations where available.

During diagnosis, capture evidence before tuning: timestamps, service symptoms, JDK/build, command used, recording duration, load conditions, and relevant configuration. Correlate JFR, GC, application, and infrastructure signals instead of drawing a conclusion from one isolated sample.

G.10 Microbenchmarking with JMH

- **OpenJDK JMH project** - Official project page for the Java Microbenchmark Harness. JMH is an OpenJDK Code Tools project, not a Java SE API.
- **Official JMH repository and README** - Build instructions, supported modes, annotations, and project guidance. Use the generated harness and the recommended build setup.
- **Official JMH samples** - Executable examples covering dead-code elimination, constant folding, state scope, setup, parameters, profilers, and other benchmark traps.

JMH handles much of the measurement machinery, but it cannot decide whether a benchmark represents a production workload. State the hypothesis, isolate the operation carefully, consume results, parameterize realistic inputs, inspect allocation and generated behavior where relevant, use forks, report uncertainty, and retain the full environment. Benchmark results are evidence for the tested configuration, not timeless truths about a Java construct.

G.11 Security and Serialization

- **Java SE 21 security developer guide** - Oracle's versioned guide to Java security APIs and mechanisms, including cryptography, providers, secure communication, authentication, and related topics.
- **Java security overview for JDK 21** - Architecture and terminology for security providers, cryptographic services, public-key infrastructure, secure communication, and access control.
- **Oracle Secure Coding Guidelines for Java SE** - Vendor-maintained secure coding guidance. It is useful for threat-aware design but is updated independently of the JLS and should be checked for its applicable platform scope.
- **Java serialization filters guide for JDK 21** - Oracle guidance for allowlists, reject lists, resource limits, and filter factories when native serialization cannot be avoided.
- **Java Object Serialization Specification** - The detailed stream and object-graph contract. Use it alongside, not instead of, a threat model for untrusted input.

Security behavior changes with JDK updates, library versions, deployment configuration, and threat intelligence. For a real system, also consult supported-version advisories and the primary documentation for the framework, container, operating system, identity provider, and protocol in use.

G.12 Maven Primary Documentation

- **Maven build lifecycle guide** - Default, clean, and site lifecycles; phases; goals; packaging; and safe command selection.
- **Maven dependency mechanism guide** - Transitive dependencies, mediation, scopes, management, optional dependencies, and exclusions.
- **Maven reproducible builds guide** - Project configuration and verification practices for reproducible artifacts.
- **Maven Wrapper documentation** - Running a project with a declared Maven distribution rather than depending on an arbitrary machine-wide installation.

Maven's online documentation can describe the current Maven line rather than the version in an older repository. Confirm the wrapper or CI version, plugin versions, effective POM, active profiles, and repository settings before diagnosing resolution or lifecycle behavior.

G.13 Gradle Primary Documentation

- **Gradle User Manual** - Official entry point for builds, tasks, plugins, dependency management, performance, authoring, and reference material.
- **Gradle dependency management** - Configurations, variants, metadata, repositories, constraints, platforms, and dependency resolution.
- **Gradle dependency locking** - Lock-state generation, update, use, and limitations for repeatable resolution.
- **Gradle dependency verification** - Checksums, signatures, trust configuration, and verification metadata for supply-chain protection.
- **Gradle Java toolchains** - Declaring, discovering, selecting, and provisioning Java installations for compilation, testing, and execution.
- **Gradle security guidance** - Official guidance on wrapper integrity, dependency verification, credentials, and other build-security concerns.

The `current` URLs follow Gradle's current release. For a repository, use its wrapper version and switch the documentation selector to that exact version before relying on DSL behavior or defaults.

G.14 JUnit Primary Documentation

- **JUnit official site** - Project entry point for releases, documentation, community information, and related modules.
- **JUnit User Guide** - Official guide for the JUnit Platform, Jupiter programming and extension models, parameterized tests, suites, build integration, IDE support, parallel execution, and migration topics.

The `current` guide moves with JUnit releases. Pin the version used by the build and open the matching versioned guide for exact annotations, engines, launcher APIs, and configuration parameters. In an interview, distinguish the JUnit Platform, a test engine such as Jupiter, and the assertions or mocking libraries selected by the project.

G.15 Suggested Reading Routes

Use a route based on the question rather than reading every source front to back.

- **Language puzzle:** Compile a minimal example with JDK 21, then consult JLS Chapters 4, 5, 8, 14, 15, or 18. Explain the rule and one practical implication.
- **Memory visibility or race:** State the shared invariant, identify synchronization actions, use JLS 17.4 and the relevant `java.util.concurrent` contract, and test only as supporting evidence.
- **Collection behavior:** Start at the interface contract, then the concrete API, then JEP 431 if design motivation matters. Do not infer a guarantee merely from current source code.
- **Class loading or bytecode:** Use JVMs Chapters 4 through 6. Use `javap` to inspect an example and label OpenJDK-specific observations.
- **Java 21 feature:** Read the relevant JEP for motivation, then the JLS or API for final semantics, and finally a JDK guide for operational advice.
- **GC or latency incident:** Read the HotSpot VM and GC guides, collect JFR and GC evidence with documented tools, and evaluate the exact deployed JDK under representative load.
- **Microbenchmark:** Start with JMH's README and samples. Pre-register the question, preserve benchmark code and environment, and report distributions or uncertainty rather than only the best score.
- **Build failure:** Identify the wrapper and runtime versions, then use the exact Maven or Gradle documentation. Capture dependency insight, effective configuration, repositories, and toolchain selection.
- **Test design:** Use the version-matched JUnit guide, separate unit from integration boundaries, and make assertions about externally meaningful behavior.
- **Security decision:** Start with a threat model and current supported-version guidance. Use the Java security and serialization sources for platform behavior, then consult the primary documentation of every external component involved.

The final habit is version discipline. A precise source from the wrong JDK, collector, database, build tool, or test framework can still produce the wrong answer. Record the relevant versions, distinguish guarantees from observations, and prefer the smallest authoritative source that directly answers the question.